

FIREFLY SOFTWARE FOUNDATION



PyFly

by Example

Event-Driven Python Microservices
with the Firefly Framework



PyFly by Example

Microservicios Python orientados a eventos con el framework Firefly
Firefly Software Foundation

Copyright © 2026 Firefly Software Foundation.

Publicado bajo la Licencia Apache, Versión 2.0 (la "Licencia"); no puedes utilizar este material salvo en cumplimiento de la Licencia. Puedes obtener una copia de la Licencia en <https://www.apache.org/licenses/LICENSE-2.0>.

Salvo que lo exija la legislación aplicable o se acuerde por escrito, el software y la documentación distribuidos bajo la Licencia se distribuyen "TAL CUAL", SIN GARANTÍAS NI CONDICIONES DE NINGÚN TIPO, ya sean expresas o implícitas. Consulta la Licencia para conocer el régimen específico de permisos y limitaciones que la rigen.

Primera edición, 2026.

Los capítulos originales de Lumen corresponden a PyFly 26.6.x. El Apéndice E documenta la implementación de aplicaciones web y administración de modelos del checkout 26.9.5 que acompaña al libro; ejecuta su catálogo desde ese checkout.

Publicado por la Firefly Software Foundation.

Para Jacinto Arias y el equipo de Taidy —

por empujarnos, una y otra vez, a construir un framework de verdad para Python.

Gracias a vosotros, ahora por fin sí se harán las cosas de una única forma.

Una. Sola. Forma. (Le damos una semana.)

Prefacio

El Python empresarial ha significado durante mucho tiempo ensamblar una docena de bibliotecas independientes —una para inyección de dependencias, otra para enrutamiento, otra más para acceso asíncrono a la base de datos— sin un idioma común que las una. **PyFly** cambia eso. Ofrece la experiencia cohesionada de convención sobre configuración que Spring Boot dio al mundo Java, reconstruida desde cero para Python 3.12+ y `async / await`.

Este libro enseña PyFly **haciendo**. Construyes una aplicación real desde una carpeta vacía hasta un servicio seguro, observable y orientado a eventos, haciendo concreto cada concepto antes de pasar al siguiente. Y lo más importante: el código de estas páginas no es pseudocódigo ilustrativo, sino que está tomado de un **proyecto real que compila, arranca y supera sus pruebas** con PyFly v26.6.110. Cada listado se verificó contra el ejemplo en ejecución, de modo que lo que lees es lo que realmente funciona.

El **Apéndice E** amplía el libro con el catálogo independiente de `samples/webapp`, verificado contra el checkout actual de PyFly. Explica HTML nativo, formularios, URLs y archivos estáticos, errores personalizados y administración de los mismos modelos SQLAlchemy/Beanie. Los capítulos de Lumen conservan sus referencias históricas de versión.

Para quién es este libro

Este libro es para desarrolladores de Python de nivel intermedio que se sienten cómodos con `async / await`, las anotaciones de tipos y los fundamentos de los servicios HTTP. No necesitas experiencia previa con frameworks: si has construido algo con FastAPI, Flask o SQLAlchemy, estás bien preparado.

Los desarrolladores de Spring Boot se sentirán especialmente como en casa. Allí donde PyFly refleja un concepto de Spring —beans, estereotipos, transacciones declarativas, eventos de aplicación— una llamada de **Equivalencia con Spring** traza el paralelismo de forma explícita, para que mapees lo que ya sabes en lugar de aprender desde cero.

Lo que vas a construir

Cada capítulo hace avanzar **Lumen**, un servicio de monedero (wallet) digital y libro mayor. El recorrido sigue un arco deliberado, una parte cada vez:

- **Inicio rápido — Construye Lumen paso a paso.** Antes de la inmersión profunda, un único recorrido guiado te lleva desde una carpeta vacía hasta una funcionalidad de monedero en ejecución y probada —abrir un monedero, depositar, leer el saldo por HTTP— para que veas la forma completa de una aplicación PyFly antes de centrarte en cualquier parte concreta. Cada capítulo posterior expande luego una porción de lo que construiste aquí.
- **Parte I — Fundamentos (Capítulos 1–4).** Generas el andamiaje del primer servicio Lumen con `pyfly new`, lo ejecutas bajo un servidor ASGI, conectas el contenedor de inyección de dependencias de PyFly, vinculas configuración tipada y perfiles, y expones tus primeros endpoints REST validados.
- **Parte II — Modelar y persistir (Capítulos 5–7).** Introduces el patrón repositorio sobre un puerto, persistes monederos con SQLAlchemy asíncrono (SQLite, sin necesidad de infraestructura), modelas el dominio con un objeto de valor `Money` y una raíz de agregado `Wallet`, y separas las lecturas de las escrituras con manejadores de comando y consulta de CQRS despachados a través de un bus.
- **Parte III — Orientada a eventos (Capítulos 8–10).** El agregado emite eventos de dominio; un escuchador los proyecta; un **libro mayor con event sourcing** (un patrón en el que el estado se reconstruye a partir de un flujo de eventos)

reconstruye cada saldo reproduciendo su flujo de eventos; y esos mismos eventos fluyen hacia Kafka o RabbitMQ para otros servicios.

- **Parte IV — Hacia los microservicios (Capítulos 11–13).** Lumen va más allá de su propio proceso: un cliente HTTP tipado llama a un servicio externo de Pagos, una **saga de transferencia** orquestada mueve dinero entre monederos y *compensa* cuando un paso falla, y los patrones de caché y resiliencia mantienen el sistema rápido y tolerante a fallos.
- **Parte V — Asegurar · Observar · Publicar (Capítulos 14–18).** Aseguras los endpoints con JWT y `@secure`, haces el servicio observable con métricas, trazas, comprobaciones de salud y el panel de administración, pruebas toda la pila, lo conectas con el mundo exterior mediante programación, notificaciones y webhooks, y finalmente lo extiendes y lo despliegas a producción.

Para la última página tendrás un servicio funcional, probado, observable y seguro, y el modelo mental para extenderlo.

Cómo usar este libro

Lee de forma secuencial. Cada capítulo se apoya en el anterior, y la base de código de Lumen crece de forma incremental; saltarte capítulos deja huecos.

Escribe tú mismo cada listado. Leer y teclear el código a la vez es la manera de que los patrones se asienten. Resiste la tentación de copiar y pegar hasta que hayas escrito cada listado al menos una vez.

Ejecútalo. Lumen funciona de verdad: `uv run pyfly run` arranca el servicio y `uv run --extra dev pytest` lo ejercita. Cada vez que un capítulo añada una funcionalidad, arranca la aplicación o las pruebas y míralo funcionar. Ver JSON real que vuelve de un endpoint real vale por cien diagramas.

Cada capítulo cierra con un **Resumen** de lo que cambió en la base de código de Lumen y un conjunto de **Ejercicios** que dan un paso más allá. Los ejercicios son opcionales, pero recomendables para cualquier cosa que pretendas aplicar de inmediato.

Convenciones en breve

Las convenciones tipográficas y estructurales —los pies de los listados de código, los tipos de llamada y la numeración de las figuras— se demuestran, con ejemplos en vivo, en la sección **Convenciones** que viene a continuación.

El código de acompañamiento

El proyecto Lumen completo y ejecutable vive en el directorio `samples/lumen` del framework. Es un único proyecto PyFly por capas — `interfaces` , `models` , `core` , `web` — que haces crecer capítulo a capítulo; el código terminado que hay allí es el destino al que este libro te conduce. Configúralo una sola vez con `uv sync` y úsalo para comparar tu trabajo, ponerte al día si te quedas atrás o, simplemente, ejecutar las partes sobre las que estás leyendo.

Convenciones

Esta página explica las convenciones tipográficas y estructurales que se usan a lo largo del libro.

Listados de código

Cada ejemplo de código de varias líneas tiene una **pestaña con el nombre del archivo** en la esquina superior izquierda que muestra el archivo al que pertenece, y un pie "**Listado N.N**" debajo del bloque que lo identifica por capítulo y número de secuencia. Por ejemplo:

wallet/domain/wallet.py

```
from pyfly.core import component
from dataclasses import dataclass, field
from decimal import Decimal

@Component
@dataclass
class Wallet:
    id: str
    balance: Decimal = field(default=Decimal("0.00"))

    def deposit(self, amount: Decimal) → None:
        if amount ≤ 0:
            raise ValueError("Deposit amount must be positive")
        self.balance += amount
```

Listado 5.1 — Raíz del agregado Wallet

Las referencias de código en línea dentro de la prosa usan fuente **monoespaciada**, como en "el decorador `@component` registra la clase en el contenedor de PyFly".

Notas al margen

En los márgenes y en el cuerpo aparecen cuatro estilos de notas al margen:

NOTA

Las notas aportan contexto complementario o aclaran una sutileza del texto principal: merece la pena leerlas, pero no son bloqueantes.

CONSEJO

Los consejos comparten un atajo, un idiom o una buena práctica que te ahorrará tiempo en proyectos reales.

ADVERTENCIA

Las advertencias señalan un error habitual o un punto delicado que puede provocar problemas difíciles de depurar si se ignora.

EQUIVALENCIA CON SPRING

Las notas de equivalencia con Spring relacionan un concepto de PyFly directamente con su equivalente en Spring Boot: ideales para quienes migran desde el ecosistema de la JVM.

Figuras

Los diagramas se numeran como **Figura N.N** y llevan un pie debajo de la imagen. Se incrustan como SVG en línea, de modo que se renderizan con nitidez a cualquier nivel de zoom tanto en la edición en pantalla como en la impresión. Te encontrarás con la primera en la página inicial del Capítulo 1.

Contenido

Inicio Rápido

Construye Lumen Paso a Paso	14
-----------------------------------	----

PARTE I Fundamentos

1. ¿Por qué PyFly?	56
2. Inyección de Dependencias y el Contexto de Aplicación	83
3. Configuración, Perfiles y Secretos	123
4. Tu Primera API HTTP	158

PARTE II Modelar y Persistir el Dominio

5. Persistencia y el Patrón Repositorio	194
6. Diseño Guiado por el Dominio	241
7. CQRS: Comandos y Consultas	286

PARTE III Arquitectura Orientada a Eventos

8. Eventos de Dominio y Arquitectura Orientada a Eventos	339
9. Event Sourcing del Libro Mayor	376
10. Mensajería con Kafka y RabbitMQ	429

PARTE IV Hacia los Microservicios

11. Dividir el Monolito: Clientes HTTP y el BFF	485
12. Transacciones Distribuidas: Sagas, Workflows y TCC	528
13. Caché y Resiliencia	579

PARTE V Asegurar, Observar y Desplegar

14. Seguridad, Sesiones e Identidad	632
15. Observabilidad y el Panel de Administración	710
16. Pruebas de Aplicaciones PyFly	775
17. Programación, Notificaciones, Webhooks y Callbacks	831
18. Extender PyFly y Llevarlo a Producción	880

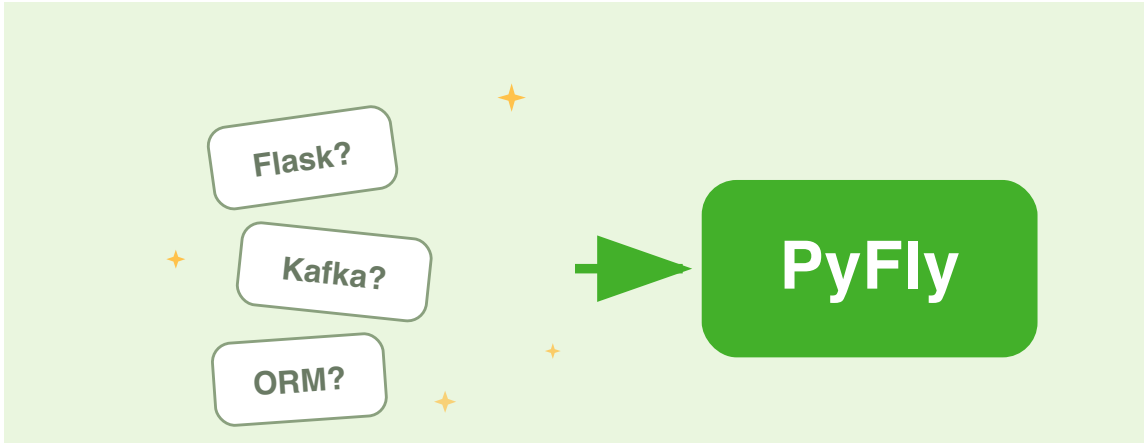
Apéndices

A. Chuleta de Spring Boot → PyFly	927
B. MongoDB y Datos Documentales	971
C. ECM: Contenido y Firma Electrónica	998
D. CLI y Resolución de Problemas	1013
E. Aplicaciones Web Nativas y Administración de Modelos	1028
Glosario	1042

Inicio Rápido

INICIO RÁPIDO

Construye Lumen paso a paso



Bienvenido. Esto es lo primerísimo que vas a construir con PyFly, y lo vamos a hacer con calma. Al final de este capítulo habrás pasado de una *carpeta vacía* a una porción *en ejecución y probada* de un servicio real de monedero digital: abrir un monedero, persistirlo en una base de datos, leer su saldo de vuelta por HTTP y reaccionar a un evento de dominio. Cada concepto recibe un fragmento de código pequeño y completo y un punto de control "Ejecútalo" para que lo veas funcionar antes de seguir adelante.

Esto es un *recorrido*, no la inmersión profunda. Cada paso adelanta un tema que la Parte I y la Parte II tratan a fondo más tarde. El objetivo aquí es ganar impulso: para cuando llegues al Capítulo 1 ya habrás conocido la inyección de dependencias, la configuración, HTTP, la persistencia, CQRS y los eventos —en pequeño— y el resto del libro completará el *porqué*.

La aplicación que construyes se llama **Lumen**: un monedero digital con sabor a DDD. Un monedero puede abrirse, recibir depósitos y permitir retiradas, protegiendo una regla central —**el saldo nunca se vuelve negativo**— y modelando el dinero con aritmética entera exacta para que nunca haya desviaciones de coma flotante. Es la misma aplicación que construye el libro entero, así que nada de lo que aprendas aquí es desechable.

 NOTA

Este capítulo está escrito sobre PyFly **v26.6.110**. Cada listado está tomado del proyecto real y en ejecución `samples/Lumen` que acompaña al libro: el código compila, arranca y pasa sus pruebas. Reconocerás estos mismos archivos de nuevo, con más profundidad, en capítulos posteriores.

Paso 1 — Requisitos previos e instalación

PyFly es un framework de Python, y la forma más fluida de trabajar con él es a través de **uv**, el rápido gestor de paquetes y proyectos de Python de Astral. `uv` se encarga de tu versión de Python, de tu entorno virtual y de tus dependencias en una sola herramienta, y la herramienta de línea de comandos `pyfly` se ejecuta a través de él.

Necesitas dos cosas:

- **Python 3.12 o más reciente.** PyFly usa funciones de tipado modernas (`StrEnum` , uniones `X | None` , genéricos de PEP 695).
- **uv.** Instálalo una vez, en todo el sistema.

Instala `uv` con la orden oficial de una línea:

```
curl -LsSf https://astral.sh/uv/install.sh | sh    # macOS / Linux
# Windows (PowerShell):
# powershell -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Ejecútalo

Confirma que ambas herramientas están disponibles:

```
uv --version
uv python install 3.12    # ensures a 3.12+ interpreter is available
```

Deberías ver impresa una versión de uv, y uv informará de que Python 3.12 está instalado (o ya presente).

💡 SIGUIENDO EL EJEMPLO TERMINADO

Todo lo que hay en este capítulo existe, ya terminado, en el repositorio del libro bajo `samples/lumen`. Si en algún momento quieres comparar tu código con el original —o simplemente ejecutarlo—, clona el repositorio y haz:

```
cd samples/lumen
uv sync --extra dev    # framework + pytest
uv run pyfly run --server uvicorn
```

Puedes construir junto a él, copiando un archivo a la vez, o leer la versión terminada cuando un paso no esté claro. Ambas cosas funcionan.

Paso 2 — Andamiaje del proyecto

PyFly incluye un generador de proyectos, `pyfly new`, igual que Spring tiene el Spring Initializr. Escribe una distribución de proyecto convencional para que no empieces desde una página en blanco. Como `pyfly` vive dentro del paquete del framework, lo primerísimo que hacemos es crear un directorio de proyecto y añadirle PyFly.

Para este recorrido construiremos los directorios a mano a medida que avancemos — así cada archivo queda visible y nada se esconde detrás de un generador —, pero el generador está ahí cuando quieras tomar ventaja:

```
pyfly new lumen --archetype hexagonal --features web,data-relational
```

Esa orden crea una carpeta `lumen/`, un `pyproject.toml`, un `pyfly.yaml` y un árbol de fuentes por capas. Creemos la misma forma nosotros mismos para que veas exactamente para qué sirve cada pieza. Empieza con el proyecto y sus dependencias:

```
mkdir lumen && cd lumen
uv init --package --name lumen
uv add "pyfly[cli,web,data-relational]" "pydantic ≥ 2.5"
uv add --dev "pytest ≥ 8" "pytest-asyncio ≥ 0.24" "httpx ≥ 0.27"
```

Los tres extras de PyFly que acabas de añadir se corresponden con las tres cosas que Lumen necesita: `cli` aporta la propia orden `pyfly`, `web` aporta el servidor ASGI y `data-relational` aporta SQLAlchemy 2 (async) más `aiosqlite` para que podamos persistir en un archivo SQLite sin ninguna base de datos externa que instalar.

La distribución del proyecto

Las aplicaciones PyFly siguen una estructura por capas que separa el contrato público, el modelo de dominio, la lógica de aplicación y el borde web. Crea estos paquetes bajo

```
src/lumen :
```

```

lumen/
├── pyproject.toml
├── pyfly.yaml           # framework configuration
├── src/lumen/
│   ├── interfaces/     # the public contract: DTOs + enums
│   │   ├── dtos/v1/
│   │   └── enums/v1/
│   ├── models/         # the domain model + persistence
│   │   ├── entities/v1/
│   │   └── repositories/
│   └── core/           # application logic: commands, queries,
handlers
├── services/
├── mappers/
├── web/               # the HTTP edge: controllers
│   └── controllers/
├── app.py             # the application class
└── main.py            # the ASGI entry point

```

Cada capa tiene un único cometido. `interfaces` es la frontera con la que habla el resto del código (y otros servicios). `models` contiene los ricos objetos de dominio y las filas en que se persisten. `core` contiene las operaciones de negocio. `web` las expone por HTTP. Las rellenaremos capa por capa.

🍃 EQUIVALENCIA CON SPRING

`pyfly new` es el equivalente del Spring Initializr (`start.spring.io`). Las funcionalidades `web` y `data-relational` son las contrapartes en PyFly de los starters `spring-boot-starter-web` y `spring-boot-starter-data-jpa` : nombrar una funcionalidad arrastra exactamente las dependencias y la autoconfiguración que esa funcionalidad necesita, y nada más.

La clase de aplicación

Dos archivos convierten un paquete en una aplicación PyFly. El primero, `app.py`, declara la propia aplicación: qué paquetes escanear en busca de componentes y qué niveles del framework activar.

`lumen/app.py`

```
from __future__ import annotations

from pyfly.core import pyfly_application
from pyfly.starters.domain import enable_domain_stack

@enable_domain_stack
@pyfly_application(
    name="lumen",
    version="1.0.0",
    description="Lumen – a DDD digital-wallet service built on the PyFly framework.",
    scan_packages=[
        "lumen.models.repositories",
        "lumen.core.services.wallets",
        "lumen.web.controllers",
    ],
)
class LumenApplication:
    pass
```

Listado 0.1 — La clase de aplicación

`@pyfly_application` marca la clase como una aplicación PyFly y `scan_packages` le dice al contenedor de inyección de dependencias dónde buscar los componentes que declararás: tus repositorios, servicios, manejadores de comandos/consultas y

controladores. `@enable_domain_stack` activa los niveles de dominio en los que nos apoyaremos más tarde: CQRS, el motor transaccional, la capa de datos relacional y los eventos.

El punto de entrada ASGI

El segundo archivo, `main.py`, es lo que un servidor ASGI importa y sirve realmente. Arranca PyFly —carga la configuración, escanea tus paquetes y construye el contexto de aplicación— y luego entrega la aplicación web resultante a Starlette.

`lumen/main.py`

```
from __future__ import annotations

from lumen.app import LumenApplication
from pyfly.core import PyFlyApplication
from pyfly.web.adapters.starlette import create_app

# Bootstrap: load config, scan packages, build the DI context.
_pyfly = PyFlyApplication(LumenApplication)

app = create_app(
    title="lumen",
    version="1.0.0",
    description="Lumen — a DDD digital-wallet service built on the PyFly\nframework.",
    context=_pyfly.context,
)
```

Listado 0.2 — El punto de entrada ASGI

❗ NOTA

El `samples/lumen/main.py` real añade un gancho `lifespan` y un montaje `/static`. Esos son refinamientos que conocerás en el Capítulo 4; los dos elementos esenciales — `PyFlyApplication(LumenApplication)` para arrancar y `create_app(...)` para construir la aplicación web— son exactamente lo que ves aquí.

Configuración

PyFly lee `pyfly.yaml` desde la raíz del proyecto. Crea uno que nombre la aplicación, establezca el puerto HTTP y active los niveles que necesitamos. Todo está anidado bajo una clave de nivel superior `pyfly`.

`pyfly.yaml`

```
pyfly:
  app:
    name: lumen
    version: 1.0.0
  server:
    # App on 8080; the actuator + admin default to the management port 9090.
    port: 8080
  cqrs:
    enabled: true
  transactional:
    enabled: true
  eda:
    provider: memory          # in-memory event bus, no broker needed
  data:
    relational:
      enabled: true
      url: "sqlite+aiosqlite:///./lumen.db"
      ddl-auto: create        # create tables on startup
```

Listado 0.3 — `pyfly.yaml`

Vale la pena dedicar un momento ahora —y un capítulo más adelante— a unas pocas claves. `pyfly.server.port` es el puerto HTTP de la aplicación — `8080` por defecto, exactamente como el `server.port` de Spring—. `data.relational` apunta a un archivo SQLite (`lumen.db`) y `ddl-auto: create` le dice al framework que cree el esquema de la base de datos al arrancar, así que no hay ningún paso de migración que ejecutar para este recorrido. `eda.provider: memory` nos da un bus de eventos en proceso.

⚠ ADVERTENCIA

Si vienes de un PyFly más antiguo, ten en cuenta que la clave del puerto es `pyfly.server.port` (sobreescritura por entorno `PYFLY_SERVER_PORT`). Las antiguas `pyfly.web.port` / `PYFLY_WEB_PORT` se eliminaron: a partir de ahora establece el puerto bajo `pyfly.server` .

Ejecútalo

Aun sin endpoints todavía, la aplicación arranca. Iníciala:

```
uv run pyfly run --server uvicorn
```

Verás el banner de PyFly, registros de arranque estructurados y una línea que te dice que el servidor está escuchando en `http://0.0.0.0:8080` . La opción `--server uvicorn` selecciona el servidor Uvicorn (viene con `pyfly[web]`); para desarrollo, añade `--reload` para reiniciar automáticamente cuando edites un archivo.

PyFly también expone un **endpoint de salud** para que los orquestadores puedan saber que la aplicación está viva. Los endpoints del actuador y el panel de administración se ejecutan en un *puerto de gestión separado*, `9090` por defecto, lo que mantiene los endpoints operativos fuera de tu puerto público de aplicación. En otra terminal:

```
curl -s localhost:9090/actuator/health
```

```
{"status": "UP"}
```

DOS PUERTOS, A PROPÓSITO

La aplicación sirve tu API en `8080`; el **puerto de gestión** `9090` sirve `/actuator/health`, `/actuator/info` y el panel de administración. Este es el comportamiento de `management.server.port` de Spring Boot. El puerto de gestión está *abierto y sin autenticación por defecto* —está bien en una red privada, pero en producción establecerías `pyfly.management.security.enabled: true` para protegerlo, o `pyfly.management.server.port: -1` para deshabilitar por completo los endpoints de gestión—. Por defecto, solo `health` e `info` se exponen por HTTP; expón más (métricas, entorno, ...) con `pyfly.management.endpoints.web.exposure.include`.

Detén el servidor con `Ctrl-C`. El shell está vacío, pero los cimientos están vivos: el contenedor de inyección de dependencias se construye, el servidor arranca y el informe de salud funciona. Ahora le damos algo que hacer.

Paso 3 — La primera porción del dominio

Empezamos donde DDD dice que hay que empezar: por el modelo. Dos objetos cargan con todo el dominio: `Money`, un objeto de valor para importes exactos, y `Wallet`, el agregado que posee el saldo.

Money — un objeto de valor

`Money` es el *objeto de valor* de manual: no tiene identidad, dos instancias con el mismo importe y moneda son intercambiables, y nunca cambia. Almacenamos los importes como **unidades menores** enteras (céntimos) más un código de moneda ISO-4217, de modo que la aritmética es exacta: `Money(1050, EUR)` son 10,50 € y no hay redondeo de coma flotante del que preocuparse.

Primero, la diminuta enumeración de moneda de la que depende, bajo `interfaces/enums/v1/`:

```
lumen/interfaces/enums/v1/currency.py
```

```
from __future__ import annotations

from enum import StrEnum

class Currency(StrEnum):
    """ISO-4217 currency codes Lumen wallets can hold."""

    EUR = "EUR"
    USD = "USD"
    GBP = "GBP"
```

Listado 0.4 — La enumeración `Currency`

Ahora `Money` en sí, bajo `models/entities/v1/`. Se construye sobre `pyfly.domain.ValueObject` y es una dataclass congelada, así que la igualdad es estructural y las instancias son inmutables. La aritmética devuelve objetos `Money` nuevos y se niega a mezclar monedas.

lumen/models/entities/v1/money.py

```

from __future__ import annotations

from dataclasses import dataclass

from lumen.interfaces.enums.v1.currency import Currency
from pyfly.domain import BusinessRuleViolation, ValueError

@dataclass(frozen=True)
class Money(ValueObject):
    """An exact monetary amount in a single currency (minor units)."""

    amount: int
    currency: Currency

    def __post_init__(self) → None:
        if not isinstance(self.amount, int) or isinstance(self.amount, bool):
            raise BusinessRuleViolation(
                "money-amount-integer", "amount must be an integer number of
minor units"
            )

    @classmethod
    def zero(cls, currency: Currency) → Money:
        """The additive identity for *currency* (a zero balance)."""
        return cls(amount=0, currency=currency)

    def add(self, other: Money) → Money:
        self._assert_same_currency(other)
        return Money(amount=self.amount + other.amount,
currency=self.currency)

    def subtract(self, other: Money) → Money:
        self._assert_same_currency(other)
        return Money(amount=self.amount - other.amount,
currency=self.currency)

```

```

@property
def is_positive(self) → bool:
    return self.amount > 0

@property
def is_negative(self) → bool:
    return self.amount < 0

@property
def major_units(self) → float:
    """The amount as a major-unit decimal (cents / 100)."""
    return round(self.amount / 100, 2)

def _assert_same_currency(self, other: Money) → None:
    if self.currency is not other.currency:
        raise BusinessRuleViolation(
            "money-currency-mismatch",
            f"cannot combine {self.currency.value} with
{other.currency.value}",
        )

```

Listado 0.5 — El objeto de valor Money

`BusinessRuleViolation` es la señal del framework de que se ha quebrantado una regla de dominio —aquí, "los importes son unidades menores enteras" y "no puedes sumar euros a dólares"—. Fíjate en que no hay HTTP, ni base de datos, ni cableado del framework: un objeto de valor es dominio puro.

Wallet — el agregado

El `Wallet` es la *raíz de agregado*: el objeto que posee la invariante. El estado solo cambia a través de métodos que revelan la intención (`open`, `deposit`, `withdraw`), cada uno de los cuales protege la regla **el saldo nunca se vuelve negativo** y registra un *evento de dominio* que describe lo que ocurrió. Construido sobre `pyfly.domain.AggregateRoot`, lanza eventos con `raise_event(...)`, que drenaremos y publicaremos en el Paso 9.

lumen/models/entities/v1/wallet_entity.py

```

from __future__ import annotations

from dataclasses import dataclass
from datetime import UTC, datetime

from lumen.interfaces.enums.v1.currency import Currency
from lumen.models.entities.v1.money import Money
from pyfly.domain import AggregateRoot, BusinessRuleViolation, DomainEvent

@dataclass(frozen=True)
class WalletOpened(DomainEvent):
    wallet_id: str = ""
    owner_id: str = ""
    currency: str = ""

@dataclass(frozen=True)
class FundsDeposited(DomainEvent):
    wallet_id: str = ""
    amount: int = 0
    currency: str = ""
    balance: int = 0

class Wallet(AggregateRoot[str]):
    """Wallet aggregate root - owns the ``balance ≥ 0`` invariant."""

    __slots__ = ("owner_id", "balance", "created_at")

    def __init__(
        self,
        id: str,
        owner_id: str,
        balance: Money,
        created_at: datetime | None = None,
    ) → None:

```

```

    super().__init__(id)
    self.owner_id = owner_id
    self.balance = balance
    self.created_at = created_at or datetime.now(UTC)

    @property
    def currency(self) → Currency:
        return self.balance.currency

    @classmethod
    def open(cls, wallet_id: str, owner_id: str, currency: Currency) →
Wallet:
        """Open a new, empty wallet; raises :class:`WalletOpened`."""
        if not owner_id.strip():
            raise BusinessRuleViolation("wallet-owner-required", "owner_id is
required")
        wallet = cls(id=wallet_id, owner_id=owner_id,
balance=Money.zero(currency))
        wallet.raise_event(
            WalletOpened(wallet_id=wallet_id, owner_id=owner_id,
currency=currency.value)
        )
        return wallet

    def deposit(self, amount: Money) → None:
        """Credit *amount* to the balance; raises :class:`FundsDeposited`."""
        self._assert_currency(amount)
        if not amount.is_positive:
            raise BusinessRuleViolation("wallet-deposit-positive", "deposit
amount must be > 0")
        self.balance = self.balance.add(amount)
        assert self.id is not None
        self.raise_event(
            FundsDeposited(
                wallet_id=self.id,
                amount=amount.amount,
                currency=amount.currency.value,
                balance=self.balance.amount,
            )

```

```

    )

    def _assert_currency(self, amount: Money) → None:
        if amount.currency is not self.balance.currency:
            raise BusinessRuleViolation(
                "wallet-currency-mismatch",
                f"wallet holds {self.balance.currency.value}, got
{amount.currency.value}",
            )

```

Listado 0.6 — El agregado Wallet y sus eventos de dominio

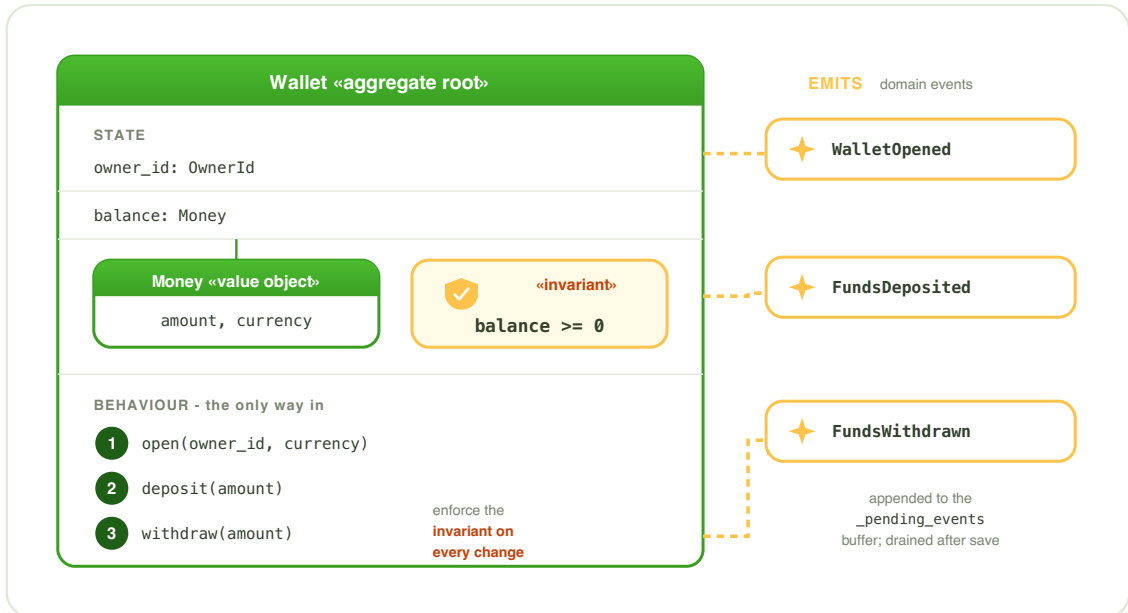


Figura 0.1 — El agregado Wallet posee su invariante; todos los cambios de estado pasan por sus métodos.

❗ NOTA

El `Wallet` terminado en `samples/lumen` también tiene un método `withdraw` y un evento `FundsWithdrawn`, que siguen la misma forma —los hemos dejado fuera aquí para que este primer listado sea corto—. El Capítulo 6 construye el agregado completo.

Ejecútalo

No necesitas el servidor en ejecución para ejercitar el dominio. Abre un REPL de Python dentro del proyecto y maneja el modelo directamente:

```
uv run python
```

```
>>> from lumen.interfaces.enums.v1.currency import Currency
>>> from lumen.models.entities.v1.money import Money
>>> from lumen.models.entities.v1.wallet_entity import Wallet
>>> w = Wallet.open("wlt-1", "alice", Currency.EUR)
>>> w.deposit(Money(1500, Currency.EUR))
>>> w.balance.amount, w.balance.currency.value
(1500, 'EUR')
>>> w.deposit(Money(100, Currency.USD))      # wrong currency → rejected
pyfly.domain.exceptions.BusinessRuleViolation: wallet holds EUR, got USD
```

El agregado impone sus propias reglas en Python puro, sin necesidad de infraestructura.

Paso 4 — Persístelo

Un monedero que vive solo en memoria no sirve de mucho. Necesitamos guardarlo. La capa de datos de PyFly te da un *repositorio al estilo de Spring Data* sobre SQLAlchemy, y como configuramos SQLite no hay ninguna base de datos que instalar.

La fila de persistencia

El agregado es rico; la fila en la que se persiste es plana. Mapeamos `Wallet` a un `WalletEntity` —una fila por monedero, con el saldo dividido en una columna entera (`balance_minor`) y una columna de moneda—. Hereda la `Base` declarativa del framework, lo que le permite conservar como clave primaria el propio id de cadena del agregado.

lumen/models/entities/v1/wallet_orm.py

```

from __future__ import annotations

from datetime import UTC, datetime

from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column

from pyfly.data.relational.sqlalchemy import Base, UtcDateTime

class WalletEntity(Base):
    """One persisted wallet row, keyed by the aggregate's own string id."""

    __tablename__ = "wallets"

    id: Mapped[str] = mapped_column(String(64), primary_key=True)
    owner_id: Mapped[str] = mapped_column(String(255), nullable=False,
index=True)
    currency: Mapped[str] = mapped_column(String(3), nullable=False)
    balance_minor: Mapped[int] = mapped_column(nullable=False, default=0)
    created_at: Mapped[datetime] = mapped_column(
        UtcDateTime(), default=lambda: datetime.now(UTC)
    )

```

Listado 0.7 — La fila de persistencia `WalletEntity`

Como la clase es subclase de `Base`, importarla registra la tabla `wallets`; con `ddl-auto: create` el framework crea esa tabla al arrancar.

El repositorio

En lugar de escribir SQL a mano, creas una subclase del `Repository[Entity, IdType]` genérico del framework. Esa única declaración le dice al framework el tipo de la entidad y el tipo de la clave primaria, y a cambio obtienes gratis toda la superficie del

repositorio asíncrono: `save`, `find_by_id`, `find_all`, `count`, `delete`, paginación y más. Cada llamada se ejecuta en la unidad de trabajo del código que la hace, o en una breve propia que confirma la escritura antes de devolver.

`lumen/models/repositories/wallet_repository.py`

```
from __future__ import annotations

from lumen.models.entities.v1.wallet_orm import WalletEntity
from pyfly.container import repository
from pyfly.data.relational.sqlalchemy import Repository

@repository
class WalletRepository(Repository[WalletEntity, str]):
    """CRUD for :class:`WalletEntity`, plus a convenience upsert."""

    async def find_by_owner_id(self, owner_id: str) → list[WalletEntity]:
        """All wallets owned by *owner_id* (derived query stub)."""
        ...

    async def upsert(self, entity: WalletEntity) → WalletEntity:
        """Insert *entity*, or update the row with the same id."""
        session = self._require_session()
        merged = await session.merge(entity)
        await session.flush()
        return merged
```

Listado 0.8 — Un repositorio al estilo de Spring Data

Dos cosas aquí son Spring Data puro. `find_by_owner_id` es una **consulta derivada**: su cuerpo es un esbozo elidido (`...`), y al arrancar el framework analiza el *nombre* del método y compila por ti un `SELECT ... WHERE owner_id = :owner_id` real. `upsert` es una pequeña comodidad sobre `session.merge` para que un manejador pueda persistir un monedero tanto si es nuevo como si ya existe, con una sola llamada.

El decorador `@repository` registra la clase como un componente gestionado —un *bean* en el contenedor de inyección de dependencias— para que pueda inyectarse en los manejadores que escribimos a continuación.

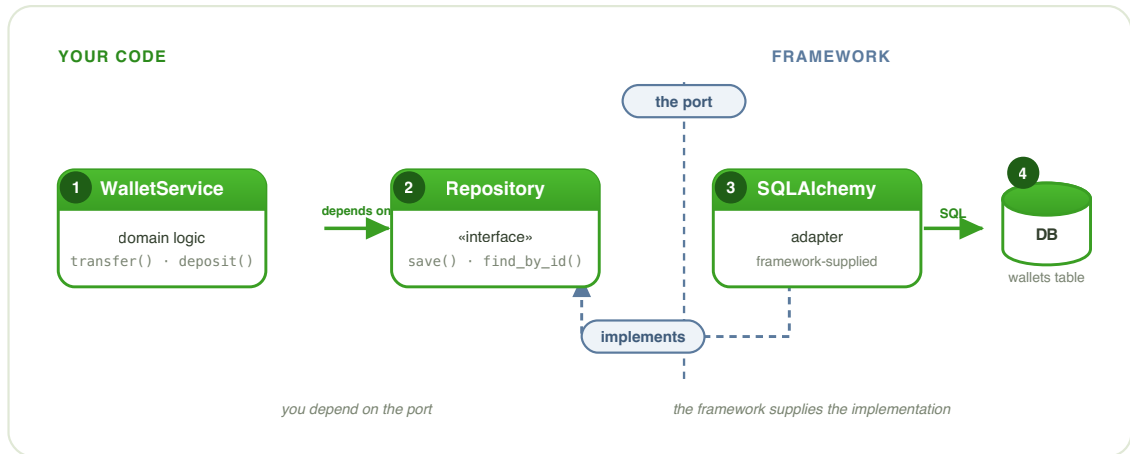


Figura 0.2 — Un Repository del framework convierte una declaración tipada en una superficie CRUD completa.

🌿 EQUIVALENCIA CON SPRING

`Repository[WalletEntity, str]` es el análogo directo del `JpaRepository<WalletEntity, String>` de Spring Data. Tú declaras la interfaz; el framework proporciona la implementación. Las consultas derivadas (`findByOwnerId` en Spring, `find_by_owner_id` aquí) se analizan a partir del nombre del método exactamente de la misma manera.

Paso 5 — Un camino de escritura con CQRS

Ahora cableamos el modelo al repositorio a través de un *comando*. PyFly usa **CQRS** — Command Query Responsibility Segregation (segregación de responsabilidad entre comandos y consultas)—, lo que significa que las escrituras fluyen por un camino (los comandos) y las lecturas por otro (las consultas). Un comando es un objeto pequeño e inmutable que describe la intención; un manejador (handler) lo ejecuta.

El comando

`OpenWallet` lleva los datos necesarios para abrir un monedero y se valida a sí mismo antes de que nada se ejecute. Es una dataclass congelada que extiende `Command[str]` —el `str` dice "este comando, cuando se maneja, produce un id de monedero"—.

lumen/core/services/wallets/open_wallet_command.py

```

from __future__ import annotations

from dataclasses import dataclass

from lumen.interfaces.enums.v1.currency import Currency
from pyfly.cqrs import Command, ValidationResult

@dataclass(frozen=True)
class OpenWallet(Command[str]):
    """Open a new wallet. Returns the generated wallet id."""

    owner_id: str
    currency: Currency

    async def validate(self) → ValidationResult: # type: ignore[override]
        if not self.owner_id.strip():
            return ValidationResult.failure("owner_id", "Owner id is
required")
        return ValidationResult.success()

```

Listado 0.9 — El comando OpenWallet

El manejador

El manejador es donde ocurre el trabajo: generar un id, crear el agregado `Wallet`, persistirlo a través del repositorio y luego drenar y publicar los eventos del agregado. Se ejecuta dentro de `@transactional()`, que abre una unidad de trabajo a la que se une cada llamada al repositorio del método, confirma en caso de éxito y revierte en caso de fallo. El repositorio y el publicador de eventos los inyecta el contenedor —tú solo los declaras en `__init__`—.

lumen/core/services/wallets/open_wallet_handler.py

```

from __future__ import annotations

from uuid import uuid4

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

from lumen.core.mappers.wallet_mapper import to_entity
from lumen.core.services.wallets.event_publishing import publish_domain_events
from lumen.core.services.wallets.open_wallet_command import OpenWallet
from lumen.models.entities.v1.wallet_entity import Wallet
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.data.relational.sqlalchemy import transactional
from pyfly.eda import EventPublisher

@command_handler
@service
class OpenWalletHandler(CommandHandler[OpenWallet, str]):
    """Open a new, empty wallet."""

    def __init__(
        self,
        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
    ) → None:
        super().__init__()
        self._repository = repository
        self._events = events
        self._session_factory = session_factory

    @transactional()
    async def do_handle(self, command: OpenWallet) → str: # type:
        ignore[override]
        wallet_id = f"wlt-{uuid4()}"

```

```

wallet = Wallet.open(
    wallet_id=wallet_id,
    owner_id=command.owner_id,
    currency=command.currency,
)
await self._repository.upsert(to_entity(wallet))
await publish_domain_events(self._events, wallet.clear_events())
return wallet_id

```

Listado 0.10 — El manejador de OpenWallet

El manejador llama a `to_entity(wallet)` —un pequeño mapeador que aplanar el agregado en una fila `WalletEntity`—. Créalo bajo `core/mappers/`. (Añadiremos la proyección del lado de lectura que también necesita en el siguiente paso.)

`lumen/core/mappers/wallet_mapper.py`

```

from __future__ import annotations

from lumen.models.entities.v1.wallet_entity import Wallet
from lumen.models.entities.v1.wallet_orm import WalletEntity

def to_entity(wallet: Wallet) → WalletEntity:
    """Flatten a :class:`Wallet` aggregate into a persistable row."""
    assert wallet.id is not None
    return WalletEntity(
        id=wallet.id,
        owner_id=wallet.owner_id,
        currency=wallet.currency.value,
        balance_minor=wallet.balance.amount,
        created_at=wallet.created_at,
    )

```

Listado 0.11 — Mapeando el agregado a su fila

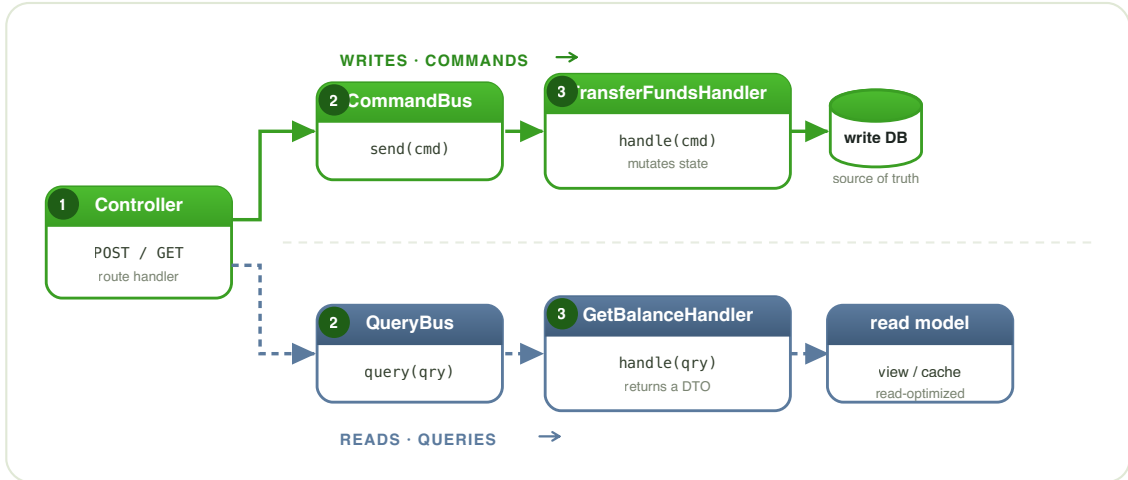


Figura 0.3 — Un comando fluye por el bus hasta su manejador; las consultas toman un camino separado.

🌿 EQUIVALENCIA CON SPRING

`@command_handler` + `@service` registra un manejador al que el bus de comandos despacha —muy parecido a un `@Service` de Spring cuyo método maneja una petición—. `@transactional()` es la contraparte en PyFly del `@Transactional` de Spring: gestiona la unidad de trabajo para que la persistencia se confirme por completo o se revierta por completo.

Paso 6 — Un camino de lectura

Las lecturas toman el otro carril. Una *consulta* hace una pregunta; un *manejador de consultas* la responde, normalmente proyectando una fila de base de datos sobre un DTO pequeño y hecho a propósito. Leeremos solo el saldo.

El DTO y la consulta

La respuesta del saldo es un diminuto modelo Pydantic bajo `interfaces/dtos/v1/`:

`lumen/interfaces/dtos/v1/balance_dto.py`

```
from __future__ import annotations

from pydantic import BaseModel

from lumen.interfaces.enums.v1.currency import Currency


class BalanceDto(BaseModel):
    """Lightweight balance projection for the balance endpoint."""

    id: str
    currency: Currency
    balance_minor: int
    balance: float
```

Listado 0.12 — El modelo de respuesta *BalanceDto*

La consulta lleva solo el id del monedero y declara que devuelve un `BalanceDto` o `None` :

lumen/core/services/wallets/get_balance_query.py

```

from __future__ import annotations

from dataclasses import dataclass

from lumen.interfaces.dtos.v1.balance_dto import BalanceDto
from pyfly.cqrs import Query

@dataclass(frozen=True)
class GetBalance(Query[BalanceDto | None]):
    """Look up just the balance of a wallet by its identifier."""

    wallet_id: str

```

Listado 0.13 — La consulta GetBalance

El manejador y la proyección

Añade el mapeador del lado de lectura a `wallet_mapper.py` —una pequeña función que proyecta una fila sobre el DTO, calculando el saldo en unidades mayores—:

lumen/core/mappers/wallet_mapper.py

```

from __future__ import annotations

from lumen.interfaces.dtos.v1.balance_dto import BalanceDto
from lumen.interfaces.enums.v1.currency import Currency
from lumen.models.entities.v1.wallet_orm import WalletEntity

def entity_to_balance_dto(entity: WalletEntity) → BalanceDto:
    """Project a persisted row onto the lightweight balance DTO."""
    return BalanceDto(
        id=entity.id,
        currency=Currency(entity.currency),
        balance_minor=entity.balance_minor,
        balance=round(entity.balance_minor / 100, 2),
    )

```

Listado 0.14 — *Proyectando una fila sobre el DTO de saldo*

El manejador de la consulta carga la fila por id y la proyecta —devolviendo `None` cuando no existe tal monedero—:

lumen/core/services/wallets/get_balance_handler.py

```

from __future__ import annotations

from lumen.core.mappers.wallet_mapper import entity_to_balance_dto
from lumen.core.services.wallets.get_balance_query import GetBalance
from lumen.interfaces.dtos.v1.balance_dto import BalanceDto
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cqrs import QueryHandler, query_handler

@query_handler
@service
class GetBalanceHandler(QueryHandler[GetBalance, BalanceDto | None]):
    def __init__(self, repository: WalletRepository) → None:
        super().__init__()
        self._repository = repository

    async def do_handle(self, query: GetBalance) → BalanceDto | None: #
        type: ignore[override]
        entity = await self._repository.find_by_id(query.wallet_id)
        return entity_to_balance_dto(entity) if entity is not None else None

```

Listado 0.15 — El manejador de GetBalance

Fíjate en la asimetría que te da CQRS: el lado de escritura rehidrata el agregado completo para proteger las invariantes; el lado de lectura toca solo las columnas que necesita la vista del saldo y nunca construye un agregado. Cada lado está moldeado para su propio cometido.

Paso 7 — Exponlo por HTTP

El dominio funciona, persiste y tiene caminos de escritura y de lectura. Ahora le ponemos un borde web. Un **controlador** mapea las peticiones HTTP sobre comandos y consultas y las despacha a través del bus —no contiene lógica de negocio propia—.

Primero el DTO de petición para abrir un monedero, bajo `interfaces/dtos/v1/`:

`lumen/interfaces/dtos/v1/open_wallet_request.py`

```
from __future__ import annotations

from pydantic import BaseModel, Field

from lumen.interfaces.enums.v1.currency import Currency


class OpenWalletRequest(BaseModel):
    """Wallet-opening request payload."""

    owner_id: str = Field(min_length=1, max_length=64,
description="Identifier of the wallet owner")
    currency: Currency = Field(default=Currency.EUR, description="ISO-4217
currency the wallet holds")
```

Listado 0.16 — La carga útil `OpenWalletRequest`

Ahora el controlador, bajo `web/controllers/`. El contenedor de inyección de dependencias inyecta los buses de comandos y de consultas; cada manejador construye un comando o una consulta y lo espera con `await` sobre el bus. Las anotaciones de parámetros declaran de dónde provienen los datos:

`Valid[Body[...]]` vincula y valida el cuerpo JSON, `PathVar[str]` vincula un segmento de la URL.

lumen/web/controllers/wallet_controller.py

```

from __future__ import annotations

from lumen.core.services.wallets.get_balance_query import GetBalance
from lumen.core.services.wallets.open_wallet_command import OpenWallet
from lumen.interfaces.dtos.v1.balance_dto import BalanceDto
from lumen.interfaces.dtos.v1.open_wallet_request import OpenWalletRequest
from pyfly.container import rest_controller
from pyfly.cqrs import DefaultCommandBus, DefaultQueryBus
from pyfly.kernel import ResourceNotFoundException
from pyfly.web import Body, PathVar, Valid, get_mapping, post_mapping,
request_mapping

@rest_controller
@request_mapping("/api/v1/wallets")
class WalletController:
    """Digital-wallet REST API: open a wallet, read its balance."""

    def __init__(self, commands: DefaultCommandBus, queries: DefaultQueryBus)
→ None:
        self._commands = commands
        self._queries = queries

    @post_mapping("", status_code=201)
    async def open_wallet(self, request: Valid[Body[OpenWalletRequest]]) →
dict[str, str]:
        wallet_id = await self._commands.send(
            OpenWallet(owner_id=request.owner_id, currency=request.currency)
        )
        return {"wallet_id": wallet_id}

    @get_mapping("/{wallet_id}/balance")
    async def wallet_balance(self, wallet_id: PathVar[str]) → BalanceDto:
        result = await self._queries.query(GetBalance(wallet_id=wallet_id))
        if result is None:
            raise ResourceNotFoundException(
                f"Wallet {wallet_id!r} not found",

```

```

        code="WALLET_NOT_FOUND",
        context={"wallet_id": wallet_id},
    )
    return result

```

Listado 0.17 — El controlador de monederos

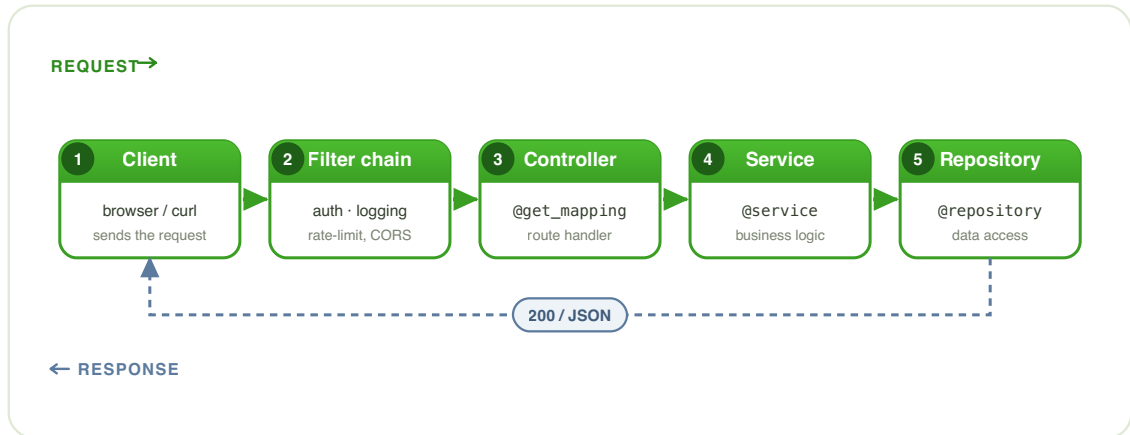


Figura 0.4 — Una petición se vincula a un manejador, que despacha un comando o una consulta a través del bus.

Ejecútalo

Inicia el servidor:

```
uv run pyfly run --server uvicorn
```

En otra terminal, abre un monedero:

```
curl -s -X POST localhost:8080/api/v1/wallets \
  -H 'content-type: application/json' \
  -d '{"owner_id":"alice","currency":"EUR"}'
```

```
{"wallet_id":"wlt-7d2c1a9e-..."}
```

Lee el saldo de vuelta (sustituye el id que obtuviste arriba):

```
curl -s localhost:8080/api/v1/wallets/wlt-7d2c1a9e-.../balance
```

```
{"id":"wlt-7d2c1a9e-...", "currency":"EUR", "balance_minor":0, "balance":0.0}
```

Un monedero recién abierto tiene saldo cero —exactamente lo que `Money.zero(EUR)` produjo allá en la fábrica `open` del agregado—. La petición viajó desde HTTP, a través del bus de comandos, hasta el manejador, a través del repositorio, hasta SQLite, y de vuelta por el camino de lectura. Ese es todo el arco, de extremo a extremo.

DOCUMENTACIÓN INTERACTIVA GRATIS

Mientras el servidor se ejecuta, abre `http://localhost:8080/docs` en un navegador. PyFly generó un documento OpenAPI y una interfaz Swagger UI a partir de tu controlador y tus DTOs —puedes probar los endpoints ahí mismo, sin código adicional—.

Paso 8 — Demuéstralo con una prueba

Ejecutar `curl` a mano está bien una vez; una prueba lo demuestra para siempre. PyFly está diseñado para ser testeable sin un servidor en ejecución —puedes despachar comandos y consultas directamente a través de los buses—. Escribe una prueba bajo `tests/`:

tests/test_quickstart.py

```

from __future__ import annotations

import pytest
from lumen.core.services.wallets.get_balance_query import GetBalance
from lumen.core.services.wallets.open_wallet_command import OpenWallet
from lumen.interfaces.enums.v1.currency import Currency

from pyfly.cqrs import DefaultCommandBus, DefaultQueryBus

@pytest.mark.asyncio
async def test_open_wallet_then_read_balance(
    command_bus: DefaultCommandBus,
    query_bus: DefaultQueryBus,
) → None:
    wallet_id = await command_bus.send(OpenWallet(owner_id="alice",
currency=Currency.EUR))
    assert wallet_id.startswith("wlt-")

    balance = await query_bus.query(GetBalance(wallet_id=wallet_id))
    assert balance is not None
    assert balance.balance_minor == 0
    assert balance.currency is Currency.EUR

```

Listado 0.18 — Una prueba de extremo a extremo a través de los buses

Los parámetros `command_bus` y `query_bus` son *fixtures*: arrancan el contexto de aplicación una vez y te entregan buses cableados, los mismos componentes que el controlador usa en producción. (El `samples/lumen/tests/conftest.py` terminado define estas fixtures; cópialo cuando construyas tu propia batería de pruebas —el Capítulo 16 lo explica al completo—.)

Ejecútalo

```
uv run --extra dev pytest -q
```

```
. [100%]  
1 passed in 0.42s
```

Verde. Ahora tienes una funcionalidad de monedero que no solo se ejecuta, sino que está *verificada* —la misma prueba se ejecuta en CI en cada cambio—.

🍃 EQUIVALENCIA CON SPRING

Despachar a través de los buses en una prueba refleja las pruebas de porción (slice tests) de Spring Boot: ejercitas beans reales cableados sin levantar el servidor HTTP. Las fixtures `command_bus` / `query_bus` son el equivalente en PyFly de un `ApplicationContext` de Spring inyectado en un `@SpringBootTest`.

Paso 9 — Una probada de eventos

El agregado ha estado registrando eventos de dominio todo este tiempo —

`WalletOpened`, `FundsDeposited` — y el manejador los drena con

`wallet.clear_events()` y los publica. Hasta ahora nada ha *escuchado*. Añadamos un pequeño oyente que reaccione.

Primero, el puente de publicación que el manejador ya importa. Convierte cada evento de dominio drenado en una carga útil y lo publica en el bus de eventos, bajo `core/services/wallets/`:

lumen/core/services/wallets/event_publishing.py

```

from __future__ import annotations

import dataclasses
from collections.abc import Iterable
from typing import Any

from lumen.core.services.listeners.wallet_audit_listener import
WALLET_EVENTS_DESTINATION
from pyfly.domain import DomainEvent
from pyfly.eda import EventPublisher

def _to_payload(event: DomainEvent) → dict[str, Any]:
    """Flatten a frozen-dataclass domain event into a JSON-friendly dict."""
    payload: dict[str, Any] = dataclasses.asdict(event)
    payload.setdefault("event_type", event.event_type)
    return payload

async def publish_domain_events(publisher: EventPublisher, events:
Iterable[DomainEvent]) → None:
    """Publish each drained domain event on the wallet events channel."""
    for event in events:
        await publisher.publish(
            destination=WALLET_EVENTS_DESTINATION,
            event_type=event.event_type,
            payload=_to_payload(event),
        )

```

Listado 0.19 — Publicando los eventos de dominio drenados

Ahora el oyente, bajo `core/services/listeners/`. Es un simple `@service` cuyo método está marcado con `@event_listener`; al arrancar, PyFly lo descubre y lo suscribe al bus —sin cableado a mano—. Aquí mantiene un diminuto registro de auditoría en memoria y un total acumulado por monedero.

```
lumen/core/services/listeners/wallet_audit_listener.py
```

```
from __future__ import annotations

from pyfly.container import service
from pyfly.eda import EventEnvelope, event_listener

# The logical channel the wallet handlers publish domain events to.
WALLET_EVENTS_DESTINATION = "wallet.events"

@service
class WalletAuditListener:
    """In-memory audit log + running-total projection over wallet events."""

    def __init__(self) → None:
        self._running_totals: dict[str, int] = {}

    @event_listener(event_types=["WalletOpened", "FundsDeposited"])
    async def on_wallet_event(self, envelope: EventEnvelope) → None:
        payload = dict(envelope.payload)
        wallet_id = str(payload.get("wallet_id", ""))
        if envelope.event_type == "WalletOpened":
            self._running_totals.setdefault(wallet_id, 0)
        elif envelope.event_type == "FundsDeposited":
            amount = int(payload.get("amount", 0))
            self._running_totals[wallet_id] =
self._running_totals.get(wallet_id, 0) + amount

    def running_total(self, wallet_id: str) → int:
        """Net funds for *wallet_id*, in minor units."""
        return self._running_totals.get(wallet_id, 0)
```

Listado 0.20 — Un oyente de eventos de dominio

Añade el paquete de oyentes a `scan_packages` en `app.py` para que el contenedor lo descubra:

```
scan_packages=[
    "lumen.models.repositories",
    "lumen.core.services.wallets",
    "lumen.core.services.listeners", # ← add this
    "lumen.web.controllers",
],
```

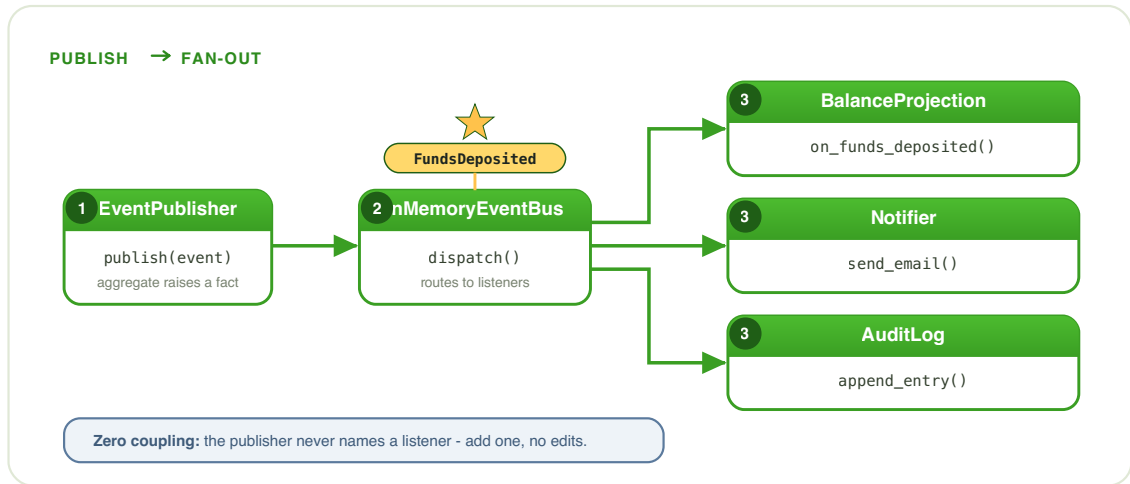


Figura 0.5 — Un manejador publica eventos de dominio; los oyentes se suscriben y reaccionan, desacoplados del comando.

Ejecútalo

Abre un monedero, luego haz un depósito, y el total acumulado del oyente se actualiza como efecto secundario de esos comandos —sin que el comando sepa que el oyente existe—. Ese desacoplamiento es el sentido mismo de los eventos: añades reacciones (registros de auditoría, notificaciones, proyecciones) sin tocar el código que las disparó.

🍃 EQUIVALENCIA CON SPRING

`@event_listener` es la contraparte en PyFly del `@EventListener` de Spring. Publicar a través de un `EventPublisher` y suscribirse con un método marcado es el mismo modelo de publicación/suscripción que el `ApplicationEventPublisher` de Spring y los beans anotados con `@EventListener`.

✓ Lo que construiste

Acabas de construir —y probar— una porción vertical real de un servicio: un modelo de dominio, una base de datos, un camino de escritura, un camino de lectura, un borde HTTP y una reacción a eventos. Cada una de esas cosas fue un *adelanto*. El resto del libro desarma cada una y la reconstruye como es debido, con el razonamiento, las alternativas y los detalles de producción.

Aquí tienes el mapa de lo que acabas de hacer al capítulo que profundiza:

En este Inicio rápido...	Se profundiza en
Viste el contenedor construir e inyectar tus beans (<code>@repository</code> , <code>@service</code>)	Capítulo 2 — Inyección de dependencias y el contexto de aplicación
Configuraste la aplicación con <code>pyfly.yaml</code> y el puerto de gestión	Capítulo 3 — Configuración, perfiles y secretos
Expusiste una API HTTP con <code>@rest_controller</code> , vinculación y validación	Capítulo 4 — Tu primera API HTTP
Persististe con un <code>Repository</code> del framework sobre SQLAlchemy/SQLite	Capítulo 5 — Persistencia y el patrón Repositorio
Modelaste <code>Money</code> , <code>Wallet</code> , invariantes y eventos de dominio	Capítulo 6 — Diseño orientado al dominio
Separaste escrituras y lecturas con comandos, consultas y el bus	Capítulo 7 — CQRS: comandos y consultas
Publicaste y reaccionaste a eventos de dominio con <code>@event_listener</code>	Capítulo 8 — Eventos de dominio y arquitectura orientada a eventos
(Próximamente) reconstruiste el estado a partir de un registro de eventos	Capítulo 9 — Event sourcing del libro mayor
(Próximamente) llamaste a otros servicios y dividiste el monolito	Capítulos 11–12 — Clientes HTTP, el BFF y las sagas
(Próximamente) lo aseguraste, observaste y desplegaste	Capítulos 14–18 — Seguridad, observabilidad, pruebas y producción

Cuando estés listo para el *porqué* de todo esto, pasa la página al Capítulo 1.

Pruébalo tú mismo

Si quieres seguir avanzando por tu cuenta primero, tres pequeñas extensiones se construyen directamente sobre lo que ya tienes:

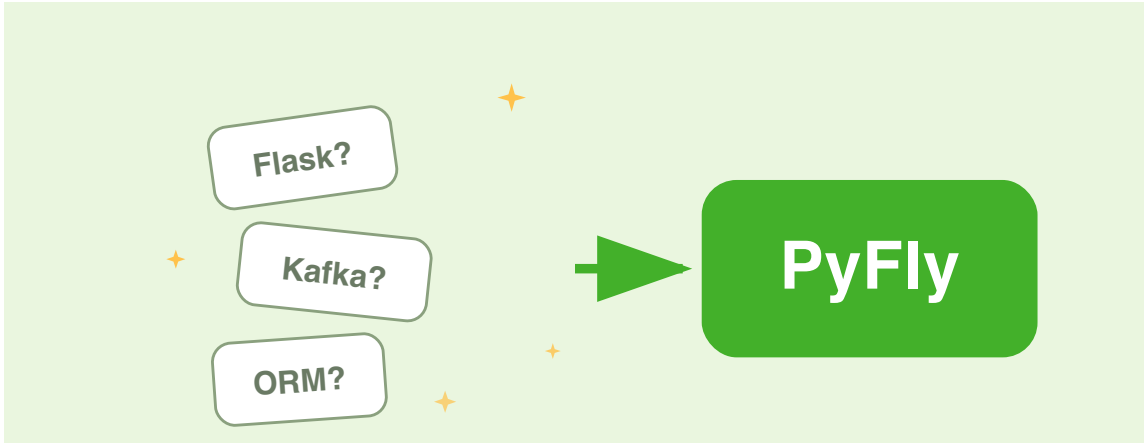
1. **Añade un endpoint de depósito.** Ya tienes el evento `FundsDeposited` y el método `deposit` del agregado. Añade un comando `DepositFunds` + manejador (modélalos sobre `OpenWallet`), una ruta `POST /{wallet_id}/deposit`, y observa cómo trepa el total acumulado del oyente.
2. **Añade un camino de `withdraw`** que se niegue a sobregirar —la invariante `balance ≥ 0` del agregado debería rechazarlo, y tu manejador debería exponerlo como un error limpio—. Añade un evento de dominio `FundsWithdrawn` que refleje el evento `FundsDeposited` mostrado en el Listado 0.6.
3. **Escribe una prueba para el oyente**, comprobando que abrir un monedero y depositar deja el total acumulado esperado —demostrando el camino de eventos de extremo a extremo—.

PARTE I

Fundamentos

CAPÍTULO 1

¿Por qué PyFly?



Al final de este capítulo habrás instalado PyFly, generado el esqueleto del servicio de monedero **Lumen** y lo habrás ejecutado en local, con registro estructurado, un endpoint de salud en vivo y documentación interactiva de la API ya funcionando, todo ello sin una sola línea de código repetitivo.

El problema de la cohesión

Imagina tu primer día en un nuevo microservicio en Python. Antes de escribir una sola línea de lógica de negocio, te enfrentas a dos semanas de decisiones de arquitectura.

¿A qué framework web recurres? FastAPI, Flask, Starlette, Django: cada uno es razonable, cada uno introduce sus propios modismos. ¿Qué ORM? ¿SQLAlchemy (síncrono o asíncrono?), Tortoise, Beanie? ¿Cómo conectas las dependencias? ¿Con

dependency-injector, python-inject o un módulo de fábrica hecho a mano? ¿Cómo gestionas la configuración? ¿Con pydantic-settings, python-dotenv, dynaconf? ¿Y cómo debería organizarse el proyecto? Cada equipo inventa su propia respuesta.

Con el tiempo ensamblas una pila a medida, la unes con buenas intenciones y la pones en producción. Seis meses después, un segundo equipo empieza un nuevo servicio y toma decisiones completamente distintas. Ahora tienes dos bases de código con convenciones incompatibles, estrategias de pruebas diferentes, patrones de despliegue distintos y ningún entendimiento compartido de cómo funciona nada.

Python te da una elección infinita. Lo que no te da es cohesión.

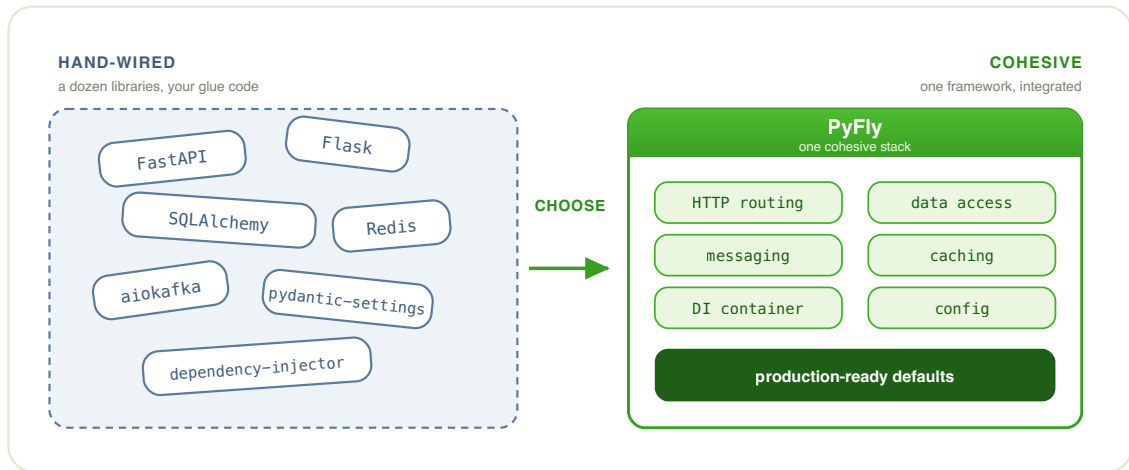


Figura 1.1 — Elección infinita, ninguna cohesión.

El problema del ensamblaje de la pila no es un fallo de habilidades: es una carencia de herramientas. Los desarrolladores de Java lo resolvieron hace años con Spring Boot: un único framework con criterio propio que toma decisiones sensatas por ti, te deja sobrescribir lo que importa y aplica un modismo coherente en cada servicio. PyFly aporta esa misma disciplina a Python.

¿Qué es PyFly?

La solución al problema de la cohesión no es prohibir la elección, sino tomar un buen conjunto de decisiones y empaquetarlas como un framework. Eso es precisamente lo que hace PyFly.

PyFly es un **framework cohesivo, de pila completa y nativamente asíncrono** para construir aplicaciones Python de nivel de producción: tanto microservicios como monolitos y bibliotecas. Toma las decisiones de la pila por ti: inyección de dependencias, enrutamiento HTTP, acceso a base de datos, mensajería, caché, seguridad, observabilidad, todo integrado, todo coherente, todo con valores por defecto listos para producción desde el primerísimo `pyfly run`.

Por dentro, PyFly delega en bibliotecas asíncronas probadas en batalla del ecosistema de Python (Starlette para HTTP, SQLAlchemy (async) para datos relacionales, structlog para el registro, Pydantic para la validación), pero tú nunca las importas directamente. Dependes de **los puertos de PyFly** (clases `Protocol` de Python) y el contenedor de inyección de dependencias conecta los adaptadores concretos en el arranque. Cambia PostgreSQL por MongoDB, o Kafka por RabbitMQ, sin tocar una sola línea de lógica de negocio.

PyFly es la **implementación oficial en Python del Firefly Framework**, una plataforma empresarial probada en batalla construida originalmente para Java (más de 40 módulos en producción). Aporta el mismo modelo de programación a Python 3.12+, no como una conversión sino como una reimplementación nativa diseñada en torno a `async/await` y las anotaciones de tipo.

Su arquitectura se asienta sobre cuatro capas (fundamento, aplicación, infraestructura e integración), cada una compuesta por módulos enfocados que encajan limpiamente entre sí.

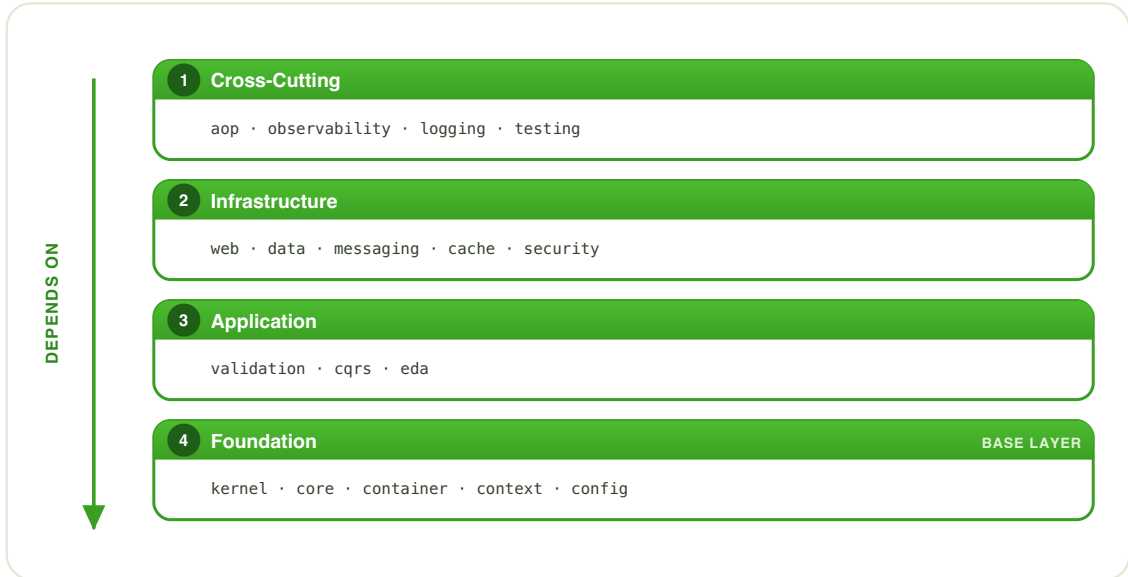


Figura 1.2 — Las cuatro capas de módulos de PyFly.

Cada capa sigue el mismo principio de **arquitectura hexagonal**: tu código vive en el centro y depende solo de los puertos; los adaptadores viven en los bordes y pueden intercambiarse sin perturbar el núcleo. Este diseño permite que el framework crezca contigo: empieza con un sencillo servicio REST y gradúate a CQRS, mensajería orientada a eventos u orquestación de sagas sin reescribir el fundamento que construiste el primer día.

🍃 EQUIVALENCIA CON SPRING

Si vienes de Spring Boot, PyFly te resultará familiar casi de inmediato.

`@pyfly_application` es tu `@SpringBootApplication`. `@service`, `@rest_controller` y `@repository` son exactamente los estereotipos que conoces. La inyección por constructor a partir de las anotaciones de tipo refleja `@Autowired` sin XML ni magia de reflexión. La jerarquía de configuración de `pyfly.yaml` (valores por defecto → perfil → variables de entorno) se corresponde directamente con `application.yaml` + perfiles.

Una llamada de atención de **Equivalencia con Spring** como esta aparece a lo largo del libro allí donde los conceptos se alinean lo suficiente como para ahorrarte el trabajo mental de traducción.

Instalar PyFly

Para concretar estas ideas, a lo largo del libro construirás **Lumen**, una plataforma fintech de monederos. Lumen empieza de forma sencilla: una API REST que registra asientos en el libro mayor e informa de los saldos de los monederos. Para el capítulo final abarcará varios servicios que se comunican mediante eventos, con sagas que coordinan transferencias entre servicios. Empezar con un esqueleto bien estructurado te ahorra el dolor de adaptar la arquitectura más tarde, así que el generador de esqueletos de PyFly crea esa estructura por ti desde el principio.

Iremos paso a paso, sin prisa. Aquí nada presupone experiencia previa con PyFly: si sabes ejecutar un comando en una terminal, podrás seguir el ritmo.

❗ TÉRMINO NUEVO: uv

`uv` es un gestor de paquetes y ejecutor de proyectos de Python muy rápido (piensa en `pip` + `virtualenv` + un ejecutor de tareas, todo en un único binario). PyFly lo recomienda, y cada comando de este libro que empieza con `uv run` significa simplemente "ejecuta esto dentro del entorno virtual del proyecto". Si solo has usado `pip`, no pierdes nada: `uv` lee el mismo `pyproject.toml` estándar.

Paso 1 — Comprueba los prerequisites. PyFly tiene como objetivo Python 3.12+ y usa `uv`. Confirma que ambos están instalados:

terminal

```
python --version
# Python 3.12.0 or later

uv --version
# uv 0.5.0 or later
```

Listado 1.1 — Verificar los prerequisites

Si alguno de los comandos falta o informa de una versión más antigua, instálalo antes de continuar (las instrucciones de instalación de `uv` son un breve script en su sitio web). Todo lo demás que PyFly necesita, lo trae él por ti.

Paso 2 — Genera el esqueleto del proyecto. Un único comando genera un directorio de proyecto completo y con forma de producción:

terminal

```
# web-api archetype, web feature
uv run pyfly new lumen --archetype web-api --features web
```

Listado 1.2 — Generar el esqueleto del servicio de monedero Lumen

❗ TÉRMINOS NUEVOS: ESQUELETO (SCAFFOLD) / ARQUETIPO

Generar el esqueleto (scaffold) es crear un esqueleto de proyecto ya hecho para empezar a partir de código que funciona en lugar de una carpeta vacía. Un *arquetipo* es la plantilla que usa la generación de esqueletos: `web-api` es la plantilla de servicio REST. La opción `--features web` añade la dependencia del servidor ASGI (la pieza que realmente acepta las peticiones HTTP). `uv run pyfly new` llama al registro de arquetipos de PyFly y escribe el directorio completo de una sola vez.

Paso 3 — Entra en el proyecto e instala las dependencias.**terminal**

```
cd lumen

# Install dependencies (including the pyfly CLI and ASGI server)
uv sync
```

Listado 1.3 — Entrar en el proyecto y sincronizar las dependencias

`uv sync` lee las dependencias declaradas en `pyproject.toml` y las instala en un entorno virtual local del proyecto, incluido el propio comando `pyfly` (disponible mediante el extra `cli`). La primera sincronización descarga los paquetes; las sincronizaciones posteriores son casi instantáneas porque uv cachea todo.

💡 PRUÉBALO: CONFIRMA QUE LA CLI FUNCIONA

Desde la raíz del proyecto, ejecuta `uv run pyfly --help`. Deberías ver la lista de comandos de PyFly (`new`, `run` y compañía). Si eso se imprime, tu cadena de herramientas está sana y estás listo para inspeccionar el proyecto generado.

 CONSEJO

Ejecuta `pyfly new` **sin argumentos** para entrar en el modo interactivo. Te guía por la selección de arquetipo y características con navegación mediante las teclas de flecha, algo práctico cuando quieres preseleccionar extras como soporte de datos relacionales o de mensajería.

El generador de esqueletos imprime la disposición generada en forma de árbol:

```

Created web-api project
lumen-demo/
├── .env.example
├── .gitignore
├── Dockerfile
├── README.md
├── pyfly.yaml
├── pyproject.toml
├── src/lumen_demo/__init__.py
├── src/lumen_demo/app.py
├── src/lumen_demo/controllers/__init__.py
├── src/lumen_demo/controllers/health_controller.py
├── src/lumen_demo/controllers/todo_controller.py
├── src/lumen_demo/main.py
├── src/lumen_demo/models/__init__.py
├── src/lumen_demo/models/todo.py
├── src/lumen_demo/repositories/__init__.py
├── src/lumen_demo/repositories/todo_repository.py
├── src/lumen_demo/services/__init__.py
├── src/lumen_demo/services/todo_service.py
├── tests/__init__.py
├── tests/conftest.py
└── tests/test_todo_service.py

```

Next steps:

```

cd lumen-demo
uv sync --group dev
pyfly run --reload

```

La aparente sencillez es deliberada. `pyproject.toml` te da un proyecto conforme a los estándares desde el primer día, sin la deuda heredada de `setup.py`. `pyfly.yaml` es la única fuente de verdad para cada ajuste de tiempo de ejecución, desde el nivel de registro hasta la URL de la base de datos, de modo que la configuración nunca se dispersa por una docena de archivos. La disposición `src/` evita que el paquete `lumen` sea importable accidentalmente sin una instalación explícita, detectando

temprano los errores de ruta de importación. El generador de esqueletos también crea controladores, servicios y repositorios de muestra ejecutables, de modo que tengas código que funciona para estudiar de inmediato en lugar de un cascarón vacío.

Dos archivos que importan

Comprender pronto la estructura de los puntos de entrada del esqueleto te ahorrará confusión más adelante. El esqueleto genera dos archivos que trabajan juntos: `app.py` declara la aplicación y `main.py` expone el `app` ASGI que importa el servidor. Cada uno tiene una responsabilidad distinta.

TÉRMINO NUEVO: ASGI

ASGI (Asynchronous Server Gateway Interface) es el contrato estándar entre un servidor web de Python y una aplicación web asíncrona: el sucesor moderno y asíncrono de WSGI. No tienes que implementarlo; PyFly le entrega al servidor un objeto `app` ASGI ya hecho. Recuerda solo esto: el *servidor* (Uvicorn o Granian) le habla ASGI a tu *app*.

Paso 1 — Abre la declaración de la aplicación. Mira `src/lumen/app.py` :

lumen/app.py

```

from pyfly.core import pyfly_application

@pyfly_application(
    name="lumen-demo",
    scan_packages=[
        "lumen_demo.controllers",
        "lumen_demo.services",
        "lumen_demo.repositories",
    ],
)
class Application:
    pass

```

Listado 1.4 — Declaración de la aplicación (@pyfly_application)

Por corto que sea, cada parámetro tiene su peso:

- **name** — la identidad del servicio. Aparece en cada evento de registro estructurado, en el título de OpenAPI y en las cargas útiles de las comprobaciones de salud: un identificador inequívoco cuando estás leyendo registros agregados de una docena de servicios.
- **scan_packages** — la instrucción que impulsa todo el sistema de inyección de dependencias. PyFly recorre cada módulo de la lista y registra en el `ApplicationContext` cualquier clase decorada con `@service`, `@rest_controller`, `@repository` o `@configuration`. Añade una nueva clase de servicio en cualquier punto de esos paquetes y se conecta automáticamente; no se requiere código de registro explícito.

El cuerpo de la clase `Application` está intencionadamente vacío. Nunca la instancias tú mismo: lo hace `PyFlyApplication` durante el arranque: carga la configuración desde `pyfly.yaml`, configura el registro estructurado, imprime el banner de arranque, escanea los paquetes, inicializa el `ApplicationContext` y registra los tiempos de arranque.

📌 TÉRMINOS NUEVOS: LA INYECCIÓN DE DEPENDENCIAS Y EL APPLICATIONCONTEXT

La *inyección de dependencias* significa que una clase declara lo que necesita en su constructor y el framework suministra esos colaboradores, en lugar de que la clase los construya ella misma. El `ApplicationContext` (a menudo simplemente "el contenedor") es el registro que contiene cada objeto conectado; PyFly llama a cada uno un *bean*, el mismo término que usa Spring. `scan_packages` es lo que lo puebla.

Paso 2 — Abre el punto de entrada ASGI. Ahora mira `src/lumen/main.py`:

lumen/main.py

```

from collections.abc import AsyncIterator
from contextlib import asynccontextmanager

from starlette.applications import Starlette

from pyfly.core import PyFlyApplication
from pyfly.web.adapters.starlette import create_app

from lumen_demo.app import Application

# Bootstrap: load config, scan packages, build DI context
_pyfly = PyFlyApplication(Application)

@asynccontextmanager
async def _lifespan(app: Starlette) → AsyncIterator[None]:
    """Manage application startup and shutdown lifecycle."""
    _pyfly._route_metadata = getattr(
        app.state, "pyfly_route_metadata", []
    )
    _pyfly._docs_enabled = getattr(
        app.state, "pyfly_docs_enabled", False
    )
    _pyfly._host = str(_pyfly.config.get("pyfly.server.host", "0.0.0.0"))
    _pyfly._port = int(_pyfly.config.get("pyfly.server.port", 8080))
    await _pyfly.startup()
    yield
    await _pyfly.shutdown()

app = create_app(
    title="lumen-demo",
    version="0.1.0",
    context=_pyfly.context,
    lifespan=_lifespan,
)

```

Listado 1.5 — Punto de entrada ASGI (main.py)

Este es el archivo que descubre `pyfly run` (y cualquier servidor ASGI como Uvicorn). El objeto `app` a nivel de módulo es una aplicación Starlette: `create_app` monta cada `@rest_controller` encontrado en el contexto de inyección de dependencias y conecta el gancho del ciclo de vida (lifespan) para que el arranque y el apagado ocurran limpiamente. El decorador `@pyfly_application` por sí solo **no** crea el `app` ASGI: `main.py` es el puente explícito entre el mundo de la inyección de dependencias (`app.py`) y el mundo HTTP.

LO QUE ACABA DE PASAR

Dos archivos pequeños dividen un trabajo en dos. `app.py` responde a *qué es esta aplicación y dónde vive su código* (nombre, versión, paquetes a escanear). `main.py` responde a *cómo la alcanza un servidor web*: arranca PyFly en un `ApplicationContext` y luego expone un único objeto `app` ASGI. Cuando más adelante añadas un controlador o un servicio, no editarás ninguno de los dos archivos: solo dejas la nueva clase en uno de los paquetes escaneados y PyFly la conecta en el siguiente arranque.

Archivos del proyecto

Los otros dos archivos que editarás con más frecuencia son `pyproject.toml` y `pyfly.yaml`. Cada uno desempeña un papel distinto: `pyproject.toml` gestiona las dependencias, mientras que `pyfly.yaml` es dueño de cada ajuste de tiempo de ejecución.

Paso 1 — Lee el manifiesto de dependencias. `pyproject.toml` registra las dependencias del proyecto. Fíjate en los extras:

lumen/pyproject.toml

```

[project]
name = "lumen-demo"
version = "0.1.0"
description = "lumen-demo - built with PyFly"
requires-python = "≥ 3.12"
dependencies = [
    "pyfly[web]",
]

[dependency-groups]
dev = [
    "pytest ≥ 8.0",
    "pytest-asyncio ≥ 0.23",
    "pytest-cov ≥ 5.0",
    "ruff ≥ 0.3",
    "mypy ≥ 1.8",
]

```

Listado 1.6 — pyproject.toml (secciones clave)

❗ TÉRMINO NUEVO: EXTRAS

Un *extra* es un paquete opcional de dependencias nombrado entre corchetes:

`pyfly[web]` significa "instala PyFly más su grupo opcional `web`". Los extras mantienen tu instalación ligera: incorporas un controlador de base de datos, una CLI o un servidor más rápido solo cuando realmente lo usas.

El extra `web` en `pyfly[web]` agrupa Uvicorn junto con el adaptador ASGI. El extra `cli` (aíñádelo cuando necesites el comando `pyfly` fuera de `uv run`) proporciona las herramientas de línea de comandos. Otros extras (`data-relational`, `granian`) son opcionales; los añades solo cuando una característica los necesita. La muestra del monedero Lumen, por ejemplo, declara `pyfly[cli,web,data-relational]` porque se conecta a SQLite.

Paso 2 — Lee la configuración de tiempo de ejecución. `pyfly.yaml` es donde viven todos los ajustes de tiempo de ejecución:

`lumen/pyfly.yaml`

```
pyfly:
  app:
    name: lumen-demo
    version: 0.1.0
    module: lumen_demo.main:app

  server:
    port: 8080
    type: "auto"
    event-loop: "auto"
    workers: 0

  actuator:
    endpoints:
      enabled: true

  admin:
    enabled: true

  logging:
    level:
      root: INFO
```

Listado 1.7 — `pyfly.yaml` (valor por defecto del esqueleto)

La clave `module` le dice a `pyfly run` exactamente dónde vive el `app` ASGI: `lumen_demo.main:app`. La aplicación escucha en `pyfly.server.port` (por defecto `8080`); el tipo de servidor, los endpoints del actuador y el nivel de registro también tienen valores por defecto sensatos, así que solo sobrescribes lo que realmente necesitas cambiar.

🍃 EQUIVALENCIA CON SPRING

`pyfly.server.port` es el equivalente directo de `server.port` de Spring Boot, hasta el mismo valor por defecto `8080`. La variable de entorno de sobrescritura correspondiente es `PYFLY_SERVER_PORT` (el `SERVER_PORT` de Spring). La identidad de la aplicación sigue el mismo patrón: `pyfly.app.name` y `pyfly.app.version` reflejan `spring.application.name`.

⚠ CLAVE RENOMBRADA: USA SERVER.PORT, NO WEB.PORT

Versiones anteriores de PyFly usaban `pyfly.web.port` (y la variable de entorno `PYFLY_WEB_PORT`) para el puerto de escucha. Ambas fueron **eliminadas** en la v26.06.102 en favor de `pyfly.server.port` / `PYFLY_SERVER_PORT`. Si copias una configuración antigua y el puerto parece ignorado, casi siempre es por esto: renombra la clave bajo `server:` y la sobrescritura surte efecto.

📄 SERVIDOR POR DEFECTO: GRANIAN FRENTE A UVICORN

El servidor ASGI por defecto de PyFly es **Granian**, un servidor de alto rendimiento basado en Rust. La dependencia `pyfly[web]` del esqueleto agrupa **Uvicorn** en su lugar (instalación más ligera). Los dos son intercambiables: pasa `--server uvicorn` en la línea de comandos, o añade `pyfly[granian]` a tus dependencias para usar Granian. Por esta razón, la muestra del monedero Lumen se ejecuta con `--server uvicorn`.

Tu primera ejecución

Con el proyecto en su sitio, arranca el servidor.

Paso 1 — Arranca el servidor. Desde la raíz del proyecto, ejecuta:

terminal

```
uv run pyfly run --server uvicorn
```

Listado 1.8 — Ejecutar el servidor con uv

❗ POR QUÉ `--SERVER UVICORN`

El servidor por defecto de PyFly es Granian, pero el extra `pyfly[web]` trae Uvicorn (una instalación más ligera). Pasar `--server uvicorn` le dice a PyFly que use el servidor que realmente tienes. Si alguna vez ves un error sobre que Granian no está instalado, esta opción es la solución, o bien añade `pyfly[granian]` y omite la opción.

Tras un momento verás el banner ASCII de PyFly seguido de un flujo de eventos estructurados de arranque:

```

      -----
----- .-.-/ ----\ | ---.-.
\___ <  | \|  _\| |<  | |
| |_> >__ | | | | |_\___ |
|  _// ____| | | | ____/ ____|
|_|  V                \/>

```

```

:: PyFly Framework :: (v26.06.110) (Python 3.13.13)
Copyright 2026 Firefly Software Foundation. | Apache License 2.0
2026-06-07 20:34:32,442 [INFO] pyfly.core: starting_application |
    app=lumen-demo version=0.1.0 python=3.13.13 pid=72300
2026-06-07 20:34:32,442 [INFO] pyfly.core: runtime_environment |
    os=Darwin os_version=25.5.0 arch=arm64 cpus=11
2026-06-07 20:34:32,442 [INFO] pyfly.core: loaded_config |
    source=pyfly-defaults.yaml (framework defaults)
2026-06-07 20:34:32,442 [INFO] pyfly.core: loaded_config |
    source=pyfly.yaml
2026-06-07 20:34:32,442 [INFO] pyfly.core: scanned_package |
    package=lumen-demo.controllers beans_found=1
2026-06-07 20:34:32,580 [INFO] pyfly.core: bean_summary |
    total=127 services=6 repositories=2 controllers=4
2026-06-07 20:34:32,580 [INFO] pyfly.core: mapped_endpoints | count=5
2026-06-07 20:34:32,580 [INFO] pyfly.core: request_mapping |
    method=POST path=/api/v1/wallets handler=open_wallet
2026-06-07 20:34:32,580 [INFO] pyfly.core: api_documentation |
    swagger_ui=http://0.0.0.0:8080/docs
2026-06-07 20:34:32,580 [INFO] pyfly.core: management_server |
    url=http://0.0.0.0:9090 endpoints=actuator + admin
2026-06-07 20:34:32,580 [INFO] pyfly.core: admin_dashboard |
    url=http://0.0.0.0:9090/admin
2026-06-07 20:34:32,581 [INFO] pyfly.core: server_started |
    server=uvicorn host=0.0.0.0 port=8080 workers=1

```

Lee estas líneas de registro como la historia de lo que el framework hizo en tu nombre. Cada evento tiene un nombre en lugar de un mensaje en bruto, de modo que los agregadores de registros puedan filtrar por clave:

1. `starting_application` — PyFly se anuncia con el nombre y la versión del servicio que declaraste, dando a cada agregador de registros un filtro limpio desde el primerísimo evento.
2. `runtime_environment` — el sistema operativo, la arquitectura y el número de CPU se emiten una vez en el arranque, lo que facilita correlacionar el comportamiento con el entorno más adelante.
3. `loaded_config` (x2) — la configuración está en capas: el framework trae `pyfly-defaults.yaml` con valores por defecto seguros para producción; tu `pyfly.yaml` sobrescribe solo lo que necesitas. Activa un perfil (p. ej., `--profile prod`) y verás una tercera entrada.
4. `scanned_package` — el contenedor de inyección de dependencias encontró un `@rest_controller` en el paquete de controladores web. `beans_found` crece a medida que añades servicios y repositorios.
5. `bean_summary` — el tamaño total del contexto de inyección de dependencias, incluidos los beans internos del framework. Incluso con 127 beans registrados, PyFly arranca en mucho menos de un segundo.
6. `management_server` — los endpoints del actuador y el panel de administración se sirven en un puerto de gestión *separado* (por defecto `9090`), independiente del `8080` de la aplicación.
7. `admin_dashboard` — la URL exacta de la interfaz de administración en vivo, `http://0.0.0.0:9090/admin`.
8. `server_started` — Uvicorn está aceptando conexiones en el puerto de la aplicación.

i LO QUE ACABA DE PASAR

Ejecutaste un solo comando y PyFly hizo el trabajo de toda una jornada de código repetitivo: cargó la configuración en capas, escaneó tus paquetes, conectó el contenedor de inyección de dependencias, mapeó tus rutas HTTP, publicó documentación interactiva de la API, levantó un servidor de gestión separado con comprobaciones de salud y un panel de administración, y empezó a aceptar peticiones, todo registrado como eventos con nombre y filtrables por máquina. Deja este servidor ejecutándose en su terminal; lo llamarás desde una segunda terminal en los siguientes pasos. Pulsa **Ctrl+C** cuando quieras detenerlo.

🍃 EQUIVALENCIA CON SPRING

Servir el actuador y el panel de administración en un puerto dedicado refleja el `management.server.port` de Spring Boot. En PyFly la clave es `pyfly.management.server.port` (por defecto `9090`). Igualala a `pyfly.server.port` para un comportamiento de puerto único, o ponla a `-1` para deshabilitar por completo los endpoints de gestión. Por defecto, este puerto está **abierto y sin autenticación** (también equivalencia con Spring): bien para el desarrollo local, pero en producción asegúralo con `pyfly.management.security.enabled: true`.

Dos endpoints ya están en vivo antes de que hayas escrito una sola línea de lógica de aplicación. Abre una *segunda* terminal (deja el servidor ejecutándose en la primera) y prueba la comprobación de salud.

Paso 2 — Comprueba el endpoint de salud en vivo. Vive en el puerto de gestión, `9090`:

terminal

```
curl -s localhost:9090/actuador/health
```

Listado 1.9 — Comprobación de salud (puerto de gestión 9090)

❗ TÉRMINO NUEVO: ACTUADOR

El *actuador* es la capa de operaciones incorporada de PyFly: un pequeño conjunto de endpoints HTTP que informan sobre la aplicación en ejecución: `/actuator/health`, `/actuator/info` y (cuando los exposes) métricas, entorno y más. Es el mismo concepto, y la misma ruta base `/actuator`, que Spring Boot Actuator. Por defecto solo se exponen por HTTP `health` e `info`; amplía eso con `pyfly.management.endpoints.web.exposure.include`.

Deberías ver un documento JSON con un `"status": "UP"` de nivel superior y un mapa `components`, una entrada por cada indicador de salud que el framework registró:

```
{
  "status": "UP",
  "components": {
    "cache_health_indicator": {
      "status": "UP",
      "details": {"adapter": "InMemoryCache", "latencyMs": 0.01}
    },
    "cqrs_health_indicator": {
      "status": "UP",
      "details": {"command_handlers": 3, "query_handlers": 2}
    },
    "eda_health": {
      "status": "UP",
      "details": {"adapter": "InMemoryEventBus"}
    },
    "db_health_indicator": {
      "status": "UP",
      "details": {"database": "sqlite"}
    }
  }
}
```

Paso 3 — Abre un monedero. Ahora llama al primer endpoint de negocio. A diferencia del actuador, este vive en el puerto de la *aplicación*, `8080`:

terminal

```
curl -s -X POST localhost:8080/api/v1/wallets \
  -H 'content-type: application/json' \
  -d '{"owner_id":"u-1","currency":"EUR"}'
```

Listado 1.10 — Abrir un monedero (puerto de aplicación 8080)

La respuesta es el identificador del nuevo monedero (tu UUID será distinto):

```
{"wallet_id": "wlt-c5bbb2a7-dd49-4321-932e-e4c6bfa5cc2c"}
```

LO QUE ACABA DE PASAR

Ese único `curl` viajó por toda la pila que el framework construyó para ti: el cuerpo JSON se validó contra el modelo `OpenWalletRequest`, el `WalletController` lo convirtió en un comando `OpenWallet`, el bus de comandos lo despachó a su manejador (handler), se creó y se persistió una raíz de agregado `Wallet`, y el nuevo id volvió como JSON. Construirás cada una de esas capas por ti mismo en capítulos posteriores; por ahora, fíjate en que ya funciona de extremo a extremo.

Paso 4 — Explora la documentación interactiva. Abre `http://localhost:8080/docs` en tu navegador para la interfaz interactiva de Swagger; `http://localhost:8080/redoc` te da ReDoc; la especificación OpenAPI en bruto está en `http://localhost:8080/openapi.json`. La documentación vive en el puerto de la aplicación junto a tu API.

NOTA

Las tres URL de documentación se emiten en el registro de arranque (entradas `api_documentation`), así que nunca tienes que recordarlas. El Panel de Administración de PyFly (una vista en vivo de los beans, los endpoints, los indicadores de salud y los eventos de registro recientes) se abre en `http://localhost:9090/admin` en el puerto de gestión (su URL exacta es la línea `admin_dashboard` del registro de arranque).

Paso 5 — Ejecuta las pruebas generadas. El esqueleto trae una batería de pruebas que funciona para que puedas confirmar la cadena de herramientas de extremo a extremo. Instala las herramientas de desarrollo y luego ejecuta pytest:

terminal

```
uv sync --group dev
uv run --group dev pytest -q
```

Listado 1.11 — Ejecutar la batería de pruebas

SALIDA ESPERADA

Deberías ver una breve sucesión de puntos (o líneas `PASSED`) y un resumen verde como `3 passed in 0.42s`; el número exacto depende del arquetipo. Cualquier resumen verde significa que PyFly, tu entorno virtual y pytest están todos conectados correctamente. (La muestra del monedero Lumen que estudias a lo largo del libro trae una batería mucho mayor, pero se ejecuta del mismo modo: `uv run --group dev pytest -q`.)

Eso completa el ciclo: instalar, generar el esqueleto, ejecutar, llamar y probar, todo antes de escribir una línea de tu propia lógica.

✓ Lo que construiste

En menos de cinco minutos instalaste PyFly, generaste el esqueleto de un servicio con forma de producción y lo ejecutaste en local. La tabla siguiente resume lo que Lumen ya entrega (enteramente desde el framework) antes de que hayas escrito una sola línea de lógica de aplicación.

Capacidad	Cómo la obtuviste
Registro JSON estructurado con metadatos de tiempo de ejecución	Valor por defecto del framework: emitido en cada arranque
Documentación interactiva de la API (Swagger UI + ReDoc)	Incorporada; siempre activa, desactívala mediante <code>pyfly.yaml</code>
Endpoint de salud (<code>/actuator/health</code> en el puerto 9090)	<code>actuator.endpoints.enabled: true</code> en el esqueleto
Configuración en capas consciente de los perfiles	<code>pyfly.yaml</code> + la opción <code>--profile</code>
Contenedor de inyección de dependencias con autodescubrimiento	<code>scan_packages</code> en <code>@pyfly_application</code>
Tiempos de arranque, resumen de beans, mapa de endpoints	Emitidos como eventos de registro estructurado en cada arranque
Panel de Administración (<code>/admin</code> en el puerto de gestión 9090)	Habilitado en el esqueleto por defecto

Esa es la promesa de PyFly: decisiones sensatas tomadas por ti, convenciones coherentes en cada servicio y valores por defecto listos para producción desde la primerísima ejecución.

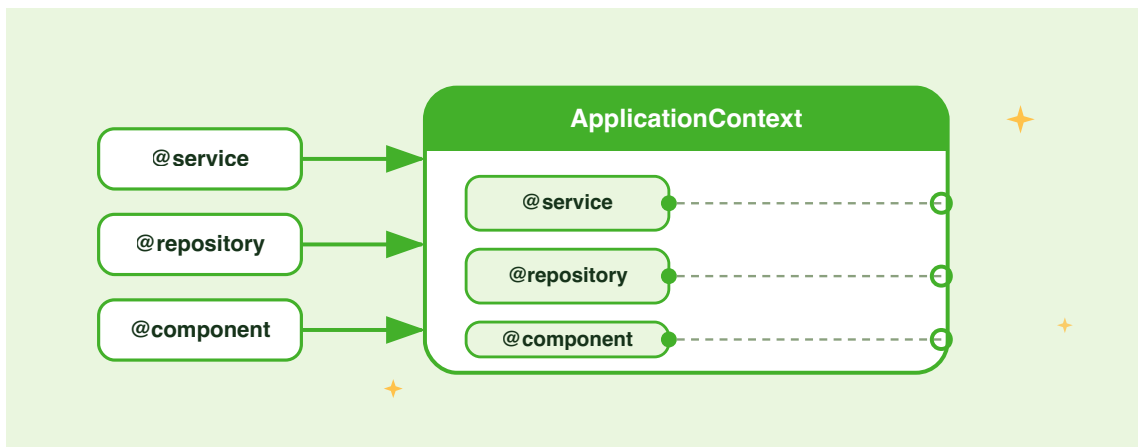
Pruébalo tú mismo

- 1. Renombra el servicio.** Abre `pyfly.yaml` y cambia `app.name` de `lumen-demo` a `lumen-wallet`. Actualiza también el parámetro `name` en `@pyfly_application`. Vuelve a ejecutar con `uv run pyfly run --server uvicorn` y confirma que el nuevo nombre aparece en la línea de registro `starting_application`.
- 2. Explora la documentación en vivo.** Con el servidor en ejecución, abre `http://localhost:8080/docs`. Fíjate en el nombre y la versión del servicio en la parte superior de la interfaz de Swagger. Después navega a `http://localhost:8080/redoc` y compara los dos renderizadores de documentación. Comprueba la respuesta de salud en `http://localhost:9090/actuator/health` (recuerda: el actuator vive en el puerto de gestión, no en el puerto de la aplicación) y anota qué indicadores de salud ya están informando. Abre `http://localhost:9090/admin` para ver la misma información en el panel en vivo.
- 3. Cambia a Granian.** Añade `pyfly[granian]` a la lista `dependencies` en `pyproject.toml`, ejecuta `uv sync` y luego arranca el servidor sin la opción `--server uvicorn`. Compara la línea de registro `server_started`: el campo `server` ahora debería decir `granian`.
- 4. Asigna archivos a responsabilidades.** Mira los dos archivos de punto de entrada `app.py` y `main.py`. Escribe una descripción de una sola frase de lo que cada uno es responsable. ¿De dónde viene el contenedor de inyección de dependencias? ¿Dónde vive el objeto `app` ASGI? ¿Cuál editarías para cambiar los paquetes a escanear?

- 5. Colapsa a un único puerto.** Añade `pyfly.management.server.port: 8080` bajo la sección `management:` hermana de `server` en `pyfly.yaml` (de modo que sea igual a `pyfly.server.port`). Vuelve a ejecutar el servidor y observa el registro de arranque: la línea `management_server` desaparece y `http://localhost:8080/actuator/health` ahora responde en el puerto de la aplicación. Después prueba `pyfly.management.server.port: -1`, vuelve a ejecutar y confirma que las rutas del actuador y de administración han desaparecido por completo.

CAPÍTULO 2

Inyección de dependencias y el contexto de aplicación



En el capítulo anterior le diste a Lumen su punto de entrada de aplicación y observaste cómo arrancaba el contenedor. Ahora vas a declarar los primeros componentes reales de Lumen — un `WalletRepository` que hereda del `Repository` estilo Spring Data del framework, un `WalletEntity` que mapea la fila de persistencia y un manejador CQRS que depende de ambos — y dejarás que PyFly los conecte entre sí a partir de nada más que anotaciones de tipos. Sin factorías, sin `new` manual, sin código de pegamento.

Este capítulo es práctico. Construiremos cada pieza en pasos pequeños y numerados, y después de cada hito hay un punto de control **Ejecútalo** que muestra el comando exacto que debes escribir y la salida que deberías ver en pantalla. Si tu salida coincide,

vas por buen camino; si no, la diferencia te indica con precisión qué corregir. Vas siguiendo el ejercicio dentro de `samples/Lumen` con PyFly **v26.6.110** instalado (`uv sync` desde el directorio de Lumen lo descarga).

NUEVO TÉRMINO: CONTENEDOR

A lo largo del capítulo hablamos de *el contenedor*. Un **contenedor** es simplemente el objeto que PyFly crea al arrancar y que sabe cómo construir, conectar y entregar todos los objetos de tu aplicación. Nunca lo construyes tú mismo — cuando ejecutas la aplicación, PyFly levanta el contenedor, lo llena con tus beans y lo mantiene vivo hasta el apagado. Piénsalo como un almacén inteligente que tanto guarda tus objetos como los ensambla bajo demanda.

Antes de que aparezca una sola línea de código de Lumen, conviene detenerse en *por qué* eso importa. En un proyecto Python convencional escribirías algo como:

```
handler = DepositFundsHandler(
    repository=InMemoryWalletRepository(),
    events=InMemoryEventBus(),
)
```

en algún punto cercano a la ruta de arranque. Esa línea parece inofensiva, pero fija cada decisión — qué clase de repositorio, qué bus de eventos — en el punto de construcción. Cambia el repositorio por un adaptador de Postgres y tendrás que encontrar cada lugar de construcción. Añade un doble de prueba y necesitarás reestructurar el cableado. La **inyección de dependencias** invierte esta relación: las clases *declaran* lo que necesitan, y el contenedor *decide* qué proporcionar. El resultado es código abierto a la extensión pero cerrado a la modificación — el `DepositFundsHandler` que escribes hoy aceptará un adaptador de base de datos de producción en la Parte II sin un solo cambio en su código fuente.

Estereotipos: declarar tus beans

Antes de que el contenedor pueda conectar nada, necesita saber qué clases gestionar. Un **bean** es cualquier objeto que el contenedor crea, conecta y posee. Haces que una clase sea visible para el contenedor aplicando un **decorador de estereotipo** — una anotación ligera que registra la clase y señala su rol arquitectónico.

NUEVOS TÉRMINOS: BEAN Y ESTEREOTIPO

Un **bean** no es más que una instancia que el contenedor posee — la construye, le suministra sus dependencias y (normalmente) mantiene una única copia compartida. Un **estereotipo** es el decorador que pones en una clase para decir "contenedor, esto es tuyo, por favor gestiónalo". La palabra *estereotipo* está tomada de Spring; simplemente significa "un rol etiquetado". Poner `@service` en una clase es todo el acto de registro — no hay un fichero de configuración aparte que editar.

PyFly incluye cinco estereotipos:

Decorador	Significado
<code>@service</code>	Capa de lógica de negocio: operaciones de dominio, orquestación de casos de uso.
<code>@component</code>	Bean gestionado genérico sin un rol arquitectónico específico.
<code>@repository</code>	Capa de acceso a datos: bases de datos, almacenamiento externo, puertos.
<code>@configuration</code>	Clase de configuración que puede contener métodos factoría <code>@bean</code> .
<code>@rest_controller</code>	Capa HTTP: gestiona peticiones y devuelve respuestas JSON.

Los cinco estereotipos son **equivalentes a nivel de contenedor**: comparten la misma factoría interna `_make_stereotype()` y aceptan los mismos argumentos de palabra clave opcionales (`name`, `scope`, `profile`, `condition`). Las diferencias significativas son la etiqueta `__pyfly_stereotype__` — usada por la capa web para descubrir controladores y por el contexto para encontrar clases `@configuration` — y la claridad arquitectónica que cada nombre aporta a quienes leen tu código. Elegir `@repository` en lugar de `@component` no cuesta nada técnicamente, pero le dice a todo futuro lector exactamente para qué sirve la clase.

Funcionan tanto la forma simple como la forma con paréntesis:

```
@service          # bare — all defaults
class SimpleService:
    pass

@service(name="wallet_svc") # with keyword args
class NamedService:
    pass
```

El arranque con `scan_packages`

El contenedor solo descubre beans en los paquetes que se le ha indicado escanear. En `lumen/app.py`, `@pyfly_application` lista cada subpaquete que el contenedor debe inspeccionar en busca de declaraciones de estereotipo:

lumen/app.py

```

from pyfly.core import pyfly_application
from pyfly.starters.domain import enable_domain_stack

@enable_domain_stack
@pyfly_application(
    name="lumen",
    version="1.0.0",
    description=(
        "Lumen — a DDD digital-wallet service"
        " built on the PyFly framework."
    ),
    scan_packages=[
        "lumen.models.repositories",
        "lumen.core.services.wallets",
        "lumen.core.services.transfers",
        "lumen.core.services.listeners",
        "lumen.web.controllers",
    ],
)
class LumenApplication:
    pass
    
```

Listado 2.1 — Punto de entrada de la aplicación con `scan_packages`

Cómo funciona. `@pyfly_application` registra `LumenApplication` como la raíz de la aplicación y siembra el contenedor con las autoconfiguraciones del framework.

`scan_packages` es la lista exacta de rutas de paquetes Python que el contenedor recorre al arrancar, recolectando cada clase decorada con un estereotipo. Cualquier paquete que no esté listado aquí es invisible para el contenedor — la fuente más habitual de confusión del tipo "¿por qué no se encuentra mi bean?" al añadir nuevos

subpaquetes. `@enable_domain_stack` activa en una sola línea las autoconfiguraciones de CQRS, el motor transaccional, event sourcing, datos relacionales y el motor de reglas.

Lee ese listado como cuatro decisiones:

- **Paso 1 — nombrar la aplicación.** `name="lumen"` y `version="1.0.0"` se convierten en la identidad que reportan el banner de arranque y el endpoint `/actuador/info`. (Estos son el nombre y la versión de la *aplicación*; la versión del framework es aparte — aquí, v26.6.110.)
- **Paso 2 — describirla.** `description=...` son metadatos orientados a personas que se muestran en la documentación de API generada.
- **Paso 3 — listar los paquetes a escanear.** Cada entrada de `scan_packages` es una ruta Python con puntos que el contenedor importará e inspeccionará. Si una clase con un estereotipo vive en un paquete que *no* está en esta lista, el contenedor nunca la verá.
- **Paso 4 — habilitar el stack.** `@enable_domain_stack` enciende las autoconfiguraciones de la capa de dominio para que los buses de CQRS, la sesión transaccional y la capa de datos relacional existan todos antes de que tus beans se conecten.

💡 **CONSEJO: ESCANEA EL PAQUETE, NO LA CLASE**

Las entradas de `scan_packages` son rutas de *paquete* (`lumen.web.controllers`), nunca rutas de módulo o clase individuales. El contenedor recorre el paquete y descubre cada clase decorada con un estereotipo dentro de él. Cuando añades un nuevo manejador bajo `lumen.core.services.wallets`, se recoge automáticamente — sin necesidad de editar `scan_packages`. Solo tocas esta lista cuando introduces un subpaquete completamente nuevo.

Ejecútalo. Desde la raíz del proyecto Lumen, arranca la aplicación y observa cómo el contenedor se ensambla a sí mismo:

```
cd samples/lumen
uv run pyfly run --server uvicorn
```

Deberías ver el banner seguido de líneas estructuradas de arranque — el contenedor reportando exactamente lo que escaneó y conectó:

```
:: PyFly Framework :: (v26.6.110) (Python 3.12.13)

pyfly.core: starting_application | app=lumen version=1.0.0
pyfly.core: scanned_package | package=lumen.models.repositories beans_found=1
pyfly.core: scanned_package | package=lumen.core.services.wallets
beans_found=7
pyfly.core: scanned_package | package=lumen.core.services.transfers
beans_found=2
pyfly.core: scanned_package | package=lumen.core.services.listeners
beans_found=1
pyfly.core: scanned_package | package=lumen.web.controllers beans_found=1
pyfly.core: bean_summary | total=137 services=10 repositories=1 controllers=4
configurations=19
pyfly.core: server_started | server=uvicorn host=0.0.0.0 port=8080 workers=1
pyfly.core: application_started | app=lumen startup_time_s=0.143
beans_initialized=137
```

Las líneas `scanned_package` son `scan_packages` haciendo su trabajo: una línea por entrada, cada una reportando cuántos beans encontró. La línea final `application_started` — el equivalente del "Started Application in N seconds" de Spring Boot — es tu señal de que el contexto arrancó limpiamente. Pulsa `Ctrl-C` para detener el servidor.

NUEVO TÉRMINO: EL PUERTO DE GESTIÓN

Aparecen dos puertos al arrancar. La API HTTP de tu aplicación escucha en `pyfly.server.port` (por defecto **8080**). Los endpoints operativos — la comprobación de salud, info y el panel de administración — escuchan por separado en el **puerto de gestión** `pyfly.management.server.port` (por defecto **9090**), que está abierto y sin autenticación por defecto. No tocarás ninguno de los dos en este capítulo, pero vale la pena saber por qué se encienden dos puertos. (La clave antigua `pyfly.web.port` se eliminó en v26.6.102; usa siempre `pyfly.server.port` ahora.)

Qué acaba de pasar. Un solo comando hizo mucho. PyFly cargó `pyfly.yaml`, importó cada paquete de `scan_packages`, encontró cada clase decorada con un estereotipo, le pidió al contenedor que las construyera en orden de dependencias y reportó los totales — 137 beans, de los cuales la mayoría son beans de autoconfiguración del framework y solo un puñado son tuyos por ahora. A partir de aquí, el resto del capítulo trata de *añadir* a ese recuento de beans: una entidad, un repositorio y un manejador de comandos, cada uno descubierto exactamente por este mecanismo.

EQUIVALENCIA CON SPRING

`scan_packages` es el equivalente de `@ComponentScan(basePackages = {...})` de Spring. La semántica es idéntica: lista cada subpaquete que quieras que el framework inspeccione, y registrará todo lo que encuentre. La línea de log `application_started` refleja el resumen de arranque "Started Application in N seconds (process running for M)" de Spring Boot.

La entidad y el repositorio

Lumen almacena los monederos en una base de datos relacional. Dos clases asumen esta responsabilidad: `WalletEntity` (la fila de persistencia) y `WalletRepository` (el bean de acceso a datos). Las construiremos en orden: primero la forma de la fila, luego el bean de acceso a datos que la lee y la escribe.

NUEVO TÉRMINO: ENTIDAD

Una **entidad** es la forma en base de datos de un registro — aquí, un monedero, almacenado como una fila en una tabla `wallets`. Es deliberadamente simple: solo columnas tipadas, sin comportamiento. (Lumen mantiene aparte el objeto de dominio *rico*, la raíz de agregado `Wallet`; lo conocerás en el Capítulo 6. Por ahora, la entidad es simplemente la forma en que un monedero se escribe en la base de datos y se lee de ella.)

La entidad. `WalletEntity` es una clase mapeada con SQLAlchemy que hereda el `Base` del framework. Constrúyela campo a campo:

- **Paso 1 — heredar `Base`.** Heredar del `Base` declarativo de PyFly es lo que inscribe la clase en los metadatos del ORM para que el framework pueda crear su tabla.
- **Paso 2 — nombrar la tabla.** `__tablename__ = "wallets"` es la tabla SQL a la que esta clase mapea.
- **Paso 3 — declarar la clave primaria.** `id` es una columna `str` marcada `primary_key=True` — el monedero conserva su propio id de dominio (`wlt-...`) en lugar de un número generado.
- **Paso 4 — declarar las columnas restantes.** `owner_id`, `currency`, `balance_minor` (el saldo en unidades menores — céntimos — de modo que nunca haya un error de redondeo de coma flotante) y una marca de tiempo `created_at`.

lumen/models/entities/v1/wallet_orm.py

```

from __future__ import annotations

from datetime import UTC, datetime

from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column

from pyfly.data.relational.sqlalchemy import Base, UtcDateTime


class WalletEntity(Base):
    """One persisted wallet row, keyed by the aggregate's own id."""

    __tablename__ = "wallets"

    id: Mapped[str] = mapped_column(
        String(64), primary_key=True
    )
    owner_id: Mapped[str] = mapped_column(
        String(255), nullable=False, index=True
    )
    currency: Mapped[str] = mapped_column(String(3), nullable=False)
    balance_minor: Mapped[int] = mapped_column(
        nullable=False, default=0
    )
    created_at: Mapped[datetime] = mapped_column(
        UtcDateTime(), default=lambda: datetime.now(UTC)
    )
    
```

 Listado 2.2a — *WalletEntity: la fila de persistencia*

Heredar `Base` (la base declarativa de PyFly) registra la tabla `wallets` en `Base.metadata`; el ciclo de vida del motor del framework la crea al arrancar. No se necesita más cableado.

Ejecútalo. Con `ddl-auto: create` establecido en `pyfly.yaml`, el framework construye el esquema a partir de tus entidades mapeadas en el momento en que arranca la aplicación. Vuelve a arrancar la aplicación y busca las líneas de la capa de datos:

```
uv run pyfly run --server uvicorn
```

```
pyfly.data.relational.schema: Initializing database schema (ddl-auto=create)
pyfly.data.relational.schema: Database schema initialized (1 tables)
```

`1 tables` es tu tabla `wallets` — creada puramente porque `WalletEntity` hereda `Base`. No hay script de migración que ejecutar ni `CREATE TABLE` que escribir a mano. (Lumen usa SQLite por defecto, así que la base de datos no es más que un fichero `lumen.db` en el directorio del proyecto.)

NUEVO TÉRMINO: REPOSITORIO

Un **repositorio** es el objeto con el que habla tu código cuando quiere cargar o guardar entidades. En lugar de escribir SQL, llamas a métodos como `find_by_id` o `save`. La clase base `Repository` de PyFly genera esos métodos por ti a partir de los tipos de la entidad y de la clave, así que el repositorio que escribes está casi vacío — tú lo declaras, y el framework rellena la implementación. (CRUD, usado más abajo, no es más que el acrónimo de Create, Read, Update, Delete — las cuatro operaciones básicas de datos.)

El repositorio. `WalletRepository` hereda del `Repository` genérico del framework `Repository[WalletEntity, str]`. Los dos argumentos de tipo le dicen al framework el *tipo de entidad* (`WalletEntity`) y el *tipo de la clave primaria* (`str`); a partir de ahí genera e inyecta una superficie CRUD asíncrona completa — `find_by_id`, `save`, `find_all`, `find_all(pageable)`, `delete`, `delete_by_id`, `count` y más —, en la que cada llamada usa la sesión de la unidad de trabajo en curso:

lumen/models/repositories/wallet_repository.py

```

from __future__ import annotations

from lumen.models.entities.v1.wallet_orm import WalletEntity
from pyfly.container import repository
from pyfly.data import Page, Pageable
from pyfly.data.relational.sqlalchemy import (
    Repository,
    Specification,
)

def balance_at_least(min_minor: int) → Specification[WalletEntity]:
    """Reusable predicate: wallets with balance ≥ min_minor."""
    return Specification(
        lambda root, q: q.where(root.balance_minor ≥ min_minor)
    )

@repository
class WalletRepository(Repository[WalletEntity, str]):
    """CRUD + derived + specification queries for WalletEntity."""

    # Derived query – compiled from the name by the post-processor
    async def find_by_owner_id(
        self, owner_id: str
    ) → list[WalletEntity]:
        """All wallets owned by owner_id (derived query stub)."""
        ...

    # Specification query – composable predicate + pagination
    async def find_rich(
        self, min_minor: int, pageable: Pageable
    ) → Page[WalletEntity]:
        """Page of wallets with balance ≥ min_minor."""
        return await self.find_all_by_spec_paged(
            balance_at_least(min_minor), pageable
        )
    
```

```
# Upsert: one call for INSERT or UPDATE
async def upsert(self, entity: WalletEntity) → WalletEntity:
    """Persist entity whether the row is new or already exists."""
    session = self._require_session()
    merged = await session.merge(entity)
    await session.flush()
    return merged
```

Listado 2.2b — `WalletRepository`: subclase del `Repository` del framework

Cómo funciona. `@repository` le dice al contenedor que gestione `WalletRepository` como un bean de DI. El framework lee `Repository[WalletEntity, str]` al arrancar, genera internamente la implementación CRUD y registra la clase — inyectas `WalletRepository` directamente por tipo en cualquier parte de la aplicación. No hay interfaz de puerto escrita a mano ni adaptador aparte que mantener: **el framework suministra e inyecta la implementación; tú dependes de la propia clase del repositorio por tipo.**

Los tres métodos extra muestran los puntos de extensión que el framework expone por encima del CRUD heredado. Míralos de uno en uno:

- **Paso 1 — una consulta derivada.** `find_by_owner_id` es una **consulta derivada**: el `RepositoryBeanPostProcessor` (un componente de arranque que edita los beans después de que se construyan) analiza el *nombre* del método y compila un `SELECT ... WHERE owner_id = :owner_id` real. Tú escribes solo el cuerpo vacío (`...`); el framework suministra el SQL. La convención de nombres es la API — `find_by_<column>` se convierte en `WHERE <column> = ?`.
- **Paso 2 — una consulta con Specification.** `find_rich` compone un predicado `Specification` reutilizable (aquí, `balance_at_least`) y lo ejecuta con paginación y ordenación mediante el heredado `find_all_by_spec_paged` . Las Specifications

son la forma de construir una cláusula `WHERE` componible y con seguridad de tipos cuando un nombre de método resultaría inmanejable.

- **Paso 3 — un upsert.** `upsert` es una conveniencia ligera sobre `session.merge` para que un manejador de comandos pueda persistir una entidad tanto si es nueva (INSERT) como si ya existe (UPDATE) con una sola llamada. Como el monedero posee su propio id, ambos casos se basan en la misma clave primaria.

Qué acaba de pasar. Declaraste un repositorio cuyo cuerpo está casi enteramente vacío y, sin embargo, ahora expone una superficie CRUD asíncrona completa más una consulta derivada, una consulta de specification y un upsert. El estereotipo `@repository` lo registró como bean; el framework leyó la base `Repository[WalletEntity, str]`, generó la implementación y la hizo inyectable por tipo. Tú escribiste la intención; PyFly escribió la fontanería.

🔔 CONSEJO: CONFIRMA QUE EL REPOSITORIO ESTÁ REGISTRADO

Vuelve a ejecutar `uv run pyfly run --server uvicorn` y mira la línea `bean_summary : repositories=1`. Ese único repositorio registrado es tu `WalletRepository`. Si alguna vez añades un segundo repositorio y no aparece en este recuento, la causa habitual es que su paquete falta en `scan_packages`.

🍃 EQUIVALENCIA CON SPRING

`@service`, `@component`, `@repository` y `@configuration` mapean directamente a `@Service`, `@Component`, `@Repository` y `@Configuration` de Spring. `@rest_controller` refleja `@RestController`. `Repository[E, ID]` refleja el `JpaRepository<E, ID>` de Spring Data: declara los tipos de entidad y de clave; el framework genera e inyecta la implementación completa. Los métodos de consulta derivada (con nombres como `find_by_owner_id`) se compilan a SQL al arrancar — el mismo mecanismo que la derivación de consultas de Spring Data a partir de los nombres de los métodos.

Inyección por constructor

Con el repositorio declarado, necesitas un manejador que lo use. Ahí es donde se hace visible la capacidad más importante del contenedor: nunca llamas tú mismo a los constructores. Declaras lo que una clase *necesita* como parámetros de `__init__` con anotaciones de tipo, y el contenedor los rellena automáticamente. Esto es la **inyección por constructor**, y es el enfoque recomendado para todas las dependencias obligatorias.

NUEVO TÉRMINO: INYECCIÓN

La **inyección** es el acto del contenedor de *entregar* a una clase los objetos de los que depende, en lugar de que la clase los construya por sí misma. Con la inyección *por constructor*, simplemente listas las dependencias como parámetros tipados de `__init__`; el contenedor lee esas anotaciones de tipo y pasa los beans correspondientes cuando construye el objeto. La clase nunca dice *cómo* obtener sus dependencias — solo *qué* necesita.

El modelo mental es una simple lista de deseos: lista los tipos que necesitas; el contenedor entrega las instancias correctas. Si una dependencia no existe al arrancar, obtienes de inmediato un claro `NoSuchBeanError` — no un críptico `AttributeError` — tres marcos de pila más adentro en tiempo de ejecución.

Apilar decoradores de manejador sobre `@service`

En el diseño CQRS de Lumen, cada manejador del lado de escritura lleva dos decoradores: `@command_handler` (o `@query_handler`) **apilado sobre** `@service`. El patrón es innegociable: `@service` registra la clase como un bean; el decorador CQRS añade únicamente metadatos de enrutado (`__pyfly_command_type__` o `__pyfly_query_type__`) para que el bus de comandos/consultas pueda despachar al

manejador correcto. Sin `@service`, el contenedor nunca ve la clase y el bus de comandos lanza `CommandHandlerNotFoundException` en el momento del despacho (el bus de consultas lanza `QueryHandlerNotFoundException` cuando falta un manejador de consultas).

Antes de leer el listado, esta es la forma de lo que estás a punto de escribir, paso a paso:

- **Paso 1 — registrar el bean.** Pon `@service` directamente sobre la clase. Esta es la línea que hace que el contenedor lo gestione.
- **Paso 2 — añadir metadatos de enrutado.** Apila `@command_handler` *encima* de `@service`. Lee `CommandHandler[DepositFunds, int]` y registra "este bean maneja comandos `DepositFunds`".
- **Paso 3 — declarar dependencias en `__init__`.** Lista el repositorio, el publicador de eventos y la factoría de sesiones como parámetros tipados. Esta única firma es la especificación completa de cableado — el contenedor la lee y suministra los tres.
- **Paso 4 — escribir la lógica de negocio en `do_handle`.** Envuélvela en `@transactional()` para que toda la secuencia cargar-mutar-guardar sea una única unidad de trabajo confirmada.

El `DepositFundsHandler` muestra el patrón completo:

lumen/core/services/wallets/deposit_funds_handler.py

```

from __future__ import annotations

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

from lumen.core.mappers.wallet_mapper import to_aggregate, to_entity
from lumen.core.services.wallets.deposit_funds_command import (
    DepositFunds,
)
from lumen.core.services.wallets.event_publishing import (
    publish_domain_events,
)
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.data.relational.sqlalchemy import LockMode, transactional
from pyfly.domain import AggregateNotFound
from pyfly.eda import EventPublisher

@command_handler
@service
class DepositFundsHandler(CommandHandler[DepositFunds, int]):
    """Credit funds to an existing wallet; returns new balance."""

    def __init__(
        self,
        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
    ) → None:
        super().__init__()
        self._repository = repository
        self._events = events
        self._session_factory = session_factory

    @transactional()

```

```

async def do_handle( # type: ignore[override]
    self, command: DepositFunds
) → int:
    entity = await self._repository.find_by_id(
        command.wallet_id, lock=LockMode.PESSIMISTIC_WRITE
    )
    if entity is None:
        raise AggregateNotFound("Wallet", command.wallet_id)

    wallet = to_aggregate(entity)
    wallet.deposit(
        Money(amount=command.amount, currency=wallet.currency)
    )
    await self._repository.upsert(to_entity(wallet))

    await publish_domain_events(
        self._events, wallet.clear_events()
    )
    return wallet.balance.amount

```

Listado 2.3 — *DepositFundsHandler: @command_handler + @service apilados*

Cómo funciona. Cinco decisiones son visibles en este listado:

- `@service` registra la clase como un bean singleton. Sin él, el contenedor nunca ve la clase.
- `@command_handler` (aplicado encima de `@service`, así que se ejecuta *después* del registro) lee el primer argumento genérico de `CommandHandler[DepositFunds, int]` y registra que este bean maneja comandos `DepositFunds`.
- La firma de `__init__` es la especificación completa de cableado: `repository: WalletRepository` — el bean CRUD generado por el framework; `events: EventPublisher` — resuelto por la autoconfiguración de CQRS; `session_factory: async_sessionmaker[AsyncSession]` — la factoría de conexiones compartida

proporcionada por la autoconfiguración relacional. Los tres se resuelven por tipo;

`DepositFundsHandler` nunca importa una clase concreta.

- `@transactional()` sobre `do_handle` envuelve todo el cuerpo en una única unidad de trabajo confirmada. El decorador liga una unidad de trabajo a la tarea en curso, en el origen de datos de `session_factory`; cada llamada al repositorio del cuerpo se une a ella, y confirma en caso de éxito (o revierte en caso de error).

`lock=LockMode.PESSIMISTIC_WRITE` retiene la fila del monedero hasta entonces, así que dos cambios del mismo monedero nunca se pisan (el Capítulo 5 explica por qué).

- La lógica de negocio sigue la secuencia estándar CQRS/DDD: cargar la entidad, rehidratar el agregado mediante el mapeador, mutar a través de métodos de dominio que imponen invariantes, persistir mediante `upsert`, drenar y publicar los eventos. El monedero se guarda *antes* de que se publiquen los eventos, de modo que cualquier oyente que consulte el repositorio encuentre el registro actualizado.

Un manejador del lado de lectura usa el mismo patrón de apilado, solo que con

`@query_handler` y `QueryHandler`:

```

from pyfly.container import service
from pyfly.cqrs import QueryHandler, query_handler
from lumen.models.repositories.wallet_repository import WalletRepository

@query_handler
@service
class GetWalletHandler(QueryHandler[GetWallet, WalletDto | None]):
    def __init__(self, repository: WalletRepository) → None:
        super().__init__()
        self._repository = repository

    async def do_handle(
        self, query: GetWallet
    ) → WalletDto | None:
        entity = await self._repository.find_by_id(query.wallet_id)
        return entity_to_dto(entity) if entity is not None else None
    
```

El contenedor resuelve las dependencias de forma **recursiva**. Cuando construye `DepositFundsHandler` también construye `WalletRepository` (el bean CRUD generado por el framework), el `EventPublisher` y el `async_sessionmaker` — ninguno de los cuales el manejador necesita conocer.

Qué acaba de pasar. Declaraste un manejador que necesita tres colaboradores y no escribiste ni una línea de código de cableado. El contenedor leyó las anotaciones de tipo de `__init__`, construyó cada dependencia (y *sus* dependencias, de forma recursiva) y entregó el objeto terminado a quien pida `DepositFundsHandler`. Cambiar el bus de eventos en memoria por Kafka más adelante no tocará esta clase en absoluto — pide un `EventPublisher` y se conforma con lo que sea que el contenedor proporcione.

Ejecútalo. La forma más segura de confirmar que todo el grafo se conecta es el test de integración que arranca el contexto de aplicación *real* y conduce un depósito a través del bus de comandos. Desde la raíz del proyecto Lumen:

```
uv run --extra dev pytest tests/test_app_context_integration.py -q
```

```
.
[100%]
1 passed in 0.19s
```

Ese único punto es el contenedor demostrándose a sí mismo: escaneó los paquetes, generó el repositorio, resolvió el `EventPublisher` y la factoría de sesiones, construyó `DepositFundsHandler` con los tres inyectados y ejecutó de extremo a extremo un ciclo de vida `open → deposit → withdraw → reload`. Si faltara una dependencia, este test fallaría en el *arranque* con un `NoSuchBeanError` mucho antes de que se ejecutara cualquier aserción — que es exactamente el fallo temprano y ruidoso que el contenedor está diseñado para darte.

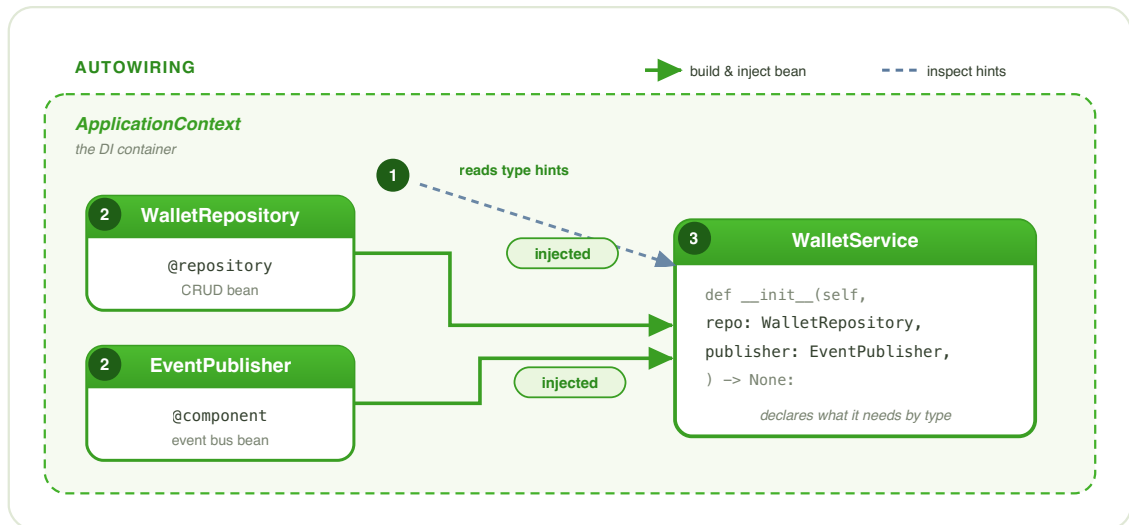


Figura 2.1 — El contenedor inyecta dependencias a partir de las anotaciones de tipo.

🍃 EQUIVALENCIA CON SPRING

La inyección por constructor en PyFly es funcionalmente idéntica a la inyección por constructor con `@Autowired` de Spring. En el Spring moderno ni siquiera escribes `@Autowired` — el framework infiere la inyección a partir del único constructor, igual que PyFly lee las anotaciones de tipo de `__init__`. El modelo mental es el mismo: declara lo que necesitas, deja que el contenedor lo proporcione.

💡 CONSEJO

Prefiere la inyección por constructor para las dependencias obligatorias. Las hace visibles en la firma de la clase, te permite escribir tests unitarios de Python puro sin contenedor (`handler = DepositFundsHandler(repo=MockRepo(), events=MockBus())`) y previene errores accidentales de dependencias faltantes en el arranque en lugar de en tiempo de ejecución.

El contenedor y el ApplicationContext

El sistema de DI de PyFly tiene dos capas, y entender la frontera entre ellas te ahorrará tiempo real de depuración. Una capa gestiona grafos de objetos; la otra gestiona el ciclo de vida completo de la aplicación. Confundirlas es una fuente habitual de confusión.

Container (de `pyfly.container`) es el motor de DI de bajo nivel. Almacena objetos **Registration**, resuelve tipos por las anotaciones del constructor, gestiona scopes, aplica la desambiguación con `@primary` y maneja las búsquedas por nombre basadas en **Qualifier**. No tiene conciencia del ciclo de vida — es una máquina pura de "dame un T".

`ApplicationContext` (de `pyfly.context`) es el orquestador de alto nivel. Envuelve a `Container` y añade toda la secuencia de arranque: filtrado de perfiles, evaluación de condiciones, procesamiento de `@configuration` / `@bean`, tejido de `BeanPostProcessor`, hooks de `@post_construct` / `@pre_destroy`, publicación de eventos y autoconfiguración. Interactúas con el `ApplicationContext` en el código de aplicación; el `Container` en crudo es un detalle de implementación, accesible mediante `ctx.container` como vía de escape.

Piénsalo así: `Container` es la planta de fabricación — sabe cómo construir cosas. `ApplicationContext` es el jefe de producción — decide qué se construye, en qué orden y qué pasa cuando la fábrica abre o cierra.

Reglas de resolución

Cuando el contenedor necesita resolver un tipo `T`, aplica cuatro reglas en estricto orden de prioridad:

1. **Registro directo** — si `T` está registrado directamente, resuélvelo.
2. **Vínculo de interfaz** — si `T` es un `Protocol` o ABC con exactamente una implementación vinculada, resuelve esa implementación.
3. **Desambiguación con `@primary`** — si hay múltiples implementaciones vinculadas, gana la decorada con `@primary`.
4. **Error** — `NoSuchBeanError` cuando nada coincide; `NoUniqueBeanError` cuando existen múltiples candidatos sin `@primary`.

El paso 4 es deliberadamente ruidoso. Una dependencia faltante o ambigua es un error de configuración, y sacarlo a la luz en el arranque en lugar de enterrarlo en una traza de ejecución es una de las garantías clave del contenedor.

@primary

`@primary` resuelve la ambigüedad cuando varios beans satisfacen la misma interfaz.

Ponlo en la implementación que quieres como predeterminada. Esto surge siempre que tienes un puerto `@runtime_checkable Protocol` con más de un adaptador registrado — un patrón habitual para infraestructura intercambiable (almacén de caché, bus de mensajes, canal de notificación).

Por ejemplo, supón que tu aplicación define un protocolo `CacheStore` e incluye dos adaptadores — uno en proceso para el desarrollo local y uno de Redis para producción:

```
from pyfly.container import repository, primary

@primary
@Repository
class InMemoryCacheStore(CacheStore):
    """Default: active in development and tests."""
    ...

@Repository
class RedisCacheStore(CacheStore):
    """Production cache — activated by profile or condition."""
    ...
```

Sin `@primary`, resolver `CacheStore` con dos implementaciones registradas lanza:

```
NoUniqueBeanError: Multiple beans of type 'CacheStore' found
but none is marked @primary
Candidates: ['InMemoryCacheStore', 'RedisCacheStore']
```

El mensaje nombra a cada candidato en competencia para que puedas tomar una decisión deliberada en lugar de adivinar cuál habría elegido el contenedor. Mover `@primary` de un adaptador al otro es el único cambio necesario para conmutar el almacén de respaldo de la aplicación — nada en el código de servicio cambia.

Ten en cuenta que el propio `WalletRepository` de Lumen es una subclase del `Repository` del framework, así que solo se registra un bean y no se necesita `@primary`. `@primary` es relevante siempre que construyas a mano un par puerto/adaptador con múltiples implementaciones de adaptador.

@order

El contenedor inicializa los beans singleton de forma anticipada durante el arranque, pero algunos beans realmente deben estar listos antes que otros — un filtro de seguridad que debe envolver cada petición entrante, o un migrador de esquema que debe ejecutarse antes de tocar cualquier repositorio. `@order` te da control explícito sobre la secuencia de inicialización.

Los valores más bajos se resuelven primero durante la pasada de arranque anticipado. Las constantes `HIGHEST_PRECEDENCE` (`-(2**31)`) y `LOWEST_PRECEDENCE` (`2**31 - 1`) marcan los extremos:

```
from pyfly.container import order, HIGHEST_PRECEDENCE, service

@order(HIGHEST_PRECEDENCE)
@service
class SecurityInitializer:
    """Must be ready before any other service."""
    ...
```

`@order` afecta a la resolución de singletons durante el arranque, a la secuencia en la que se ejecutan las instancias de `BeanPostProcessor` y a la ordenación de los resultados de `get_beans_of_type()`.

Qualifier — resolución de beans por nombre

La inyección basada en tipos cubre la mayoría de los escenarios. No obstante, de vez en cuando necesitas realmente una *instancia* concreta en lugar de cualquier implementación que satisfaga el tipo — el caso clásico es una clase `@configuration` que produce dos beans del mismo tipo (digamos, una conexión de base de datos primaria y una de réplica de lectura) donde un servicio aguas abajo debe recibir una específica.

Selecciona un bean concreto por nombre con `Annotated[T, Qualifier("name")]`:

```
from typing import Annotated
from pyfly.container import Qualifier, service

@service
class ReportService:
    def __init__(
        self,
        db: Annotated[object, Qualifier("analytics_db")],
    ) → None:
        self.db = db # receives the bean named "analytics_db"
```

El contenedor llama a `resolve_by_name("analytics_db", expected_type=T)` y verifica la asignabilidad — un nombre mal escrito que apunte al tipo equivocado lanza `NoSuchBeanError` con un mensaje claro en lugar de inyectar silenciosamente el objeto incorrecto.

Factorías de beans: @configuration y @bean

Los decoradores de estereotipo funcionan de maravilla para las clases que posees, pero no toda dependencia es una clase que controlas. Los clientes de terceros necesitan argumentos de constructor conocidos solo en tiempo de ejecución; beans relacionados comparten estado de configuración; algunas familias de beans se expresan con mayor claridad como una única factoría. Para todas estas situaciones, PyFly proporciona el patrón `@configuration` / `@bean` — código de factoría explícito que aun así participa plenamente en la maquinaria de resolución y de ciclo de vida del contenedor.

NUEVOS TÉRMINOS: @CONFIGURATION Y @BEAN

Una clase `@configuration` es un lugar donde poner **métodos factoría**. Un método `@bean` es una de esas factorías: el contenedor lo *llama* durante el arranque y registra como bean lo que sea que devuelva. Recurras a este patrón cuando una dependencia no puede simplemente estereotiparse — por ejemplo, un objeto de terceros que no posees, o uno que necesita una construcción a medida. La **anotación del tipo de retorno** del método es lo que el contenedor usa para registrar el resultado, así que es obligatoria.

Una clase `@configuration` actúa como una factoría. Sus métodos `@bean` se llaman durante la secuencia de arranque, y el valor de retorno de cada método se registra como un bean cuyo tipo proviene de la anotación de retorno del método. Lee el listado de abajo en dos pasos:

- **Paso 1 — marcar la clase `@configuration`**. Esto le dice al contexto que la escanee en busca de métodos `@bean` antes de construir cualquier bean de estereotipo.
- **Paso 2 — escribir un método `@bean` con una anotación de retorno.**
`event_publisher(self) → EventPublisher` construye un `InMemoryEventBus` y — porque la anotación dice `EventPublisher` — lo registra *como* un

`EventPublisher`. Cualquier cosa que pida un `EventPublisher` recibe ahora esta instancia.

`lumen/infra_config.py`

```
from pyfly.container import configuration, bean
from pyfly.eda import EventPublisher, InMemoryEventBus

@Configuration
class LumenInfraConfig:
    """Wires infrastructure beans that require explicit construction."""

    @bean
    def event_publisher(self) → EventPublisher:
        """In-memory event bus – replace with Kafka adapter in production."""
        return InMemoryEventBus()
```

Listado 2.4 — Producir un bean `EventPublisher` mediante `@configuration`

Cómo funciona. `@configuration` le dice al contexto que escanee `LumenInfraConfig` en busca de métodos `@bean` durante el arranque, antes de construir cualquier bean de estereotipo. La anotación de retorno `EventPublisher` es la clave: el contexto la lee y registra la instancia `InMemoryEventBus` producida como un `EventPublisher`, no como un `InMemoryEventBus`. Esa distinción importa — cuando `DepositFundsHandler` pide después un `EventPublisher`, recibe la instancia `InMemoryEventBus` sin saber ni importarle el tipo concreto.

Cambiar a un adaptador de Kafka para producción significa reemplazar `InMemoryEventBus()` por `KafkaEventPublisher(settings.kafka_url)` en un solo método. El resto del código queda intacto.

❗ NOTA: CÓMO OBTIENE LUMEN REALMENTE SU EVENTPUBLISHER

El listado de arriba muestra el patrón `@configuration` / `@bean` que escribirías para construir a mano un bean. Lumen en sí *no* necesita esto para su bus de eventos: establecer `eda.provider: memory` en `pyfly.yaml` pide a la autoconfiguración de EDA del framework que registre por ti un bean `EventPublisher` (el mismo `InMemoryEventBus` que ves en el cableado `events_is InMemoryEventBus` al arrancar). Por eso `DepositFundsHandler` puede simplemente pedir un `EventPublisher` — la autoconfiguración ya suministró uno. Recurre a `@bean` cuando necesites un bean que el framework *no* proporciona de fábrica.

Los métodos `@bean` también pueden declarar parámetros; el contenedor los resuelve automáticamente:

```
@configuration
class MessagingConfig:

    @bean
    def audited_publisher(self, base: EventPublisher) → EventPublisher:
        """Wrap the base publisher with audit logging."""
        return AuditingEventPublisher(base)
```

Parámetros de @bean

Parámetro	Por defecto	Descripción
<code>name</code>	nombre del método	Nombre del bean para la resolución por nombre.
<code>scope</code>	<code>Scope.SINGLETON</code>	Scope de ciclo de vida del bean producido.
<code>primary</code>	<code>False</code>	Marca este como el candidato primario para su interfaz.
<code>profile</code>	<code>""</code>	Crea el bean solo cuando la expresión de perfil coincide.

NOTA

La anotación del tipo de retorno en un método `@bean` es **obligatoria**. El contexto la lee para saber bajo qué tipo de interfaz registrar el bean producido. Omitirla hará que el bean sea inalcanzable por tipo.

Scopes

Cada bean tiene un **scope** que controla cuánto vive su instancia. Acertar con el scope tiene menos que ver con el rendimiento y más con la corrección: compartir un objeto con estado diseñado para un solo uso produce condiciones de carrera; recrear un singleton en cada resolución desperdicia recursos y anula la caché. El enum `Scope` define tres valores que cubren la inmensa mayoría de las necesidades del mundo real.

Scope.SINGLETON (por defecto) — se crea una instancia en la primera resolución y se reutiliza durante toda la vida de la aplicación. Los singletons se instancian de forma anticipada durante `ApplicationContext.start()`, ordenados por `@order`. Casi todos los beans de aplicación deberían ser singletons.

Scope.TRANSIENT — se crea una instancia nueva en cada resolución. Úsalo para objetos con estado, no compartibles:

`lumen/contexts.py`

```
from pyfly.container import component, Scope

@component(scope=Scope.TRANSIENT)
class TransferContext:
    """Carries state for a single wallet transfer operation."""

    def __init__(self) → None:
        self.steps: list[str] = []
        self.rolled_back: bool = False
```

Listado 2.5 — Un bean transitorio para contexto por operación

Cómo funciona. `TransferContext` acumula los pasos de una transferencia de varios saltos para que una saga pueda revertirlos en orden inverso si algo falla. Compartir una única instancia entre peticiones concurrentes mezclaría su estado; `Scope.TRANSIENT` garantiza que cada resolución produzca un `TransferContext` fresco y vacío. El contenedor sigue gestionando la clase — inyectándola, filtrándola por perfil, postprocesándola — pero nunca cachea el resultado.

Scope.REQUEST — acotado a una sola petición HTTP. Se crea una instancia nueva cuando llega una petición y se descarta cuando termina. Úsalo para beans de la capa web que llevan estado específico de la petición, como el usuario autenticado actual.

```
from pyfly.container import component, Scope
```

```
@component(scope=Scope.REQUEST)
```

```
class CurrentUser:
    user_id: str = ""
    roles: list[str] = []
```

Una regla práctica rápida:

- **SINGLETON** — el bean no tiene estado, o su estado es seguro de compartir entre todos los llamadores (pools de conexiones, cachés, objetos de servicio).
- **TRANSIENT** — el bean acumula estado por operación que no debe filtrarse entre operaciones (sagas, builders, portadores de contexto).
- **REQUEST** — el bean lleva estado por petición HTTP que debe estar aislado entre peticiones concurrentes (usuario autenticado, ID de traza acotado a la petición).

Ciclo de vida y condiciones

La construcción y el cableado son solo la mitad de la historia. Los beans de infraestructura reales necesitan *actuar* después de construirse — reservar un pool de hilos, precargar una caché, suscribirse a una cola de mensajes — y necesitan *deshacer* esas acciones limpiamente al apagarse. PyFly te da dos hooks de ciclo de vida para esto, además de una familia de decoradores condicionales que controlan si un bean participa o no en el contenedor en absoluto.

@post_construct y @pre_destroy

Una vez que el contenedor construye un bean e inyecta todas sus dependencias, a menudo necesitas una inicialización de una sola vez — abrir un pool de conexiones, calentar una caché, registrar un oyente. Marca un método con `@post_construct` y el contexto lo llamará después de que se complete la construcción. Se admiten tanto métodos síncronos como `async`:

`lumen/wallet_audit_listener.py`

```
from pyfly.container import service
from pyfly.context import post_construct, pre_destroy
import logging

logger = logging.getLogger(__name__)

@service
class WalletAuditListenerWithLifecycle:
    def __init__(self) → None:
        self._entries: list[dict] = []

    @post_construct
    async def on_start(self) → None:
        logger.info("wallet_audit_listener_ready")

    @pre_destroy
    async def on_stop(self) → None:
        logger.info("wallet_audit_listener_shutting_down")
```

Listado 2.6 — Hooks de ciclo de vida en un bean `@service`

Cómo funciona. `on_start` se dispara *después* de que el constructor retorne y todas las dependencias inyectadas estén establecidas — lo que hace seguro emitir consultas al repositorio, abrir conexiones o publicar un evento de aplicación. La palabra clave

`async` funciona sin ninguna configuración extra: el contexto llama a `await on_start()` cuando detecta una corrutina, y recurre a una llamada directa para los métodos síncronos.

`@pre_destroy` es la contraparte, llamada durante `ApplicationContext.stop()` antes de descartar el bean. Los beans se destruyen en orden **inverso** al de inicialización, de modo que un oyente arrancado después del repositorio se detiene antes que él.

Ejecútalo. Añade el bean de arriba a un paquete escaneado, luego arranca y detén la aplicación para ver cómo se disparan ambos hooks. La línea de `@post_construct` aparece durante la pasada de arranque; pulsar `Ctrl-C` dispara la línea de `@pre_destroy`:

```
uv run pyfly run --server uvicorn
```

```
... wallet_audit_listener_ready          # @post_construct, during startup
^C
... shutting_down | app=lumen
... wallet_audit_listener_shutting_down  # @pre_destroy, during shutdown
```

Ver `..._ready` antes de `application_started` confirma que el hook se ejecuta como parte de la pasada de arranque anticipado; ver `..._shutting_down` después del `Ctrl-C` confirma el desmontaje simétrico.

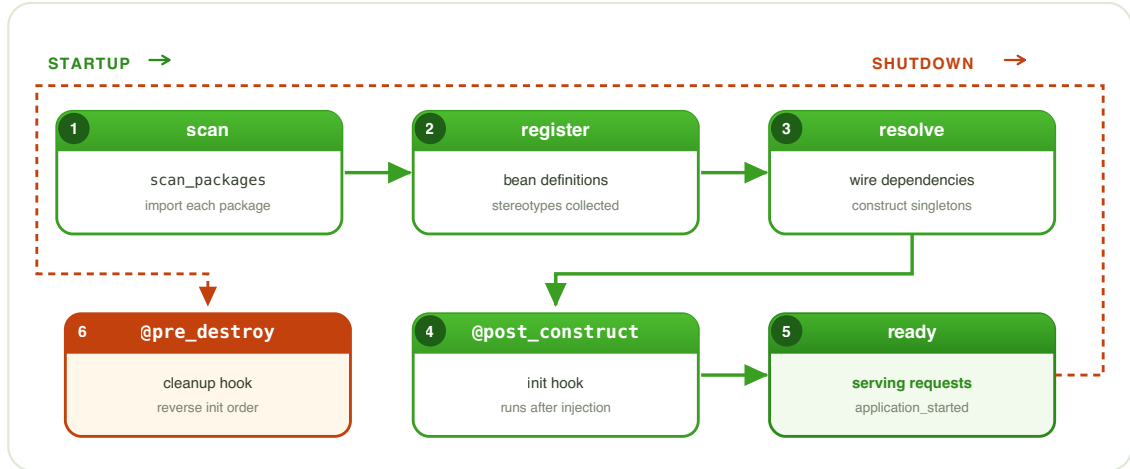


Figura 2.2 — El ciclo de vida de un bean.

Beans condicionales

Las condiciones responden a una pregunta poderosa: *¿debería existir este bean en absoluto, dado el entorno actual?* Son la forma en que la misma base de código funciona en desarrollo (adaptadores baratos en memoria), en CI (Testcontainers) y en producción (infraestructura real) — sin una sola sentencia `if` en tu código de servicio.

Los decoradores condicionales se evalúan con una estrategia de dos pasadas durante `ApplicationContext.start()`:

Pasada 1 (antes de procesar el `@configuration` del usuario) evalúa: -

`@conditional_on_property(key, having_value="...")` — la clave de configuración debe existir y, opcionalmente, coincidir con un valor. -

`@conditional_on_class("module.name")` — el módulo Python debe ser importable. -

El invocable `condition` en un decorador de estereotipo.

Pasada 2 (después de procesar el `@configuration` del usuario) evalúa: -

`@conditional_on_bean(SomeType)` — registra solo si ya existe otro bean de ese tipo. -

`@conditional_on_missing_bean(SomeType)` — registra solo si todavía no existe ningún bean de ese tipo.

El diseño de dos pasadas es deliberado. Las condiciones de la Pasada 1 dependen de hechos externos — ficheros de configuración y paquetes instalados — que se conocen antes de construir cualquier bean. Las condiciones de la Pasada 2 dependen de *qué beans se registraron*, información disponible solo después de que la Pasada 1 se estabilice. Procesarlas en orden garantiza que cada condición se evalúe contra una vista estable y predecible del contenedor.

El patrón más poderoso es "**predeterminado con anulación**" — proporciona un respaldo que cede automáticamente ante cualquier implementación proporcionada por el usuario:

lumen/notifications.py

```

from pyfly.container import service
from pyfly.context import conditional_on_missing_bean, conditional_on_property
import logging

logger = logging.getLogger(__name__)

class NotificationPort:
    async def send(self, owner_id: str, message: str) → None:
        ...

@conditional_on_property("lumen.smtp.host")
@service
class SmtplibNotificationAdapter:
    """Real email sender – only active when SMTP is configured."""

    async def send(self, owner_id: str, message: str) → None:
        logger.info("smtp_send owner=%s", owner_id)

@conditional_on_missing_bean(NotificationPort)
@service
class LoggingNotificationFallback:
    """Log-only fallback – active whenever no real sender is wired."""

    async def send(self, owner_id: str, message: str) → None:
        logger.info("notification_fallback owner=%s", owner_id)
    
```

Listado 2.7 — Predeterminado con anulación usando @conditional_on_missing_bean

Cómo funciona. Lee las dos clases como una cadena de intención.

`SmtplibNotificationAdapter` se activa solo cuando `lumen.smtp.host` está presente en la configuración, manteniendo los entornos de desarrollo libres de clientes de correo a medio configurar. `LoggingNotificationFallback` se activa siempre que no haya

registrado ningún `NotificationPort` real — en la práctica, cualquier entorno donde SMTP no esté configurado. El respaldo no comprueba *por qué* el adaptador real está ausente; simplemente llena el hueco.

Por tanto, cualquier manejador que inyecte `NotificationPort` siempre recibe *algo* — sin `NoSuchBeanError`, sin guarda de `None`. En desarrollo y en CI obtienes salida de log estructurada; en producción obtienes correo real. La elección se hace enteramente en la configuración, sin cambio de código y sin ninguna ramificación en la lógica de servicio.

💡 CONSEJO

El par `@conditional_on_missing_bean` / `@conditional_on_property` es la forma en que funciona toda la autoconfiguración propia de PyFly. Cada subsistema (caché, mensajería, cliente HTTP) incluye un bean predeterminado que se aparta automáticamente en el momento en que registras tu propia implementación.

✓ Lo que construiste

Lumen tiene ahora un `WalletEntity` mapeado a la tabla `wallets`, un `WalletRepository` que hereda del `Repository[WalletEntity, str]` del framework (lo que aporta una superficie CRUD asíncrona completa, una consulta derivada y una consulta de specification), y un `DepositFundsHandler` conectado al repositorio, al publicador de eventos y a la factoría de sesiones — todo solo mediante anotaciones de tipo. Viste por qué `@command_handler` y `@query_handler` **deben apilarse sobre** `@service` — los decoradores CQRS añaden metadatos de enrutado, pero `@service` es lo que registra el bean. Viste que el framework autogenera e inyecta la implementación de `Repository` para que dependas de la propia clase del repositorio por tipo, sin necesidad de un par puerto/adaptador escrito a mano. También viste cómo

`@primary` resuelve la ambigüedad cuando dos adaptadores compiten por un puerto construido a mano, cómo `@post_construct` / `@pre_destroy` delimitan la vida de un bean, y cómo `@conditional_on_missing_bean` habilita predeterminados que ceden automáticamente ante implementaciones reales.

El hilo conductor es consistente: tú declaras la intención con decoradores y anotaciones de tipo; PyFly proporciona las instancias. Esa separación te permite probar cada clase de forma aislada, intercambiar adaptadores sin tocar la lógica de negocio y conducir toda la configuración de la aplicación desde un fichero YAML — todo lo cual se vuelve esencial a medida que Lumen crece a lo largo del resto del libro.

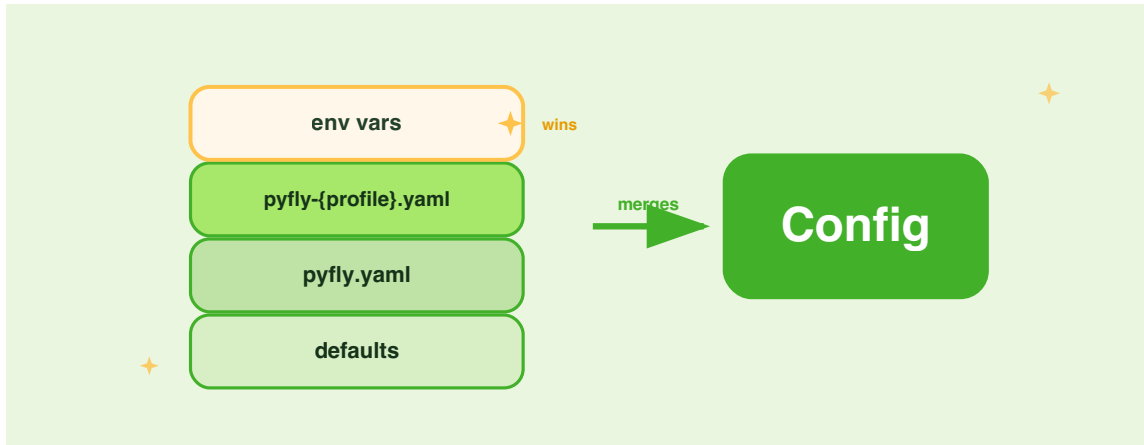
Pruébalo tú mismo

- 1. Practica `@primary` con un puerto construido a mano.** Define un protocolo `CacheStore` con un único método `async def get(key)`. Registra dos adaptadores `@repository` — uno en proceso y uno "remoto" simulado — ambos heredando `CacheStore`. Arranca la aplicación y observa el `NoUniqueBeanError`. Luego añade `@primary` al adaptador en proceso y observa cómo el arranque tiene éxito. A continuación, prueba a inyectar el simulado por nombre: anota un parámetro del constructor como `Annotated[CacheStore, Qualifier("remote_cache")]` después de registrarlo con `@repository(name="remote_cache")`.
- 2. Añade un `@post_construct` que registre metadatos de arranque.** Extiende `WalletAuditListener` con un método `async def on_ready(self)` decorado con `@post_construct`. Dentro de él, registra el nombre de clase de cualquier dependencia inyectada. Ejecuta `pyfly run --reload`, arranca el servidor y confirma que la línea de log aparece después de los propios mensajes de arranque del framework.

3. **Haz un bean condicional a una propiedad.** Añade un `WalletAuditService` decorado con `@conditional_on_property("lumen.audit.enabled", having_value="true")`. Abre `pyfly.yaml` y omite la clave. Verifica que el servicio está ausente de la lista de beans al arrancar. Luego añade `lumen.audit.enabled: "true"` a `pyfly.yaml` y vuelve a ejecutar — confirma que aparece. Así es exactamente como controlas subsistemas opcionales sin tocar el código de servicio.

CAPÍTULO 3

Configuración, perfiles y secretos



Lumen ya tiene servicios cableados: un `WalletService` respaldado por un repositorio, un publicador de eventos y un ciclo de vida de inyección de dependencias completo. El problema es que cada entorno donde se ejecuta Lumen —tu portátil, un clúster de staging compartido, un despliegue de producción reforzado— necesita ajustes diferentes: puertos distintos, URL de base de datos distintas, verbosidad de log distinta, secretos distintos. Codificar esas diferencias a fuego en el código es frágil; repartirlas entre una docena de llamadas a `os.environ` resulta ilegible e imposible de auditar.

Este capítulo muestra cómo PyFly resuelve eso con un único `pyfly.yaml`, un sistema de precedencia de cuatro capas, ficheros de superposición específicos por entorno y clases de configuración fuertemente tipadas. Al terminar, Lumen tendrá una historia de configuración limpia que escala desde `pyfly run` en tu portátil hasta un despliegue de producción en contenedores, sin tocar una línea de lógica de negocio.

pyfly.yaml: tu única fuente de ajustes

Toda aplicación no trivial tiene al menos dos audiencias para su configuración: un desarrollador que quiere logs detallados y una base de datos local relajada, y un sistema de producción que exige logs JSON estructurados, un pool de conexiones real y ningún modo de depuración. La solución ingenua — `if os.getenv("ENV") == "prod":` repartido entre una docena de ficheros — se vuelve rápidamente imposible de auditar. La respuesta de PyFly es un único fichero YAML (o TOML) canónico que contiene todo lo que tu aplicación sabe de sí misma, con mecanismos separados para lo que cambia entre entornos.

PyFly autodescubre este fichero en la raíz de tu proyecto. El framework comprueba los candidatos en orden — `pyfly.yaml`, `pyfly.toml`, `config/pyfly.yaml`, `config/pyfly.toml` — y carga el primero que encuentra.

NUEVO TÉRMINO: AUTODESCUBRIMIENTO

"Autodescubrimiento" simplemente significa que no tienes que decirle a PyFly dónde está el fichero de configuración. Dejas `pyfly.yaml` en la raíz de tu proyecto y el framework lo encuentra al arrancar. Sin argumento de ruta, sin llamada de registro.

Veamos el `pyfly.yaml` base de Lumen una sección cada vez, y luego leámoslo en su conjunto.

Paso 1 — Identifica el servicio. El primer bloque nombra la aplicación y le da una versión. Estos dos valores fluyen hacia el banner de arranque, el endpoint de salud y los metadatos de trazas, de modo que cada parte del sistema reporta la misma identidad:

```
pyfly:  
  app:  
    name: lumen  
    version: 1.0.0
```

Paso 2 — Elige el puerto de escucha. La API de negocio de PyFly escucha en `pyfly.server.port`. El valor por defecto es `8080`, así que esta línea es técnicamente redundante: se escribe por claridad. (Si has leído material más antiguo de PyFly que mencionaba `pyfly.web.port`, ten en cuenta que esa clave se eliminó en la v26.06.102; usa ahora `pyfly.server.port`.)

```
server:  
  port: 8080
```

Paso 3 — Activa las características de dominio que usa Lumen. Los bloques restantes activan la observabilidad, CQRS, el motor transaccional, event sourcing, la caché, el bus de eventos en memoria y la capa de datos relacional. Cada bloque es una característica; activas lo que necesitas y dejas el resto en los valores por defecto del framework.

Aquí está el fichero completo. Fíjate en que `pyfly:` es la única clave de nivel superior: todo lo que Lumen le dice al framework vive bajo ella:

pyfly.yaml

```

pyfly:
  app:
    name: lumen
    version: 1.0.0
  banner:
    mode: console
  server:
    # App on 8080; actuator + admin default to the management port 9090.
    port: 8080
  observability:
    metrics:
      enabled: true
    tracing:
      enabled: false
  cqrs:
    enabled: true
  transactional:
    enabled: true
    persistence:
      provider: in-memory
  event sourcing:
    enabled: true
  cache:
    provider: in-memory
    # Event-Driven Architecture: the in-memory bus (no broker needed).
    # Setting the provider registers the EventPublisher bean that the
    # wallet command handlers publish domain events through and that the
    # @event_listener audit projection auto-subscribes to.
  eda:
    provider: memory
    # Relational data layer (SQLAlchemy + SQLite via aiosqlite). The
    # framework creates the schema on startup (ddl-auto=create) and backs
    # the WalletRepository (a framework Repository[WalletEntity, str]) that
    # the command handlers persist through inside @transactional().
  data:
    relational:
      enabled: true

```

```
url: "sqlite+aiosqlite:///./lumen.db"
ddl-auto: create
```

Listado 3.1 — Fichero de configuración base de Lumen

EJECÚTALO

Desde la raíz del proyecto Lumen, arranca la aplicación y observa qué fichero carga el framework:

```
cd samples/lumen
uv sync
uv run pyfly run
```

El banner de arranque imprime la versión del framework y, a continuación, PyFly registra cada fuente de configuración que ha fusionado (una línea `loaded_config` por capa) antes de que la aplicación se enlace al puerto `8080`:

```
:: PyFly Framework :: (v26.06.110) (Python 3.12.13)
Copyright 2026 Firefly Software Foundation. | Apache License 2.0
no_active_profiles message=No active profiles set, falling back to
default
loaded_config source=pyfly-defaults.yaml (framework defaults)
loaded_config source=.../samples/lumen/pyfly.yaml
Uvicorn running on http://0.0.0.0:8080
```

Déjala en marcha; pronto la golpearás con `curl`. Pulsa `Ctrl+C` para detenerla.

Tres cosas merecen mención. Primera, la clave de nivel superior `pyfly:` está reservada exclusivamente para los ajustes del framework: servidor web, observabilidad, CQRS, EDA, acceso a datos y perfiles viven todos ahí. Tus propias claves de aplicación van bajo un nombre de nivel superior diferente (como `lumen:`). Segunda, `pyfly.app.name` y `pyfly.app.version` identifican el servicio en todas

partes: el banner de arranque, los endpoints de salud y los metadatos de trazas leen estos valores. Tercera, el bloque `pyfly.data.relational.*` configura la capa SQLAlchemy/aiosqlite; `url`, `ddl-auto` y `enabled` son sus tres claves centrales.

Las claves anidadas se corresponden directamente con el acceso por notación de puntos a través de `Config.get()`. Para leer un valor anidado, unes la ruta de la clave con puntos: `pyfly.server.port` recorre `pyfly:`, luego `server:` y luego `port:`:

```
config.get("pyfly.app.name")           # "lumen"
config.get("pyfly.server.port")        # 8080
config.get("pyfly.data.relational.url") # "sqlite+aiosqlite:///./lumen.db"
config.get("pyfly.eda.provider")       # "memory"
```

`Config.get()` usa **coincidencia relajada de segmentos**: `ddl-auto` y `ddl_auto` se resuelven a la misma clave. Tu YAML puede usar kebab-case (el estilo YAML convencional) y tu código Python puede usar snake_case; no hace falta recordar qué forma usaste en el fichero.

PyFly usa `PyYAML` (`yaml.safe_load`) para el análisis de YAML; los tipos nativos de YAML se conservan. El entero `8080` en YAML llega como un `int` de Python, sin necesidad de analizar cadenas.

LO QUE ACABA DE PASAR

Escribiste un único fichero YAML y PyFly lo convirtió en un objeto de configuración tipado y consultable. El bloque `pyfly:` le dijo al framework qué características activar (capa de datos, CQRS, bus de eventos); `Config.get("...")` vuelve a leer cualquier valor por su ruta con puntos; y la coincidencia relajada significa que nunca tienes que recordar si escribiste `ddl-auto` o `ddl_auto`. Ese único objeto es la fuente de la verdad sobre la que se construye el resto de este capítulo.

 CONSEJO

También puedes usar TOML si tu equipo prefiere una sintaxis estilo INI con tipado estricto. Renombra el fichero a `pyfly.toml` y usa la sintaxis de tablas de TOML — `[pyfly.web]`, `[pyfly.data.relational]` — en lugar del anidamiento de YAML. Cada característica descrita en este capítulo funciona de forma idéntica con ambos formatos.

Cómo se organiza la configuración por capas

Un único fichero funciona bien con un solo entorno. Los proyectos reales tienen tres o cuatro —desarrollo, test, staging, producción— y las diferencias entre ellos suelen ser pequeñas: una URL de base de datos aquí, un nivel de log allá. Duplicar el fichero entero para cada entorno es una carga de mantenimiento; la primera vez que alguien actualice el puerto en un fichero y olvide los demás, tendrás un bug de deriva de configuración.

PyFly evita esto superponiendo cuatro fuentes de configuración, cada una fusionada en profundidad sobre la anterior. Las capas posteriores siempre ganan:

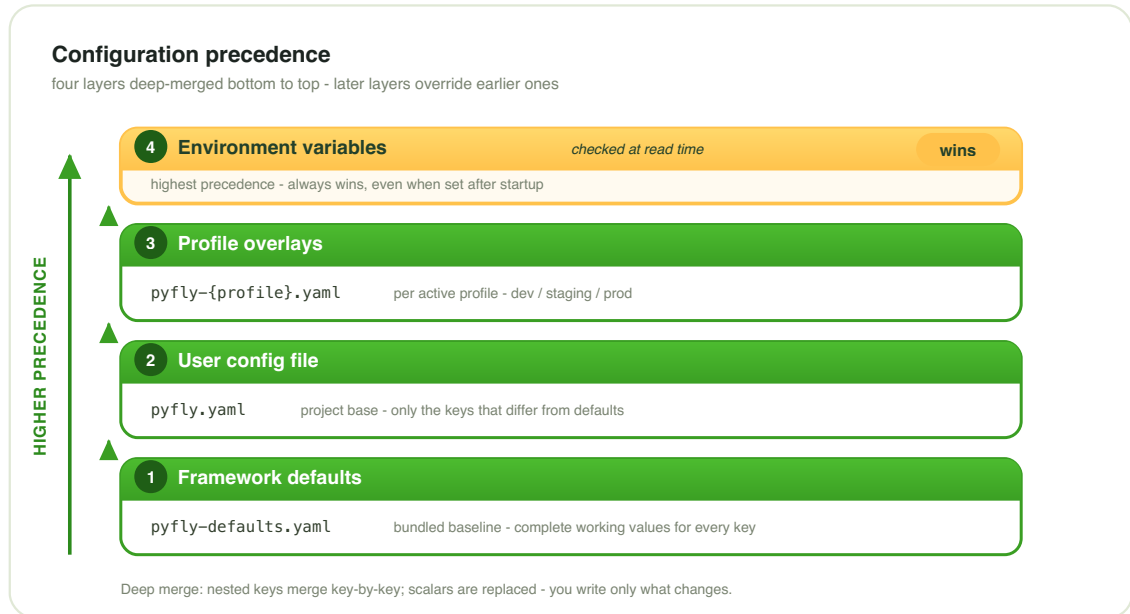


Figura 3.1 — Precedencia de configuración (las capas posteriores ganan).

Capa 1 — Valores por defecto del framework. El `pyfly-defaults.yaml` empaquetado dentro del paquete `pyfly.resources` proporciona un valor por defecto sensato para cada clave que lee el framework. Nunca editas este fichero: se carga vía `importlib.resources` y funciona correctamente en distribuciones empaquetadas. El framework siempre parte de una línea base completa y funcional.

Capa 2 — Fichero de configuración del usuario. Tu `pyfly.yaml` (o `pyfly.toml`). Incluye solo las claves que difieren de los valores por defecto del framework. En el Listado 3.1, `pyfly.server.port: 8080` coincide con el valor por defecto: se incluye por claridad, no por necesidad.

Capa 3 — Ficheros de superposición de perfil. Para cada perfil activo, PyFly busca un fichero llamado `pyfly-{profile}.yaml` junto al fichero base y lo fusiona en profundidad. Las superposiciones de perfil contienen solo las claves que cambian.

Capa 4 — Variables de entorno. Se comprueban en el **momento de lectura** en cada llamada a `Config.get()`, no se fijan al arrancar. Esto significa que una variable de entorno establecida después de que la aplicación arranque sigue ganando: el comportamiento correcto para despliegues en contenedores donde los secretos se inyectan en tiempo de ejecución. Las variables de entorno siempre anulan todo lo demás.

Fusión en profundidad, no reemplazo

Las capas se combinan mediante una fusión en profundidad recursiva (`Config._deep_merge()`). Los diccionarios anidados se fusionan clave por clave; los valores escalares se reemplazan. La distinción importa en la práctica: sin fusión en profundidad, una superposición de producción que cambiara solo `pyfly.server.port` borraría la clave `host` que está junto a ella en la misma sección `server:`, y el bloque `web.docs` no relacionado en una sección hermana. Con la fusión en profundidad, escribes solo lo que pretendes cambiar.

Para concretarlo, considera un fichero base y una superposición de producción:

```
# pyfly.yaml (base)
pyfly:
  server:
    port: 8080
    host: "0.0.0.0"
  web:
    docs:
      enabled: true
```

```
# pyfly-prod.yaml (overlay)
pyfly:
  server:
    port: 443
```

Tras la fusión, la configuración efectiva es:

```

pyfly:
  server:
    port: 443          # overridden by prod overlay
    host: "0.0.0.0"    # preserved from base
  web:
    docs:
      enabled: true    # preserved from base (sibling section untouched)

```

Solo las claves que difieren aparecen en la superposición. Todo lo demás se conserva de la capa inferior.

🌿 EQUIVALENCIA CON SPRING

Este modelo de cuatro capas se corresponde directamente con la jerarquía de configuración de Spring Boot: `application.yaml` (valores por defecto embebidos en el jar) → tu `application.yaml` → `application-{profile}.yaml` → variables de entorno. El comportamiento de fusión en profundidad, la regla de que la variable de entorno siempre gana y el paso temprano de resolución de perfiles son todas decisiones deliberadas de equivalencia.

Perfiles

El sistema de capas proporciona el mecanismo para variar la configuración entre entornos. Los perfiles proporcionan el vocabulario para nombrar esos entornos y activarlos limpiamente, sin ninguna lógica `if/else` en el código de tu aplicación.

Un **perfil** es una variante de entorno con nombre: `dev`, `test`, `staging`, `prod`. Activar un perfil carga un fichero de superposición y puede incluir o excluir beans condicionalmente.

📘 NUEVO TÉRMINO: FICHERO DE SUPERPOSICIÓN

Una "superposición" es un pequeño fichero YAML que contiene *solo las claves que cambian* para un entorno. PyFly lo fusiona sobre el `pyfly.yaml` base. Nunca repites la configuración completa: simplemente declaras las diferencias, y la fusión en profundidad de la sección anterior rellena todo lo demás.

Activar perfiles

PyFly debe saber qué perfiles están activos *antes* de cargar la configuración completa, porque necesita saber qué ficheros de superposición fusionar. Esta **resolución temprana de perfiles** sigue un orden de prioridad deliberado:

1. **Variable de entorno** `PYFLY_PROFILES_ACTIVE` — máxima prioridad; separada por comas para múltiples perfiles.
2. `pyfly.profiles.active` **en el fichero de configuración base** — alternativa cuando la variable de entorno no está establecida.
3. **Pasada programáticamente** — vía `Config.from_file("pyfly.yaml", active_profiles=["prod"])`.

En producción, anula con una variable de entorno: sin cambio de código, sin edición de fichero:

```
PYFLY_PROFILES_ACTIVE=prod uv run pyfly run
```

Ficheros de superposición de perfil

Para cada perfil activo `{name}`, PyFly busca `pyfly-{name}.yaml` junto al fichero base. Añadiremos tres superposiciones a Lumen, cada una conteniendo solo las claves que difieren de la base. Constrúyelas una a una.

Paso 1 — La superposición de desarrollo. Crea `pyfly-dev.yaml` junto a `pyfly.yaml`. Dev quiere el bucle de retroalimentación más ruidoso posible: trazas completas, cada consulta SQL volcada a la terminal y los internos del framework visibles en `DEBUG`.

`pyfly-dev.yaml`

```
pyfly:
  web:
    debug: true
  data:
    relational:
      echo: true
  logging:
    level:
      root: "DEBUG"
```

Listado 3.2 — Superposición de desarrollo: logging detallado, modo de depuración

Tres claves cubren todo lo que el entorno de desarrollo necesita: modo de depuración para trazas detalladas, eco de SQL para que cada consulta aparezca en la terminal y nivel de log `DEBUG` para que los internos del framework sean visibles. Todo lo demás llega sin cambios desde el fichero base. Fíjate en que `echo` vive bajo `pyfly.data.relational.*`, de forma coherente con la estructura del fichero base.

Paso 2 — La superposición de test. Crea `pyfly-test.yaml`. El entorno de test quiere lo opuesto a dev: silencio. Silencia el banner para que la salida de las pruebas siga siendo legible, desactiva la persistencia real (las pruebas unitarias simulan el repositorio) y eleva el umbral de log para que las pruebas que pasan no impriman nada.

`pyfly-test.yaml`

```
pyfly:
  banner:
    mode: "OFF"
  data:
    relational:
      enabled: false
  logging:
    level:
      root: "WARNING"
```

Listado 3.3 — Superposición de test: SQLite en memoria, banner silenciado

La superposición de test silencia el banner de arranque para que la salida de las pruebas se mantenga limpia, desactiva la persistencia de datos (las pruebas unitarias simulan la capa de repositorio) y eleva el umbral de log a `WARNING` para que las pruebas que pasan no produzcan ruido.

Paso 3 — La superposición de producción. Crea `pyfly-prod.yaml`. Producción cambia muchos interruptores a la vez: una URL real de PostgreSQL, logs JSON estructurados, los docs interactivos desactivados y un banner silencioso.

`pyfly-prod.yaml`

```
pyfly:
  server:
    port: 443
  web:
    debug: false
    docs:
      enabled: false
  data:
    relational:
      enabled: true
      url: "postgresql+asyncpg://prod-db:5432/lumen"
  logging:
    level:
      root: "WARNING"
    format: "json"
  banner:
    mode: "OFF"
```

Listado 3.4 — Superposición de producción: base de datos real, logging JSON, docs desactivados

La superposición de producción toma varias decisiones deliberadas. Desactiva los docs interactivos de la API: no quieres una interfaz de Swagger en vivo en un endpoint de producción. Cambia el logging a formato `json` para que agregadores como Datadog o CloudWatch puedan analizar campos estructurados en lugar de raspar texto legible por humanos. Apunta `pyfly.data.relational.url` a la instancia real de PostgreSQL. Establece `pyfly.server.port: 443`, aunque en la práctica anularás esto con `PYFLY_SERVER_PORT` desde tu pipeline de despliegue para que ningún detalle de topología entre en el repositorio.

EJECÚTALO

Con los tres ficheros de superposición en su sitio, activa el perfil dev y observa cómo ocurre la fusión. La variable de entorno `PYFLY_PROFILES_ACTIVE` le dice a PyFly qué superposiciones cargar antes de que lea el resto de la configuración:

```
PYFLY_PROFILES_ACTIVE=dev uv run pyfly run
```

El log de arranque ahora lista la superposición dev entre las fuentes fusionadas y, como dev establece `pyfly.data.relational.echo: true`, cada sentencia SQL aparece en la terminal en cuanto la aplicación toca la base de datos:

```
active_profiles profiles=['dev']
loaded_config source=pyfly-defaults.yaml (framework defaults)
loaded_config source=.../samples/lumen/pyfly.yaml
loaded_config source=.../samples/lumen/pyfly-dev.yaml (profile: dev)
INFO sqlalchemy.engine.Engine BEGIN (implicit)
```

Detén la aplicación, ejecútala de nuevo *sin* la variable de entorno y la superposición dev desaparece de la lista de fuentes: los valores por defecto más silenciosos del fichero base toman el control. Ese es todo el mecanismo de perfiles en un experimento: una variable de entorno introduce y retira una capa entera.

CONSEJO

Múltiples perfiles se separan por comas en la variable de entorno y se aplican en orden, de modo que el último perfil gana en los conflictos:

`PYFLY_PROFILES_ACTIVE=prod,metrics` aplica primero `pyfly-prod.yaml` y luego `pyfly-metrics.yaml`. Usa esto para componer aspectos transversales: un perfil `metrics` puede activar el raspado de Prometheus sin duplicar toda tu configuración de prod.

Beans con ámbito de perfil

A veces la diferencia entre entornos no es un valor sino si un componente existe siquiera. Un cargador de semillas que rellena monederos de prueba nunca debe ejecutarse en producción. Un registrador de auditoría detallado que graba cada campo de la petición es útil en desarrollo pero un riesgo de cumplimiento en prod.

El parámetro `profile` de cualquier estereotipo controla cuándo participa un bean en el contenedor. La expresión admite negación y OR separado por comas:

```
from pyfly.container import service

@service(profile="dev")
class DevSeedLoader:
    """Seeds the database with test wallets - only in dev."""
    ...

@service(profile="!prod")
class VerboseAuditLogger:
    """Detailed audit logging - active everywhere except prod."""
    ...
```

NUEVO TÉRMINO: BEAN

Un "bean" es cualquier objeto que el contenedor de inyección de dependencias crea y gestiona por ti: un `@service`, `@repository`, `@command_handler`, etc. (el término viene directamente de Spring). "Con ámbito de perfil" significa que el contenedor solo crea el bean cuando un perfil que coincide está activo.

`Environment.accepts_profiles()` evalúa las expresiones de perfil durante la primera pasada de `ApplicationContext.start()`. Los beans cuya expresión no coincide con el conjunto activo se eliminan antes de que tenga lugar cualquier resolución: nunca se

instancian, nunca se cablean, nunca están presentes en el contenedor. El resultado es un contenedor estructuralmente diferente por entorno, sin ninguna sentencia `if` en el código de tu aplicación.

LO QUE ACABA DE PASAR

Los perfiles te dieron dos herramientas independientes. Los *ficheros de superposición* (`pyfly-{name}.yaml`) cambian los *valores* de configuración por entorno. El parámetro `profile=` de un estereotipo cambia qué *beans existen* por entorno. Ambos están gobernados por el mismo conjunto de perfiles activos —establecido una sola vez vía `PYFLY_PROFILES_ACTIVE` —, de modo que una única variable de entorno remodela tanto tus ajustes como tu grafo de objetos, con cero lógica condicional en tu código de negocio.

Ajustes con tipos seguros mediante `@config_properties`

Las búsquedas por clave de cadena como

`config.get("pyfly.data.relational.url")` funcionan para lecturas ocasionales, pero no escalan. Cada llamada es una lectura aislada sin información de tipos: debes acordarte de llamar a `float()` sobre el resultado, y una errata en una clave aflora en la primera petición en producción, no al arrancar. Para cualquier cosa más allá de un puñado de valores dispersos, el enfoque correcto es agrupar los ajustes relacionados en una clase Python tipada que se rellena una vez al arrancar y se inyecta donde haga falta.

`@config_properties` resuelve exactamente esto enlazando una sección de configuración a una dataclass de Python tipada.

📌 NUEVO TÉRMINO: ENLACE (BINDING)

"Enlazar" (binding) significa copiar valores del árbol de configuración a los campos de un objeto tipado. Una `@dataclass` de Python es una clase cuyos campos se declaran con anotaciones de tipo (`url: str` , `pool_size: int`); tras el enlace, lees `props.pool_size` y obtienes un `int` de verdad, no una cadena que tienes que convertir tú mismo.

Declarar una clase de propiedades

Decora una `@dataclass` con `@config_properties(prefix="...")`. El `prefix` identifica la sección de configuración que se va a enlazar; los nombres de los campos deben coincidir con las claves bajo esa sección (kebab/snake intercambiables).

Paso 1 — Escribe la clase. Aquí está la propia `RelationalProperties` del framework, que enlaza el bloque `pyfly.data.relational.*`:

pyfly/config/properties/data.py

```

from dataclasses import dataclass, field

from pyfly.core.config import config_properties

@config_properties(prefix="pyfly.data.relational")
@dataclass
class RelationalProperties:
    """Typed binding for pyfly.data.relational.* (abridged)."""

    enabled: bool = False
    url: str | None = None
    echo: EchoSetting = False # True, False or "debug"
    ddl_auto: str | None = None
    pool: PoolProperties = field(default_factory=PoolProperties)
    # ... sqlite, read_replica, datasources, migrations, schema

```

Listado 3.5 — RelationalProperties: ajustes tipados para la capa de datos

El decorador establece `__pyfly_config_prefix__` en la clase y la marca como un bean inyectable. Los tipos de los campos deben ser `int`, `float`, `bool` o `str` para la coerción automática; los tipos más complejos se dejan tal cual.

Fíjate en que cada campo lleva un valor por defecto. Esto es intencionado: la clase es autodocumentada y puede construirse y usarse en pruebas unitarias sin ningún fichero YAML en disco; basta con instanciar `RelationalProperties()`. Dos valores por defecto están vacíos a propósito. `url` vale `None` porque una aplicación relacional sin `pyfly.data.relational.url` falla al arrancar (solo el perfil `dev` recurre a un fichero SQLite local, con un aviso), y `ddl_auto` sin fijar se resuelve como `create` para una base de datos embebida como SQLite y como `none` para un servidor de base de datos. Los ajustes anidados, como `pool.size` y `pool.recycle`, son dataclasses propias.

Paso 2 — Aplica el patrón a tus propios ajustes. El mismo decorador funciona para la configuración a nivel de aplicación. Así sería una clase `WalletProperties` para las reglas de negocio de Lumen:

```
from dataclasses import dataclass
from pyfly.core import config_properties

@config_properties(prefix="lumen.wallet")
@dataclass
class WalletProperties:
    daily_transfer_limit: float = 10_000.0
    default_currency: str = "USD"
```

Paso 3 — Añade el YAML correspondiente. Pon el bloque bajo la clave de nivel superior `lumen:` (fuera de `pyfly:`, ya que este es el espacio de nombres propio de tu aplicación, no el del framework) y el framework lo enlaza automáticamente, sin necesidad de ningún registro especial:

```
lumen:
  wallet:
    daily-transfer-limit: 10000.0
    default-currency: USD
```

Enlazar e inyectar

Llama a `config.bind(PropertiesClass)` para producir una instancia tipada y rellena. `Config` está registrado como un bean singleton, así que puedes inyectarlo en cualquier servicio y enlazar desde ahí:

lumen/wallet_service.py

```

from pyfly.container import service
from pyfly.core import Config
from pyfly.config.properties import RelationalProperties
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.eda import EventPublisher

@service
class WalletService:
    def __init__(
        self,
        repo: WalletRepository,
        events: EventPublisher,
        config: Config,
    ) → None:
        self.repo = repo
        self.events = events
        self.db: RelationalProperties = config.bind(
            RelationalProperties
        )

    async def transfer(
        self, from_id: str, to_id: str, amount: float
    ) → dict:
        if not self.db.enabled:
            raise RuntimeError("Relational layer not enabled")
        # ... perform transfer using self.db.url for diagnostics ...
        return {"from": from_id, "to": to_id, "amount": amount}

```

Listado 3.6 — Inyectar *RelationalProperties* vía *config.bind()*

Cuando el contenedor de inyección de dependencias arranca Lumen, `WalletService.__init__` recibe el singleton `Config` compartido e inmediatamente llama a `config.bind(RelationalProperties)`. Esa llamada resuelve los valores enlazados una sola vez, al arrancar, y los almacena en `self.db`: una dataclass de

Python sencilla con tipos reales. A partir de ese momento, `transfer()` lee `self.db.enabled` como un `bool`, con autocompletado completo del IDE y sin código de análisis en ninguna parte.

`config.bind()` funciona en cinco pasos:

1. Lee `__pyfly_config_prefix__` de la clase.
2. Llama a `effective_section(prefix)` — una copia resuelta del subárbol con los marcadores `${...}` expandidos, las anulaciones de variables de entorno aplicadas y las claves solo de entorno inyectadas.
3. Hace coincidir las claves de la sección con los campos de la dataclass mediante búsqueda relajada (kebab/snake intercambiables).
4. Aplica coerción de tipos a los campos cuyos valores llegaron como cadenas (por ejemplo, desde variables de entorno).
5. Construye la dataclass con los kwargs recopilados; los campos ausentes de la configuración usan los valores por defecto de la dataclass.

El detalle crítico en el paso 2 es que `effective_section()` aplica la pila completa de cuatro capas —valores por defecto, fichero, superposición de perfil, variables de entorno— antes de que se construya la dataclass. Para cuando `bind()` termina, `RelationalProperties` refleja lo que sea que diga la superposición de producción o una variable de entorno en tiempo de ejecución, no solo el YAML base.

i LO QUE ACABA DE PASAR

Reemplazaste las búsquedas dispersas de cadena `config.get("...")` por un único objeto tipado. `config.bind(RelationalProperties)` lee toda la sección `pyfly.data.relational.*` una sola vez, aplica la precedencia de cuatro capas, fuerza las cadenas a los tipos correctos y te devuelve una dataclass sencilla. A partir de entonces tu servicio lee `self.db.enabled` como un `bool` con autocompletado del IDE, y una errata en el nombre de un campo se detecta al arrancar, no en el momento de la petición en producción.

i EJECÚTALO

Puedes demostrar que la capa de variables de entorno alcanza una propiedad enlazada con una pequeña comprobación. El `pyfly.yaml` base establece `pyfly.data.relational.enabled: true`; aquí lo anulamos desde el entorno. Desde la raíz del proyecto Lumen:

```
PYFLY_DATA_RELATIONAL_ENABLED=false uv run python -c "
from pyfly.core import Config
from pyfly.config.properties import RelationalProperties
db = Config.from_file('pyfly.yaml').bind(RelationalProperties)
print('enabled =', db.enabled, type(db.enabled).__name__)
"
```

Salida esperada: la variable de entorno gana sobre el YAML y la cadena `"false"` se fuerza a un `bool`:

```
enabled = False bool
```

Inyectar valores individuales con Value

Para ajustes aislados que no justifican una clase de propiedades completa, PyFly proporciona un descriptor `Value`. Decláralo como un campo a nivel de clase y el contenedor de inyección de dependencias lo resuelve en el momento de creación del bean, exactamente como el `@Value("${...}")` de Spring Boot:

```
from pyfly.container import service
from pyfly.core import Value

@service
class WalletService:
    # Resolved from pyfly.app.name in the merged config.
    app_name: str = Value("${pyfly.app.name}")
    # Falls back to 10000 when the key is absent.
    transfer_limit: float = Value(
        "${lumen.wallet.daily-transfer-limit:10000}"
    )
```

`Value("${key}")` lanza `KeyError` al arrancar cuando la clave falta y no se proporciona un valor por defecto: una garantía de fallo rápido que mantiene los bugs de configuración faltante fuera de producción. `Value("${key:default}")` usa el valor por defecto delimitado por dos puntos cuando la clave está ausente.

Coerción de tipos

Los tipos nativos de YAML llegan correctamente tipados: los enteros, booleanos y flotantes no necesitan coerción. Cuando un valor llega de una variable de entorno (siempre una cadena) y el campo de destino tiene un tipo no de cadena, `bind()` aplica la coerción automáticamente:

Tipo de destino	Regla de coerción
<code>int</code>	<code>int(value)</code>
<code>float</code>	<code>float(value)</code>
<code>bool</code>	<code>value.lower() in ("true", "1", "yes")</code>
<code>str</code>	no se necesita coerción

La ruta de dataclass de `bind()` trata `"true"`, `"1"` y `"yes"` como `True`; la ruta de anulación de `get()` /variable de entorno en tiempo de lectura admite además `"on"`.

Llamar a `bind()` sobre una clase no decorada con `@config_properties` lanza `ValueError` inmediatamente: una señal clara de fallo rápido al arrancar en lugar de un bug silencioso de valor incorrecto en el momento de la petición.

🍃 EQUIVALENCIA CON SPRING

`@config_properties` es la respuesta de PyFly al `@ConfigurationProperties` de Spring Boot. El modelo mental es idéntico: anota un POJO (aquí, una dataclass) con un prefijo y el framework le enlaza la sección de configuración correspondiente con coerción de tipos completa. `Value("${...}")` se corresponde con el `@Value("${...}")` de Spring: misma sintaxis de expresión, misma garantía de fallo rápido ante valores faltantes. La combinación de `pyfly.yaml` + superposiciones de perfil + `@config_properties` + `Value` se corresponde con `application.yaml` + `application-{profile}.yaml` + `@ConfigurationProperties` + `@Value`: los mismos conceptos, con modismos pythónicos.

Variables de entorno y secretos

Los ficheros son el hogar adecuado para la configuración que varía por entorno pero que es seguro versionar: puertos, niveles de log, nombres de host de bases de datos. Son el hogar equivocado para los secretos: las contraseñas, las claves de API, los tokens de firma y las credenciales de bases de datos nunca deben entrar en el control de versiones. La cuarta capa de la pila de configuración existe específicamente para recibir estos valores en el momento del despliegue, desde un gestor de secretos o un pipeline de CI/CD, sin que ninguno de ellos toque el sistema de ficheros.

Las variables de entorno son la cuarta capa y la de mayor prioridad. PyFly las comprueba en cada llamada a `Config.get()` —en el momento de lectura, no al arrancar—, de modo que siempre ganan, incluso cuando se establecen después de que el proceso empiece.

Convención de nombres

Cada clave de configuración con notación de puntos se corresponde con una variable de entorno con prefijo `PYFLY_` mediante una transformación mecánica de tres pasos:

1. Elimina el prefijo `pyfly.` (si está presente).
2. Reemplaza los puntos (`.`) y guiones (`-`) por guiones bajos (`_`).
3. Pon el resultado en mayúsculas y antepón `PYFLY_`.

Clave de configuración	Variable de entorno
<code>pyfly.app.name</code>	<code>PYFLY_APP_NAME</code>
<code>pyfly.server.port</code>	<code>PYFLY_SERVER_PORT</code>
<code>pyfly.management.server.port</code>	<code>PYFLY_MANAGEMENT_SERVER_PORT</code>
<code>pyfly.web.debug</code>	<code>PYFLY_WEB_DEBUG</code>
<code>pyfly.data.relational.url</code>	<code>PYFLY_DATA_RELATIONAL_URL</code>
<code>pyfly.data.relational.pool.size</code>	<code>PYFLY_DATA_RELATIONAL_POOL_SIZE</code>
<code>pyfly.logging.level.root</code>	<code>PYFLY_LOGGING_LEVEL_ROOT</code>
<code>pyfly.eda.provider</code>	<code>PYFLY_EDA_PROVIDER</code>
<code>pyfly.profiles.active</code>	<code>PYFLY_PROFILES_ACTIVE</code>

Para las claves específicas de la aplicación que no empiezan por `pyfly.`, la ruta completa con puntos se transforma de la misma manera (sin eliminar prefijo):

```
lumen.wallet.daily-transfer-limit
→ PYFLY_LUMEN_WALLET_DAILY_TRANSFER_LIMIT
```

La regla es consistente: cuando necesitas decirle a un operador de Kubernetes qué variable de entorno controla un ajuste dado, la respuesta es siempre "aplica la transformación de tres pasos" en lugar de rebuscar en el código fuente del framework.

Las variables de entorno siempre ganan

Activar el perfil de producción y anular la URL de la base de datos para una instancia de contenedor concreta es un único comando:

```
PYFLY_PROFILES_ACTIVE=prod \
  PYFLY_DATA_RELATIONAL_URL="postgresql+asyncpg://rds-prod:5432/lumen" \
  PYFLY_SERVER_PORT=8080 \
  uv run pyfly run
```

Aquí, `PYFLY_SERVER_PORT=8080` anula el `port: 443` de la superposición de prod. La pila de precedencia se resuelve así:

1. Valores por defecto del framework → `port: 8080`
2. Configuración base → `port: 8080` (sin cambios)
3. Superposición de prod → `port: 443`
4. Variable de entorno → `port: 8080` (gana)

El puerto efectivo es `8080`. Este es un patrón útil durante una migración por fases: mantén `port: 443` en la superposición como el valor por defecto de producción previsto y luego usa una variable de entorno temporal para mantener el servicio en `8080` para un experimento de división de tráfico. Cuando el experimento termina, elimina la variable de entorno y la superposición toma el control: sin necesidad de editar ficheros.

Mantener los secretos fuera de los ficheros

Los ficheros de configuración nunca deben contener credenciales, claves de API o secretos de firma. El `pyfly-defaults.yaml` se distribuye con un secreto JWT de marcador de posición (`"change-me-in-production"`) que existe solo para mantener el framework ejecutable nada más sacarlo de la caja. Reemplázalo antes de pasar a producción:

```
PYFLY_SECURITY_JWT_SECRET="$(vault kv get -field=jwt_secret secret/lumen)"
```

⚠ NUNCA VERSIONES SECRETOS

No pongas contraseñas, claves de API, credenciales de bases de datos ni secretos JWT en `pyfly.yaml`, `pyfly-prod.yaml` ni en ningún fichero que entre en el control de versiones. Usa variables de entorno provenientes de un gestor de secretos (HashiCorp Vault, AWS Secrets Manager, Kubernetes Secrets o similar). La capa de variables de entorno existe precisamente para recibir estos valores en el momento del despliegue, no en el momento del desarrollo.

Una nota sobre las claves solo de entorno

`Config.bind()` también gestiona los valores que existen *solo* como variables de entorno, sin entrada correspondiente en ningún fichero YAML. `effective_section()` inyecta estas claves solo de entorno en la sección enlazada para que `bind()` vea el mismo valor que vería `get()`. Añade un nuevo campo a una clase

`@config_properties`, establécelo exclusivamente vía una variable de entorno en tu pipeline de despliegue, y se rellena correctamente incluso cuando los ficheros YAML aún no se han actualizado:

```
# No YAML entry for pyfly.data.relational.echo?
# Set it exclusively via env var – bind() still picks it up.
PYFLY_DATA_RELATIONAL_ECHO=true uv run pyfly run
```

Esta es una escotilla de escape práctica durante despliegues incrementales: el equipo que despliega puede inyectar un nuevo valor antes de que el fichero YAML se actualice y se revise, y la aplicación lo toma sin un cambio de código.

⚠ NOMBRES DE CAMPO CON VARIAS PALABRAS E INYECCIÓN SOLO DE ENTORNO

La inyección solo de entorno trata cada guion bajo en un nombre `PYFLY_*` como un separador de ruta, de modo que `PYFLY_DATA_RELATIONAL_POOL_MAX_OVERFLOW` se lee como la ruta anidada `pool` → `max` → `overflow`, no como el campo `pool.max_overflow`. Para segmentos de una sola palabra como `echo`, o `pool.size` (exactamente `pool` → `size`), esto es inequívoco y `bind()` los inyecta limpiamente. Para un campo de varias palabras como `max_overflow`, dale a la clave un hogar real en tu YAML (aunque sea solo `max-overflow: 10` bajo `pool:`) para que la variable de entorno anule una hoja existente en lugar de depender de la inyección solo de entorno. La lectura en tiempo de lectura `config.get("pyfly.data.relational.pool.max-overflow")` siempre devuelve el valor de entorno de todos modos, porque `get()` mapea la clave completa con puntos en un solo paso.

✓ Lo que construiste

Lumen ahora tiene una historia de configuración limpia a través de tres entornos. Un `pyfly.yaml` contiene la línea base compartida — `pyfly.app.name`, `pyfly.eda.provider`, `pyfly.data.relational.*` y cualquier otra perilla del framework que use Lumen—. `pyfly-dev.yaml`, `pyfly-test.yaml` y `pyfly-prod.yaml` contienen solo los deltas por entorno. Activar un perfil requiere una única variable de entorno (`PYFLY_PROFILES_ACTIVE=prod`). Los ajustes tipados viven en dataclasses `@config_properties`, enlazadas al arrancar con coerción de tipos completa, de modo que los servicios leen campos tipados en lugar de llamar a `float(os.environ.get(...))` en código de servicio disperso. Los valores individuales se inyectan limpiamente vía `Value("${key}")`, que falla rápido al arrancar cuando la clave falta. Los secretos se quedan en las variables de entorno, nunca en los ficheros.

La pila de cuatro capas —valores por defecto → fichero → superposición de perfil → variables de entorno— te da un único modelo mental que funciona desde `pyfly run` en tu portátil hasta un contenedor blindado con secretos inyectados en el momento del despliegue, sin tocar una línea de lógica de negocio.

Puertos de aplicación y de gestión

PyFly separa el puerto de **aplicación** del puerto de **gestión**, reflejando los `server.port` / `management.server.port` de Spring Boot. Nada más sacarlo de la caja, la API de negocio escucha en `pyfly.server.port` (**8080**) mientras que los endpoints del actuador (`/actuator/*`) y el panel de administración (`/admin`) se sirven en un `pyfly.management.server.port` dedicado (**9090**). Esto mantiene las comprobaciones de salud, el raspado de Prometheus y la consola de administración fuera del puerto público: expones solo el 8080 a internet y accedes al 9090 desde dentro del clúster.

Clave	Variable de entorno	Por defecto	Propósito
<code>pyfly.server.port</code>	<code>PYFLY_SERVER_PORT</code>	<code>8080</code>	Puerto HTTP de la aplicación
<code>pyfly.server.host</code>	<code>PYFLY_SERVER_HOST</code>	<code>0.0.0.0</code>	Dirección de enlace de la aplicación
<code>pyfly.management.server.port</code>	<code>PYFLY_MANAGEMENT_SERVER_PORT</code>	<code>9090</code>	Puerto de gestión (actuador + admin)
<code>pyfly.management.server.address</code>	<code>PYFLY_MANAGEMENT_SERVER_ADDRESS</code>	host de la app	Dirección de enlace de gestión

El puerto de gestión es un segundo escuchador **en proceso** —no procesos de worker adicionales—, que comparte el mismo bucle de eventos y los mismos beans, así que funciona con cualquier adaptador de servidor (Granian, Uvicorn, Hypercorn). Dos valores cambian la topología: establece `pyfly.management.server.port` **igual** al puerto de la app para servir todo en un único puerto (el comportamiento previo a la `v26.06.102`), o establéclo en `-1` para desactivar por completo los endpoints web de gestión.

⚠ EL PUERTO DE GESTIÓN ESTÁ ABIERTO POR DEFECTO

Por equivalencia con Spring Boot, el puerto de gestión (9090) está **abierto y sin autenticación** por defecto: está pensado para vivir en una red interna protegida por aislamiento de red, no por los filtros de login de la app. Antes de exponerlo a cualquier lugar alcanzable, asegúralo con `pyfly.management.security.enabled: true`, que aplica los filtros de seguridad, sesión y CSRF de la app también al puerto de gestión. Por defecto, solo los endpoints del actuador `health` e `info` se exponen por HTTP; amplíalos con `pyfly.management.endpoints.web.exposure.include` (por ejemplo `"health,info,metrics,prometheus"`).

i EJECÚTALO

Arranca Lumen y golpea el puerto de gestión directamente. El endpoint de salud vive en el 9090, no en el 8080 de la aplicación:

```
uv run pyfly run          # in one terminal
curl http://localhost:9090/actuator/health # in another
```

Salida esperada: un pequeño documento JSON que informa de que el servicio está activo:

```
{"status": "UP"}
```

La misma ruta en el puerto de la app (`curl http://localhost:8080/actuator/health`) no responderá, porque el actuador escucha solo en el puerto de gestión.

🍃 EQUIVALENCIA CON SPRING

`pyfly.server.port` $\equiv$ `server.port` de Spring, `pyfly.server.host` $\equiv$ `server.address`, y `pyfly.management.server.port` $\equiv$ `management.server.port`. Establecer un puerto de gestión distinto ejecuta el actuador en su propio conector, exactamente como hace Spring Boot. `pyfly.management.security.enabled` y `pyfly.management.endpoints.web.exposure.include` reflejan los ajustes de seguridad de gestión de Spring y `management.endpoints.web.exposure.include`.

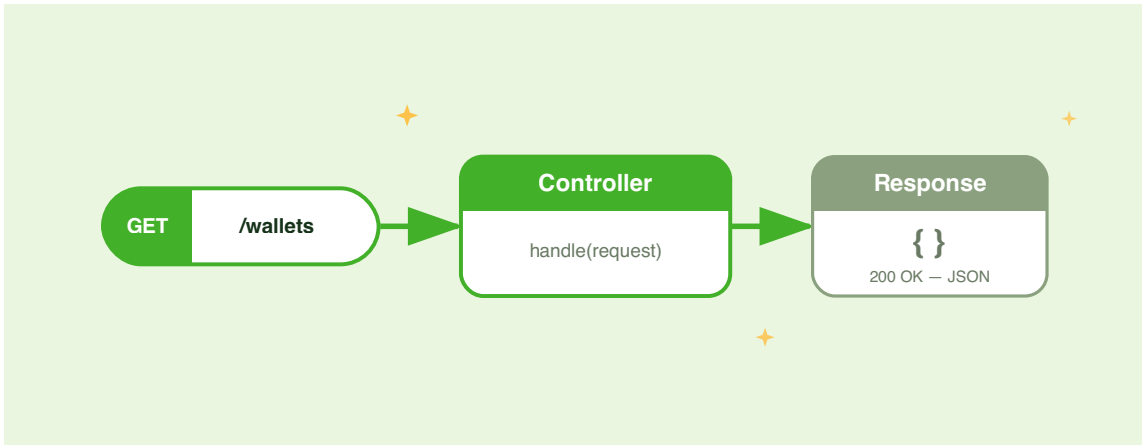
🍃 Pruébalo tú mismo

- Añade una superposición de staging.** Crea `pyfly-staging.yaml` con una URL de PostgreSQL para una base de datos de pruebas compartida bajo `pyfly.data.relational.url`, `pyfly.data.relational.enabled: true` y logging en `INFO`. Actívala con `PYFLY_PROFILES_ACTIVE=staging uv run pyfly run` y verifica desde el log de arranque que la fuente de staging se cargó. Compara la configuración efectiva con la que produciría la superposición de prod.
- Enlaza una nueva propiedad tipada y úsala.** Añade un campo `max_wallets_per_owner: int = 5` a una nueva clase `WalletProperties` decorada con `@config_properties(prefix="lumen.wallet")`, y una clave correspondiente `lumen.wallet.max-wallets-per-owner: 5` en `pyfly.yaml` (fuera del bloque `pyfly:`). Inyecta `Config` en `WalletService`, llama a `config.bind(WalletProperties)` y añade una guarda en `open_wallet` que lance `ValueError` cuando el propietario ya posee el número máximo de monederos. Escribe una prueba rápida que anule el límite a `1` estableciendo `PYFLY_LUMEN_WALLET_MAX_WALLETS_PER_OWNER=1` y verificando que el error se dispare en el segundo monedero.

- 3. Anula un valor vía una variable de entorno y observa la precedencia.** Establece `PYFLY_SERVER_PORT=9090` antes de arrancar Lumen. Comprueba el log de arranque y confirma que el servidor se enlaza al `9090`, no al `8080` de `pyfly.yaml`. Luego desestablece la variable de entorno y reinicia: el puerto debería revertir a `8080`. Este ejercicio hace concreta la naturaleza en tiempo de lectura de la resolución de variables de entorno: la variable de entorno siempre gana, y eliminarla restaura inmediatamente el valor del fichero sin ningún cambio de código.

CAPÍTULO 4

Tu primera API HTTP



Lumen tiene los servicios cableados, una historia de configuración limpia y un ciclo de vida que abarca desde el desarrollo hasta la producción. Lo único que falta es una forma de que el mundo exterior hable con él. Este capítulo cierra la Parte I convirtiendo el dominio del monedero en una API REST limpia y validada: una con documentación OpenAPI automática, respuestas de error estructuradas en las que los clientes pueden confiar y las convenciones gestionadas por el framework que ya esperas del resto de PyFly.

📘 APLICACIONES HTML Y ADMINISTRACIÓN DE MODELOS

El Apéndice E desarrolla el catálogo ejecutable: `ModelAndView`, `Form[T]`, `reverse`, `static_url`, errores HTML y CRUD de entidades/documentos existentes. La administración de datos siempre exige autenticación; en un puerto separado también exige `pyfly.management.security.enabled: true`.

Controladores y mapeos de rutas

Todo framework web debe responder a dos preguntas: ¿cómo encuentra una petición el manejador correcto, y cómo obtiene ese manejador las dependencias que necesita? Los frameworks que responden a estas preguntas con mecanismos separados te obligan a mantener un fichero de router, un fichero de pegamento de DI y un andamiaje de documentación en tres sitios distintos. PyFly colapsa las tres preocupaciones en una sola clase.

Un **controlador** en PyFly es una clase Python corriente que el contenedor de DI gestiona y a la que la capa web enruta las peticiones. Dos decoradores lo marcan:

`@rest_controller`, de `pyfly.container`, lo registra como bean y fija su estereotipo; `@request_mapping`, de `pyfly.web`, fija el prefijo de URL que hereda cada manejador de la clase.

Algunos términos de ese párrafo reaparecerán a lo largo del capítulo, así que fijémoslos de una vez. Un **bean** es simplemente un objeto que el contenedor de DI crea y te entrega: tú nunca llamas a `WalletController()`; el framework lo construye y conserva una única instancia compartida. Un **estereotipo** es una etiqueta que el framework estampa en una clase para saber *qué tipo* de bean es: `@rest_controller` estampa «esto es un controlador web», que es la señal que la maquinaria de arranque utiliza para ir a buscar rutas dentro de él. Un **manejador** (handler) es un método `async def` del controlador que responde a un tipo de petición. Con esas tres palabras en la mano, el resto del capítulo se lee como prosa llana.

Los manejadores de ruta son simples métodos `async def`, cada uno decorado con `@get_mapping`, `@post_mapping`, `@put_mapping`, `@patch_mapping` o `@delete_mapping`. Cada decorador de mapeo acepta una ruta relativa opcional y un `status_code` opcional. La URL completa es la ruta base de `@request_mapping` concatenada con la ruta relativa del decorador del método.

Un ejemplo de lectura previo a CQRS

El Capítulo 7 introduce el bus completo de comandos/consultas CQRS que Lumen usa en producción. Aquí, en la Parte I, la mecánica de la capa web se enseña sobre el mismo dominio del monedero, respaldado por un almacén en memoria simple en lugar del bus. La estructura del controlador, los imports y las formas de los decoradores son *idénticos* a los del Capítulo 7; solo cambia el destino del despacho.

lumen/web/controllers/wallet_controller.py

```

from __future__ import annotations

from pyfly.container import rest_controller
from pyfly.kernel import ResourceNotFoundException
from pyfly.web import (
    Body,
    PathVar,
    QueryParam,
    Valid,
    get_mapping,
    post_mapping,
    request_mapping,
)

from lumen.interfaces.dtos.v1.balance_dto import BalanceDto
from lumen.interfaces.dtos.v1.deposit_request import DepositRequest
from lumen.interfaces.dtos.v1.open_wallet_request import OpenWalletRequest
from lumen.interfaces.dtos.v1.wallet_dto import WalletDto

# -----
# In-memory store (replaced by a database repository in Chapter 5)
# -----
_wallets: dict[str, WalletDto] = {}

@rest_controller
@request_mapping("/api/v1/wallets")
class WalletController:
    """Digital-wallet REST API: open, deposit, inspect.

    In Part I the controller holds a minimal in-memory store so you can
    focus on the web-layer mechanics – decorators, binding, validation, and
    error handling – without persistence or CQRS machinery. Chapter 7
    replaces the store with DefaultCommandBus / DefaultQueryBus dispatching.
    """

```

```

@post_mapping("", status_code=201)
async def open_wallet(
    self, request: Valid[Body[OpenWalletRequest]]
) → dict[str, str]:
    import uuid
    wallet_id = str(uuid.uuid4())
    wallet = WalletDto(
        id=wallet_id,
        owner_id=request.owner_id,
        currency=request.currency,
        balance_minor=0,
        balance=0.0,
        created_at=__import__("datetime").datetime.now(
            __import__("datetime").timezone.utc
        ),
    )
    _wallets[wallet_id] = wallet
    return {"wallet_id": wallet_id}

@get_mapping("/{wallet_id}")
async def get_wallet(self, wallet_id: PathVar[str]) → WalletDto:
    result = _wallets.get(wallet_id)
    if result is None:
        raise ResourceNotFoundException(
            f"Wallet {wallet_id!r} not found",
            code="WALLET_NOT_FOUND",
            context={"wallet_id": wallet_id},
        )
    return result

@get_mapping("/{wallet_id}/balance")
async def get_balance(self, wallet_id: PathVar[str]) → BalanceDto:
    wallet = _wallets.get(wallet_id)
    if wallet is None:
        raise ResourceNotFoundException(
            f"Wallet {wallet_id!r} not found",
            code="WALLET_NOT_FOUND",
            context={"wallet_id": wallet_id},
        )

```

```

    return BalanceDto(
        id=wallet.id,
        currency=wallet.currency,
        balance_minor=wallet.balance_minor,
        balance=wallet.balance,
    )

@post_mapping("/{wallet_id}/deposit")
async def deposit(
    self,
    wallet_id: PathVar[str],
    request: Valid[Body[DepositRequest]],
) → dict[str, int | str]:
    wallet = _wallets.get(wallet_id)
    if wallet is None:
        raise ResourceNotFoundException(
            f"Wallet {wallet_id!r} not found",
            code="WALLET_NOT_FOUND",
            context={"wallet_id": wallet_id},
        )
    new_balance = wallet.balance_minor + request.amount
    _wallets[wallet_id] = wallet.model_copy(
        update={
            "balance_minor": new_balance,
            "balance": new_balance / 100,
        }
    )
    return {"wallet_id": wallet_id, "balance_minor": new_balance}

@get_mapping("")
async def list_wallets(
    self,
    owner_id: QueryParam[str] = None,
) → list[WalletDto]:
    wallets = list(_wallets.values())
    if owner_id is not None:
        wallets = [w for w in wallets if w.owner_id == owner_id]
    return wallets

```

Listado 4.1 — *WalletController usando los decoradores web reales de PyFly*

Vale la pena examinar cuatro decisiones de diseño de este listado.

`@rest_controller` hace dos cosas a la vez: registra `WalletController` como bean singleton en el contenedor de DI y fija el marcador `__pyfly_stereotype__` que `ControllerRegistrar` utiliza para descubrir y montar rutas en el arranque. Emparejarlo con `@request_mapping("/api/v1/wallets")` significa que cada decorador a nivel de método hereda ese prefijo: escribes la ruta base una sola vez.

Esta versión no tiene ningún `__init__` con colaboradores inyectados. El diccionario en memoria `_wallets` es un almacén a nivel de módulo que basta para la Parte I. El Capítulo 5 introduce los repositorios; el Capítulo 7 muestra el patrón de producción: un constructor que recibe `DefaultCommandBus` y `DefaultQueryBus` del contenedor de DI, despachando comandos y consultas a través del bus en lugar de leer `_wallets` directamente.

Cada manejador devuelve un modelo Pydantic (`WalletDto` , `BalanceDto`) o un `dict` corriente. El framework serializa el valor de retorno a JSON y fija la cabecera `Content-Type` : el manejador nunca construye un objeto de respuesta. El argumento `status_code=201` de `@post_mapping` produce un 201 Created en caso de éxito; todos los demás manejadores usan 200 por defecto.

Los cinco decoradores de mapeo aceptan los mismos dos parámetros:

Parámetro	Por defecto	Descripción
<code>path</code>	<code>""</code>	Ruta relativa añadida a la base. Usa <code>{name}</code> para variables de ruta.
<code>status_code</code>	<code>200</code>	Código de estado HTTP para una respuesta correcta.

`@post_mapping("", status_code=201)` mapea `POST /api/v1/wallets` y devuelve 201 en caso de éxito. `@get_mapping("/{wallet_id}")` mapea `GET /api/v1/wallets/{wallet_id}`. Las rutas se concatenan en el arranque; las barras duplicadas o finales se normalizan automáticamente.

Construir el controlador, paso a paso

Si estás escribiendo esto desde cero, el listado anterior aterriza todo de golpe. Aquí tienes el mismo controlador ensamblado en el orden en que realmente lo construirías, para que cada decorador tenga un trabajo que hacer antes de que llegue el siguiente.

Paso 1 — Crea el fichero y la clase. Crea `src/lumen/web/controllers/wallet_controller.py` y define una clase vacía decorada con los dos decoradores a nivel de clase. Esto basta para que PyFly descubra el controlador en el arranque, incluso antes de que tenga una sola ruta.

```
from pyfly.container import rest_controller
from pyfly.web import request_mapping

@rest_controller
@request_mapping("/api/v1/wallets")
class WalletController:
    """Digital-wallet REST API: open, deposit, inspect."""
```

Paso 2 — Añade el primer manejador. Dale a la clase un método `async def` y márcalo con un decorador de mapeo. `@post_mapping("", status_code=201)` mapea `POST /api/v1/wallets` —la ruta vacía significa «la ruta base sin nada añadido»— y promete un `201 Created` en caso de éxito.

Paso 3 — Añade los manejadores restantes. Repite el patrón: un `async def` por ruta, cada uno con su propio decorador de mapeo y ruta relativa. El conjunto completo del Listado 4.1 te da abrir, obtener, saldo, depósito y listar.

Paso 4 — Conecta el almacén. El diccionario `_wallets` a nivel de módulo es la única «base de datos» que necesita la Parte I. Cada manejador lee de él y escribe en él directamente; el Capítulo 5 lo cambia por un repositorio real sin tocar un solo decorador.

 NOTA

Fíjate en lo que *no* escribiste: ningún fichero de router que mapee URLs a funciones, ninguna llamada de registro en `main.py`, ninguna entrada manual de OpenAPI. Los decoradores son el registro. En el arranque, `ControllerRegistrar` encuentra cada bean `@rest_controller` y monta sus rutas por ti.

 PRUÉBALO

Arranca el servidor y confirma que las rutas están activas. Desde la raíz del proyecto:

```
uv run pyfly run --server uvicorn
```

El banner de arranque informa de la versión del framework y del puerto al que está enlazado:

```
:: PyFly Framework :: (v26.06.110) (Python 3.13.13)
```

En una segunda terminal, abre un monedero y vuelve a leerlo:

```
curl -s -X POST localhost:8080/api/v1/wallets \
  -H 'Content-Type: application/json' \
  -d '{"owner_id": "alice", "currency": "EUR"}'
```

Deberías ver un cuerpo **201** con el id generado:

```
{"wallet_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890"}
```

Copia ese id y obtén el monedero:

```
curl -s localhost:8080/api/v1/wallets/a1b2c3d4-e5f6-7890-abcd-ef1234567890
```

```
{"id": "a1b2c3d4-...", "owner_id": "alice", "currency": "EUR",
  "balance_minor": 0, "balance": 0.0, "created_at": "2026-06-15T10:30:00+00:00"}
```

Qué acaba de pasar. Dos decoradores a nivel de clase registraron el controlador y fijaron su prefijo de URL; un decorador a nivel de método convirtió un `async def` en una ruta activa; el framework hizo el enrutamiento, la serialización JSON y la gestión del código de estado. Escribiste la intención de negocio, no la fontanería.

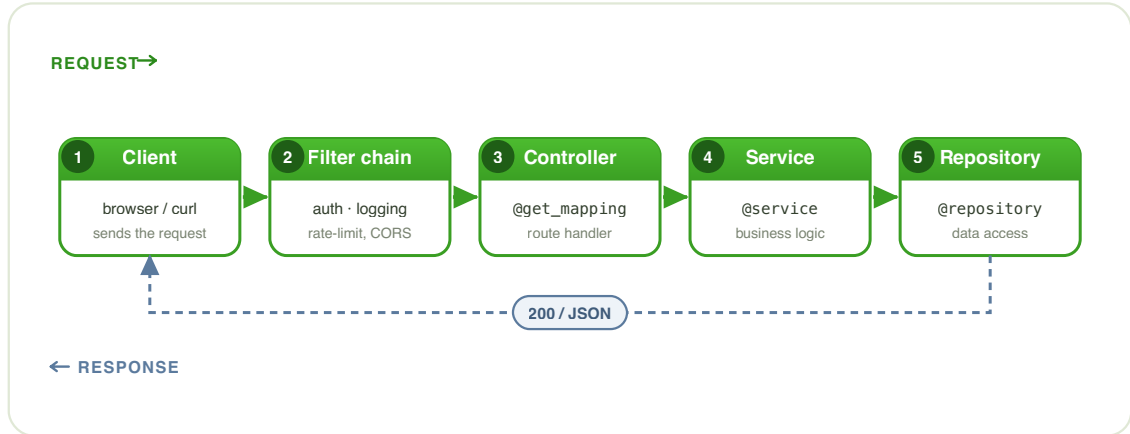


Figura 4.1 — Cómo fluye una petición hasta tu manejador.

🌿 EQUIVALENCIA CON SPRING

`@rest_controller` + `@request_mapping` + `@get_mapping` / `@post_mapping` es una traducción directa de `@RestController` + `@RequestMapping` + `@GetMapping` / `@PostMapping` de Spring. Los métodos manejadores devuelven valores directamente (no `ResponseEntity`) y el framework los convierte a JSON: exactamente el patrón que Spring fomenta con `@ResponseBody` en `@RestController`.

Vincular los datos de la petición

Una petición transporta datos en varios sitios a la vez: la ruta de la URL identifica el recurso, la cadena de consulta lleva filtros y paginación, el cuerpo lleva la carga útil y las cabeceras llevan metadatos. La mayoría de los frameworks abordan esto con mecanismos separados, cada uno con sus propias convenciones. PyFly los unifica bajo una sola idea: **las anotaciones de tipo genéricas en los parámetros del manejador declaran de dónde vienen los datos.**

Este enfoque hace que las firmas de los manejadores se documenten a sí mismas. La lista de parámetros de cualquier manejador te dice exactamente qué partes de la petición lee y qué tipos espera, sin abrir un fichero de router ni consultar la documentación.

En términos llanos, **vincular** (binding) es que el framework copia una pieza de la petición entrante en uno de los parámetros de tu manejador, convirtiéndola al tipo que pediste por el camino. Tú declaras *qué quieres y de dónde viene* con una anotación de tipo; PyFly hace la extracción, el parseo y la coerción de tipos antes de que se ejecute el cuerpo de tu método.

El `ParameterResolver` inspecciona la firma de cada manejador en el arranque y construye un plan de resolución, de modo que no hay sobrecarga de introspección por petición. Cinco tipos de vinculación cubren cada parte de una petición HTTP:

PathVar[T] — variables de ruta

Extrae un segmento con nombre de la ruta de la URL. El nombre del parámetro debe coincidir con un `{placeholder}` de la ruta.

```
@get_mapping("/{wallet_id}")
async def get_wallet(self, wallet_id: PathVar[str]) → WalletDto:
    ...

@get_mapping("/{wallet_id}/transactions/{txn_id}")
async def get_transaction(
    self,
    wallet_id: PathVar[str],
    txn_id: PathVar[str],
) → dict:
    ...
```

`PathVar` coacciona el segmento de cadena en bruto a `T` automáticamente.

`PathVar[int]`, `PathVar[float]` y `PathVar[UUID]` funcionan todos: la coerción llama a `int(value)`, `float(value)` y `UUID(value)` respectivamente.

QueryParam[T] — parámetros de consulta

Extrae un valor de la cadena de consulta, con soporte para valores por defecto y valores opcionales.

```
@get_mapping("")
async def list_wallets(
    self,
    owner_id: QueryParam[str] = None,
    page: QueryParam[int] = 1,
    size: QueryParam[int] = 20,
) → list[WalletDto]:
    ...
```

Un parámetro es **obligatorio** cuando no tiene un valor por defecto en Python y su tipo no admite `None`. Un `QueryParam` obligatorio ausente lanza

`InvalidRequestException` (HTTP 400). Para hacer un parámetro opcional, dale un valor por defecto o anótalo como `QueryParam[str | None]`.

🔗 PRUÉBALO

Con el servidor en marcha y al menos un monedero abierto, ejercita el manejador

`list_wallets` : primero sin filtro, luego con el parámetro de consulta opcional `owner_id` :

```
curl -s 'localhost:8080/api/v1/wallets'
curl -s 'localhost:8080/api/v1/wallets?owner_id=alice'
```

El primero devuelve todos los monederos; el segundo devuelve solo los de Alice. Como `owner_id` tiene un valor por defecto de `None` , omitirlo es perfectamente válido: ningún 400. La variable de ruta se comporta igual a la inversa: pide un id de monedero que no existe y obtendrás un 404 limpio, que la siguiente sección disecciona.

Body[T] — cuerpo de la petición

Deserializa el cuerpo de la petición JSON (o XML). Cuando `T` es un `BaseModel` de Pydantic, se llama a `model_validate_json()` automáticamente.

```
@post_mapping("", status_code=201)
async def open_wallet(
    self, request: Valid[Body[OpenWalletRequest]]
) → dict[str, str]:
    ...
```

Header[T] y Cookie[T]

Extraen valores de las cabeceras y cookies de la petición. Para las cabeceras, el nombre del parámetro se convierte de `snake_case` a `kebab-case` automáticamente:

```
@get_mapping("/me")
async def get_my_wallets(
    self,
    x_api_key: Header[str],
    session_id: Cookie[str | None],
) → List[WalletDto]:
    ...
```

`x_api_key: Header[str]` lee la cabecera `x-api-key`. Una cabecera o cookie obligatoria ausente lanza `InvalidRequestException` (HTTP 400), igual que un parámetro de consulta ausente.

💡 CONSEJO

Los cinco tipos de vinculación siguen la misma regla de **obligatorio frente a opcional**: sin valor por defecto + tipo que no admite `None` = obligatorio (HTTP 400 cuando está ausente); cualquier valor por defecto o `T | None` = opcional. La regla es uniforme en `QueryParam`, `Header` y `Cookie`: la aprendes una vez y se aplica en todas partes.

Qué acaba de pasar. Aprendiste todo el vocabulario de vinculación como cinco anotaciones paralelas — `PathVar`, `QueryParam`, `Body`, `Header`, `Cookie` — que se leen como prosa en la firma de un manejador y comparten una única regla de obligatorio-frente-a-opcional. El framework lee la anotación, saca el valor del sitio correcto, lo coacciona a tu tipo y te entrega un argumento listo para usar. No hay nada más que cablear.

Validación con Valid[T]

La vinculación le dice al framework *de dónde* vienen los datos. La validación le dice *qué aspecto deben tener esos datos* antes de que tu manejador los vea siquiera. Sin una capa que intercepte la entrada incorrecta pronto, la lógica de validación se

dispersa por los métodos de servicio, bloques `if` manuales ensucian el código de negocio y distintos manejadores producen respuestas de error inconsistentes según dónde resulte que capturen el problema.

PyFly resuelve esto a nivel de tipo. El `BaseModel` de Pydantic te da restricciones a nivel de campo gratis. `Valid[T]` es el marcador de PyFly que convierte un `ValidationError` de Pydantic en una **respuesta 422 estructurada** en lugar de dejar que escale a un 500.

Una breve glosa antes del código. Un **DTO** —Data Transfer Object, objeto de transferencia de datos— es una clase pequeña que describe la *forma* de los datos que cruzan el cable: qué campos debe llevar una petición, o qué campos devolverá una respuesta. Los DTO de Lumen son modelos Pydantic corrientes, de modo que las declaraciones de campo hacen también de reglas de validación. La **validación** es el acto de comprobar los datos entrantes contra esas reglas y rechazarlos limpiamente si no encajan, antes de que se ejecute nada del código de tu manejador.

DTO de Pydantic para Lumen

Los DTO de petición y respuesta usados en la API del monedero de Lumen viven bajo `lumen/interfaces/dtos/v1/`: un fichero por DTO. El nombre del directorio codifica una convención que vale la pena señalar: `interfaces` contiene los contratos que ve el mundo exterior, y `v1` los versiona para que una futura forma de carga útil `v2` pueda convivir junto a la antigua sin romper a los clientes existentes. Aquí los tienes completos.

```
lumen/interfaces/dtos/v1/open_wallet_request.py
```

```
from __future__ import annotations

from pydantic import BaseModel, Field

from lumen.interfaces.enums.v1.currency import Currency


class OpenWalletRequest(BaseModel):
    """Wallet-opening request payload."""

    owner_id: str = Field(
        min_length=1,
        max_length=64,
        description="Identifier of the wallet owner",
    )
    currency: Currency = Field(
        default=Currency.EUR,
        description="ISO-4217 currency the wallet holds",
    )
```

Listado 4.2a — *OpenWalletRequest*: carga útil de apertura de monedero

`lumen/interfaces/dtos/v1/deposit_request.py`

```
from __future__ import annotations

from pydantic import BaseModel, Field

class DepositRequest(BaseModel):
    """Deposit/withdrawal request payload.

    Shared by POST /{id}/deposit and POST /{id}/withdraw – both move a
    positive amount of money in the wallet's own currency.
    """

    amount: int = Field(
        gt=0,
        description="Amount in minor units (cents); must be positive",
    )
```

Listado 4.2b — *DepositRequest*: carga útil de depósito/retirada

```
lumen/interfaces/dtos/v1/wallet_dto.py
```

```
from __future__ import annotations

from datetime import datetime

from pydantic import BaseModel

from lumen.interfaces.enums.v1.currency import Currency


class WalletDto(BaseModel):
    """Full wallet representation returned to clients.

    ``balance_minor`` is in minor units (cents); ``balance`` is the same
    value rendered as a major-unit decimal for human-friendly display.
    """

    id: str
    owner_id: str
    currency: Currency
    balance_minor: int
    balance: float
    created_at: datetime
```

Listado 4.2c — *WalletDto: respuesta completa del monedero*


```
lumen/interfaces/dtos/v1/balance_dto.py
```

```
from __future__ import annotations

from pydantic import BaseModel

from lumen.interfaces.enums.v1.currency import Currency


class BalanceDto(BaseModel):
    """Lightweight balance projection for the balance endpoint."""

    id: str
    currency: Currency
    balance_minor: int
    balance: float
```

Listado 4.2d — BalanceDto: proyección ligera del saldo

Estos son modelos Pydantic puros: PyFly no les añade nada. Vale la pena desglosar cuatro decisiones de diseño.

`OpenWalletRequest.owner_id` usa `Field(min_length=1, max_length=64)`. El límite inferior evita monederos fantasma a partir de IDs de propietario en cadena vacía que contaminarían silenciosamente tus datos; el límite superior mantiene los identificadores dentro de un ancho de columna razonable cuando llegue la capa de base de datos en el Capítulo 5.

`currency` es un enum `Currency` (un `StrEnum` con `EUR`, `USD`, `GBP`). Usar un enum en lugar de una cadena en bruto significa que Pydantic rechaza `"XYZ"` en el momento de la deserialización: nunca validas el código de divisa tú mismo.

`Field(default=Currency.EUR)` proporciona un valor por defecto razonable para que los llamantes puedan omitir el campo en monederos en EUR.

`DepositRequest.amount` es un `int` con `Field(gt=0)`. Almacenar el dinero en unidades menores evita el redondeo de coma flotante: `1050` significa 10,50 € en un monedero en EUR. La restricción `gt=0` convierte un depósito cero o negativo en un error de cliente 422, no en una decisión de lógica de negocio: la restricción vive en el tipo, y Pydantic la aplica antes de que se ejecute tu manejador.

`WalletDto` y `BalanceDto` son modelos de respuesta. Devolver un modelo Pydantic tipado en lugar de un `dict` corriente permite al framework generar esquemas de respuesta OpenAPI precisos y da a los clientes un contrato legible por máquina.

Usar `Valid[T]` en un manejador

Envuelve `Body[T]` en `Valid` para optar por errores 422 estructurados cuando falle la validación:

```
@post_mapping("", status_code=201)
async def open_wallet(
    self, request: Valid[Body[OpenWalletRequest]]
) → dict[str, str]:
    ...

@post_mapping("/{wallet_id}/deposit")
async def deposit(
    self,
    wallet_id: PathVar[str],
    request: Valid[Body[DepositRequest]],
) → dict[str, int | str]:
    ...
```

`Valid[Body[OpenWalletRequest]]` le dice dos cosas al resolver: vincula desde el cuerpo de la petición (`Body`) y ejecuta la validación de Pydantic antes de que se ejecute el manejador (`Valid`). Cuando el cuerpo falla la validación, el resolver captura

el `ValidationError` y lanza una `ValidationException` con `code="VALIDATION_ERROR"` y un array `context.errors` que contiene cada detalle a nivel de campo.

Lo que ve el cliente en caso de fallo

Para ver la validación en acción, envía un `POST /api/v1/wallets` con un `owner_id` vacío:

```
POST /api/v1/wallets
Content-Type: application/json

{"owner_id": ""}
```

🔍 PRUÉBALO

Con el servidor en marcha, envía la carga útil incorrecta y observa el `422`:

```
curl -s -w '\nHTTP %{http_code}\n' -X POST localhost:8080/api/v1/wallets \
-H 'Content-Type: application/json' \
-d '{"owner_id": ""}'
```

La opción `-w '\nHTTP %{http_code}\n'` imprime la línea de estado después del cuerpo, para que puedas confirmar que es `HTTP 422`: no el `201` que devuelve una petición válida, ni un `500`. El cuerpo es el sobre estructurado que se muestra a continuación. Prueba una segunda variante —

`-d '{"owner_id": "alice", "currency": "XYZ"}'` — para ver cómo el enum `Currency` rechaza un código desconocido con la misma forma de sobre.

La respuesta es HTTP 422:

```

{
  "error": {
    "message": "Validation failed: owner_id: String should have at least 1
character",
    "code": "VALIDATION_ERROR",
    "status": 422,
    "path": "/api/v1/wallets",
    "timestamp": "2026-06-07T10:30:00+00:00",
    "transaction_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "context": {
      "errors": [
        {
          "type": "string_too_short",
          "loc": ["owner_id"],
          "msg": "String should have at least 1 character",
          "input": "",
          "ctx": {"min_length": 1}
        }
      ]
    }
  }
}

```

Cada error de campo lleva un `type` (legible por máquina), un `loc` (ruta hasta el campo que falla), un `msg` (legible por humanos) y un `input` (el valor rechazado). Los consumidores de la API parsean este array de forma determinista: sin rascar cadenas de error.

La diferencia entre `Body[T]` a secas y `Valid[Body[T]]` es exactamente esta:

Anotación	Cuando falla la validación
<code>Body[T]</code>	Se propaga el <code>ValidationError</code> de Pydantic en bruto: puede convertirse en un 500 sin gestión adicional
<code>Valid[Body[T]]</code>	Capturado, convertido en <code>ValidationException</code> , produce siempre un 422 estructurado

Usa `Valid[Body[T]]` en cada endpoint que acepte entrada del usuario.

🍃 EQUIVALENCIA CON SPRING

`Valid[Body[T]]` mapea directamente a la combinación de `@Valid` + `@RequestBody` de Spring en un método `@RestController`. En Spring escribes `@PostMapping public ResponseEntity create(@Valid @RequestBody OpenWalletRequest body)`; en PyFly escribes `async def open_wallet(self, request: Valid[Body[OpenWalletRequest]])`. La forma de la respuesta 422 (errores a nivel de campo con rutas de ubicación) refleja la carga útil de `MethodArgumentNotValidException` de Spring Boot 3.

Qué acaba de pasar. Las reglas de validación nunca salieron del DTO.

`Field(min_length=1)` en `owner_id`, el enum `Currency` y `Field(gt=0)` en `amount` son toda la especificación, y envolver el cuerpo en `Valid` convirtió cualquier infracción de esas reglas en un `422` predecible y legible por máquina antes de que se ejecutara tu manejador. Escribiste las restricciones una vez, sobre los datos; el framework las aplicó en todas partes donde llegan los datos.

Errores en los que los clientes pueden confiar

Una API bien diseñada falla de forma ruidosa, consistente e informativa. Los clientes nunca deben parsear trazas de pila de excepciones ni adivinar qué fue mal a partir de un 500 genérico. El reto es lograr esto sin dispersar lógica específica de HTTP por todo tu código de servicio: el código de estado HTTP es una preocupación de infraestructura, no de negocio.

La jerarquía de excepciones de PyFly es la columna vertebral de su historia de errores. Cada excepción del árbol lleva tres cosas: un `message` legible por humanos, un `code` legible por máquina y un dict `context` opcional para detalle de depuración. El manejador global de excepciones de la capa web mapea cada subclase al código de estado HTTP correcto automáticamente: tú haces `raise`, el framework responde.

El árbol de excepciones

```
PyFlyException
├── BusinessException          → 400 (catch-all)
│   ├── ValidationException    → 422
│   ├── ResourceNotFoundException → 404
│   ├── ConflictException      → 409
│   ├── InvalidRequestException → 400
│   └── ...
├── SecurityException          → 403
│   ├── UnauthorizedException  → 401
│   └── ForbiddenException     → 403
├── InfrastructureException     → 502 (catch-all)
│   ├── ServiceUnavailableException → 503
│   └── CircuitBreakerException → 503
└── ...
```

La jerarquía es deliberadamente plana. `BusinessException` cubre cualquier cosa que sea culpa del llamante; `InfrastructureException` cubre cualquier cosa que sea culpa del sistema. Las subclases fijan el código de estado. Cuando un nuevo error de dominio no encaja en una subclase existente, extiende el padre más cercano y el código de estado viene gratis.

Impórtalas de `pyfly.kernel`:

```
from pyfly.kernel import (
    ResourceNotFoundException,
    ConflictException,
    ValidationException,
    InvalidRequestException,
)
```

Lánzalas desde el código del manejador sin preocuparte por HTTP:

```
raise ResourceNotFoundException(
    f"Wallet {wallet_id!r} not found",
    code="WALLET_NOT_FOUND",
    context={"wallet_id": wallet_id},
)
```

El manejador global la captura, la mapea a 404 y emite una respuesta JSON estructurada:

```
{
  "error": {
    "message": "Wallet 'w-999' not found",
    "code": "WALLET_NOT_FOUND",
    "status": 404,
    "path": "/api/v1/wallets/w-999",
    "timestamp": "2026-06-07T10:30:00+00:00",
    "transaction_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "context": {
      "wallet_id": "w-999"
    }
  }
}
```

🔍 PRUÉBALO

Pide un monedero que nunca se abrió y observa cómo el framework convierte tu `raise` en un `404` limpio:

```
curl -s -w '\nHTTP %{http_code}\n' localhost:8080/api/v1/wallets/w-999
```

La línea de estado dice `HTTP 404` y el cuerpo es el sobre de arriba, con `"code": "WALLET_NOT_FOUND"` y tu `context` trasladado al pie de la letra. Nunca escribiste un código de estado en `get_wallet: ResourceNotFoundException` se mapea a 404 por ti. Fíjate en el `transaction_id` de la respuesta; cópialo y haz `grep` en el log de tu servidor para encontrar la petición exacta.

El `transaction_id` es gratis: el `TransactionIdFilter` asigna un UUID a cada petición y lo enhebra por todas las respuestas de error. Los clientes lo registran; el soporte lo usa para encontrar la entrada de log de servidor correspondiente. Un solo ID es todo lo que se necesita para reconstruir lo que pasó.

Qué acaba de pasar. Tu manejador expresó un hecho de dominio —«este monedero no existe»— lanzando una excepción tipada con un mensaje, un código y algo de contexto. El manejador global de la capa web hizo la traducción a HTTP: eligió el código de estado a partir de la clase de la excepción, lo envolvió todo en el sobre de error estándar y estampó un `transaction_id`. Las preocupaciones de HTTP se mantuvieron completamente fuera de tu código de negocio.

RFC 7807

El sobre de error por defecto — `{"error": {...}}` — es el formato propio de PyFly. Si tu equipo prefiere el estándar del IETF, fija `pyfly.web.problem-details.enabled: true` en `pyfly.yaml`. Con esa opción activada, la misma `ResourceNotFoundException` produce una respuesta `application/problem+json` con `type`, `title`, `status`, `detail` e `instance` como los miembros estándar de RFC 7807, más `code` y `transactionId` como miembros de extensión de PyFly. Ambos modos usan la misma jerarquía de excepciones y el mismo mapeo de estados.

Negociación de contenido y OpenAPI

JSON y XML

Devolver un `dict` o un modelo Pydantic no es del todo el final de la historia. En algún punto entre la sentencia `return` de tu manejador y los bytes que recibe el cliente, el framework decide un formato de cable. En lugar de codificar JSON a fuego, PyFly pasa el valor de retorno por una cadena ordenada de `HttpMessageConverter`: importante para APIs empresariales que deben servir a socios de XML o negociar el formato más ligero para clientes móviles.

JSON es el predeterminado. Cuando no se envía ninguna cabecera `Accept`, la respuesta es `application/json`. Cuando el cliente envía `Accept: application/xml`, el conversor de XML toma el relevo y serializa el mismo valor de retorno como XML, sin cambio alguno en el código de tu manejador:

```
GET /api/v1/wallets/w-001    Accept: application/json
→ {"id": "w-001", ...}

GET /api/v1/wallets/w-001    Accept: application/xml
→ <response><id>w-001</id>...</response>
```

La misma negociación se aplica a los datos entrantes: un parámetro `Body[T]` o `Valid[Body[T]]` acepta cuerpos de petición tanto con `Content-Type: application/json` como con `Content-Type: application/xml`. JSON es el recurso de reserva cuando no hay ninguna cabecera `Content-Type` presente.

Documentación autogenerada

Las especificaciones de API mantenidas manualmente se desvían. A medida que cambian las rutas, se renombran los parámetros y se añaden nuevos modelos, las especificaciones escritas a mano se quedan atrás respecto al código. PyFly elimina esto por completo generando la documentación a partir de los mismos metadatos que impulsan el enrutamiento: la especificación está siempre sincronizada porque es la misma fuente.

Tan pronto como Lumen arranca, tres endpoints de documentación quedan activos sin coste alguno:

Endpoint	Propósito
<code>/docs</code>	Swagger UI: documentación interactiva, pruébalo-ya
<code>/redoc</code>	ReDoc: documentación de referencia limpia, de dos paneles
<code>/openapi.json</code>	Especificación OpenAPI 3.0 en bruto

El `OpenAPIGenerator` introspecciona los metadatos de ruta de `ControllerRegistrar` —cada ruta, método, variable de ruta, parámetro de consulta y esquema de petición/respuesta (a partir de la introspección de modelos Pydantic)— y ensambla la especificación en el arranque. Nunca escribes la especificación a mano. Los endpoints de documentación viven en el puerto de la **aplicación** (8080) junto a tu API; están activados por defecto (`pyfly.web.docs.enabled: true`). Desactívalos en producción con `pyfly.web.docs.enabled: false` en `pyfly.yaml` .

NOTA

No confundas los endpoints de documentación con el **panel de administración**. `/docs` , `/redoc` y `/openapi.json` describen *tu* API y se sirven en el puerto de la app (8080). El Panel de Administración de PyFly (`/admin`) y los endpoints de salud del actuador (`/actuator/*`) describen el *proceso en ejecución* y se sirven en el puerto de **gestión** separado (`pyfly.management.server.port` , por defecto 9090), introducido en el Capítulo 3. Son dos listeners distintos con dos audiencias distintas.

PRUÉBALO

Con el servidor en marcha, obtén la especificación en bruto y confirma que tus rutas están en ella:

```
curl -s localhost:8080/openapi.json | head -c 200
```

Verás la cabecera de OpenAPI 3.0 y el comienzo del mapa `paths`. Luego abre `http://localhost:8080/docs` en un navegador. Verás `POST /api/v1/wallets`, `GET /api/v1/wallets/{wallet_id}`, `POST /api/v1/wallets/{wallet_id}/deposit` y los demás: cada uno con los esquemas de petición y respuesta correctos derivados de tus modelos Pydantic, y el parámetro de consulta `owner_id` de `list_wallets` ya documentado con su tipo y valor por defecto. Haz clic en «Try it out» en `POST /api/v1/wallets` para abrir un monedero real directamente desde el navegador.

Qué acaba de pasar. No escribiste ni una línea de documentación de API y, sin embargo, apareció una especificación completa, interactiva y siempre exacta. Los mismos metadatos de ruta y modelo que impulsan la gestión de peticiones impulsan también la documentación, de modo que las dos nunca pueden desviarse una de otra.

El servidor por debajo

Lumen ya tiene rutas, vinculaciones, validación y documentación. La última pregunta es qué escucha realmente en el puerto 8080. La respuesta importa: distintos servidores hacen distintos compromisos en rendimiento, soporte de versión HTTP, compatibilidad con el SO y herramientas del ecosistema. Atar una aplicación a un único servidor a nivel de framework te obliga a aceptar esos compromisos de forma permanente.

Un **servidor ASGI** es el proceso que realmente acepta conexiones TCP, parsea HTTP y llama a tu aplicación: la capa entre el socket del sistema operativo y tus manejadores. PyFly no codifica uno a fuego. En el arranque, `ServerAutoConfiguration` ejecuta una selección en cascada basada en lo que está instalado:

Prioridad	Servidor	Característica
1. ^a	Granian	Impulsado por Rust/tokio; el mayor rendimiento con un solo worker
2. ^a	Uvicorn	Estándar del ecosistema; el mejor soporte de herramientas
3. ^a	Hypercorn	HTTP/2 y HTTP/3 nativos

Los tres arrancan a través del mismo protocolo `ApplicationServerPort`, de modo que tu código es completamente ajeno a cuál se está ejecutando. Sobrescríbelo con `pyfly.server.type: uvicorn` en `pyfly.yaml` o con la opción de CLI `--server`:

```
pyfly run --server uvicorn --reload      # development: auto-reload
pyfly run --server granian --workers 4  # production: multi-worker
```

💡 PRUÉBALO

Para el desarrollo del día a día, ejecuta con auto-reload para que el servidor se reinicie en cada guardado:

```
uv run pyfly run --reload
```

PyFly registra el servidor elegido y el puerto enlazado en el arranque. Como `--reload` requiere un vigilante de ficheros integrado, PyFly selecciona **Uvicorn** para el modo reload independientemente del orden de la cascada. Edita un manejador, guarda y observa cómo el log informa del reinicio; luego vuelve a ejecutar cualquier `curl` de antes y ve tu cambio en vivo sin detener el proceso.

El bucle de eventos también es enchufable: `uvloop` (Linux/macOS) y `winloop` (Windows) se seleccionan automáticamente cuando están instalados, entregando una mejora de rendimiento de 2 a 4× sobre el asyncio por defecto. Instálalos con `uv add "pyfly[web-fast]"`.

💡 CONSEJO

Para desarrollo, `pyfly run --reload` es todo lo que necesitas: elige el mejor servidor y bucle de eventos disponibles automáticamente. Para producción, pasa un recuento de workers positivo y explícito para escalar a través de los núcleos: `pyfly run --server granian --workers 4`, como en el ejemplo de arriba. Un valor de `--workers` igual a 0 o negativo se resuelve a un solo worker, de modo que el multi-worker es siempre una opción explícita. Las opciones de la CLI siempre sobrescriben `pyfly.yaml`.

✓ Lo que construiste

La Parte I está completa.

En cuatro capítulos pasaste de un andamiaje vacío a un servicio con forma de producción. Lumen ahora **arranca** (`@pyfly_application`, banner de arranque, logging estructurado), está **cableado** (servicios y repositorios conectados mediante inyección por constructor sin código de pegamento), **configurado** (`pyfly.yaml` de cuatro capas + superposiciones de perfil + secretos por variable de entorno, `WalletProperties` tipado) y **sirve**: una API REST validada en `/api/v1/wallets` con vinculación de cuerpo (`PathVar`, `QueryParam` y `Valid[Body[T]]`); errores 422 estructurados a partir de restricciones de Pydantic; mapeo de error-de-dominio-a-estado desde la jerarquía de excepciones; modelos de respuesta tipados (`WalletDto`, `BalanceDto`) que impulsan la generación de esquemas OpenAPI; y un servidor ASGI enchufable corriendo por debajo.

Cada parte de esta pila sigue el mismo principio hexagonal que has visto a lo largo del libro: tu código depende de puertos y decoradores; el framework cablea los adaptadores. Cambia el almacén en memoria por un adaptador de PostgreSQL en el Capítulo 5, sustituye el despacho directo por un bus CQRS completo en el Capítulo 7, o habilita respuestas XML: nada de eso requiere tocar la estructura de decoradores del controlador ni las formas de los DTO.

La Parte II lleva a Lumen más lejos: datos persistentes con SQLAlchemy, eventos de dominio, resiliencia con cortacircuitos (circuit breakers) y seguridad con JWT. Los cimientos que construiste aquí se llevan adelante intactos.

Pruébalo tú mismo

Cada ejercicio es pequeño y autocontenido. Después de cada cambio, reinicia con `uv run pyfly run --reload` y vuelve a ejecutar el `curl` sugerido para confirmar el comportamiento. Si tienes instaladas las dependencias de desarrollo, también puedes ejecutar la suite de pruebas del proyecto en cualquier momento para asegurarte de que nada ha regresionado:

```
uv run --extra dev pytest
```

Deberías ver una fila de puntos que pasan y una línea de resumen `passed`.

1. **Añade un endpoint `DELETE /api/v1/wallets/{wallet_id}`**. Elimina el monedero de `_wallets` y devuelve 204 No Content. Lanza `ResourceNotFoundException` si el monedero no existe. Decóralo con `@delete_mapping("/{wallet_id}", status_code=204)`: PyFly convierte un retorno `None` con `status_code=204` en una respuesta 204 sin cuerpo. Verifícalo con `curl -X DELETE http://localhost:8080/api/v1/wallets/{id}`.

2. Añade filtrado por divisa a `list_wallets`. Añade un parámetro `currency:`

`QueryParam[str] = None` y filtra `_wallets.values()` cuando no sea `None`.

Pruébalo con `GET /api/v1/wallets?currency=EUR` y confirma que solo se devuelven los monederos en EUR; confirma que `GET /api/v1/wallets` sin el parámetro devuelve todos los monederos. Luego haz que el parámetro sea obligatorio eliminando el valor por defecto: observa la respuesta 400 cuando lo omites de la petición.

3. Lanza una subclase específica de dominio de `ResourceNotFoundException`.

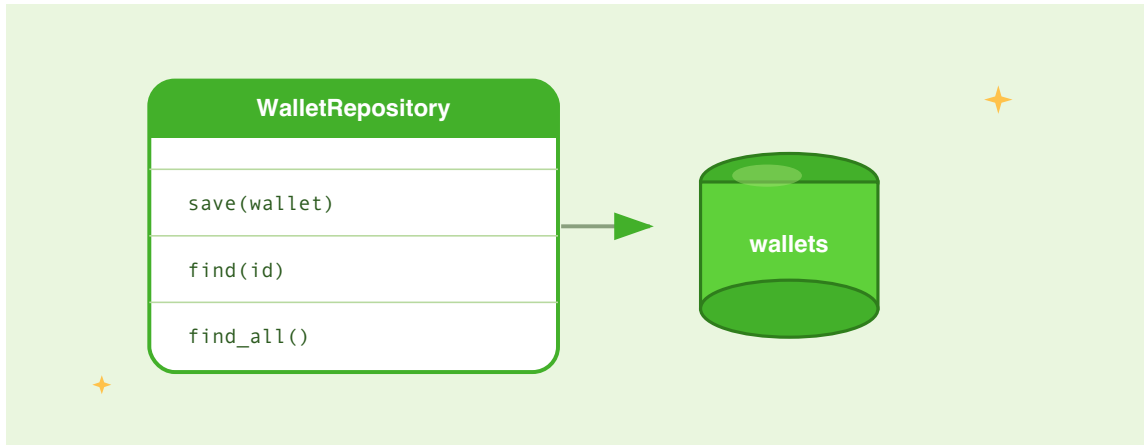
Crea un `WalletNotFoundError` que herede de `ResourceNotFoundException` y lleve un campo `currency` extra en `context`. Lánzalo desde `get_wallet` y `get_balance` en lugar de `ResourceNotFoundException` a secas. Verifica que la respuesta de error JSON incluye el campo de contexto extra sin ningún cambio en el manejador global: la jerarquía lo mapea a 404 automáticamente.

PARTE II

Modelar y Persistir el Dominio

CAPÍTULO 5

Persistencia y el patrón Repositorio



Lumen tiene una API de monederos (wallets) que funciona, pero cada monedero desaparece en el momento en que reinicias el proceso. Ha llegado la hora de hacer que los monederos sean duraderos.

El enfoque ingenuo es esparcir llamadas a `select()` y `session.commit()` de SQLAlchemy por los manejadores de comandos. PyFly ofrece algo mucho mejor: una **capa de repositorios al estilo de Spring Data**. Declaras una interfaz — `class WalletRepository(Repository[WalletEntity, str])` — y el framework *la implementa por ti*. El CRUD asíncrono completo viene gratis. Los métodos de consulta se derivan de sus **nombres**. La paginación, la ordenación, los filtros componibles y las proyecciones de lectura son ciudadanos de primera clase. No hay ningún adaptador escrito a mano ni SQL en el código de la aplicación.

Este capítulo reconstruye la persistencia de Lumen sobre esa capa, exactamente como lo hace el ejemplo en ejecución: la entidad de SQLAlchemy, el repositorio con sus consultas derivadas y por especificación, `Page / Pageable / Sort`, las proyecciones para las vistas de lectura y la junta transaccional que mantiene íntegro el agregado `Wallet`. Todo lo que aparece aquí se ejecuta contra un fichero SQLite real con cero infraestructura externa: los 43 tests del ejemplo pasan sobre él. Este capítulo se dirige a PyFly **v26.6.110**.

Construiremos la capa de persistencia pieza a pieza, y en cada hito hay un recuadro **Ejecútalo** con el comando exacto que debes escribir y la salida que deberías ver. Si estás siguiendo el ejemplo Lumen, trabaja desde la raíz del proyecto (`samples/lumen`), donde viven `pyfly.yaml` y `pyproject.toml` ; todos los comandos de abajo dan por hecho ese directorio.

❗ EJECÚTALO: VE EL PROBLEMA PRIMERO

Antes de añadir la persistencia, vale la pena sentir el hueco que cierra. Abre un monedero, luego detén y reinicia el proceso: con el almacén en memoria de la Parte I el monedero ha desaparecido. El resto de este capítulo hace que sobreviva a ese reinicio.

```
# Terminal 1 – start the app
uv run pyfly run --server uvicorn
# ... startup banner, then: Uvicorn running on http://0.0.0.0:8080

# Terminal 2 – open a wallet, then read it back
curl -s -X POST localhost:8080/api/v1/wallets \
  -H 'content-type: application/json' \
  -d '{"owner_id": "alice", "currency": "EUR"}'
```

Recuperas el nuevo id, y luego la lectura del saldo confirma que existe:

```
{"wallet_id": "wlt-7f3c..."}
```

Ahora pulsa **Ctrl+C** en la Terminal 1, arranca la aplicación de nuevo y pide el saldo de ese monedero. Antes de este capítulo, la fila nunca se escribió en disco, así que ha desaparecido:

```
{"detail": "Wallet 'wlt-7f3c...' not found", "code": "WALLET_NOT_FOUND"}
```

Al final de este capítulo la misma secuencia devuelve el saldo después de un reinicio.

El repositorio, en una frase

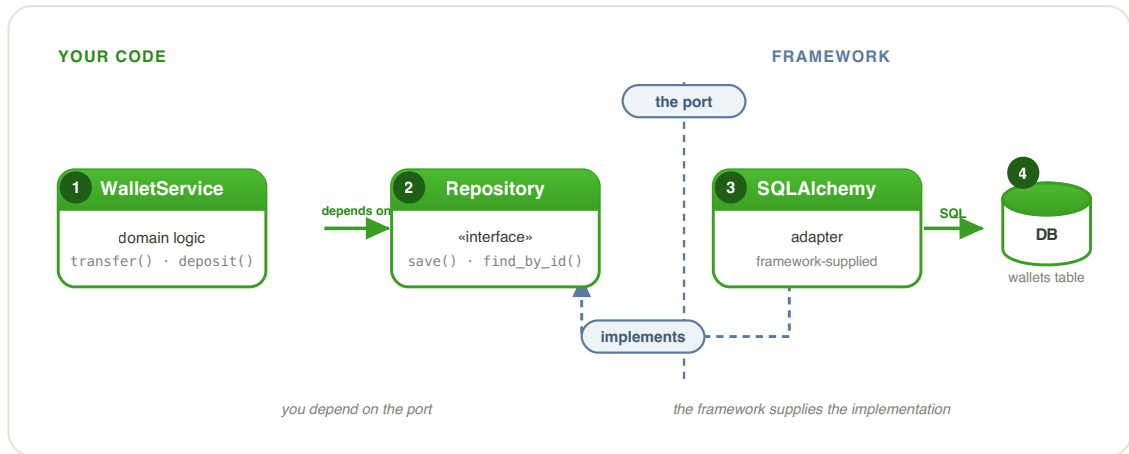


Figura 5.1 — Tu código depende del repositorio; el framework suministra la implementación de SQLAlchemy que hay detrás.

Un repositorio de PyFly es una clase que hereda del genérico `Repository[Entity, ID]` y va marcada con el estereotipo `@repository`. Esa es toda la declaración. A partir de los dos parámetros de tipo el framework aprende el **tipo de entidad** y el **tipo de la clave primaria**, y desde ahí proporciona una superficie completa de acceso a datos asíncrono — `save`, `find_by_id`, `find_all`, `delete` / `delete_by_id`, `count`, `exists_by_id`, además de paginación y consultas por especificación—, y cada llamada encuentra su sesión de base de datos en la unidad de trabajo en la que se ejecuta.

Este es el patrón Repositorio tal como lo popularizó Spring Data, trasladado a un Python asíncrono idiomático. Tú escribes *qué* quieres (el método) y el framework escribe *cómo* (el SQL).

🍃 EQUIVALENCIA CON SPRING

`Repository[T, ID]` es el `JpaRepository<T, ID>` de PyFly. Heredar de él para obtener el CRUD, derivar consultas a partir de los nombres de método, `Pageable / Page`, `Specification` y las proyecciones por interfaz están todos trasladados casi nombre por nombre desde Spring Data JPA. Si has escrito un `interface OrderRepository extends JpaRepository<Order, UUID>` de Spring, ya conoces la forma de este capítulo.

La entidad: una fila por monedero

Antes de que un repositorio pueda almacenar algo, necesita una **entidad**: la forma en disco de un monedero, una fila plana por agregado. (Una *entidad*, en esta capa, es simplemente una clase de Python que se mapea a una tabla de la base de datos; cada instancia es una fila.) Las entidades de PyFly son modelos ordinarios de SQLAlchemy 2.0 construidos sobre una base declarativa que el framework exporta.

La construiremos campo a campo. Crea el fichero `src/lumen/models/entities/v1/wallet_orm.py` y añade las piezas por orden.

Paso 1 — importa la base declarativa del framework. Cada entidad hereda de una *base declarativa*: una clase de SQLAlchemy que registra cada tabla que defines para que el framework pueda crearlas todas al arrancar. PyFly exporta una como `Base`:

```
from pyfly.data.relational.sqlalchemy import Base
```

Paso 2 — nombra la tabla y declara las columnas. Hereda de `Base`, fija `__tablename__` y escribe un atributo tipado por columna. La entidad completa es corta:

lumen/models/entities/v1/wallet_orm.py

```

from __future__ import annotations

from datetime import UTC, datetime

from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column

from pyfly.data.relational.sqlalchemy import Base, UtcDateTime

class WalletEntity(Base):
    """One persisted wallet row, keyed by the aggregate's string id."""

    __tablename__ = "wallets"

    id: Mapped[str] = mapped_column(String(64), primary_key=True)
    owner_id: Mapped[str] = mapped_column(
        String(255), nullable=False, index=True
    )
    currency: Mapped[str] = mapped_column(String(3), nullable=False)
    balance_minor: Mapped[int] = mapped_column(nullable=False, default=0)
    created_at: Mapped[datetime] = mapped_column(
        UtcDateTime(), default=lambda: datetime.now(UTC)
    )

```

Listado 5.1 — *WalletEntity*: la fila de persistencia de SQLAlchemy

La sintaxis `Mapped[T]` / `mapped_column(...)` es el estilo de SQLAlchemy 2.0: cada anotación de tipo dirige tanto el tipo del atributo de Python como el DDL de columna generado (la sentencia `CREATE TABLE`), de modo que cada columna tiene una única fuente de verdad. Como `WalletEntity` hereda de `Base`, importar este módulo registra la tabla `wallets` en `Base.metadata` —el registro de todas las tablas que conoce la base—, y el ciclo de vida del motor del framework la crea entonces al arrancar. `UtcDateTime` es el tipo de columna de PyFly para los instantes: guarda un

`datetime` UTC con zona horaria en todos los backends (`TIMESTAMPTZ` en PostgreSQL, `DATETIME(6)` en MySQL y MariaDB), mientras que una columna `DateTime` normal en PostgreSQL no lleva zona horaria y rechaza el valor con zona que escribe el mapeador.

QUÉ ACABA DE PASAR

Escribiste una clase, sin SQL. Los cinco atributos tipados se convirtieron en cinco columnas; `primary_key=True` marcó `id` como la clave; `index=True` en `owner_id` acelerará la consulta de "monederos propiedad de X" que construyes más adelante; `nullable=False` y `default=...` fijan las restricciones. Importar este módulo basta para que el framework sepa que la tabla existe: nunca llamas a `CREATE TABLE` tú mismo.

Hay dos decisiones de diseño que conviene destacar.

Base, no BaseEntity. PyFly trae dos bases declarativas. `BaseEntity` te da una clave primaria sustituta de tipo **UUID** más cuatro columnas de auditoría (`created_at` , `updated_at` , `created_by` , `updated_by`) rellenas automáticamente: el valor por defecto adecuado para la mayoría de las tablas. Lumen usa deliberadamente la `Base` simple en su lugar, porque el agregado `Wallet` ya es dueño de su identidad: un id de cadena con la forma `wlt-...` . Heredar de `Base` deja que la fila conserve esa clave primaria de **cadena**, de modo que la fila y el agregado comparten una identidad en lugar de que la fila invente una segunda, sustituta.

Unidades menores en enteros. Los importes viven en `balance_minor` como céntimos enteros, nunca como un número en coma flotante. Las columnas de coma flotante acumulan error de redondeo a lo largo de millones de transacciones; la aritmética entera se mantiene exacta. Un saldo de `2500` significa 25,00 €: el decimal en unidad mayor se calcula solo en los bordes, para mostrarlo.

 RECURRE A BASEENTITY POR DEFECTO

A menos que tu agregado sea dueño de una clave natural como lo es `Wallet`, prefiere `class Order(BaseEntity)`. Obtienes una PK de tipo UUID y columnas de auditoría gratis, y el `AuditingEntityListener` rellena `created_by` / `updated_by` a partir del contexto de seguridad en cada inserción y actualización. Lumen es la excepción, no la regla.

El repositorio: CRUD gratis

Ahora, la pieza central. El `WalletRepository` de Lumen hereda de `Repository[WalletEntity, str]` —tipo de entidad `WalletEntity`, tipo de clave primaria `str`— y se registra con `@repository`. (Un *estereotipo* como `@repository` es un decorador de clase que le dice al contenedor del framework: "gestióname una instancia de esto". Esa instancia gestionada es un **bean**: un objeto que el contenedor crea una vez y entrega a cualquiera que lo pida. La *inyección de dependencias*, o *DI*, es que el contenedor haga esa entrega por ti, de modo que un manejador nunca construye su propio repositorio.) Esa única declaración basta para que el framework suministre toda la superficie CRUD —siendo *CRUD* las cuatro operaciones básicas de tabla: Crear, Leer, Actualizar, Borrar (Create, Read, Update, Delete).

Crea `src/lumen/models/repositories/wallet_repository.py` en dos pasos.

Paso 1 — importa lo que la declaración necesita. Tres imports: tu entidad, el estereotipo `@repository` y la base genérica `Repository`.

```
from lumen.models.entities.v1.wallet_orm import WalletEntity
from pyfly.container import repository
from pyfly.data.relational.sqlalchemy import Repository
```

Paso 2 — hereda de la base genérica y márcala con `@repository`. Los dos parámetros de tipo llevan todo el cableado:

`lumen/models/repositories/wallet_repository.py`

```
from __future__ import annotations

from lumen.models.entities.v1.wallet_orm import WalletEntity
from pyfly.container import repository
from pyfly.data import Page, Pageable
from pyfly.data.relational.sqlalchemy import Repository, Specification

@repository
class WalletRepository(Repository[WalletEntity, str]):
    """CRUD + derived + specification queries for WalletEntity.

    The @repository stereotype registers this as a DI bean. The
    framework reads the entity/PK types from the
    Repository[WalletEntity, str] base; every call joins the unit
    of work of the @transactional method that makes it, or runs in
    a short unit of its own.
    """

    # (query methods follow — see the next sections)
```

Listado 5.2 — `WalletRepository`: heredar del repositorio del framework

No hay `__init__`, ni SQL, ni clase adaptadora. Con solo esa declaración, cualquier manejador que inyecte un `WalletRepository` ya puede llamar a:

Método	Devuelve	Qué hace
<code>save(entity)</code>	<code>T</code>	Inserta una fila nueva o fusiona una existente
<code>find_by_id(id)</code>	<code>T None</code>	Carga por clave primaria
<code>find_all(**filters)</code>	<code>list[T]</code>	Todas las filas, filtros de igualdad opcionales
<code>find_all(sort)</code>	<code>list[T]</code>	Todas las filas en un orden <code>Sort</code> dado
<code>find_all(pageable)</code>	<code>Page[T]</code>	Consulta paginada + ordenada (ver abajo)
<code>stream_all(sort)</code>	<code>AsyncIterator[T]</code>	Recorre en flujo cada fila (el análogo de <code>Flux<T></code>)
<code>delete(entity)</code>	<code>None</code>	Borra una entidad dada
<code>delete_by_id(id)</code>	<code>None</code>	Borra por clave primaria (sin efecto si no existe)
<code>delete_all(entities=None)</code>	<code>None</code>	Borra las entidades dadas (o todas las filas)
<code>delete_all_by_id(ids)</code>	<code>None</code>	Borra muchas filas por clave primaria
<code>count()</code>	<code>int</code>	Cuenta todas las filas de la tabla
<code>exists_by_id(id)</code>	<code>bool</code>	Si existe una fila con este id
<code>save_all(entities)</code>	<code>list[T]</code>	Inserción/actualización en bloque
<code>find_all_by_id(ids)</code>	<code>list[T]</code>	Carga muchas filas por clave primaria

Método	Devuelve	Qué hace
<code>find_all_by_spec(spec)</code>	<code>list[T]</code>	Filas que satisfacen una <code>Specification</code>
<code>find_all_by_spec_paged(...)</code>	<code>Page[T]</code>	Consulta <code>Specification</code> paginada + ordenada

Eso es más que suficiente para la mayoría de las entidades. Lumen añade tres métodos propios por encima —una consulta derivada, una consulta por especificación y un upsert— que las siguientes secciones van construyendo.

🍃 EQUIVALENCIA CON SPRING

Esta superficie heredada es exactamente la jerarquía de repositorios de Spring Data, trasladada nombre por nombre y estable a partir de PyFly **v26.6.110**: `CrudRepository` → `ReactiveSortingRepository` → `PagingAndSortingRepository`. `save / save_all`, `find_by_id`, `find_all`, `exists_by_id`, `count` y la familia `delete*` se corresponden con sus equivalentes de Spring; `find_all(pageable)` → `Page[T]` es `findAll(Pageable)`, y `find_all_by_spec*` es el `JpaSpecificationExecutor`. Si conoces `JpaRepository<T, ID>`, ya conoces esta tabla.

Cómo conoce los tipos el framework

Cuando escribes `Repository[WalletEntity, str]`, el hook `__init_subclass__` de la clase base inspecciona `__orig_bases__` en el momento de la definición de la clase y extrae el tipo de entidad (`WalletEntity`) y el tipo del id (`str`) de los parámetros genéricos. (`__init_subclass__` es un hook de Python que se ejecuta una vez, de forma automática, cuando se *define* una subclase, así que esto ocurre en tiempo de importación, antes de crear ningún objeto.) El repositorio no guarda ninguna `AsyncSession` —el manejador de conexión-y-transacción de la base de datos por el que pasa cada consulta—. Cada llamada busca la unidad de trabajo ligada a la tarea

en curso y usa su sesión o, fuera de una transacción, abre una unidad breve propia (la sección "Transacciones", más abajo, muestra qué significa eso para tus escrituras). No se pasa nada manualmente: los parámetros de tipo *son* el cableado.

EJECÚTALO: CONFIRMA QUE EL REPOSITORIO SE CABLEA

La prueba más rápida de que la entidad y el repositorio están sanos es la batería de tests, que ejercita el repositorio contra un fichero SQLite real sin servidor. Desde la raíz del proyecto Lumen:

```
uv run --extra dev pytest tests/test_sql_wallet_repository.py -q
```

Deberías ver pasar todos los tests del repositorio:

```
..... [100%]
5 passed in 0.30s
```

Estos tests construyen el repositorio directamente y ejercitan `upsert`, `find_by_id`, `count`, la consulta derivada y la ruta de especificación: los mismos métodos que construye este capítulo. Si están en verde, las columnas de tu entidad y la declaración `Repository[WalletEntity, str]` son correctas.

Consultas derivadas: el nombre del método es la consulta

El CRUD cubre las búsquedas por clave primaria. Las aplicaciones reales también necesitan consultar por otras columnas: "todos los monederos propiedad de este cliente". En la mayoría de los frameworks escribirías el SQL a mano. En PyFly declaras un **esbozo** (stub) —un método sin cuerpo, solo `...`— y dejas que el framework compile la consulta *a partir del nombre del método*.

Paso 1 — declara el esbozo. Añade un método a `WalletRepository`. El *nombre* describe la consulta; el cuerpo es literalmente `...`:

```
lumen/models/repositories/wallet_repository.py
```

```
@repository
class WalletRepository(Repository[WalletEntity, str]):

    # derived query: compiled from the method name by the post-processor
    async def find_by_owner_id(
        self, owner_id: str
    ) → list[WalletEntity]:
        """All wallets owned by *owner_id* (derived query stub)."""
        ...
```

Listado 5.3 — Una consulta derivada: declarada como esbozo, compilada a partir de su nombre

Paso 2 — deja que el framework rellene el cuerpo. No escribes más código. Al arrancar, un `BeanPostProcessor` —el `RepositoryBeanPostProcessor`— hace el trabajo. (Un *BeanPostProcessor* es un hook que el contenedor ejecuta sobre cada bean justo después de crearlo; este se especializa en repositorios.) Examina el repositorio, detecta que `find_by_owner_id` es un esbozo, analiza el **nombre** convirtiéndolo en una consulta parseada y reemplaza el esbozo por una implementación real que ejecuta `SELECT ... FROM wallets WHERE owner_id = :owner_id`. Llamar a `await repo.find_by_owner_id("alice")` ahora devuelve exactamente las filas de ese propietario.

❗ QUÉ ACABA DE PASAR

Declaraste un método y obtuviste una consulta funcional: el framework leyó la *intención* del nombre `find_by_owner_id` y escribió el SQL por ti. La nomenclatura no es magia; sigue una gramática, que se cubre a continuación. La idea clave: en esta capa describes *qué* quieres mediante cómo nombras el método, y el post-procesador suministra el *cómo*.

La gramática es la convención de Spring Data. Un nombre de método es un **prefijo** seguido de un **sujeto** construido a partir de nombres de campo, operadores, conectores y una cláusula de ordenación opcional:

Parte	Tokens
Prefijo	<code>find_by</code> · <code>count_by</code> · <code>exists_by</code> · <code>delete_by</code>
Conectores	<code>_and_</code> · <code>_or_</code>
Operadores	<code>_greater_than</code> · <code>_less_than</code> · <code>_between</code> · <code>_in</code> · <code>_like</code> · <code>_containing</code> · <code>_is_null</code> · <code>_is_not_null</code>
Ordenación	<code>_order_by_<field>_<asc\ desc></code>

Cada cláusula consume el número correspondiente de argumentos del método (la igualdad y las comparaciones toman uno; `_between` toma dos; `_is_null` / `_is_not_null` no toman ninguno). Algunos ejemplos sobre un hipotético repositorio de pedidos:

```

@repository
class OrderRepository(Repository[Order, UUID]):
    async def find_by_status(self, status: str) → list[Order]: ...

    async def find_by_customer_id_and_status(
        self, customer_id: str, status: str
    ) → list[Order]: ...

    async def find_by_total_greater_than(
        self, min_total: float
    ) → list[Order]: ...

    async def find_by_total_between(
        self, low: float, high: float
    ) → list[Order]: ...

    async def count_by_status(self, status: str) → int: ...

    async def exists_by_customer_id(self, customer_id: str) → bool: ...

    async def find_by_status_order_by_created_at_desc(
        self, status: str
    ) → list[Order]: ...

```

El prefijo decide la *forma* del resultado: `find_by` devuelve una lista, `count_by` devuelve un `int`, `exists_by` devuelve un `bool` y `delete_by` emite un `DELETE` y devuelve el número de filas eliminadas. Nunca escribes el SQL; nombras el método y anotas el tipo de retorno.

💡 CUANDO UN NOMBRE SE VUELVA ABSURDO, USA @QUERY

Los nombres derivados son perfectos hasta dos o tres predicados. Pasado eso, se vuelven ilegibles. Para cualquier cosa más compleja, coloca un decorador `@query("SELECT w FROM WalletEntity w WHERE ...")` (al estilo JPQL, o `native=True` para SQL en bruto) sobre el esbozo y escribe la consulta de forma explícita. El mismo patrón de esbozo más decorador; solo que tú suministras el texto de la consulta en lugar de codificarlo en el nombre.

Paginación: Page, Pageable y Sort

Un endpoint de listado nunca debería devolver *todos* los monederos. La *paginación* es la solución estándar: devolver una **página** de filas de tamaño fijo a la vez, más los metadatos suficientes para que el cliente pueda pedir la siguiente. Los tipos de paginación de PyFly — `Pageable` (qué página, qué tamaño, qué orden), `Sort` (la ordenación) y `Page[T]` (el fragmento más los metadatos)— se heredan directamente de la superficie CRUD a través de `find_all(pageable)`.

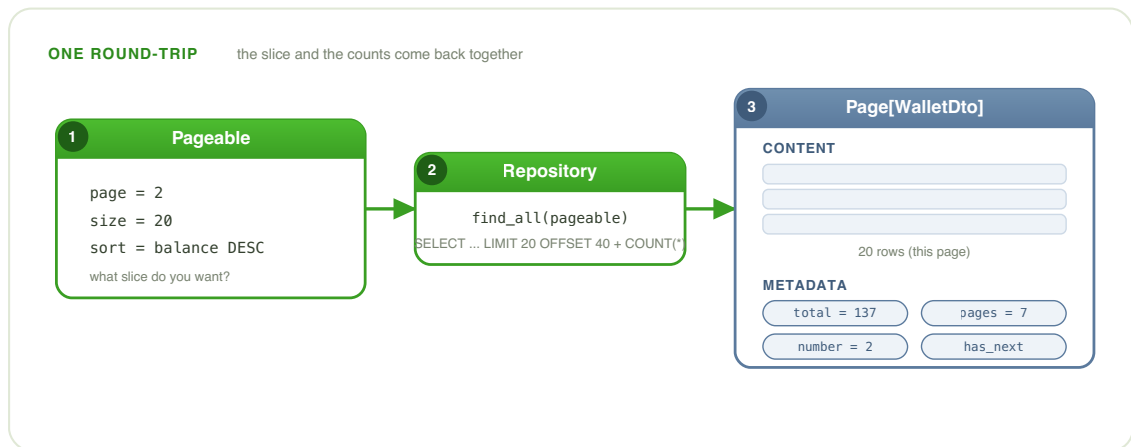


Figura 5.2 — Un solo viaje de ida y vuelta: un `Pageable` (página, tamaño, orden) entra en `find_all` y vuelve un `Page` con el fragmento de filas más los totales que el cliente necesita para dibujar los controles de paginación.

Hay tres piezas pequeñas que ensamblar: el manejador que llama a `find_all(pageable)`, el `Page[T]` que devuelve y el controlador que construye el `Pageable` a partir de la petición. Las veremos en ese orden.

El manejador de consulta `ListWallets` de Lumen es toda la historia en tres líneas:

`lumen/core/services/wallets/list_wallets_handler.py`

```
@query_handler
@service
class ListWalletsHandler(
    QueryHandler[ListWallets, Page[WalletDto]]
):
    def __init__(self, repository: WalletRepository) → None:
        super().__init__()
        self._repository = repository

    async def do_handle( # type: ignore[override]
        self, query: ListWallets
    ) → Page[WalletDto]:
        page = await self._repository.find_all(
            query.pageable
        )
        return page.map(entity_to_dto)
```

Listado 5.4 — Paginando con `find_all(pageable)`, y luego mapeando la página

`find_all(pageable)` hace tres cosas en una sola llamada: cuenta el número total de filas coincidentes, aplica la ordenación del `Pageable` y corta el resultado con `LIMIT / OFFSET`. Devuelve un `Page[WalletEntity]`. El manejador llama entonces a `page.map(entity_to_dto)` para convertir cada fila en un `WalletDto` **sin perder los metadatos de paginación**: `.map` traslada `total`, `page`, `size` y el resto a la nueva página.

Un `Page[T]` expone todo lo que un cliente necesita para renderizar un paginador:

Miembro	Significado
<code>items</code>	Las filas de esta página (<code>list[T]</code>)
<code>total</code>	Total de filas coincidentes en todas las páginas
<code>page</code>	Número de página actual (base 1)
<code>size</code>	Máximo de elementos por página
<code>total_pages</code>	<code>ceil(total / size)</code>
<code>has_next</code>	Si existe una página siguiente
<code>has_previous</code>	Si existe una página anterior
<code>map(fn)</code>	Transforma los elementos, conservando los metadatos

El propio `Pageable` se construye en el borde: el controlador convierte los parámetros de consulta `?page=&size=` en un `Pageable` con un `Sort` compartido de más reciente primero:

```
lumen/web/controllers/wallet_controller.py
```

```
#: Newest-first ordering shared by the list endpoints.
_NEWEST_FIRST = Sort.by("created_at").descending()

@get_mapping("")
async def list_wallets(
    self, page: QueryParam[int] = 1, size: QueryParam[int] = 20
) → PageDto[WalletDto]:
    """A page of wallets, newest first."""
    result = await self._queries.query(
        ListWallets(pageable=Pageable.of(page, size, _NEWEST_FIRST))
    )
    return PageDto.from_page(result)
```

Listado 5.5 — Construyendo un Pageable a partir de los parámetros de consulta (controlador)

`Sort.by("created_at").descending()` nombra la columna y la dirección; `Pageable.of(page, size, sort)` lo empaqueta con las coordenadas de la página. El manejador devuelve un `Page` del framework, y el controlador lo pliega en un `PageDto` serializable —un reflejo Pydantic plano de la página— de modo que `GET /api/v1/wallets?page=1&size=20` devuelve un JSON como `{"items": [...], "total": 42, "page": 1, "total_pages": 3, "has_next": true, ...}`.

❗ EJECÚTALO: PASA POR LAS PÁGINAS DE LOS MONEDEROS

Con la capa relacional activada (la sección "Activarlo" de más abajo la enciende; el ejemplo Lumen ya la trae habilitada), abre un par de monederos y luego pide la primera página. Desde una aplicación en ejecución:

```
curl -s 'localhost:8080/api/v1/wallets?page=1&size=20'
```

La respuesta lleva las filas y los metadatos del paginador: fíjate en `total`, `page`, `total_pages` y `has_next` junto a `items`:

```
{
  "items": [
    {
      "id": "wlt-...", "owner_id": "alice", "currency": "EUR",
      "balance_minor": 0, "balance": 0.0, "created_at": "..."}
  ],
  "total": 1, "page": 1, "size": 20, "total_pages": 1,
  "has_next": false, "has_previous": false
}
```

Esos son exactamente los miembros de `Page[T]` de la tabla de arriba, serializados por `PageDto`. El cliente renderiza un paginador directamente a partir de esta forma, sin necesidad de una consulta de conteo adicional.

Especificaciones: filtros componibles y reutilizables

Las consultas derivadas responden a preguntas fijas. A veces quieres un **predicado reutilizable** —una condición `WHERE` que puedes nombrar una vez y reutilizar— que compones en el lugar de la llamada: "monederos con al menos este saldo", combinado libremente con otras condiciones. Eso es lo que es una `Specification`: un objeto pequeño que envuelve un fragmento `WHERE`, componible con `&` (AND), `|` (OR) y `~` (NOT).

La construimos en dos pasos: una factoría que *devuelve* una `Specification`, y luego un método de repositorio que la *ejecuta*.

Paso 1 — escribe una factoría que devuelva una `Specification`. Toma el parámetro (el saldo mínimo) y devuelve un objeto predicado. **Paso 2 — añade un método de repositorio que lo ejecute** a través del `find_all_by_spec_paged` heredado. Lumen hace ambas cosas en un solo fichero:

`lumen/models/repositories/wallet_repository.py`

```
def balance_at_least(min_minor: int) → Specification[WalletEntity]:
    """Wallets whose balance is at least *min_minor*.

    Returned as a Specification, so it composes via & / | / ~ and
    runs through find_all_by_spec / find_all_by_spec_paged.
    """
    return Specification(
        lambda root, q: q.where(root.balance_minor ≥ min_minor)
    )

@Repository
class WalletRepository(Repository[WalletEntity, str]):

    async def find_rich(
        self, min_minor: int, pageable: Pageable
    ) → Page[WalletEntity]:
        """A page of wallets with balance ≥ min_minor."""
        return await self.find_all_by_spec_paged(
            balance_at_least(min_minor), pageable
        )
```

Listado 5.6 — Una factoría de `Specification` y un método que la ejecuta paginada

Una `Specification` envuelve un invocable $(\text{root}, q) \rightarrow q$: dada la clase de entidad (`root`) y un `Select` de SQLAlchemy, devuelve la sentencia con un predicado añadido. `balance_at_least(1000)` produce el predicado `balance_minor  $\geq$  1000`. Como las especificaciones se componen con operadores de Python, puedes construir filtros arbitrariamente complejos a partir de piezas pequeñas:

```
rich = balance_at_least(1000)
in_eur = Specification(
    lambda root, q: q.where(root.currency == "EUR")
)
rich_eur = rich & in_eur           # AND
rich_or_eur = rich | in_eur       # OR
not_rich = ~rich                  # NOT
```

Una especificación se ejecuta de dos maneras. `find_all_by_spec(spec)` devuelve como lista todas las filas coincidentes; `find_all_by_spec_paged(spec, pageable)` aplica el predicado, cuenta las coincidencias, ordena y corta, devolviendo un `Page[T]`. `find_rich` usa la forma paginada, así que el endpoint de monederos ricos está él mismo paginado. El manejador refleja exactamente el manejador de listado, mapeando las filas a DTOs:

lumen/core/services/wallets/list_rich_wallets_handler.py

```

@query_handler
@service
class ListRichWalletsHandler(
    QueryHandler[ListRichWallets, Page[WalletDto]]
):
    def __init__(self, repository: WalletRepository) → None:
        super().__init__()
        self._repository = repository

    async def do_handle( # type: ignore[override]
        self, query: ListRichWallets
    ) → Page[WalletDto]:
        page = await self._repository.find_rich(
            query.min_minor, query.pageable
        )
        return page.map(entity_to_dto)

```

Listado 5.7 — El manejador de monederos ricos ejecuta la ruta de Specification

GET /api/v1/wallets/rich?min_minor=1000&page=1&size=20 devuelve ahora una página de monederos con 10,00 € o más, de más reciente primero.

❗ EJECÚTALO: FILTRA A LOS MONEDEROS RICOS

Abre un monedero e ingresa 25,00 € en él (2500 unidades menores), abre otro y déjalo vacío, y luego pide los monederos con al menos 10,00 €:

```
curl -s 'localhost:8080/api/v1/wallets/rich?
min_minor=1000&page=1&size=20'
```

Solo vuelve el monedero con fondos, y `total` cuenta únicamente las coincidencias: el monedero vacío queda filtrado por la `Specification`:

```
{"items": [{"id": "wlt-...", "balance_minor": 2500, "balance":
25.0, ...}],
"total": 1, "page": 1, "total_pages": 1, "has_next": false}
```

El mismo predicado (`balance_at_least`) que se ejecuta aquí puede combinarse con otros usando `&`, `|` y `~`: esa es la recompensa de escribir el filtro como una `Specification` en lugar de como una consulta puntual.

❗ FILTROS SIN LAMBDA

Para el caso común —igualdad y un puñado de comparaciones— ni siquiera necesitas escribir una lambda. `FilterOperator.gte("balance_minor", 1000) & FilterOperator.eq("currency", "EUR")` produce la misma `Specification` componible a partir de métodos factoría estáticos, y `FilterUtils.by(currency="EUR")` construye una a partir de argumentos por palabra clave (consulta por ejemplo, Query-by-Example). Lumen usa aquí una lambda explícita porque la intención se lee con claridad; ambos estilos producen una `Specification` que puedes pasar a los mismos métodos del repositorio.

Proyecciones: lee solo las columnas que necesitas

El endpoint de saldo no necesita la fila entera, solo el id, la moneda y un saldo calculado. PyFly admite **proyecciones por interfaz**, la idea de Spring Data de declarar el subconjunto de campos que quiere una vista de lectura y dejar que el framework copie exactamente esos. (Una *proyección* es una vista de solo lectura sobre un subconjunto de las columnas de una entidad: nombras los pocos campos que te importan, y el framework copia únicamente esos, dejando sin leer el resto de la fila.)

Construir una requiere tres piezas: la clase de proyección, el mapeador que sabe cómo rellenarla y el manejador que la usa. Las veremos por orden.

Paso 1 — declara la proyección. Una proyección es una clase marcada con `@projection`. En Lumen es una dataclass concreta:

```
lumen/interfaces/dtos/v1/balance_dto.py
```

```
from dataclasses import dataclass

from pyfly.data import projection

@projection
@dataclass
class BalanceView:
    """Projection: just the fields the balance view needs.

    id, currency and balance_minor are copied straight from the
    WalletEntity; balance is a computed major-unit decimal supplied
    by a registered transform on the mapper.
    """

    id: str
    currency: str
    balance_minor: int
    balance: float
```

Listado 5.8 — BalanceView: una @projection solo de los campos del saldo

Paso 2 — registra la proyección en un Mapper. Un `Mapper` es el ayudante del framework que copia los campos de la entidad en una proyección. Lee esos cuatro campos de un `WalletEntity` y construye la vista. Tres (`id`, `currency`, `balance_minor`) se copian tal cual; el cuarto (`balance`, el decimal en unidad mayor) lo suministra un *transform*: una pequeña función registrada contra un nombre de campo que calcula un valor que la entidad no almacena directamente:

lumen/core/mappers/wallet_mapper.py

```

from pyfly.data import Mapper

_mapper = Mapper()
_mapper.register_projection(
    WalletEntity,
    BalanceView,
    transforms={"balance": lambda e: round(e.balance_minor / 100, 2)},
)

def entity_to_balance_dto(entity: WalletEntity) → BalanceDto:
    """Project a row onto the balance DTO via the projection."""
    view = _mapper.project(entity, BalanceView)
    return BalanceDto(
        id=view.id,
        currency=Currency(view.currency),
        balance_minor=view.balance_minor,
        balance=view.balance,
    )

```

Listado 5.9 — Registrando y ejecutando la proyección mediante Mapper

Paso 3 — usa la proyección desde un manejador de lectura.

`Mapper.project(entity, BalanceView)` lee solo los campos declarados, aplica el transform de `balance` y devuelve un `BalanceView`. El manejador de consulta carga entonces la fila por id y la proyecta:

lumen/core/services/wallets/get_balance_handler.py

```

@query_handler
@service
class GetBalanceHandler(QueryHandler[GetBalance, BalanceDto | None]):
    def __init__(self, repository: WalletRepository) → None:
        super().__init__()
        self._repository = repository

    async def do_handle( # type: ignore[override]
        self, query: GetBalance
    ) → BalanceDto | None:
        entity = await self._repository.find_by_id(query.wallet_id)
        return (
            entity_to_balance_dto(entity)
            if entity is not None
            else None
        )

```

Listado 5.10 — El manejador de lectura del saldo: busca por id y luego proyecta

EJECÚTALO: LEE SOLO EL SALDO

El endpoint de saldo devuelve únicamente los cuatro campos proyectados, no la fila entera. Contra una aplicación en ejecución con un monedero con fondos:

```
curl -s localhost:8080/api/v1/wallets/wlt-.../balance
```

```
{
  "id": "wlt-...",
  "currency": "EUR",
  "balance_minor": 2500,
  "balance": 25.0
}
```

Sin `owner_id`, sin `created_at`: la proyección declaró cuatro campos, así que se leen y se devuelven cuatro campos. `balance` (el `25.0` en unidad mayor) es el transform calculado; el resto se copia directamente de la fila.

⚠ UNA PROYECCIÓN DEBE PODER INSTANCIARSE

Spring permite que una proyección sea una interfaz pelada y devuelve un proxy en tiempo de ejecución. Python no tiene tal proxy, así que una proyección de PyFly debe ser un tipo **concreto** que el mapeador pueda construir: aquí, una `@dataclass`. Marcar un *Protocol* como `@projection` no funcionará: un Protocol no puede instanciarse, y `Mapper.project` no tiene nada que construir. Usa una dataclass (o cualquier clase plana con campos coincidentes) y estarás a salvo.

Transacciones y la junta del agregado

La superficie del repositorio es limpia, pero tres sutilezas honestas deciden qué hacen tus escrituras: qué escrituras se confirman juntas, si una escritura inserta o actualiza, y qué pasa cuando dos peticiones cambian el mismo monedero a la vez. Lumen maneja las tres de forma deliberada.

Una unidad de trabajo por comando

Esto es lo más importante que hay que entender sobre la capa de datos. Hay dos verbos de base de datos fáciles de confundir. **Volcar** (flush) es enviar el SQL pendiente (el `INSERT / UPDATE`) a la base de datos para que sea visible a las lecturas posteriores de *esta* conexión, pero todavía dentro de una transacción abierta que puede deshacerse. **Confirmar** (commit) es hacer esos cambios permanentes y visibles para todos. Un flush sin commit se revierte cuando termina su transacción.

Cada llamada al repositorio se ejecuta dentro de una **unidad de trabajo**. (Una *unidad de trabajo* es un lote de cambios de todo o nada: o bien cada escritura que contiene se confirma junta, o bien —si algo falla— ninguna lo hace.) El repositorio no guarda una sesión propia: cada llamada busca la unidad ligada a la tarea en curso, y de dónde sale esa unidad decide qué se confirma:

- **Fuera de una transacción**, la llamada abre una unidad breve propia. Una lectura (`find*`, `count*`, `exists*`, ...) se ejecuta y termina; cualquier otro método — `save`, `upsert`, `delete` — vuelca y **confirma** antes de devolver. Una llamada, una confirmación.
- **Dentro de `@transactional()`**, cada llamada se une a la unidad del método: `save` y `upsert` solo vuelcan, y la unidad confirma una sola vez, cuando el método devuelve, o revierte todas las escrituras cuando lanza una excepción.

Un manejador de comandos es más de una llamada. El ingreso de Lumen carga el monedero, deja que el agregado compruebe el importe, guarda el nuevo saldo y publica los eventos; como confirmaciones separadas, un fallo a medio camino dejaría la mitad hecha. Por eso un manejador que escribe decora su `do_handle` con `@transactional()`. Los manejadores de Lumen además inyectan el `async_sessionmaker` —la factoría que entrega sesiones— como `self._session_factory`, que indica el origen de datos en el que se ejecuta la unidad (sin ese atributo, la unidad se ejecuta en el origen de datos por defecto de la aplicación, que en Lumen es la misma base de datos):

lumen/core/services/wallets/open_wallet_handler.py

```

@command_handler
@service
class OpenWalletHandler(CommandHandler[OpenWallet, str]):
    """Open a new, empty wallet."""

    def __init__(
        self,
        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
    ) → None:
        super().__init__()
        self._repository = repository
        self._events = events
        # @transactional resolves the unit-of-work session from here.
        self._session_factory = session_factory

    @transactional()
    async def do_handle( # type: ignore[override]
        self, command: OpenWallet
    ) → str:
        wallet_id = f"wlt-{uuid4()}"
        wallet = Wallet.open(
            wallet_id=wallet_id,
            owner_id=command.owner_id,
            currency=command.currency,
        )
        await self._repository.upsert(to_entity(wallet))

        await publish_domain_events(
            self._events, wallet.clear_events()
        )

        return wallet_id

```

Listado 5.11 — Un manejador de escritura: @transactional() confirma la unidad de trabajo

`@transactional()` (importado de `pyfly.data.relational.sqlalchemy`) liga una nueva unidad de trabajo a la tarea en curso, en el origen de datos que hay detrás de `self._session_factory`; cada llamada al repositorio dentro del método se une a ella, y la unidad **confirma si tiene éxito** y revierte si falla. Quita el decorador y el monedero se sigue guardando —el `upsert` confirma en una unidad propia—, pero ya no forma un único paso de todo o nada con el resto del manejador. Los manejadores de lectura anteriores de este capítulo no necesitan `@transactional`: una lectura suelta se ejecuta en una unidad de lectura breve propia.

⚠ CAMBIÓ EN V26.09.08

Hasta la v26.09.07 el repositorio compartía una sola sesión inyectada, y una llamada hecha fuera de `@transactional` solo volcaba: nadie la confirmaba, y la escritura desaparecía al parar la aplicación. Desde la v26.09.08 una llamada así confirma en su propia unidad. El código que contaba con que esas escrituras se descartaran, o con confirmar más tarde a mano una sesión compartida, debe envolver las llamadas en `@transactional` para que formen una sola unidad.

❗ QUÉ ACABA DE PASAR

La regla en una línea: **las lecturas no necesitan nada; una escritura que abarca varias llamadas necesita `@transactional()`**. Una llamada suelta al repositorio confirma por sí sola; el decorador convierte la carga, las comprobaciones, el guardado y los eventos en una sola unidad que se confirma —o se revierte— de una vez. Hace tres cosas por ti: abre la transacción, deja que cada llamada al repositorio dentro del método se una a ella y confirma al final (o revierte ante una excepción).

upsert, no save, para un agregado que es dueño de su id

Fíjate en que el manejador llama a `self._repository.upsert(...)`, no a `save(...)`. (*Upsert* es el verbo combinado para "insertar si es nuevo, actualizar si ya está presente": una sola llamada para ambos casos.) Esa es la segunda sutileza. El

agregado `Wallet` genera su *propia* clave primaria por adelantado (`wlt-...`), así que para cuando un ingreso o una retirada persisten un monedero ya cargado, ya existe una fila con ese id, y la escritura debe actualizarla en lugar de insertar una segunda fila con la misma clave.

Lumen deja clara esa intención con un método `upsert` construido sobre `session.merge`, que inserta cuando el id es nuevo y actualiza cuando ya existe:

`lumen/models/repositories/wallet_repository.py`

```
@repository
class WalletRepository(Repository[WalletEntity, str]):

    async def upsert(self, entity: WalletEntity) → WalletEntity:
        """Insert *entity* or update the existing row with the same id.

        Uses session.merge so a freshly-mapped entity carrying the
        aggregate's id persists whether or not a row already exists –
        the aggregate owns its primary key, so identity is never
        ambiguous. Flushes so the write is visible in the current
        unit of work; the surrounding @transactional commits it
        (outside one, the call's own unit does).
        """
        session = self._require_session()
        merged = await session.merge(entity)
        await session.flush()
        return merged
```

Listado 5.12 — `upsert`: una sola llamada tanto para `INSERT` como para `UPDATE`

`_require_session()` es el accesor heredado que devuelve la sesión de la unidad en la que se ejecuta la llamada: la unidad del manejador dentro de `@transactional`, o la unidad breve de la propia llamada fuera de él. `merge` se basa en la clave primaria, así que tanto la primera escritura (apertura) como toda escritura posterior (ingreso, retirada) toman la misma ruta de código. Desde la v26.09.08 el `save()` heredado

sigue el `save` de Spring Data y haría lo mismo: una entidad nueva se inserta, y una cuya clave ya está fijada se fusiona con su fila, así que un segundo guardado del mismo id la actualiza (usa la instancia que devuelve `save()`). Hasta la v26.09.07 `save()` emitía siempre un `INSERT`, y un segundo guardado de la misma clave lanzaba `IntegrityError`: por eso Lumen tiene `upsert`. Se queda como la forma explícita de expresar la intención.

❗ QUÉ ACABA DE PASAR

Dos preguntas deciden cada escritura: *¿se confirmó esta fila?* y *¿esta escritura insertó o actualizó?* `@transactional()` responde a la primera (confirma la unidad de trabajo); `upsert / merge` responde a la segunda (una sola ruta de código tanto para INSERT como para UPDATE, porque el agregado es dueño de su id). Acierta en ambas y un monedero que abres, ingresas en él y luego lees de vuelta tras un reinicio devuelve el saldo correcto, que es exactamente lo que afirma el test del repositorio de más abajo contra un motor *recién creado*.

Dos retiradas a la vez

La tercera sutileza es la concurrencia. El agregado `Wallet` rechaza un descubierto, pero solo del saldo con el que se cargó. Dos retiradas de 60 de un monedero con 100 pueden cargar 100 cada una, pasar cada una la comprobación y guardar 40 cada una: las dos tienen éxito, y los mismos fondos se gastan dos veces. Una unidad de trabajo por sí sola no lo impide; con el nivel de aislamiento por defecto las dos unidades confirman. Lumen cierra el hueco con un **bloqueo pesimista**: la retirada (y el ingreso) leen la fila con `LockMode.PESSIMISTIC_WRITE` dentro de su unidad `@transactional`:

lumen/core/services/wallets/withdraw_funds_handler.py

```

from pyfly.data.relational.sqlalchemy import LockMode, transactional

@command_handler
@service
class WithdrawFundsHandler(CommandHandler[WithdrawFunds, int]):

    @transactional()
    async def do_handle( # type: ignore[override]
        self, command: WithdrawFunds
    ) → int:
        # Lock the row until the unit ends: a concurrent withdrawal
        # waits here.
        entity = await self._repository.find_by_id(
            command.wallet_id, lock=LockMode.PESSIMISTIC_WRITE
        )
        if entity is None:
            raise AggregateNotFound("Wallet", command.wallet_id)

        wallet = to_aggregate(entity)
        wallet.withdraw(
            Money(amount=command.amount, currency=wallet.currency)
        )
        await self._repository.upsert(to_entity(wallet))

        await publish_domain_events(
            self._events, wallet.clear_events()
        )
        return wallet.balance.amount

```

Listado 5.12a — Bloquea la fila y deja que el agregado compruebe el saldo

En PostgreSQL, MySQL y MariaDB la lectura es `SELECT ... FOR UPDATE`: una segunda retirada del mismo monedero espera en su lectura hasta que termina la primera unidad, y entonces ve el saldo que dejó esa unidad. SQLite no tiene bloqueos de fila;

allí una unidad de escritura retiene el único bloqueo de escritura de la base de datos desde su `BEGIN IMMEDIATE`, con el mismo efecto. Un bloqueo necesita una transacción de lectura-escritura: fuera de `@transactional` (o dentro de `@transactional(read_only=True)`) `find_by_id(..., lock=...)` lanza `IllegalTransactionStateError`.

Hay otras dos formas de mantener un invariante bajo concurrencia. Una **actualización con guarda** comprueba y cambia en una sola sentencia

(`UPDATE ... SET balance_minor = balance_minor - :amount WHERE id = :id AND balance_minor ≥ :amount`) y no necesita transacción a su alrededor; la usa la saga de transferencias de Lumen (Capítulo 12). El **bloqueo optimista** (`VersionedMixin`)

añade una columna de versión y hace fallar la confirmación perdedora con

`OptimisticLockingFailureException` (HTTP 409). Todas las rutas que escriben la fila deben usar el mismo enfoque: un escritor que se salta el bloqueo aún puede sobrescribirla.

EJECÚTALO: DOS RETIRADAS EN CARRERA

El ejemplo incluye un test que arranca la aplicación y lanza en carrera dos retiradas de 60 de un monedero con 100, a través del bus de comandos y a través de la saga de transferencias. Desde la raíz del proyecto Lumen:

```
uv run --extra dev pytest tests/test_concurrent_balance_changes.py -q
```

```
.s.s [100%]
2 passed, 2 skipped in 0.30s
```

Cada carrera deja pasar exactamente una retirada y un saldo de 40. Los dos casos omitidos son las mismas carreras sobre PostgreSQL; se ejecutan cuando

`LUMEN_TEST_POSTGRES_URL` indica un servidor en el que los tests pueden crear bases de datos.

La junta mapeadora agregado ↔ entidad

Hay un límite más, y es una característica, no un accidente. Lumen mantiene dos tipos distintos:

- **Wallet** — la *raíz de agregado* de DDD del Capítulo 6. Es dueña del invariante `balance ≥ 0`, expone métodos que revelan la intención (`open`, `deposit`, `withdraw`) y lanza eventos de dominio. No sabe nada de SQLAlchemy.
- **WalletEntity** — la *fila de persistencia*. Es un modelo plano de SQLAlchemy con columnas y sin comportamiento.

Un pequeño mapeador los une, una función pura en cada sentido:

lumen/core/mappers/wallet_mapper.py

```

def to_entity(wallet: Wallet) → WalletEntity:
    """Flatten a Wallet aggregate into a persistable row."""
    assert wallet.id is not None
    return WalletEntity(
        id=wallet.id,
        owner_id=wallet.owner_id,
        currency=wallet.currency.value,
        balance_minor=wallet.balance.amount,
        created_at=wallet.created_at,
    )

def to_aggregate(entity: WalletEntity) → Wallet:
    """Rehydrate a Wallet aggregate from a persistence row."""
    currency = Currency(entity.currency)
    return Wallet(
        id=entity.id,
        owner_id=entity.owner_id,
        balance=Money(amount=entity.balance_minor, currency=currency),
        created_at=entity.created_at,
    )

```

Listado 5.13 — El mapeador agregado ↔ fila

El lado de escritura llama a `to_entity` antes del `upsert`; el lado de lectura o bien rehidrata con `to_aggregate` (cuando un comando necesita el agregado rico) o bien proyecta directamente a un DTO (cuando una consulta solo necesita datos). Mantener la fila separada del agregado significa que las preocupaciones de persistencia —tipos de columna, nulabilidad, el baile del merge— nunca se filtran al modelo de dominio, y que los invariantes del dominio nunca limitan el esquema de la tabla. El repositorio almacena filas; el mapeador es la junta que mantiene puro el agregado.

i LA REHIDRATACIÓN SE SALTA LA FACTORÍA

`to_aggregate` llama al **constructor** de `Wallet` directamente, nunca a la factoría `Wallet.open`. La factoría es para monederos *nuevos*: valida las entradas y lanza un evento `WalletOpened`. Una fila cargada desde la base de datos ya representa un monedero válido y confirmado: volver a ejecutar la factoría volvería a disparar ese evento y a comprobar reglas que pasaron hace mucho. El constructor fija los campos en silencio, produciendo un `Wallet` indistinguible de uno recién abierto pero sin eventos espurios.

Activarlo

Activar la capa relacional es configuración, no código: tres claves en `pyfly.yaml`.

Añade un bloque `data.relational`:

`pyfly.yaml`

```
pyfly:
  data:
    relational:
      enabled: true
      url: "sqlite+aiosqlite:///./lumen.db"
      ddl-auto: create
```

Listado 5.14 — Configuración de la capa de datos relacional

`enabled: true` activa la autoconfiguración relacional, que construye el motor asíncrono de SQLAlchemy y el `async_sessionmaker`, registra el `async_sessionmaker` que inyectan los manejadores, e instala el `RepositoryBeanPostProcessor` que compila tus esbozos de consulta derivada. `url` es una cadena de conexión estándar de SQLAlchemy: SQLite vía `aiosqlite` aquí para un desarrollo de cero infraestructura, `postgresql+asyncpg://...` en producción. `ddl-auto: create` ejecuta

`Base.metadata.create_all` al arrancar, así que la tabla `wallets` (descubierta porque `WalletEntity` hereda de `Base`) se construye automáticamente la primera vez que arranca la aplicación. En SQLite `create` es además el valor por defecto; Lumen lo escribe explícitamente.

La huella de dependencias es minúscula: `pyfly[data-relational]` arrastra `sqlalchemy[asyncio]`, `alembic` y `aiosqlite`, y nada más. Sin servidor de base de datos, sin instalación de drivers, que es exactamente por lo que el ejemplo se ejecuta en cualquier sitio.

❗ EJECÚTALO: EL MONEDERO QUE DESAPARECE, ARREGLADO

Vuelve a ejecutar el experimento de la introducción, ahora con la persistencia activada. Abre un monedero, detén la aplicación, arráncala de nuevo y lee el saldo de vuelta:

```
# Terminal 1
uv run pyfly run --server uvicorn

# Terminal 2 - open a wallet
curl -s -X POST localhost:8080/api/v1/wallets \
  -H 'content-type: application/json' \
  -d '{"owner_id": "alice", "currency": "EUR"}'
# → {"wallet_id": "wlt-..."}

# Ctrl+C in Terminal 1, then start it again, then:
curl -s localhost:8080/api/v1/wallets/wlt-.../balance
```

Esta vez el monedero sobrevive: su fila se confirmó en `lumen.db` en disco:

```
{"id": "wlt-...", "currency": "EUR", "balance_minor": 0, "balance": 0.0}
```

Mira en el directorio del proyecto y verás el fichero SQLite `lumen.db` que el motor creó en el primer arranque, con la tabla `wallets` dentro. Todo el capítulo se reduce a esto: el monedero sobrevive al proceso.

🔗 CICLO DE VIDA DEL ESQUEMA EN PRODUCCIÓN

`ddl-auto: create` es lo correcto para desarrollo y ejemplos: crea las tablas que faltan y deja en paz las existentes (una columna nueva necesita una migración). Si no se fija, `ddl-auto` vale `create` solo para una base de datos embebida como SQLite y `none` en un servidor de base de datos, como en Spring Boot, así que apuntar `url` a PostgreSQL no crea el esquema por ti. En producción gestiónalo con migraciones de Alembic (`pyfly db migrate` y `pyfly db upgrade` , o `pyfly.data.relational.migrations.enabled: true` para aplicarlas al arrancar), y fija `ddl-auto: validate` para que el arranque falle cuando falte una tabla o una columna que necesita un modelo. `update` no es una estrategia: con él, o con cualquier valor desconocido, el arranque falla nombrando la clave, y lo mismo ocurre con `create` junto a las migraciones de arranque. El código de la aplicación no cambia: solo el ajuste `ddl-auto` y la canalización de migración.

Demostrar que funciona

Como el repositorio es una clase ordinaria, puedes probarlo directamente contra un fichero SQLite real, sin contexto de aplicación, sin HTTP. El test del repositorio de Lumen crea una base de datos temporal, ejecuta `Base.metadata.create_all` y ejercita la superficie de principio a fin, incluido el `RepositoryBeanPostProcessor` que compila la consulta derivada (el mismo procesador que ejecuta el `ApplicationContext` en vivo):

```
lumen/tests/test_sql_wallet_repository.py
```

```
def _make_repo(session: AsyncSession) → WalletRepository:
    repo = WalletRepository(WalletEntity, session)
    # Mirror the ApplicationContext: compile derived-query stubs.
    RepositoryBeanPostProcessor().after_init(repo, "walletRepository")
    return repo
```

```
@pytest.mark.asyncio
```

```
async def test_derived_find_by_owner_id(sqlite_factory) → None:
```

```
    factory, _ = sqlite_factory
```

```
    async with factory() as session:
```

```
        repo = _make_repo(session)
```

```
        await repo.upsert(_entity("wlt-1", "alice", 100))
```

```
        await repo.upsert(_entity("wlt-2", "alice", 200))
```

```
        await repo.upsert(_entity("wlt-3", "bob", 300))
```

```
        await session.commit()
```

```
        owned = await repo.find_by_owner_id("alice")
```

```
        assert sorted(w.id for w in owned) == ["wlt-1", "wlt-2"]
```

```
        assert await repo.find_by_owner_id("nobody") == []
```

```
@pytest.mark.asyncio
```

```
async def test_specification_find_rich_paged_and_sorted(
```

```
    sqlite_factory,
```

```
) → None:
```

```
    factory, _ = sqlite_factory
```

```
    async with factory() as session:
```

```
        repo = _make_repo(session)
```

```
        # age_days drives created_at for newest-first ordering.
```

```
        await repo.upsert(_entity("wlt-poor", "a", 50, age_days=3))
```

```
        await repo.upsert(_entity("wlt-mid", "b", 1000, age_days=2))
```

```
        await repo.upsert(_entity("wlt-rich", "c", 5000, age_days=1))
```

```
        await session.commit()
```

```
        # balance_minor ≥ 1000, newest first, page size 1.
```

```
        newest_first = Sort.by("created_at").descending()
```

```

page = await repo.find_rich(1000, Pageable.of(1, 1, newest_first))
assert page.total == 2 # mid + rich
assert page.total_pages == 2
assert page.has_next is True
assert [w.id for w in page.items] == ["wlt-rich"]

# The bare predicate also works through find_all_by_spec.
rich = await repo.find_all_by_spec(balance_at_least(5000))
assert [w.id for w in rich] == ["wlt-rich"]

```

Listado 5.15 — Probando el CRUD, la consulta derivada y la ruta de Specification

El primer test ejercita la consulta derivada: tres monederos de entrada, dos propietarios de salida, y `find_by_owner_id("alice")` devuelve exactamente los dos: prueba de que el framework compiló `WHERE owner_id = :owner_id` a partir del nombre del método. El segundo ejercita la ruta de `Specification`: afirma el filtro de umbral (`total == 2`, solo mid y rich cumplen ≥ 1000), la ordenación de más reciente primero (`wlt-rich` es el más reciente de los dos), los metadatos de la página (`total_pages == 2`, `has_next`) y que el mismo predicado `balance_at_least` también se ejecuta sin paginar a través de `find_all_by_spec`.

El fixture refleja lo que hace el framework al arrancar —construir el motor, ejecutar `Base.metadata.create_all` dentro de un bloque `begin()` para que el DDL se confirme, devolver una factoría de sesiones—, de modo que el test ejercita la misma tabla exacta que crea la aplicación. Los tests entregan al repositorio una sesión propia (su *modo manual*): la usa tal cual y el test la confirma; la aplicación arrancada construye el repositorio sin ella, como se describió antes. Otros tests del mismo fichero prueban que `upsert` hace un ida y vuelta a través de un motor *recién creado* (durabilidad a través de una reconexión) y que `find_all(pageable)` cuenta y corta correctamente una tabla de cinco monederos.

❗ EJECÚTALO: DEMUESTRA TODA LA CAPA EN VERDE

Ejecuta el fichero de test del repositorio de principio a fin. Desde la raíz del proyecto Lumen:

```
uv run --extra dev pytest tests/test_sql_wallet_repository.py -v
```

Cada test nombrado informa **PASSED** : ida y vuelta del CRUD, durabilidad a través de la reconexión, la consulta derivada, la ruta de especificación y la paginación:

```
tests/
test_sql_wallet_repository.py::test_upsert_inserts_then_updates_and_persists PASSED
tests/test_sql_wallet_repository.py::test_find_by_id_unknown_returns_none PASSED
tests/test_sql_wallet_repository.py::test_derived_find_by_owner_id PASSED
tests/
test_sql_wallet_repository.py::test_specification_find_rich_paged_and_sorted PASSED
tests/
test_sql_wallet_repository.py::test_find_all_pageable_counts_and_pages PASSED
5 passed in 0.31s
```

Ejecuta toda la batería (`uv run --extra dev pytest -q`) para confirmar que el resto de Lumen sigue pasando junto a la capa de persistencia.

🍃 EQUIVALENCIA CON SPRING

Construir el repositorio directamente contra una base de datos real en el mismo proceso refleja el slice `@DataJpaTest` de Spring, que arranca una base de datos H2 y la capa JPA de forma aislada para probar repositorios sin el contexto completo.

`Base.metadata.create_all` es el análogo de `spring.jpa.hibernate.ddl-auto=create`, y ejecutar `RepositoryBeanPostProcessor` a mano hace las veces del proxy de Spring que materializa las consultas derivadas sobre un `JpaRepository` al arrancar. PyFly tiene además un slice `@DataTest` cuyos tests revierten todas sus unidades de trabajo (Capítulo 16).

✓ Lo que construiste

Lumen ahora persiste los monederos a través de la capa de repositorios al estilo de Spring Data de PyFly:

- **Entidad** — `WalletEntity(Base)`, una fila de SQLAlchemy 2.0 con una clave primaria de cadena (el propio id del agregado) y saldos en unidades menores enteras.
- **Repositorio** — `WalletRepository(Repository[WalletEntity, str])`, marcado con `@repository`. El framework suministra el CRUD asíncrono completo (`save`, `find_by_id`, `find_all`, `delete` / `delete_by_id`, `delete_all` / `delete_all_by_id`, `count`, `exists_by_id`, `save_all`, `find_all_by_id`, `stream_all`, paginación, especificaciones) sin ningún adaptador escrito a mano.
- **Consulta derivada** — `find_by_owner_id`, declarada como un esbozo `...` y compilada a partir de su nombre por el `RepositoryBeanPostProcessor`.
- **Paginación** — `find_all(pageable)` devolviendo un `Page[T]` con `total` / `total_pages` / `has_next`, mapeado a DTOs con `Page.map`, expuesto en `GET / api/v1/wallets`.

- **Especificación** — `balance_at_least(n)` compuesta con `&` `|` `~` y ejecutada vía `find_all_by_spec_paged`, expuesta en `GET /api/v1/wallets/rich`.
- **Proyección** — `@projection BalanceView`, una dataclass concreta sobre la que el `Mapper` proyecta las filas para la vista de lectura del saldo.
- **Transacciones** — una llamada suelta al repositorio confirma en una unidad propia; los manejadores de escritura se decoran con `@transactional()` para que la carga, las comprobaciones, el guardado y los eventos se confirmen juntos; `upsert / session.merge` persiste un agregado que es dueño de su id; `find_by_id(..., lock=LockMode.PESSIMISTIC_WRITE)` mantiene `balance ≥ 0` cuando compiten retiradas; y el mapeador agregado ↔ entidad mantiene puro el modelo de dominio.

Escribiste interfaces y esbozos; el framework escribió el SQL. Esa es la recompensa del patrón Repositorio.

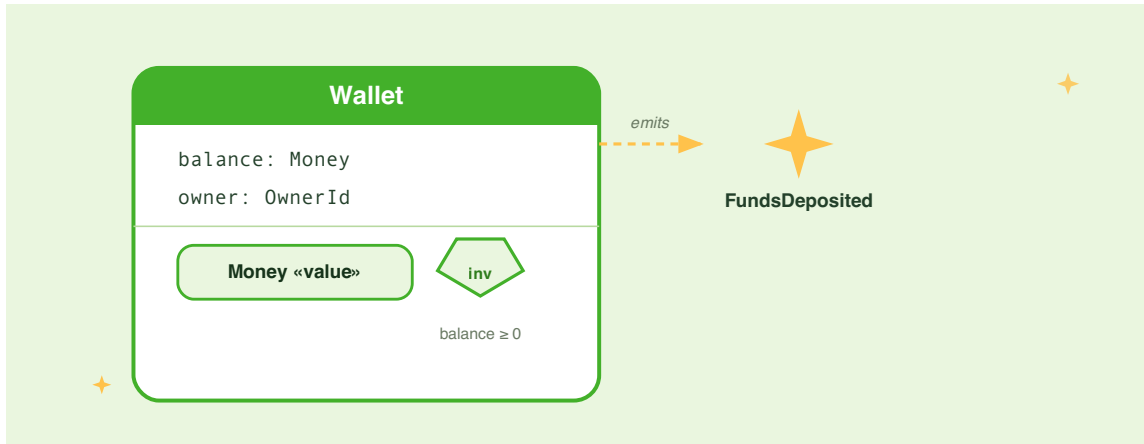
Pruébalo tú mismo

1. **Añade un contador derivado.** Declara `async def count_by_currency(self, currency: str) → int: ...` en `WalletRepository` (cuerpo `...`). Escribe un test que haga `upsert` de monederos en dos monedas y afirme el conteo de cada una, confirmando que el prefijo `count_by` compila a `SELECT COUNT(*) ... WHERE currency = :currency` sin ningún SQL por tu parte.
2. **Compón dos especificaciones.** Define una segunda factoría `in_currency(code: str) → Specification[WalletEntity]` (predicado `currency = code`), y luego añade un método de repositorio que ejecute `balance_at_least(min_minor) & in_currency(code)` a través de `find_all_by_spec_paged`. Prueba que devuelve solo los monederos ricos en la moneda elegida, de más reciente primero.

3. **Sigue el rastro del límite transaccional.** En un test que arranque la aplicación (como hace `tests/test_app_context_integration.py`), haz que `WithdrawFundsHandler.do_handle` lance una excepción justo después del `upsert`, envía una retirada y afirma que el saldo no se movió: la unidad revirtió la escritura. Luego quita el decorador `@transactional()` (y el argumento `lock=`, que necesita una transacción) y repite: el `upsert` confirma ahora en su propia unidad antes del fallo, y el saldo se mueve. Esa es la diferencia entre una unidad de trabajo y una confirmación por llamada.
4. **Proyecta una vista diferente.** Añade una dataclass `@projection OwnerView` con solo `id` y `owner_id`, regístrala en un `Mapper` y escribe un test sin manejador que cargue un `WalletEntity` y lo proyecte, verificando que solo se leen las dos columnas declaradas y que se ignora el resto de la fila.

CAPÍTULO 6

Diseño guiado por el dominio



La funcionalidad del monedero de Lumen funciona. Los depósitos llegan, los saldos se actualizan y la base de datos lo persiste todo entre reinicios. Pero fíjate bien en la capa de servicio y notarás algo incómodo: la comprobación de descubierto vive en el método del servicio, no en el propio monedero. La validación de divisa es una comparación de propiedades repartida entre un puñado de sentencias `if`. Nada impide que un futuro desarrollador —o tu yo futuro a las 11 de la noche— se salte esas defensas llamando directamente a `repo.save(entity)`.

El **diseño guiado por el dominio** (Domain-Driven Design) resuelve esto haciendo que el modelo sea responsable de sus propias reglas. Los datos dejan de ser una bolsa pasiva de valores que cualquier invocador puede mutar; se convierten en un objeto con criterio propio: uno que hace cumplir sus invariantes, anuncia lo que ha ocurrido y coopera con la capa de persistencia mientras permanece libre de cualquier importación de base de datos.

Este capítulo asciende el monedero a un agregado DDD en condiciones: un objeto de valor `Money`, una raíz de agregado `Wallet` que protege la regla de descubierto y la regla de coincidencia de divisa, y un conjunto de `DomainEvent`s que se emiten cada vez que el monedero cambia de estado. Un mapeador ligero convierte entre el rico modelo de dominio y el registro plano de persistencia; ninguno de los dos lados necesita conocer la forma del otro.

Entidades y objetos de valor

Antes de poder construir un modelo que haga cumplir sus propias reglas, necesitas un vocabulario para los dos tipos fundamentalmente distintos de objetos que aparecen en todo dominio.

Piensa en qué hace que dos monederos sean distintos. Aunque dos monederos contengan exactamente cien euros, siguen siendo monederos separados que pertenecen a propietarios separados: te importa *cuál* tienes. Ahora piensa en el importe en sí. Cien euros son cien euros; el objeto Python exacto que contiene el valor es irrelevante. Si un depósito añade cincuenta euros al saldo de un monedero, no quieres actualizar el importe existente en su sitio: quieres derivar un importe completamente nuevo que registre el resultado. Mutar en el sitio invita a errores de aliasing en los que dos partes del código comparten sin saberlo una referencia al mismo objeto y ven los cambios del otro.

LA JERGA, EN LENGUAJE LLANO

Algunas palabras se repiten a lo largo de este capítulo. Una **invariante** es una regla que siempre debe cumplirse; para un monedero, "el saldo nunca es negativo". Un **agregado** es un pequeño grupo de objetos que cambian juntos y deben mantenerse coherentes como conjunto. Un **error de aliasing** ocurre cuando dos fragmentos de código sostienen accidentalmente el *mismo* objeto y uno lo muta, sorprendiendo al otro. Ten presentes estos tres conceptos; el resto del capítulo trata en gran medida de prevenir el último y garantizar el primero dentro del segundo.

El DDD da nombre a estos dos roles: **entidades** y **objetos de valor**, y el módulo `pyfly.domain` de PyFly los convierte en conceptos de primera clase:

Concepto	Base de PyFly	Igualdad	Mutación
<code>Entity[TID]</code>	<code>Entity</code>	Identidad: iguales solo cuando coincide el <code>id</code>	Permitida mediante métodos propios
<code>ValueObject</code>	<code>ValueObject</code>	Estructural: se comparan todos los campos	Prohibida; <code>replace(**changes)</code> crea una nueva instancia

Las entidades transitorias (las que tienen `id=None`) se comparan iguales solo por la identidad de objeto de Python, así que puedes meter entidades en conjuntos y diccionarios sin preocuparte por colisiones de hash de objetos sin guardar.

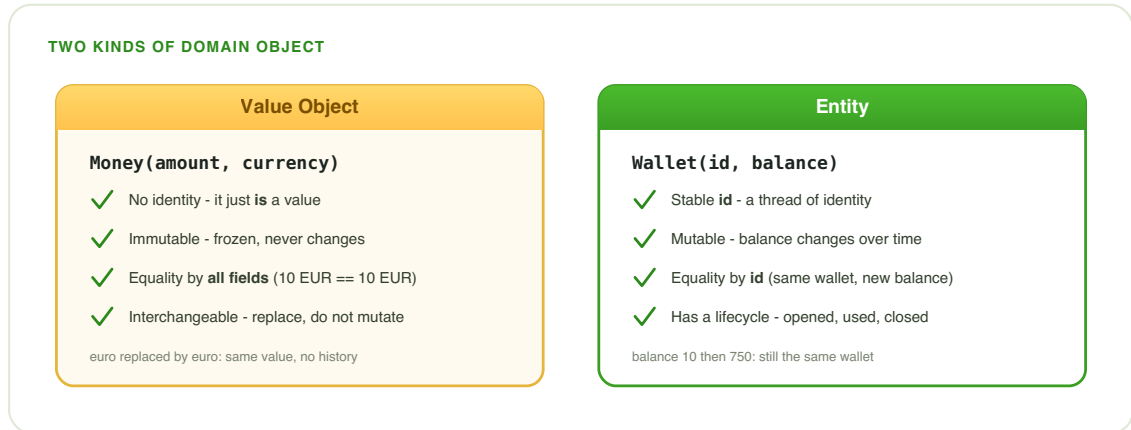


Figura 6.1 — Dos clases de objeto de dominio: un objeto de valor Money (sin identidad, inmutable, comparado por sus campos) frente a una entidad Wallet (con identidad estable, mutable, comparada por id).

El dinero es el objeto de valor de manual. Un importe de cien euros no es un objeto específico que sigues en el tiempo; es un valor. Dos instancias separadas de `Money(100, "EUR")` son iguales. Un depósito no muta el importe existente: produce uno nuevo, dejando el original intacto y el modelo libre de efectos secundarios ocultos.

Aquí está el objeto de valor `Money` para Lumen:

lumen/models/entities/v1/money.py

```

from __future__ import annotations

from dataclasses import dataclass

from lumen.interfaces.enums.v1.currency import Currency
from pyfly.domain import BusinessRuleViolation, ValueError

@dataclass(frozen=True)
class Money(ValueObject):
    """An exact monetary amount in a single currency.

    ``amount`` is in minor units (e.g. cents): ``Money(1050,
    Currency.EUR)`` is €10.50. Arithmetic returns new ``Money``
    instances and refuses to mix currencies.
    """

    amount: int
    currency: Currency

    def __post_init__(self) → None:
        if not isinstance(self.amount, int) or isinstance(self.amount, bool):
            raise BusinessRuleViolation(
                "money-amount-integer",
                "amount must be an integer number of minor units",
            )

    @classmethod
    def zero(cls, currency: Currency) → Money:
        """The additive identity for *currency* (a zero balance)."""
        return cls(amount=0, currency=currency)

    def add(self, other: Money) → Money:
        """Return ``self + other``; both must share a currency."""
        self._assert_same_currency(other)
        return Money(amount=self.amount + other.amount,
            currency=self.currency)

```

```

def subtract(self, other: Money) → Money:
    """Return `self - other`; both must share a currency."""
    self._assert_same_currency(other)
    return Money(amount=self.amount - other.amount,
currency=self.currency)

@property
def is_positive(self) → bool:
    return self.amount > 0

@property
def is_negative(self) → bool:
    return self.amount < 0

@property
def major_units(self) → float:
    """The amount rendered as a major-unit decimal (cents / 100)."""
    return round(self.amount / 100, 2)

def _assert_same_currency(self, other: Money) → None:
    if self.currency is not other.currency:
        raise BusinessRuleViolation(
            "money-currency-mismatch",
            f"cannot combine {self.currency.value} "
            f"with {other.currency.value}",
        )

def __str__(self) → str:
    return f"{self.major_units:.2f} {self.currency.value}"

```

Listado 6.1 — Money: un objeto de valor inmutable con aritmética consciente de la divisa

Cómo funciona. El importe se almacena en **unidades menores** —céntimos enteros, peniques o cualquiera que sea la denominación más pequeña de la divisa— para eliminar por completo el redondeo en coma flotante. Los cálculos financieros que usan `float` son una fuente crónica de errores de un céntimo de más o de menos que solo

afloran en producción, normalmente durante la conciliación. Almacenar 10,50 € como `amount=1050` mantiene toda la aritmética exacta. `__post_init__` rechaza de inmediato los importes que no sean enteros con `BusinessRuleViolation("money-amount-integer")`, así que un `float` perdido como `10.5` nunca entra en silencio en el modelo.

El campo `currency` usa el enum `Currency` (`Currency.EUR`, `Currency.USD`, `Currency.GBP`) en lugar de una cadena pelada, descartando erratas en el momento de la construcción. Las comparaciones de divisa dentro de `_assert_same_currency` usan la comprobación de identidad de Python (`is`) —lanzando `BusinessRuleViolation("money-currency-mismatch")` si difieren— para que el error aflore exactamente donde se cometió el fallo.

Tanto `add` como `subtract` devuelven una *nueva* instancia de `Money` en lugar de modificar `self`, consecuencia directa de `frozen=True`. Esta garantía de inmutabilidad significa que el agregado que contiene un valor `Money` nunca puede quedar parcialmente actualizado: o bien el reemplazo completo tiene éxito, o bien el valor antiguo permanece en su sitio. Las propiedades `is_positive` e `is_negative` exponen el signo sin filtrar el entero crudo. `major_units` convierte a un decimal para mostrarlo, y `__str__` da formato como `"10.50 EUR"` mediante `currency.value`.

Constrúyelo paso a paso. Si estás creando `Money` desde cero, este es el orden en que escribirlo, y por qué importa cada línea.

Paso 1 — Declara los dos campos y congela la clase. Añade `amount: int` y `currency: Currency` a una clase decorada con `@dataclass(frozen=True)` que herede de `ValueObject`. `frozen=True` es lo que hace que el objeto sea inmutable y te da la igualdad estructural de regalo: dos instancias de `Money` con el mismo importe y divisa ahora son `==`.

Paso 2 — Protege el constructor. Añade `__post_init__` para rechazar cualquier cosa que no sea un entero simple. La comprobación `isinstance(self.amount, bool)` es deliberada: en Python `True` es un `int`, y no quieres que `Money(True, ...)` se cuele.

Paso 3 — Añade la factoría `zero`. Un monedero se abre con saldo cero, así que `Money.zero(currency)` se lee mejor en el punto de llamada que `Money(0, currency)`.

Paso 4 — Añade `add` y `subtract`, cada uno pasando primero por `_assert_same_currency`. Devolver un `Money` completamente nuevo (sin mutar nunca `self`) es lo que previene el error de aliasing de la apertura de la sección.

Paso 5 — Añade los ayudantes de visualización: `is_positive`, `is_negative`, `major_units` y `__str__`.

Ejecútalo. `Money` no depende del runtime del framework ni de una base de datos, así que puedes ejercitar todas las reglas desde un prompt de Python. Arranca uno con `uv run python` desde el directorio `samples/lumen` y escribe:


```
>>> from lumen.models.entities.v1.money import Money
>>> from lumen.interfaces.enums.v1.currency import Currency
>>> ten_fifty = Money(1050, Currency.EUR)
>>> str(ten_fifty)
'10.50 EUR'
>>> ten_fifty.add(Money(450, Currency.EUR))
Money(amount=1500, currency=<Currency.EUR: 'EUR'>)
>>> ten_fifty.add(Money(100, Currency.USD))
Traceback (most recent call last):
...
pyfly.domain.exceptions.BusinessRuleViolation: cannot combine EUR with USD
>>> Money(10.5, Currency.EUR)
Traceback (most recent call last):
...
pyfly.domain.exceptions.BusinessRuleViolation: amount must be an integer number of minor units
```

Las dos trazas son la clave: el modelo rechaza una suma entre divisas y un importe en coma flotante *en el momento en que cometes el fallo*, no tres capas más abajo durante la conciliación.

Lumen incluye estos comportamientos exactos como pruebas. Ejecútalas con:

```
uv run --extra dev pytest tests/test_money.py -q
```

y deberías ver pasar las seis:

```
..... [100%]
6 passed in 0.00s
```

Qué acaba de pasar. Has construido un objeto de valor imposible de usar mal: no se puede mutar, no puede mezclar divisas y no puede contener un `float`. Cualquier otra pieza del modelo del monedero se apoyará en estas garantías, por lo cual `Money` va primero.

i UNIDADES MENORES FRENTE A DECIMAL

Almacenar el dinero como céntimos enteros es una convención; otra es el `decimal.Decimal` de Python con una escala fija. Ambas son válidas. Lo que importa es elegir una y ceñirse a ella dentro del contexto delimitado (bounded context). Para Lumen, las unidades menores enteras mantienen el modelo libre de configuración de precisión en tiempo de importación, y `__post_init__` impone la restricción con `BusinessRuleViolation("money-amount-integer")` para que un `float` nunca entre en silencio en el modelo.

🍃 EQUIVALENCIA CON SPRING

`ValueObject` refleja el conjunto `@ValueObject` / `@Embeddable` del ecosistema JPA de Spring y la interfaz marcadora `ValueObject` de Spring Modulith. El dataclass con `frozen=True` se corresponde con el tipo `record` de Java introducido en Java 16: inmutable, igualdad basada en valor, sintaxis concisa. La anotación `@ValueObject` de `jMolecules` expresa la misma intención.

La raíz de agregado

`Money` resuelve el problema de representación: ahora los importes son inmutables y conscientes de la divisa. Pero Lumen todavía necesita algo que *posea* el saldo del monedero y decida cuándo se permite un depósito o una retirada. Ese es el papel de la **raíz de agregado**.

Una entidad se convierte en raíz de agregado cuando posee un grupo de objetos relacionados y actúa como el único punto de entrada para todos los cambios dentro de ese grupo. La raíz de agregado es la **frontera de coherencia**: ningún código externo entra y muta directamente un objeto interno. Todos los cambios fluyen a través de los

métodos de la raíz, que hacen cumplir las reglas. Este es el diseño que evita el atajo de las 11 de la noche descrito en la introducción del capítulo: una vez que todo cambio debe fluir a través de la raíz, no hay puerta trasera.

`AggregateRoot[TID]` amplía `Entity[TID]` con una sola adición: un búfer interno de **eventos de dominio pendientes**. Cada método que cambia el estado llama a `self.raise_event(event)` para registrar lo ocurrido. Cuando el repositorio guarda el agregado, el servicio de aplicación vacía ese búfer con `clear_events()` y publica los eventos en el bus de eventos. Verás el ciclo completo de publicación en la sección Eventos de dominio; por ahora, concéntrate en el propio agregado.

Aquí está la raíz de agregado `Wallet`:

lumen/models/entities/v1/wallet_entity.py

```

from __future__ import annotations

from dataclasses import dataclass
from datetime import UTC, datetime

from lumen.interfaces.enums.v1.currency import Currency
from lumen.models.entities.v1.money import Money
from pyfly.domain import AggregateRoot, BusinessRuleViolation, DomainEvent

# — Domain events


---



@dataclass(frozen=True)
class WalletOpened(DomainEvent):
    wallet_id: str = ""
    owner_id: str = ""
    currency: str = ""

@dataclass(frozen=True)
class FundsDeposited(DomainEvent):
    wallet_id: str = ""
    amount: int = 0
    currency: str = ""
    balance: int = 0

@dataclass(frozen=True)
class FundsWithdrawn(DomainEvent):
    wallet_id: str = ""
    amount: int = 0
    currency: str = ""
    balance: int = 0

# — Aggregate root

```

```

class Wallet(AggregateRoot[str]):
    """Wallet aggregate root — owns the ``balance ≥ 0`` invariant."""

    __slots__ = ("owner_id", "balance", "created_at")

    def __init__(
        self,
        id: str,
        owner_id: str,
        balance: Money,
        created_at: datetime | None = None,
    ) → None:
        super().__init__(id)
        self.owner_id = owner_id
        self.balance = balance
        self.created_at = created_at or datetime.now(UTC)

    @property
    def currency(self) → Currency:
        return self.balance.currency

# — Factory method —


---



    @classmethod
    def open(cls, wallet_id: str, owner_id: str, currency: Currency) →
Wallet:
    """Open a new, empty wallet; raises WalletOpened."""
    if not owner_id.strip():
        raise BusinessRuleViolation(
            "wallet-owner-required", "owner_id is required"
        )
    wallet = cls(
        id=wallet_id,
        owner_id=owner_id,
        balance=Money.zero(currency),
    )
    wallet.raise_event(

```

```

        WalletOpened(
            wallet_id=wallet_id,
            owner_id=owner_id,
            currency=currency.value,
        )
    )
    return wallet

# — Behaviour —————

def deposit(self, amount: Money) → None:
    """Credit *amount* to the balance; raises FundsDeposited."""
    self._assert_currency(amount)
    if not amount.is_positive:
        raise BusinessRuleViolation(
            "wallet-deposit-positive",
            "deposit amount must be > 0",
        )
    self.balance = self.balance.add(amount)
    assert self.id is not None
    self.raise_event(
        FundsDeposited(
            wallet_id=self.id,
            amount=amount.amount,
            currency=amount.currency.value,
            balance=self.balance.amount,
        )
    )

def withdraw(self, amount: Money) → None:
    """Debit *amount*; refuses to overdraw. Raises FundsWithdrawn."""
    self._assert_currency(amount)
    if not amount.is_positive:
        raise BusinessRuleViolation(
            "wallet-withdrawal-positive",
            "withdrawal amount must be > 0",
        )
    remaining = self.balance.subtract(amount)
    if remaining.is_negative:

```

```

        raise BusinessRuleViolation(
            "wallet-insufficient-funds",
            f"cannot withdraw {amount}; balance is {self.balance}",
        )
    self.balance = remaining
    assert self.id is not None
    self.raise_event(
        FundsWithdrawn(
            wallet_id=self.id,
            amount=amount.amount,
            currency=amount.currency.value,
            balance=self.balance.amount,
        )
    )

# — Helpers —————

def _assert_currency(self, amount: Money) → None:
    if amount.currency is not self.balance.currency:
        raise BusinessRuleViolation(
            "wallet-currency-mismatch",
            f"wallet holds {self.balance.currency.value}, "
            f"got {amount.currency.value}",
        )

```

Listado 6.2 — Wallet: la raíz de agregado que posee el saldo e impone sus reglas

Cómo funciona. La frontera del agregado se impone en tres niveles. Primero, `__slots__` bloquea el conjunto de atributos, y `balance` y `owner_id` son deliberadamente públicos-pero-propios: solo los métodos del propio agregado (`deposit` , `withdraw`) los mutan, mientras que la propiedad `currency` es una comodidad de solo lectura que delega en `balance.currency` . Segundo, el classmethod factoría `open` es la única forma legítima de crear un monedero nuevo: el invocador suministra el `wallet_id` (para que la capa de aplicación controle la generación de IDs), `open` valida que `owner_id` no esté en blanco, inicializa el saldo

con `Money.zero(currency)` y encola de inmediato `WalletOpened`. Usar una factoría en lugar de llamar a `__init__` directamente asegura que el evento de apertura *nunca* se olvide, ni siquiera en un fixture de prueba. Tercero, los eventos de dominio — `WalletOpened`, `FundsDeposited`, `FundsWithdrawn` — son dataclasses congelados. `FundsDeposited` y `FundsWithdrawn` llevan un campo `balance` (el saldo posterior a la operación en unidades menores), de modo que un suscriptor nunca necesita volver a consultar al agregado para conocer el estado actual.

Constrúyelo paso a paso. La clase `Wallet` tiene más piezas móviles que `Money`, así que este es el orden para ensamblarla.

Paso 1 — Define primero las tres clases de evento (`WalletOpened`, `FundsDeposited`, `FundsWithdrawn`), cada una un `@dataclass(frozen=True)` que herede de `DomainEvent`. Tienen que existir antes de que el agregado pueda referenciarlas. Cubrimos los eventos a fondo en la sección dentro de dos; por ahora son simplemente los registros que el monedero emitirá.

Paso 2 — Declara el agregado. Hereda de `AggregateRoot[str]` (el `[str]` dice que el id es una cadena), define `__slots__` para bloquear los nombres de atributo y escribe `__init__` para almacenar `owner_id`, `balance` y `created_at`. Llama a `super().__init__(id)` primero para que la clase base configure el id y el búfer interno de eventos.

Paso 3 — Añade la propiedad de solo lectura `currency` que delega en `self.balance.currency`. Es una comodidad que evita que los invocadores tengan que bajar dos niveles hasta `wallet.balance.currency`.

Paso 4 — Escribe el classmethod factoría `open`. Valida `owner_id`, construye el monedero con un saldo `Money.zero(currency)` y llama a `self.raise_event(WalletOpened(...))`. Hacer `open` un classmethod —en lugar de esperar que los invocadores usen `__init__` más un evento manual— garantiza que el evento de apertura nunca se olvide.

Paso 5 — Escribe `deposit` y `withdraw`. Cada uno valida primero (coincidencia de divisa, importe positivo y, para `withdraw`, fondos suficientes), *después* muta `self.balance`, *después* lanza su evento. El orden importa: nunca mutes antes de que hayan pasado todas las comprobaciones, o puedes dejar el monedero en un estado a medio cambiar.

Paso 6 — Añade el ayudante privado `_assert_currency` que comparten ambas transiciones.

RAISE_EVENT NO LANZA UNA EXCEPCIÓN

A pesar del nombre, `raise_event` no lanza nada. *Añade* un evento a un búfer interno del agregado (`AggregateRoot._pending_events`). Nada se publica en ese momento. Un paso posterior —el servicio de aplicación, tras un guardado exitoso— vacía el búfer con `clear_events()` y entrega los eventos al bus de eventos. Piensa en `raise_event` como "anota que esto ocurrió", no como "aborta".

Ejecútalo. Igual que `Money`, el agregado `Wallet` es Python puro: no hace falta base de datos. Desde `uv run python`:

```
>>> from lumen.models.entities.v1.wallet_entity import Wallet
>>> from lumen.models.entities.v1.money import Money
>>> from lumen.interfaces.enums.v1.currency import Currency
>>> w = Wallet.open("wlt-1", "owner-1", Currency.EUR)
>>> w.deposit(Money(1000, Currency.EUR))
>>> w.withdraw(Money(400, Currency.EUR))
>>> str(w.balance)
'6.00 EUR'
>>> [e.event_type for e in w.pending_events()]
['WalletOpened', 'FundsDeposited', 'FundsWithdrawn']
>>> w.withdraw(Money(9999, Currency.EUR))
Traceback (most recent call last):
...
pyfly.domain.exceptions.BusinessRuleViolation: cannot withdraw 99.99 EUR; balance is 6.00 EUR
```

Fíjate en los tres eventos que esperan en el búfer tras las operaciones exitosas, y fíjate en que el intento de descubierto se lanzó *antes* de tocar el saldo: `pending_events()` sigue mostrando tres, no cuatro. La batería de pruebas de Lumen afirma exactamente esto:

```
uv run --extra dev pytest tests/test_wallet_aggregate.py -q
```

```
..... [100%]
6 passed in 0.0Xs
```

Qué acaba de pasar. El monedero ahora posee sus reglas. No hay forma de dejarlo en descubierto, no hay forma de alimentarlo con la divisa equivocada y no hay forma de cambiar su estado sin dejar atrás un evento que registre lo ocurrido. La capa de servicio ya no tiene que recordar nada de eso.

El diagrama de abajo muestra el panorama completo: estado, invariantes y los eventos que el monedero emite.

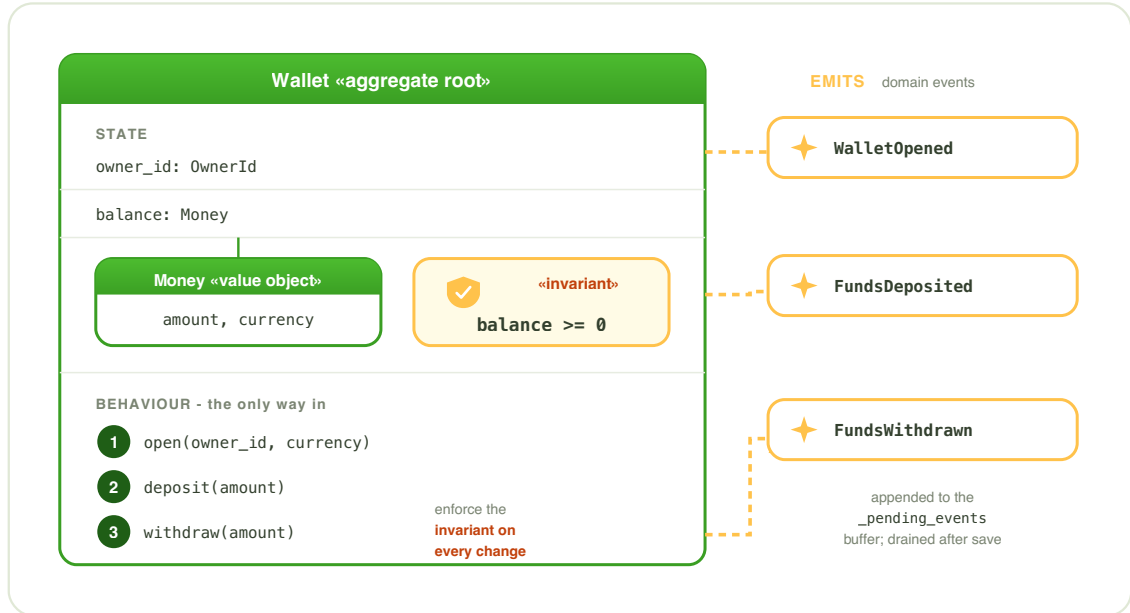


Figura 6.2 — El agregado Wallet: estado, invariantes y los eventos que emite.

🍃 EQUIVALENCIA CON SPRING

`AggregateRoot[str]` se corresponde con `org.jmolecules.ddd.types.AggregateRoot<A, ID>` de `jMolecules` y con `AbstractAggregateRoot<A>` de `Spring Data`, que ofrece el mismo mecanismo `registerEvent()` / `@DomainEvents` / `@AfterDomainEventPublication`. El patrón es idéntico en espíritu: el agregado acumula eventos en un búfer; el repositorio los vacía tras un guardado exitoso; un `DomainEventPublisher` los despacha. El `raise_event` + `clear_events` de `PyFly` es el equivalente Python de `registerEvent` + `@AfterDomainEventPublication`.

Proteger las invariantes

La raíz de agregado solo es valiosa si las reglas que impone son genuinamente inalcanzables por cualquier otra vía. Eso es lo que significa **invariante** en DDD: una condición que el modelo debe mantener sin importar cómo se le llame, quién lo llame o cuántos servicios existan en la aplicación. Una invariante no es una sugerencia: es una restricción que no puede violarse porque el modelo no expone ningún mecanismo para hacerlo.

El `Wallet` de Lumen tiene tres invariantes:

1. El saldo nunca debe bajar de cero (sin descubierto).
2. Los fondos solo pueden depositarse o retirarse en la divisa nativa del monedero.
3. Los importes de depósito y retirada deben ser estrictamente positivos.

Las tres se imponen dentro de los métodos del agregado. La excepción del framework para esto es `BusinessRuleViolation` de `pyfly.domain`. Toma dos argumentos obligatorios: un identificador `rule` estable y legible por máquina, y un `message` legible por humanos. Los identificadores de Lumen — `"wallet-insufficient-funds"`, `"wallet-currency-mismatch"`, `"wallet-deposit-positive"`, `"wallet-withdrawal-positive"` — son etiquetas en kebab-case que viajan en el cuerpo de respuesta RFC 7807 y en los campos de log estructurado.

`BusinessRuleViolation` amplía `pyfly.kernel.BusinessException`, así que el mapeador de detalles de problema (problem-details) RFC 7807 del Capítulo 4 la traduce automáticamente a una respuesta HTTP 422, sin necesidad de un manejador adicional.

i QUÉ SIGNIFICA AQUÍ RFC 7807

RFC 7807 es el estándar web para "detalles de problema": una forma JSON pequeña y predecible (`type`, `title`, `status`, `detail`, más tus propios campos) que devuelve una API cuando algo falla. El mapeador de PyFly convierte automáticamente cualquier `BusinessException` en uno de estos, de modo que el identificador `rule` que pones en `BusinessRuleViolation` acaba en el cuerpo de respuesta donde un cliente puede leerlo sin analizar prosa en inglés.

Comprueba que la invariante se sostiene. La promesa de una invariante es que una operación *fallida* deja el modelo exactamente como estaba. Puedes demostrarlo desde `uv run python`:

```
>>> from lumen.models.entities.v1.wallet_entity import Wallet
>>> from lumen.models.entities.v1.money import Money
>>> from lumen.interfaces.enums.v1.currency import Currency
>>> from pyfly.domain import BusinessRuleViolation
>>> w = Wallet.open("wlt-1", "owner-1", Currency.EUR)
>>> w.deposit(Money(500, Currency.EUR))
>>> w.clear_events()           # drain the open + deposit events
[WalletOpened(...), FundsDeposited(...)]
>>> try:
...     w.withdraw(Money(501, Currency.EUR))
... except BusinessRuleViolation as exc:
...     print(exc.rule)
...
wallet-insufficient-funds
>>> str(w.balance)           # unchanged – the rule fired before any
mutation
'5.00 EUR'
>>> w.pending_events()       # and no event was queued for the failed
attempt
[]
```

Esa es toda la garantía en tres líneas: el identificador de la regla es estable y legible por máquina, el saldo no se movió y ningún evento se filtró por una operación que nunca ocurrió.

⚠ MANTÉN LAS INVARIANTES EN EL MODELO, NO EN EL SERVICIO

Devolver la comprobación de descubierto a `WalletService` crea dos problemas. Primero, cualquier código que llame a `repo.save(entity)` directamente se salta la comprobación por completo. Segundo, acabas duplicando la regla en cada vía que modifica un monedero: el servicio, un trabajo en segundo plano, un comando de administración. Cuando la regla cambia —digamos que el equipo de producto introduce un colchón de descubierto configurable— hay exactamente un sitio que actualizar: el método del agregado. Esa es toda la cuestión.

La diferencia entre una defensa a nivel de servicio y una invariante de agregado es la exigibilidad. Una defensa de servicio es una convención; una invariante de agregado es una restricción física impuesta por el encapsulamiento. Para concretarlo, así es como se ve el enfoque a nivel de servicio y por qué es frágil:

lumen/wallet_service_before.py

```
# DO NOT DO THIS – rules that belong in the model
from pyfly.container import service

@service
class WalletServiceBefore:

    async def withdraw(self, wallet_id: str, amount: float) → None:
        # Rule lives here – but anyone calling repo.save directly skips it
        wallet = {"id": wallet_id, "balance": 50.0, "currency": "EUR"}
        if wallet["balance"] < amount:
            raise ValueError("Insufficient funds")
        wallet["balance"] -= amount
        # ... save
```

Listado 6.3 — Antes: reglas de negocio dispersas por el servicio (frágil)

Y así es como se ve el servicio después de que el modelo asume la propiedad:

lumen/wallet_service_after.py

```

from pyfly.container import service
from pyfly.domain import AggregateNotFound

from lumen.interfaces.enums.v1.currency import Currency
from lumen.models.entities.v1.money import Money

@service
class WalletService:

    async def withdraw(
        self,
        wallet_id: str,
        amount_cents: int,
        currency: Currency,
    ) → None:
        # The service orchestrates; the aggregate decides.
        wallet = await self._repo.find(wallet_id)
        if wallet is None:
            raise AggregateNotFound("Wallet", wallet_id)
        wallet.withdraw(Money(amount=amount_cents, currency=currency))
        await self._repo.save(wallet)
        # Events are drained and published by the repository/service boundary

```

Listado 6.4 — Después: el servicio delega en el agregado

Cómo funciona. La versión "después" se lee como una instrucción:

`wallet.withdraw(...)` significa "pídele al monedero que retire". El servicio no sabe —ni le importa— qué implica eso. Confía en que el agregado o bien tendrá éxito, o bien lanzará una `BusinessRuleViolation`. Ese patrón de orquestador ligero tiene un beneficio práctico para el flujo de trabajo del equipo: un desarrollador nuevo puede implementar un endpoint `transfer` sin leer `WalletService` en absoluto. Las restricciones viven en `Wallet`: un único sitio donde mirar.

El identificador `rule` de `BusinessRuleViolation` también importa. Cadenas como `"wallet-insufficient-funds"` y `"wallet-currency-mismatch"` viajan en el cuerpo de respuesta RFC 7807, donde el código del cliente puede emparejarlas sin analizar mensajes de texto libre. También aparecen en los campos de log estructurado, lo que hace que escribir alertas de producción sea sencillo.

AGGREGATENOTFOUND

`AggregateNotFound` es la segunda excepción de dominio en `pyfly.domain`. Lánzala cuando el agregado solicitado no exista: se corresponde con una respuesta de detalles de problema 404 mediante el mismo manejador RFC 7807. El constructor toma el nombre del tipo de agregado y el ID: `AggregateNotFound("Wallet", wallet_id)`.

Eventos de dominio

Tu agregado ya hace cumplir sus invariantes y controla todos los cambios de estado. Pero Lumen acabará necesitando reaccionar a esos cambios: actualizar un registro de auditoría, enviar una notificación push, disparar la detección de fraude, publicar un asiento en el libro mayor. La solución tentadora es poner esos efectos secundarios directamente dentro de `deposit` y `withdraw`. Eso acopla el modelo de dominio a la infraestructura: de repente tu monedero necesita conocer los topics de Kafka y las plantillas de correo, y cada prueba unitaria arrastra una conexión a un broker.

Los **eventos de dominio** cortan ese acoplamiento. Un evento de dominio registra algo que *ocurrió* dentro del agregado: en pasado, un hecho inmutable. El agregado no sabe qué se hará con el hecho; solo lo registra. Los consumidores aguas abajo — escuchadores de eventos, proyectores, servicios de notificación — se suscriben y reaccionan en su propio contexto, sin que el agregado dependa jamás de ellos.

`DomainEvent` de `pyfly.domain` es una base de dataclass congelado que autorrellena tres campos cuando se crea una instancia:

- `event_id` — un UUID v4 que identifica de forma única esta ocurrencia.
- `occurred_at` — una marca de tiempo UTC en el momento de la construcción.
- `event_type` — una propiedad que por defecto es el nombre de la clase de la subclase (`"WalletOpened"` , `"FundsDeposited"` , `"FundsWithdrawn"`).

Ya viste los tres eventos del monedero definidos en el Listado 6.2. Aquí están aislados, con una mirada explícita a lo que obtienes de la base:

```
lumen/models/entities/v1/wallet_entity.py
```

```
from dataclasses import dataclass
from pyfly.domain import DomainEvent

@dataclass(frozen=True)
class WalletOpened(DomainEvent):
    wallet_id: str = ""
    owner_id: str = ""
    currency: str = ""

@dataclass(frozen=True)
class FundsDeposited(DomainEvent):
    wallet_id: str = ""
    amount: int = 0
    currency: str = ""
    balance: int = 0 # balance after the deposit, in minor units

@dataclass(frozen=True)
class FundsWithdrawn(DomainEvent):
    wallet_id: str = ""
    amount: int = 0
    currency: str = ""
    balance: int = 0 # balance after the withdrawal, in minor units

# Each event carries event_id (UUID), occurred_at (UTC datetime),
# and event_type (class name) – all set by DomainEvent.__post_init__.

def demonstrate_event_fields() → None:
    evt = FundsDeposited(
        wallet_id="w-1",
        amount=5000,
        currency="EUR",
        balance=15000,
    )
```

```
print(evt.event_id)      # e.g. "3fa85f64-5717-4562-b3fc-2c963f66afa6"
print(evt.occurred_at)   # e.g. datetime(2026, 6, 7, 9, 30, 0,
tzinfo=UTC)
print(evt.event_type)    # "FundsDeposited"
```

Listado 6.5 — Eventos de dominio y los campos que `DomainEvent` proporciona automáticamente

Cómo funciona. Cada clase de evento declara solo los campos exclusivos de esa ocurrencia. Todo lo demás viene de `DomainEvent`: un `event_id` UUID para el procesamiento idempotente, `occurred_at` para el rastro de auditoría y `event_type` —el nombre de la clase— para el enrutamiento en el consumidor sin inspeccionar la clase Python. Todos los campos tienen como valor por defecto cero o cadena vacía para que la maquinaria del dataclass con `frozen=True` pueda ofrecer construcción mediante argumentos por palabra clave sin requerir argumentos posicionales.

Fíjate en que `FundsDeposited` lleva tanto `amount` (la transacción) como `balance` (el saldo posterior a la operación, en unidades menores). Un suscriptor que actualice un saldo de modelo de lectura no necesita ninguna llamada de vuelta al agregado ni a la base de datos: todo está en el evento. Ese diseño autocontenido mantiene simples a los consumidores y elimina viajes de ida y vuelta adicionales.

MODELO DE LECTURA, EN LENGUAJE LLANO

Un **modelo de lectura** (read-model) es una copia separada de los datos, optimizada para consultas y moldeada para mostrarlos: un total de panel, un índice de búsqueda, una caché. Como `FundsDeposited` ya lleva el `balance` posterior a la operación, un servicio que mantenga tal copia puede aplicar el evento a ciegas sin volver a cargar nunca el monedero. Construiremos modelos de lectura como es debido en un capítulo posterior; aquí, basta con anotar por qué poner `balance` en el evento merece la pena.

Ejecútalo. Puedes ver los tres campos autorrellenados en cualquier evento desde `uv run python :`

```
>>> from lumen.models.entities.v1.wallet_entity import FundsDeposited
>>> evt = FundsDeposited(wallet_id="w-1", amount=5000, currency="EUR",
balance=15000)
>>> evt.event_type
'FundsDeposited'
>>> evt.event_id          # a fresh UUID, set by DomainEvent.__post_init__
'3fa85f64-5717-4562-b3fc-2c963f66afa6'
>>> evt.occurred_at      # a UTC timestamp, also set automatically
datetime.datetime(2026, 6, 16, 9, 30, 0, tzinfo=datetime.timezone.utc)
```

Declaraste cuatro campos; obtuviste siete, porque `DomainEvent` aporta `event_id`, `occurred_at` y la propiedad `event_type`. Ese es todo el atractivo de heredar de él: cada evento se autoidentifica con cero código adicional.

El ciclo de vida del evento abarca dos fases. Dentro del agregado: cuando `wallet.deposit(amount)` tiene éxito, llama a `self.raise_event(FundsDeposited(...))`, añadiendo el evento a un búfer privado de `AggregateRoot`. Todavía no se publica nada. En la frontera del servicio: después de que el repositorio guarde el agregado y la transacción confirme, el servicio de aplicación vacía el búfer y publica. Esta secuencia de *guardar primero, publicar después* garantiza que nunca se despache un evento por un cambio que no llegó a persistir. El Listado 6.6 muestra esa frontera al completo:

lumen/wallet_application_service.py

```

import uuid

from pyfly.container import service
from pyfly.eda import EventPublisher

from lumen.interfaces.enums.v1.currency import Currency
from lumen.models.entities.v1.money import Money
from lumen.models.entities.v1.wallet_entity import Wallet

@service
class WalletApplicationService:

    def __init__(
        self,
        repo: object,          # typed as WalletRepository in practice
        events: EventPublisher,
    ) → None:
        self._repo = repo
        self._events = events

    async def open_wallet(self, owner_id: str, currency: Currency) → str:
        wallet_id = str(uuid.uuid4())
        wallet = Wallet.open(
            wallet_id=wallet_id, owner_id=owner_id, currency=currency
        )
        await self._repo.save(wallet)
        for event in wallet.clear_events():
            await self._events.publish(event)
        return wallet_id

    async def deposit(
        self,
        wallet_id: str,
        amount_cents: int,
        currency: Currency,
    ) → None:

```

```

wallet = await self._repo.find(wallet_id)
wallet.deposit(Money(amount=amount_cents, currency=currency))
await self._repo.save(wallet)
for event in wallet.clear_events():
    await self._events.publish(event)

```

Listado 6.6 — Vaciar los eventos de dominio tras un guardado exitoso

Cómo funciona. `open_wallet` llama a `Wallet.open`, que encola internamente un evento `WalletOpened`; tras el `save`, `wallet.clear_events()` devuelve ese único evento y `publish` lo despacha. `deposit` sigue el mismo patrón de tres pasos: cargar, mutar, guardar; luego vaciar. El bucle `for event in wallet.clear_events()` es deliberadamente explícito en lugar de ocultarse dentro del repositorio, porque el servicio de aplicación es el sitio adecuado para decidir *cuándo* ocurre la publicación: después de la frontera de la transacción, no antes.

El ciclo de publicación, paso a paso. Cada método de comando del servicio de aplicación sigue el mismo ritmo de cuatro tiempos. Apréndelo una vez y cada caso de uso futuro (transferencia, reembolso, congelación) se escribirá solo:

Paso 1 — Cargar (o crear). Para un monedero nuevo, llama a la factoría `Wallet.open`; para uno existente, `await self._repo.find(wallet_id)`.

Paso 2 — Mutar mediante un método de comportamiento. Llama a `wallet.deposit(...)` o `wallet.withdraw(...)`. Aquí es donde se ejecutan las invariantes y se encolan los eventos en el búfer del agregado.

Paso 3 — Guardar. `await self._repo.save(wallet)`. Nada se ha publicado todavía, a propósito: si el guardado falla, ningún evento escapa.

Paso 4 — Vaciar y publicar. `for event in wallet.clear_events(): await self._events.publish(event)`. Esto se ejecuta solo después de que el guardado tenga éxito, de modo que nunca se despacha un evento por un cambio que no persistió.

Qué acaba de pasar. El agregado decide *qué* ocurrió y lo registra; el servicio de aplicación decide *cuándo* se entera el mundo. Mantener esas dos responsabilidades separadas es la razón por la que el monedero permanece libre de cualquier código de broker, cola o transacción, y por la que el orden de publicación es "guardar primero, publicar después".

💡 ORDEN DE LOS EVENTOS

`raise_event` añade al búfer en el orden de llamada. `clear_events` lo vacía y lo limpia, devolviendo los eventos en el mismo orden. Si un único método del agregado lanza varios eventos (una operación por lotes, por ejemplo), llegan al bus de eventos en el orden en que se lanzaron: los más antiguos primero.

Dominio frente a persistencia

Con el modelo de dominio y sus eventos en su sitio, queda una tensión: ¿cómo llega el agregado `Wallet` a la base de datos?

El atajo tentador es anotar `Wallet` directamente con campos `Mapped[]` de SQLAlchemy y un `__tablename__`. Eso fusiona dos preocupaciones que cambian a ritmos muy distintos: las reglas de negocio evolucionan con el producto; las definiciones de columna evolucionan con el esquema. Mezclarlas significa que un

cambio de esquema te obliga a tocar el agregado, y un cambio de regla arriesga romper accidentalmente un mapeo de columna. También arrastra SQLAlchemy a cada prueba unitaria.

La alternativa son dos modelos que coexisten sin conocerse el uno al otro, y un mapeador ligero que convierte entre ellos:

Modelo	Contiene	Conoce
<code>Wallet</code>	Reglas de negocio, eventos de dominio, invariantes	Nada fuera de <code>pyfly.domain</code>
<code>WalletEntity</code>	Cinco columnas: <code>id</code> , <code>owner_id</code> , <code>currency</code> , <code>balance_minor</code> , <code>created_at</code>	Solo SQLAlchemy + <code>pyfly.data</code>

`Wallet` es Python puro: sin anotaciones `Mapped[]`, sin `__tablename__`. Puedes instanciarlo en una prueba unitaria con dos líneas y ejercitar todas las invariantes sin una conexión a base de datos. `WalletEntity` es persistencia pura: hereda de `Base` de `pyfly.data.relational.sqlalchemy`, lleva columnas tipadas de SQLAlchemy 2.0 y no sabe nada de reglas de dominio ni de eventos. El `Repository[WalletEntity, str]` del framework (Capítulo 5) almacena y recupera filas; un mapeador ligero convierte entre la fila y el agregado en cada cruce.

El Listado 6.7 muestra `WalletEntity` —la fila del ORM— seguido de las dos funciones del mapeador que cruzan la frontera:

lumen/models/entities/v1/wallet_orm.py

```

from __future__ import annotations

from datetime import UTC, datetime

from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column

from pyfly.data.relational.sqlalchemy import Base, UtcDateTime

class WalletEntity(Base):
    """One persisted wallet row, keyed by the aggregate's own string id."""

    __tablename__ = "wallets"

    id: Mapped[str] = mapped_column(String(64), primary_key=True)
    owner_id: Mapped[str] = mapped_column(
        String(255), nullable=False, index=True
    )
    currency: Mapped[str] = mapped_column(String(3), nullable=False)
    balance_minor: Mapped[int] = mapped_column(nullable=False, default=0)
    created_at: Mapped[datetime] = mapped_column(
        UtcDateTime(), default=lambda: datetime.now(UTC)
    )

```

Listado 6.7 — *WalletEntity*: la fila de persistencia de SQLAlchemy

lumen/core/mappers/wallet_mapper.py

```

from __future__ import annotations

from lumen.interfaces.enums.v1.currency import Currency
from lumen.models.entities.v1.money import Money
from lumen.models.entities.v1.wallet_entity import Wallet
from lumen.models.entities.v1.wallet_orm import WalletEntity

def to_entity(wallet: Wallet) → WalletEntity:
    """Flatten a Wallet aggregate into a persistable row."""
    assert wallet.id is not None
    return WalletEntity(
        id=wallet.id,
        owner_id=wallet.owner_id,
        currency=wallet.currency.value,
        balance_minor=wallet.balance.amount,
        created_at=wallet.created_at,
    )

def to_aggregate(entity: WalletEntity) → Wallet:
    """Rehydrate a Wallet aggregate from a persistence row."""
    currency = Currency(entity.currency)
    return Wallet(
        id=entity.id,
        owner_id=entity.owner_id,
        balance=Money(amount=entity.balance_minor, currency=currency),
        created_at=entity.created_at,
    )

```

Listado 6.8 — wallet_mapper: funciones puras que cruzan la frontera dominio/persistencia

Cómo funciona. `WalletEntity` hereda de `Base` en lugar de llevar lógica de dominio alguna, de modo que importarla registra la tabla `wallets` en `Base.metadata`; el `EngineLifecycle` del framework crea la tabla al arrancar cuando se establece `ddl-`

`auto=create`. La clave primaria es el propio id de cadena del agregado (`wlt-...`), no un sustituto, de modo que la fila y el `Wallet` comparten una sola identidad y no hace falta traducción.

`to_entity` escribe `wallet.balance.amount` (un entero) directamente en `balance_minor`: sin conversión a `float`. `to_aggregate` reconstruye un enum `Currency` a partir de la cadena ISO-4217 almacenada mediante `Currency(entity.currency)`, y luego construye un `Money` a partir del valor entero crudo en unidades menores. El campo `created_at` se preserva en el viaje de ida y vuelta, de modo que los agregados rehidratados conservan su marca de tiempo original. No hay ningún cruce de frontera en coma flotante por ninguna parte.

Construye el mapeador paso a paso. Un mapeador no es más que dos funciones puras. No hay magia del framework que cablear; las escribes y las llamas en los momentos adecuados.

Paso 1 — Escribe `to_entity(wallet)`. Lee cada pieza del agregado y cópiala en la fila plana: `wallet.id`, `wallet.owner_id`, `wallet.currency.value` (la cadena ISO, no el enum), `wallet.balance.amount` (las unidades menores enteras crudas) y `wallet.created_at`. El `assert wallet.id is not None` hace explícita la precondition: solo persistes monederos que ya tienen un id.

Paso 2 — Escribe `to_aggregate(entity)`, lo inverso. Reconstruye el enum `Currency` a partir de la cadena almacenada con `Currency(entity.currency)`, envuelve el entero de nuevo en un `Money` y construye un `Wallet`. Pasar `created_at=entity.created_at` preserva la marca de tiempo original a través del viaje de ida y vuelta.

Paso 3 — Llámalas en la frontera, no dentro del agregado. El manejador de comandos llama a `to_entity` justo antes de `repo.save(...)` y a `to_aggregate` justo después de `repo.find(...)`. La clase `Wallet` nunca importa la fila, y la fila nunca importa el `Wallet`.

Ejecútalo. El viaje de ida y vuelta es Python puro: no se requiere una base de datos en vivo para demostrar que preserva todos los campos. Desde `uv run python`:

```
>>> from lumen.models.entities.v1.wallet_entity import Wallet
>>> from lumen.models.entities.v1.money import Money
>>> from lumen.interfaces.enums.v1.currency import Currency
>>> from lumen.core.mappers import wallet_mapper
>>> w = Wallet.open("wlt-1", "owner-1", Currency.EUR)
>>> w.deposit(Money(2500, Currency.EUR))
>>> row = wallet_mapper.to_entity(w)           # aggregate → flat row
>>> (row.id, row.currency, row.balance_minor)
('wlt-1', 'EUR', 2500)
>>> back = wallet_mapper.to_aggregate(row)     # flat row → aggregate
>>> str(back.balance), back.owner_id
('25.00 EUR', 'owner-1')
```

El entero `2500` cruza en ambas direcciones intacto: ningún `float` aparece jamás en la frontera de persistencia, que es toda la razón por la que `Money` almacena unidades menores.

Qué acaba de pasar. Mantuviste dos modelos que nunca se importan el uno al otro y los uniste con dos funciones diminutas. Un cambio de esquema ahora toca solo `WalletEntity`; un cambio de regla toca solo `Wallet`. Ninguno puede romper al otro por accidente.

El mapeador es deliberadamente estrecho. No hace cumplir reglas: eso lo hacen `Wallet.__init__` y los métodos de comportamiento. No publica eventos: eso lo hace el servicio de aplicación. Solo traduce la forma.

El repositorio. El servicio de aplicación nunca interactúa con `WalletEntity` directamente. En su lugar, un manejador de comandos llama a `wallet_mapper.to_entity(wallet)` antes de persistir y a `wallet_mapper.to_aggregate(entity)` después de cargar, mientras que el `WalletRepository(Repository[WalletEntity, str])` del framework gestiona todo el SQL. El Capítulo 5 cubre `Repository` al completo; el punto clave aquí es que el propio `Wallet` nunca importa SQLAlchemy: el agregado permanece libre de preocupaciones de persistencia a ambos lados de la frontera del mapeador.

🍃 EQUIVALENCIA CON SPRING

Esta estructura de dos modelos más mapeador es el equivalente Python del patrón defendido en *Implementing Domain-Driven Design* de Vaughn Vernon para Spring: una `WalletJpaEntity` anotada con `@Entity` (la fila de persistencia), un objeto de dominio `Wallet` (el agregado) y un `WalletAssembler` o un mapeador generado por MapStruct que traduce entre ellos. El `JpaRepository<WalletJpaEntity, String>` de Spring Data JPA se corresponde con el `Repository[WalletEntity, str]` de PyFly. La estructura es idéntica; el código repetitivo es menor.

Especificaciones para las reglas de negocio

El agregado protege bien las operaciones que cambian el estado: no puedes dejar un monedero en descubierto ni depositar la divisa equivocada. Pero no todas las reglas tratan sobre la mutación. Algunas reglas son comprobaciones de elegibilidad: "antes de mostrar a este usuario el botón de retirada, ¿está el monedero en un estado operable?" o "de diez mil monederos, ¿cuáles cumplen los requisitos para el bono de fidelidad?". Estos son predicados de solo lectura, y codificarlos como métodos del agregado abarrotaría `Wallet` con lógica de consulta no relacionada con las transiciones de estado.

El **patrón Especificación** lo resuelve limpiamente. Una especificación es un predicado con nombre y reutilizable: un único método `is_satisfied_by(obj) → bool` envuelto en un objeto que se compone con otros usando operadores booleanos. Como cada regla es su propia clase, puedes nombrar las reglas con claridad, reutilizarlas entre servicios y combinarlas en tiempo de ejecución según el contexto, algo que una cadena de `if` no puede hacer.

`Specification[T]` de `pyfly.domain` es un predicado en memoria componible. Hereda de ella, implementa `is_satisfied_by` y combina instancias con `&` (y), `|` (o) y `~` (no). Una especificación también es directamente invocable, así que puedes pasarla al `filter` integrado de Python sin código adaptador alguno.

DOS TIPOS DE ESPECIFICACIÓN

`pyfly.domain.Specification` es el predicado en memoria que se usa dentro de los servicios de dominio. `pyfly.data.relational.sqlalchemy.Specification` (Capítulo 5) es el predicado de consulta consciente de la base de datos que empuja la regla hacia abajo, hasta el SQL. Los dos coexisten. Las especificaciones de dominio son para la lógica de negocio; las especificaciones de datos son para las consultas.

Aquí hay una especificación que expresa la regla "elegible para retirada":

lumen/domain/specs.py

```

from lumen.interfaces.enums.v1.currency import Currency
from lumen.models.entities.v1.wallet_entity import Wallet
from pyfly.domain import Specification

class HasPositiveBalance(Specification[Wallet]):
    """The wallet has at least one cent remaining."""

    def is_satisfied_by(self, wallet: Wallet) → bool:
        return wallet.balance.is_positive

class IsInCurrency(Specification[Wallet]):
    """The wallet holds a specific currency."""

    def __init__(self, currency: Currency) → None:
        self._currency = currency

    def is_satisfied_by(self, wallet: Wallet) → bool:
        return wallet.balance.currency is self._currency

# Compose: a wallet is eligible for withdrawal if it has a positive
# balance in the requested currency.
def eligible_for_withdrawal(currency: Currency) → Specification[Wallet]:
    return HasPositiveBalance() & IsInCurrency(currency)

# Use as a predicate:
def filter_eligible(
    wallets: list[Wallet],
    currency: Currency,
) → list[Wallet]:
    spec = eligible_for_withdrawal(currency)
    return list(filter(spec, wallets))

```

Listado 6.9 — EligibleForWithdrawal: una Especificación de dominio componible

Cómo funciona. `HasPositiveBalance` delega en `wallet.balance.is_positive` (una propiedad, sin paréntesis). `IsInCurrency` usa comparación de identidad (`is`) porque `Currency` es un `StrEnum` con miembros singleton. La factoría `eligible_for_withdrawal` las combina con `&` , produciendo un compuesto cuyo `is_satisfied_by` devuelve `True` solo cuando pasan ambas comprobaciones. Como `Specification` implementa `__call__` , pasas el compuesto directamente a `filter()` : no hace falta un envoltorio lambda.

Construye una especificación paso a paso.

Paso 1 — Hereda de `Specification[Wallet]` e implementa el único método requerido, `is_satisfied_by(self, wallet) → bool` . Ese es todo el contrato: devuelve `True` o `False` , nunca lances.

Paso 2 — Si la regla necesita un parámetro (como una divisa con la que coincidir), tómallo en `__init__` y almacénalo, como hace `IsInCurrency` . Una regla sin parámetros como `HasPositiveBalance` se salta esto.

Paso 3 — Compón con los operadores booleanos. `HasPositiveBalance() & IsInCurrency(currency)` construye una nueva especificación cuyo `is_satisfied_by` es verdadero solo cuando lo son ambas mitades. `|` es o, `~` es no.

Paso 4 — Úsala como predicado. Como `Specification` implementa `__call__` , el compuesto es un invocable, así que puedes entregarlo directamente al `filter()` integrado de Python sin lambda de por medio.

Ejecútalo. Las especificaciones son objetos corrientes. Coloca las clases del Listado 6.9 en un módulo (digamos `src/lumen/domain/specs.py`) y pruébalas desde `uv run python :`

```

>>> from lumen.models.entities.v1.wallet_entity import Wallet
>>> from lumen.models.entities.v1.money import Money
>>> from lumen.interfaces.enums.v1.currency import Currency
>>> from lumen.domain.specs import eligible_for_withdrawal
>>> empty = Wallet.open("wlt-1", "owner-1", Currency.EUR)
>>> funded = Wallet.open("wlt-2", "owner-2", Currency.EUR)
>>> funded.deposit(Money(1000, Currency.EUR))
>>> usd = Wallet.open("wlt-3", "owner-3", Currency.USD)
>>> usd.deposit(Money(1000, Currency.USD))
>>> spec = eligible_for_withdrawal(Currency.EUR)
>>> spec(funded), spec(empty), spec(usd)
(True, False, False)
>>> [w.id for w in filter(spec, [empty, funded, usd])]
['wlt-2']

```

Solo el monedero EUR con fondos satisface ambas mitades del compuesto, así que `filter` conserva únicamente ese.

Qué acaba de pasar. Expresaste una regla de negocio de solo lectura como un objeto con nombre y reutilizable en lugar de un `if` en línea. Se compone con otras reglas en tiempo de ejecución, pasa directamente a `filter` y es trivial de probar unitariamente de forma aislada; y, crucialmente, vive *fuera* del agregado, que es donde corresponden las comprobaciones de elegibilidad.

La disciplina de diseño clave: una especificación es un *predicado*, no una *defensa*.

Devuelve `True` o `False` y nunca lanza. Las invariantes del agregado (descubierto, divisa que no coincide) corresponden a dentro de `deposit` y `withdraw` porque deben *impedir* un cambio de estado. Las especificaciones corresponden a los servicios y a los manejadores de consulta porque *seleccionan* o *clasifican*: nunca mutan.

Las especificaciones son especialmente útiles allí donde las reglas se combinan dinámicamente: una búsqueda de administración que añade filtros según el rol del operador, o un trabajo por lotes que parte una lista en monederos elegibles y no elegibles. Cada clase tiene exactamente un método y ningún efecto secundario, lo que hace triviales las pruebas unitarias aisladas.

💡 SPECIFICATION.OF PARA LAMBDAS RÁPIDAS

Para predicados puntuales que no necesitan una clase, usa el método factoría: `spec = Specification.of(lambda w: w.balance.amount ≥ 1000, name="minimum-balance")`. Se compone con `&`, `|` y `~` de la misma forma que una subclase completa.

✓ Lo que construiste

El monedero de Lumen es ahora un modelo de dominio de primera clase.

`Money` es un `ValueObject` congelado que almacena importes como unidades menores enteras, impone la homogeneidad de divisa mediante `__assert_same_currency` y se reemplaza en lugar de mutarse. `__post_init__` rechaza los `float` de inmediato. `is_positive` e `is_negative` son propiedades; `major_units` y `__str__` se encargan de la visualización.

`Wallet(AggregateRoot[str])` es la frontera de coherencia. Su factoría `open` y los métodos de comportamiento `deposit` / `withdraw` hacen cumplir las tres invariantes — sin descubierto, sin operaciones entre divisas, sin importes no positivos — lanzando `BusinessRuleViolation` con un identificador de regla estable. Cada cambio de estado encola un evento de dominio (`WalletOpened`, `FundsDeposited`, `FundsWithdrawn`); el

saldo posterior a la operación se registra en cada evento, de modo que los suscriptores no necesitan ninguna llamada de vuelta. Tras un guardado exitoso, el servicio de aplicación vacía los eventos con `clear_events()` y los entrega a `EventPublisher`.

La capa de persistencia solo ve `WalletEntity` (cinco columnas, sin lógica de dominio). `to_aggregate` en `wallet_mapper` rehidrata la fila en un `Wallet`, y `to_entity` la aplanas de vuelta; el `WalletRepository(Repository[WalletEntity, str])` del framework gestiona todo el SQL sin que el agregado importe jamás SQLAlchemy. `Specification[Wallet]` te da un predicado componible e invocable para comprobaciones de elegibilidad que viven fuera de la frontera del agregado.

El controlador queda intacto. El servicio encogió. Las reglas las hace cumplir el objeto que las posee.

Pruébalo tú mismo

1. **Añade un método `transfer_to` y un objeto de valor `DailyLimit`.** Añade un `DailyLimit(ValueObject)` con `max_amount: int` y `currency: Currency`, decorado con `@dataclass(frozen=True)`. Luego añade `Wallet.transfer_to(target: Wallet, amount: Money) → None`. El método debería llamar a `self.withdraw(amount)` y `target.deposit(amount)` en secuencia, lanzando `BusinessRuleViolation("transfer-currency-mismatch", ...)` si los dos monederos contienen divisas distintas. Como `Wallet` usa `__slots__`, añade `"_frozen"` a la tupla si también haces el ejercicio 2. Verifica que ambos monederos acumulan cada uno un evento `FundsWithdrawn` / `FundsDeposited`, respectivamente, y que una transferencia entre monederos que no coinciden se lanza antes de modificar ninguno de los dos saldos.

2. Añade un evento `WalletFrozen` y un comportamiento `freeze()`. Define

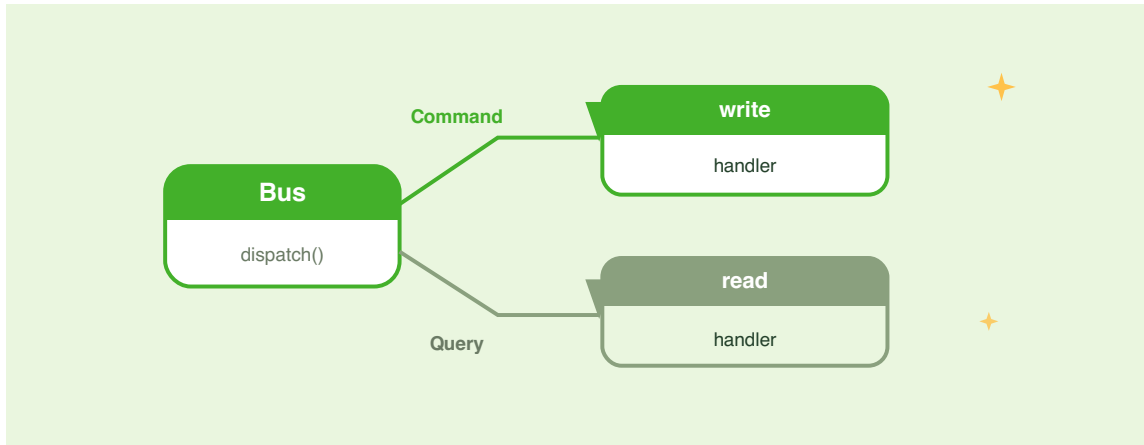
`WalletFrozen(DomainEvent)` con `wallet_id: str = ""` y `reason: str = ""`.
 Añade `"frozen"` a `__slots__` y un atributo `frozen: bool` (por defecto `False`) a `Wallet.__init__`. Añade un método `freeze(reason: str) → None` que ponga `self.frozen = True` y llame a `self.raise_event(WalletFrozen(...))`. Protege `deposit` y `withdraw` con `if self.frozen: raise BusinessRuleViolation("wallet-frozen", ...)` al principio de cada método.
 Luego escribe un escuchador de eventos usando `@event_listener` de `pyfly.eda` que registre una advertencia estructurada cada vez que se publique un evento `WalletFrozen`.

3. Expresa una regla de negocio como una Especificación. Escribe un

`MinimumBalance(Specification[Wallet])` que compruebe si el saldo de un monedero está en o por encima de un importe umbral (en unidades menores) pasado a su `__init__`. Combínalo con `IsInCurrency` del Listado 6.9 usando `&` para producir una función factoría `premium_eligible(currency: Currency, threshold: int)`. Llama a `list(filter(premium_eligible(Currency.EUR, 50000), wallets))` sobre una lista de monederos de prueba y afirma que solo aparecen en el resultado los monederos con al menos 500,00 EUR (50 000 céntimos).

CAPÍTULO 7

CQRS: comandos y consultas



El monedero (wallet) de Lumen es ya un ciudadano de primera clase del dominio. El agregado `Wallet` impone sus propias invariantes, emite eventos de dominio y persiste a través de una frontera de repositorio limpia. El controlador, sin embargo, sigue llamando a `WalletApplicationService` directamente: un método por operación, con lecturas y escrituras compartiendo la misma ruta de código. Ese diseño está bien a pequeña escala, pero muestra fricciones a medida que el sistema crece. El equipo quiere cachear los saldos de los monederos, mantener un único rastro de auditoría para cada escritura, añadir reglas de autorización a operaciones concretas y probar cada pieza de lógica en aislamiento completo de las demás.

CQRS —Command Query Responsibility Segregation, segregación de responsabilidad entre comandos y consultas— aborda todo esto trazando una línea clara entre las dos cosas que un servicio puede hacer: *cambiar el estado* y *leer el estado*. Las escrituras se convierten en **comandos**: mensajes fuertemente tipados, con nombre e inmutables

que fluyen a través de un `CommandBus`. Las lecturas se convierten en **consultas**: mensajes igualmente tipados que fluyen a través de un `QueryBus`. Cada bus ejecuta una tubería fija —validación, autorización, ejecución y, después (para los comandos), publicación de eventos de dominio—. Tu manejador (handler) implementa exactamente una intención; el bus se encarga de todo lo demás.

Al final de este capítulo, el controlador de Lumen despacha comandos y consultas en lugar de llamar al servicio directamente. `OpenWallet`, `DepositFunds` y `WithdrawFunds` recorren la ruta de comandos; `GetWallet`, `GetBalance`, `ListWallets` y `ListRichWallets` recorren la ruta de consultas. El agregado `Wallet` que construiste en el Capítulo 6 permanece intacto: CQRS no reemplaza el modelo de dominio; es el mecanismo de entrega de instrucciones hacia él.

NUEVA JERGA, EN TÉRMINOS SENCILLOS

Un **bus** aquí no es hardware: es un único objeto al que le entregas un mensaje, y él averigua qué manejador debe ejecutarse. Un **manejador** es una clase pequeña que hace el trabajo para exactamente un tipo de mensaje. Un **DTO** (objeto de transferencia de datos) es una forma plana —id, propietario, saldo— que pones en el cable como JSON; está deliberadamente separado de tu objeto de dominio rico. Una **proyección** es una porción de solo lectura de tus datos, moldeada para una vista concreta. Conocerás cada uno de estos a medida que construimos, una pieza cada vez, así que no te preocupes si ahora mismo te resultan abstractos.

Este capítulo está construido en torno a PyFly **v26.6.110**, y cada listado está tomado literalmente del ejemplo de Lumen en `samples/lumen/src/lumen`. Iremos despacio: construiremos primero los comandos, luego sus manejadores, después las consultas y, por último, conectaremos todo en el controlador, ejecutando la aplicación y las pruebas en cada hito para que puedas ver cómo cobran vida las piezas antes de añadir la siguiente.

Por qué separar las lecturas de las escrituras

Imagina Lumen al final del Capítulo 6. `WalletController` llama a `WalletApplicationService.credit(wallet_id, amount)`. Esa llamada muta el estado, pero nada en la firma del método lo hace evidente. El equipo quiere añadir una caché de saldos. ¿Dónde va? ¿Dentro de `credit`? ¿En un decorador alrededor del servicio? La pregunta revela el problema: a un único método de servicio se le pide servir a dos amos —la ruta de escritura, que siempre debe tocar la base de datos, y la ruta de lectura, que debería evitarla siempre que sea posible—. Atornillar la caché a un método de escritura es incómodo en el mejor de los casos y peligroso en el peor.

Las escrituras y las lecturas tienen formas fundamentalmente distintas. Una escritura lleva intención y datos: «deposita 1 500 unidades menores en el monedero wlt-001». Una lectura lleva una pregunta: «¿cuál es el saldo actual del monedero wlt-001?». La primera debe alcanzar la base de datos cada vez. La segunda es repetible: preguntar dos veces debería devolver la misma respuesta sin duplicar la carga de la base de datos. Canalizar ambas a través del mismo método mezcla preocupaciones que escalan de forma distinta, se prueban de forma distinta y necesitan un comportamiento transversal distinto.

El beneficio más profundo es la **claridad de intención**. Cuando un compañero lee `wallet_service.credit(wallet_id, amount)`, tiene que inspeccionar la implementación para saber si es seguro llamarlo dos veces, si publica eventos y si es idempotente. Cuando lee `DepositFunds(wallet_id=..., amount=...)`, la intención es inequívoca; y si la intención resulta estar equivocada, cambias el nombre del comando, no la firma del servicio.

Tres beneficios concretos importan para Lumen:

Escalado independiente. Las lecturas suelen superar a las escrituras en un orden de magnitud o más. Una vez que las dos rutas están separadas, el bus puede cachear los resultados de las consultas sin tocar la ruta de escritura. Puedes enrutar las consultas a una réplica de lectura y los comandos a la base de datos primaria con un cambio de configuración, no de código.

Manejadores enfocados. Cada manejador implementa exactamente una operación.

`DepositFundsHandler` carga un monedero, impulsa su comportamiento de dominio, lo persiste y drena eventos; nada más. `GetBalanceHandler` carga un monedero y devuelve una proyección ligera; nada más. Como los manejadores son clases Python planas con dependencias inyectadas, puedes probar cada uno por unidad en aislamiento completo de la capa HTTP.

Preocupaciones transversales centralizadas. La validación, la autorización y las trazas distribuidas se implementan una sola vez en la tubería del bus y se aplican uniformemente a cada manejador, sin código repetitivo en el propio manejador. Añadir autorización por operación más adelante es cuestión de sobrescribir `authorize()` en el comando; el bus garantiza que se ejecute antes de que se llegue siquiera a `do_handle`.

Comandos y manejadores de comandos

Antes de escribir una sola línea de código de manejador, da nombre a las intenciones de tu sistema. En el dominio de monederos de Lumen pueden ocurrir tres cosas: se puede abrir un monedero, se pueden depositar fondos y se pueden retirar fondos. Cada una es un **comando**: un mensaje con nombre e inmutable que expresa una intención. El bus lo entrega; el manejador actúa sobre él; el agregado de dominio

impone las reglas. Los comandos no son llamadas a métodos disfrazadas de objetos: son contratos explícitos que viven en tu base de código como ciudadanos de primera clase.

Un comando es un dataclass congelado que hereda de `Command[R]`, donde `R` es el tipo que devuelve el manejador. El parámetro genérico es documentación y una pista para el verificador de tipos; el bus no lo impone en tiempo de ejecución.

¿QUÉ ES `COMMAND[R]` ?

El `[R]` de `Command[R]` es un *parámetro de tipo genérico*: un marcador de posición para «lo que sea que devuelva este comando». `OpenWallet(Command[str])` dice «al enviarme, recibes de vuelta un `str`» (el nuevo id del monedero).

`DepositFunds(Command[int])` dice «al enviarme, recibes de vuelta un `int`» (el nuevo saldo). Tu editor y tu verificador de tipos usan esto para detectar errores; en tiempo de ejecución el bus simplemente devuelve lo que devolvió el manejador.

Los comandos de Lumen viven en tres archivos separados bajo

`lumen/core/services/wallets/`, uno por intención. Los construiremos uno a uno.

Paso 1 — Escribe el comando `OpenWallet`. Crea `open_wallet_command.py`. Lleva los dos datos necesarios para abrir un monedero —quién es su propietario y qué moneda contiene— y un gancho `validate()` que rechaza un propietario en blanco antes incluso de que el bus busque un manejador.

lumen/core/services/wallets/open_wallet_command.py

```

from __future__ import annotations

from dataclasses import dataclass

from lumen.interfaces.enums.v1.currency import Currency
from pyfly.cqrs import Command, ValidationResult

@dataclass(frozen=True)
class OpenWallet(Command[str]):
    """Open a new wallet. Returns the generated wallet id."""

    owner_id: str
    currency: Currency

    async def validate(self) → ValidationResult: # type: ignore[override]
        if not self.owner_id.strip():
            return ValidationResult.failure(
                "owner_id", "Owner id is required"
            )
        return ValidationResult.success()

```

Listado 7.1 — OpenWallet: un comando congelado con validación incorporada

Paso 2 — Escribe los comandos `DepositFunds` y `WithdrawFunds`. Cada uno lleva un `wallet_id` al que apuntar y un `amount` en unidades menores, y valida que el id esté presente y que el importe sea positivo. Son deliberadamente gemelos casi idénticos: la misma forma, dirección opuesta.

```
lumen/core/services/wallets/deposit_funds_command.py
```

```
from __future__ import annotations

from dataclasses import dataclass

from pyfly.cqrs import Command, ValidationResult

@dataclass(frozen=True)
class DepositFunds(Command[int]):
    """Deposit ``amount`` minor units. Returns the new balance."""

    wallet_id: str
    amount: int

    async def validate(self) → ValidationResult: # type: ignore[override]
        if not self.wallet_id.strip():
            return ValidationResult.failure(
                "wallet_id", "Wallet id is required"
            )
        if self.amount ≤ 0:
            return ValidationResult.failure(
                "amount", "Deposit amount must be > 0"
            )
        return ValidationResult.success()
```

Listado 7.2 — *DepositFunds: importe en unidades menores, sin campo de moneda*

lumen/core/services/wallets/withdraw_funds_command.py

```

from __future__ import annotations

from dataclasses import dataclass

from pyfly.cqrs import Command, ValidationResult

@dataclass(frozen=True)
class WithdrawFunds(Command[int]):
    """Withdraw ``amount`` minor units. Returns the new balance."""

    wallet_id: str
    amount: int

    async def validate(self) → ValidationResult: # type: ignore[override]
        if not self.wallet_id.strip():
            return ValidationResult.failure(
                "wallet_id", "Wallet id is required"
            )
        if self.amount ≤ 0:
            return ValidationResult.failure(
                "amount", "Withdrawal amount must be > 0"
            )
        return ValidationResult.success()

```

Listado 7.3 — *WithdrawFunds*: la misma forma que *DepositFunds*

Cuatro decisiones de diseño están horneadas en cada comando:

- `frozen=True` hace que el dataclass sea inmutable en el mismo momento en que se construye. Los campos no pueden mutarse accidentalmente en una capa de la tubería antes de llegar a otra, y los mensajes inmutables son hasheables por defecto, lo cual es útil al almacenarlos o compararlos en las pruebas.

- `validate()` es un gancho asíncrono que el bus llama antes de despachar el manejador. `OpenWallet.validate` comprueba que `owner_id` no esté en blanco; `DepositFunds.validate` y `WithdrawFunds.validate` comprueban que el importe sea positivo. Estas precondiciones corresponden al comando: no requieren ninguna consulta a la base de datos y no pertenecen al agregado de dominio. El agregado impone invariantes que necesitan estado cargado (descubierto, coincidencia de moneda); los comandos imponen invariantes que se pueden conocer únicamente a partir de los campos. Mantener separadas estas dos capas significa que el agregado nunca se llama con datos estructuralmente incorrectos.
- **Sin campo `currency`** en `DepositFunds` ni en `WithdrawFunds`. La propia moneda del monedero es la única moneda válida para un depósito o una retirada, y el repositorio la resuelve una vez que el agregado está cargado. Llevar una moneda en el comando invitaría a discrepancias; el agregado impone la invariante a partir de su propio estado.
- **Nomenclatura en modo imperativo:** `DepositFunds`, no `WalletDeposit` ni `DepositFundsCommand`. Esto hace que el registro de comandos se lea como un rastro de auditoría de negocio —una secuencia de cosas que *ocurrieron*— en lugar de una lista de operaciones técnicas.

LO QUE ACABA DE OCURRIR

Ahora tienes tres archivos pequeños, cada uno describiendo *una cosa que el sistema puede hacer*, sin más lógica que un par de comprobaciones de campo. Todavía no hay manejadores: estos comandos no hacen nada por sí solos. Esa es la idea: un comando es un sobre, no el trabajador que lo abre. A continuación escribirás los trabajadores (los manejadores) que realmente llevan a cabo cada intención.

Implementar un manejador de comandos

Un manejador de comandos hereda de `CommandHandler[C, R]` e implementa exactamente un método: `do_handle`. Tú escribes el *qué*; el bus lo envuelve con el *cómo*.

Ambos decoradores en cada manejador son obligatorios. `@command_handler` registra la clase en el `HandlerRegistry` introspeccionando el primer argumento de tipo genérico; no se necesita registro manual. `@service` conecta el manejador al contenedor de inyección de dependencias de PyFly para que los argumentos del constructor se resuelvan e inyecten automáticamente en el arranque. El orden importa: `@command_handler` arriba, `@service` justo debajo. Sin `@service`, el contenedor de inyección de dependencias nunca instancia la clase y el bus no puede encontrar el manejador; sin `@command_handler`, el registro nunca mapea el tipo de comando a la clase. Omitir cualquiera de los dos decoradores es un fallo silencioso: el bus lanza «no handler found» en el momento del despacho.

`@transactional()` convierte `do_handle` en una unidad de trabajo confirmada.

Los manejadores de comandos inyectan `session_factory:` `async_sessionmaker[AsyncSession]` y la almacenan como `self._session_factory`. Cuando `@transactional()` ejecuta `do_handle`, liga una unidad de trabajo a la tarea en curso en el origen de datos de esa factoría; cada llamada al repositorio del método se une a ella, y confirma en caso de éxito y revierte ante cualquier excepción. Sin `@transactional()` cada llamada al repositorio confirmaría por su cuenta, así que un manejador que carga, comprueba, guarda y publica ya no tendría éxito o fallaría como un solo paso.

❗ FLUSH FRENTE A COMMIT, EN TÉRMINOS SENCILLOS

Un **flush** empuja tus cambios pendientes a la conexión de la base de datos para que las consultas posteriores en la *misma* sesión puedan verlos, pero siguen estando dentro de una transacción abierta que se puede revertir. Un **commit** los hace permanentes. Dentro de `@transactional()` el repositorio solo hace flush, y la unidad hace commit una sola vez, al final del manejador. Fuera de él, cada llamada al repositorio es una unidad breve que hace commit por sí misma. El decorador es lo que hace que todo el manejador persista, o falle, de una vez.

Paso 3 — Escribe `OpenWalletHandler`. Ahora construye el trabajador para el primer comando. Crea `open_wallet_handler.py`, apila `@command_handler` sobre `@service`, inyecta el repositorio, el publicador de eventos y la factoría de sesiones, e implementa el único método `do_handle`.

lumen/core/services/wallets/open_wallet_handler.py

```

from __future__ import annotations

from uuid import uuid4

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

from lumen.core.mappers.wallet_mapper import to_entity
from lumen.core.services.wallets.event_publishing import publish_domain_events
from lumen.core.services.wallets.open_wallet_command import OpenWallet
from lumen.models.entities.v1.wallet_entity import Wallet
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.data.relational.sqlalchemy import transactional
from pyfly.eda import EventPublisher

@command_handler
@service
class OpenWalletHandler(CommandHandler[OpenWallet, str]):
    """Open a new, empty wallet."""

    def __init__(
        self,
        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
    ) → None:
        super().__init__()
        self._repository = repository
        self._events = events
        # @transactional resolves the unit-of-work session from here.
        self._session_factory = session_factory

    @transactional()
    async def do_handle(self, command: OpenWallet) → str: # type:
        ignore[override]

```

```

wallet_id = f"wlt-{uuid4()}"
wallet = Wallet.open(
    wallet_id=wallet_id,
    owner_id=command.owner_id,
    currency=command.currency,
)
await self._repository.upsert(to_entity(wallet))
await publish_domain_events(self._events, wallet.clear_events())
return wallet_id

```

Listado 7.4 — *OpenWalletHandler: unidad de trabajo @transactional() + upsert*

Recorre `do_handle` paso a paso. `f"wlt-{uuid4()}"` genera un identificador estable con prefijo. `Wallet.open(...)` llama a la factoría, que impone la precondition de propietario no vacío y bufferiza un evento `WalletOpened`. `to_entity(wallet)` mapea el agregado a una fila plana `WalletEntity`. `repository.upsert(...)` llama a `session.merge` —una sola llamada que inserta si no existe ninguna fila o actualiza si existe— y luego hace flush. El agregado es dueño de su id, así que tanto INSERT como UPDATE usan la misma cadena estable como clave. (Hasta la v26.09.07 `save` siempre insertaba y un segundo guardado del mismo id lanzaba `IntegrityError`; desde la v26.09.08 `save` también fusiona una entidad cuya clave está fijada, y `upsert` se queda como la forma explícita.) `wallet.clear_events()` drena el búfer y `publish_domain_events` reenvía cada evento al bus de EDA. El decorador `@transactional()` confirma la unidad de trabajo al salir. El manejador devuelve el ID del monedero, que fluye de vuelta al controlador como el valor de retorno de `send`.

Observa el requisito del constructor: `super().__init__()` es obligatorio en `CommandHandler`. Si lo omites, la contabilidad interna de la clase base —contexto de correlación, ganchos de ciclo de vida— nunca se inicializa. El repositorio, `EventPublisher` y `session_factory` los inyecta el contenedor de inyección de dependencias a partir de las anotaciones de tipo; no se necesita ninguna configuración de factoría.

Paso 4 — Escribe los manejadores de depósito y retirada. Estos dos añaden un movimiento que `OpenWalletHandler` no necesitaba: *cargan* un monedero existente antes de actuar sobre él. La forma es la misma en ambos, diferenciándose solo en si llaman a `wallet.deposit(...)` o a `wallet.withdraw(...)`.

```
lumen/core/services/wallets/deposit_funds_handler.py
```

```
from __future__ import annotations

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

from lumen.core.mappers.wallet_mapper import to_aggregate, to_entity
from lumen.core.services.wallets.deposit_funds_command import DepositFunds
from lumen.core.services.wallets.event_publishing import publish_domain_events
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cQRS import CommandHandler, command_handler
from pyfly.domain import AggregateNotFound
from pyfly.data.relational.sqlalchemy import LockMode, transactional
from pyfly.eda import EventPublisher
```

```
@command_handler
```

```
@service
```

```
class DepositFundsHandler(CommandHandler[DepositFunds, int]):
```

```
    """Credit funds to an existing wallet; returns the new balance."""
```

```
    def __init__(
        self,
        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
    ) → None:
        super().__init__()
        self._repository = repository
        self._events = events
        self._session_factory = session_factory
```

```
@transactional()
```

```
    async def do_handle(self, command: DepositFunds) → int: # type:
        ignore[override]
```

```
        entity = await self._repository.find_by_id(
            command.wallet_id, lock=LockMode.PESSIMISTIC_WRITE
```

```
)  
if entity is None:  
    raise AggregateNotFound("Wallet", command.wallet_id)  
  
wallet = to_aggregate(entity)  
wallet.deposit(Money(amount=command.amount, currency=wallet.currency))  
await self._repository.upsert(to_entity(wallet))  
  
await publish_domain_events(self._events, wallet.clear_events())  
return wallet.balance.amount
```

Listado 7.5 — *DepositFundsHandler*: *find_by_id* → *to_aggregate* → *act* → *upsert*

lumen/core/services/wallets/withdraw_funds_handler.py

```

from __future__ import annotations

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

from lumen.core.mappers.wallet_mapper import to_aggregate, to_entity
from lumen.core.services.wallets.event_publishing import publish_domain_events
from lumen.core.services.wallets.withdraw_funds_command import WithdrawFunds
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cQRS import CommandHandler, command_handler
from pyfly.domain import AggregateNotFound
from pyfly.data.relational.sqlalchemy import LockMode, transactional
from pyfly.eda import EventPublisher

@command_handler
@service
class WithdrawFundsHandler(CommandHandler[WithdrawFunds, int]):
    """Debit funds from an existing wallet; returns the new balance."""

    def __init__(
        self,
        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
    ) → None:
        super().__init__()
        self._repository = repository
        self._events = events
        self._session_factory = session_factory

    @transactional()
    async def do_handle(self, command: WithdrawFunds) → int: # type:
        ignore[override]
        entity = await self._repository.find_by_id(
            command.wallet_id, lock=LockMode.PESSIMISTIC_WRITE

```

```

    )
    if entity is None:
        raise AggregateNotFound("Wallet", command.wallet_id)

    wallet = to_aggregate(entity)
    wallet.withdraw(Money(amount=command.amount,
currency=wallet.currency))
    await self._repository.upsert(to_entity(wallet))

    await publish_domain_events(self._events, wallet.clear_events())
    return wallet.balance.amount

```

Listado 7.6 — *WithdrawFundsHandler*: patrón idéntico, el descubierto lo rechaza el agregado

`DepositFundsHandler` y `WithdrawFundsHandler` siguen el patrón clásico: **find** → **to_aggregate** → **act** → **to_entity** → **upsert** → **drain**. `repository.find_by_id` devuelve la fila plana `WalletEntity`, leída con un bloqueo pesimista (`LockMode.PESSIMISTIC_WRITE`) para que un cambio concurrente del mismo monedero espere a que termine esta unidad (Capítulo 5); `to_aggregate(entity)` rehidrata el objeto de dominio rico para que las invariantes del agregado estén en alcance. `Money` se construye a partir del `amount` del comando y de la moneda del *monedero* —nunca una moneda del propio comando— porque el monedero es dueño de esa invariante. Si `wallet.withdraw` rechaza (el saldo quedaría negativo), lanza `BusinessRuleViolation`, que se propaga como HTTP 422 sin una sola línea de código de manejo de errores en el manejador.

Fíjate en lo que está ausente: ningún bloque try/except, ninguna llamada de registro, ninguna configuración de trazas. Todo eso pertenece a la tubería del bus. El manejador es una expresión pura de la intención de negocio.

i LO QUE ACABA DE OCURRIR

El lado de escritura está ya completo: tres comandos y tres manejadores. Enviar `OpenWallet` crea y persiste un monedero nuevo; enviar `DepositFunds` o `WithdrawFunds` carga uno, impulsa su comportamiento de dominio y lo guarda. La pila `@command_handler` + `@service` significa que PyFly los descubre y conecta en el arranque: nunca los llamas directamente y nunca los registras a mano.

Ejecútalo: confirma que el lado de escritura funciona de principio a fin. El ejemplo de Lumen ya incluye una prueba que ejercita la ruta completa de comandos. Desde el directorio `samples/lumen`, ejecuta solo esa prueba:

terminal

```
uv run --extra dev pytest tests/test_cqrs_flow.py::test_full_wallet_lifecycle
-q
```

Listado 7.4a — Ejercitar la ruta de comandos

Deberías ver una única prueba que pasa:

```
1 passed in 0.42s
```

Esa única prueba abre un monedero, deposita 1 500 unidades menores, retira 500 y afirma que el saldo queda en 1 000, demostrando que `OpenWalletHandler`, `DepositFundsHandler` y `WithdrawFundsHandler` confirman todos a través del bus. Si ves `0 items collected`, no estás en el directorio `samples/lumen`; haz `cd` allí primero. Si ves `no handler found`, comprueba dos veces que ambos decoradores estén presentes en cada manejador: esa es la causa más común con diferencia.

El mapeador entidad↔agregado

Los manejadores de comandos no interactúan con el repositorio a través del agregado de dominio. Interactúan a través de una fila plana `WalletEntity` —la forma de persistencia que entiende el `Repository[WalletEntity, str]` del framework— y usan `wallet_mapper` para traducir entre los dos mundos:

```
# Aggregate → row (before upsert)
to_entity(wallet)      # Wallet → WalletEntity

# Row → aggregate (after find_by_id)
to_aggregate(entity)   # WalletEntity → Wallet
```

Esta separación mantiene el agregado libre de anotaciones de SQLAlchemy y el repositorio libre de lógica de dominio. El mapeador es un único módulo; cambiar el esquema de almacenamiento toca un archivo, no cada manejador.

Enviar un comando

El `CommandBus` es el único punto de entrada para todas las escrituras. La autoconfiguración de PyFly registra un `DefaultCommandBus` como singleton en el contenedor de inyección de dependencias; decláralo como argumento del constructor y el framework lo inyecta. Enviar un comando es una única llamada con `await`:

```
from pyfly.cqrs import DefaultCommandBus
from lumen.core.services.wallets.open_wallet_command import OpenWallet
from lumen.core.services.wallets.deposit_funds_command import DepositFunds
from lumen.interfaces.enums.v1.currency import Currency

wallet_id: str = await command_bus.send(
    OpenWallet(owner_id="u-1", currency=Currency.EUR)
)
balance: int = await command_bus.send(
    DepositFunds(wallet_id=wallet_id, amount=1500)
)
```

`send` es una corrutina: siempre hazle `await`. El valor de retorno es lo que devolvió `do_handle`: un ID de monedero `str` para `OpenWallet`, y el nuevo saldo como un `int` (unidades menores) para `DepositFunds` y `WithdrawFunds`. Si algo en la tubería falla —validación, autorización o el propio manejador—, la excepción se envuelve en `CommandProcessingException` y se propaga fuera de `send`, donde el manejador de errores global la mapea al código de estado HTTP apropiado.

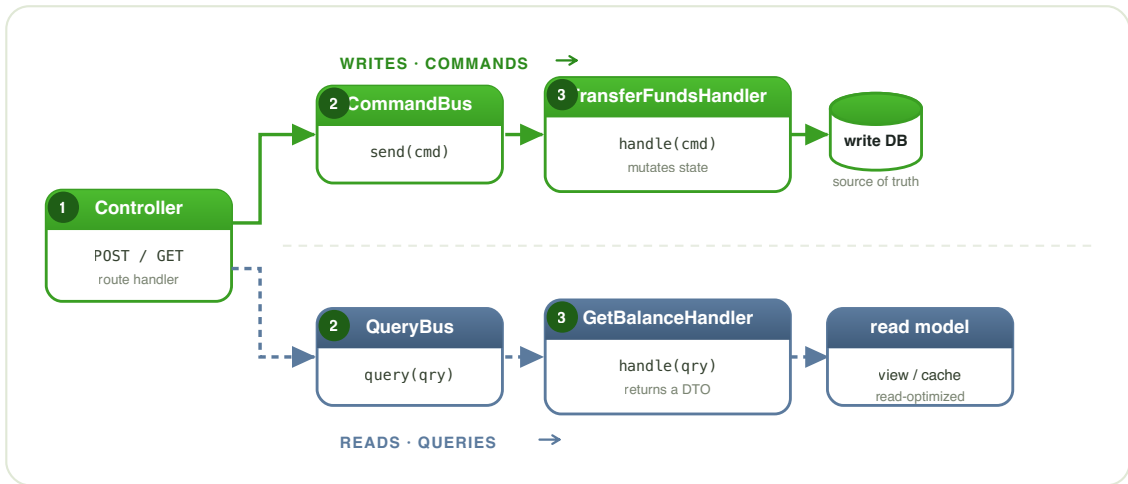


Figura 7.1 — Los comandos fluyen al modelo de escritura; las consultas, al modelo de lectura.

🍃 EQUIVALENCIA CON SPRING

`CommandBus.send(command)` es el equivalente en Python de `CommandGateway.send(command)` o `CommandGateway.sendAndWait(command)` del framework Axon. Cada clase de manejador de comandos corresponde a un método anotado con `@CommandHandler` en Axon, o a un `@MessageHandler` en el modelo `ApplicationEventPublisher` de Spring Modulith. El decorador `@command_handler` es la contrapartida de PyFly de `@CommandHandler`: registra el manejador en el registro introspeccionando el parámetro de tipo genérico, exactamente igual que Axon resuelve los métodos manejadores por el tipo del parámetro. La apilación de `@service` refleja el hecho de que en Spring cada bean `@CommandHandler` es también un `@Component` de Spring: el registro y la inyección son inseparables. El decorador `@transactional()` se corresponde directamente con `@Transactional` de Spring: ambos abren una unidad de trabajo, confirman en caso de éxito y revierten ante cualquier excepción, con las mismas reglas de propagación y de reversión. `save()` sigue el `save` de Spring Data (persiste una entidad nueva, fusiona una existente); el `upsert` de Lumen deja explícita la fusión con `session.merge`.

Consultas y manejadores de consultas

Los comandos viajan en una dirección: hacia el modelo de escritura. Las consultas son el viaje de vuelta: le piden al sistema una proyección del estado actual y esperan una respuesta, no un efecto secundario.

Una **consulta** es un dataclass congelado que hereda de `Query[R]`, donde `R` es el tipo del resultado. Como los comandos, las consultas son mensajes inmutables, pero no llevan intención de cambiar el estado. `query_bus.query(GetBalance(...))` carga datos frescos del repositorio y devuelve un DTO tipado. Las consultas no necesitan `@transactional()`: las lecturas no mutan el estado, así que no hay nada que confirmar ni revertir.

Las consultas devuelven **DTO de lectura** en lugar de agregados de dominio. La separación es deliberada. Si `GetWalletHandler` devolviera un agregado `Wallet`, la capa de API quedaría acoplada a cada campo del agregado: un cambio en el modelo de dominio podría romper silenciosamente el contrato de la API. Un modelo Pydantic `WalletDto` dedicado proyecta exactamente los campos que necesita la respuesta HTTP. ¿Añades un campo a `Wallet`? La proyección cambia solo si lo incluyes explícitamente en el DTO. ¿Eliminas un campo de `Wallet`? La proyección compila hasta que la limpies.

El lado de lectura refleja el lado de escritura paso a paso —mensaje de consulta y luego manejador de consulta—, pero con dos simplificaciones: sin `@transactional()` (nada que confirmar) y sin publicación de eventos (nada cambió).

Paso 5 — Escribe las consultas de búsqueda única. Crea `get_wallet_query.py` y `get_balance_query.py`. Ambas llevan solo un `wallet_id`; lo que difiere es lo que prometen devolver.

```
lumen/core/services/wallets/get_wallet_query.py
```

```
from __future__ import annotations

from dataclasses import dataclass

from lumen.interfaces.dtos.v1.wallet_dto import WalletDto
from pyfly.cqrs import Query

@dataclass(frozen=True)
class GetWallet(Query[WalletDto | None]):
    """Look up a wallet by its identifier."""

    wallet_id: str
```

Listado 7.7 — `GetWallet`: una consulta de búsqueda única que devuelve un `WalletDto` completo

```
lumen/core/services/wallets/get_balance_query.py
```

```
from __future__ import annotations

from dataclasses import dataclass

from lumen.interfaces.dtos.v1.balance_dto import BalanceDto
from pyfly.cqrs import Query

@dataclass(frozen=True)
class GetBalance(Query[BalanceDto | None]):
    """Look up just the balance of a wallet by its identifier."""

    wallet_id: str
```

Listado 7.8 — *GetBalance*: una consulta más ligera que devuelve solo la proyección del saldo

Ambas consultas llevan solo `wallet_id`. `GetWallet` devuelve un `WalletDto` —la representación completa, incluyendo `id`, `owner_id`, `currency`, `balance_minor`, `balance` y `created_at`—. `GetBalance` devuelve un `BalanceDto` —una proyección más ligera que omite `owner_id` y `created_at`—. Un sondeo de saldo no necesita el propietario; dejar fuera esos campos ahorra ancho de banda y evita exponer accidentalmente la titularidad de la cuenta en una respuesta que quien llama podría registrar. Mantener las dos consultas separadas significa que puedes ajustar cada una de forma independiente —caché, autorización o un almacén de lectura dedicado— sin tocar la otra.

Paso 6 — Escribe los manejadores de las consultas de búsqueda única. Los manejadores de consultas viven bajo el mismo paquete `wallets/` que los comandos. **Se aplica la misma apilación** `@query_handler + @service : @query_handler` registra la clase en el registro de manejadores; `@service` la conecta al contenedor de inyección de dependencias. Ambos decoradores son obligatorios por las mismas

razones que en los manejadores de comandos. Fíjate en cuánto más pequeños son que los manejadores de comandos: sin factoría de sesiones, sin publicador de eventos, solo el repositorio y una proyección de una línea.

`lumen/core/services/wallets/get_wallet_handler.py`

```
from __future__ import annotations

from lumen.core.mappers.wallet_mapper import entity_to_dto
from lumen.core.services.wallets.get_wallet_query import GetWallet
from lumen.interfaces.dtos.v1.wallet_dto import WalletDto
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cqrs import QueryHandler, query_handler

@query_handler
@service
class GetWalletHandler(QueryHandler[GetWallet, WalletDto | None]):
    def __init__(self, repository: WalletRepository) → None:
        super().__init__()
        self._repository = repository

    async def do_handle(self, query: GetWallet) → WalletDto | None: # type:
        ignore[override]
        entity = await self._repository.find_by_id(query.wallet_id)
        return entity_to_dto(entity) if entity is not None else None
```

Listado 7.9 — `GetWalletHandler`: `find_by_id` → `entity_to_dto` → `return`

lumen/core/services/wallets/get_balance_handler.py

```

from __future__ import annotations

from lumen.core.mappers.wallet_mapper import entity_to_balance_dto
from lumen.core.services.wallets.get_balance_query import GetBalance
from lumen.interfaces.dtos.v1.balance_dto import BalanceDto
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cqrs import QueryHandler, query_handler

@query_handler
@service
class GetBalanceHandler(QueryHandler[GetBalance, BalanceDto | None]):
    def __init__(self, repository: WalletRepository) → None:
        super().__init__()
        self._repository = repository

    async def do_handle(self, query: GetBalance) → BalanceDto | None: #
        type: ignore[override]
        entity = await self._repository.find_by_id(query.wallet_id)
        return entity_to_balance_dto(entity) if entity is not None else None

```

Listado 7.10 — *GetBalanceHandler*: vista *@projection* mediante *Mapper.project*

Ambos manejadores delegan la proyección en `wallet_mapper` —el único módulo dueño de la forma del DTO—. `entity_to_dto` rellena los seis campos de `WalletDto` directamente desde la fila. `entity_to_balance_dto` toma un camino distinto: llama a `Mapper.project(entity, BalanceView)` contra una interfaz marcada con `@projection` que declara exactamente los cuatro campos que necesita el endpoint de saldo, con una transformación registrada que calcula `balance` (unidades mayores) a partir de `balance_minor`. El mapeador copia solo esos campos declarados —un

equivalente del lado de lectura de las proyecciones por interfaz de Spring Data—. Ninguno de los dos manejadores toca el modelo Pydantic directamente; renombrar un campo toca un archivo.

Consultas paginadas y por especificación

El lado de lectura no se detiene en las búsquedas de un único recurso. Los sistemas de producción necesitan listas con metadatos de paginación y la capacidad de filtrar por predicados en tiempo de ejecución. El framework gestiona ambas cosas a través de la clase base `Repository`.

PAGEABLE Y SPECIFICATION, EN TÉRMINOS SENCILLOS

Un `Pageable` agrupa tres cosas que necesita un endpoint de listado: qué página quieres, qué tamaño tiene cada página y cómo ordenar. Una `Specification` es un filtro reutilizable y componible —piénsalo como una cláusula `WHERE` que puedes construir como objeto y combinar con `&` (and), `|` (or) y `~` (not) antes de que llegue siquiera al SQL—. Ambas provienen de la capa de datos del framework; tú no escribes el SQL.

Paso 7 — Escribe las consultas de listado y sus manejadores. `ListWallets`

envuelve un `Pageable` (número de página, tamaño, ordenación) y le pide al repositorio una porción contada, ordenada y limitada. `ListRichWallets` añade un umbral `min_minor` y lo ejecuta a través de una `Specification` componible. Construye primero los dos mensajes de consulta y luego sus manejadores.


```
lumen/core/services/wallets/list_wallets_query.py
```

```
from __future__ import annotations

from dataclasses import dataclass

from lumen.interfaces.dtos.v1.wallet_dto import WalletDto
from pyfly.data import Page, Pageable
from pyfly.cqrs import Query

@dataclass(frozen=True)
class ListWallets(Query[Page[WalletDto]]):
    """List wallets, one page at a time."""

    pageable: Pageable
```

Listado 7.11 — *ListWallets*: una consulta que lleva un *Pageable*

```
lumen/core/services/wallets/list_rich_wallets_query.py
```

```
from __future__ import annotations

from dataclasses import dataclass

from lumen.interfaces.dtos.v1.wallet_dto import WalletDto
from pyfly.data import Page, Pageable
from pyfly.cqrs import Query

@dataclass(frozen=True)
class ListRichWallets(Query[Page[WalletDto]]):
    """List wallets whose balance is at least ``min_minor``, paged."""

    min_minor: int
    pageable: Pageable
```

Listado 7.12 — *ListRichWallets*: añade un umbral de saldo para el filtrado por *Specification*

lumen/core/services/wallets/list_wallets_handler.py

```

from __future__ import annotations

from lumen.core.mappers.wallet_mapper import entity_to_dto
from lumen.core.services.wallets.list_wallets_query import ListWallets
from lumen.interfaces.dtos.v1.wallet_dto import WalletDto
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cqrs import QueryHandler, query_handler
from pyfly.data import Page

@query_handler
@service
class ListWalletsHandler(QueryHandler[ListWallets, Page[WalletDto]]):
    def __init__(self, repository: WalletRepository) → None:
        super().__init__()
        self._repository = repository

    async def do_handle(self, query: ListWallets) → Page[WalletDto]:
        # type: ignore[override]
        page = await self._repository.find_all(query.pageable)
        return page.map(entity_to_dto)

```

Listado 7.13 — ListWalletsHandler: find_all(pageable) + Page.map

lumen/core/services/wallets/list_rich_wallets_handler.py

```

from __future__ import annotations

from lumen.core.mappers.wallet_mapper import entity_to_dto
from lumen.core.services.wallets.list_rich_wallets_query import
ListRichWallets
from lumen.interfaces.dtos.v1.wallet_dto import WalletDto
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cqrs import QueryHandler, query_handler
from pyfly.data import Page

@query_handler
@service
class ListRichWalletsHandler(QueryHandler[ListRichWallets, Page[WalletDto]]):
    def __init__(self, repository: WalletRepository) → None:
        super().__init__()
        self._repository = repository

    async def do_handle(self, query: ListRichWallets) → Page[WalletDto]: #
type: ignore[override]
        page = await self._repository.find_rich(query.min_minor,
query.pageable)
        return page.map(entity_to_dto)

```

Listado 7.14 — ListRichWalletsHandler: Specification + find_all_by_spec_paged

`find_all(pageable)` se hereda de la clase base `Repository` del framework. Cuenta el total de filas, aplica la ordenación del `Pageable` y rebana con `LIMIT / OFFSET`, devolviendo un `Page[WalletEntity]` que lleva `items`, `total`, `page`, `size`, `total_pages`, `has_next` y `has_previous`. `Page.map(entity_to_dto)` transforma los elementos sin tocar los metadatos. El controlador envuelve el resultado en un `PageDto` para el cable.

`find_rich` está definido en el propio `WalletRepository` y delega en el heredado `find_all_by_spec_paged`. Construye una `Specification` —un predicado `WHERE` componible— y la pasa junto con el `Pageable`. El framework añade la cláusula `WHERE`, la ordenación y el `LIMIT / OFFSET`, y luego ejecuta una consulta de conteo para el total. El manejador llama a `repo.find_rich(query.min_minor, query.pageable)` y mapea la página exactamente como antes.

Ejecutar una consulta pasa por `QueryBus.query`:

```
from pyfly.cqrs import DefaultQueryBus
from lumen.core.services.wallets.get_balance_query import GetBalance

balance_dto = await query_bus.query(GetBalance(wallet_id="wlt-001"))
```

El valor de retorno es lo que devolvió `do_handle` —un `BalanceDto` o `None`—. `None` significa que el monedero no se encontró. El controlador es responsable de traducir eso a HTTP 404, manteniendo las preocupaciones HTTP fuera del manejador.

❗ LAS CONSULTAS DEVUELVEN NONE, NO EXCEPCIONES

Los manejadores de consultas devuelven `None` cuando el recurso no se encuentra, en lugar de lanzar `AggregateNotFound`. Esto es una convención deliberada: una consulta que no encuentra nada no es un error, es una respuesta. El controlador convierte un resultado `None` en una respuesta 404, manteniendo la preocupación HTTP fuera del manejador.

i LO QUE ACABA DE OCURRIR

Ambos lados de CQRS existen ya. Tres comandos y tres manejadores cambian el estado; cuatro consultas y cuatro manejadores lo leen. Ninguno de ellos sabe nada de HTTP, y ninguno se registra a mano: los decoradores lo hacen en el arranque. Lo único que falta es la frontera HTTP que convierte una petición web en un mensaje y el resultado de un mensaje en una respuesta web. Esa es el controlador, que construyes a continuación.

Ejecútalo: confirma que cada manejador está registrado. Antes de tocar el controlador, demuestra que el bus descubrió todos tus manejadores. Arranca la aplicación y luego pregúntale al indicador de salud de CQRS cuántos manejadores encontró. Desde el directorio `samples/lumen`:

terminal

```
uv run pyfly run --server uvicorn
```

Listado 7.14a — Arrancar Lumen

En una segunda terminal, consulta el endpoint de salud del actuador. En v26.6.110 el actuador vive en su propio puerto de gestión, **9090** por defecto, no en el 8080 de la aplicación:

terminal

```
curl -s localhost:9090/actuator/health | python -m json.tool
```

Listado 7.14b — Contar los manejadores registrados

Busca el bloque `cqrs_health_indicator`. Con todos los manejadores de comandos y consultas de este capítulo en su sitio, reporta tres manejadores de comandos y cuatro de consultas:

```
"cQRS_health_indicator": {
  "status": "UP",
  "details": {"command_handlers": 3, "query_handlers": 4}
}
```

Si un recuento es menor de lo que esperas, a un manejador le falta uno de sus dos decoradores y el registro nunca lo mapeó. Detén el servidor con `Ctrl-C` cuando termines.

EL ACTUADOR VIVE AHORA EN SU PROPIO PUERTO

En PyFly v26.6.110 la API de negocio y el actuador se ejecutan en puertos **separados**, al estilo de Spring. Tus endpoints de monedero escuchan en `pyfly.server.port` (por defecto **8080**), mientras que los endpoints del actuador y el panel de administración escuchan en `pyfly.management.server.port` (por defecto **9090**), que está abierto y sin autenticar por defecto. Lumen mantiene los valores por defecto, así que la salud está en `localhost:9090/actuator/health` y la API de monederos está en `localhost:8080/api/v1/wallets`. La exposición HTTP por defecto del actuador es `health,info`; expón más mediante `pyfly.management.endpoints.web.exposure.include`. Bloquea el puerto de gestión en producción con `pyfly.management.security.enabled: true`, o desactívalo por completo con `pyfly.management.server.port: -1`.

Conectar el bus al controlador

El controlador es la frontera HTTP del sistema. Su único trabajo es traducir una petición HTTP en un mensaje de dominio y mapear el resultado de vuelta a una respuesta HTTP. Todo lo que hay en medio pertenece al bus y a los manejadores, y esa frontera se vuelve mucho más limpia una vez que el controlador despacha comandos y consultas en lugar de llamar a métodos de servicio directamente.

Antes de CQRS, `WalletController` mantenía una referencia a `WalletApplicationService` y llamaba a sus métodos directamente. Cada vez que la interfaz del servicio cambiaba —un nuevo parámetro, un método renombrado, un tipo de retorno distinto—, el controlador tenía que cambiar también. Con CQRS, el controlador sabe una sola cosa: qué mensaje enviar.

El controlador inyecta `DefaultCommandBus` y `DefaultQueryBus` por tipo —las **clases de bus concretas**, no protocolos abstractos—. Esta es la importación correcta:

```
from pyfly.cqrs import DefaultCommandBus, DefaultQueryBus
```

¿Por qué clases concretas? La autoconfiguración de CQRS de PyFly registra exactamente una instancia de cada bus en el contenedor de inyección de dependencias. Inyectar por el tipo concreto es inequívoco: sin despacho por protocolo, y el verificador de tipos ve toda la superficie de `send` / `query`. Usar un alias de protocolo requeriría un enlace explícito en el contenedor; el tipo concreto funciona de fábrica.

Orden de las rutas: por qué los manejadores de un solo recurso se llaman `wallet_*`

El framework registra las rutas de un controlador en **orden alfabético por nombre de método**; el enrutador de Starlette aplica entonces la coincidencia «gana el primero registrado». Esto significa que un segmento literal como `/rich` debe registrarse *antes* que la variable de ruta `/ {wallet_id}`; de lo contrario, cada petición `GET /api/v1/wallets/rich` coincidiría con la ruta variable y buscaría un monedero cuyo id es la cadena `"rich"`.

Los manejadores de colección se llaman `list_wallets` y `list_rich_wallets`; los manejadores de un solo recurso se llaman `wallet_detail` y `wallet_balance`.

Alfabéticamente, `l` va antes que `w`, así que las rutas de colección (`GET /`, `GET /`

`rich`) se registran siempre por delante de las rutas parametrizadas (`GET /{wallet_id}` , `GET /{wallet_id}/balance`). Si renombras `wallet_detail` por algo que ordene antes que `list_*` , la ruta `/rich` se romperá silenciosamente.

Paso 8 — Conecta los buses al controlador. Sustituye la antigua dependencia de `WalletApplicationService` por los dos buses, y luego convierte cada endpoint en una sola línea: construye un comando o consulta a partir de la petición, despáchalo y devuelve el resultado. Aquí está el controlador completo.

lumen/web/controllers/wallet_controller.py

```

from __future__ import annotations

from lumen.core.services.wallets.deposit_funds_command import DepositFunds
from lumen.core.services.wallets.get_balance_query import GetBalance
from lumen.core.services.wallets.get_wallet_query import GetWallet
from lumen.core.services.wallets.list_rich_wallets_query import
ListRichWallets
from lumen.core.services.wallets.list_wallets_query import ListWallets
from lumen.core.services.wallets.open_wallet_command import OpenWallet
from lumen.core.services.wallets.withdraw_funds_command import WithdrawFunds
from lumen.interfaces.dtos.v1.balance_dto import BalanceDto
from lumen.interfaces.dtos.v1.deposit_request import DepositRequest
from lumen.interfaces.dtos.v1.open_wallet_request import OpenWalletRequest
from lumen.interfaces.dtos.v1.page_dto import PageDto
from lumen.interfaces.dtos.v1.wallet_dto import WalletDto
from pyfly.container import rest_controller
from pyfly.cqrs import DefaultCommandBus, DefaultQueryBus
from pyfly.data import Pageable, Sort
from pyfly.kernel import ResourceNotFoundException
from pyfly.web import (
    Body, PathVar, QueryParam, Valid,
    get_mapping, post_mapping, request_mapping,
)

#: Newest-first ordering shared by the list endpoints.
_NEWEST_FIRST = Sort.by("created_at").descending()

@rest_controller
@request_mapping("/api/v1/wallets")
class WalletController:
    """Digital-wallet REST API: open, deposit, withdraw, list, inspect."""

    def __init__(
        self, commands: DefaultCommandBus, queries: DefaultQueryBus
    ) → None:
        self._commands = commands

```

```

    self._queries = queries

# --- commands -----

@post_mapping("", status_code=201)
async def open_wallet(
    self, request: Valid[Body[OpenWalletRequest]]
) → dict[str, str]:
    wallet_id = await self._commands.send(
        OpenWallet(owner_id=request.owner_id, currency=request.currency)
    )
    return {"wallet_id": wallet_id}

@post_mapping("/{wallet_id}/deposit")
async def deposit(
    self,
    wallet_id: PathVar[str],
    request: Valid[Body[DepositRequest]],
) → dict[str, int | str]:
    balance = await self._commands.send(
        DepositFunds(wallet_id=wallet_id, amount=request.amount)
    )
    return {"wallet_id": wallet_id, "balance_minor": balance}

@post_mapping("/{wallet_id}/withdraw")
async def withdraw(
    self,
    wallet_id: PathVar[str],
    request: Valid[Body[DepositRequest]],
) → dict[str, int | str]:
    balance = await self._commands.send(
        WithdrawFunds(wallet_id=wallet_id, amount=request.amount)
    )
    return {"wallet_id": wallet_id, "balance_minor": balance}

# --- paged / specification queries (registered before /{wallet_id}) --

@get_mapping("")
async def list_wallets(

```

```

        self, page: QueryParam[int] = 1, size: QueryParam[int] = 20
    ) → PageDto[WalletDto]:
        result = await self._queries.query(
            ListWallets(pageable=Pageable.of(page, size, _NEWEST_FIRST))
        )
        return PageDto.from_page(result)

@get_mapping("/rich")
async def list_rich_wallets(
    self,
    min_minor: QueryParam[int] = 0,
    page: QueryParam[int] = 1,
    size: QueryParam[int] = 20,
) → PageDto[WalletDto]:
    result = await self._queries.query(
        ListRichWallets(
            min_minor=min_minor,
            pageable=Pageable.of(page, size, _NEWEST_FIRST),
        )
    )
    return PageDto.from_page(result)

# --- single-wallet queries (named wallet_* so they sort after list_*) -

@get_mapping("/{wallet_id}")
async def wallet_detail(self, wallet_id: PathVar[str]) → WalletDto:
    result = await self._queries.query(GetWallet(wallet_id=wallet_id))
    if result is None:
        raise ResourceNotFoundException(
            f"Wallet {wallet_id!r} not found",
            code="WALLET_NOT_FOUND",
            context={"wallet_id": wallet_id},
        )
    return result

@get_mapping("/{wallet_id}/balance")
async def wallet_balance(self, wallet_id: PathVar[str]) → BalanceDto:
    result = await self._queries.query(
        GetBalance(wallet_id=wallet_id)
    )

```

```

    )
    if result is None:
        raise ResourceNotFoundException(
            f"Wallet {wallet_id!r} not found",
            code="WALLET_NOT_FOUND",
            context={"wallet_id": wallet_id},
        )
    return result

```

Listado 7.15 — *WalletController: DefaultCommandBus + DefaultQueryBus + endpoints de listado paginado*

Compara el constructor con su forma anterior a CQRS. Antes, el controlador tomaba `WalletApplicationService` —una clase concreta cuyas firmas de método filtraban lógica de negocio hacia la capa HTTP—. Ahora toma `DefaultCommandBus` y `DefaultQueryBus` —dos canales opacos—. El controlador sabe *qué* enviar; no sabe nada sobre *cómo* se procesa el mensaje.

Mira `open_wallet`. Antes, llamaba a `self._service.open_wallet(owner_id=..., currency=...)` —un contrato posicional que se rompe cada vez que el servicio gana un nuevo parámetro—. Ahora construye `OpenWallet(owner_id=request.owner_id, currency=request.currency)` —un objeto con nombre e inmutable cuyos campos son su propia API—. ¿Añades un campo al comando? El controlador permanece igual hasta que decidas rellenarlo.

Los DTO de petición (`OpenWalletRequest`, `DepositRequest`) son modelos Pydantic en `lumen/interfaces/dtos/v1/`. `OpenWalletRequest` valida la longitud de `owner_id` y restringe `currency` al enum `Currency`. `DepositRequest` lo comparten tanto el endpoint de depósito como el de retirada: ambos mueven un `amount` positivo en la propia moneda del monedero. Las restricciones a nivel de campo en esos DTO las impone `Valid[Body[...]]` antes incluso de que se llame al manejador.

Los endpoints de listado paginado (`list_wallets` , `list_rich_wallets`) construyen un `Pageable` a partir de los parámetros de la cadena de consulta, despachan la consulta a través del bus y envuelven el `Page[WalletDto]` resultante en un `PageDto` para el cable. `PageDto` es un modelo Pydantic que refleja todos los campos de metadatos de `Page` — `total` , `total_pages` , `has_next` , `has_previous` —, de modo que los clientes obtienen sobres de paginación consistentes sin un serializador personalizado.

Los métodos `wallet_detail` y `wallet_balance` muestran la única preocupación HTTP que queda en el controlador: traducir un resultado de consulta `None` a un 404 mediante `ResourceNotFoundException` . Ese mapeo corresponde aquí porque 404 es un código de estado HTTP y el manejador deliberadamente no tiene conocimiento de HTTP. Los tipos de retorno se declaran como `WalletDto` y `BalanceDto` —modelos Pydantic que el framework serializa a JSON automáticamente—.

💡 DEJA QUE EL BUS LANCE

No necesitas capturar `CommandProcessingException` ni `QueryProcessingException` en el controlador a menos que quieras personalizar la forma del error. El manejador de excepciones global mapea `AggregateNotFound` a 404 y `BusinessRuleViolation` a 422 —igual que antes—. Las excepciones del bus propagan esas originales de forma transparente.

Ejecútalo: recorre la ruta HTTP completa. La rebanada vertical está completa: petición HTTP → comando/consulta → bus → manejador → dominio → repositorio → respuesta. Demuéstralo desde fuera. Arranca la aplicación (`uv run pyfly run --server uvicorn`) y luego, en una segunda terminal, abre un monedero:

terminal

```
curl -s -X POST localhost:8080/api/v1/wallets \
  -H 'content-type: application/json' \
  -d '{"owner_id":"u-1","currency":"EUR"}'
```

Listado 7.15a — Abrir un monedero por HTTP

El endpoint `open_wallet` despacha `OpenWallet` y devuelve el id generado:

```
{"wallet_id": "wlt-c5bbb2a7-dd49-4321-932e-e4c6bfa5cc2c"}
```

Copia ese id, deposita en él y luego vuelve a leer el saldo:

terminal

```
curl -s -X POST localhost:8080/api/v1/wallets/
wlt-c5bbb2a7-dd49-4321-932e-e4c6bfa5cc2c/deposit \
  -H 'content-type: application/json' -d '{"amount":1500}'

curl -s localhost:8080/api/v1/wallets/
wlt-c5bbb2a7-dd49-4321-932e-e4c6bfa5cc2c/balance
```

Listado 7.15b — Depositar y luego leer el saldo

El depósito devuelve el nuevo saldo en unidades menores; la consulta de saldo devuelve el `BalanceDto`:

```
{"wallet_id": "wlt-c5bbb2a7-dd49-4321-932e-e4c6bfa5cc2c", "balance_minor":
1500}
{"wallet_id": "wlt-c5bbb2a7-dd49-4321-932e-e4c6bfa5cc2c", "balance_minor": 150
0, "balance": 15.0}
```

Por último, confirma que la decisión de orden de rutas de antes da sus frutos: lista los monederos y el subconjunto «ricos»:

terminal

```
curl -s 'localhost:8080/api/v1/wallets?page=1&size=20'
curl -s 'localhost:8080/api/v1/wallets/rich?min_minor=1000'
```

Listado 7.15c — Ambas rutas de listado resuelven correctamente

Ambas devuelven un sobre `PageDto` (`items` , `total` , `total_pages` , `has_next` , `has_previous`). La llamada `/rich` resuelve a `list_rich_wallets` , *no* a `wallet_detail` buscando un monedero cuyo id sea la cadena literal `"rich"` , precisamente porque `list_*` ordena antes que `wallet_*` . Si `/rich` alguna vez devuelve un 404, ese orden se ha roto; revisa la regla de nombres de método de arriba.

 USA UN ID REAL

El id de monedero de arriba es ilustrativo: el tuyo será distinto en cada llamada a `open_wallet` . Pega el id que devuelva tu propio `POST /api/v1/wallets` en las URL de depósito y saldo, o recibirás un 404.

La tubería del manejador

Una sola llamada a `send` o `query` desencadena más que solo el manejador. Entender la tubería te dice dónde poner cada preocupación transversal y, lo que es igual de importante, dónde *no* ponerla.

❗ ¿QUÉ ES UNA «TUBERÍA»?

Una **tubería** es simplemente una secuencia fija de pasos que el bus ejecuta alrededor de tu manejador, como una cadena de montaje. Tu mensaje entra por un extremo, pasa por la validación y la autorización, se maneja y (para los comandos) se publican sus eventos, antes de que el resultado salga por el otro extremo. Tú escribes solo el paso del medio (`do_handle()`); el bus es dueño del resto, de forma idéntica para cada mensaje.

La tubería se define una sola vez, en el bus, y se aplica uniformemente a cada manejador. Nunca escribes lógica de tubería dentro de un manejador. El orden es estricto:

Paso	Dónde se define	Aplica a	Resultado del fallo
Validación de precondiciones de negocio	gancho <code>validate()</code> en el mensaje	Comandos + Consultas	<code>CqrsValidationException</code> (HTTP 422)
Autorización	gancho <code>authorize()</code> en el mensaje	Comandos + Consultas	<code>AuthorizationException</code> (HTTP 403)
Ejecución del manejador	<code>do_handle()</code>	Comandos + Consultas	Excepciones de dominio (4xx/5xx)
Publicación de eventos de dominio	tubería del bus (post-manejador)	Solo comandos	—
Limpieza del ID de correlación	tubería del bus (bloque finally)	Comandos + Consultas	—

Validación

Sin un paso de validación estructurado, cada manejador empezaría con sus propias cláusulas de guarda: comprobar que este campo no esté en blanco, comprobar que aquel importe sea positivo. Esa lógica se duplicaría entre manejadores y se probaría solo a través de rutas de integración. Centralizar la validación en el propio mensaje resuelve ambos problemas.

El bus invoca `validate()` antes de buscar el manejador. Si la validación falla, el bus lanza `CqrsValidationException` sin llegar nunca al manejador.

El gancho de validación es también el lugar adecuado para precondiciones entre campos que se pueden conocer únicamente a partir de los campos —demasiado simples para el agregado de dominio, demasiado específicas de la aplicación para el modelo de petición—:

```
@dataclass(frozen=True)
class DepositFunds(Command[int]):
    wallet_id: str
    amount: int

    async def validate(self) → ValidationResult:
        if not self.wallet_id.strip():
            return ValidationResult.failure(
                "wallet_id", "Wallet id is required"
            )
        if self.amount ≤ 0:
            return ValidationResult.failure(
                "amount", "Deposit amount must be > 0"
            )
        return ValidationResult.success()
```

Autorización

Una vez que un mensaje es estructuralmente válido, el bus pregunta: ¿está *permitido* a quien llama realizar esta operación? La autorización responde antes de que ocurra cualquier acceso a la base de datos —más eficiente y más seguro, ya que nunca cargas datos sensibles solo para descartarlos porque quien llama carecía de permiso—.

Tanto los comandos como las consultas exponen un gancho `authorize()`. Devuelve `AuthorizationResult.success()` para permitir la ejecución, o `AuthorizationResult.failure(resource, message)` para denegarla. El bus lanza `AuthorizationException` al denegar, mapeando a HTTP 403 mediante el manejador de errores global.

Una regla práctica limpia: usa `authorize()` en el comando para comprobaciones a **nivel de operación** —quién tiene permiso para llamar a este comando en absoluto— y deja las decisiones a **nivel de recurso** (¿puede quien llama acceder a *este monedero concreto*?) al manejador, que tiene el agregado cargado en alcance:

lumen/cqrs/commands_auth.py

```

from __future__ import annotations
from dataclasses import dataclass

from pyfly.cqrs.authorization.types import AuthorizationResult
from pyfly.cqrs import Command

@dataclass(frozen=True)
class CloseWallet(Command[None]):
    """Close a wallet. Only internal service accounts may do this."""
    wallet_id: str
    requested_by: str

    async def authorize(self) → AuthorizationResult:
        internal_accounts = {"ops-service", "compliance-bot"}
        if self.requested_by not in internal_accounts:
            return AuthorizationResult.failure(
                "wallet",
                "Only internal service accounts may close wallets",
            )
        return AuthorizationResult.success()

```

Listado 7.16 — Gancho de autorización en un comando

`CloseWallet.authorize` comprueba un conjunto conocido de cuentas de servicio internas. Si `requested_by` no está en el conjunto, la autorización falla antes de que se llame al manejador. El conjunto normalmente provendría de un valor de configuración o de un claim de token inyectado en la frontera del controlador —aquí está codificado a fuego por legibilidad—. El punto clave es que la comprobación vive dentro del comando, no dispersa por el código del manejador.

Trazas distribuidas

Cuando una única petición HTTP desencadena múltiples comandos —y cada comando puede llamar a servicios aguas abajo—, necesitas una forma de coser juntos todos los registros y spans. Eso es lo que proporciona `CorrelationContext`.

Ambos buses establecen un ID de correlación al inicio de cada ejecución de la tubería. Si el mensaje ya lleva un ID (establecido mediante

`command.set_correlation_id(id)`), se usa ese ID; de lo contrario, se genera un nuevo UUID. El ID anterior siempre se restaura en un bloque `finally`, de modo que los despachos de comandos anidados dentro de la misma petición no pisotean la traza externa.

`CorrelationContext` se propaga a través de las cadenas de `await` mediante las `contextvars` de Python —no hay necesidad de pasar el ID manualmente por cada argumento de función—. Para la propagación entre servicios, serializa el contexto en las cabeceras salientes y restáuralo en el lado receptor:

```
from pyfly.cqrs.tracing.correlation import CorrelationContext

# On the sending side
headers = CorrelationContext.create_context_headers()
# {"X-Correlation-ID": "...", "X-Trace-ID": "...", "X-Span-ID": "..."}

# On the receiving side
CorrelationContext.extract_context_from_headers(headers)
```

Las tres cabeceras — `X-Correlation-ID`, `X-Trace-ID` y `X-Span-ID` — siguen la nomenclatura de W3C Trace Context, así que son compatibles con infraestructura instrumentada con OpenTelemetry de fábrica.

🔔 DÓNDE PONER LA LÓGICA TRANSVERSAL

La tubería del bus es el hogar adecuado para las preocupaciones que se aplican a *todas* las operaciones: validación, autorización, trazas y métricas. El manejador es el hogar adecuado para las preocupaciones específicas de *una* operación: cargar el agregado, impulsar el comportamiento, guardar, drenar eventos. Si te encuentras añadiendo un `try/except` a cada manejador, o copiando la misma comprobación de precondition en varios manejadores, eso pertenece a la tubería —ya sea como un gancho `validate()` en el comando o como un servicio a nivel de bus—. La tubería escala uniformemente; el código repetitivo del manejador, no.

✓ Lo que construiste

La Parte II está completa. Lumen tiene ahora una rebanada vertical completa desde HTTP hasta el dominio y de vuelta —una construida sobre decisiones arquitectónicas que escalarán sin reescribir—.

En el Capítulo 5 le diste persistencia al sistema: un `WalletRepository` que subclasifica `Repository[WalletEntity, str]` —el repositorio genérico estilo Spring Data del framework, que proporciona `find_by_id`, `find_all(pageable)`, `find_all_by_spec_paged` y más de fábrica, en el que cada llamada se ejecuta en la unidad de trabajo de quien llama o en una breve propia—. En el Capítulo 6 promoviste el monedero a un agregado DDD propiamente dicho: `Money` como objeto de valor inmutable, `Wallet(AggregateRoot[str])` como frontera de consistencia que impone las invariantes de descubierto, coincidencia de moneda e importe positivo, con los eventos de dominio `WalletOpened`, `FundsDeposited` y `FundsWithdrawn` bufferizados en el agregado y drenados al bus de eventos tras un guardado exitoso.

En este capítulo separaste el modelo de escritura del modelo de lectura. `OpenWallet`, `DepositFunds` y `WithdrawFunds` son mensajes de comando congelados y validados que fluyen a través de `DefaultCommandBus` —una tubería que ejecuta validación, autorización, ejecución del manejador, publicación de eventos de dominio y trazas distribuidas automáticamente para cada comando—. Cada manejador de comandos lleva `@transactional()` en `do_handle`: el decorador liga una unidad de trabajo en el origen de datos de `self._session_factory`, cada llamada al repositorio se une a ella, y confirma en caso de éxito y revierte en caso de fallo. Los manejadores de ingreso y retirada leen el monedero con un bloqueo pesimista, así que los cambios concurrentes de un mismo monedero hacen cola en lugar de pisarse. La persistencia pasa por `repository.upsert` —respaldado por `session.merge`—, de modo que INSERT y UPDATE comparten una única ruta de código con clave en el propio id del agregado.

`GetWallet` y `GetBalance` son mensajes de consulta que fluyen a través de `DefaultQueryBus` —la misma tubería sin el paso de publicación de eventos, y sin `@transactional()` porque las lecturas no confirman—. `GetBalanceHandler` proyecta a través de una interfaz `BalanceView` marcada con `@projection` y `Mapper.project`, copiando solo los campos declarados y aplicando una transformación de unidades mayores registrada. `ListWallets` y `ListRichWallets` completan el lado de consultas: `find_all(pageable)` devuelve un `Page[WalletEntity]` contado, ordenado y limitado por offset; `find_all_by_spec_paged` ejecuta un predicado `Specification` componible sobre la misma maquinaria de paginación. Ambos usan `Page.map(entity_to_dto)` para proyectar los elementos sin tocar los metadatos.

Cada manejador lleva la pila `@command_handler` + `@service` (o `@query_handler` + `@service`): el primer decorador registra la clase introspeccionando su argumento de tipo genérico; el segundo la conecta al contenedor de inyección de dependencias para que las dependencias del constructor se inyecten automáticamente.

`WalletController` ya no sabe nada de la capa de servicio. Inyecta `DefaultCommandBus` y `DefaultQueryBus`, construye un comando o consulta a partir de la petición HTTP, lo despacha y o bien devuelve el resultado o bien lanza una excepción de dominio. Los métodos manejadores de un solo recurso se llaman `wallet_detail` y `wallet_balance` —una elección deliberada para que ordenen alfabéticamente *después* de los métodos de colección `list_wallets` y `list_rich_wallets`, garantizando que el segmento literal `/rich` se registre antes que la ruta variable `/ {wallet_id}` —.

Añadir un nuevo comando significa ahora tres cosas: definir un dataclass congelado, implementar un `do_handle` decorado con `@command_handler` + `@service` y anotado con `@transactional()`, y añadir un endpoint que llame a `self._commands.send`. La tubería se aplica automáticamente.

Ejecútalo: todo el capítulo, en un solo comando. Desde el directorio `samples/lumen`, ejecuta las pruebas del flujo CQRS una última vez para confirmar que cada pieza que construiste este capítulo sigue encajando:

terminal

```
uv run --extra dev pytest tests/test_cqrs_flow.py -q
```

Listado 7.17 — Verificar la rebanada CQRS completa

Pasan los cinco escenarios: el ciclo de vida de camino feliz, la consulta de no encontrado, el descubierto rechazado, el depósito no positivo rechazado y el depósito a un monedero desconocido rechazado:

```
5 passed in 0.61s
```

Vale la pena detenerse en esos últimos cuatro casos: cada uno ejercita la *tubería*, no el manejador. El descubierto lo rechaza el agregado y aflora como `CommandProcessingException`; el depósito no positivo nunca llega a un manejador en absoluto porque `validate()` lo rechaza primero. No escribiste ni una sola línea de código de manejo de errores para conseguir nada de eso.

Pruébalo tú mismo

- 1. Traza el ciclo de vida completo en la batería de pruebas.** Abre `samples/lumen/tests/test_cqrs_flow.py` y ejecútalo contra una base de datos real usando Testcontainers (Capítulo 11). La prueba `test_full_wallet_lifecycle` abre un monedero, deposita 1 500 unidades menores, retira 500 y luego consulta tanto `GetWallet` como `GetBalance`. Recórrela con un depurador: confirma que `wallet.clear_events()` drena los eventos `FundsDeposited` y `FundsWithdrawn` después de cada llamada a `upsert`, y que `GetWallet` devuelve un `WalletDto` con `balance_minor = 1000` y `balance = 10.0`.
- 2. Observa `upsert` frente a `save`.** En una prueba, sustituye `upsert` por `save` en `DepositFundsHandler` e ingresa dos veces en el mismo monedero: desde la v26.09.08 `save()` fusiona una entidad cuya clave ya está fijada, así que ambos depósitos se confirman (hasta la v26.09.07 el segundo lanzaba `IntegrityError`). Restaura `upsert`, abre `WalletRepository.upsert` y traza cómo `session.merge` resuelve de la misma forma la clave primaria existente.
- 3. Añade una consulta `ListByOwner`.** Define `ListByOwner(Query[list[WalletDto]])` con un campo `owner_id: str`. Implementa `ListByOwnerHandler` —decorado con `@query_handler` + `@service` — que llame a `WalletRepository.find_by_owner_id(query.owner_id)` (el stub de

consulta derivada ya existe) y mapee la lista de resultados con `entity_to_dto`. Añade un endpoint `GET /api/v1/wallets/by-owner/{owner_id}` a `WalletController`. Asegúrate de que el nombre del nuevo método de endpoint ordene antes que `wallet_detail` para que Starlette haga coincidir primero el segmento literal `/by-owner/...`.

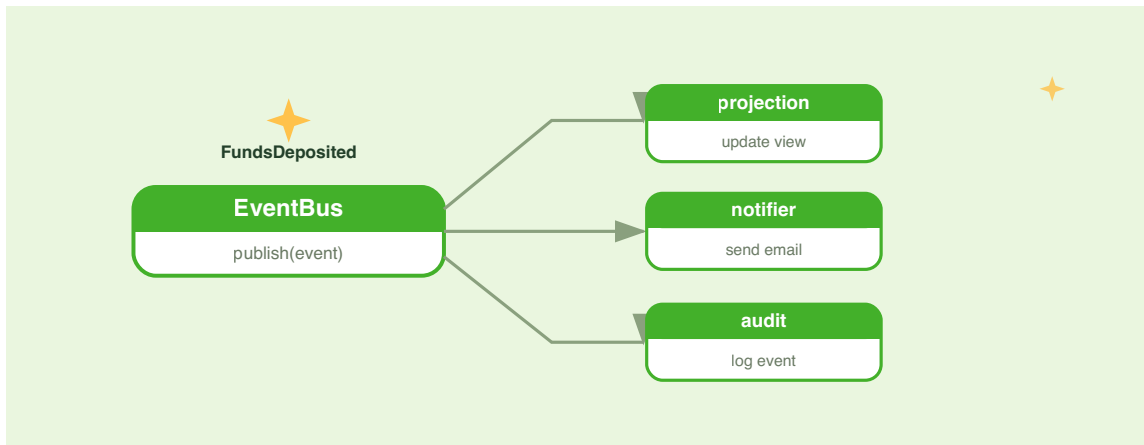
- Añade autorización a `WithdrawFunds`**. Extiende `WithdrawFunds` con un campo `initiated_by: str`. Sobrescribe `authorize()` para que devuelva `AuthorizationResult.failure("withdraw", "Initiator is required")` cuando `initiated_by` esté en blanco, y `AuthorizationResult.success()` en caso contrario. Actualiza `WithdrawFundsHandler.do_handle` para registrar `command.initiated_by` en la carga útil del evento `FundsWithdrawn`. Escribe una prueba que llame a `await WithdrawFunds(wallet_id="wlt-1", amount=100, initiated_by="").authorize()` y afirme que el resultado deniega la autorización.

PARTE III

Arquitectura Orientada a Eventos

CAPÍTULO 8

Eventos de dominio y arquitectura orientada a eventos



El monedero (wallet) de Lumen guarda correctamente, valida con rigor y despacha cada escritura a través de un comando tipado. Pero fíjate en lo que hacen los manejadores de comandos después de `repo.add`: nada. Los eventos de dominio que la raíz de agregado `Wallet` acumula en su buffer — `WalletOpened`, `FundsDeposited`, `FundsWithdrawn` — se vacían y se descartan. El pipeline del bus que el Capítulo 7 prometió que "publicaría eventos de dominio" todavía no tiene adónde enviarlos.

La carencia importa en la práctica. Lumen necesita un modelo de lectura del saldo que se mantenga sincronizado sin recargar el agregado en cada consulta, una notificación de bienvenida cuando se abre un monedero nuevo y un rastro de auditoría inmutable de cada movimiento financiero por cumplimiento normativo. Las tres funcionalidades dependen de saber *que algo ocurrió* —no quién lo solicitó ni cómo se gestionó.

Los **eventos de dominio** son la respuesta. En lugar de que un manejador de comandos llame directamente al servicio de notificaciones —o de acoplar el registro de auditoría al repositorio— publicas el evento: un registro conciso, con marca de tiempo e inmutable de un hecho. Cada parte interesada se suscribe de forma independiente. El manejador que guardó el monedero no necesita saber qué hace el auditor, ni siquiera que existe un auditor. Puedes añadir un nuevo suscriptor meses después sin tocar una sola línea del código de los manejadores existentes.

Este capítulo construye el lado reactivo de la arquitectura de Lumen. Conectarás `EventPublisher` a los manejadores de comandos, introducirás el puente `publish_domain_events` que vacía el buffer del agregado y reenvía cada evento al bus, y escribirás un `WalletAuditListener` que se suscribe usando `@event_listener` y mantiene dos proyecciones en memoria: un rastro de auditoría inmutable y un total de depósitos acumulado. Al final del capítulo, la ruta de escritura y la infraestructura de lectura quedarán totalmente desacopladas: cada lado evoluciona sin que el otro se entere.

Lo construiremos gradualmente, una pequeña pieza cada vez. Cada funcionalidad viene con un recorrido numerado, el comando exacto que ejecutar y la salida que deberías esperar ver. Si has seguido el hilo desde el Capítulo 7, ya tienes los manejadores de comandos del monedero y el agregado `Wallet` en su sitio; este capítulo está verificado con PyFly v26.6.110 y el ejemplo de Lumen bajo `samples/lumen`.

📌 NOTA: LA JERGA, EN LENGUAJE LLANO

En este capítulo se repite un puñado de términos. Un **evento de dominio** es un registro pequeño y congelado de un hecho de negocio que ya ocurrió —por ejemplo, "se depositaron fondos". Un **publicador** es lo que anuncia esos hechos; un **oyente** (o *suscriptor*) es código que reacciona a ellos. Un **bus** es la centralita intermedia que lleva cada anuncio desde el publicador hasta todos los oyentes interesados. Una **proyección** es un modelo de lectura que un oyente va construyendo a partir de eventos —aquí, un rastro de auditoría y un total acumulado. Un **puerto** es una interfaz (un `Protocol` de Python) de la que depende tu código para que la implementación concreta que hay detrás pueda intercambiarse sin cambiar a quien la invoca. Ten presentes estos cinco; el resto del capítulo trata mayormente de conectarlos.

Dos tipos de eventos

Antes de tocar nada de código, conviene ser precisos sobre qué significa "evento" en PyFly. El framework usa la palabra para dos cosas distintas, y confundirlas lleva al bus equivocado, a la API de suscripción equivocada y a sutiles sorpresas en tiempo de ejecución.

Los **eventos de aplicación** (`pyfly.context.events`) son notificaciones del ciclo de vida del framework. `ContextRefreshedEvent` se dispara cuando el contenedor de inyección de dependencias termina de cablear; `ApplicationReadyEvent` se dispara cuando el servidor HTTP empieza a aceptar conexiones; `ContextClosedEvent` se dispara durante el apagado. El `ApplicationEventBus` los despacha a los suscriptores emparejados por el tipo de clase de Python —son fontanería de infraestructura para el arranque, deliberadamente separada de cualquier concepto de negocio.

Los **eventos de dominio** (`pyfly.eda`) son hechos a nivel de negocio: *se abrió un monedero, se depositaron fondos, se completó una transferencia*. El puerto `EventPublisher` envuelve cada carga útil en un `EventEnvelope` y la enruta por el nombre de la clase del evento de dominio — `"WalletOpened"`, `"FundsDeposited"`, `"FundsWithdrawn"` — de modo que los oyentes se suscriben a hechos de negocio con nombre y no a detalles de implementación. Los eventos de dominio son el tema de este capítulo.

La distinción determina lo que puedes hacer con cada tipo. El `ApplicationEventBus` despacha a invocables indexados por clase; `InMemoryEventBus` enruta por nombre de clase y puede intercambiarse por un adaptador respaldado por Kafka sin tocar el código de los suscriptores. La regla es sencilla: usa eventos de ciclo de vida para el arranque de infraestructura y eventos de dominio para todo lo que tenga significado de negocio.

NOTA: LOS EVENTOS DE APLICACIÓN SIGUEN SIENDO ÚTILES

Si necesitas precalentar una caché en cuanto la aplicación esté lista, `@app_event_listener` sobre `ApplicationReadyEvent` es la herramienta adecuada. Los dos sistemas conviven; puedes usar ambos en el mismo servicio.

Publicar eventos

El puerto `EventPublisher`

La primera pregunta que suele hacer un nuevo miembro del equipo es: "¿qué clase importo para disparar un evento?". La respuesta, deliberadamente, no es una clase: es un protocolo. `EventPublisher` es un **puerto** en el sentido de la arquitectura hexagonal: cualquier código que necesite publicar un evento depende de esta interfaz,

y la implementación del bus se inyecta desde fuera. Esa decisión de diseño es lo que te permite ejecutar `InMemoryEventBus` localmente hoy e intercambiarlo por un adaptador de Kafka en producción sin tocar un solo manejador.

El protocolo expone dos métodos:

```
from pyfly.eda import EventPublisher

class EventPublisher(Protocol):
    def subscribe(self, event_type_pattern: str, handler: EventHandler) → None: ...

    async def publish(
        self,
        destination: str,
        event_type: str,
        payload: dict,
        headers: dict[str, str] | None = None,
    ) → None: ...
```

`publish` envuelve tus datos en un `EventEnvelope` antes de la entrega —nunca construyes el sobre tú mismo. `subscribe` registra manejadores programáticamente, aunque en la práctica usarás el decorador `@event_listener` en su lugar, porque permite que el `ApplicationContext` cablee las suscripciones automáticamente en el arranque.

El bean del bus solo existe cuando `pyfly.eda.provider` está configurado. Para Lumen, `pyfly.yaml` lo establece en `memory`:

pyfly.yaml

```
pyfly:
  eda:
    provider: memory
# ... other keys omitted for brevity
```

Listado 8.0 — Activar el bus EDA en memoria en `pyfly.yaml`

Sin esta línea el bean `EventPublisher` no se registra y cualquier manejador que declare `events: EventPublisher` en su constructor fallará al arrancar.

Encendamos ese bus antes de usarlo.

Paso 1 — Abre el `pyfly.yaml` de Lumen. Localiza el bloque de nivel superior `pyfly:`. Ya verás claves como `app`, `server` y `data` de los capítulos anteriores.

Paso 2 — Añade el bloque `eda`. Inserta las dos líneas que se muestran en el Listado 8.0 como hijo de `pyfly:`, al mismo nivel de indentación que `server:` y `data:`. La indentación es significativa en YAML, así que alinéalas con exactitud.

Paso 3 — Guarda el archivo. Esa es toda la configuración que necesita el bus en memoria —sin broker, sin cadena de conexión, sin dependencia adicional.

❗ NOTA: EJECÚTALO

Arranca la aplicación y confirma que levanta limpiamente con el bus registrado:

```
uv run pyfly run --server uvicorn
```

En el log de arranque deberías ver la aplicación levantar en el puerto de app por defecto y los endpoints de gestión en el puerto de gestión separado:

```
INFO starting_application name=lumen version=1.0.0
INFO uvicorn running on http://127.0.0.1:8080
INFO management endpoints on http://127.0.0.1:9090
```

Si en cambio el proceso termina con un error que menciona que no se pudo resolver una dependencia `EventPublisher`, el bloque `eda` falta o está mal indentado —vuelve al Paso 2.

❗ NOTA: DÓNDE VIVEN LA APP Y EL PANEL

A partir de la v26.6.110, la aplicación escucha en `pyfly.server.port` (por defecto `8080`), mientras que el actuador y el panel de administración corren en un puerto de gestión **separado**, `pyfly.management.server.port` (por defecto `9090`). El puerto de gestión está abierto y sin autenticar por defecto; establece `pyfly.management.security.enabled: true` para protegerlo, o `pyfly.management.server.port: -1` para desactivar por completo los endpoints de gestión. Nada de esto afecta al bus EDA —es puramente en proceso—, pero conviene saber qué puerto es cuál cuando empieces a jugar con la app en ejecución más adelante en el capítulo.

Qué acaba de pasar. Has cambiado un único interruptor de configuración y PyFly ha registrado un bean `EventPublisher` por ti. A partir de ahora, cualquier `@service` que pida `events: EventPublisher` en su constructor recibe el bus en memoria automáticamente. Todavía no se publica nada —eso viene a continuación— pero la fontanería está en su sitio.

El EventEnvelope

Cada evento de dominio llega a sus oyentes envuelto en un `EventEnvelope`. Piénsalo como la capa de metadatos que transforma un diccionario de Python pelado en un hecho trazable, auditable y de primera clase. Es una dataclass congelada —inmutable una vez creada— que empareja la carga útil con el contexto que cada oyente necesita.

Campo	Tipo	Por defecto	Descripción
<code>event_type</code>	<code>str</code>	obligatorio	El nombre de la clase del evento de dominio, p. ej. <code>"FundsDeposited"</code> . Se usa para el enrutamiento.
<code>payload</code>	<code>dict[str, Any]</code>	obligatorio	Los datos del evento.
<code>destination</code>	<code>str</code>	obligatorio	Canal o topic lógico, p. ej. <code>"wallet.events"</code> .
<code>event_id</code>	<code>str</code>	UUID automático	ID único de esta instancia de evento.
<code>timestamp</code>	<code>datetime</code>	<code>datetime.now(UTC)</code>	Hora de creación en UTC.
<code>headers</code>	<code>dict[str, str]</code>	<code>{}</code>	Metadatos arbitrarios: IDs de correlación, contexto de traza, etc.

Tres campos merecen atención especial. `event_id` es un UUID estable generado por el bus en el momento de la publicación —tu **clave de idempotencia** para semántica de exactamente una vez, disponible en cada oyente sin trabajo adicional. `timestamp` registra cuándo se observó el hecho, no cuándo lo procesa el oyente, así que se mantiene exacto incluso si un oyente corre con retraso. `headers` transporta preocupaciones transversales como IDs de traza distribuida —metadatos que no tienen nada que ver con la carga útil de negocio pero que importan enormemente para la observabilidad. Como el sobre está congelado, los manejadores pueden pasarlo con seguridad a través de fronteras asíncronas sin copias defensivas.

`event_type` contiene el **nombre de clase** del evento de dominio — `"WalletOpened"`, `"FundsDeposited"` o `"FundsWithdrawn"` — no una ruta separada por puntos. Los oyentes se suscriben por esos mismos nombres de clase, de modo que el contrato de suscripción lo define el modelo de dominio, no convenciones de cadena inventadas fuera de él.

Qué acaba de pasar. No te eche para atrás la tabla de seis campos: en el código del día a día solo tocas dos de ellos. Cuando publicas proporcionas `event_type`, `payload` y `destination`; el bus rellena `event_id`, `timestamp` y `headers` por ti. Cuando reaccionas, lees `envelope.event_type` para saber *qué* ocurrió y `envelope.payload` para conocer los detalles. Los otros tres campos están ahí cuando los necesitas (idempotencia, ordenación, trazas) y discretamente fuera del camino cuando no.

Los eventos de dominio en el agregado Wallet

El agregado `Wallet` lanza eventos de dominio tipados, como dataclasses congeladas. El nombre de clase de cada evento se convierte en su `event_type` de enrutamiento en el bus:

lumen/models/entities/v1/wallet_entity.py

```

from __future__ import annotations

from dataclasses import dataclass
from datetime import UTC, datetime

from lumen.interfaces.enums.v1.currency import Currency
from lumen.models.entities.v1.money import Money
from pyfly.domain import AggregateRoot, BusinessRuleViolation, DomainEvent

@dataclass(frozen=True)
class WalletOpened(DomainEvent):
    wallet_id: str = ""
    owner_id: str = ""
    currency: str = ""

@dataclass(frozen=True)
class FundsDeposited(DomainEvent):
    wallet_id: str = ""
    amount: int = 0
    currency: str = ""
    balance: int = 0

@dataclass(frozen=True)
class FundsWithdrawn(DomainEvent):
    wallet_id: str = ""
    amount: int = 0
    currency: str = ""
    balance: int = 0

class Wallet(AggregateRoot[str]):
    """Wallet aggregate root - owns the ``balance ≥ 0`` invariant."""

    def deposit(self, amount: Money) → None:

```

```

"""Credit *amount*; raises FundsDeposited."""
self._assert_currency(amount)
self.balance = self.balance.add(amount)
self.raise_event(
    FundsDeposited(
        wallet_id=self.id,
        amount=amount.amount,
        currency=amount.currency.value,
        balance=self.balance.amount,
    )
)
# ... open() and withdraw() follow the same pattern

```

Listado 8.1 — Eventos de dominio lanzados por el agregado Wallet

`DomainEvent` es una dataclass congelada base. Su propiedad `event_type` devuelve `type(self).__name__` —el nombre de clase— que es exactamente lo que `EventPublisher` usa como clave de enrutamiento. `raise_event` acumula el evento en el buffer del agregado; el manejador de comandos vacía ese buffer llamando a `wallet.clear_events()` tras una persistencia con éxito.

El puente de publicación

En lugar de repetir el bucle de vaciado en cada manejador de comandos, Lumen lo extrae en una única corrutina `publish_domain_events`. El puente serializa cada evento vaciado con `dataclasses.asdict`, y luego llama a `publisher.publish` con el nombre de clase como `event_type` y `"wallet.events"` como canal lógico:

lumen/core/services/wallets/event_publishing.py

```

from __future__ import annotations

import dataclasses
from collections.abc import Iterable
from typing import Any

from lumen.core.services.listeners.wallet_audit_listener import (
    WALLET_EVENTS_DESTINATION,
)
from pyfly.domain import DomainEvent
from pyfly.eda import EventPublisher

def _to_payload(event: DomainEvent) → dict[str, Any]:
    """Flatten a frozen-dataclass domain event into a dict."""
    payload: dict[str, Any] = dataclasses.asdict(event)
    payload.setdefault("event_type", event.event_type)
    return payload

async def publish_domain_events(
    publisher: EventPublisher, events: Iterable[DomainEvent]
) → None:
    """Publish each drained domain event on the wallet events channel.

    The envelope's ``event_type`` is the domain event class name
    (``WalletOpened`` / ``FundsDeposited`` / ``FundsWithdrawn``).
    """
    for event in events:
        await publisher.publish(
            destination=WALLET_EVENTS_DESTINATION,
            event_type=event.event_type,
            payload=_to_payload(event),
        )

```

Listado 8.2 — `publish_domain_events` conecta los eventos vaciados al bus EDA

`WALLET_EVENTS_DESTINATION` es la constante `"wallet.events"` definida en `wallet_audit_listener.py` y compartida por publicador y oyente para que el nombre del canal no pueda divergir. `event.event_type` es la propiedad del nombre de clase en `DomainEvent`: `"WalletOpened"`, `"FundsDeposited"` o `"FundsWithdrawn"`.

Cablear el publicador en los manejadores de comandos

En el Capítulo 7, los manejadores de comandos cargaban agregados, dirigían el comportamiento de dominio y guardaban —dejando los eventos acumulados tirados por el suelo. Ahora cierras esa carencia. Inyecta un `EventPublisher` junto al `WalletRepository` y un `async_sessionmaker`, decora `do_handle` con `@transactional()`, y tras `repo.upsert(...)` vacía el buffer del agregado y publica cada evento a través del puente.

El decorador `@transactional()` (de `pyfly.data.relational.sqlalchemy`) liga una unidad de trabajo a la tarea en curso en el origen de datos del `async_sessionmaker` inyectado; cada llamada al repositorio del método se une a ella, y hace commit en caso de éxito y rollback en caso de fallo. Eso significa que la secuencia cargar → mutar → guardar es una unidad de trabajo confirmada, y un guardado que falla detiene el manejador antes de que se publique ningún evento.

Aquí está el cambio, desglosado en las cuatro ediciones que harás a `DepositFundsHandler`.

Paso 1 — Añade el publicador al constructor. Junto al parámetro existente

`repository`, acepta `events: EventPublisher` y guárdalo como `self._events`.

Tipéalo como el *protocolo* `EventPublisher`, nunca como `InMemoryEventBus` —eso es lo que mantiene al manejador ignorante de qué bus está corriendo.

Paso 2 — Acepta la fábrica de sesiones. Añade `session_factory:`

`async_sessionmaker[AsyncSession]` y guárdalo como `self._session_factory`. El decorador `@transactional()` lee este atributo para elegir el origen de datos en el que se ejecuta su unidad de trabajo (sin él, la unidad se ejecuta en el origen de datos por defecto de la aplicación), así que el nombre importa.

Paso 3 — Decora `do_handle` con `@transactional()`. Esto envuelve toda la secuencia cargar-mutar-guardar en una sola transacción confirmada.

Paso 4 — Vacía y publica tras guardar. Como último paso dentro de `do_handle`,

después de `self._repository.upsert(...)`, llama a `await publish_domain_events(self._events, wallet.clear_events())`.

`wallet.clear_events()` devuelve los eventos acumulados y vacía el buffer, de modo que nunca se publican dos veces.

Juntando esas cuatro ediciones obtienes el `DepositFundsHandler` actualizado:

lumen/core/services/wallets/deposit_funds_handler.py

```

from __future__ import annotations

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

from lumen.core.mappers.wallet_mapper import to_aggregate, to_entity
from lumen.core.services.wallets.deposit_funds_command import DepositFunds
from lumen.core.services.wallets.event_publishing import publish_domain_events
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.domain import AggregateNotFound
from pyfly.data.relational.sqlalchemy import LockMode, transactional
from pyfly.eda import EventPublisher

@command_handler
@service
class DepositFundsHandler(CommandHandler[DepositFunds, int]):
    """Credit funds to an existing wallet; returns the new balance."""

    def __init__(
        self,
        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
    ) → None:
        super().__init__()
        self._repository = repository
        self._events = events
        self._session_factory = session_factory

    @transactional()
    async def do_handle(self, command: DepositFunds) → int:
        entity = await self._repository.find_by_id(
            command.wallet_id, lock=LockMode.PESSIMISTIC_WRITE
        )

```

```

if entity is None:
    raise AggregateNotFound("Wallet", command.wallet_id)

wallet = to_aggregate(entity)
wallet.deposit(Money(amount=command.amount, currency=wallet.currency))
await self._repository.upsert(to_entity(wallet))

await publish_domain_events(self._events, wallet.clear_events())
return wallet.balance.amount

```

Listado 8.3 — *DepositFundsHandler*: unidad de trabajo *@transactional* y luego publicar

Tres decisiones de diseño merecen mención. Primero, `events: EventPublisher` está tipado como el protocolo, no como `InMemoryEventBus` —el contenedor de inyección de dependencias inyecta la implementación que esté registrada, así que el manejador nunca sabe ni le importa qué bus está activo. Segundo, la llamada de publicación se sitúa *después* de `self._repository.upsert(...)` dentro de la unidad de trabajo `@transactional()`: si la persistencia falla, la excepción sale del manejador antes de llegar a `publish_domain_events`, y el decorador hace rollback. Tercero, el manejador trabaja directamente con la entidad ORM mediante los mappers `to_aggregate` / `to_entity` —el agregado se rehidrata desde la fila, se muta y se mapea de vuelta a una fila antes del upsert. La publicación sigue ejecutándose antes de que la unidad haga commit, lo que deja un hueco que la nota de más abajo cierra con el outbox transaccional. Por ahora, el bus en memoria nunca falla.

📌 NOTA: EJECÚTALO

Con la aplicación en ejecución (`uv run pyfly run --server uvicorn`), abre un monedero y deposita en él desde una segunda terminal:

```
# Open a wallet - returns its id
curl -s -X POST http://localhost:8080/api/v1/wallets \
  -H 'content-type: application/json' \
  -d '{"owner_id": "u-1", "currency": "EUR"}'
# {"wallet_id": "wlt-..."}

# Deposit 1500 minor units (15.00 EUR) into that wallet
curl -s -X POST http://localhost:8080/api/v1/wallets/wlt-.../deposit \
  -H 'content-type: application/json' \
  -d '{"amount": 1500}'
# {"wallet_id": "wlt-...", "balance": 1500}
```

La respuesta HTTP confirma el saldo, pero la evidencia más interesante está en el log de la aplicación: como el depósito publicó un evento `FundsDeposited` y el oyente de auditoría (que construirás en la siguiente sección) reacciona a él, verás una línea de log `wallet_audit_observed` para `event_type=FundsDeposited`. ¿Todavía no hay oyente? Entonces la publicación ocurre en silencio —que es precisamente el sentido: el manejador no sabe si alguien está escuchando.

Qué acaba de pasar. El manejador de comandos ahora hace una cosa más tras guardar: vacía los eventos que el agregado acumuló y se los entrega al bus. La ordenación crucial es *guardar primero, publicar segundo*, todo dentro de una transacción. Si el guardado falla, los eventos nunca se publican. El manejador ganó cuatro líneas y cero conocimiento nuevo —sigue sin tener ni idea de qué, si es que algo, reaccionará.

❗ PUBLICAR DESPUÉS DEL COMMIT: EL OUTBOX TRANSACCIONAL

El bus en memoria entrega en `publish()`: el oyente de auditoría se ejecuta dentro de la unidad de trabajo del manejador, *antes* de que haga commit, y un bus de broker envía en ese momento también. Un commit que falle después deja el evento entregado para un cambio que nunca se persistió. Desde la v26.09.08 PyFly cierra ese hueco para cualquier `EventPublisher` con un solo interruptor:

```
pyfly:
  eda:
    provider: memory          # o kafka, rabbitmq, redis
    outbox:
      enabled: true           # desactivado por defecto
      store: auto             # sql, mongo o auto
```

Cada `publish()` se añade entonces a un outbox en la unidad de trabajo de quien llama —las tablas `pyfly_outbox_*` del origen de datos principal, o colecciones en MongoDB— y un reenviador lo entrega al bus solo después de que la unidad haga commit: una unidad que hace rollback no publica nada, y una que hace commit publica al menos una vez, con el id del evento en la cabecera `x-pyfly-event-id` para que los consumidores dedupliquen con él. Los proveedores `database` y `postgres` ya son un outbox. El Capítulo 10 vuelve sobre ello para Kafka y RabbitMQ.

Otras dos piezas del framework tocan este camino. El `DomainEventPublisher` (activo por defecto) vacía los eventos pendientes de un agregado cuando su unidad hace commit y se los entrega a los oyentes `@app_event_listener` de la aplicación; Lumen llama a `wallet.clear_events()` dentro de la unidad, así que el publicador no encuentra nada pendiente y ningún evento sale dos veces. Fija `pyfly.eda.domain-events.destination` para que sea él quien los publique a través del bus de EDA, y elimina la llamada escrita a mano. Y un `@app_event_listener(phase=TransactionPhase.AFTER_COMMIT)` solo se ejecuta cuando la unidad en la que se publicó ha hecho commit.

El `OpenWalletHandler` sigue el mismo patrón:

lumen/core/services/wallets/open_wallet_handler.py

```

from __future__ import annotations

from uuid import uuid4

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

from lumen.core.mappers.wallet_mapper import to_entity
from lumen.core.services.wallets.event_publishing import publish_domain_events
from lumen.core.services.wallets.open_wallet_command import OpenWallet
from lumen.models.entities.v1.wallet_entity import Wallet
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.data.relational.sqlalchemy import transactional
from pyfly.eda import EventPublisher

@command_handler
@service
class OpenWalletHandler(CommandHandler[OpenWallet, str]):
    """Open a new, empty wallet."""

    def __init__(
        self,
        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
    ) → None:
        super().__init__()
        self._repository = repository
        self._events = events
        # @transactional resolves the unit-of-work session from here.
        self._session_factory = session_factory

    @transactional()
    async def do_handle(self, command: OpenWallet) → str:
        wallet_id = f"wlt-{uuid4()}"

```

```

wallet = Wallet.open(
    wallet_id=wallet_id,
    owner_id=command.owner_id,
    currency=command.currency,
)
await self._repository.upsert(to_entity(wallet))

await publish_domain_events(self._events, wallet.clear_events())
return wallet_id

```

Listado 8.4 — *OpenWalletHandler: @transactional, upsert y luego publicar WalletOpened*

`return wallet_id` viene *después* de la llamada de publicación —el manejador cumple su contrato solo una vez que cada hecho producido por la operación ha sido despachado. El id del monedero se genera localmente con `uuid4()` en lugar de delegarlo a un método del repositorio; `to_entity` mapea el agregado a una fila antes del upsert para que el `Repository` del framework reciba el modelo ORM que espera.

El atajo `@publish_result`

En servicios más sencillos donde el valor de retorno de un método es la carga útil del evento —común en código que no ha adoptado el patrón de agregado completo—

`@publish_result` elimina por completo la llamada manual de publicación:

lumen/eda/publish_result_example.py

```

from pyfly.eda import publish_result
from pyfly.eda.adapters.memory import InMemoryEventBus

bus = InMemoryEventBus()

@publish_result(bus, destination="wallet.events",
event_type="FundsTransferred")
async def transfer_funds(source_id: str, target_id: str, amount: int) → dict:
    # Business logic omitted – the returned dict IS the event payload.
    return {
        "source_id": source_id,
        "target_id": target_id,
        "amount": amount,
    }

```

Listado 8.5 — @publish_result auto-publica el valor de retorno del método

Cuando `transfer_funds` retorna, el decorador intercepta el resultado y llama a `bus.publish` con él como carga útil —sin necesidad de bucle repetitivo. `destination` y `event_type` quedan fijados en el momento de la decoración, manteniendo limpia la función de negocio. `@publish_result` también acepta un predicado `condition` opcional: el evento se publica solo cuando el resultado satisface la prueba, lo cual resulta útil para flujos de trabajo condicionales en los que no toda ejecución con éxito debe difundirse.

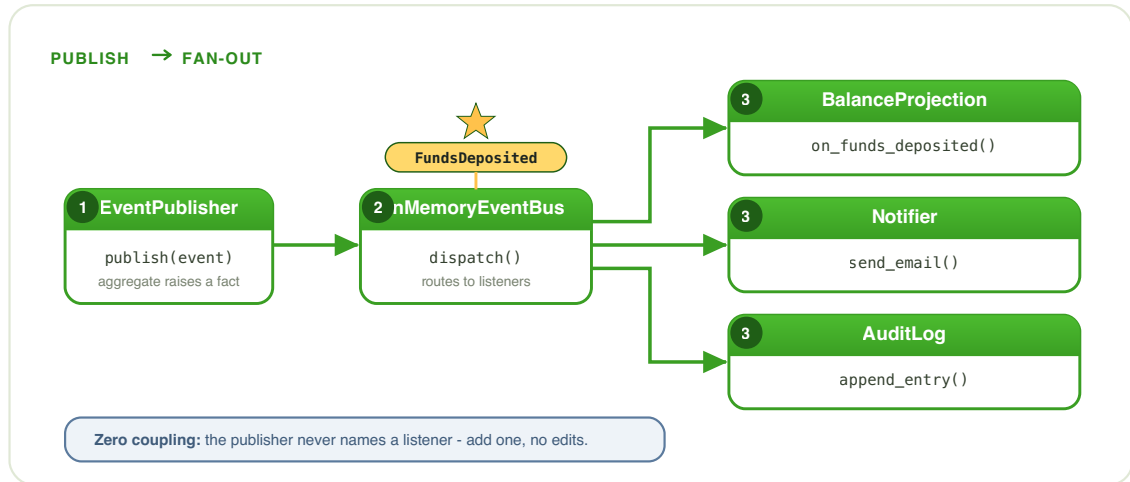


Figura 8.1 — Un publicador, muchos oyentes independientes.

🌿 EQUIVALENCIA CON SPRING

`EventPublisher` es el equivalente en PyFly del `ApplicationEventPublisher` de Spring. Llamar a `publisher.publish(...)` equivale a `applicationEventPublisher.publishEvent(event)`. El decorador `@event_listener` (siguiente sección) refleja el `@EventListener` de Spring para reacciones síncronas, en la misma transacción. `@publish_result` logra lo que los desarrolladores de Spring a menudo cablean manualmente con consejos AOP `@AfterReturning`.

Reaccionar con `@event_listener`

Publicar un evento es solo la mitad de la película. Un evento al que nadie reacciona no es más que una entrada de log. El valor del modelo orientado a eventos reside en las *reacciones* que habilita —comportamientos independientes que se activan en respuesta al mismo hecho publicado, cada uno ajeno a los demás.

El decorador `@event_listener` de PyFly es la forma más sencilla de registrar una reacción. Decora cualquier método asíncrono con los nombres de clase que le interesan, y el `ApplicationContext` cablea la suscripción durante el arranque —sin necesidad de una referencia al bus en el momento de la decoración.

```
from pyfly.eda import event_listener, EventEnvelope

@event_listener(event_types=["FundsDeposited"])
async def on_funds_deposited(envelope: EventEnvelope) → None:
    ...
```

`event_types` acepta nombres de clase exactos. Los oyentes dentro de una clase `@service` reciben un `EventEnvelope` como único argumento. Como el emparejamiento ocurre a nivel del bus —no dentro de tu función— un único método oyente puede suscribirse a varios tipos de evento en una sola declaración.

WalletAuditListener

El oyente de producción de Lumen es `WalletAuditListener`. Se suscribe a los tres eventos de dominio del monedero y mantiene dos proyecciones en memoria: un **rastros de auditoría** ordenado y un **total neto de depósitos acumulado** por monedero.

Lo ensamblaremos en pequeñas piezas. Lee primero los cuatro pasos, y luego estudia el listado completo más abajo —es el mismo código, mostrado entero.

Paso 1 — Declara una clase `@service` simple. Un oyente no es más que un bean de servicio con algo de estado. Dale un `__init__` que inicialice las dos proyecciones:

```
self._entries: list[AuditEntry] = [] para el rastro de auditoría y
self._running_totals: dict[str, int] = {} para los totales por monedero.
```

Paso 2 — Escribe el método de reacción. Añade un `async def on_wallet_event(self, envelope: EventEnvelope) → None`. Recibe un `EventEnvelope` y nada más.

Paso 3 — Estámpalo con `@event_listener` . Decora el método con

`@event_listener(event_types=["WalletOpened", "FundsDeposited", "FundsWithdrawn"])` . Un método puede suscribirse a los tres nombres de clase en una sola declaración. Este decorador no se suscribe de inmediato — *estampa* el método con metadatos que el `ApplicationContext` encuentra en el arranque y usa para auto-suscribirlo al bean `EventPublisher` .

Paso 4 — Proyecta el evento. Dentro del método, añade un `AuditEntry` por cada evento, y luego ramifica según `envelope.event_type` para ajustar el total acumulado: ponlo a cero en `WalletOpened` , súmalo en `FundsDeposited` , réstalo en `FundsWithdrawn` . Expón accesores de lectura (`entries` , `entries_for` , `running_total`) para que otro código pueda consultar las proyecciones.

lumen/core/services/listeners/wallet_audit_listener.py

```

from __future__ import annotations

import logging
from dataclasses import dataclass
from datetime import datetime

from pyfly.container import service
from pyfly.eda import EventEnvelope, event_listener

logger = logging.getLogger(__name__)

WALLET_EVENTS_DESTINATION = "wallet.events"

@dataclass(frozen=True)
class AuditEntry:
    """One observed domain event, captured for the audit trail."""

    event_type: str
    wallet_id: str
    event_id: str
    occurred_at: datetime
    payload: dict[str, object]

@service
class WalletAuditListener:
    """In-memory audit log + running-total projection over wallet events."""

    def __init__(self) → None:
        self._entries: list[AuditEntry] = []
        self._running_totals: dict[str, int] = {}

    @event_listener(
        event_types=["WalletOpened", "FundsDeposited", "FundsWithdrawn"]
    )
    async def on_wallet_event(self, envelope: EventEnvelope) → None:

```

```

"""Project every wallet domain event into the read models."""
payload = dict(envelope.payload)
wallet_id = str(payload.get("wallet_id", ""))

self._entries.append(
    AuditEntry(
        event_type=envelope.event_type,
        wallet_id=wallet_id,
        event_id=str(payload.get("event_id", envelope.event_id)),
        occurred_at=envelope.timestamp,
        payload=payload,
    )
)

if envelope.event_type == "WalletOpened":
    self._running_totals.setdefault(wallet_id, 0)
elif envelope.event_type == "FundsDeposited":
    amount = int(payload.get("amount", 0))
    self._running_totals[wallet_id] = (
        self._running_totals.get(wallet_id, 0) + amount
    )
elif envelope.event_type == "FundsWithdrawn":
    amount = int(payload.get("amount", 0))
    self._running_totals[wallet_id] = (
        self._running_totals.get(wallet_id, 0) - amount
    )

logger.info(
    "wallet_audit_observed",
    extra={"event_type": envelope.event_type, "wallet_id": wallet_id},
)

@property
def entries(self) → list[AuditEntry]:
    """A snapshot of the audit log, in observation order."""
    return list(self._entries)

def entries_for(self, wallet_id: str) → list[AuditEntry]:
    """The audit entries recorded for one wallet."""

```

```

    return [e for e in self._entries if e.wallet_id == wallet_id]

    def running_total(self, wallet_id: str) → int:
        """Net funds (deposited minus withdrawn) for wallet_id, minor
        units."""
        return self._running_totals.get(wallet_id, 0)

```

Listado 8.6 — *WalletAuditListener*: rastro de auditoría + proyección de total acumulado

Esto es lo que hace el oyente, paso a paso.

`@event_listener(event_types=["WalletOpened", "FundsDeposited", "FundsWithdrawn"])` le dice al `ApplicationContext` que suscriba `on_wallet_event` a esos tres nombres de clase. Como la clase es un bean `@service`, PyFly la descubre en el arranque y cablea las suscripciones automáticamente —nunca llamas a `bus.subscribe` a mano.

`on_wallet_event` recibe un `EventEnvelope`. `envelope.event_type` es el nombre de clase del evento de dominio lanzado. `envelope.payload` es el dict producido por `dataclasses.asdict` en el puente de publicación, así que sus claves coinciden exactamente con los nombres de los campos de la dataclass — `wallet_id`, `amount`, `currency`, `balance`.

El método añade un `AuditEntry` por cada evento, y luego ramifica según `event_type` para actualizar el total acumulado. Fíjate en lo que falta: ninguna importación del agregado `Wallet`, ninguna llamada al repositorio, ningún conocimiento de cómo se procesó el depósito. La proyección reacciona puramente al hecho publicado.

Qué acaba de pasar. Has escrito una reacción autónoma. El decorador `@event_listener` es toda la historia del cableado —no hay ninguna llamada `bus.subscribe(...)` en ningún sitio de este archivo. En el arranque, el `ApplicationContext` escanea tus beans `@service`, encuentra el método estampado

`on_wallet_event` y lo suscribe al bus para cada uno de los tres nombres de clase. El oyente y los manejadores de comandos nunca se referencian entre sí; están conectados únicamente a través de los eventos que acuerdan nombrar.

💡 CONSEJO: METADATOS DEL SOBRE EN LAS PROYECCIONES

`envelope.timestamp` te da la hora autoritativa del evento —cuándo se registró el hecho, no cuándo corrió el oyente. Guárdala en tu modelo de lectura y obtienes una columna `occurred_at` barata gratis, sin desviación de reloj entre escritor y lector.

Probar el oyente de extremo a extremo

La suite de pruebas ejercita la ruta completa de publicar-y-recibir sin mocks. El `conftest` cablea un `InMemoryEventBus` compartido, refleja el paso de descubrimiento de `@event_listener` suscribiendo `on_wallet_event` a cada nombre de clase declarado, y registra manejadores de comandos reales que comparten la misma referencia al bus:

```
# tests/conftest.py (abbreviated)
from pyfly.eda.adapters.memory import InMemoryEventBus

@pytest_asyncio.fixture
async def event_bus() → InMemoryEventBus:
    yield InMemoryEventBus()

@pytest_asyncio.fixture
async def audit_listener(event_bus: InMemoryEventBus) → WalletAuditListener:
    listener = WalletAuditListener()
    method = listener.on_wallet_event
    for pattern in method.__pyfly_event_patterns__:
        event_bus.subscribe(pattern, method)
    yield listener
```

Con ese cableado en su sitio, la prueba envía comandos reales y hace aserciones sobre los modelos de lectura del oyente:

lumen/tests/test_event_listener.py

```

from __future__ import annotations

import pytest
from lumen.core.services.listeners import WalletAuditListener
from lumen.core.services.wallets.deposit_funds_command import DepositFunds
from lumen.core.services.wallets.open_wallet_command import OpenWallet
from lumen.core.services.wallets.withdraw_funds_command import WithdrawFunds
from lumen.interfaces.enums.v1.currency import Currency

from pyfly.cqrs import DefaultCommandBus

@pytest.mark.asyncio
async def test_listener_observes_wallet_events(
    command_bus: DefaultCommandBus,
    audit_listener: WalletAuditListener,
) → None:
    wallet_id = await command_bus.send(
        OpenWallet(owner_id="u-1", currency=Currency.EUR)
    )
    await command_bus.send(DepositFunds(wallet_id=wallet_id, amount=1500))
    await command_bus.send(WithdrawFunds(wallet_id=wallet_id, amount=400))

    entries = audit_listener.entries_for(wallet_id)
    assert [e.event_type for e in entries] == [
        "WalletOpened",
        "FundsDeposited",
        "FundsWithdrawn",
    ]

    deposited = entries[1]
    assert deposited.payload["amount"] == 1500
    assert deposited.payload["currency"] == "EUR"
    assert deposited.payload["balance"] == 1500

```

```
# running_total = deposited - withdrawn
assert audit_listener.running_total(wallet_id) == 1100
```

Listado 8.7 — Prueba de extremo a extremo: los comandos publican, el oyente proyecta

La prueba demuestra la cadena completa: `OpenWalletHandler` → `publish_domain_events` → `InMemoryEventBus` → `WalletAuditListener.on_wallet_event` → `audit_listener.entries_for(...)`. Sin mocks, sin fakes —la ruta de código de producción corre tal como está escrita.

📌 NOTA: EJECÚTALO

Ejecuta la prueba del oyente de eventos desde la raíz del proyecto Lumen:

```
uv run --extra dev pytest tests/test_event_listener.py -q
```

Deberías ver pasar la suite:

```
...
[100%]
3 passed in 0.XXs
```

Las tres pruebas cubren el camino feliz (abrir, depositar, retirar y luego comprobar el rastro de auditoría y el total acumulado), el caso de proyección vacía antes de que se ejecute ningún comando, y el caso negativo en el que una retirada con descubierto lanza una excepción y por tanto *no* publica ningún evento —así que no debe aparecer en el registro de auditoría. Si una prueba falla con un error de atributo inexistente sobre `__pyfly_event_patterns__`, el decorador `@event_listener` no está aplicado a `on_wallet_event`; revisa el Paso 3 de la sección anterior.

Lo que hace convincente a este diseño es que añadir el oyente no requirió ningún cambio en los manejadores de comandos, en el agregado `Wallet` ni en ningún repositorio. `DepositFundsHandler` no tiene ni idea de que existe una proyección. Ambos lados son totalmente independientes —cada uno es una consecuencia del mismo hecho publicado, conectados únicamente por el bus.

Cuando los oyentes fallan: estrategias de error

Un oyente que se comporta mal plantea una pregunta puntiaguda: ¿debería el fallo detener toda la cadena de entrega, o debería el bus seguir notificando a los oyentes restantes? La respuesta correcta depende del papel del oyente. PyFly te da control explícito en lugar de imponer una única política.

Por defecto, `InMemoryEventBus` invoca a los oyentes secuencialmente y propaga cualquier excepción —el comportamiento correcto para desarrollo, donde un oyente que falla debe aflorar ruidosamente. En producción normalmente necesitas un control más fino.

`ErrorStrategy` es un enum que gobierna cómo se comporta el bus cuando un oyente lanza una excepción:

```
from pyfly.eda import ErrorStrategy
```

Miembro	Valor	Comportamiento
<code>IGNORE</code>	<code>"IGNORE"</code>	Traga la excepción silenciosamente. El procesamiento continúa con el siguiente manejador.
<code>LOG_AND_CONTINUE</code>	<code>"LOG_AND_CONTINUE"</code>	Registra el error y luego continúa. El valor por defecto más seguro para oyentes no críticos.
<code>RETRY</code>	<code>"RETRY"</code>	Reintenta la entrega. El número de reintentos y el back-off se configuran por separado.
<code>DEAD_LETTER</code>	<code>"DEAD_LETTER"</code>	Mueve el evento fallido a un destino de cartas muertas para inspección posterior.
<code>FAIL_FAST</code>	<code>"FAIL_FAST"</code>	Propaga la excepción de inmediato. No se invoca ningún manejador más.

💡 CONSEJO: AJUSTA LA ESTRATEGIA A LA CRITICIDAD DEL OYENTE

Un oyente de auditoría debería usar `LOG_AND_CONTINUE` —un registrador de auditoría roto no debe detener una transacción financiera. Una proyección que alimenta las respuestas de consulta podría justificar `RETRY` para asegurar que el modelo de lectura se mantenga consistente. Un notificador puede tolerar `IGNORE`, ya que un correo de bienvenida perdido no es un problema de integridad de datos.

⚠ ADVERTENCIA: EFECTOS SECUNDARIOS E IDEMPOTENCIA

Si un oyente realiza un efecto secundario —escribir una fila en la base de datos, enviar un correo— y el bus reintenta la entrega tras un fallo transitorio, el efecto puede ejecutarse más de una vez. Diseña los oyentes para que sean idempotentes: escribe una fila solo si el `event_id` no se ha registrado ya, envía un correo solo si la bandera de bienvenida no está ya activada. El `envelope.event_id` (un UUID estable generado por el bus) es tu clave de idempotencia.

En memoria hoy, un broker mañana

`InMemoryEventBus` es la implementación lista para usar —el `EventPublisher` por defecto que proporciona el `ApplicationContext`. Corre enteramente en proceso: `publish` es una llamada asíncrona directa, no hay serialización, y los eventos no entregados se desvanecen si el proceso muere. Para desarrollo local, pruebas de integración y monolitos que no necesitan entrega entre procesos, eso es perfectamente aceptable.

Entender cómo funciona internamente el bus en memoria facilita razonar sobre el comportamiento en los límites —y apreciar exactamente qué cambia cuando intercambias por un broker.

```

from pyfly.eda.adapters.memory import InMemoryEventBus

bus = InMemoryEventBus()

bus.subscribe("FundsDeposited", my_handler)

await bus.publish(
    destination="wallet.events",
    event_type="FundsDeposited",
    payload={"wallet_id": "w-001", "amount": 5000, "currency": "EUR", "balance": 5000},
)

```

Una llamada a `publish` ejecuta cuatro pasos en secuencia:

1. Envuelve los argumentos en un `EventEnvelope` con un `event_id` generado y un `timestamp` en UTC.
2. Itera cada par registrado `(pattern, handler)`.
3. Para cada par donde `fnmatch.fnmatch(event_type, pattern)` es `True`, llama al manejador con el sobre.
4. Los manejadores corren secuencialmente en orden de suscripción.

Las suscripciones usan el `fnmatch` de Python, así que `"Funds*"` empareja tanto con `"FundsDeposited"` como con `"FundsWithdrawn"`, y `"*"` empareja con todo. La invocación secuencial del paso 4 hace que el orden de los oyentes sea determinista — útil en pruebas — pero también significa que un oyente lento retrasa a todos los posteriores. Los adaptadores respaldados por broker normalmente despachan en paralelo; ten presente esa diferencia al razonar sobre el rendimiento.

Como cada oyente en Lumen depende del *protocolo* `EventPublisher`, no de `InMemoryEventBus` directamente, la implementación se intercambia sin tocar un solo oyente. El Capítulo 10 introduce adaptadores de Kafka y RabbitMQ; cambiar a cualquiera de ellos es un cambio de configuración — `WalletAuditListener` sigue funcionando sin modificación.

❗ NOTA: INMEMORYEVENTBUS Y LAS PRUEBAS

`InMemoryEventBus` es también la herramienta adecuada para las pruebas. Inyecta un `InMemoryEventBus` fresco como fixture, suscribe un manejador que capture, ejercita tu manejador de comandos y haz aserciones sobre los objetos `EventEnvelope` que el manejador recibió —incluyendo `event_type`, `payload`, `event_id` y `timestamp`. Sin mockear, sin fakes, solo el bus real con entradas controladas.

✓ Lo que construiste

La Parte III está abierta.

Este capítulo cerró el ciclo que el Capítulo 7 inició. `Wallet` lanzó eventos de dominio en el Capítulo 6; los manejadores de comandos los publicaron aquí; `WalletAuditListener` reacciona a esos hechos sin saber nada de la ruta de comandos que los disparó.

La arquitectura es genuinamente orientada a eventos dentro de un solo proceso. Aquí tienes una referencia rápida de cada pieza:

Pieza	Papel
<code>EventPublisher</code>	Puerto —un protocolo que cumple cualquier implementación de bus
<code>InMemoryEventBus</code>	Adaptador por defecto —en proceso, sin configuración; activado por <code>pyfly.eda.provider: memory</code>
<code>EventEnvelope</code>	Transporta la carga útil + <code>event_id</code> , <code>timestamp</code> , <code>destination</code> , <code>headers</code>
<code>@event_listener(event_types=[...])</code>	Decorador de suscripción —nombres de clase; el contexto lo cablea en el arranque
<code>publish_domain_events</code>	Puente —vacía <code>wallet.clear_events()</code> , serializa con <code>dataclasses.asdict</code> , llama a <code>publisher.publish</code>
<code>ErrorStrategy</code>	Controla la gestión de fallos: <code>IGNORE</code> , <code>LOG_AND_CONTINUE</code> , <code>RETRY</code> , <code>DEAD_LETTER</code> , <code>FAIL_FAST</code>

Tres principios se trasladan al resto de la Parte III: **guarda antes de publicar** —los oyentes nunca deben ver un hecho que no llegó a persistirse, y con el outbox transaccional nunca uno que no se confirmó; **diseña los oyentes para la idempotencia** —los reintentos deben ser seguros; **depende del puerto, no del adaptador** —el bus puede intercambiarse sin tocar el código de los oyentes.

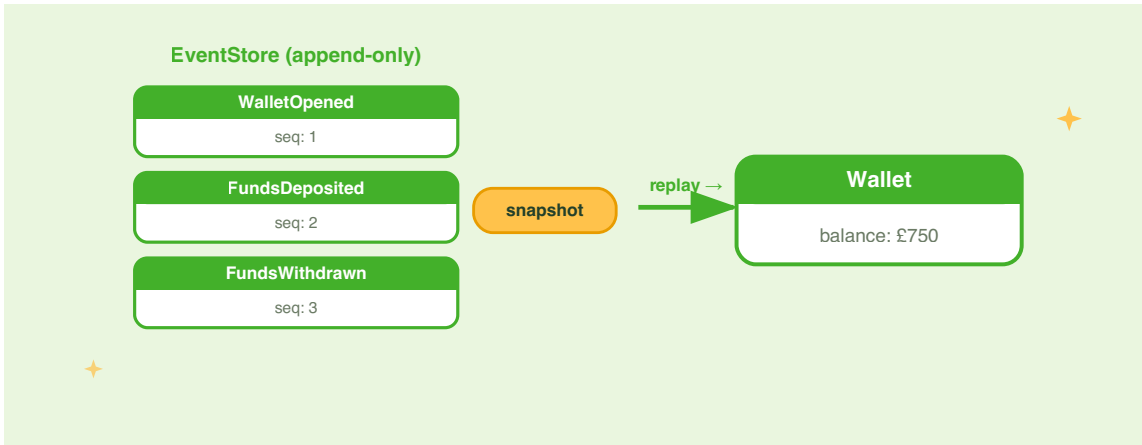
El Capítulo 9 lleva la idea del evento más lejos. En lugar de mantener un modelo de lectura separado junto a un agregado mutable, almacenas los eventos mismos como el sistema de registro —aplicando event sourcing (suministro de eventos) al libro mayor para que cada saldo histórico sea calculable desde primeros principios.

Pruébalo tú mismo

- 1. Añade un oyente `FraudDetector`**. Crea un servicio `FraudDetector` que se suscriba a `"FundsDeposited"` usando `@event_listener(event_types=["FundsDeposited"])`. Si el `amount` en la carga útil supera `1_000_000` (diez mil euros en unidades menores), registra una advertencia que incluya el `envelope.event_id`, el `envelope.timestamp` y el `wallet_id` de la carga útil. Verifica que se dispara publicando un evento `FundsDeposited` directamente a un `InMemoryEventBus` en una prueba unitaria, y comprueba que la advertencia se activó.
- 2. Extiende `WalletAuditListener` con filtrado por evento**. Añade un método `entries_by_type(self, event_type: str) → list[AuditEntry]` que devuelva solo las entradas con un `event_type` coincidente. Escribe una prueba que abra un monedero, haga dos depósitos y una retirada, y compruebe que `entries_by_type("FundsDeposited")` devuelve exactamente dos entradas.
- 3. Observa el comportamiento de la estrategia de error**. Crea un oyente cuyo manejador lance `RuntimeError("failure")` incondicionalmente. Regístralo junto a un manejador que capture añadiendo a una lista en un `InMemoryEventBus`. Configura `ErrorStrategy.LOG_AND_CONTINUE` y confirma que el manejador que captura sigue recibiendo el evento a pesar del fallo. Luego cambia a `ErrorStrategy.FAIL_FAST` y confirma que el manejador que captura *no* recibe el evento.

CAPÍTULO 9

Event sourcing del libro mayor



El Capítulo 8 dejó sin resolver una grieta estructural. El `WalletAuditListener` mantiene un modelo de lectura rápido reaccionando a eventos de dominio, pero el estado canónico del monedero sigue siendo una fila de la tabla `wallets`, que guarda una única columna `balance`. Cada vez que el saldo cambia, el valor anterior desaparece para siempre. Si un auditor de cumplimiento pregunta "¿cuál era el saldo del monedero `w-001` a las 14:32 del 3 de marzo?", la respuesta honesta es: no puedes saberlo. La base de datos solo recuerda el presente.

El **event sourcing** (almacenamiento de eventos como fuente de verdad) le da la vuelta a este diseño. En lugar de almacenar el estado *actual* y descartar cada cambio, almacenas la *secuencia de cambios* y derivas el estado actual reproduciéndolos. La tabla `wallets` desaparece. En su lugar hay un **flujo de eventos** (event stream): un registro de solo anexo de cada evento `LedgerOpened`, `Credited` y `Debited` que el

libro mayor haya producido jamás. El saldo en cualquier instante es una función pura de los eventos hasta ese momento. Puedes rebobinar hasta las 14:32 de cualquier fecha porque tienes un registro completo de todo lo ocurrido entre entonces y ahora.

Un libro mayor financiero es el dominio ideal para el event sourcing. Los contables han comprendido durante siglos que la autoridad de un libro mayor proviene de sus asientos, no de un total acumulado al pie de la columna. El total acumulado es un *hecho derivado*; los asientos son la *fuentes de verdad*. El módulo `pyfly.eventsourcing` de PyFly lleva esa intuición contable al código: los agregados emiten eventos de dominio, un `EventStore` los registra de forma inmutable, un repositorio reproduce el flujo para reconstruir el estado y un `ProjectionRunner` construye modelos de lectura encima.

Este capítulo construye el agregado `LedgerAccount`, un objeto de dominio orientado a eventos creado a propósito que convive junto al `Wallet` con estado almacenado del Capítulo 6. Verás cada componente del módulo `pyfly.eventsourcing` y cómo el almacén de eventos, los snapshots, las proyecciones y el outbox transaccional colaboran para darle a Lumen un libro mayor que es a la vez auditable y eficiente.

Del estado a los eventos

La forma más clara de captar el cambio es comparar qué aspecto tiene la base de datos en cada modelo.

En el **modelo de almacenamiento de estado**, la base de datos guarda solo el estado actual del monedero:

wallet_id	owner_id	balance_cents	currency	updated_at
w-001	u-42	8500	EUR	2026-03-03 17:11

Cada `deposit` y `withdraw` sobrescribe `balance_cents` . La historia se pierde. Sabes que el monedero contiene 85,00 EUR ahora mismo; no puedes saber cómo llegó hasta ahí.

En el **modelo de almacenamiento de eventos**, la base de datos guarda el flujo de eventos:

stream_id	seq	event_type	payload	occurred_at
led-001	1	LedgerOpened	<code>{"currency":"EUR","owner_id":"u-42"}</code>	2026-03-01 09:00
led-001	2	Credited	<code>{"amount":10000,"balance":10000}</code>	2026-03-01 09:01
led-001	3	Debited	<code>{"amount":1500,"balance":8500}</code>	2026-03-03 17:11

El saldo actual sigue siendo 85,00 EUR, pero ahora puedes leer cada decisión que condujo hasta él. Un auditor, un regulador o un investigador de fraude pueden reproducir el flujo desde cualquier desplazamiento y ver exactamente qué ocurrió y cuándo.

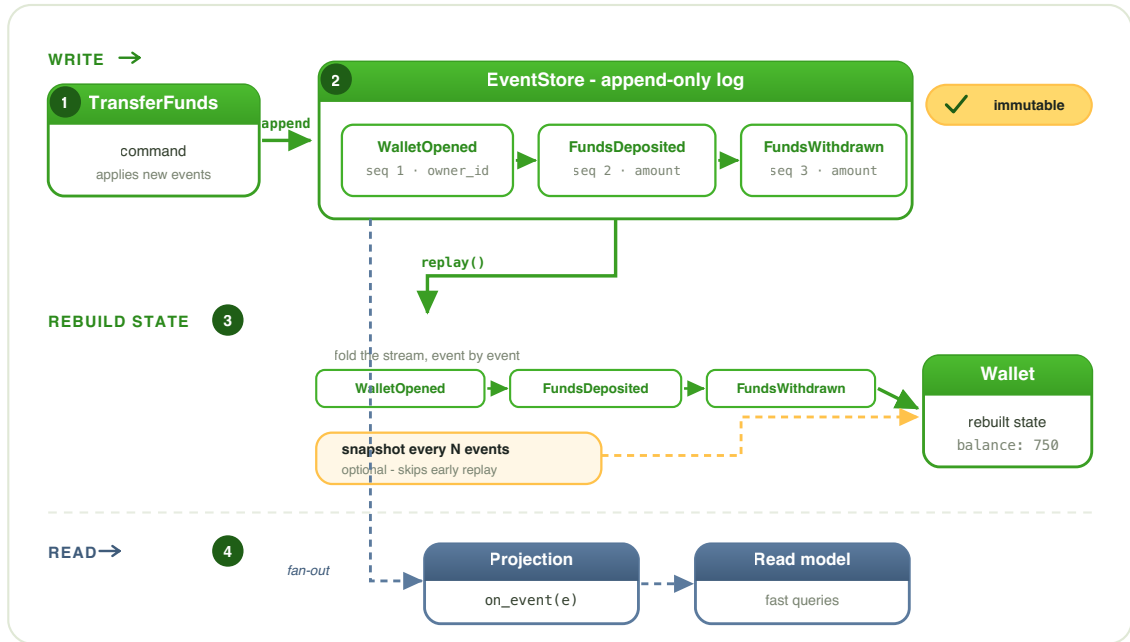


Figura 9.1 — Almacenamiento de estado frente a almacenamiento de eventos: un modelo conserva una instantánea del presente; el otro conserva cada hecho que condujo hasta él.

La contrapartida es real. El almacenamiento de eventos encarece las lecturas por defecto —debes reproducir el flujo para calcular el saldo actual— y exige disciplina en torno a la evolución del esquema (los eventos son inmutables; no puedes renombrar un campo a posteriori). Ambas preocupaciones tienen solución en PyFly: los **snapshots** aceleran la reproducción de flujos largos, y los **upcasters** traducen las formas de eventos antiguas a las nuevas durante la carga. Verás ambos antes del final de este capítulo.

i LOS EVENTOS COMO SISTEMA DE REGISTRO

El event sourcing no es lo mismo que la arquitectura orientada a eventos. El Capítulo 8 usó EDA: el agregado almacenaba su estado de forma normal y publicaba eventos de dominio como efecto secundario. El event sourcing va más allá: los eventos *son* el estado. No hay una columna `balance` separada que mantener sincronizada; el saldo lo calcula el repositorio cada vez que carga el agregado.

Una base separada para los agregados con event sourcing

Antes de escribir una línea de código del libro mayor, importa una distinción de nombres. El Capítulo 6 construyó `Wallet` sobre `pyfly.domain.AggregateRoot`, la clase base con estado almacenado. `LedgerAccount` usa una clase base **diferente**: `pyfly.eventsourcing.AggregateRoot`. Las dos viven en paquetes separados y, deliberadamente, no están relacionadas.

Aspecto	<code>Wallet</code> del Capítulo 6	<code>LedgerAccount</code> del Capítulo 9
Clase base	<code>pyfly.domain.AggregateRoot</code>	<code>pyfly.eventsourcing.AggregateRoot</code>
Evento de dominio	<code>pyfly.domain.DomainEvent</code>	<code>pyfly.eventsourcing.DomainEvent</code>
El estado vive en	una fila de base de datos	el flujo de eventos
Repositorio	<code>WalletRepository</code> (R2DBC)	<code>LedgerAccountRepository</code> (<code>EventSourcedRepository</code>)

`pyfly.eventsourcing.AggregateRoot` aporta la maquinaria de event sourcing a través de cuatro miembros:

- `when(EventType, handler)` — registra un manejador (handler) para una clase de evento dada. El manejador recibe `(aggregate, event)` como dos argumentos y realiza la mutación: una lambda, una función libre o una sola línea que delega en un método privado.
- `apply(event)` — encamina un evento recién creado a través de su manejador registrado y lo encola para el almacén de eventos. Ambas cosas ocurren atómicamente: el estado en memoria se actualiza de inmediato, sin ningún viaje de ida y vuelta al almacén.
- `replay(event_type, event)` — vuelve a ejecutar un evento persistido a través del mismo manejador *sin* volver a encolarlo. El repositorio llama a esto durante la carga para reconstruir el estado a partir del flujo almacenado.
- `version` — un contador entero que se incrementa después de cada evento despachado; el almacén lo usa como token de concurrencia optimista.

El orden de despacho es: primero el manejador registrado con `when()` ; luego un método llamado `on_{event_type}` si existe en el agregado; si no se encuentra ninguno, se lanza `EventHandlerException` . Un manejador ausente corrompería silenciosamente el estado reconstruido, así que el agregado falla de forma ruidosa en lugar de seguir adelante.

⚠ MANEJADOR DE DOS ARGUMENTOS: LA TRAMPA MÁS COMÚN

Todo manejador registrado con `when()` se invoca como `handler(aggregate, event)`: dos argumentos. Un método ligado como `self._on_opened` tiene firma `(self, event)`, lo que lo convierte en un invocable de un solo argumento desde el exterior. Pasar un método ligado directamente provoca un `TypeError` en tiempo de ejecución. El patrón usado a lo largo de este capítulo es una lambda de una sola línea — `lambda agg, evt: agg._on_opened(evt)` — que es correctamente de dos argumentos y mantiene la lógica real en un método privado donde puede probarse unitariamente y comprobarse de tipos de forma independiente.

El agregado con event sourcing

En el Capítulo 6, `Wallet` mantenía `_balance: Money` como estado de Python directo: `deposit` le sumaba, `withdraw` le restaba. En la versión con event sourcing, el agregado nunca muta sus propios campos directamente. Cada cambio de estado está mediado por un evento de dominio: el método de comportamiento *aplica* el evento, el manejador del evento *actualiza los campos* y el `EventStore` persiste el evento. Al cargar, el repositorio reproduce todos los eventos almacenados a través de los mismos manejadores, reconstruyendo el estado en memoria evento a evento.

Esta doble indirección —aplicar, luego manejar— es la mecánica central del event sourcing. Impone una disciplina estricta: cada transición de estado se registra exactamente una vez como un evento, y el estado actual del agregado siempre es demostrable a partir de su historia.

❗ JERGA: AGREGADO, MANEJADOR, FOLD

Un **agregado** es una única frontera de consistencia: un objeto que posee un conjunto de estado relacionado y las reglas que lo mantienen válido. `LedgerAccount` es un agregado: su saldo y su regla de descubierto viven juntos. Un **manejador** (a veces llamado *apply-handler*) es la pequeña función que toma un evento y actualiza los campos del agregado. Un **fold** es el término de la programación funcional para recorrer una lista y acumular un resultado elemento a elemento; reproducir un flujo de eventos hasta obtener un saldo es exactamente un fold sobre la lista de eventos. Verás "fold" usado como sinónimo de "el manejador se ejecuta sobre cada evento".

Constructor sin argumentos. `LedgerAccount.__init__` no toma argumentos. Esto es obligatorio porque `EventSourcedRepository` llama a la fábrica como `LedgerAccount()` y luego asigna `.id` antes de reproducir el flujo. Nunca construyas un libro mayor nuevo pasando argumentos a `__init__`; en su lugar, llama al classmethod `open`.

Construiremos el agregado en cuatro movimientos y luego ejecutaremos las pruebas unitarias para demostrar que cada invariante se cumple. Lee primero el listado completo y después recorre los pasos que aparecen debajo.

Aquí están los eventos de dominio y el agregado:

lumen/models/entities/v1/ledger_account.py

```

from __future__ import annotations

from dataclasses import dataclass

from lumen.interfaces.enums.v1.currency import Currency
from lumen.models.entities.v1.money import Money
from pyfly.domain import BusinessRuleViolation
from pyfly.eventsourcing import AggregateRoot, DomainEvent

# --- Domain events – the durable facts of the ledger -----

@dataclass
class LedgerOpened(DomainEvent):
    """The ledger was opened for an owner in a single currency."""
    account_id: str = ""
    owner_id: str = ""
    currency: str = ""

@dataclass
class Credited(DomainEvent):
    """Money moved *into* the ledger (a deposit / inbound transfer leg)."""
    account_id: str = ""
    amount: int = 0
    currency: str = ""
    balance: int = 0

@dataclass
class Debited(DomainEvent):
    """Money moved *out of* the ledger (withdrawal / outbound transfer
leg)."""
    account_id: str = ""
    amount: int = 0
    currency: str = ""
    balance: int = 0

```



```

# --- Event-sourced aggregate root -----

class LedgerAccount(AggregateRoot):
    """An event-sourced money-movement ledger.

    Zero-arg constructible so it can serve as the repository's factory –
    EventSourcedRepository calls LedgerAccount() then assigns .id before
    replaying the stream. Use the open() classmethod to create new ledgers.
    """

    def __init__(self) → None:
        super().__init__()
        self.owner_id: str = ""
        self.currency: Currency = Currency.EUR
        self.balance: Money = Money.zero(Currency.EUR)
        # Register apply-handlers. _dispatch calls handler(aggregate, event).
        # Use a lambda so the callable is two-arg; delegate to a private
        # method to keep the real logic type-checked and unit-testable.
        self.when(LedgerOpened, lambda agg, evt: agg._on_opened(evt))
        self.when(Credited, lambda agg, evt: agg._on_credited(evt))
        self.when(Debited, lambda agg, evt: agg._on_debited(evt))

# --- factory -----

@classmethod
def open(
    cls, account_id: str, owner_id: str, currency: Currency
) → "LedgerAccount":
    """Open a new empty ledger; appends LedgerOpened."""
    if not owner_id.strip():
        raise BusinessRuleViolation(
            "ledger-owner-required", "owner_id is required"
        )
    account = cls()
    account.id = account_id
    account.apply(
        LedgerOpened(

```

```

        account_id=account_id,
        owner_id=owner_id,
        currency=currency.value,
    )
)
return account

# --- commands (validate invariants, then apply) -----

def credit(self, amount: Money) → None:
    """Record money entering the ledger; appends Credited."""
    self._assert_currency(amount)
    if not amount.is_positive:
        raise BusinessRuleViolation(
            "ledger-credit-positive", "credit amount must be > 0"
        )
    new_balance = self.balance.add(amount)
    self.apply(
        Credited(
            account_id=self.id,
            amount=amount.amount,
            currency=amount.currency.value,
            balance=new_balance.amount,
        )
    )

def debit(self, amount: Money) → None:
    """Record money leaving; refuses to overdraw. Appends Debited."""
    self._assert_currency(amount)
    if not amount.is_positive:
        raise BusinessRuleViolation(
            "ledger-debit-positive", "debit amount must be > 0"
        )
    remaining = self.balance.subtract(amount)
    if remaining.is_negative:
        raise BusinessRuleViolation(
            "ledger-insufficient-funds",
            f"cannot debit {amount}; balance is {self.balance}",
        )

```

```

    self.apply(
        Debited(
            account_id=self.id,
            amount=amount.amount,
            currency=amount.currency.value,
            balance=remaining.amount,
        )
    )

# --- apply-handlers (pure folds, shared by apply + replay) -----

def _on_opened(self, event: object) → None:
    self.owner_id = event.owner_id          # type: ignore[attr-defined]
    self.currency = Currency(event.currency) # type: ignore[attr-defined]
    self.balance = Money.zero(self.currency)

def _on_credited(self, event: object) → None:
    self.balance = Money(
        event.balance, Currency(event.currency) # type: ignore[attr-
defined]
    )

def _on_debited(self, event: object) → None:
    self.balance = Money(
        event.balance, Currency(event.currency) # type: ignore[attr-
defined]
    )

# --- helpers -----

def _assert_currency(self, amount: Money) → None:
    if amount.currency is not self.currency:
        raise BusinessRuleViolation(
            "ledger-currency-mismatch",
            f"ledger holds {self.currency.value}, "
            f"got {amount.currency.value}",
        )

```

Listado 9.1 — LedgerAccount: un agregado con event sourcing que deriva su saldo de la reproducción

Cómo funciona. `__init__` registra tres manejadores `when()` —uno por clase de evento— cada uno como una lambda de dos argumentos que delega en un método privado. Dentro de los manejadores no ocurre ninguna aritmética; los métodos de comportamiento (`credit` , `debit`) poseen toda la validación y calculan el nuevo estado antes de construir el evento. El manejador simplemente *aplica* el resultado ya calculado.

`account.apply(Credited(...))` hace dos cosas atómicamente: anexa el evento al búfer de eventos pendientes (para que el repositorio pueda persistirlo) y lo despacha de inmediato al manejador de `Credited` (para que `balance` se actualice en memoria). El estado en memoria del agregado es, por tanto, siempre coherente con sus eventos pendientes, incluso antes de guardar.

El método de fábrica `open` llama a `apply(LedgerOpened(...))` en lugar de fijar los campos directamente. Eso es intencionado: si cargaras este agregado desde su flujo de eventos, `LedgerOpened` pasaría por exactamente el mismo manejador `_on_opened` y produciría resultados idénticos. La ruta de escritura y la ruta de reproducción son el mismo código; esa simetría es la garantía de corrección del event sourcing.

El contador `version` empieza en cero y se incrementa después de cada evento despachado. Tras `open` , `account.version = 1` ; tras un crédito, `account.version = 2` . Volverás a ver este número cuando el `EventStore` imponga la concurrencia optimista.

Construyéndolo paso a paso. El listado resulta denso la primera vez. Aquí está el mismo código como cuatro movimientos deliberados: el orden en el que realmente lo escribirías.

Paso 1 — Define los hechos duraderos. Escribe las tres dataclasses de evento (`LedgerOpened` , `Credited` , `Debited`) que extienden `pyfly.eventsourcing.DomainEvent` . Dale a cada campo un valor por defecto

(`account_id: str = ""` , `amount: int = 0`). Los valores por defecto no son cosméticos: el repositorio reconstruye estas dataclasses a partir de un payload almacenado, y una dataclass con campos obligatorios no podría reconstruirse cuando un evento antiguo es anterior a un campo más nuevo.

Paso 2 — Prepara el agregado vacío. Escribe `__init__` sin parámetros. Inicializa los campos a valores por defecto neutros (`owner_id = ""` , `balance = Money.zero(Currency.EUR)`) y registra un manejador `when()` por clase de evento, cada uno como una lambda de dos argumentos que delega en un método privado. En este punto, el agregado es un lienzo en blanco que sabe cómo reaccionar a los eventos pero no ha visto ninguno.

Paso 3 — Añade la fábrica y los comandos. Escribe el classmethod `open` y los métodos `credit` / `debit`. Cada uno valida primero sus invariantes (coincidencia de divisa, importe positivo, sin descubierto), calcula el saldo resultante y solo entonces llama a `self.apply(SomeEvent(...))`. La validación vive en el comando; el manejador nunca valida.

Paso 4 — Escribe los folds puros. Escribe `_on_opened` , `_on_credited` , `_on_debited`. Cada uno lee campos del evento y los asigna al agregado: sin aritmética, sin validación, sin excepciones. Como estos mismos tres métodos se ejecutan tanto en la ruta de escritura como en la de reproducción, mantenerlos tontos es lo que garantiza que un libro mayor recargado sea igual al que está en vivo.

 POR QUÉ LA VALIDACIÓN PERTENECE AL COMANDO, NO AL MANEJADOR

El manejador se vuelve a ejecutar cada vez que el agregado se carga desde su historia. Si la validación viviera en el manejador, estarías volviendo a comprobar la regla de descubierto contra datos que ya la pasaron hace años; y peor aún, una regla que *haya cambiado* desde entonces podría rechazar un evento histórico que era perfectamente válido cuando ocurrió. Valida una vez, en la escritura que produce el evento; confía en el evento para siempre después.

`pending_events()` devuelve los eventos encolados desde el último guardado. Las pruebas unitarias accionan el agregado de forma aislada —sin necesidad de repositorio—, lo que hace que la verificación de invariantes sea sencilla:

tests/test_ledger_event_sourcing.py

```

def test_open_emits_ledger_opened_and_starts_empty() → None:
    account = LedgerAccount.open(
        "led-1", owner_id="owner-1", currency=Currency.EUR
    )
    assert account.id == "led-1"
    assert account.owner_id == "owner-1"
    assert account.currency is Currency.EUR
    assert account.balance == Money.zero(Currency.EUR)
    # apply() queued the event for the store and bumped the version.
    [event] = account.pending_events()
    assert isinstance(event, LedgerOpened)
    assert event.account_id == "led-1"
    assert event.currency == "EUR"
    assert account.version == 1

def test_credit_then_debit_track_the_balance() → None:
    account = LedgerAccount.open("led-2", owner_id="o", currency=Currency.EUR)
    account.credit(Money(1000, Currency.EUR))
    account.debit(Money(400, Currency.EUR))
    assert account.balance == Money(600, Currency.EUR)
    kinds = [type(e).__name__ for e in account.pending_events()]
    assert kinds == ["LedgerOpened", "Credited", "Debited"]
    assert account.version == 3

def test_debit_cannot_overdraw() → None:
    account = LedgerAccount.open("led-3", owner_id="o", currency=Currency.EUR)
    account.credit(Money(500, Currency.EUR))
    with pytest.raises(BusinessRuleViolation) as exc:
        account.debit(Money(501, Currency.EUR))
    assert exc.value.rule == "ledger-insufficient-funds"
    # Invariant held: balance unchanged, no Debited event queued.
    assert account.balance == Money(500, Currency.EUR)
    assert [type(e).__name__ for e in account.pending_events()] == [
        "LedgerOpened",

```

```
"Credited",
]
```

Listado 9.2 — Pruebas unitarias: el agregado en aislamiento, comandos e invariantes

Ejecútalo. Estas tres pruebas no necesitan ni base de datos ni almacén de eventos: el agregado se sostiene enteramente por sí mismo. Ejecuta solo las pruebas unitarias de este archivo desde la raíz del proyecto Lumen:

```
uv run --extra dev pytest tests/test_ledger_event_sourcing.py -q -k "open or
credit or debit or currency"
```

Deberías ver pasar las pruebas en aislamiento:

```
.... [100%]
4 passed, 7 deselected in 0.05s
```

Los cuatro puntos son las cuatro pruebas que solo usan el agregado (las tres mostradas arriba más una comprobación de divisa no coincidente); las siete pruebas deseleccionadas son las respaldadas por el almacén, que ejecutaremos más adelante. Si ves un `TypeError` sobre el número de argumentos, casi con seguridad has caído en la trampa del método ligado de la advertencia anterior: pasar `self._on_opened` directamente a `when()` en lugar de envolverlo en una lambda de dos argumentos.

Qué acaba de ocurrir. Construiste un objeto de dominio que registra *qué cambió* en lugar de *qué es*. `open` emitió un hecho `LedgerOpened` y subió la versión a 1. `credit` y `debit` validaron cada uno su regla, calcularon el nuevo saldo y emitieron un hecho, de modo que un crédito seguido de un débito dejó tres hechos encolados y la versión en 3. Cuando una regla falló (la prueba de descubierto), el comando lanzó antes de llamar a `apply`, así que ningún evento defectuoso entró jamás en el búfer. El saldo en memoria del agregado y su lista de eventos pendientes se mantuvieron perfectamente acompañados, y nada tocó todavía una base de datos.

💡 `ON_{EVENT_TYPE}` COMO ALTERNATIVA

En lugar de lambdas `when()`, puedes definir un método en el agregado con el nombre del `event_type` del evento, que es el **nombre de clase** del evento, así que `on_LedgerOpened(self, evt)` (coincidiendo con la dataclass `LedgerOpened`) se descubre automáticamente. Usa `when()` para una sola línea concisa y métodos con nombre para manejadores que necesiten varias sentencias o variables locales. El orden de despacho es: primero el manejador `when()`; luego el método `on_{event_type}`; luego `EventHandlerException` si no existe ninguno.

🍃 EQUIVALENCIA CON SPRING

`AggregateRoot + apply() + when()` es el equivalente en PyFly de `@Aggregate + AggregateLifecycle.apply(event) + @EventSourcingHandler` de Axon Framework. Axon usa el descubrimiento de manejadores guiado por anotaciones (`@EventSourcingHandler`); PyFly usa el registro con `when()` o la convención de métodos `on_*`. La mecánica de reproducción —cargar eventos del almacén, llamar a los mismos manejadores, reconstruir el estado— es idéntica en ambos frameworks.

El EventStore

El agregado sabe cómo producir y reproducir eventos. El `EventStore` sabe cómo persistirlos y recuperarlos. Estas son preocupaciones deliberadamente separadas: el agregado es lógica de negocio pura sin E/S; el almacén de eventos es E/S pura sin lógica de negocio.

El protocolo `EventStore` expone dos operaciones centrales:

- `append(aggregate_id, aggregate_type, events, *, expected_version)` — persiste un lote de eventos para un flujo. Lanza `ConcurrencyError` si la versión real del flujo no coincide con `expected_version`.

- `load(aggregate_id, *, after_sequence=0)` — devuelve la secuencia ordenada de objetos `StoredEventEnvelope` desde el primer evento (o desde `after_sequence`) hasta el más reciente.

`InMemoryEventStore` es la implementación lista para usar. Al igual que `InMemoryEventBus` en el Capítulo 8, se ejecuta enteramente en el proceso, sin E/S: ideal para desarrollo y pruebas. Un despliegue de producción cambia a un adaptador duradero como el `SQLAlchemyEventStore` de PyFly, que desde la v26.09.08 anexa en la unidad de trabajo de quien llama: dentro de `@transactional`, los eventos y la instantánea de un agregado se confirman o se revierten con el resto de la transacción de negocio.

`EventSourcedRepository` envuelve el `EventStore` y gestiona el ciclo completo de guardado/carga. El código de aplicación nunca llama al almacén directamente; llama a `repo.save(aggregate)` y `repo.load(aggregate_id)`, y el repositorio se encarga del resto.

Todas las importaciones provienen de dos ubicaciones:

```
# Core event-sourcing types – all in the base package
from pyfly.eventsourcing import (
    AggregateRoot,
    DomainEvent,
    EventStore,
    InMemoryEventStore,
    SnapshotStore,
    InMemorySnapshotStore,
    StoredEventEnvelope,
)
# The generic repository lives in the .repository submodule
from pyfly.eventsourcing.repository import EventSourcedRepository
```

El LedgerAccountRepository

Subclasificas `EventSourcedRepository` por dos motivos: para pasar la fábrica concreta y el almacén de snapshots a través de un único constructor bien nombrado y —opcionalmente— para sobrescribir `_envelope_to_event` de modo que los eventos reproducidos sean dataclasses tipadas reales en lugar de la bolsa de atributos genérica que produce la clase base.

❗ JERGA: SOBRE (ENVELOPE) Y BOLSA DE ATRIBUTOS

Un `StoredEventEnvelope` es la *forma de cable* que el almacén persiste: envuelve el `payload` del evento (un dict plano) junto con los campos de contabilidad que el almacén necesita — `aggregate_id`, `aggregate_type`, `sequence`, `event_type`, `event_id` y `occurred_at` —. Piénsalo como el sobre con la dirección alrededor de la carta. Una **bolsa de atributos** es el objeto genérico de sustitución que crea el repositorio base al cargar si no sobrescribes nada: un objeto sin nombre con las claves del payload fijadas como atributos. Funciona —los manejadores solo leen atributos—, pero no es tu dataclass `Credited` real, así que no puede comprobarse de tipos ni probarse con `isinstance`. La sobrescritura de abajo cambia ese anonimato por la cosa real.

Construye el repositorio en tres pequeños pasos.

Paso 1 — Asigna los nombres de clase de vuelta a las dataclasses. El almacén registra el `event_type` de cada evento como la cadena del nombre de clase (`"Credited"`). Para reconstruir la dataclass real al cargar, necesitas una búsqueda de esa cadena a la clase. Eso es `_EVENT_TYPES`: una entrada por tipo de evento.

Paso 2 — Subclasifica y reenvía el constructor.

`LedgerAccountRepository.__init__` toma el `store` (y un almacén de snapshots opcional `snapshots`) y reenvía todo a `super().__init__`, suministrando `factory=LedgerAccount` para que la clase base sepa cómo crear un agregado en blanco.

Paso 3 — Sobrescribe `_envelope_to_event` para una reproducción tipada. Busca el `event_type` en `_EVENT_TYPES`, filtra el payload almacenado quedándote con los campos que la dataclass realmente declara (usando `__dataclass_fields__`) y construye el evento real. Recurre a la hidratación genérica de la clase base para cualquier tipo de evento que no reconozcas.

lumen/models/repositories/ledger_repository.py

```

from __future__ import annotations

from typing import ClassVar

from lumen.models.entities.v1.ledger_account import (
    Credited,
    Debited,
    LedgerAccount,
    LedgerOpened,
)
from pyfly.eventsourcing import (
    DomainEvent,
    EventStore,
    SnapshotStore,
    StoredEventEnvelope,
)
from pyfly.eventsourcing.repository import EventSourcedRepository

# Map a stored event_type (the event class name) back to its dataclass.
_EVENT_TYPES: dict[str, type[DomainEvent]] = {
    LedgerOpened.__name__: LedgerOpened,
    Credited.__name__: Credited,
    Debited.__name__: Debited,
}

class LedgerAccountRepository(EventSourcedRepository[LedgerAccount]):
    """Loads/saves LedgerAccount aggregates via the event store."""

    SNAPSHOT_INTERVAL: ClassVar[int] = 100

    def __init__(
        self,
        store: EventStore,
        *,
        snapshots: SnapshotStore | None = None,
    ) → None:

```

```

super().__init__(
    store,
    factory=LedgerAccount,
    snapshots=snapshots,
    snapshot_interval=self.SNAPSHOT_INTERVAL,
)

@staticmethod
def _envelope_to_event(envelope: StoredEventEnvelope) → object:
    """Rebuild the concrete event dataclass from a stored payload.

    Overrides the base-class generic hydration so that replayed events
    are the same dataclasses the aggregate applied on the write side.
    Unknown fields are ignored for forward-compatibility.
    """
    event_cls = _EVENT_TYPES.get(envelope.event_type)
    if event_cls is None:
        # Fall back to generic hydration for unrecognised event types.
        return EventSourcedRepository._envelope_to_event(envelope)
    field_names = {
        f.name for f in event_cls.__dataclass_fields__.values()
    }
    kwargs = {
        k: v for k, v in envelope.payload.items() if k in field_names
    }
    return event_cls(**kwargs)

```

Listado 9.3 — LedgerAccountRepository: reproducción tipada mediante _envelope_to_event

Cómo funciona. `super().__init__(store, factory=LedgerAccount, ...)` indica al repositorio base que llame a `LedgerAccount()` —sin argumentos— para crear un agregado en blanco, asigne `.id` y luego reproduzca el flujo. `factory` acepta cualquier invocable sin argumentos; pasar la propia clase (`factory=LedgerAccount`) es equivalente a `factory=lambda: LedgerAccount()`.

La sobrescritura de `_envelope_to_event` busca la cadena `event_type` almacenada en `_EVENT_TYPES`, usa `__dataclass_fields__` para filtrar el payload a campos conocidos y reconstruye la dataclass real. Los campos desconocidos se descartan silenciosamente —compatibilidad hacia delante en la práctica—: si una versión futura del evento añade un campo que el manejador antiguo no reconoce, el libro mayor sigue reproduciéndose en lugar de fallar. El repliegue a la clase base, al final, gestiona cualquier tipo de evento que el repositorio no conozca.

❗ JERGA: COMPATIBILIDAD HACIA DELANTE (FORWARD-COMPATIBILITY)

Un lector es **compatible hacia delante** cuando puede ingerir datos escritos por una versión *más nueva* de sí mismo sin romperse: simplemente ignora las partes que no entiende. Descartar campos desconocidos del payload es exactamente eso: un escritor v2 puede añadir un `reference_code` a `Credited`, y un lector v1 aún en ejecución sigue plegando el evento correctamente en lugar de fallar ante el campo sorpresa.

Ejecútalo. Una prueba enfocada ejercita esta sobrescritura directamente: le entrega al repositorio un sobre construido a mano y comprueba que recibe de vuelta una dataclass `Credited` real:

```
uv run --extra dev pytest tests/test_ledger_event_sourcing.py -q -k
typed_replay
```

```
. [100%]
1 passed, 10 deselected in 0.05s
```

Qué acaba de ocurrir. Cableaste la frontera de persistencia sin escribir una sola línea de SQL ni de código de almacenamiento. El `EventSourcedRepository` base ya sabe cómo drenar los eventos pendientes en `save` y reproducirlos en `load`; tu subclase añadió solo dos cosas: una fábrica sin argumentos para poder construir un libro mayor en blanco, y un `_envelope_to_event` tipado para que los eventos que reproduce sean

exactamente las mismas dataclasses que el agregado emitió en la ruta de escritura. Esa simetría es de lo que se trata todo: la escritura y la reproducción ejecutan código idéntico.

Guardar, cargar y la prueba de la reproducción

Este es el momento hacia el que todo el capítulo ha estado avanzando: demostrar que un libro mayor recargado a partir de nada más que sus eventos almacenados es igual al libro mayor en vivo que los produjo. La prueba de abajo lo hace en dos mitades.

Paso 1 — Escribe un libro mayor y persístelo. Abre un libro mayor, acréditalo, debítalo y luego llama a `repo.save(account)`. El repositorio drena los tres eventos pendientes al almacén y limpia el búfer del agregado.

Paso 2 — Recarga desde cero y compara. Crea un repositorio *completamente nuevo* y llama a `load("acct-1")`. Nada en esta segunda mitad comparte memoria con el primer objeto. El saldo del libro mayor recuperado solo puede ser correcto si se recalculó reproduciendo el flujo almacenado, que es exactamente la prueba que queremos.

La siguiente prueba demuestra el ciclo completo de guardado y carga:

tests/test_ledger_event_sourcing.py

```

@pytest.mark.asyncio
async def test_balance_survives_reload_by_replay(
    event_store: InMemoryEventStore,
) → None:
    # 1. Open + credit + debit, then persist the pending events.
    account = LedgerAccount.open(
        "acct-1", owner_id="owner-7", currency=Currency.EUR
    )
    account.credit(Money(2500, Currency.EUR)) # +25.00
    account.debit(Money(1000, Currency.EUR)) # -10.00 → 15.00
    assert account.balance == Money(1500, Currency.EUR)

    repo = LedgerAccountRepository(event_store)
    await repo.save(account)
    # After committing, the aggregate has no pending events left.
    assert account.pending_events() == []

    # 2. Reconstruct from a *fresh* repository + aggregate. Nothing here
    #     carries the in-memory object's state – load() rebuilds the
    #     LedgerAccount purely by replaying the stored event stream.
    fresh_repo = LedgerAccountRepository(event_store)
    recovered = await fresh_repo.load("acct-1")

    assert recovered is not None
    assert recovered is not account
    assert recovered.owner_id == "owner-7"
    assert recovered.currency is Currency.EUR
    # The load-by-replay proof: balance recomputed from events, not stored.
    assert recovered.balance == Money(1500, Currency.EUR)
    # Three events were folded in: LedgerOpened, Credited, Debited.
    assert recovered.version == 3
    # A reconstructed aggregate has nothing pending – it was not "changed".
    assert recovered.pending_events() == []

```

Listado 9.4 — Prueba estelar: el saldo sobrevive a una recarga por reproducción

Cómo funciona. `repo.save(account)` llama a `store.append("acct-1", "LedgerAccount", pending_events, expected_version=0)`. Los tres eventos — `LedgerOpened`, `Credited`, `Debited` — se serializan en objetos `StoredEventEnvelope` y se escriben en el flujo en orden; el búfer de eventos pendientes del agregado se limpia a continuación.

`fresh_repo.load("acct-1")` llama a `store.load("acct-1")`, recibe los tres sobres, construye un `LedgerAccount()` en blanco mediante la fábrica, asigna `.id = "acct-1"` y pasa cada sobre por `_envelope_to_event` antes de reproducirlo. Los manejadores `_on_opened`, `_on_credited` y `_on_debited` se ejecutan en secuencia; tras los tres, `recovered.balance.amount` vale `1500`: el mismo valor que mantenía el agregado en vivo, calculado sin ningún estado compartido entre los dos objetos.

La prueba usa *dos instancias de repositorio independientes* que comparten el mismo almacén en memoria — `repo` para la escritura, `fresh_repo` para la lectura—. Esto demuestra que la reproducción, no la identidad de objetos dentro del proceso, es la fuente de verdad.

Ejecútalo. Ejecuta la prueba estelar más la prueba de inspección del flujo en crudo que la sigue:

```
uv run --extra dev pytest tests/test_ledger_event_sourcing.py -q -k
"reload_by_replay or immutable_event_stream"
```

```
.. [100%]
2 passed, 9 deselected in 0.05s
```

Qué acaba de ocurrir. Demostraste que el event sourcing realmente funciona de principio a fin. El primer libro mayor vivió solo en memoria; tras `save`, sus hechos vivieron solo en el almacén. Un segundo objeto de libro mayor, no relacionado,

reconstruyó entonces el mismo saldo exacto de 15,00 EUR plegando esos hechos de vuelta a través de los mismos manejadores, sin ninguna columna de saldo almacenada a la vista. La versión aterrizó en 3 (un evento por `LedgerOpened` / `Credited` / `Debited`), y el libro mayor reconstruido no tenía nada pendiente porque cargar no es lo mismo que cambiar.

El almacén de eventos también expone los sobres en crudo para inspección, útil para auditorías y pruebas:

tests/test_ledger_event_sourcing.py

```
@pytest.mark.asyncio
async def test_store_holds_the_immutable_event_stream(
    event_store: InMemoryEventStore,
) → None:
    account = LedgerAccount.open(
        "acct-2", owner_id="o", currency=Currency.GBP
    )
    account.credit(Money(800, Currency.GBP))
    account.debit(Money(300, Currency.GBP))
    await LedgerAccountRepository(event_store).save(account)

    envelopes = await event_store.load("acct-2")
    assert [e.event_type for e in envelopes] == [
        "LedgerOpened", "Credited", "Debited"
    ]
    assert [e.sequence for e in envelopes] == [1, 2, 3]
    assert all(e.aggregate_type == "LedgerAccount" for e in envelopes)
    # The Debited payload carries the post-debit running balance.
    assert envelopes[2].payload["balance"] == 500
    assert await event_store.latest_version("acct-2") == 3
```

Listado 9.5 — El almacén contiene el flujo de eventos inmutable

ARGUMENTO DE FÁBRICA

`EventSourcedRepository` acepta un invocable `factory` que produce una instancia de agregado en blanco. La fábrica debe devolver un agregado en su estado inicial de `__init__` —sin eventos aplicados— porque el repositorio aplicará la historia completa él mismo. Pasar una fábrica que construye un agregado ya mutado (por ejemplo, una que llama a `open()` internamente) corromperá la reproducción: `_on_opened` se ejecutaría dos veces, la segunda sobrescribiendo los campos que los eventos reales ya fijaron.

Concurrencia optimista

Dos peticiones concurrentes —un crédito desde la app móvil y un débito automático de comisiones desde un trabajo en segundo plano— pueden cargar el mismo libro mayor en la misma versión, aplicar cada una su propio cambio y luego intentar guardar. Sin una protección de concurrencia, un guardado gana silenciosamente, los eventos del otro se pierden, los números de secuencia colisionan y el saldo resultante es erróneo.

La **concurrencia optimista** previene esto. Antes de anexar eventos nuevos, el `EventStore` compara la versión *actual* del flujo contra la versión *esperada* que el repositorio registró en el momento de la carga. Si coinciden, el anexo prosigue y la versión avanza. Si no coinciden —porque otro escritor ya anexó—, se lanza `ConcurrencyError` y la petición perdedora debe reintentar desde una carga fresca.

❗ JERGA: BLOQUEO OPTIMISTA FRENTE A PESIMISTA

El bloqueo *pesimista* asume que el conflicto es probable, así que toma un bloqueo por adelantado: ningún otro escritor puede tocar la fila hasta que lo liberes. La concurrencia *optimista* asume que el conflicto es raro, así que no toma ningún bloqueo en absoluto: deja proceder a ambos escritores y solo comprueba, en el momento de guardar, si alguien más cambió el flujo primero. El perdedor reintenta. Para la mayoría de los libros mayores, dos escritores simultáneos sobre la *misma* cuenta son poco habituales, así que la estrategia optimista evita el coste del bloqueo en la ruta sin conflicto, abrumadoramente común.

El `expected_version` lo pasa el repositorio de forma implícita: registra la versión en la que cargó el agregado y se la suministra al almacén al guardar. Nunca gestionas números de versión en el código de aplicación.

La progresión de la versión es determinista: tras un guardado y una recarga, las escrituras posteriores avanzan la versión sin conflicto:

tests/test_ledger_event_sourcing.py

```

@pytest.mark.asyncio
async def test_continues_appending_after_a_reload(
    event_store: InMemoryEventStore,
) → None:
    repo = LedgerAccountRepository(event_store)
    account = LedgerAccount.open(
        "acct-3", owner_id="o", currency=Currency.EUR
    )
    account.credit(Money(1000, Currency.EUR))
    await repo.save(account)

    # Reload, mutate again, save again – version must advance, no conflict.
    reloaded = await repo.load("acct-3")
    assert reloaded is not None
    assert reloaded.version == 2
    reloaded.debit(Money(250, Currency.EUR))
    await repo.save(reloaded)

    final = await repo.load("acct-3")
    assert final is not None
    assert final.balance == Money(750, Currency.EUR)
    assert final.version == 3
    assert await event_store.latest_version("acct-3") == 3

```

Listado 9.6 — Continuar anexando tras una recarga: el bloqueo optimista avanza correctamente

Cómo funciona. Tras el primer guardado, el flujo está en la versión 2. Al cargar, `reloaded.version == 2`. `repo.save(reloaded)` anexa con `expected_version=2`; el almacén avanza a 3 y tiene éxito. La carga `final` reproduce los tres eventos y confirma el saldo correcto.

Ejecútalo.

```
uv run --extra dev pytest tests/test_ledger_event_sourcing.py -q -k  
continues_appending
```

```
. [100%]  
1 passed, 10 deselected in 0.05s
```

Qué acaba de ocurrir. Un libro mayor recargado recordó su versión, de modo que el siguiente guardado se alineó limpiamente detrás de los eventos ya presentes en el flujo: sin conflicto, números de secuencia en orden, saldo correcto. Esta es la *ruta feliz* de la concurrencia optimista: un escritor cada vez. En el momento en que dos escritores compiten, el `expected_version` del segundo ya no coincidiría y el almacén lanzaría `ConcurrencyError` en su lugar, que es el caso que la advertencia de abajo te dice cómo manejar.

⚠ MANEJA SIEMPRE CONCURRENCYERROR

Cuando dos escritores compiten, el guardado perdedor lanza `ConcurrencyError`. Desde la v26.09.08 es la `OptimisticLockingFailureException` del núcleo, así que uno no capturado se convierte en un HTTP 409 y un oyente de mensajes lo reintenta. Tu servicio de aplicación aún debe decidir qué hacer: reintentar el ciclo completo de cargar-mutar-guardar (apropiado para escrituras de baja contención), o exponer un 409 Conflict al llamante (apropiado cuando el llamante debería reenviar con datos frescos). Nunca tragues el error en silencio: un error de concurrencia tragado deja el flujo en un estado inconsistente.

Snapshots

El event sourcing cambia simplicidad de escritura por coste de lectura. Cargar un libro mayor que ha registrado 10 000 movimientos de dinero significa reproducir 10 000 eventos cada vez que se necesita el agregado. Para la mayoría de los libros mayores el flujo se mantiene corto; para cuentas de alta frecuencia puede crecer hasta volverse prohibitivamente largo.

Los **snapshots** abordan esto. Un snapshot es un punto de control serializado del estado del agregado en una versión concreta. Al cargar, el repositorio busca primero el snapshot más reciente, deserializa el estado directamente hasta esa versión y luego reproduce solo los eventos que llegaron después de él. Un snapshot en la versión 9 000 reduce una reproducción de 10 000 eventos a 1 000 eventos.

`InMemorySnapshotStore` almacena los snapshots en memoria. Pásalo a `EventSourcedRepository` junto con el almacén de eventos mediante el argumento clave `snapshots`, exactamente como lo acepta `LedgerAccountRepository.__init__`:

```
store = InMemoryEventStore()
snapshots = InMemorySnapshotStore()
repo = LedgerAccountRepository(store, snapshots=snapshots)
```

El repositorio decide cuándo hacer un snapshot automáticamente usando

`snapshot_interval` (por defecto `100`, fijado por `LedgerAccountRepository.SNAPSHOT_INTERVAL`). Tras cada `save`, comprueba si la nueva versión del agregado **cruza** un múltiplo de `snapshot_interval`:


```
# crosses_interval is True when this batch pushes the stream past a  
# 100-event boundary - e.g., version 95 → 105 crosses the 100 mark.  
crossed = (  
    (aggregate.version // snapshot_interval)  
    > (previous_version // snapshot_interval)  
)
```

Esta lógica de cruce de intervalo (en lugar de divisibilidad exacta) maneja el caso común en el que un único lote de guardado se sitúa a horcajadas sobre el umbral. Una importación masiva que añade 10 eventos lleva la versión de 95 a 105 y dispara correctamente un snapshot, aunque ni 95 ni 105 sean exactamente divisibles por 100.

La costura del snapshot es inofensiva cuando el flujo es más corto que el intervalo; la siguiente prueba lo demuestra:

tests/test_ledger_event_sourcing.py

```

@pytest.mark.asyncio
async def test_snapshot_store_round_trips_the_ledger() → None:
    """With a snapshot store wired, reload still yields the right state.

    The ledger's stream is far shorter than the snapshot interval, so this
    proves the snapshot seam is harmless and the repository falls back
    to a full replay.
    """
    store = InMemoryEventStore()
    snapshots = InMemorySnapshotStore()
    repo = LedgerAccountRepository(store, snapshots=snapshots)

    account = LedgerAccount.open(
        "acct-4", owner_id="o", currency=Currency.EUR
    )
    account.credit(Money(5000, Currency.EUR))
    await repo.save(account)

    recovered = await repo.load("acct-4")
    assert recovered is not None
    assert recovered.balance == Money(5000, Currency.EUR)

```

Listado 9.7 — Almacén de snapshots cableado: la recarga sigue produciendo el estado correcto

Cómo funciona. Tras el guardado, el repositorio comprueba: ¿es

`version 2 // 100 > 0 // 100` ? No; el umbral de snapshot no se ha cruzado, así que no se toma ningún snapshot. La siguiente `load` realiza una reproducción completa de los dos eventos y devuelve el saldo correcto. Una vez que un libro mayor cruza una frontera de 100 eventos, el repositorio serializa el estado del agregado en un sobre de snapshot. La siguiente carga encuentra el snapshot, deserializa directamente hasta esa versión y luego pide al almacén de eventos los eventos con un número de secuencia mayor que la versión del snapshot, reduciendo el coste de reproducción solo al delta.

Ejecútalo.

```
uv run --extra dev pytest tests/test_ledger_event_sourcing.py -q -k
snapshot_store_round_trips
```

```
. [100%]
1 passed, 10 deselected in 0.05s
```

Qué acaba de ocurrir. Cableaste un almacén de snapshots en el repositorio y el libro mayor aún se recargó correctamente, que es exactamente lo que debería pasar con un flujo corto. Como los dos eventos nunca cruzaron el intervalo de 100 eventos, no se escribió ningún snapshot y la carga recurrió a una simple reproducción completa. Los snapshots son optimización pura: cablear uno no cuesta nada para flujos cortos y rinde discretamente una vez que una cuenta se vuelve de alta frecuencia. Puedes demostrar la corrección de tu libro mayor con el almacén de snapshots presente o ausente: la respuesta es idéntica.

💡 EL INTERVALO DE SNAPSHOT EN PRODUCCIÓN

Un `snapshot_interval` de 100 es el valor por defecto y un punto de partida sensato. Para libros mayores de alta frecuencia podrías bajarlo; para cuentas que solo cambian unas pocas veces al día, un intervalo más alto reduce el coste de almacenamiento de snapshots. Los snapshots son una optimización, no un requisito de corrección: eliminarlos deja el sistema correcto pero más lento.

Cableado de arranque y autoconfiguración

El módulo `pyfly.eventsourcing` se incluye en el paquete **base** `pyfly`, sin dependencia extra. Habilitarlo lleva dos pasos: fija `pyfly.eventsourcing.enabled: true` en `pyfly.yaml` y anota la aplicación con `@enable_domain_stack`. La autoconfiguración de PyFly registra entonces automáticamente los beans `event_store` y `snapshot_store`.

❗ JERGA: BEAN Y AUTOCONFIGURACIÓN

Un **bean** es sencillamente un objeto que el framework crea una vez y entrega a cualquiera que lo pida: la misma idea de inyección de dependencias que viste en el Capítulo 2. La **autoconfiguración** es una clase de métodos de fábrica productores de beans que PyFly activa *solo cuando se cumple una condición*; aquí, la guarda `@conditional_on_property("pyfly.eventsourcing.enabled", having_value="true")`. Activa ese único indicador de configuración y los beans `event_store` y `snapshot_store` aparecen en el contenedor; déjalo apagado y la maquinaria de event sourcing permanece latente. Nunca llamas tú mismo a la fábrica.

La batería de pruebas lo confirma directamente:

tests/test_ledger_event_sourcing.py

```
def test_auto_configuration_registers_event_store_beans() → None:
    """enable_domain_stack activates this auto-config when
    pyfly.eventsourcing.enabled=true (set in pyfly.yaml), registering
    the in-memory event/snapshot stores the ledger repository depends on."""
    from pyfly.core.config import Config
    from pyfly.eventsourcing.auto_configuration import (
        EventSourcingAutoConfiguration,
    )

    auto = EventSourcingAutoConfiguration()
    cfg = Config() # empty config → providers default to "memory"
    assert isinstance(auto.event_store(cfg), InMemoryEventStore)
    assert isinstance(auto.snapshot_store(cfg), InMemorySnapshotStore)
```

Listado 9.8 — La autoconfiguración registra los beans del almacén de eventos

Ejecútalo.

```
uv run --extra dev pytest tests/test_ledger_event_sourcing.py -q -k
auto_configuration
```

```
. [100%]
1 passed, 10 deselected in 0.05s
```

Qué acaba de ocurrir. Con el indicador `enabled` activado, la autoconfiguración produjo los dos almacenes en memoria de los que depende el repositorio del libro mayor, sin que tú instanciaras ninguno. Observa que los métodos de fábrica toman un argumento `Config`: así es como leen `pyfly.eventsourcing.store.provider` para decidir entre el valor por defecto `memory` y un adaptador respaldado por SQL. Pasar un `Config()` vacío los deja en `memory`, que es lo que la prueba comprueba.

En el código de aplicación, el repositorio se cablea mediante inyección de dependencias:

```
# pyfly.yaml
pyfly:
  eventsourcing:
    enabled: true
```

```
# In a service or handler – the beans are injected automatically
@Component
class LedgerService:
    def __init__(
        self,
        event_store: EventStore,
        snapshot_store: SnapshotStore,
    ) → None:
        self._repo = LedgerAccountRepository(
            event_store, snapshots=snapshot_store
        )
```

Proyecciones

El almacén de eventos es el sistema de registro, pero la mayoría de las consultas de aplicación —"¿cuál es el saldo actual?", "muéstrame todos los libros mayores del propietario u-42", "¿qué cuentas están por encima de 1 000 EUR?"— no deben reproducir flujos de eventos en cada lectura. Deberían golpear un modelo de lectura precalculado: una tabla optimizada para consultas, mantenida sincronizada por un proceso en segundo plano que consume el flujo de eventos.

Ese proceso en segundo plano es una **proyección**. Una proyección se suscribe al flujo de eventos y actualiza un modelo de lectura cada vez que llega un evento relevante.

PyFly proporciona `FunctionProjection` y `ProjectionRunner` en

```
pyfly.eventsourcing.projection:
```

❗ JERGA: MODELO DE LECTURA, PROYECCIÓN, MODELO DE ESCRITURA

El **modelo de escritura** es la cara que has construido hasta ahora: el agregado y su flujo de eventos, optimizado para *registrar los cambios correctamente*. Un **modelo de lectura** es una copia separada de los datos, con forma de consulta, optimizada para *responder preguntas rápido* (una fila por libro mayor con su saldo actual, digamos). Una **proyección** es el proceso que mantiene un modelo de lectura sincronizado reproduciendo el flujo de eventos sobre él. Esta separación —un modelo para escrituras, otro para lecturas— es el patrón **CQRS** que viste en el Capítulo 7, ahora respaldado por un flujo de eventos en lugar de una tabla relacional.

- `FunctionProjection(name, handler_fn)` — envuelve una función asíncrona que recibe un `StoredEventEnvelope` y actualiza el modelo de lectura.
- `ProjectionRunner(projection, store)` — acciona la proyección iterando el `EventStore` en orden de número de secuencia y llamando al manejador para cada sobre.

Una proyección se arma en tres piezas. **Paso 1:** escribe el manejador, una función `async` que toma un sobre y actualiza el modelo de lectura (aquí un `dict` plano; en producción, una tabla de base de datos). **Paso 2:** envuélvelo, `FunctionProjection("balance_ledger", handler)` convierte la función pelada en una proyección con nombre. **Paso 3:** accionalo, `ProjectionRunner(projection, store)` conecta la proyección al almacén de eventos y, al arrancar, le alimenta cada sobre almacenado en orden de secuencia.

Aquí hay una `BalanceLedgerProjection` que construye un modelo de lectura de saldos a partir del flujo de eventos:

```
lumen/event sourcing/balance_projection.py
```

```
from __future__ import annotations

import asyncio

from pyfly.event sourcing import InMemoryEventStore
from pyfly.event sourcing.projection import FunctionProjection,
ProjectionRunner

# The in-process read model – in production, replace with a DB table.
_balance_store: dict[str, dict] = {}

async def _handle_envelope(envelope: object) → None:
    """Update the balance read model for each ledger event."""
    event_type: str = getattr(envelope, "event_type", "")
    payload: dict = getattr(envelope, "payload", {})

    if event_type == "LedgerOpened":
        _balance_store[payload["account_id"]] = {
            "account_id": payload["account_id"],
            "owner_id": payload.get("owner_id", ""),
            "balance_cents": 0,
            "currency": payload.get("currency", ""),
        }
    elif event_type in ("Credited", "Debited"):
        account_id = payload["account_id"]
        if account_id in _balance_store:
            _balance_store[account_id]["balance_cents"] = (
                payload["balance"]
            )

def build_projection(store: InMemoryEventStore) → ProjectionRunner:
    projection = FunctionProjection("balance_ledger", _handle_envelope)
    return ProjectionRunner(projection, store)
```



```

async def demo_projection(store: InMemoryEventStore) → None:
    runner = build_projection(store)
    # start() launches a background polling task and returns immediately;
    # the read model is populated asynchronously as the loop drains the
    # store. Poll until the projection has caught up, then stop the runner.
    await runner.start()
    try:
        for _ in range(50):
            if "led-001" in _balance_store:
                break
            await asyncio.sleep(0.05)
    finally:
        await runner.stop()

    balance = _balance_store.get("led-001", {})
    print(f"Balance read model: {balance}")

```

Listado 9.9 — *BalanceLedgerProjection*: un modelo de lectura construido a partir del flujo de eventos

Cómo funciona. `FunctionProjection("balance_ledger", _handle_envelope)` envuelve el manejador asíncrono. `ProjectionRunner(projection, store)` lo enlaza con el `InMemoryEventStore`. `await runner.start()` lanza una tarea de sondeo en segundo plano y retorna *de inmediato*; no se bloquea hasta que el almacén se drena. La tarea itera sobre `store.stream_all(...)`, llamando a `_handle_envelope` para cada sobre nuevo en orden y avanzando su posición en el flujo global del almacén para no reprocesar nunca un evento. Como la población ocurre asíncronamente, la demo sondea `_balance_store` hasta que la proyección se ha puesto al día, y luego llama a `await runner.stop()` para detener el bucle antes de leer el resultado. Solo después de que la proyección haya procesado los sobres, `_balance_store` refleja el estado actual de cada libro mayor del almacén.

La proyección es deliberadamente sin estado: solo lee `envelope.event_type` y `envelope.payload`. No se carga ningún agregado; no se llama a ningún repositorio. El modelo de lectura es barato de reconstruir: detén el runner, limpia `_balance_store`, llama a `start()` de nuevo. Esta propiedad de reconstruir-desde-la-historia es exclusiva del event sourcing; los modelos de almacenamiento de estado ya han descartado la historia.

En producción, `_handle_envelope` escribiría en una base de datos real (PostgreSQL, Redis, Elasticsearch). Pasa al `ProjectionRunner` un `CheckpointStore` (`checkpoints=`, como el bean `projection_checkpoint_store`) para que los reinicios continúen desde el último evento procesado en lugar de reproducir todo desde el principio; con `SqlAlchemyCheckpointStore` en la base de datos del modelo de lectura, cada lote y su checkpoint se confirman en una sola unidad de trabajo. El patrón de proyección es idéntico con independencia del almacenamiento subyacente.

PROYECCIONES FRENTE A LISTENERS DEL CAPÍTULO 8

La `BalanceProjection` del Capítulo 8 (Listado 8.4) era un suscriptor `@event_listener` sobre el `InMemoryEventBus`: reaccionaba a los eventos a medida que se publicaban. La `BalanceLedgerProjection` de este capítulo lee directamente del `EventStore`: puede reproducir la historia desde el principio, ponerse al día con el presente y continuar consumiendo eventos futuros. Ambas mantienen un modelo de lectura de saldos; la proyección sobre el almacén de eventos es reconstruible desde la historia; el listener del bus no lo es.

El outbox transaccional

Considera la llamada `repo.save(account)` del Listado 9.4: tres eventos se anexan al almacén de eventos. Ahora supón que esos eventos también necesitan llegar a un broker externo: Kafka, RabbitMQ, otro microservicio. El enfoque ingenuo es llamar a `broker.publish(envelope)` inmediatamente después de `store.append(...)`. Pero ¿y si el proceso se cae entre el anexo y la publicación? Los eventos están en el almacén, pero el broker nunca los recibió. El servicio aguas abajo nunca se enteró del crédito.

El patrón del **outbox transaccional** resuelve esto. En lugar de publicar directamente, encolas el evento en un *outbox*: un intermediario duradero. El outbox persiste el evento en la misma transacción de base de datos que el cambio que lo produjo. Un trabajador en segundo plano separado (el *relay*) drena el outbox y reenvía cada evento al broker con semántica de al menos una vez. Si el relay se cae, reinicia y reintenta desde el último evento no confirmado.

El `TransactionalOutbox` de PyFly vive en `pyfly.eventsourcing`. Está respaldado por tablas (las tablas `pyfly_outbox_*` del framework), así que necesita un origen de datos: un nombre del registro, un `AsyncEngine` como aquí, o el origen por defecto de la aplicación. Acepta una corrutina `publish` y un límite `max_attempts`, y expone dos métodos:

- `enqueue(envelope)` — escribe un sobre de evento en el outbox, en la unidad de trabajo ligada al origen de datos del outbox (o en una unidad breve propia).
- `start()` — comprueba (o crea) las tablas del outbox y arranca el relay en segundo plano que llama a `publish(envelope)` por cada elemento confirmado, reintentando hasta `max_attempts` veces en caso de fallo.

lumen/eventsourcing/outbox_demo.py

```

from __future__ import annotations

import asyncio

from sqlalchemy.ext.asyncio import create_async_engine

from pyfly.eventsourcing import InMemoryEventStore, TransactionalOutbox

from lumen.interfaces.enums.v1.currency import Currency
from lumen.models.entities.v1.ledger_account import LedgerAccount
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.ledger_repository import (
    LedgerAccountRepository,
)

# Simulated broker: collect published envelopes for inspection.
_published: list = []

async def _broker_publish(envelope: object) → None:
    _published.append(envelope)

async def demo_outbox() → None:
    engine = create_async_engine("sqlite+aiosqlite:///./ledger.db")
    store = InMemoryEventStore()
    repo = LedgerAccountRepository(store)
    outbox = TransactionalOutbox(
        publish=_broker_publish, datasource=engine, max_attempts=5
    )
    await outbox.start() # creates the pyfly_outbox_* tables if missing
    try:
        account = LedgerAccount.open("led-004", "u-11", Currency.EUR)
        account.credit(Money(5000, Currency.EUR))
        await repo.save(account)

```

```

# Write the stored envelopes into the outbox table.
for envelope in await store.load("led-004"):
    await outbox.enqueue(envelope)

# The relay publishes each one once its enqueue has committed.
for _ in range(50):
    if len(_published) == 2: # LedgerOpened + Credited
        break
    await asyncio.sleep(0.05)
    assert len(_published) == 2
finally:
    await outbox.stop()
    await engine.dispose()

```

Listado 9.10 — *TransactionalOutbox: entrega fiable de al menos una vez a un broker*

Cómo funciona. El outbox mantiene los sobres en una tabla, así que los eventos pendientes sobreviven a un reinicio. `_broker_publish` es la función de entrega; sustitúyela por tu productor de Kafka o RabbitMQ. `max_attempts=5` significa que el relay reintenta una entrega fallida hasta cinco veces antes de mandar el sobre a la cola de mensajes muertos (dead-letter). El relay trabaja en segundo plano: la demo espera a que lleguen los dos sobres y luego detiene el outbox. Un `enqueue` dentro de un método `@transactional` sobre el origen de datos del outbox forma parte de esa unidad: su evento existe exactamente cuando la unidad se confirma, y un rollback lo retira. (Hasta la v26.09.07 el outbox era un diccionario en el proceso: un evento encolado por una unidad que se revertía se publicaba igualmente, y un reinicio perdía lo pendiente. Ahora necesita una capa de datos; sin ella, `start()` lanza `IllegalTransactionStateError`.)

La garantía crítica: el outbox se drena de forma independiente de la petición que creó los eventos. Si el proceso se cae después de `repo.save(account)` pero antes de que el outbox termine de vaciarse, el siguiente reinicio retoma desde donde lo dejó y completa la entrega. El estado del agregado en el almacén de eventos ya es correcto; solo se interrumpió la entrega del lado del broker.

⚠ AL MENOS UNA VEZ, NO EXACTAMENTE UNA VEZ

El outbox garantiza que cada evento llega al broker *al menos una vez*. Si el relay entrega un evento y luego se cae antes de marcarlo como confirmado, el evento se entrega de nuevo en el reinicio. Tus consumidores del broker —y los servicios aguas abajo— deben ser idempotentes: usa el `envelope.event_id` como clave de deduplicación. El Capítulo 10 muestra dónde encuentra un consumidor de Kafka o RabbitMQ un id estable con el que deduplicar.

El outbox transaccional es el puente entre el event sourcing y la mensajería orientada a eventos. El Capítulo 10 retoma exactamente aquí, presentando los productores de Kafka y los exchanges de RabbitMQ y mostrando cómo configurar el relay para una entrega fiable a cada uno.

🍃 EQUIVALENCIA CON SPRING

El patrón del outbox transaccional es bien conocido en el ecosistema Spring con el mismo nombre. El `EventPublicationRegistry` de Spring Modulith y el `@TransactionalEventListener(phase = AFTER_COMMIT)` de Spring aproximan la misma garantía: el evento se registra de forma duradera antes de ser despachado. El almacén de eventos de Axon Server cumple un papel similar para las aplicaciones basadas en Axon: los eventos se escriben primero en el almacén, y los grupos de proyección / procesadores de eventos los consumen con garantías de al menos una vez desde el registro almacenado. El `TransactionalOutbox` de PyFly es el equivalente portable de ese patrón, sin requerir un servidor de eventos dedicado.

Avanzado: upcasting y multitenencia

Dos preocupaciones aparecen en todo sistema con event sourcing de larga vida. Esta sección las presenta brevemente; un tratamiento completo queda fuera del alcance de este libro.

Upcasting

Los eventos son inmutables. Una vez que un evento `Credited` se escribe en el flujo, no puedes volver atrás y añadir un campo `reference_code`. Pero los requisitos del producto cambian: dentro de tres meses el equipo de finanzas necesitará un código de referencia en cada crédito para la conciliación. Los eventos nuevos lo incluyen; los antiguos no.

Un **upcaster** es una función que transforma una forma de evento antigua en la forma actual durante la reproducción. El `EventStore` lo llama de forma transparente; el agregado nunca ve la forma antigua. Registras upcasters por tipo de evento y por versión:

```
# Conceptual – upcaster API varies by adapter  
def upcast_credited_v1(payload: dict) → dict:  
    payload.setdefault("reference_code", "LEGACY")  
    return payload
```

El upcaster se ejecuta cuando el `EventStore` carga un evento cuya versión de esquema es inferior a la actual. Los datos antiguos se vuelven legibles sin una migración; los datos nuevos se escriben en el esquema actual.

Multitenencia

Cuando varios inquilinos (tenants) comparten el mismo almacén de eventos, los eventos deben acotarse por inquilino para que un inquilino nunca pueda leer ni reproducir el flujo de otro. PyFly te da dos costuras para esto; ninguna vive en `EventSourcedRepository` (su constructor toma solo `store`, `factory`, `snapshots` y `snapshot_interval`: no hay parámetro `tenant_id`).

La primera costura está en el **propio sobre**. `StoredEventEnvelope` lleva un campo dedicado `tenant_id`, y `StoredEventEnvelope.of(...)` lo acepta como argumento clave:

```
envelope = StoredEventEnvelope.of(
    aggregate_id="led-001",
    aggregate_type="LedgerAccount",
    sequence=0,
    event=Credited(...),
    tenant_id="tenant-A",
)
```

El adaptador SQL lo persiste como una columna `tenant_id` en `pyfly_event_store`, de modo que un almacén consciente de inquilinos puede filtrar cada consulta por ella. Esto mantiene limpio el `aggregate_id` a la vez que particiona el registro por inquilino.

La segunda costura es un **patrón que implementas tú mismo**: prefija cada `aggregate_id` con el identificador del inquilino — `"tenant-A::led-001"` en lugar de `"led-001"` — cuando llames a `repo.save` y `repo.load`. Un envoltorio fino consciente de inquilinos alrededor del repositorio puede aplicar y retirar el prefijo para que el código de aplicación nunca lo vea:


```

class TenantLedgerRepository:
    def __init__(self, repo: LedgerAccountRepository, tenant_id: str) → None:
        self._repo = repo
        self._prefix = f"{tenant_id}::"

    async def load(self, account_id: str) → LedgerAccount | None:
        return await self._repo.load(self._prefix + account_id)

```

Las proyecciones deben acotar sus modelos de lectura de forma similar: normalmente incluyendo `tenant_id` como columna en la tabla del modelo de lectura y filtrando por ella en el momento de la consulta.

ELEGIR EVENT SOURCING

El event sourcing añade complejidad operativa: upcasters, gestión de snapshots, procedimientos de reconstrucción de proyecciones, monitorización del relay del outbox. Elígelo deliberadamente para dominios donde la auditabilidad y las consultas de viaje en el tiempo son requisitos de primera clase: libros mayores financieros, historiales médicos, registros de cadena de suministro. Para dominios con mucho CRUD donde lo único que importa es el estado actual, el almacenamiento de estado es más simple y suficiente.

Ejecuta el capítulo entero

Ejecutaste cada pieza de forma aislada a medida que la construías. Ahora ejecuta la batería completa del libro mayor para confirmar que todo sigue pasando en conjunto:

```
uv run --extra dev pytest tests/test_ledger_event_sourcing.py -q
```

```

.....
[100%]
11 passed in 0.06s

```

Once puntos: las cuatro pruebas de aislamiento del agregado, la prueba estelar de recarga por reproducción, la inspección del flujo en crudo, la sobrescritura de reproducción tipada, la comprobación de concurrencia tras la recarga, el round-trip del snapshot, la búsqueda de libro mayor desconocido y la comprobación del cableado de autoconfiguración. Con todas en verde, el agregado `LedgerAccount` y su repositorio están completos y son correctos.

✓ Lo que construiste

El `LedgerAccount` de Lumen es ahora un libro mayor totalmente orientado a eventos que coexiste con el `Wallet` con estado almacenado del Capítulo 6.

Las dos bases de agregado están separadas por diseño: `Wallet` extiende `pyfly.domain.AggregateRoot` y almacena su saldo en una fila de base de datos; `LedgerAccount` extiende `pyfly.eventsourcing.AggregateRoot` y deriva su saldo de un flujo de eventos inmutable. La distinción no es cosmética: la maquinaria de event sourcing (`apply` , `replay` , `when`) vive solo en la clase base de eventsourcing.

Los tres eventos de dominio — `LedgerOpened` , `Credited` , `Debited` — son `dataclass` es que extienden `pyfly.eventsourcing.DomainEvent` , cada uno con valores por defecto en sus campos para que el repositorio pueda reconstruirlos desde un payload almacenado. Los manejadores se registran con `when()` como lambdas de dos argumentos que delegan en métodos privados; la trampa del método ligado a evitar es que un método ligado ya es de un solo argumento desde el exterior, lo que provoca un `TypeError` en el momento del despacho.

`LedgerAccountRepository` es una subclase fina de `EventSourcedRepository[LedgerAccount]` que pasa la fábrica sin argumentos y sobrescribe `_envelope_to_event` para hidratar dataclasses concretas durante la

reproducción. La prueba estelar lo confirmó: tras abrir, acreditar y debitar un libro mayor, un repositorio y un agregado completamente nuevos reconstruyeron el saldo correcto puramente reproduciendo el flujo almacenado, sin columna de saldo almacenada, sin estado compartido.

El contador `version` acciona la protección de concurrencia optimista: el almacén rechaza cualquier `append` cuyo `expected_version` no coincida con la versión real del flujo, forzando al escritor perdedor a reintentar desde una carga fresca.

`InMemorySnapshotStore` resultó inofensivo cuando el flujo es más corto que el intervalo de snapshot y acelerará la reproducción una vez que un libro mayor cruce el umbral.

`BalanceLedgerProjection` usó `FunctionProjection` y `ProjectionRunner` para mantener un modelo de lectura de saldos rápido a partir del flujo de eventos en crudo, uno que puede reconstruirse desde la historia en cualquier momento, a diferencia del enfoque de listener del bus del Capítulo 8. `TransactionalOutbox` conectó luego el almacén de eventos con el mundo del broker, encolando eventos para una entrega de al menos una vez y reintentando en caso de fallo, de modo que ningún hecho se pierda silenciosamente entre el almacén y los consumidores aguas abajo. El Capítulo 10 retoma exactamente ahí, presentando los adaptadores de Kafka y RabbitMQ a los que apunta el relay del outbox.

Pruébalo tú mismo

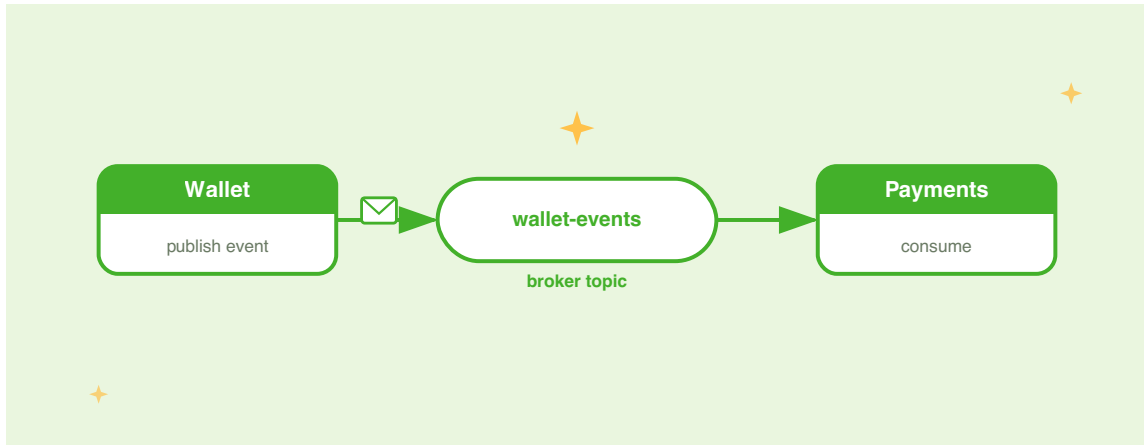
1. **Reproduce hasta un punto en el tiempo.** Aplica diez créditos de 100 céntimos cada uno a un `LedgerAccount`. Luego carga el agregado manualmente llamando a `await store.load("led-X")`, filtra los sobres a los que tengan `sequence ≤ 5` y reproduce solo esos a través de un `LedgerAccount()` nuevo. Comprueba que el

saldo resultante es igual a 400 céntimos (cuatro créditos después de la apertura) en lugar de 1 000 céntimos (diez créditos). Esta es la "consulta de viaje en el tiempo" que los modelos de almacenamiento de estado no pueden ofrecer.

2. **Implementa un evento `Transferred` y un guardado de doble agregado.** Añade un `Transferred(DomainEvent)` con los campos `source_id: str`, `target_id: str`, `amount: int`, `currency: str`. Añade un método `transfer_to(target: LedgerAccount, amount: Money) → None` a `LedgerAccount` que llame a `self.debit(amount)` y `target.credit(amount)` en secuencia, y luego aplique un evento `Transferred` sobre `self`. Cablea una corrutina `demo_transfer` que abra dos libros mayores, acredite 10 000 céntimos en el primero, transfiera 3 000 céntimos al segundo, guarde ambos agregados de forma independiente, recargue ambos desde el almacén y compruebe que los saldos son 7 000 y 3 000 céntimos respectivamente.
3. **Añade una `OwnerLedgerProjection`.** Escribe una segunda `FunctionProjection` llamada `owner_ledger` cuyo manejador mantenga un `dict[str, list[dict]]` que asigne cada `owner_id` a una lista cronológica de registros de transacción. Cada registro debe incluir `event_type`, `amount` (del payload para los eventos `Credited / Debited`) y el número de secuencia del sobre. Aliméntala con el mismo `InMemoryEventStore`, abre tres libros mayores para el mismo propietario, realiza una mezcla de créditos y débitos, arranca el runner de la proyección y comprueba que la lista de transacciones del propietario tiene el número correcto de entradas en el orden correcto.

CAPÍTULO 10

Mensajería con Kafka y RabbitMQ



El servicio de monederos de Lumen es genuinamente orientado a eventos. Los comandos fluyen a través de manejadores tipados; los eventos de dominio se publican en un bus en proceso; los oyentes reaccionan de forma independiente sin acoplarse a la ruta de escritura. En el Capítulo 8 construiste `WalletAuditListener`, un servicio que reacciona a los eventos `WalletOpened`, `FundsDeposited` y `FundsWithdrawn` sin conocer los manejadores de comandos. En el Capítulo 9 fuiste más allá, almacenando esos eventos *como la fuente de la verdad* para que cada saldo histórico se pueda calcular desde primeros principios.

Hay un límite que ninguno de los dos capítulos cruzó: la red. `InMemoryEventBus` vive dentro del proceso de Python. En el momento en que otro servicio —un futuro `PaymentsService` que liquide transferencias, o un `NotificationsService` que envíe alertas push— necesite reaccionar a los hechos de Lumen, necesitas un **broker de**

mensajería (message broker): un componente de infraestructura independiente que almacena eventos de forma duradera, los enruta a suscriptores en otros procesos y los reproduce cuando un consumidor se reinicia tras una caída.

Este capítulo lleva la base orientada a eventos de Lumen al otro lado de ese límite. Verás cómo PyFly envuelve la complejidad de Apache Kafka y RabbitMQ tras una única abstracción limpia — `MessageBrokerPort` — de modo que el código de aplicación nunca sepa qué broker se está ejecutando debajo. Publicarás los eventos de monedero de Lumen en topics reales, los consumirás con el decorador `@message_listener`, elegirás el formato de serialización adecuado para tus requisitos de evolución de esquema, gestionarás los mensajes envenenados con colas de mensajes muertos integradas en el decorador y protegerás tu servicio frente a caídas del broker con cortacircuitos (circuit breakers) y reintentos.

Al final del capítulo, los eventos de integración de Lumen fluyen a través de los límites de proceso, listos para los servicios de la Parte IV que los consumirán.

Lo construiremos gradualmente, una pieza cada vez. Cada funcionalidad viene con un recorrido numerado, el comando exacto que ejecutar y la salida que deberías esperar ver. Si has seguido el hilo desde el Capítulo 8, ya tienes `EventPublisher` conectado a los manejadores de comandos del monedero y el `WalletAuditListener` reaccionando en proceso; este capítulo está verificado contra PyFly v26.6.110 y el ejemplo de Lumen ubicado en `samples/lumen`. Nada de lo que hay aquí requiere un clúster de Kafka o RabbitMQ en ejecución para seguir adelante: PyFly incluye un broker en memoria que satisface el mismo contrato, de modo que puedes leer, ejecutar y probar cada listado antes de tocar siquiera Docker.

i LA JERGA, EN LENGUAJE SENCILLO

Un puñado de términos se repiten en este capítulo. Un **broker de mensajería** es un servidor independiente (Kafka o RabbitMQ) que almacena mensajes y los entrega a otros procesos. Un **topic** es un canal con nombre en el broker; los publicadores escriben en él y los suscriptores leen de él. Un **productor** (o *publicador*) coloca mensajes en un topic; un **consumidor** (u *oyente*) los retira y reacciona. Un **grupo de consumidores** es una etiqueta que permite que varias copias del mismo servicio compartan el trabajo, de modo que cada mensaje se gestione una sola vez. Un **serializador** convierte un objeto de Python en los **bytes** en bruto que el broker almacena; un **deserializador** convierte esos bytes de nuevo en un objeto en el otro lado. Una **cola de mensajes muertos** (DLQ) es un área de retención para mensajes que no pudieron procesarse. Un **adaptador** es el controlador concreto del broker que se oculta tras la interfaz **MessageBrokerPort**. Ten presentes estos ocho; el resto del capítulo trata, sobre todo, de conectarlos.

Una abstracción, muchos brokers

Por qué importa una abstracción

Antes de escribir una línea de código de Kafka o RabbitMQ, vale la pena preguntarse: ¿por qué introduce PyFly una capa de abstracción en absoluto? Tanto **aiokafka** como **aio-pika** exponen APIs asíncronas perfectamente usables. La respuesta es la misma razón por la que dependes de **EventPublisher** en lugar de **InMemoryEventBus**: la abstracción es lo que te permite intercambiar infraestructura sin tocar la lógica de negocio.

Sin una abstracción, cada servicio que produce o consume un mensaje importa tipos específicos de Kafka o de RabbitMQ. Cambiar de broker —o ejecutar Kafka en producción y un broker en memoria en CI— significa cambiar rutas de importación,

firmas de constructores y código repetitivo de bucles de consumo en cada archivo afectado. Con `MessageBrokerPort`, el cambio es una modificación de YAML. Los oyentes y publicadores que componen tu lógica de negocio nunca cambian.

La abstracción también da frutos en las pruebas. `InMemoryMessageBroker` satisface el protocolo del port. Inyéctalo allí donde se espera un `MessageBrokerPort` y escribe pruebas rápidas y deterministas sin dependencia de Docker. El Capítulo 16 lo concreta.

El protocolo `MessageBrokerPort`

`MessageBrokerPort` es un `@runtime_checkable Protocol`. Úsalo como anotación de tipo en todo tu código; llama a `isinstance(obj, MessageBrokerPort)` en tiempo de ejecución si necesitas verificar que un bean inyectado satisface el contrato.

El protocolo define cuatro métodos:


```

from pyfly.messaging import MessageBrokerPort

class MessageBrokerPort(Protocol):
    async def publish(
        self,
        topic: str,
        value: bytes,
        *,
        key: bytes | None = None,
        headers: dict[str, str] | None = None,
    ) → None: ...

    async def subscribe(
        self,
        topic: str,
        handler: MessageHandler,
        group: str | None = None,
    ) → None: ...

    async def start(self) → None: ...

    async def stop(self) → None: ...

```

Los cuatro métodos:

`publish` envía un único mensaje al topic con nombre. `value` son bytes en bruto: el protocolo deja la serialización deliberadamente en tus manos; codifica la carga útil antes de llamar a `publish` y decodifícala dentro de tu manejador. `key` y `headers` son de tipo keyword-only, de modo que quien llama no pueda transponerlos por accidente. `key` dirige la asignación de particiones de Kafka para las garantías de orden; RabbitMQ lo ignora. `headers` transportan metadatos transversales como `event-type` e identificadores de correlación.

`subscribe` registra un `MessageHandler` asíncrono para un topic. El parámetro opcional `group` se corresponde con los grupos de consumidores de Kafka y las colas de consumidores en competencia de RabbitMQ. Despliega tres instancias de un servicio, todas suscribiéndose con el mismo `group`, y solo una instancia procesa cada mensaje. Omite `group` para una semántica de difusión —cada suscriptor recibe cada mensaje—, lo cual resulta útil para analíticas que necesitan una copia de cada evento.

`start` crea las conexiones y comienza a consumir. Registra todas las suscripciones *antes* de llamar a `start`, y luego llámalo una sola vez durante el arranque de la aplicación.

`stop` drena los mensajes en vuelo y cierra las conexiones de forma limpia. El ciclo de vida de la aplicación de PyFly llama a `stop` automáticamente durante el apagado, así que rara vez necesitarás invocarlo a mano.

La dataclass `Message`

Cada manejador recibe un `Message`: una dataclass congelada que transporta el sobre completo de un mensaje recibido:

```
from pyfly.messaging import Message

msg = Message(
    topic="wallet.events",
    value=b'{"wallet_id": "w-001", "amount": 5000}',
    key=b"w-001",
    headers={"event-type": "FundsDeposited"},
)
```

Campo	Tipo	Valor por defecto	Descripción
topic	str	obligatorio	El topic o cola en el que llegó el mensaje.
value	bytes	obligatorio	La carga útil en bruto. La decodificas dentro de tu manejador.
key	bytes \ \nNone	None	Clave de partición o enrutamiento. Kafka la usa para la asignación de particiones.
headers	dict[str, str]	{}	Metadatos de cadena adjuntados por el publicador.

La dataclass está congelada: una vez que el broker te entrega un `Message` , sus campos son inmutables, seguros para pasar a través de límites asíncronos sin copia defensiva e inmunes a la mutación accidental dentro de los manejadores.

Kafka frente a RabbitMQ: elegir el broker adecuado

Antes de sumergirnos en la configuración, ayuda entender dónde encaja cada broker. La tabla siguiente resume las concesiones clave; ninguna de las dos opciones es universalmente correcta.

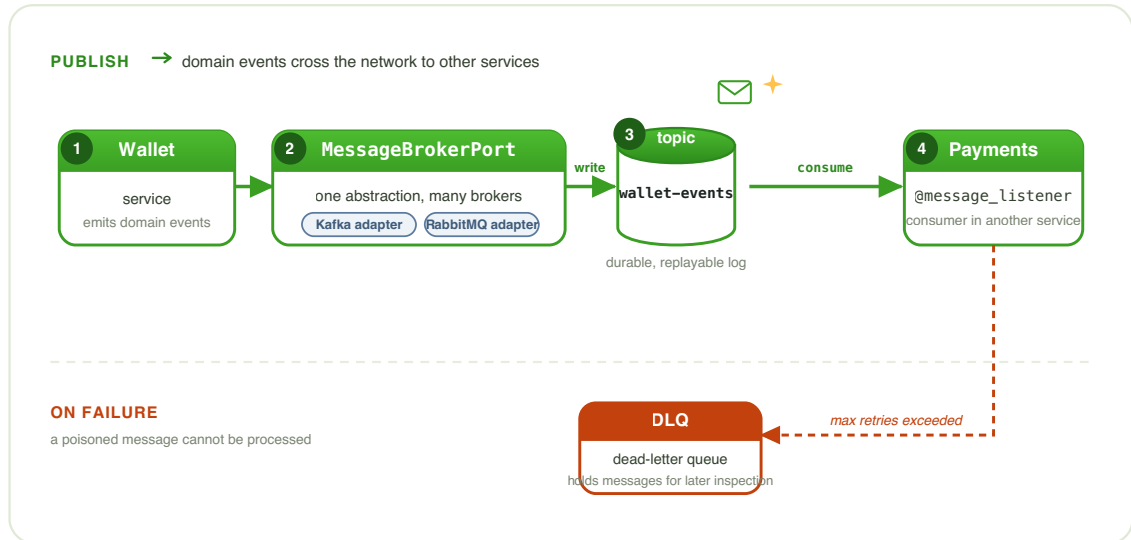


Figura 10.1 — MessageBrokerPort se sitúa entre el código de aplicación y los adaptadores de broker.

Dimensión	Apache Kafka	RabbitMQ
Modelo	Registro de confirmación (commit log) distribuido; los consumidores mantienen su propio offset	Broker de mensajería; los mensajes se eliminan de la cola tras la confirmación
Retención	Configurable (de días a indefinida); los consumidores pueden reproducir desde cualquier offset	Los mensajes se eliminan al entregarse; colas de mensajes muertos para los fallidos
Rendimiento	Millones de mensajes/segundo; optimizado para streaming	Decenas de miles/segundo; optimizado para enrutamiento de tareas
Orden	Garantizado dentro de una partición (productores con clave)	Garantizado dentro de una sola cola (FIFO)
Grupos de consumidores	Balanceo de carga nativo a nivel de partición	Colas de consumidores en competencia; un mensaje por consumidor
Evolución de esquema	Funciona bien con Avro/Protobuf + Schema Registry	Funciona bien con JSON; el acoplamiento de esquema es responsabilidad del usuario
Cuándo elegirlo	Streaming de eventos, registros de auditoría, reproducción, alto rendimiento	Colas de tareas, patrones RPC, enrutamiento por mensaje con bindings complejos
Extra de PyFly	<code>uv add "pyfly[kafka]"</code>	<code>uv add "pyfly[rabbitmq]"</code>

Para Lumen, Kafka es el ajuste natural: los eventos de monedero forman un flujo ordenado por monedero, merece la pena reproducirlos cuando un nuevo consumidor se conecta y, con el tiempo, alimentarán analíticas de alto rendimiento. Los ejemplos de este capítulo muestran ambos adaptadores de forma intercambiable: desde la perspectiva de tu código, la elección es un detalle de configuración.

INSTALAR AMBOS ADAPTADORES

Si quieres dar soporte a cualquiera de los dos brokers en una sola instalación, `uv add "pyfly[eda]"` incorpora tanto `aiokafka` como `aio-pika`. La autoconfiguración selecciona entonces Kafka si `aiokafka` es importable, RabbitMQ si `aio_pika` es importable, y recurre al broker en memoria si ninguno está presente.

Configurar los adaptadores

Conectar un broker a Lumen es una tarea de configuración, no de codificación. Añades un extra al proyecto, agregas un bloque `pyfly.messaging` a `pyfly.yaml` y PyFly hace el resto: construye el adaptador correcto y lo registra bajo el bean `MessageBrokerPort`, de modo que cualquier cosa que pida ese port reciba inyectado el broker en ejecución. Empezaremos con el broker que no necesita infraestructura alguna y luego pasaremos a Kafka y RabbitMQ.

❗ ACTIVA LA MENSAJERÍA CON UNA SOLA CLAVE

En la v26.6.110, el subsistema de mensajería solo se conecta por sí mismo cuando la clave `pyfly.messaging.provider` está **presente** en tu configuración. Sin clave no hay bean `MessageBrokerPort`: un deliberado "desactivado por defecto" para que una app sin mensajería no necesite incorporar ninguna biblioteca de broker. Una vez establecida la clave, su valor (`"memory"`, `"kafka"`, `"rabbitmq"` o `"auto"`) selecciona el adaptador. Las claves complementarias viven bajo el mismo bloque:

`pyfly.messaging.kafka.bootstrap-servers` y `pyfly.messaging.rabbitmq.url`.

Empieza con el broker en memoria

El `pyfly.yaml` de Lumen ya se ejecuta sobre el bus EDA en memoria del Capítulo 8. Para poner en marcha la abstracción de *mensajería* sin levantar Docker, añade una única línea `provider`.

Paso 1 — Habilitar el broker en memoria. Abre `pyfly.yaml` y añade un bloque `messaging` bajo `pyfly`:

```
pyfly:
  messaging:
    provider: "memory"
```

Paso 2 — Añadir un oyente para que haya algo que despertar. Coloca el oyente independiente del Listado 10.4 (unas páginas más adelante) en `src/lumen/messaging/payments_consumer.py`. En el arranque, PyFly descubre la función marcada y la suscribe por ti: no escribes ninguna llamada a `subscribe()`.

🔗 EJECÚTALO

Arranca la app. El broker en memoria no necesita ningún servidor externo, así que esto funciona en un portátil sin nada instalado:

```
uv run pyfly run
```

El banner de arranque informa de la versión del framework y el puerto enlazado (`pyfly.server.port`, `8080` por defecto en la v26.6.110):

```
:: PyFly Framework :: (v26.06.110) (Python 3.13.13)
app=lumen version=1.0.0 ... started_in=0.42s port=8080
```

No se imprime nada más todavía: no se ha publicado ningún mensaje. Eso es lo esperado: el broker está en ejecución y el oyente está suscrito, esperando. Las secciones siguientes le darán eventos que transportar.

Qué acaba de ocurrir. Una línea de YAML cambió el bean `MessageBrokerPort` de "no presente" a un broker en memoria funcional, y el framework autosuscribió tu `@message_listener` a él durante el arranque. Sin Kafka, sin RabbitMQ, sin Docker; y, sin embargo, *exactamente el mismo código* se ejecutará contra un broker real en cuanto cambies esa única línea a `"kafka"`. Esa propiedad de cambiar-sin-recompilar es todo el sentido de la abstracción.

Kafka

Cuando estés listo para un broker real, añade `pyfly[kafka]` a tu proyecto y apunta el provider a Kafka.

Paso 1 — Instalar el extra de Kafka. Esto incorpora `aiokafka`, el controlador asíncrono que envuelve el `KafkaAdapter` de PyFly:

```
uv add "pyfly[kafka]"
```


Paso 2 — Declarar el broker en `pyfly.yaml`. Cambia el provider y enumera tus brokers:

```
pyfly:
  messaging:
    provider: "kafka"
    kafka:
      bootstrap-servers: "kafka-1:9092,kafka-2:9092"
```

Eso es todo lo que PyFly necesita para autoconfigurar un `KafkaAdapter` y registrarlo como el bean `MessageBrokerPort`. (`bootstrap-servers` es una lista separada por comas de pares `host:port` —las direcciones de uno o más brokers del clúster—; el cliente descubre el resto a partir de cualquiera de ellos.) Para la mayoría de los servicios el YAML es suficiente; si necesitas opciones avanzadas de productor, construye el adaptador manualmente como un `@bean` dentro de una clase `@configuration`.

RabbitMQ

```
pyfly:
  messaging:
    provider: "rabbitmq"
    rabbitmq:
      url: "amqp://user:password@rabbitmq-host:5672/"
```

`RabbitMQAdapter` usa por defecto un exchange directo duradero llamado `"pyfly"`.

Para personalizar el nombre del exchange, construye el adaptador manualmente:

lumen/messaging/config.py

```

from pyfly.container import configuration, bean
from pyfly.messaging import MessageBrokerPort
from pyfly.messaging.adapters.rabbitmq import RabbitMQAdapter

@Configuration
class BrokerConfig:
    """Wire up the message broker bean."""

    @bean
    def broker(self) → MessageBrokerPort:
        return RabbitMQAdapter(
            url="amqp://user:password@rabbitmq-host:5672/",
            exchange_name="lumen-events",
        )

```

Listado 10.1 — Nombre de exchange de RabbitMQ personalizado mediante @bean

Cómo funciona. `@configuration` marca la clase como una fábrica que el contenedor de DI llama durante el arranque. `@bean` sobre `broker` le indica al contenedor que llame a `broker()` una vez, almacene en caché el resultado y lo inyecte allí donde se solicite `MessageBrokerPort`. Cualquier `@service` que declare `MessageBrokerPort` en su constructor recibe esta instancia automáticamente, sin necesidad de importar `RabbitMQAdapter` en la clase consumidora.

Autodetección

Cuando `provider` es `"auto"`, PyFly sondea los paquetes instalados en orden y elige el primer broker que encuentra:

Prioridad	Biblioteca comprobada	Adaptador seleccionado
1	<code>aiokafka</code>	<code>KafkaAdapter</code>
2	<code>aio_pika</code>	<code>RabbitMQAdapter</code>
3	<i>(reserva)</i>	<code>InMemoryMessageBroker</code>

`provider: "memory"` es distinto de `"auto"`: *siempre* selecciona el broker en memoria con independencia de lo que esté instalado, que es exactamente lo que quieres en las pruebas. Un `provider: "kafka"` o `"rabbitmq"` explícito omite el sondeo por completo y exige que la biblioteca de ese adaptador esté presente.

El patrón práctico es un YAML por entorno. Establece `provider: "memory"` en `pyfly-test.yaml` y `provider: "kafka"` en `pyfly-prod.yaml`, y cada ejecución de prueba y de producción usará el adaptador apropiado sin cambios de código.

🔔 EJECÚTALO

Puedes confirmar qué adaptador seleccionó PyFly sin enviar un solo mensaje. Con la mensajería habilitada, arranca la app y busca la línea del broker en el registro de arranque:

```
uv run pyfly run
```

```
pyfly.messaging provider=memory broker=InMemoryMessageBroker started
```

Cambia `provider` a `"kafka"` (con `pyfly[kafka]` instalado y un broker accesible) y reinicia; la misma línea informa ahora de `broker=KafkaAdapter`. El código de negocio que publica y consume no cambió: solo lo hizo el YAML.

Publicar eventos de integración

De eventos en proceso a eventos de integración

En el Capítulo 8, los manejadores de comandos de Lumen drenaban el búfer de eventos de la raíz de agregado `Wallet` con `wallet.clear_events()` y publicaban cada evento de dominio a través de `EventPublisher`. `WalletAuditListener` se suscribía usando `@event_listener` y reaccionaba dentro del mismo proceso.

El patrón de **evento de integración** cruza el límite de proceso. Mientras que un *evento de dominio* describe lo que ocurrió dentro de un agregado —un hecho privado, disponible para los oyentes del mismo proceso—, un evento de integración es una representación pública y depurada del mismo hecho: diseñada para consumidores externos, estable entre versiones y serializada a bytes para su transporte por un broker.

Para Lumen, el evento de integración de un depósito transporta solo lo que un consumidor externo necesita: el identificador del monedero, el importe en unidades menores, el código de moneda y el saldo resultante. No expone los detalles internos de implementación del agregado.

Cómo drena Lumen los eventos hacia el broker

Lumen separa el puente de publicación de los manejadores de comandos, de modo que cada manejador publique eventos de forma idéntica. `publish_domain_events` (en `lumen/core/services/wallets/event_publishing.py`) itera los eventos drenados, convierte cada dataclass congelada en un dict y llama a `EventPublisher.publish`:

```
# lumen/core/services/wallets/event_publishing.py (real Lumen code)
from pyfly.eda import EventPublisher
from pyfly.domain import DomainEvent

async def publish_domain_events(
    publisher: EventPublisher,
    events: Iterable[DomainEvent],
) → None:
    for event in events:
        payload = dataclasses.asdict(event)
        payload.setdefault("event_type", event.event_type)
        await publisher.publish(
            destination="wallet.events",
            event_type=event.event_type,    # "WalletOpened" /
            "FundsDeposited" / ...
            payload=payload,
        )
```

La firma de `EventPublisher.publish` es:

```
async def publish(
    self,
    destination: str,
    event_type: str,
    payload: dict[str, Any],
    headers: dict[str, str] | None = None,
) → None: ...
```

`destination` es el nombre lógico del canal (`"wallet.events"`). `event_type` es el nombre de la clase del evento de dominio — `"WalletOpened"`, `"FundsDeposited"` o `"FundsWithdrawn"` —, que es exactamente lo que filtran los suscriptores `@event_listener`.

Cada manejador de comandos conecta `EventPublisher` mediante el constructor y llama a `publish_domain_events` tras persistir:

lumen/core/services/wallets/deposit_funds_handler.py

```

from __future__ import annotations

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

from lumen.core.mappers.wallet_mapper import to_aggregate, to_entity
from lumen.core.services.wallets.deposit_funds_command import DepositFunds
from lumen.core.services.wallets.event_publishing import publish_domain_events
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.data.relational.sqlalchemy import LockMode, transactional
from pyfly.domain import AggregateNotFound
from pyfly.eda import EventPublisher

@command_handler
@service
class DepositFundsHandler(CommandHandler[DepositFunds, int]):
    """Credit funds to an existing wallet; returns the new balance."""

    def __init__(
        self,
        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
    ) → None:
        super().__init__()
        self._repository = repository
        self._events = events
        self._session_factory = session_factory

    @transactional()
    async def do_handle(self, command: DepositFunds) → int:
        entity = await self._repository.find_by_id(
            command.wallet_id, lock=LockMode.PESSIMISTIC_WRITE
        )

```

```

if entity is None:
    raise AggregateNotFound("Wallet", command.wallet_id)

wallet = to_aggregate(entity)
wallet.deposit(Money(
    amount=command.amount,      # integer minor units (e.g. 5000 =
€50.00)
    currency=wallet.currency,
))
await self._repository.upsert(to_entity(wallet))

# Drain pending events and forward them to the EDA bus.
await publish_domain_events(
    self._events, wallet.clear_events()
)
return wallet.balance.amount

```

Listado 10.2 — *DepositFundsHandler* drena eventos mediante *EventPublisher*

Decisiones de diseño clave:

`events: EventPublisher` es el port, no el adaptador. El contenedor de DI inyecta el bus que esté configurado —en memoria en las pruebas, un bus respaldado por broker en producción—. Este manejador nunca menciona Kafka ni RabbitMQ.

`command.amount` es el depósito en unidades menores enteras (p. ej., `5000` significa 50,00 € para un monedero en EUR). El evento de dominio `FundsDeposited` registra el mismo campo `amount`, más `currency` (una cadena como `"EUR"`) y `balance` (el nuevo saldo en unidades menores).

`wallet.clear_events()` drena la lista de eventos pendientes del agregado y los devuelve. Llamarlo *después* de `repository.add` garantiza que los eventos describan un hecho que persistió. Publicar antes de guardar crearía eventos fantasma: hechos sobre cosas que nunca ocurrieron.

Los eventos de dominio emitidos durante un depósito son instancias de:

```
@dataclass(frozen=True)
class FundsDeposited(DomainEvent):
    wallet_id: str = ""
    amount: int = 0      # integer minor units
    currency: str = ""   # e.g. "EUR"
    balance: int = 0     # new balance after deposit, minor units
```

Cuando `publish_domain_events` publica este evento, `event_type` es el nombre de la clase `"FundsDeposited"`, *no* una cadena con puntos como `"wallet.fundsdeposited"`.

Publicar un evento de integración directamente en el broker

Cuando un servicio independiente que se ejecuta en un proceso distinto necesita recibir los eventos de monedero de Lumen, el bus EDA debe estar respaldado por un adaptador de broker real. La carga útil que circula por el cable es el mismo dict que ven los oyentes en proceso. Un `EventPublisher` respaldado por broker se encarga del transporte, y el outbox transaccional (`pyfly.eda.outbox.enabled`, Capítulo 8) hace que espere al commit.

Ayuda ver la publicación primero en su forma más pequeña posible. El siguiente listado es una sencilla función `async` —sin clase, sin decorador— que toma un `MessageBrokerPort`, construye la carga útil y llama a `publish`. Constrúyela en tres movimientos:

Paso 1 — Codificar la carga útil a bytes. `MessageBrokerPort.publish` solo ve `bytes`, así que la función serializa el evento con `json.dumps(...).encode()`. El `.encode()` convierte la cadena JSON en bytes UTF-8 que el broker puede almacenar literalmente.

Paso 2 — Elegir una clave de partición. Pasar `key=wallet_id.encode()` le indica a Kafka que enrute cada mensaje de un monedero dado a la misma partición, lo que preserva su orden. (RabbitMQ ignora la clave, así que incluirla es inofensivo en cualquier caso.)

Paso 3 — Adjuntar la cabecera de tipo de evento. `headers={"event-type": "FundsDeposited"}` permite a un consumidor decidir si le interesa este mensaje *antes* de deserializar el cuerpo: enrutamiento barato.

lumen/messaging/deposit_publisher.py

```

from __future__ import annotations

import json

from pyfly.messaging import MessageBrokerPort

async def publish_deposit_event(
    broker: MessageBrokerPort,
    wallet_id: str,
    amount: int,
    currency: str,
    balance: int,
) → None:
    """Encode a FundsDeposited integration event and publish to the topic."""
    payload = json.dumps({
        "wallet_id": wallet_id,
        "amount": amount,          # integer minor units
        "currency": currency,      # e.g. "EUR"
        "balance": balance,        # new balance, minor units
        "event_type": "FundsDeposited",
    }).encode()

    await broker.publish(
        "wallet.events",
        payload,
        key=wallet_id.encode(),
        headers={"event-type": "FundsDeposited"},
    )

```

Listado 10.3 — Publicar un evento de integración de monedero en un topic de Kafka

Decisiones de diseño clave:

`broker: MessageBrokerPort` es el port, no el adaptador. El contenedor de DI inyecta el adaptador que esté configurado —Kafka en producción, el broker en memoria en las pruebas—.

`key=wallet_id.encode()` es la clave de enrutamiento. En Kafka, todos los mensajes que comparten la misma clave aterrizan en la misma partición, entregándolos a los consumidores en orden de publicación: crítico para un libro mayor donde el depósito antes de la retirada debe preservarse. En RabbitMQ la clave se ignora (el enrutamiento usa el binding del exchange), así que este campo es seguro de incluir con independencia del broker en ejecución.

`headers={"event-type": "FundsDeposited"}` usa el nombre de la clase del evento de dominio, no una ruta con puntos como `"wallet.fundsdeposited"`. Los consumidores pueden inspeccionar el tipo de evento sin decodificar la carga útil, lo cual es útil para el enrutamiento y el filtrado sin una deserialización completa.

Qué acaba de ocurrir. Cruzaste el límite de proceso. El mismo hecho `FundsDeposited` que `WalletAuditListener` consumió en proceso en el Capítulo 8 son ahora bytes en un topic, direccionables por cualquier servicio que se conecte al broker; y la función que los colocó ahí no nombra ningún broker, solo el port `MessageBrokerPort`. Intercambia el adaptador configurado y este código permanece sin cambios.

⚠ PUBLICA DESPUÉS DE GUARDAR, NO ANTES

Drena y publica siempre los eventos *después* de `repository.upsert(...)`. Si el guardado falla, ningún mensaje llega al broker. Aun así, una publicación directa se ejecuta antes de que la unidad de trabajo haga commit, y el broker conserva el mensaje si la unidad se revierte después. El outbox transaccional (`pyfly.eda.outbox.enabled: true`, Capítulo 8) añade cada publicación en la unidad de quien llama y la reenvía tras el commit: la garantía atómica para producción. La publicación directa que se muestra aquí es un punto de partida razonable para servicios más simples.

Consumir eventos con `@message_listener`

El problema del sondeo

Antes de los brokers, los servicios reaccionaban a los cambios de estado de otro servicio sondeando una base de datos compartida o un endpoint REST. El sondeo añade latencia (la reacción espera hasta el siguiente intervalo de sondeo), desperdicia recursos (la mayoría de los sondeos no encuentran nada nuevo) y acopla el consumidor al productor a nivel de API. Un oyente de mensajes elimina los tres problemas: el broker empuja el evento en cuanto está disponible, las conexiones inactivas consumen una CPU insignificante y el consumidor depende solo del esquema del mensaje, no de la API interna del productor.

Oyentes declarativos con `@message_listener`

`@message_listener` es el decorador de suscripción declarativa. Decora cualquier función o método asíncrono con el topic que debe consumir, y PyFly conecta la suscripción durante el arranque de la aplicación: sin referencia al bus, sin llamada a `subscribe()`, sin gestión del ciclo de vida en tu código.

La firma del decorador es:

```
def message_listener(  
    topic: str,  
    group: str | None = None,  
    *,  
    retries: int = 0,  
    retry_delay: float = 0.0,  
    dead_letter_topic: str | None = None,  
    ) → ...: ...
```

Parámetro	Tipo	Valor por defecto	Descripción
topic	str	obligatorio	El topic en el que escuchar.
group	str None	None	Nombre del grupo de consumidores.
retries	int	0	Veces que reinvocar el manejador ante un fallo.
retry_delay	float	0.0	Retardo base (segundos) entre reintentos: el intento N espera <code>retry_delay * N</code> .
dead_letter_topic	str None	None	Cuando se establece, un mensaje que sigue fallando tras <code>retries</code> se republica aquí.

El primer oyente es una función independiente, la forma más simple. Constrúyela en tres movimientos:

Paso 1 — Escribir una función asíncrona que tome un `Message`. Cada oyente recibe un argumento: el sobre congelado `Message` (`topic`, `value`, `key`, `headers`). La función debe ser `async` porque el broker la espera (`await`).

Paso 2 — Decorarla con el topic y el grupo.

`@message_listener(topic="wallet.events", group="payments-service")` es la suscripción entera. No hay ninguna llamada a `subscribe()` que escribir ni ningún bus que importar: el decorador marca la función con metadatos que el framework lee en el arranque.

Paso 3 — Decodificar dentro del manejador. El cuerpo comprueba la cabecera `event-type` y luego `json.loads(msg.value)` convierte los bytes en bruto de nuevo en un dict. El manejador decide qué le importa; aquí reacciona solo a `FundsDeposited`.

lumen/messaging/payments_consumer.py

```

from __future__ import annotations

import json

from pyfly.messaging import Message, message_listener

@message_listener(topic="wallet.events", group="payments-service")
async def on_wallet_event(msg: Message) → None:
    """React to every wallet event published to the topic."""
    event_type = msg.headers.get("event-type", "unknown")
    payload = json.loads(msg.value)

    if event_type == "FundsDeposited":
        wallet_id: str = payload["wallet_id"]
        amount: int = payload["amount"]          # minor units
        currency: str = payload["currency"]
        print(
            f"[Payments] Deposit received: "
            f"wallet={wallet_id} "
            f"amount={amount} {currency}"
        )

```

Listado 10.4 — @message_listener sobre una función independiente

Cómo funciona. El decorador almacena seis atributos de metadatos en la función envuelta — `__pyfly_message_listener__ = True`, más `__pyfly_listener_topic__`, `__pyfly_listener_group__`, `__pyfly_listener_retries__`, `__pyfly_listener_retry_delay__` y `__pyfly_listener_dlg__` —. Durante el arranque, el framework escanea todos los beans registrados, encuentra las funciones que llevan `__pyfly_message_listener__ = True` y llama a `broker.subscribe(topic, handler, group)` automáticamente. Nunca llamas a `subscribe()` a mano.

`group="payments-service"` coloca al consumidor en un grupo de consumidores.

Escala a múltiples instancias del servicio de pagos y solo una procesa cada mensaje: el broker distribuye la carga entre el grupo. Omite `group` para una semántica de difusión en la que cada instancia recibe cada mensaje.

Dentro del manejador, `msg.headers.get("event-type", "unknown")` inspecciona los metadatos del sobre antes de tocar la carga útil. El valor de la cabecera es el nombre de la clase del evento de dominio — `"FundsDeposited"`, `"WalletOpened"` o `"FundsWithdrawn"` —, coincidiendo con lo que Lumen establece en el lado del publicador.

 EJECÚTALO

Con `provider: "memory"` establecido (sin Docker), este es el viaje de ida y vuelta completo publicar → consumir en un solo lugar. Guarda el fragmento de abajo como `roundtrip.py` y ejecútalo con `uv run python roundtrip.py`:

```
import asyncio, json
from pyfly.messaging.adapters.memory import InMemoryMessageBroker
from lumen.messaging.payments_consumer import on_wallet_event

async def main() → None:
    broker = InMemoryMessageBroker()
    await broker.subscribe(
        "wallet.events", on_wallet_event, group="payments-service"
    )
    await broker.start()
    await broker.publish(
        "wallet.events",
        json.dumps({
            "wallet_id": "w-001", "amount": 5000,
            "currency": "EUR", "balance": 5000,
        }).encode(),
        headers={"event-type": "FundsDeposited"},
    )
    await asyncio.sleep(0.1)  # let the listener run
    await broker.stop()

asyncio.run(main())
```

El oyente imprime la línea que construyó a partir de la carga útil decodificada:

```
[Payments] Deposit received: wallet=w-001 amount=5000 EUR
```

Dentro de una app en ejecución *no* escribirías este cableado a mano: el decorador `@message_listener` y el bean del broker configurado hacen el `subscribe / start / stop` por ti. Este script independiente solo hace visible el viaje de ida y vuelta de forma aislada.

Qué acaba de ocurrir. Un mensaje que publicaste en `wallet.events` llegó a una función que nunca conectaste explícitamente a nada. El decorador transportó el topic y el grupo; el broker (aquí en memoria, en producción Kafka) hizo la entrega. Ese es el lado de consumo de la misma abstracción que usaste para publicar, y el cuerpo de la función es agnóstico respecto al broker de arriba abajo.

Oyentes en clases de servicio

Cuando un oyente necesita colaboradores —un repositorio, otro servicio—, decléralo como un método en una clase `@service`. PyFly inyecta las dependencias a través del constructor y conecta la suscripción del oyente después de que el bean se inicialice. La forma cambia solo ligeramente respecto a la versión independiente:

Paso 1 — Convertir la clase en un `@service`. Esto la registra en el contenedor de DI para que el framework pueda tanto inyectar su constructor como descubrir su método oyente.

Paso 2 — Declarar los colaboradores en el constructor. Aquí `smtp_client` representa un servicio de correo electrónico o push; el contenedor lo suministra. La función libre del Listado 10.4 no tenía dónde guardar tal dependencia: esa es la razón para recurrir a una clase.

Paso 3 — Decorar un *método* con `@message_listener`. La firma gana `self`, pero por lo demás el decorador y el cuerpo son idénticos a la forma de función. Como el bean se crea primero, `self._smtp` está listo para cuando llega un mensaje.

lumen/messaging/notifications_consumer.py

```

from __future__ import annotations

import json

from pyfly.container import service
from pyfly.messaging import Message, message_listener

@service
class WalletNotificationConsumer:
    """Sends push notifications when wallet events arrive via the broker."""

    def __init__(self, smtp_client: object) → None:
        # smtp_client would be an injected email/push service.
        self._smtp = smtp_client

    @message_listener(topic="wallet.events", group="notifications-service")
    async def on_wallet_event(self, msg: Message) → None:
        event_type = msg.headers.get("event-type", "unknown")

        if event_type != "WalletOpened":
            return

        payload = json.loads(msg.value)
        owner_id: str = payload.get("owner_id", "")
        wallet_id: str = payload.get("wallet_id", "")
        currency: str = payload.get("currency", "")

        print(
            f"[Notification] Welcome {owner_id}! "
            f"Your {currency} wallet {wallet_id} is ready."
        )

```

Listado 10.5 — @message_listener sobre un método de @service con dependencias

Cómo funciona. `@service` registra `WalletNotificationConsumer` en el contenedor de DI. El constructor recibe `smtp_client` mediante inyección. Después de crear el bean, el framework detecta `on_wallet_event` llevando `__pyfly_message_listener__ = True` y lo registra como un oyente de método enlazado (bound-method): `self` ya está capturado, así que cada invocación tiene acceso completo a `self._smtp`.

El retorno temprano cuando `event_type != "WalletOpened"` es una guarda de filtrado. Un único topic (`wallet.events`) transporta múltiples tipos de evento, así que cada oyente filtra los que le interesan. Esto es más simple que mantener un topic separado por tipo de evento, aunque para flujos de muy alto volumen, un topic-por-tipo es una concesión de diseño legítima.

🔍 SEMÁNTICA DE GRUPOS DE CONSUMIDORES DE UN VISTAZO

Dos servicios con nombres de grupo *distintos* reciben cada uno cada mensaje: el broker entrega una copia a cada grupo. Dos *instancias* del mismo servicio que comparten el *mismo* nombre de grupo comparten la carga: cada mensaje va exactamente a una instancia. Usa grupos distintos para fanout (pagos y notificaciones necesitan ambos el evento); usa el mismo grupo para escalado horizontal (tres instancias del servicio de pagos comparten el trabajo).

Serialización y evolución de esquema

Por qué bytes, y por qué importa

`MessageBrokerPort.publish` acepta `bytes` en bruto. Esa es una elección deliberada. Un adaptador de broker que forzara un único formato de serialización sería cómodo para los casos simples y doloroso para todo lo demás: la evolución de esquema, los

consumidores multilinguaje, los requisitos de cumplimiento y las restricciones de rendimiento empujan en direcciones distintas. Al dejar la serialización en tus manos, PyFly se mantiene al margen.

Vale la pena conocer tres formatos: JSON por su simplicidad, Avro para la evolución respaldada por un registro de esquemas, y Protobuf para entornos críticos en rendimiento o multilinguaje:

Formato	Legible por humanos	Cumplimiento de esquema	Evolución de esquema	Multilinguaje	Codificación en PyFly
JSON	Sí	Opcional	Manual (disciplina del consumidor)	Universal	<code>json.dumps(...).encode()</code>
Avro	No	Sí (vía registro)	De primera clase (<code>BACKWARD</code> / <code>FORWARD</code> / <code>FULL</code>)	Buena	Biblioteca <code>fastavro</code>
Protobuf	No	Sí (archivos <code>.proto</code>)	De primera clase (numeración de campos)	Excelente	Biblioteca <code>protobuf</code>

JSON: empieza aquí

La *serialización* es solo el acto de convertir un objeto en memoria en una secuencia plana de bytes que puedes almacenar o enviar, y la *deserialización* es lo inverso. Los tres formatos siguientes difieren únicamente en cuán compactos son esos bytes y con cuánta rigurosidad vigilan la forma de los datos.

JSON es el valor por defecto adecuado. No requiere herramientas más allá de la biblioteca estándar, cualquier lenguaje puede analizarlo y la carga útil es legible en las interfaces de monitorización del broker. El patrón de codificación son dos líneas:

```
import json

payload: bytes = json.dumps({
    "wallet_id": "w-001",
    "amount": 5000,           # integer minor units (€50.00)
    "currency": "EUR",
    "balance": 10000,        # new balance, minor units
    "event_type": "FundsDeposited",
}).encode()

await broker.publish("wallet.events", payload)
```

Decodificar en el consumidor:

```
data: dict = json.loads(msg.value)
```

La debilidad de JSON es que el esquema no se hace cumplir. Si un publicador añade un campo obligatorio y el consumidor no se ha actualizado, el consumidor se rompe silenciosamente. Para los eventos internos de Lumen, donde productor y consumidor se despliegan juntos, esto es manejable. Para los eventos compartidos con equipos externos o los topics de larga vida, necesitas garantías más fuertes.

Avro: evolución respaldada por un registro de esquemas

Los esquemas de Avro son documentos JSON que describen la forma de un mensaje. Un Schema Registry (el de Confluent es el más común, pero existen alternativas de código abierto) almacena esos esquemas y hace cumplir las reglas de compatibilidad cuando los productores registran nuevas versiones. La biblioteca `fastavro` codifica y decodifica la carga útil binaria. La ruta de publicación es el mismo

`broker.publish(...)` que ya conoces; solo cambia el paso de codificación:

Paso 1 — Declarar el esquema una vez. `WALLET_DEPOSITED_SCHEMA` enumera cada campo y su tipo Avro (`string` , `long`). Es una constante a nivel de módulo, de modo que se escribe una vez, no por mensaje.

Paso 2 — Compilarlo una vez. `fastavro.parse_schema(...)` se llama en tiempo de importación y el resultado se almacena en caché en `_PARSED` . Analizarlo en cada publicación sería trabajo desperdiciado en la ruta caliente.

Paso 3 — Codificar y publicar. `fastavro.schemaless_writer` serializa el registro en un búfer `BytesIO` ; `buf.getvalue()` entrega los bytes a `broker.publish` exactamente como hacía la ruta JSON.

lumen/messaging/avro_publisher.py

```

from __future__ import annotations

import io

import fastavro # type: ignore[import]

from pyfly.messaging import MessageBrokerPort

WALLET_DEPOSITED_SCHEMA = {
    "type": "record",
    "name": "FundsDeposited",
    "namespace": "lumen.wallet",
    "fields": [
        {"name": "wallet_id", "type": "string"},
        {"name": "amount", "type": "long", # integer minor units
        {"name": "currency", "type": "string"},
        {"name": "balance", "type": "long", # new balance, minor units
    ],
}

_PARSED = fastavro.parse_schema(WALLET_DEPOSITED_SCHEMA)

async def publish_deposit_avro(
    broker: MessageBrokerPort,
    wallet_id: str,
    amount: int,
    currency: str,
    balance: int,
) → None:
    """Encode a FundsDeposited event with Avro and publish to the topic."""
    record = {
        "wallet_id": wallet_id,
        "amount": amount, # integer minor units
        "currency": currency,
        "balance": balance,
    }

```



```

buf = io.BytesIO()
fastavro.schemaless_writer(buf, _PARSED, record)

await broker.publish(
    "wallet.events",
    buf.getvalue(),
    headers={"content-type": "avro/binary",
             "event-type": "FundsDeposited"},
)

```

Listado 10.6 — Publicar un evento de monedero con codificación Avro

Cómo funciona. `fastavro.parse_schema` compila el documento de esquema JSON una vez en el momento de la carga del módulo; nunca lo analices dentro de la función de publicación o pagarás el coste de compilación en cada llamada.

`fastavro.schemaless_writer` serializa el registro en el búfer `BytesIO` sin incrustar el esquema en cada mensaje (el registro proporciona el esquema en el lado del consumidor). `buf.getvalue()` extrae los bytes para `broker.publish`.

Las cabeceras `headers={"content-type": "avro/binary", "event-type": "FundsDeposited"}` señalan a los consumidores que se requiere decodificación Avro y transportan el tipo de evento para el enrutamiento, de forma coherente con la convención de JSON.

Protobuf: rendimiento y poliglotismo

Los Protocol Buffers compilan un archivo `.proto` en una clase generada. Producen mensajes más pequeños que JSON o Avro, y el código generado está disponible en todos los lenguajes principales, lo que convierte a Protobuf en la elección correcta cuando el consumidor es un servicio en Go o Java.

```
# Assumes a generated class lumen_pb2.FundsDeposited
from lumen_pb2 import FundsDeposited # type: ignore[import]

event = FundsDeposited(
    wallet_id="w-001",
    amount=5000,          # integer minor units
    currency="EUR",
    balance=10000,
)
payload: bytes = event.SerializeToString()

await broker.publish(
    "wallet.events",
    payload,
    headers={"content-type": "application/protobuf",
            "event-type": "FundsDeposited"},
)
```

Decodificar en el consumidor sigue el patrón espejo:

```
from lumen_pb2 import FundsDeposited # type: ignore[import]

event = FundsDeposited()
event.ParseFromString(msg.value)
```

💡 EMPIEZA CON JSON, MIGRA CUANDO SIENTAS EL DOLOR

La progresión correcta para la mayoría de los equipos es: JSON primero (rápido de entregar, fácil de depurar); añade Avro cuando múltiples equipos sean dueños de distintos lados de un topic y la deriva de esquema se convierta en un coste de coordinación real; cambia a Protobuf cuando el tamaño binario o la interoperabilidad multilenguaje sean un requisito estricto. Como el `publish` y el `@message_listener` de PyFly aceptan bytes en bruto, puedes cambiar el formato de serialización sin cambiar las llamadas a la API del broker: solo intercambia los pasos de codificación y decodificación.

Cuando la entrega falla: colas de mensajes muertos

El inevitable mensaje defectuoso

Incluso un consumidor bien diseñado acabará por encontrarse con un mensaje que no puede procesar. Una base de datos aguas abajo puede no estar disponible. La carga útil puede violar una suposición de la que el consumidor dependía. Un error de red transitorio puede interrumpir una llamada a una API de terceros a mitad del manejador. La pregunta no es si un consumidor fallará, sino qué ocurre cuando lo hace.

Sin una estrategia de mensajes muertos, un consumidor fallido o bien descarta el mensaje (pérdida de datos) o lo vuelve a encolar indefinidamente, creando un bucle infinito de reintentos que bloquea todos los mensajes posteriores: una *píldora envenenada*. Una **cola de mensajes muertos** (DLQ) es la respuesta estructurada: un topic o cola aparte donde los mensajes que no pueden procesarse tras un número configurable de intentos se aparcen para su inspección y reprocesamiento manual.

Reintento y DLQ nativos del decorador

En PyFly, el reintento y el enrutamiento a mensajes muertos están integrados en `@message_listener`: sin andamiaje de try/except, sin publicación manual en la DLQ. Declara `retries` y `dead_letter_topic` directamente en el decorador. Añades resiliencia añadiendo *argumentos*, no código:

Paso 1 — Parte de un oyente normal. El cuerpo del manejador en el siguiente listado es un consumidor corriente que decodifica la carga útil y llama a un trabajador (`_charge`).

Paso 2 — Añade `retries` (y opcionalmente `retry_delay`). `retries=3` le indica al framework que reinvoque el manejador hasta tres veces más si lanza una excepción. `retry_delay=0.5` espacia esos intentos con un retroceso lineal.

Paso 3 — Añade `dead_letter_topic`. Cuando se agotan todos los reintentos, el framework republica el mensaje allí en lugar de dejar que la excepción tumbe al consumidor. No escribes ni el bucle de reintentos ni la publicación en la DLQ.

lumen/messaging/resilient_consumer.py

```

from __future__ import annotations

import json
import logging

from pyfly.container import service
from pyfly.messaging import Message, message_listener

logger = logging.getLogger(__name__)

@service
class ResilientWalletConsumer:
    """Processes wallet events with built-in retry and DLQ fallback."""

    @message_listener(
        topic="wallet.events",
        group="payments-dlq",
        retries=3,
        retry_delay=0.5,          # waits 0.5s, 1.0s, 1.5s (linear)
        dead_letter_topic="wallet.events.DLQ",
    )
    async def on_wallet_event(self, msg: Message) → None:
        payload = json.loads(msg.value)
        event_type = msg.headers.get("event-type", "unknown")
        logger.info(
            "Processing event: type=%s wallet=%s",
            event_type,
            payload.get("wallet_id"),
        )
        # Any unhandled exception triggers a retry; after 3 retries
        # the original message is forwarded to wallet.events.DLQ.
        await self._charge(payload)

    async def _charge(self, payload: dict) → None:
        raise NotImplementedError("replace with real payment logic")

```

Listado 10.7 — Reintento y DLQ conectados a través de `@message_listener`

Los tres parámetros:

`retries=3` permite hasta tres entregas más del mensaje tras el primer fallo (cuatro en total). Los reintentos son apropiados para fallos *transitorios* (un único nodo de base de datos reiniciándose); mantén el conteo bajo y deja que la DLQ gestione los fallos sostenidos.

`retry_delay=0.5` aplica un retroceso lineal: el intento 1 espera 0,5 s, el intento 2 espera 1,0 s, el intento 3 espera 1,5 s. Si omities ambos, el oyente recibe la política del contenedor (`pyfly.messaging.listener.retry.*`): cinco intentos, separados 1 s y duplicándose, como máximo 30 s.

`dead_letter_topic="wallet.events.DLQ"` es la red de seguridad. Cuando se agotan todos los reintentos, el framework republica el mensaje original en el topic de la DLQ, preservando el `value` y la `key` originales, y añade cabeceras de diagnóstico, entre ellas:

Cabecera	Valor
<code>x-original-topic</code>	El topic del que se consumió originalmente el mensaje.
<code>x-exception</code>	El nombre de la clase de la excepción (p. ej., <code>RuntimeError</code>).

En Kafka y RabbitMQ el contenedor añade también `x-exception-message`, `x-dlt-attempts` y de dónde venía el registro. Después el mensaje se confirma para que el consumidor siga funcionando: queda aparcado, no perdido, y el siguiente mensaje del topic se procesa con normalidad.

 EJECÚTALO

Puedes ver cómo un mensaje envenenado aterriza en la DLQ sin un broker real. En el momento del cableado, el framework envuelve cada oyente con `pyfly.messaging.error_handling.wrap_listener`: el mismo ayudante hace el reintento y el envío a mensajes muertos. Acciónalo directamente para que el flujo sea visible. Guarda esto como `dlq_demo.py` y ejecuta `uv run python dlq_demo.py`:

```
import asyncio, json
from pyfly.messaging.adapters.memory import InMemoryMessageBroker
from pyfly.messaging.error_handling import wrap_listener
from pyfly.messaging.types import Message

async def always_fails(msg: Message) → None:
    raise RuntimeError("downstream unavailable")

async def main() → None:
    broker = InMemoryMessageBroker()
    await broker.start()

    async def show_dlq(msg: Message) → None:
        print("DLQ:", msg.headers["x-original-topic"],
              msg.headers["x-exception"])
    await broker.subscribe("wallet.events.DLQ", show_dlq)

    handler = wrap_listener(
        always_fails, broker,
        retries=2, dead_letter_topic="wallet.events.DLQ",
    )
    await handler(Message(
        topic="wallet.events",
        value=json.dumps({"wallet_id": "w-001"}).encode(),
    ))
    await broker.stop()

asyncio.run(main())
```

Tras dos reintentos, el envoltorio se rinde y republica en la DLQ, donde tu monitor imprime las cabeceras de diagnóstico:

```
DLQ: wallet.events RuntimeException
```

El manejador retornó con normalidad —la excepción se tragó, no se propagó—, así que en una app real el consumidor simplemente pasaría al siguiente mensaje.

Monitorizar la DLQ

Suscríbete al topic de la DLQ como a cualquier otro oyente para observar y alertar sobre los mensajes enviados a mensajes muertos:

lumen/messaging/dlq_monitor.py

```

from __future__ import annotations

import json
import logging

from pyfly.messaging import Message, message_listener

logger = logging.getLogger(__name__)

@message_listener(topic="wallet.events.DLQ", group="dlq-monitor")
async def on_dead_letter(msg: Message) → None:
    """Log every message that failed all retries."""
    original = msg.headers.get("x-original-topic", "unknown")
    exc = msg.headers.get("x-exception", "unknown")
    payload = json.loads(msg.value) if msg.value else {}
    logger.warning(
        "DLQ message: original_topic=%s exception=%s wallet=%s",
        original,
        exc,
        payload.get("wallet_id"),
    )

```

Listado 10.8 — Suscribirse al topic de la DLQ

⚠ DISEÑA LOS CONSUMIDORES PARA QUE SEAN IDEMPOTENTES

Un consumidor que alcanza el límite de reintentos de la DLQ ha consumido el mensaje. Si más tarde un operador reproduce el mensaje de la DLQ, el consumidor lo procesará de nuevo. Sin idempotencia, ese doble procesamiento puede corromper datos: abonar dos veces un monedero, enviar una notificación duplicada. Usa el identificador estable del mensaje como clave de idempotencia: antes de procesar, comprueba si ese ID ya se ha registrado en una tabla `processed_events` y omite el trabajo si así es. El paso de comprobar-y-registrar debería estar en la misma transacción de base de datos que la escritura de negocio, que es justo lo que te da la unidad de trabajo de la entrega (siguiente sección).

Transacciones de los oyentes: confirmación tras el commit

En Kafka y RabbitMQ, el contenedor de oyentes ejecuta cada entrega en una unidad de trabajo y confirma el mensaje —hace commit del offset de Kafka, envía el ack de AMQP— solo después de que esa unidad haga commit. Cada llamada al repositorio que hace el oyente se une a la unidad, así que lo que escribe una entrega se confirma de una vez o nada, y un proceso que cae antes del commit recibe el mensaje de nuevo. La unidad toma los ajustes del propio `@transactional` del oyente: un oyente declarado `@transactional(isolation=Isolation.SERIALIZABLE)` se ejecuta en `SERIALIZABLE`, uno con `read_only=True` no puede escribir, y uno declarado `REQUIRES_NEW` o `NESTED` se ejecuta en la unidad que empieza su propio decorador.

Un servicio `@transactional` al que llama el oyente también se une a la unidad de la entrega. Cuando ese servicio falla, la unidad queda marcada solo-rollback aunque el oyente capture la excepción: la entrega se revierte con `UnexpectedRollbackError` y se vuelve a intentar, y después va a la cola de mensajes muertos. Para un paso de mejor esfuerzo, como un registro de auditoría, declara el servicio `@transactional(propagation=Propagation.NESTED)`: su fallo revierte hasta su propio punto de guardado, y la entrega hace commit. Dos ajustes lo completan. Dimensiona

`pyfly.messaging.listener.retry.max-attempts` y `retry.max-delay` para que duren más que una conmutación por error de la base de datos: con los valores por defecto, una caída de más de unos 15 segundos manda a mensajes muertos lo que se consume durante ella. Y fija `pyfly.messaging.listener.transactional: false` para un oyente de alto rendimiento que nunca toca la base de datos, de modo que no se abra una unidad por mensaje.

Resiliencia: cortacircuitos y reintentos

Proteger a Lumen de una caída del broker

Un broker sano no está garantizado. Las particiones de red, las actualizaciones progresivas (rolling upgrades) y el agotamiento de recursos pueden hacer que el broker quede temporalmente no disponible. Si el manejador de comandos llama a `broker.publish(...)` y el broker está caído, te enfrentas a dos malas opciones sin una capa de resiliencia: fallar el comando entero (negarte a depositar fondos porque el broker es inalcanzable) o descartar silenciosamente el evento (el depósito tiene éxito pero el evento de integración se pierde).

Ninguna es aceptable. El outbox transaccional es la solución atómica: con `pyfly.eda.outbox.enabled: true` (Capítulo 8) cada evento se captura en la unidad de trabajo de quien llama y un reenviador lo publica tras el commit, con sus propios reintentos y mensajes muertos (`pyfly.eda.outbox.forward.*`), de modo que una caída del broker solo añade latencia, no pérdida de datos. Junto al outbox, los **cortacircuitos** (circuit breakers) y los **reintentos** protegen cualquier código que llame al broker y que escribas tú mismo frente a los fallos en cascada.

Un **cortacircuitos** es la metáfora eléctrica convertida en código: tras demasiados fallos seguidos "salta" y deja de dejar pasar llamadas durante un periodo de enfriamiento, de modo que un broker con dificultades no se vea machacado por miles de intentos de reconexión condenados al fracaso. Un **reintento** es la táctica complementaria: vuelve a probar la misma llamada unas cuantas veces, porque muchos fallos son momentáneos.

El módulo de resiliencia de PyFly (`pyfly.resilience`) proporciona ambas primitivas. El cortacircuitos se abre tras un umbral de fallos configurable y bloquea las llamadas al broker durante un periodo de enfriamiento, evitando una tormenta de reconexiones de manada (thundering herd). El decorador de reintentos gestiona los errores transitorios con un retroceso configurable. Los aplicas como un par de decoradores sobre el método de publicación:

Paso 1 — Escribe el método de publicación simple. `forward` hace una sola cosa: codifica el registro y llama a `self._broker.publish(...)`. No vive ninguna lógica de resiliencia en el cuerpo.

Paso 2 — Envuélvelo en `@circuit_breaker`. Pasa una instancia compartida de `CircuitBreaker` para que el conteo de fallos se acumule entre llamadas, no por llamada. Cuando el broker está caído, el cortacircuitos salta y falla rápido.

Paso 3 — Envuelve eso en `@retry` por fuera. El orden de los decoradores importa: `@retry` se sitúa por encima de `@circuit_breaker`, de modo que todos los intentos de reintento de una llamada ocurren antes de que el cortacircuitos registre un solo fallo. Mantén `max_attempts` bajo y deja que el cortacircuitos absorba las caídas sostenidas.

lumen/messaging/resilient_publisher.py

```

from __future__ import annotations

import json
import logging

from pyfly.container import service
from pyfly.messaging import MessageBrokerPort
from pyfly.resilience import CircuitBreaker, circuit_breaker, retry

logger = logging.getLogger(__name__)

@service
class ResilientForwarder:
    """
    Forwards records to the broker from code you write yourself.
    Applies retry and circuit-breaker protection on every publish call.
    """

    def __init__(self, broker: MessageBrokerPort) → None:
        self._broker = broker

    @retry(max_attempts=3, delay=1.0, backoff=2.0)
    @circuit_breaker(CircuitBreaker(failure_threshold=5, recovery_timeout=30))
    async def forward(
        self,
        topic: str,
        payload: dict,
        event_type: str,
    ) → None:
        """Forward a single outbox record to the broker."""
        await self._broker.publish(
            topic,
            json.dumps(payload).encode(),
            headers={"event-type": event_type},
        )
        logger.info(

```

```

        "Event forwarded: topic=%s event-type=%s",
        topic,
        event_type,
    )

```

Listado 10.9 — Publicación resiliente al broker con reintento y cortacircuitos

Los dos decoradores:

`@retry(max_attempts=3, delay=1.0, backoff=2.0)` envuelve `forward` en un bucle de reintentos de hasta tres intentos. Tras el primer fallo espera `delay` segundos (1 s); tras el segundo espera `delay * backoff` (2 s): la espera crece como `delay * backoff ** attempt`. Si el tercer intento sigue fallando, la excepción se propaga. Los reintentos sirven para fallos *transitorios* (un único nodo del broker reiniciándose); son contraproducentes para fallos *permanentes* (un topic mal configurado). Mantén `max_attempts` bajo y deja que el cortacircuitos gestione las caídas sostenidas.

`@circuit_breaker(CircuitBreaker(failure_threshold=5, recovery_timeout=30))` protege `forward` con una instancia compartida de `CircuitBreaker` que rastrea los fallos consecutivos a través de todas las llamadas. Cuando el conteo alcanza `failure_threshold`, el circuito se *abre* y las llamadas posteriores fallan inmediatamente con `CircuitBreakerException` en lugar de intentar alcanzar un broker inalcanzable, evitando una tormenta de reconexiones. Tras `recovery_timeout` segundos el circuito entra en un estado *semiabierto*: la siguiente llamada se deja pasar como sonda. Si tiene éxito, el circuito se cierra; si falla, se vuelve a abrir. El orden de los decoradores importa: `@retry` es el decorador externo, de modo que los tres intentos de una llamada lógica ocurren antes de que el cortacircuitos registre un solo fallo.

🍃 EQUIVALENCIA CON SPRING

`MessageBrokerPort` con `KafkaAdapter` es el equivalente en PyFly del `KafkaTemplate` (publicación) y el `@KafkaListener` (consumo) de Spring Kafka. `RabbitMQAdapter` refleja el `RabbitTemplate` y el `@RabbitListener` de Spring AMQP. El modelo de ciclo de vida es el mismo: registra los oyentes antes de arrancar el contenedor, y el framework gestiona los hilos de consumidor. Las colas de mensajes muertos en Spring Kafka se configuran mediante `DeadLetterPublishingRecoverer` sobre el `DefaultErrorHandler`; en Spring AMQP mediante `RabbitListenerContainerFactory` con un `MessageRecoverer`. PyFly implementa el mismo patrón de forma declarativa a través de `@message_listener(retries=..., dead_letter_topic=...)` en lugar de requerir configuración de contenedor específica del broker. Los decoradores `@retry` y `@circuit_breaker` reflejan las anotaciones `@Retryable` y `@CircuitBreaker` de Resilience4j usadas con la infraestructura de mensajería de Spring.

✓ Lo que construiste

La Parte III está completa.

Lumen es ahora plenamente orientada a eventos, con event sourcing y conectada a un broker. Aquí está dónde dejó las cosas cada capítulo.

El Capítulo 8 introdujo el modelo de dos buses: `ApplicationEventBus` para los eventos del ciclo de vida del framework, `InMemoryEventBus` para los eventos de dominio. `EventPublisher` se conectó a los manejadores de comandos para que cada mutación de un agregado produjera un hecho al que oyentes independientes — `WalletAuditListener` entre ellos — pudieran reaccionar sin conocerse entre sí. Las suscripciones usan `@event_listener(event_types=["WalletOpened", "FundsDeposited", "FundsWithdrawn"])`; los manejadores reciben un `EventEnvelope` cuyo `event_type` es el nombre de la clase del evento de dominio.

El Capítulo 9 reemplazó el enfoque de agregado mutable más modelo de lectura por el event sourcing. Cada movimiento financiero es un evento inmutable que se añade al libro mayor. El saldo actual se calcula reproduciendo el flujo de eventos.

`EventEnvelope` se convirtió en la unidad de almacenamiento, y los snapshots mantuvieron acotados los tiempos de reproducción.

Este capítulo cruzó el límite de la red. `MessageBrokerPort` es la única abstracción frente a Kafka, RabbitMQ o el broker en memoria. Intercambiar adaptadores es un cambio de configuración: ningún cambio en el código de negocio. `@message_listener` ofrece suscripciones declarativas y sin código repetitivo tanto en funciones independientes como en métodos de `@service`. Los parámetros `retries` y `dead_letter_topic` gestionan los mensajes envenenados sin andamiaje manual de try/except. Las cargas útiles se codificaron como bytes JSON, con Avro y Protobuf disponibles cuando el cumplimiento de esquema o la eficiencia binaria importan más que la simplicidad. `@retry` y `@circuit_breaker` protegen la ruta de publicación frente a fallos del broker transitorios y sostenidos.

Los eventos de dominio que fluyen a través de los tres capítulos son:

Clase de evento	Campos
<code>WalletOpened</code>	<code>wallet_id</code> , <code>owner_id</code> , <code>currency</code>
<code>FundsDeposited</code>	<code>wallet_id</code> , <code>amount</code> , <code>currency</code> , <code>balance</code>
<code>FundsWithdrawn</code>	<code>wallet_id</code> , <code>amount</code> , <code>currency</code> , <code>balance</code>

`amount` y `balance` son siempre unidades menores enteras (p. ej., `5000` para 50,00 €). `currency` es un valor de cadena del StrEnum `Currency` (`"EUR"`, `"USD"`, `"GBP"`). El valor de la cabecera `event_type` es siempre el nombre de la clase — `"FundsDeposited"` —, nunca una ruta con puntos.

Tres principios pasan a la Parte IV:

- **Depende del port, no del adaptador.** `MessageBrokerPort` se inyecta; `KafkaAdapter` es un detalle de configuración.
- **Diseña los consumidores para que sean idempotentes.** Los brokers entregan *al menos una vez*. Protégete frente al procesamiento duplicado con un identificador de mensaje estable.
- **Captura los eventos de forma atómica.** El outbox transaccional garantiza que un evento nunca se pierda, incluso cuando el broker no está disponible en el momento de la escritura.

La Parte IV introduce el `PaymentsService` y el `NotificationsService`. Ambos se suscriben a `wallet.events`. Las elecciones de adaptador y configuración hechas en este capítulo son todo lo que necesitan para empezar a recibir los hechos de Lumen en cuanto se conecten.

Pruébalo tú mismo

EJECÚTALO

Cada ejercicio de abajo termina en una prueba. Como `provider: "memory"` no necesita broker, puedes ejecutarlos sin nada instalado más allá del extra de desarrollo. Desde la raíz del proyecto Lumen:

```
uv run --extra dev pytest tests/test_messaging.py -q
```

Una ejecución en verde tiene este aspecto:

```
...
[100%]
3 passed in 0.XXs
```

Si una prueba falla con un error de atributo ausente como

`__pyfly_message_listener__`, el decorador `@message_listener` no está aplicado a tu manejador: vuelve a revisar el Paso 2 de "Oyentes declarativos".

- 1. Intercambia el adaptador en una línea.** Empieza con `provider: "memory"` en `pyfly.yaml` y añade el `@message_listener` del Listado 10.4. Escribe una prueba de integración que publique un mensaje `FundsDeposited` con `amount=5000` y `currency="EUR"` y afirme que el oyente lo recibe. Luego cambia a `provider: "kafka"` en el YAML y confirma que la misma prueba (con un broker de Kafka gestionado por Testcontainers) pasa sin cambiar el oyente ni la aserción de la prueba.
- 2. Añade un monitor de DLQ.** Crea un segundo `@message_listener` en el topic `wallet.events.DLQ` con el grupo `dlq-monitor`. Debería registrar las cabeceras `x-original-topic` y `x-exception` junto con la carga útil decodificada. Escribe

una prueba que simule un consumidor fallido lanzando `RuntimeError` dentro del manejador, configure `retries=2` y `dead_letter_topic="wallet.events.DLQ"`, y confirme que el monitor de DLQ recibe el mensaje con `x-original-topic: "wallet.events"`.

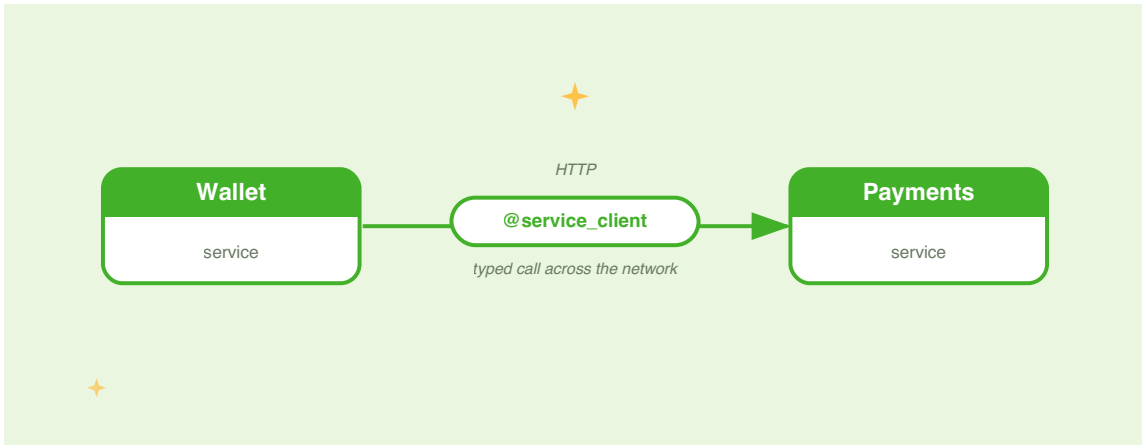
- 3. Evoluciona el esquema con Avro.** Empieza con el `WALLET_DEPOSITED_SCHEMA` del Listado 10.6. Añade un campo opcional `note` con un valor por defecto de `None` (unión Avro `["null", "string"]`, por defecto `null`). Confirma que un consumidor compilado contra el esquema original aún puede decodificar un mensaje codificado con el nuevo esquema: este es un cambio *retrocompatible* (backward-compatible). Luego intenta añadir un campo obligatorio sin un valor por defecto y observa la `SchemaParseException` que el registro lanzaría, ilustrando por qué los valores por defecto son obligatorios para una evolución segura.

PARTE IV

Hacia los Microservicios

CAPÍTULO 11

Dividir el monolito: clientes HTTP y el BFF



Cuando Lumen era un único servicio, cada capacidad vivía en el mismo proceso. El monedero, la comprobación de saldo y el procesamiento de pagos se ejecutaban juntos: sencillos de probar, simples de desplegar y perfectamente adecuados hasta que el equipo necesitó publicar funcionalidades a ritmos distintos. Entonces llegó la conversación difícil sobre la división.

La promesa de una división en microservicios es real: los equipos son dueños de sus servicios de forma independiente, los escalan por separado y los despliegan sin coordinar una ventana de publicación compartida. Pero toda división introduce un problema que el monolito nunca tuvo: la red. Lo que era una llamada a una función local se convierte en una petición HTTP que puede agotar su tiempo de espera, fallar a medias o aterrizar en un servicio saturado. Esa frontera de red no es un detalle de despliegue; es una preocupación de ingeniería de primer orden.

Este capítulo presenta `PaymentsService`, un segundo servicio al que el servicio Wallet de Lumen llama para liquidar transferencias. En lugar de montar a mano sesiones de `httpx` e ir hilvanando la lógica del cortacircuitos (circuit breaker) por cada manejador, definirás el cliente de Payments como una clase de Python corriente: una interfaz tipada y declarativa que PyFly rellena en el arranque. Al final del capítulo verás también cómo un nivel **BFF (Backend for Frontend)** se sitúa por delante de ambos servicios y compone sus capacidades en una única API centrada en el recorrido del usuario.

NUEVA JERGA, EN LENGUAJE LLANO

Algunos términos se repiten a lo largo de este capítulo. **Llamada servicio a servicio** significa que uno de tus servicios hace una petición HTTP a otro de tus servicios (Wallet llamando a Payments), en contraste con una petición que llega desde un navegador. Un **cliente declarativo** es un cliente que *describes* en vez de *implementar*: escribes las firmas de los métodos y dejas que el framework rellene la fontanería HTTP. Un **cortacircuitos (circuit breaker)** es un interruptor de seguridad que deja de llamar a un servicio remoto después de que haya fallado demasiadas veces seguidas. Un **BFF** (Backend for Frontend) es un servicio fino cuya única misión es llamar a otros servicios y remodelar sus respuestas para una aplicación concreta. Construiremos cada una de estas piezas, una a una.

Este capítulo se articula en torno a `httpx` y al paquete `pyfly.client`, ambos incluidos con el framework. Todo lo que hay aquí se ejecuta contra PyFly v26.6.110. No necesitas instalar nada nuevo: si has ido siguiendo el hilo con Lumen, las herramientas de cliente ya están en tu path.

Por qué dividir (y por qué duele)

La zona de confort del monolito

Un monolito no es un error arquitectónico: es un punto de partida arquitectónico.

Lumen comenzó como un único servicio porque un único servicio era lo correcto: un equipo, una canalización de despliegue, un conjunto de preocupaciones que razonar.

La transacción de base de datos que escribe una fila del monedero y publica un evento de dominio en la misma unidad de trabajo no era un compromiso a la baja; era la elección óptima.

La presión para dividir suele llegar desde fuera de la arquitectura. Payments necesita un registro de auditoría de cumplimiento separado. La puntuación de riesgo necesita un equipo especialista con acceso a una fuente de datos privada. El procesamiento de liquidaciones exige un rendimiento un orden de magnitud mayor que las lecturas de saldo. Cualquiera de estas razones es buena para extraer un servicio, y ninguna de ellas borra el hecho de que el resto del sistema todavía necesita llamar al servicio extraído a través de una frontera de red.

El coste de la red

Las llamadas de red fallan de maneras en que las llamadas a funciones locales no lo hacen. Un método sobre un objeto local o bien devuelve un valor o bien lanza una excepción. Una llamada HTTP a un servicio remoto puede agotar su tiempo de espera (el remoto va lento), rechazar la conexión (el remoto está caído), devolver un 503 transitorio (el remoto está sobrecargado) o tener éxito solo al tercer intento. En un monolito estos modos de fallo son irrelevantes; en un sistema distribuido son tu línea de base.

El arreglo ingenuo —usar `httpx` directamente con `try/except` alrededor de cada llamada— funciona para un único punto de llamada, pero no escala. Acabas con la lógica del cortacircuitos duplicada en cada cliente de servicio, retardos de reintento incrustados a fuego en los manejadores y valores de tiempo de espera dispersos por fragmentos de `pyfly.yaml` que nadie posee. Cuando Payments introduce un nuevo endpoint, cada quien que llame debe acordarse de añadir de nuevo todo el andamiaje de resiliencia.

El cliente HTTP tipado de PyFly elimina esa duplicación. Declaras cómo es el servicio remoto: sus endpoints, sus rutas y la forma de sus parámetros. PyFly genera la implementación en el arranque, conecta un cortacircuitos y una política de reintentos a partir de `pyfly.yaml` y registra el bean en el contenedor para que cualquier manejador que lo necesite pueda declararlo como argumento del constructor. La resiliencia se aplica una vez, de forma coherente, en la capa correcta.

Un cliente de servicio tipado

Declarativo en vez de imperativo

La idea central del módulo de cliente de PyFly es que los contratos servicio a servicio se expresan mejor como tipos que como lógica HTTP procedimental. Cuando describes `PaymentsClient` como una clase con firmas de método tipadas, obtienes una interfaz de Python que cualquier IDE puede navegar, que cualquier verificador de tipos puede comprobar y que cualquier prueba puede simular, sin importar nunca `httpx` en el código que la usa.

Dos decoradores definen ese contrato:

Decorador	Resiliencia incorporada	Úsalo para
<code>@service_client</code>	Cortacircuitos + reintentos	Llamadas servicio a servicio en producción
<code>@http_client</code>	Ninguna	Clientes ligeros, pruebas, utilidades internas

Usa `@service_client` siempre que el destino sea otro microservicio. Reserva `@http_client` para utilidades internas y dobles de prueba.

Definir el cliente de Payments

El servicio Payments expone dos endpoints: uno para crear una instrucción de pago y otro para recuperar un pago por identificador. Definir el cliente consiste en escribir la clase. Construyámosla decisión a decisión.

Paso 1 — Crea el archivo. Añade `src/lumen/sdk/payments_client.py` junto al `client.py` existente. El paquete `sdk` es donde Lumen guarda el código que *habla con* servicios, así que este es el hogar natural para un cliente de servicio.

Paso 2 — Decora la clase. Coloca `@service_client(base_url=...)` sobre una clase normal. Ese único decorador es lo que convierte una clase corriente en un cliente HTTP gestionado por PyFly: registra la URL base, activa las funciones de resiliencia y registra la clase como un bean para que el contenedor pueda inyectarla después.

Paso 3 — Declara un método por endpoint. Cada método recibe un decorador de verbo (`@post` , `@get` , `@patch` , `@delete`) que lleva la ruta. El cuerpo del método es solo `...` —una *elipsis*, el literal de Python para "aquí todavía no hay nada"—. Nunca escribes el código de la petición; PyFly lo escribe por ti en el arranque.

El cliente completo tiene este aspecto.

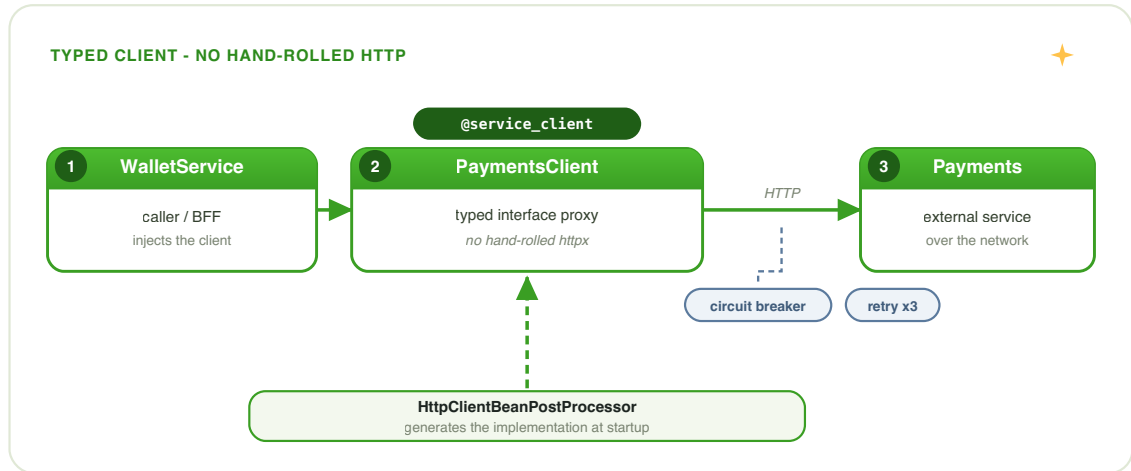


Figura 11.1 — La canalización del cliente declarativo de PyFly. Tú escribes la interfaz; `HttpClientBeanPostProcessor` genera la implementación.

lumen/sdk/payments_client.py

```

from __future__ import annotations

from pyfly.client import (
    delete,
    get,
    patch,
    post,
    service_client,
)

@service_client(
    base_url="http://payments-service:8080",
    circuit_breaker=True,
    retry=3,
    circuit_breaker_failure_threshold=5,
    circuit_breaker_recovery_timeout=60.0,
    retry_base_delay=1.0,
)
class PaymentsClient:
    """Typed HTTP client for the Payments service.

    Method stubs are replaced with real HTTP implementations by
    HttpClientBeanPostProcessor at application startup. Declare
    this class as a constructor argument to have it injected.
    """

    @post("/payments")
    async def create_payment(self, body: dict) → dict:
        """POST /payments – submit a payment instruction."""
        ...

    @get("/payments/{payment_id}")
    async def get_payment(self, payment_id: str) → dict:
        """GET /payments/:payment_id – fetch a payment by ID."""
        ...

```

```

@patch("/payments/{payment_id}/cancel")
async def cancel_payment(self, payment_id: str) → dict:
    """PATCH /payments/:payment_id/cancel – cancel pending."""
    ...

@delete("/payments/{payment_id}")
async def delete_payment(self, payment_id: str) → None:
    """DELETE /payments/:payment_id – remove a completed record."""
    ...

```

Listado 11.1 — Cliente de Payments tipado con `@service_client`

Cómo funciona — la canalización de declaración:

`@service_client(base_url=...)` estampa atributos de metadatos sobre la clase y la registra como un bean singleton en el contenedor de PyFly: el mismo mecanismo

`__pyfly_injectable__ = True` que usa `@service`. La `base_url` se almacena como `__pyfly_http_base_url__`; las opciones de resiliencia aterrizan en `__pyfly_resilience__`.

Los decoradores de verbo — `@post("/payments")`, `@get("/payments/{payment_id}")` y los demás— adjuntan cada uno dos atributos a su método: `__pyfly_http_method__` (la cadena con el verbo HTTP) y `__pyfly_http_path__` (la plantilla de ruta). El cuerpo del método en sí se convierte en un stub que lanza `NotImplementedError` y nunca debería llamarse directamente.

En el arranque, `HttpClientBeanPostProcessor.after_init()` inspecciona cada bean. Cuando encuentra una clase con `__pyfly_http_client__ = True`, crea un `HttpxClientAdapter` para la `base_url`, recorre cada método buscando `__pyfly_http_method__` y reemplaza cada stub por una implementación asíncrona real. Esa implementación usa `inspect.signature()` para enlazar los argumentos de quien llama, interpola las variables de ruta (`{payment_id}` → el valor real), separa los

parámetros restantes en cadenas de consulta o un cuerpo JSON, y llama a `client.request()`. Las respuestas con estado ≥ 400 lanzan excepciones tipadas; las respuestas correctas devuelven `response.json()`.

La interpolación de variables de ruta es posicional: cualquier parámetro cuyo nombre coincida con un `{marcador}` de la plantilla de ruta es sustituido. Para `get_payment(self, payment_id: str)`, llamar a `client.get_payment("pay-123")` envía `GET /payments/pay-123`. Para `create_payment(self, body: dict)`, llamar a `client.create_payment({"amount": 5000})` envía `POST /payments` con el dict serializado como cuerpo JSON. Los parámetros llamados `body` en métodos POST/PUT/PATCH siempre se tratan como el cuerpo JSON de la petición; todos los demás parámetros que no son de ruta en métodos GET/DELETE se convierten en parámetros de cadena de consulta.

QUÉ ACABA DE PASAR

Escribiste cuatro *firmas* de método y cero líneas de código HTTP. El decorador `@service_client` etiquetó la clase para el contenedor; los decoradores de verbo etiquetaron cada método con un verbo y una ruta. En el arranque, un componente entre bambalinas llamado `HttpClientBeanPostProcessor` lee esas etiquetas y reemplaza silenciosamente cada stub `...` por un método asíncrono funcional que construye la URL, envía la petición y vuelve a convertir la respuesta en Python. Desde el punto de vista de quien llama, `await client.get_payment("pay-123")` se ve exactamente igual que llamar a un método local: la red queda oculta.

Ejecútalo — confirma que los stubs están conectados. Hasta que el contexto de aplicación arranca, esos cuerpos `...` lanzan `NotImplementedError` a propósito, así que no puedes simplemente llamar al método en un script pelado. La forma honesta de demostrar que el cliente funciona es una prueba diminuta que arranque un contexto (o conecte el post-procesador) e inspeccione el método generado. La comprobación rápida más sencilla es confirmar los metadatos que los decoradores estamparon:

```
uv run python -c "from lumen.sdk.payments_client import PaymentsClient; \
print(PaymentsClient.__pyfly_http_base_url__); \
print(PaymentsClient.create_payment.__pyfly_http_method__, \
PaymentsClient.create_payment.__pyfly_http_path__)"
```

Salida esperada:

```
http://payments-service:8080
POST /payments
```

Si ves la URL base y `POST /payments`, los decoradores se aplicaron correctamente y el post-procesador tiene todo lo que necesita para generar la implementación real cuando la aplicación arranque.

🍃 EQUIVALENCIA CON SPRING

`@service_client` con `@get` / `@post` / `@put` / `@delete` / `@patch` es el homólogo en PyFly de `@FeignClient` de Spring Cloud OpenFeign con `@GetMapping` / `@PostMapping`, etc. En Feign anotas una interfaz; en PyFly anotas una clase con métodos stub: la intención es idéntica. Ambos frameworks generan la implementación HTTP en tiempo de arranque, inyectan el bean a través del contenedor de inyección de dependencias y admiten cortacircuitos (Feign vía Resilience4j; PyFly vía el `CircuitBreaker` incorporado). La diferencia clave es que Feign trabaja sobre interfaces de Java mientras que PyFly trabaja sobre clases de Python corrientes, lo que significa que puedes añadir métodos auxiliares junto a los métodos stub: útil para la lógica de remodelado de respuestas que pertenece dentro de la propia clase cliente.

Inyectar el cliente en un manejador

Como `PaymentsClient` es un bean singleton, cualquier `@service` o `@command_handler` puede declararlo como argumento del constructor. El contenedor de PyFly lo inyecta a través de la misma vía de autoconexión usada para repositorios y servicios de dominio.

i BEAN Y AUTOCONEXIÓN, EN BREVE

Un **bean** es simplemente un objeto que el framework crea y gestiona por ti: nunca llamas tú a su constructor. La **autoconexión** es cómo el contenedor entrega un bean a quien lo necesite: cuando una clase enumera `payments: PaymentsClient` en su `__init__`, PyFly se percata del tipo, encuentra el bean coincidente y lo pasa automáticamente. Declaras la dependencia por *tipo*; el contenedor hace la búsqueda.

El servicio Wallet de Lumen ya aplica el patrón del `WalletRepository` y el objeto de valor `Money` de capítulos anteriores. Cuando el monedero debe llamar a Payments para liquidar un reintegro, el manejador sigue ese mismo patrón en tres pasos: cargar el monedero, retirar a través de la raíz de agregado y luego llamar al servicio externo.

Paso 1 — Declara la dependencia. Añade `payments: PaymentsClient` al constructor del manejador y guárdalo en `self`. Esa es la única conexión que escribes; el contenedor suministra el cliente en vivo.

Paso 2 — Haz primero el trabajo local. Carga el monedero y llama a `wallet.withdraw(...)`, después persístelo, de modo que el propio estado del monedero quede liquidado antes de que ocurra cualquier llamada de red.

Paso 3 — Llama al servicio remoto. `await self._payments.create_payment(...)` se lee como una llamada a un método local. Los detalles de resiliencia y HTTP ya vienen horneados en el cliente inyectado.

Aquí está el manejador.

lumen/core/services/wallets/settle_transfer_handler.py

```

from __future__ import annotations

from lumen.core.mappers.wallet_mapper import to_aggregate, to_entity
from lumen.core.services.wallets.settle_transfer_command import (
    SettleTransfer,
)
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository
from lumen.sdk.payments_client import PaymentsClient
from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.domain import AggregateNotFound

@command_handler
@service
class SettleTransferHandler(CommandHandler[SettleTransfer, dict]):
    """Withdraw from the wallet and submit a payment instruction."""

    def __init__(
        self,
        repository: WalletRepository,
        payments: PaymentsClient,
    ) → None:
        super().__init__()
        self._repository = repository
        self._payments = payments

    async def do_handle(self, command: SettleTransfer) → dict:
        entity = await self._repository.find_by_id(command.wallet_id)
        if entity is None:
            raise AggregateNotFound("Wallet", command.wallet_id)

        wallet = to_aggregate(entity)
        wallet.withdraw(
            Money(amount=command.amount, currency=wallet.currency)
        )

```



```

await self._repository.upsert(to_entity(wallet))

payment = await self._payments.create_payment({
    "wallet_id": command.wallet_id,
    "amount": command.amount,
    "currency": wallet.currency.value,
    "reference": command.reference,
})
return payment

```

Listado 11.2 — *CommandHandler* que inyecta *PaymentsClient*

Cómo funciona — la vía de inyección:

`payments: PaymentsClient` en el constructor es resuelto por el contenedor en el arranque. `HttpClientBeanPostProcessor` conecta `PaymentsClient` antes de que se instancie `SettleTransferHandler`, de modo que el bean inyectado está plenamente operativo. El manejador llama a `await self._payments.create_payment(...)` exactamente como si fuera un método asíncrono local. La agrupación de conexiones (connection pooling), la propagación de cabeceras y el mapeo de errores son todos invisibles para el manejador.

`wallet.withdraw(Money(...))` se ejecuta antes de la llamada de red, y el manejador no lleva `@transactional`, así que el `upsert` confirma en una unidad de trabajo propia antes de que se contacte a Payments. (La carga y el guardado son entonces dos unidades: el Capítulo 5 muestra el bloqueo, y el Capítulo 12 la actualización con guarda, que impiden que dos liquidaciones del mismo monedero gasten los mismos fondos.) Si Payments está temporalmente no disponible, los reintentos y el cortacircuitos —descritos en la siguiente sección— gestionan la recuperación de forma transparente, sin ningún código en el manejador.

Ejecútalo — ejercita el manejador con un cliente falso. Como el manejador depende del *tipo* `PaymentsClient`, puedes sustituirlo por un suplente en una prueba sin ninguna red. Pon esto en `tests/test_settle_transfer.py` y ejecútalo:

```
uv run --extra dev pytest tests/test_settle_transfer.py -q
```

Una ejecución exitosa imprime algo como:

```
1 passed in 0.12s
```

La gracia de la prueba es la sustitución: pasas un objeto hecho a mano en lugar del `PaymentsClient` real, afirmas que el manejador llamó a `create_payment` con el cuerpo esperado y nunca abres un socket. Esa es la recompensa práctica de declarar la dependencia por tipo: al manejador ni le consta ni le importa si el cliente del otro lado es real o falseado.

Resiliencia sobre el cable

Por qué la capa de cliente es el lugar correcto para la resiliencia

La lógica de resiliencia dentro de un manejador mezcla preocupaciones de negocio con fontanería de infraestructura. Un manejador que captura `httpx.ConnectError` e implementa su propio bucle de espera está haciendo dos cosas a la vez: liquidar una transferencia y gestionar los modos de fallo de HTTP. Esas responsabilidades pertenecen a capas separadas.

`@service_client` mueve el **cortacircuitos** y la **política de reintentos** a la capa de cliente, donde corresponde. Los configuras una vez en el decorador y cada método del cliente los hereda de manera uniforme. El código del manejador permanece centrado en la operación de negocio.

Cortacircuitos

Un cortacircuitos monitoriza cada llamada al servicio remoto. Cuando

`failure_threshold` llamadas consecutivas fallan, el circuito se **abre**: las llamadas posteriores se rechazan de inmediato con `CircuitBreakerException` en lugar de esperar a un agotamiento de tiempo de espera. Esto evita que un único servicio lento o no disponible bloquee el bucle de eventos y agote las agrupaciones de conexiones de todos los llamantes.

Tras `circuit_breaker_recovery_timeout` segundos, el circuito entra en **medio abierto**: se admite una petición de sondeo. Si tiene éxito, el circuito se cierra y se reanuda la operación normal. Si falla, el circuito vuelve a abrirse y se reinicia el temporizador de recuperación.

`@service_client` conecta el cortacircuitos automáticamente. Si lo necesitas por separado:

lumen/sdk/standalone_breaker.py

```

from __future__ import annotations

from datetime import timedelta

from pyfly.client import CircuitBreaker
from pyfly.kernel.exceptions import CircuitBreakerException

breaker = CircuitBreaker(
    failure_threshold=3,
    recovery_timeout=timedelta(seconds=30),
)

async def call_with_breaker(client, payment_id: str) → dict:
    """Fetch a payment through a standalone circuit breaker."""
    try:
        return await breaker.call(
            client.get_payment,
            payment_id,
        )
    except CircuitBreakerException:
        return {"status": "unavailable", "payment_id": payment_id}

```

Listado 11.3 — Uso de CircuitBreaker por separado

Cómo funciona: `CircuitBreaker.__init__` acepta `failure_threshold` (por defecto 5) y `recovery_timeout` como un `timedelta` (por defecto 30 s). `breaker.call(func, *args)` ejecuta `func(*args)` dentro del cortacircuitos: si tiene éxito, reinicia la cuenta de fallos; si falla, incrementa la cuenta y voltea el estado a `OPEN` una vez que se alcanza el umbral. Las transiciones de estado `CLOSED → HALF_OPEN` se calculan de forma perezosa con `time.monotonic()`: no hay temporizador en segundo plano.

`CircuitBreakerException` nunca se contabiliza como un fallo. Señala que el circuito ya está abierto, así que volver a lanzarla sin registrar otro fallo impide que el tiempo de espera de recuperación se reinicie indefinidamente.

ⓘ TRES ESTADOS, EN LENGUAJE LLANO

Piensa en el cortacircuitos como un interruptor de luz con tres posiciones. **Cerrado** es lo normal: las llamadas fluyen a través. **Abierto** significa "deja de intentarlo": las llamadas fallan al instante sin tocar la red, lo que libra a tu servicio de esperar por algo que claramente está caído. **Medio abierto** es "déjame tantear el terreno": tras el tiempo de espera de recuperación, el cortacircuitos deja pasar exactamente una llamada; si funciona, el interruptor vuelve a cerrado, y si falla, salta de nuevo a abierto.

Ejecútalo — observa cómo un cortacircuitos se abre y se recupera. Puedes accionar el cortacircuitos a mano desde un REPL sin ningún servicio real involucrado. Arranca uno con `uv run python` desde `samples/lumen` y prueba:

```

uv run python -c "
import asyncio
from datetime import timedelta
from pyfly.client import CircuitBreaker, CircuitState
from pyfly.kernel.exceptions import CircuitBreakerException

async def boom():
    raise RuntimeError('payments down')

async def main():
    cb = CircuitBreaker(failure_threshold=2,
recovery_timeout=timedelta(seconds=30))
    for _ in range(2):
        try:
            await cb.call(boom)
        except RuntimeError:
            pass
    print('state after 2 failures:', cb.state.name)
    try:
        await cb.call(boom)
    except CircuitBreakerException:
        print('open circuit rejected the call without calling boom')

asyncio.run(main())
"

```

Salida esperada:

```

state after 2 failures: OPEN
open circuit rejected the call without calling boom

```

La segunda línea es justo el quid: una vez que el circuito está abierto, el cortacircuitos lanza `CircuitBreakerException` *de inmediato* en lugar de ejecutar `boom` de nuevo. Baja `recovery_timeout` a `timedelta(seconds=0)` y vuelve a ejecutarlo: la siguiente

lectura de `cb.state` reporta `HALF_OPEN` y la tercera llamada admite un sondeo (así que `boom` se ejecuta otra vez) en vez de ser rechazada. Eso es el cortacircuitos dejando que el servicio demuestre que se ha recuperado.

Política de reintentos

Los fallos transitorios —un pico momentáneo de latencia, un reinicio progresivo, un breve reinicio de conexión— no necesitan un cortacircuitos; necesitan un segundo intento. `RetryPolicy` proporciona espera exponencial con filtrado de excepciones configurable:

`lumen/sdk/standalone_retry.py`

```
from __future__ import annotations

from datetime import timedelta

from pyfly.client import RetryPolicy

policy = RetryPolicy(
    max_attempts=3,
    base_delay=timedelta(milliseconds=500),
    retry_on=(ConnectionError, TimeoutError),
)

async def resilient_fetch(client, payment_id: str) → dict:
    """Fetch a payment with retry on transient network errors."""
    return await policy.execute(
        client.get_payment,
        payment_id,
    )
```

Listado 11.4 — Uso de `RetryPolicy` por separado

Cómo funciona: `RetryPolicy.__init__` acepta `max_attempts` (por defecto 3, contando el primer intento), `base_delay` (por defecto 1 s) y `retry_on`, una tupla de tipos de excepción. La fórmula de espera es `base_delay * (2 ** attempt)`: para `base_delay=0.5 s`, los retardos son 0,5 s, 1 s, 2 s. Solo las excepciones que coinciden con `retry_on` disparan un reintento; las demás se propagan de inmediato. Esto importa: no quieres reintentar un 404 (el recurso no existe) ni un 422 (la petición es semánticamente inválida).

❗ ESPERA (BACKOFF), EN LENGUAJE LLANO

Espera (backoff) significa aguardar un poco más antes de cada reintento en vez de aporrear el servicio remoto en el instante en que falla. La espera *exponencial* duplica la espera cada vez, así que un servicio que necesita un momento para recuperarse obtiene más oxígeno con cada intento, mientras que uno sano se reintenta casi de inmediato.

Ejecútalo — observa cómo el reintento se recupera de un error transitorio. Simula una llamada que falla dos veces y luego tiene éxito:


```

uv run python -c "
import asyncio
from datetime import timedelta
from pyfly.client import RetryPolicy

attempts = {'n': 0}
async def flaky():
    attempts['n'] += 1
    if attempts['n'] < 3:
        raise ConnectionError('reset')
    return 'ok'

async def main():
    policy = RetryPolicy(max_attempts=3, base_delay=timedelta(milliseconds=1),
                        retry_on=(ConnectionError,))
    print('result:', await policy.execute(flaky))
    print('attempts:', attempts['n'])

asyncio.run(main())
"

```

Salida esperada:

```

result: ok
attempts: 3

```

Tres intentos, un éxito. Cambia `retry_on` a `(TimeoutError,)` y el primer `ConnectionError` se propaga en su lugar: prueba de que solo se reintentan las excepciones que enumeras.

Cuando `@service_client` activa ambas funciones, el post-procesador las envuelve en el orden correcto: cortacircuitos *fuera*, reintento *dentro*. Una única llamada lógica intenta hasta `max_attempts` reintentos antes de que el cortacircuitos registre un fallo. Un circuito abierto rechaza la llamada de inmediato, saltándose por completo el bucle de reintentos.

Excepciones de error tipadas

Cuando el servicio remoto devuelve una respuesta 4xx o 5xx, el método generado lanza una excepción tipada en vez de devolver la carga útil de error como si fuera un éxito. La jerarquía de excepciones vive en `pyfly.client.exceptions`: importa de ahí las clases que quieras capturar (por ejemplo, `from pyfly.client.exceptions import ServiceNotFoundException`):

Estado	Clase de excepción	retryable
400	<code>ServiceValidationException</code>	False
401 / 403	<code>ServiceAuthenticationException</code>	False
404	<code>ServiceNotFoundException</code>	False
409	<code>ServiceConflictException</code>	False
422	<code>ServiceUnprocessableEntityException</code>	False
429	<code>ServiceRateLimitException</code>	True
5xx	<code>ServiceUnavailableException</code>	True

Todas las excepciones extienden `ServiceClientException` (que a su vez es una `InfrastructureException`). La marca `retryable` en `ServiceRateLimitException` y `ServiceUnavailableException` le indica al post-procesador qué excepciones pasar a la política de reintentos. Los errores de validación 4xx y los 404 nunca se reintentan.

❗ QUÉ ACABA DE PASAR

Las tres piezas de resiliencia encajan como cajas anidadas. Las **excepciones tipadas** clasifican *qué tipo* de fallo ocurrió: un 404 no es reintentable, un 503 sí. La **política de reintentos** usa esa clasificación para decidir si volver a intentarlo. El **cortacircuitos** se sitúa fuera del bucle de reintentos y cuenta los fallos sostenidos para poder dejar de llamar a un servicio que está genuinamente caído. No escribiste nada de este pegamento: `@service_client` lo ensambló en el momento en que pusiste `circuit-breaker=True` y `retry=3`.

Configurar valores por defecto en pyfly.yaml

Las anulaciones por servicio en `@service_client` siempre tienen prioridad. Establecer valores por defecto a nivel de proceso en `pyfly.yaml` permite que los nuevos clientes hereden valores sensatos sin repetirlos en cada decorador:

pyfly.yaml

```
pyfly:
  client:
    timeout: 10
    retry:
      max-attempts: 3
      base-delay: 1.0
    circuit-breaker:
      failure-threshold: 5
      recovery-timeout: 30
```

Listado 11.5 — Valores por defecto de resiliencia de cliente en pyfly.yaml

Clave	Descripción	Por defecto
<code>pyfly.client.timeout</code>	Tiempo de espera de la petición en segundos	30
<code>pyfly.client.retry.max-attempts</code>	Total de intentos, incluido el primero	3
<code>pyfly.client.retry.base-delay</code>	Retardo base en segundos	1.0
<code>pyfly.client.circuit-breaker.failure-threshold</code>	Fallos consecutivos para abrir	5
<code>pyfly.client.circuit-breaker.recovery-timeout</code>	Segundos antes de sondear	30

`ClientAutoConfiguration` lee `pyfly.client.timeout` en el arranque y se lo pasa a `HttpClientAdapter`. Los submapas `retry` y `circuit-breaker` se reenvían como `default_retry` y `default_circuit_breaker` a `HttpClientBeanPostProcessor`. Cualquier valor establecido directamente en `@service_client(circuit_breaker_failure_threshold=...)` anula el valor por defecto.

PRECEDENCIA, EN LENGUAJE LLANO

Dos capas pueden ajustar estos mandos: los valores por defecto globales de `pyfly.yaml` y los argumentos del decorador por cliente. El decorador siempre gana. Piensa en `pyfly.yaml` como el estilo de la casa que cada nuevo cliente hereda, y en el decorador como el lugar para anular ese estilo para un servicio anterior particularmente exigente.

Ejecútalo — confirma que se lee la configuración. Tras añadir el bloque a `pyfly.yaml`, vuelve a leer los valores a través del `Config` de PyFly para asegurarte de que las claves están bien escritas (una causa habitual de "mi tiempo de espera se está ignorando" es una errata). Ejecuta esto desde la raíz del proyecto, donde vive `pyfly.yaml`:

```
uv run python -c "
from pyfly.core.config import Config
cfg = Config.from_sources('.')
print('timeout:', cfg.get('pyfly.client.timeout'))
print('cb:', cfg.get('pyfly.client.circuit-breaker'))
"
```

Salida esperada una vez que el Listado 11.5 está en su sitio:

```
timeout: 10
cb: {'failure-threshold': 5, 'recovery-timeout': 30}
```

Antes de añadir el bloque, `timeout` reporta `30` —el valor por defecto del framework procedente de `pyfly-defaults.yaml`—, lo que confirma que la anulación es lo que lo cambió. Si una clave vuelve como `None`, comprueba la indentación en `pyfly.yaml`: el anidamiento de YAML es sensible a los espacios en blanco, y un `circuit-breaker:` mal alineado aterriza en silencio bajo el padre equivocado.

💡 ESTABLECE TIEMPOS DE ESPERA BAJOS POR SERVICIO

El valor por defecto `timeout: 30` es conservador. En producción, cada servicio debería llevar una anulación en `pyfly.yaml` ajustada a su SLA. Una llamada de pagos que debería completarse en 500 ms debería tener `timeout: 2` —no 30 s— para que una instancia lenta de Payments falle rápido y el cortacircuitos pueda abrirse antes de que los hilos se acumulen.

Autenticación, descubrimiento y deduplicación

Propagar la identidad aguas abajo

Cuando el servicio Wallet llama a Payments, a menudo necesita llevar la identidad de quien llama —un JWT o un token de servicio interno— para que Payments pueda imponer sus propias reglas de autorización. El parámetro `headers` recibe un trato especial por parte del post-procesador: cuando un método stub declara `headers: dict`, el valor se reenvía como cabeceras de la petición HTTP, no se serializa como una cadena de consulta.

JWT, EN LENGUAJE LLANO

Un **JWT** (JSON Web Token) es una cadena firmada que viaja en la cabecera `Authorization` y demuestra quién es quien llama. Cuando Wallet reenvía el JWT de quien llama a Payments, Payments puede volver a comprobarlo y aplicar sus propias reglas: la identidad se transporta a través de la frontera de red en lugar de restablecerse desde cero.

Para reenviar cabeceras, añades un único parámetro opcional. No hay ningún decorador nuevo ni configuración especial:

- **Añade `headers: dict | None = None` al método.** El nombre `headers` es la palabra mágica: el post-procesador lo reconoce y enruta su valor a las cabeceras HTTP en lugar de a la cadena de consulta.
- **Pasa un dict en el punto de llamada.** El manejador suministra `headers={"Authorization": f"Bearer {token}"}`, y PyFly lo adjunta a la petición saliente.

lumen/sdk/payments_client_auth.py

```

from __future__ import annotations

from pyfly.client import get, post, service_client

@service_client(
    base_url="http://payments-service:8080",
    circuit_breaker=True,
    retry=3,
)
class AuthenticatedPaymentsClient:
    """Payments client that forwards caller identity on each request."""

    @post("/payments")
    async def create_payment(
        self,
        body: dict,
        headers: dict | None = None,
    ) → dict:
        """POST /payments – body is the JSON payload; headers forwarded."""
        ...

    @get("/payments/{payment_id}")
    async def get_payment(
        self,
        payment_id: str,
        headers: dict | None = None,
    ) → dict:
        """GET /payments/:payment_id – headers are forwarded."""
        ...

```

Listado 11.6 — Reenvío de cabeceras de autenticación por llamada

Cómo funciona: El post-procesador comprueba si un parámetro llamado `headers` está presente en los argumentos enlazados y es un `dict`. Cuando se cumplen ambas condiciones, extrae el valor del conjunto de parámetros de consulta y lo reenvía como cabeceras de la petición HTTP. El manejador pasa la cabecera `Authorization` entrante (o un token de servicio recién acuñado) como `headers={"Authorization": f"Bearer {token}"}`.

`HttpClientAdapter` también llama a `inject_headers(headers)` en cada petición, propagando las cabeceras W3C `traceparent` y `tracestate` del contexto de observabilidad actual, de modo que las trazas distribuidas se cosen a través de las fronteras de servicio sin ningún trabajo a nivel de aplicación.

PATRONES DE IDENTIDAD SERVICIO A SERVICIO

Para servicios internos en una red de confianza, un secreto compartido en una cabecera `X-Internal-Token` es el enfoque más sencillo. Para arquitecturas de confianza cero (zero-trust), considera mTLS (TLS mutuo en la capa de infraestructura) o una malla de servicios (service mesh) que inyecte certificados de identidad. Para llamadas delegadas por el usuario, reenvía el JWT original. Sea cual sea el patrón que elijas, el parámetro `headers` te da un punto de inyección limpio en el cliente declarativo.

Descubrimiento de servicios

Cuando `base_url` es una cadena estática como `http://payments-service:8080`, dependes del descubrimiento basado en DNS: un `Service` de Kubernetes o un registro de Consul resuelve `payments-service` a la IP de clúster correcta. Este es el punto de partida recomendado y suficiente para la mayoría de los despliegues.

Para entornos que necesitan resolución dinámica de URL (múltiples entornos detrás de la misma clase cliente, enrutamiento controlado por feature flags), suministra la URL a través de la configuración en su lugar:

pyfly.yaml

```
pyfly:
  client:
    timeout: 10

services:
  payments:
    base-url: "${PAYMENTS_SERVICE_URL:http://payments-service:8080}"
```

Listado 11.7 — URL base por entorno en pyfly.yaml

Un bean factoría fino lee la clave de configuración y construye el post-procesador con una factoría personalizada que inyecta la URL resuelta. La clase cliente en sí no cambia; solo cambia la factoría.

Deduplicación de peticiones

Las operaciones financieras deben ser idempotentes en la capa HTTP. Si

`create_payment` se llama, agota su tiempo de espera y se reintenta, Payments no debe crear dos registros de pago. El mecanismo estándar es una cabecera

Idempotency-Key: un identificador estable elegido por quien llama —típicamente el UUID del comando— que Payments usa para detectar y deduplicar peticiones repetidas.

lumen/core/services/wallets/settle_transfer_idempotent.py

```

from __future__ import annotations

from lumen.core.mappers.wallet_mapper import to_aggregate, to_entity
from lumen.core.services.wallets.settle_transfer_command import (
    SettleTransfer,
)
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository
from lumen.sdk.payments_client_auth import AuthenticatedPaymentsClient
from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.domain import AggregateNotFound

@command_handler
@service
class SettleTransferIdempotentHandler(
    CommandHandler[SettleTransfer, dict]
):
    """Withdraw funds and submit payment with idempotency key."""

    def __init__(
        self,
        repository: WalletRepository,
        payments: AuthenticatedPaymentsClient,
    ) → None:
        super().__init__()
        self._repository = repository
        self._payments = payments

    async def do_handle(self, command: SettleTransfer) → dict:
        entity = await self._repository.find_by_id(command.wallet_id)
        if entity is None:
            raise AggregateNotFound("Wallet", command.wallet_id)

        wallet = to_aggregate(entity)
        wallet.withdraw(

```

```

        Money(amount=command.amount, currency=wallet.currency)
    )
    await self._repository.upsert(to_entity(wallet))

    idempotency_key = str(command.transfer_id)
    return await self._payments.create_payment(
        body={
            "wallet_id": command.wallet_id,
            "amount": command.amount,
            "currency": wallet.currency.value,
        },
        headers={"Idempotency-Key": idempotency_key},
    )

```

Listado 11.8 — Idempotency-Key reenviada a través del parámetro headers

Cómo funciona: `command.transfer_id` es el identificador estable de esta operación de negocio, determinado antes de que el comando llegue al manejador. Si el manejador se llama de nuevo para el mismo comando —desde un reintento, una reentrega o una reproducción de la cola de mensajes muertos (dead-letter)— pasa la misma `Idempotency-Key`. Payments almacena la clave junto al registro de pago creado y devuelve el registro existente cuando la clave ya se ha visto antes, en lugar de crear un segundo pago. Esa deduplicación es una preocupación del lado del servidor; el trabajo del cliente es simplemente reenviar la clave de forma coherente.

El nivel de experiencia: el BFF

Por qué el frontend no puede hablar directamente con ambos servicios

Cuando una aplicación móvil o un frontend web necesita mostrar un resumen del monedero que incluya instrucciones de pago pendientes, se enfrenta a una disyuntiva: llamar a Wallet para el saldo, llamar a Payments para la lista de pendientes y fusionar

los resultados en el cliente, o hablar con una única API que haga la fusión en el lado del servidor. La primera opción incurre en dos viajes de ida y vuelta, expone la forma interna de cada servicio al cliente y obliga al cliente a implementar reintentos y manejo de errores para dos dominios de fallo independientes. La segunda opción es el **patrón BFF**.

Un **Backend for Frontend** es un servicio ligero en el *nivel de experiencia* que compone respuestas de múltiples servicios de dominio en una forma adaptada a las necesidades de un frontend concreto. Se encarga de la agregación de respuestas, del renombrado de campos para que coincidan con las convenciones del cliente y del cacheo de los resultados compuestos. Nunca toca la base de datos directamente: depende por completo de clientes de servicios de dominio.

Construir el BFF de Lumen

El SDK de Lumen ya incluye un `LumenClient` en `lumen/sdk/client.py` que envuelve un `httpx.AsyncClient` crudo: recibe un `httpx.AsyncClient` en su constructor y llama a `self._http.get(...)` / `self._http.post(...)` a mano, lanzando en caso de error con `response.raise_for_status()`. Ese es el estilo *imperativo*: útil, explícito y enteramente responsabilidad tuya mantenerlo resiliente. En el nivel BFF usas en su lugar el `@service_client` declarativo de PyFly: el código que llama tiene el mismo aspecto, pero el cortacircuitos y los reintentos vienen incorporados automáticamente.

Construiremos el BFF en cuatro piezas pequeñas, cada una en su propio archivo:

Paso 1 — Un `WalletClient` que sepa cómo alcanzar el servicio Wallet. **Paso 2 — Un `PaymentsClient`** con un endpoint extra que el BFF necesita. **Paso 3 — Un `WalletSummaryService`** que llame a ambos y fusione los resultados. **Paso 4 — Un controlador fino** que exponga la vista fusionada.

Empieza con el cliente del lado del monedero.

lumen_bff/sdk/wallet_client.py

```

from __future__ import annotations

from pyfly.client import get, service_client

@service_client(
    base_url="http://wallet-service:8080",
    circuit_breaker=True,
    retry=3,
)
class WalletClient:
    """Typed HTTP client for the Lumen Wallet service."""

    @get("/api/v1/wallets/{wallet_id}")
    async def get_wallet(self, wallet_id: str) → dict:
        """GET /api/v1/wallets/:wallet_id - fetch a wallet."""
        ...

    @get("/api/v1/wallets/{wallet_id}/balance")
    async def get_balance(self, wallet_id: str) → dict:
        """GET /api/v1/wallets/:wallet_id/balance - current balance."""
        ...

```

Listado 11.9 — WalletClient para el nivel BFF

Las rutas reflejan el controlador real de Lumen — `@request_mapping("/api/v1/wallets")` con `@get_mapping("/{wallet_id}")` — así que el cliente del BFF coincide exactamente con lo que el servicio Wallet expone.

El servicio Payments necesita un endpoint `list_pending` que permita al BFF consultar registros pendientes por monedero:

lumen_bff/sdk/payments_client_bff.py

```

from __future__ import annotations

from pyfly.client import get, post, service_client

@service_client(
    base_url="http://payments-service:8080",
    circuit_breaker=True,
    retry=3,
)
class PaymentsClient:
    """Typed HTTP client for Payments (BFF edition)."""

    @post("/payments")
    async def create_payment(self, body: dict) → dict:
        """POST /payments – submit a payment instruction."""
        ...

    @get("/payments/{payment_id}")
    async def get_payment(self, payment_id: str) → dict:
        """GET /payments/:payment_id – fetch a payment by ID."""
        ...

    @get("/payments")
    async def list_pending(self, wallet_id: str) → list:
        """GET /payments?wallet_id=... – list payments for a wallet."""
        ...

```

Listado 11.10 — *PaymentsClient* ampliado para el BFF

El servicio BFF compone entonces el saldo del monedero con la lista de pagos pendientes en una única respuesta.

i ASYNCIO.GATHER, EN LENGUAJE LLANO

`asyncio.gather(a, b)` arranca las corrutinas `a` y `b` al mismo tiempo y espera a que ambas terminen, devolviendo sus resultados como una lista. Para un BFF esto significa que dos llamadas aguas arriba se solapan en vez de encolarse una tras otra. Añadir `return_exceptions=True` cambia una cosa: en lugar de que todo el `gather` reviente cuando una llamada falla, la excepción de la llamada fallida se devuelve como el *resultado* de esa llamada, de modo que puedes inspeccionarla y aun así usar la exitosa.

lumen_bff/application/bff_service.py

```

from __future__ import annotations

import asyncio

from lumen_bff.sdk.payments_client_bff import PaymentsClient
from lumen_bff.sdk.wallet_client import WalletClient
from pyfly.container import service

@service
class WalletSummaryService:
    """Composes wallet balance and pending payments into one view.

    Calls both domain services concurrently using asyncio.gather so the
    total latency is max(wallet_latency, payments_latency) rather than
    their sum.
    """

    def __init__(
        self,
        wallet: WalletClient,
        payments: PaymentsClient,
    ) → None:
        self._wallet = wallet
        self._payments = payments

    async def get_summary(self, wallet_id: str) → dict:
        """Return a unified summary for the given wallet."""
        wallet_data, pending = await asyncio.gather(
            self._wallet.get_wallet(wallet_id),
            self._payments.list_pending(wallet_id),
            return_exceptions=True,
        )

        balance_minor: int = 0
        if isinstance(wallet_data, dict):
            balance_minor = wallet_data.get("balance_minor", 0)

```



```

pending_list: list = []
if isinstance(pending, list):
    pending_list = pending

return {
    "wallet_id": wallet_id,
    "balance_minor": balance_minor,
    "pending_payments": pending_list,
}

```

Listado 11.11 — Servicio BFF que compone Wallet + Payments

Cómo funciona — el patrón de composición:

`asyncio.gather(...)` dispara ambas llamadas aguas arriba de forma concurrente. Las llamadas a wallet y a payments se ejecutan en paralelo, así que la latencia compuesta está acotada por la más lenta de las dos en vez de por su suma: a 50 ms por servicio, las llamadas secuenciales cuestan 100 ms mientras que las concurrentes cuestan aproximadamente 55 ms.

`return_exceptions=True` es crítico para un BFF. Sin él, un único fallo aguas arriba lanza una excepción y quien llama no recibe nada. Con él, una corrutina fallida devuelve su objeto excepción como resultado en lugar de propagarlo. El servicio inspecciona cada resultado con `isinstance(wallet_data, dict)` y degrada con elegancia: devuelve una respuesta parcial con un saldo cero o una lista de pagos vacía en vez de un HTTP 500. El BFF debería hacer esa decisión explícita en la forma de su respuesta, por ejemplo incluyendo una clave `"errors"` que enumere los campos degradados.

El nombre de campo `balance_minor` sigue la convención de Lumen: las cantidades se almacenan como unidades menores enteras (céntimos) y el campo se llama `balance_minor` en todas partes: en `WalletDto`, en las respuestas de depósito/reintegro y aquí en el resumen del BFF.

Cada envoltura `@service_client` sobre `WalletClient` y `PaymentsClient` gestiona los reintentos y el cortacircuitos para su llamada aguas arriba de forma independiente. Si Payments tiene el circuito abierto, el saldo del monedero sigue apareciendo; solo la lista de pagos pendientes está vacía.

El controlador del BFF

El BFF expone su respuesta compuesta a través de un manejador web estándar de PyFly. El controlador es deliberadamente fino: su única misión es delegar en el servicio:

lumen_bff/web/controllers/summary_controller.py

```

from __future__ import annotations

from lumen_bff.application.bff_service import WalletSummaryService
from pyfly.container import rest_controller
from pyfly.web import get_mapping, request_mapping


@rest_controller
@request_mapping("/api/v1/wallets")
class WalletSummaryController:
    """Experience-tier controller for the wallet summary view."""

    def __init__(self, summary: WalletSummaryService) → None:
        self._summary = summary

    @get_mapping("/{wallet_id}/summary")
    async def get_wallet_summary(self, wallet_id: str) → dict:
        """GET /api/v1/wallets/:wallet_id/summary"""
        return await self._summary.get_summary(wallet_id)

```

Listado 11.12 — Controlador del BFF

Cómo funciona: El controlador del BFF no importa ningún modelo de dominio y no toca ningún repositorio: depende solo de `WalletSummaryService`, que a su vez depende solo de interfaces de cliente tipadas. La cadena de dependencias es controlador → servicio BFF → clientes declarativos → HTTP remoto. Cada capa es comprobable de forma independiente: el controlador con un servicio simulado (mock), el servicio con clientes simulados, y los clientes con un `HttpClientPort` simulado.

Ejecútalo — llama al endpoint compuesto. Con la aplicación BFF en ejecución (`uv run pyfly run --server uvicorn`) y los servicios Wallet y Payments alcanzables, llama a la ruta del resumen con un id de monedero que ya hayas creado:

```
curl -s http://localhost:8080/api/v1/wallets/wal-123/summary
```

Forma esperada (los valores dependen de tus datos):

```
{"wallet_id": "wal-123", "balance_minor": 5000, "pending_payments": []}
```

La respuesta única fusiona dos servicios aguas arriba. Para ver la degradación elegante en acción, detén el servicio Payments y vuelve a llamar: el `balance_minor` sigue volviendo desde Wallet, y `pending_payments` recurre a `[]` en lugar de que toda la petición devuelva un 500. Eso es `return_exceptions=True` haciendo su trabajo.

❗ PUERTO POR DEFECTO DE LA APLICACIÓN

PyFly sirve la aplicación en `pyfly.server.port`, cuyo valor por defecto es `8080` (coincidiendo con `server.port` de Spring). Anúlalo con la variable de entorno `PYFLY_SERVER_PORT` o con la clave de configuración `pyfly.server.port`. Ten en cuenta que el actuador y el panel de administración se ejecutan en un puerto de gestión *separado* — `pyfly.management.server.port`, por defecto `9090` — de modo que los endpoints de salud e información nunca chocan con tus rutas de API.

❗ ALCANCE DEL BFF Y PROPIEDAD DEL EQUIPO

Un BFF tiene como alcance un frontend o un recorrido de usuario, no uno por microservicio. Lumen podría tener un `lumen-mobile-bff` y un `lumen-web-bff`, cada uno componiendo los mismos servicios de dominio pero devolviendo formas optimizadas para sus respectivos clientes. El BFF es propiedad del equipo de frontend, no del equipo de dominio. Los servicios de dominio exponen contratos estables; los BFF adaptan esos contratos a formas específicas del cliente sin acoplar los servicios de dominio a las convenciones de ningún frontend en particular.

🍃 EQUIVALENCIA CON SPRING

El patrón BFF en PyFly refleja el enfoque API Gateway / BFF de Spring Boot, donde una aplicación fina de Spring Boot agrega respuestas de múltiples microservicios. En la pila reactiva de Spring, `Mono.zip()` proporciona la misma agregación concurrente que `asyncio.gather()` proporciona en Python. El `@FeignClient` en el BFF se corresponde con `@service_client` en PyFly; el enfoque del `WebClient` de Spring de encadenar llamadas `.flatMap()` se corresponde con la combinación de `asyncio.gather()` + manejo de errores con `isinstance` de PyFly. El modelo de propiedad por equipos —el BFF es propiedad del equipo de frontend, los servicios de dominio son propiedad de los equipos de dominio— es idéntico.

✓ Lo que construiste

Empezaste este capítulo con un único servicio Lumen y lo terminaste con una arquitectura que escala en múltiples dimensiones. Esto es lo que cambió y por qué importa.

Extrajiste PaymentsService. Payments ahora se ejecuta en su propio proceso, con su propia canalización de despliegue y su propio almacén de datos. Los manejadores de Wallet no saben nada de cómo Payments almacena los registros de pago ni qué motor de base de datos usa: todo lo que ven es la interfaz tipada que `PaymentsClient` expone.

Declaraste el cliente, no la implementación. `@service_client` con stubs `@post`, `@get`, `@patch` y `@delete` te dio una interfaz tipada que los IDE navegan y los verificadores de tipos comprueban. `HttpClientBeanPostProcessor` generó la implementación HTTP en el arranque, leyó la configuración de resiliencia de `pyfly.yaml` y registró el bean para inyectarlo en cualquier sitio.

Hiciste la red resiliente. `circuit_breaker=True` y `retry=3` en el decorador envolvieron cada método con un cortacircuitos y una política de reintentos compartidos —cortacircuitos fuera, reintento dentro— de modo que una caída sostenida de Payments abre el circuito rápido mientras que los errores transitorios se recuperan automáticamente. Las excepciones tipadas (`ServiceNotFoundException` , `ServiceUnavailableException`) dan a quien llama una señal limpia sin exponer códigos de estado HTTP en crudo.

Introdujiste el nivel BFF. `WalletSummaryService` compone dos llamadas aguas arriba con `asyncio.gather` , devuelve una respuesta parcial cuando uno de los servicios está degradado y expone un único contrato al frontend. El BFF absorbe el ciclo de publicación independiente de cada servicio de dominio y protege al frontend de sus formas internas.

Tres principios atraviesan el resto de la Parte IV:

- **Depende del cliente tipado, no de `httpx` directamente.** La declaración es tu contrato; la implementación es un detalle del framework.
- **La resiliencia pertenece a la capa de cliente.** Configúrala una vez en `@service_client` ; cada manejador que use el cliente la hereda.
- **Los BFF componen; los servicios de dominio proveen.** Los servicios de dominio poseen contratos estables y de grano fino; los BFF poseen las composiciones de grano grueso que necesitan frontends concretos.

Pruébalo tú mismo

1. Añade un cuarto endpoint y verifica la interpolación de rutas. Amplía

`PaymentsClient` con un método `@get("/payments")` que acepte `wallet_id: str` y `status: str = "pending"` como parámetros. Llámalo desde

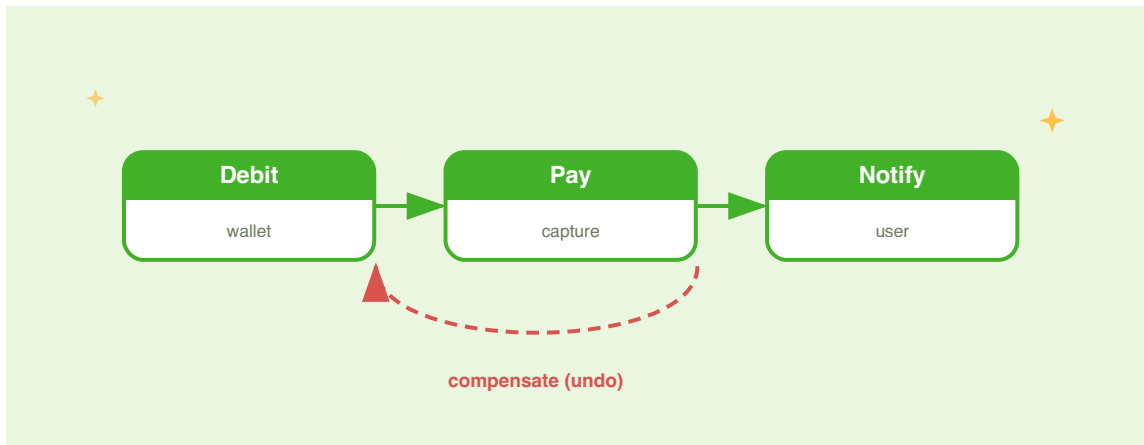
una prueba y afirma que la petición HTTP generada es `GET /payments?wallet_id=abc&status=pending`. Verifica que cambiar el valor por defecto a `status="completed"` y llamar al método sin un argumento `status` envía `status=completed` en la cadena de consulta.

2. **Prueba el BFF con servicios aguas arriba degradados.** Escribe una prueba unitaria para `WalletSummaryService.get_summary` que simule `WalletClient.get_wallet` para que tenga éxito y `PaymentsClient.list_pending` para que lance `ServiceUnavailableException` (impórtala con `from pyfly.client.exceptions import ServiceUnavailableException`). Afirma que el método devuelve un dict con el `balance_minor` correcto y una lista `pending_payments` vacía, confirmando que el repliegue a respuesta parcial funciona y que un único fallo aguas arriba no se propaga como excepción hacia quien llama al BFF. Ejecútala con `uv run --extra dev pytest tests/test_wallet_summary.py -q` y busca `1 passed`.

3. **Ajusta los umbrales del cortacircuitos para un servicio aguas arriba frágil.** Supón que Payments tiene una ventana conocida de inestabilidad durante su ejecución nocturna por lotes: devuelve 503 en aproximadamente el 20 % de las peticiones durante unos 10 segundos antes de estabilizarse. Configura `PaymentsClient` con `circuit_breaker_failure_threshold=2` y `circuit_breaker_recovery_timeout=15.0` y escribe una prueba usando un `HttpClientPort` simulado que simule dos fallos consecutivos seguidos de un éxito. Afirma que la tercera llamada (el sondeo tras el tiempo de espera de recuperación) tiene éxito y que el circuito transita de vuelta a `CLOSED`.

CAPÍTULO 12

Transacciones distribuidas: sagas, flujos de trabajo y TCC



El Capítulo 10 envió los eventos del monedero de Lumen a través de las fronteras de proceso mediante Kafka. El Capítulo 11 dividió la aplicación en servicios que cooperan entre sí y mostró cómo invocarlos por HTTP. Ambos pasos desbloquearon escala y autonomía de los equipos, pero también expusieron una nueva clase de peligro: varios agregados —o varios servicios— pueden necesitar cambiar su estado como parte de una sola operación de negocio, sin ninguna transacción ACID distribuida que te proteja.

Imagina una transferencia entre monederos de Lumen. Cargas el monedero origen y luego abonas el destino. Si se carga el origen y el abono falla —moneda incorrecta, monedero inexistente— el dueño del origen pierde dinero sin que se haya depositado nada en el otro lado. No puedes envolver dos llamadas independientes al repositorio

en un único `BEGIN ... COMMIT` cuando cada agregado posee su propia frontera de consistencia, y el commit de dos fases entre agregados independientes es operativamente frágil.

La respuesta es la **consistencia eventual con compensación explícita**. Cada paso confirma su propio almacén de forma independiente, y diseña una ruta de recuperación —una **transacción de compensación**— para cada paso que pudiera tener éxito antes de que otro posterior falle. Cuando toda la secuencia tiene éxito obtienes tu resultado de negocio; cuando cualquier paso falla, el motor recorre hacia atrás los pasos completados, invocando cada compensación para restaurar un estado consistente. Este capítulo te muestra cómo construirlo con el módulo `pyfly.transactional` de PyFly.

Modelarás la transferencia de dinero como una **saga orquestada**: una clase central que declara cada paso y su compensación, organizada en un DAG (grafo acíclico dirigido) para que el motor pueda ejecutar en paralelo los pasos independientes. Después explorarás la compensación en profundidad, el patrón **Workflow** (flujo de trabajo) para flujos de larga duración o con intervención humana, y **TCC (Try-Confirm-Cancel)** como alternativa basada en reservas. Una sección final muestra cómo la persistencia conectable permite al motor sobrevivir a una caída de proceso y reanudar automáticamente las ejecuciones obsoletas.

El problema de las escrituras distribuidas

Antes de escribir nada de código, concretemos los modos de fallo.

Dos agregados, sin red de seguridad

La transferencia entre monederos de Lumen opera sobre dos agregados `Wallet` almacenados en el mismo esquema de PostgreSQL pero tratados como objetos de dominio independientes: cada uno se carga, se muta y se guarda en su propio viaje de ida y vuelta. Los dos pasos son:

1. **Cargar el origen** — retira `amount` del `Wallet` origen (impone `balance ≥ 0`).
2. **Abonar el destino** — deposita `amount` en el `Wallet` destino (impone coincidencia de moneda).

En un monolito, ambas escrituras podrían compartir una sola transacción de base de datos. En el servicio de dominio de Lumen cada paso es una llamada independiente al repositorio. Una discrepancia de moneda en el destino, o un ID de monedero inexistente, hace que el paso 2 falle después de que el paso 1 ya haya confirmado, dejando el monedero origen cargado y el destino sin cambios. El usuario pierde dinero.

Reintentar toda la operación no es seguro: podrías cargar el origen dos veces. Saltarse silenciosamente el paso fallido deja los saldos inconsistentes. Necesitas un patrón con principios que confirme cada paso de forma independiente y deshaga de forma consistente cada paso completado cuando hay un fallo.

Consistencia eventual y compensación

Una **saga** descompone la operación en una secuencia de transacciones locales, cada una confirmando su propio almacén de forma independiente. Cuando un paso falla, el motor ejecuta **transacciones de compensación** en orden inverso para cada paso completado. Las compensaciones no son rollbacks de base de datos; son *deshacer semánticos*: nuevas operaciones hacia delante que revierten el efecto. "Reabonar el monedero origen" es una nueva operación de depósito que restaura el saldo original, no un rollback.

i LAS SAGAS SON EVENTUALMENTE CONSISTENTES

Una saga no te ofrece serializabilidad ni aislamiento. Entre el momento en que se carga el monedero origen y el momento en que se abona el monedero destino, otra petición podría leer el monedero origen y ver un saldo inferior al que finalmente tendrá. Este es el compromiso que aceptas cuando eliges operar sobre agregados independientes sin un bloqueo distribuido. Las sagas te dan *consistencia al final* —o todos los pasos hacia delante confirmaron o todos quedaron compensados— no *consistencia en cada punto*.

Una saga orquestada

El módulo `pyfly.transactional` de PyFly proporciona los decoradores `@saga` y `@saga_step`. Declaras una clase por saga, anotas cada método como un paso con su compensación, y declaras el orden de dependencias. El motor descubre la clase a través del contenedor de inyección de dependencias, construye un DAG validado en el arranque y dirige la ejecución de forma asíncrona.

i TÉRMINO NUEVO: ORQUESTACIÓN

Orquestación significa que un componente central —aquí, el `SagaEngine` de PyFly— decide el orden en que se ejecutan los pasos y qué hacer cuando uno falla. La alternativa, la *coreografía*, hace que cada servicio reaccione a eventos sin un director central. Este capítulo usa orquestación porque hace que la ruta de recuperación sea explícita y fácil de probar: el motor posee las reglas, tu clase de saga solo declara los pasos.

Construiremos la saga de transferencia en cuatro movimientos: encender el motor, declarar la clase de saga, observar el DAG que el motor construye a partir de ella y, por último, invocar el motor desde un servicio. Vamos uno a uno.

Activar el motor

El motor transaccional se activa mediante el decorador de arranque (*starter*)

`@enable_domain_stack` sobre tu clase de aplicación, y ese único decorador es todo lo que necesitas. No hace falta YAML adicional. En Lumen:

Añade el decorador de arranque. Abre tu clase de aplicación y apila

`@enable_domain_stack` por encima de `@pyfly_application`. Un decorador de arranque (*starter*) es la forma que tiene PyFly de encender un área de funcionalidad completa (aquí, el motor transaccional y su cableado de inyección de dependencias) sin que tengas que registrar cada componente a mano; el equivalente en Spring es una anotación `@EnableXxx`.

lumen/app.py

```
from pyfly.core import pyfly_application
from pyfly.starters.domain import enable_domain_stack

@EnableDomainStack
@pyfly_application(
    name="lumen",
    scan_packages=[
        "lumen.models.repositories",
        "lumen.core.services.transfers",
        # ... other packages
    ],
)

class LumenApplication:
    pass
```

Listado 12.1 — Activar el motor transaccional mediante el domain stack

Apagarlo, o encenderlo bajo un starter más reducido. `@enable_domain_stack` ya establece `pyfly.transactional.enabled: true` por ti, así que el motor está activo en cuanto el decorador está sobre la clase. Esa misma propiedad es la palanca a la que recurre en dos situaciones:

- **Para apagar el motor** bajo el domain stack, ponla en `false` en `application.yaml`: la autoconfiguración se condiciona a que el valor sea exactamente `"true"`, así que cualquier otra cosa lo deshabilita.
- **Para encenderlo bajo un starter más reducido** como `@enable_core_stack` (que *no* incluye el motor transaccional), añade tú mismo la propiedad:

```
pyfly:  
  transactional:  
    enabled: true
```

❗ PRUÉBALO: CONFIRMA QUE EL MOTOR QUEDÓ CABLEADO

Arranca la aplicación en su puerto por defecto (`pyfly.server.port` es `8080` en la v26.6.110) y observa el log de arranque:

```
uv run pyfly run
```

Entre las líneas de arranque deberías ver registrarse los componentes transaccionales, por ejemplo:

```
INFO pyfly.starters.domain domain stack enabled: transactional engine
active
INFO pyfly.transactional registered saga 'money-transfer' (2 steps)
INFO pyfly.server Uvicorn running on http://0.0.0.0:8080
```

Si pones explícitamente `transactional.enabled: false` (o habilitas solo el core stack sin añadir la propiedad), la línea de la saga nunca aparece y `SagaEngine.execute(...)` lanzará más adelante `ValueError: Saga 'money-transfer' is not registered`. Ver la línea `registered saga` es tu prueba de que el cableado funcionó.

Cómo funciona: `@enable_domain_stack` fusiona `DOMAIN_STACK_PROPERTIES` en la configuración activa, y ese diccionario ya contiene `pyfly.transactional.enabled: "true"`, así que el decorador *registra* y activa el motor en un solo movimiento. La autoconfiguración, `TransactionalEngineAutoConfiguration`, está protegida por `@conditional_on_property("pyfly.transactional.enabled", having_value="true")`; como el starter puso el valor en `"true"`, la condición coincide y la autoconfiguración cablea cada componente del motor — `SagaEngine`, `TccEngine`, `WorkflowEngine`, `SagaRegistry`, `InMemoryPersistenceAdapter` y `LoggerEventsAdapter` — en el contenedor de inyección de dependencias. El `OrchestrationBeanPostProcessor` escanea entonces cada bean producido en el

arranque: cualquier bean que lleve metadatos `__pyfly_saga__` se registra automáticamente en `SagaRegistry`. Nunca llamas a `registry.register_from_bean()` en código de producción.

❗ QUÉ ACABA DE PASAR

Un pequeño cambio —un único decorador— te dio un motor de sagas completamente cableado. `@enable_domain_stack` declaró los componentes y los encendió (establece `pyfly.transactional.enabled: true` por ti), y un post-procesador de beans de arranque encontró tus clases de saga y las registró por ti. A partir de aquí solo escribes clases de saga y llamas a `SagaEngine.execute(...)`; la fontanería está hecha.

Declarar la saga de transferencia

La transferencia entre monederos de Lumen es una saga de dos pasos: cargar el monedero origen y luego abonar el destino. Si el abono falla —moneda incorrecta o monedero inexistente— el motor compensa reabonando el origen, devolviendo ambos saldos a sus valores originales.

❗ TÉRMINO NUEVO: COMPENSACIÓN

Una *compensación* (o *transacción de compensación*) es el deshacer de un paso. No es un rollback de base de datos: para cuando compensas, la escritura original ya ha confirmado en su propio almacén. En cambio es una *nueva operación hacia delante* que revierte semánticamente el efecto. El deshacer de "cargar el origen" no es `ROLLBACK`; es "depositar la misma cantidad de vuelta en el origen". Cada paso que cambia el estado necesita una compensación que le corresponda.

Constrúyela en tres movimientos. **Paso 1:** declara la clase y apila los decoradores. **Paso 2:** escribe los pasos hacia delante y su compensación. **Paso 3:** cablea los parámetros con marcadores de inyección. El archivo completo está abajo; luego recorreremos cada movimiento.

```
lumen/core/services/transfers/money_transfer_saga.py
```

```
from __future__ import annotations

from dataclasses import dataclass
from typing import Annotated

from lumen.core.mappers.wallet_mapper import to_aggregate
from lumen.core.services.transfers.transfer_request import TransferRequest
from lumen.interfaces.enums.v1.currency import Currency
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.domain import AggregateNotFound, BusinessRuleViolation
from pyfly.transactional.saga.annotations import (
    FromStep,
    Input,
    saga,
    saga_step,
)
from pyfly.transactional.saga.core.context import SagaContext

MONEY_TRANSFER_SAGA = "money-transfer"

@dataclass(frozen=True)
class DebitResult:
    wallet_id: str
    amount: int
    currency: Currency
    balance: int

@saga(name=MONEY_TRANSFER_SAGA)
@service
class MoneyTransferSaga:
    """Debit source wallet, credit destination; compensate on failure."""

    def __init__(self, repository: WalletRepository) → None:
```



```

        self._repository = repository

# -- Step 1: debit the source -----

@saga_step(id="debit-source", compensate="recredit_source")
async def debit_source(
    self,
    request: Annotated[TransferRequest, Input()],
    ctx: SagaContext,
) → DebitResult:
    entity = await self._repository.find_by_id(
        request.source_wallet_id
    )
    if entity is None:
        raise AggregateNotFound("Wallet", request.source_wallet_id)
    wallet = to_aggregate(entity)
    # The aggregate checks amount, currency and the loaded balance.
    wallet.withdraw(
        Money(amount=request.amount, currency=request.currency)
    )
    wallet.clear_events()
    # One guarded UPDATE: refused if a concurrent debit got there first.
    balance = await self._repository.debit(
        request.source_wallet_id, request.amount
    )
    if balance is None:
        raise BusinessRuleViolation(
            "wallet-insufficient-funds",
            "a concurrent debit spent the funds",
        )
    return DebitResult(
        wallet_id=request.source_wallet_id,
        amount=request.amount,
        currency=request.currency,
        balance=balance,
    )

async def recredit_source(
    self,

```

```

        debit: Annotated[DebitResult, FromStep("debit-source")],
    ) → int:
        """Compensation: put the money back. Receives the forward step's
        result via FromStep – NOT the saga input."""
        balance = await self._repository.credit(
            debit.wallet_id, debit.amount
        )
        if balance is None:
            raise AggregateNotFound("Wallet", debit.wallet_id)
        return balance

# -- Step 2: credit the destination -----

@saga_step(id="credit-destination", depends_on=["debit-source"])
async def credit_destination(
    self,
    request: Annotated[TransferRequest, Input()],
    ctx: SagaContext,
) → int:
    entity = await self._repository.find_by_id(
        request.destination_wallet_id
    )
    if entity is None:
        raise AggregateNotFound(
            "Wallet", request.destination_wallet_id
        )
    wallet = to_aggregate(entity)
    wallet.deposit(
        Money(amount=request.amount, currency=request.currency)
    )
    wallet.clear_events()
    balance = await self._repository.credit(
        request.destination_wallet_id, request.amount
    )
    if balance is None:
        raise AggregateNotFound(
            "Wallet", request.destination_wallet_id
        )
    return balance

```

Listado 12.2 — *MoneyTransferSaga*: cargo → abono, con compensación

Cómo funciona, paso a paso:

Paso 1: la pila de decoradores. `@saga(name=MONEY_TRANSFER_SAGA)` estampa `__pyfly_saga__` en la clase con el nombre de la saga. El decorador solo adjunta metadatos: no envuelve la clase ni crea un proxy. **El requisito crítico** es que `@saga` debe apilarse *por encima de* `@service`. La anotación `@service` hace que el contenedor de inyección de dependencias instancie y escanee el bean en el arranque; el gancho `OrchestrationBeanPostProcessor.after_init()` ve entonces `__pyfly_saga__` en el bean y llama a `SagaRegistry.register_from_bean()`. Sin `@service`, la clase nunca se escanea y la saga no puede ejecutarse por nombre.

⚠ EL ORDEN DE LOS DECORADORES NO ES OPCIONAL

Lee la pila de arriba abajo: `@saga` está *por encima* de `@service`. Intercámbialos — `@service` por encima de `@saga` — y el bean sigue registrándose con la inyección de dependencias, pero los metadatos de la saga se aplican al objeto ya envuelto, el post-procesador nunca los encuentra y `execute("money-transfer")` falla con `ValueError: Saga 'money-transfer' is not registered`. Si te topas con ese error, comprueba primero el orden de los decoradores.

Paso 2: los métodos de paso. `@saga_step` adjunta los metadatos `__pyfly_saga_step__` directamente al método asíncrono —sin envoltura, sin proxy— así que `inspect.iscoroutinefunction` sigue devolviendo `True` y el motor hace correctamente `await` de la llamada. El parámetro `compensate="recredit_source"` nombra el *método de la misma clase* que se invoca al deshacer este paso. Omitir `depends_on` (o pasar `[]`) significa que el paso puede ejecutarse en cuanto el motor arranca.

Interacción con el repositorio: comprobar y cambiar en una sola sentencia. Cada paso hacia delante usa el repositorio del framework

`WalletRepository(Repository[WalletEntity, str])` de la misma forma:

1. `find_by_id(id)` — carga la fila cruda `WalletEntity` desde la base de datos.
2. `to_aggregate(entity)` — rehidrata la rica raíz de agregado `Wallet` a partir de esa fila; el agregado impone sus invariantes (un importe positivo, la misma moneda, `balance ≥ 0` para el saldo con el que se cargó).
3. Comprobar — llama a `wallet.withdraw(...)` o `wallet.deposit(...)` sobre el agregado, dejando que lance `BusinessRuleViolation` si se rompe una invariante antes de que ocurra ninguna escritura.
4. `debit(...)` o `credit(...)` — cambia el saldo guardado con una sola sentencia con guarda: `debit` ejecuta `UPDATE ... SET balance_minor = balance_minor - :amount WHERE id = :id AND balance_minor ≥ :amount` y devuelve el nuevo saldo, o `None` cuando el saldo ya no cubre el importe.

¿Por qué no cargar, mutar y hacer `upsert` como los manejadores de comandos? Porque un paso de saga nunca se une a la transacción de quien lo llama. Desde la v26.09.08 cada paso se ejecuta en una tarea propia con el estado transaccional limpio, así que confirma por su cuenta y se compensa exactamente una vez si un paso posterior falla. Sin una unidad alrededor del paso, cada llamada al repositorio es una unidad breve propia: la carga y la escritura son dos unidades, y la comprobación del agregado por sí sola podría pasar para dos transferencias en carrera. El `UPDATE` con guarda convierte la comprobación y el cambio en una sola sentencia, así que dos transferencias concurrentes nunca pueden gastar los mismos fondos, y un abono nunca pisa un cambio concurrente. (Hasta la v26.09.07 los pasos compartían la sesión de quien llamaba y confirmaban o revertían con ella.)

Paso 3: cablea los parámetros. La inyección de parámetros usa `typing.Annotated` con **instancias de marcador**, no clases desnudas:

- `Annotated[TransferRequest, Input()]` — `Input()` es una instancia (fíjate en los paréntesis); `Input` desnudo sin `()` no se resuelve.
- `Annotated[DebitResult, FromStep("debit-source")]` — lee el resultado que el paso `"debit-source"` almacenó en `SagaContext` al completarse.
- `ctx: SagaContext` — inyectado por tipo; no hace falta marcador `Annotated`.

El resolutor inspecciona las anotaciones de tipo en tiempo de ejecución mediante `typing.get_type_hints(func, include_extras=True)`.

Los métodos de compensación no reciben la entrada de la saga.

`recredit_source` toma `Annotated[DebitResult, FromStep("debit-source")]` —el valor que devolvió el paso hacia delante— no la `TransferRequest`. Devuelve la cantidad original con el mismo `credit` con guarda que usa el abono hacia delante. Las compensaciones siempre leen de `ctx.step_results` mediante `FromStep`, nunca de la entrada original.

❗ QUÉ ACABA DE PASAR

Escribiste una sola clase que contiene toda la historia de la transferencia: un paso hacia delante para cargar, su compensación para reabonar y un segundo paso hacia delante para abonar. Los decoradores le dijeron al motor *qué es cada método* (un paso, una compensación) y *cómo se conectan* (`compensate=`, `depends_on=`). No escribiste ningún bucle de orquestación ni lógica de rollback con try/except: eso es trabajo del motor. Tu código solo describe la operación de negocio y su deshacer.

El DAG de pasos

📌 TÉRMINO NUEVO: DAG

Un *DAG* —grafo acíclico dirigido— es un conjunto de pasos conectados por flechas de "debe-ejecutarse-antes", sin ciclos (ningún paso puede, directa o indirectamente, depender de sí mismo). El motor lee tus declaraciones `depends_on`, construye este grafo y lo ordena en *capas*: todo lo que está en la capa 0 no tiene dependencias sin satisfacer y se ejecuta primero; la capa 1 se ejecuta cuando termina la capa 0; y así sucesivamente. Los pasos de la misma capa son independientes, así que el motor los ejecuta a la vez. Un ciclo haría imposible la división en capas, por lo que el motor lo rechaza en el arranque y no en tiempo de ejecución.

Los dos pasos forman una cadena lineal:

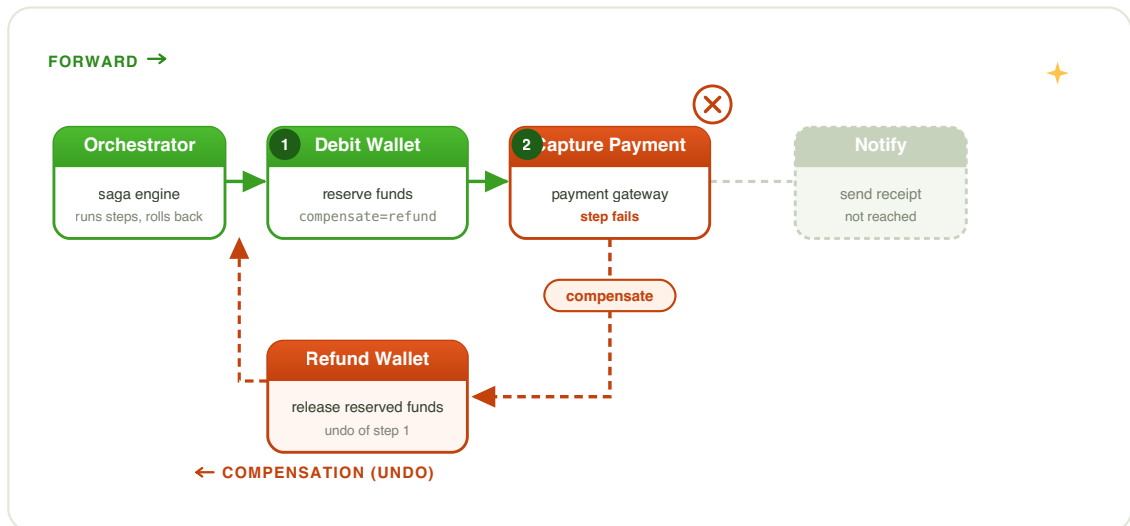


Figura 12.1 — DAG de MoneyTransferSaga: los pasos se ejecutan en orden de capa topológica; los pasos independientes de una capa se ejecutan con `asyncio.gather`.

```

Layer 0:  debit-source
          |
Layer 1:  credit-destination

```

Como `credit-destination` depende de `debit-source`, se ejecutan secuencialmente. Una saga más compleja —una comprobación de fraude y una comprobación de KYC que son independientes entre sí pero ambas alimentan un paso de captura— colocaría las dos comprobaciones en la misma capa y las ejecutaría de forma concurrente con `asyncio.gather`.

Ejecutar la saga

La clase de saga solo *describe* la operación. Para *ejecutarla* necesitas un servicio ligero que inyecte el motor y lo invoque por nombre.

Paso 1: inyecta `SagaEngine`. Declara un `@service` cuyo constructor tome un parámetro `SagaEngine`; el contenedor de inyección de dependencias te entrega el motor autoconfigurado. **Paso 2: llama a `execute`** con el nombre de la saga y la carga de entrada. **Paso 3: pliega el `SagaResult`** en un diccionario pequeño y amigable con JSON para el invocador.

lumen/core/services/transfers/transfer_service.py

```

from __future__ import annotations

from typing import Any

from lumen.core.services.transfers.money_transfer_saga import
MONEY_TRANSFER_SAGA
from lumen.core.services.transfers.transfer_request import TransferRequest
from pyfly.container import service
from pyfly.transactional.saga.core.result import SagaResult
from pyfly.transactional.saga.engine.saga_engine import SagaEngine

@service
class TransferService:
    """Run the money-transfer saga and report the outcome."""

    def __init__(self, saga_engine: SagaEngine) → None:
        self._saga_engine = saga_engine

    async def transfer(self, request: TransferRequest) → dict[str, Any]:
        result: SagaResult = await self._saga_engine.execute(
            saga_name=MONEY_TRANSFER_SAGA,
            input_data=request,
        )

        if result.success:
            debit = result.result_of("debit-source")
            return {
                "status": "completed",
                "correlation_id": result.correlation_id,
                "source_balance": debit.balance,
                "destination_balance": result.result_of("credit-destination"),
            }

        return {
            "status": "failed",
            "correlation_id": result.correlation_id,

```



```

    "failed_steps": list(result.failed_steps().keys()),
    "compensated_steps": list(result.compensated_steps().keys()),
    "error": str(result.error),
}

```

Listado 12.3 — Ejecutar la saga de transferencia de dinero

Cómo funciona: `saga_engine.execute()` resuelve `MoneyTransferSaga` desde el registro por nombre, crea un `SagaContext` con un `correlation_id` UUID autogenerado y empieza a ejecutar capas. En caso de éxito, `SagaResult.success` es `True` y `result_of("debit-source")` devuelve el `DebitResult` que produjo el paso hacia delante. En caso de fallo, `result.failed_steps()` devuelve un diccionario de ID de paso a `StepOutcome` para cada paso que agotó sus reintentos; `result.compensated_steps()` devuelve los pasos que se deshicieron correctamente.

`SagaResult` es una dataclass inmutable y congelada. Sus miembros clave:

- `result.success` — `True` cuando cada paso hacia delante se completó.
- `result.result_of(step_id)` — el valor que devolvió ese paso, o `None`.
- `result.failed_steps()` — diccionario de ID de paso → `StepOutcome` para los pasos fallidos.
- `result.compensated_steps()` — diccionario de ID de paso → `StepOutcome` para los pasos compensados.
- `result.correlation_id` — UUID para correlacionar logs y trazas entre servicios.
- `result.error` — la excepción que detuvo la saga, o `None` en caso de éxito.

❗ PRUÉBALO: EL CAMINO FELIZ Y EL CAMINO COMPENSADO

Expón `TransferService.transfer` detrás de una ruta HTTP (el Capítulo 11 cubrió los controladores) y ejercita ambos desenlaces contra la aplicación en ejecución en `pyfly.server.port` (8080).

Una transferencia válida entre dos monederos existentes de la misma moneda devuelve el resumen completado:

```
curl -s -X POST http://localhost:8080/transfers \
  -d '{"source_wallet_id":"w-1","destination_wallet_id":"w-2","amount":500,"currency":"EUR"}'
```

```
{
  "status": "completed",
  "correlation_id": "8f3c...",
  "source_balance": 9500,
  "destination_balance": 10500
}
```

Ahora apunta la transferencia a un monedero destino que no existe. `credit-destination` lanza `AggregateNotFound`, el motor compensa `debit-source` y la respuesta informa exactamente de eso:

```
curl -s -X POST http://localhost:8080/transfers \
  -d '{"source_wallet_id":"w-1","destination_wallet_id":"does-not-exist","amount":500,"currency":"EUR"}'
```

```
{
  "status": "failed",
  "correlation_id": "1a2b...",
  "failed_steps": ["credit-destination"],
  "compensated_steps": ["debit-source"],
  "error": "Wallet 'does-not-exist' not found"
}
```

La observación clave: vuelve a leer `w-1` después y su saldo está de vuelta en `9500` ; `debit-source` fue deshecho por `recredit_source` . Una transferencia fallida deja ambos monederos exactamente como empezaron.

🌿 EQUIVALENCIA CON SPRING

`@saga` / `@saga_step` reflejan `@Saga` / `@SagaStep` en la librería Java `fireflyframework-transactional-engine` . La regla de la pila de decoradores (`@saga` sobre `@service`) refleja la regla de Java de que `@Saga` debe estar sobre una clase anotada con `@Service` para que el `WorkflowBeanPostProcessor` pueda descubrirla. Los marcadores de inyección de parámetros (`Input()` , `FromStep("id")`) se corresponden directamente con `@Input` y `@FromStep` en la versión Java. El modelo asíncrono difiere: Java usa Project Reactor (`Mono<T>`) mientras que PyFly usa `async/await` nativo con `asyncio.gather` para las capas paralelas.

La compensación en profundidad

El camino feliz es directo: cada paso tiene éxito y la saga confirma. El verdadero reto de diseño es el camino infeliz. Entender qué ocurre en caso de fallo —y por qué la compensación debe diseñarse con cuidado— es lo que separa una saga fiable de una frágil.

Qué se ejecuta en caso de fallo

Cuando un paso falla tras todos los reintentos, el motor entra en *modo de compensación*. Inspecciona `SagaContext` en busca de cada paso cuyo estado sea `DONE` y luego llama a sus métodos de compensación en orden inverso al de finalización bajo la política por defecto `STRICT_SEQUENTIAL` . En `MoneyTransferSaga` ,

un monedero destino inexistente hace que `credit-destination` lance `AggregateNotFound`. El motor compensa entonces el paso que ya se había completado:

```
Forward path: debit-source ✓ → credit-destination ×  
Compensation: recredit_source (for debit-source)
```

El efecto neto: el monedero origen se restaura a su saldo original y el monedero destino nunca fue tocado, como si la transferencia nunca hubiera ocurrido.

Los métodos de compensación reciben sus argumentos a través del mismo sistema de inyección que los pasos hacia delante. `Annotated[DebitResult, FromStep("debit-source")]` lee el `DebitResult` que `debit_source` almacenó en el contexto al completarse, de modo que siempre compensas con los datos realmente confirmados, nunca con una aproximación.

❗ PRUÉBALO: DEMUESTRA LA COMPENSACIÓN EN UNA PRUEBA

No necesitas un servidor en ejecución para verificar el camino infeliz: una prueba unitaria rápida contra el motor basta, y es el tipo de prueba que escribirás para cada saga. Dirige `TransferService.transfer` con un monedero destino que no existe y luego comprueba que la compensación se ejecutó:

```
async def test_failed_transfer_compensates_the_debit(transfer_service):
    result = await transfer_service.transfer(
        TransferRequest(
            source_wallet_id="w-1",
            destination_wallet_id="does-not-exist",
            amount=500,
            currency=Currency.EUR,
        )
    )
    assert result["status"] == "failed"
    assert result["failed_steps"] == ["credit-destination"]
    assert result["compensated_steps"] == ["debit-source"]
```

Ejecuta solo esta prueba (el grupo `--extra dev` instala pytest; el Capítulo 16 cubre los fixtures en profundidad):

```
uv run --extra dev pytest -q -k compensates
```

Salida esperada:

```
1 passed in 0.05s
```

Una prueba en verde aquí es tu garantía de que una transferencia rota nunca deja dinero desaparecido.

Políticas de compensación

Cinco políticas gobiernan cómo ejecuta el motor las compensaciones. Establece el valor por defecto global en YAML o anúlalo por ejecución:

```
pyfly:
  transactional:
    saga:
      compensation_policy: STRICT_SEQUENTIAL
```

Política	Comportamiento	Úsala cuando
STRICT_SEQUENTIAL	Orden inverso, una a una. Se detiene al primer error de compensación.	El orden importa; un rollback parcial es inaceptable.
GROUPED_PARALLEL	Invierte las capas topológicas; compensa cada capa en paralelo.	Quieres velocidad sin violar la estructura de dependencias.
RETRY_WITH_BACKOFF	Orden inverso con backoff exponencial. Continúa si los reintentos tienen éxito.	Es probable que haya fallos de red transitorios durante la compensación.
CIRCUIT_BREAKER	Rastrea fallos consecutivos; se abre tras 3 y omite el resto.	Evitar fallos en cascada; la recuperación manual gestiona los pasos omitidos.
BEST_EFFORT_PARALLEL	Todas las compensaciones a la vez; los errores se registran, nunca se lanzan.	La velocidad es crítica; una reconciliación aparte gestiona los fallos parciales.

⚠ LA COMPENSACIÓN DEBE SER IDEMPOTENTE

El motor puede llamar a un método de compensación más de una vez. Si el motor cae entre la llamada a `void_payment` y la persistencia del resultado de la compensación, llamará a `void_payment` de nuevo al reiniciarse. Tus métodos de compensación deben ser seguros de llamar varias veces con los mismos argumentos. Para anulaciones de pagos, esto significa que el `PaymentsService` debe tratar una doble anulación como un no-op (devolver éxito si ya estaba anulado, no lanzar). Diseña la compensación *antes* de diseñar el paso hacia delante: la idempotencia no es una ocurrencia tardía.

Configuración de compensación por paso

Puedes anular el número de reintentos y el tiempo de espera de un paso de compensación sin cambiar el comportamiento del paso hacia delante:

lumen/transfer/transfer_saga_hardened.py

```

from pyfly.container import service
from pyfly.transactional.saga.annotations import saga, saga_step

@saga(name="money-transfer-hardened")
@service
class HardenedTransferSaga:

    @saga_step(
        id="debit-wallet",
        compensate="refund_wallet",
        depends_on=[],
        retry=3,
        backoff_ms=200,
        timeout_ms=5_000,
        compensation_retry=5,
        compensation_backoff_ms=1_000,
        compensation_timeout_ms=8_000,
        compensation_critical=True,
    )
    async def debit_wallet(self, *args: object) → None: ...

    async def refund_wallet(self, *args: object) → None: ...

```

Listado 12.4 — Reintento y tiempo de espera de compensación por paso

Cómo funciona: `compensation_retry=5` da a la compensación cinco intentos propios, independientes de los tres reintentos del paso hacia delante. `compensation_critical=True` significa que si la compensación agota todos sus reintentos y aun así falla, el motor lanza esa excepción, sacando a la superficie el *fallo de compensación* como un error observable en lugar de tragárselo silenciosamente.

Pasos de compensación externos

Cuando la lógica de compensación es lo bastante compleja como para merecer su propia clase, o cuando vive en un módulo diferente, sácala por completo:

`lumen/transfer/compensation_steps.py`

```
from typing import Annotated

from pyfly.container import service
from pyfly.transactional.saga.annotations import (
    FromStep,
    compensation_step,
)

from lumen.core.services.transfers.money_transfer_saga import DebitResult
from lumen.models.repositories.wallet_repository import WalletRepository

@compensation_step(saga="money-transfer", for_step_id="debit-source")
@service
class SourceRecreditCompensation:

    def __init__(self, repository: WalletRepository) → None:
        self._repository = repository

    async def execute(
        self,
        debit: Annotated[DebitResult, FromStep("debit-source")],
    ) → None:
        """External alternative to the inline recredit_source method."""
        ...
```

Listado 12.5 — Clase de paso de compensación externo

El `SagaRegistry` descubre las clases `@compensation_step` en el arranque junto a las clases `@saga` y las cablea en sus definiciones de paso automáticamente. El parámetro `for_step_id` debe coincidir exactamente con la cadena `id` del paso.

Flujos de trabajo y señales

`@saga` es la herramienta adecuada cuando todos los pasos se conocen de antemano y la operación se completa en minutos. Algunos procesos de negocio son inherentemente más largos: la aprobación de un préstamo a la espera de un responsable de cumplimiento, una incorporación de varios pasos bloqueada por un clic en un correo, un pago que necesita un periodo de enfriamiento antes de liquidarse. Estos encajan en el patrón **Workflow** (flujo de trabajo).

En qué se diferencian los flujos de trabajo de las sagas

	Saga	Workflow
Duración	De segundos a minutos	De minutos a días
Espera	Solo reintentos	Señales, temporizadores, flujos hijos
Intervención humana	No	Sí (<code>@wait_for_signal</code>)
Persistencia de estado	Punto de control por saga	Tras cada capa
Despliegue en DAG	Sí (capas paralelas)	Sí + primitivas de compuerta

Declarar un flujo de trabajo

lumen/transfer/approval_workflow.py

```

from __future__ import annotations

from pyfly.container import service
from pyfly.transactional.core.model import TriggerMode
from pyfly.transactional.workflow.annotations import (
    compensation_step,
    on_workflow_complete,
    on_workflow_error,
    wait_for_signal,
    workflow,
    workflow_query,
    workflow_step,
)

@workflow(
    id="large-transfer-approval",
    trigger_mode=TriggerMode.SYNC,
    timeout_ms=86_400_000,    # 24 hours
    max_retries=1,
)

@service
class LargeTransferWorkflow:
    """High-value transfers require a compliance officer to approve."""

    @workflow_step(id="enrich-request", depends_on=[])
    async def enrich_request(self, payload: dict) → dict:
        return {**payload, "risk_score": 0.12}

    @workflow_step(
        id="compliance-review",
        depends_on=["enrich-request"],
        compensatable=True,
        compensation_method="release_review",
        timeout_ms=82_800_000,
    )
    @wait_for_signal("approved", timeout_ms=82_800_000)

```

```

async def compliance_review(self) → None:
    """Suspends until a compliance officer delivers the signal."""

    @compensation_step(for_step="compliance-review")
    async def release_review(self) → None:
        """Called if the workflow is cancelled during review."""

    @workflow_step(
        id="settle-transfer",
        depends_on=["compliance-review"],
    )
    async def settle_transfer(self, payload: dict) → dict:
        return {"settled": True}

    @workflow_query(name="status")
    async def get_status(self, ctx: object) → str:
        return str(getattr(ctx, "status", "UNKNOWN"))

    @on_workflow_complete
    async def on_done(self, ctx: object) → None:
        pass # emit audit event

    @on_workflow_error
    async def on_error(self, ctx: object, err: Exception) → None:
        pass # alert on-call

```

Listado 12.6 — *LargeTransferWorkflow*: aprobación dirigida por señales para transferencias de alto valor

Cómo funciona: La pila de decoradores sigue la misma regla que las sagas:

`@workflow` por encima de `@service`. `@workflow(id=...)` toma argumentos solo por palabra clave: `id` es obligatorio; todos los demás son opcionales.

`@wait_for_signal("approved", timeout_ms=82_800_000)` se apila por encima de `@workflow_step` y le dice al motor que suspenda en ese paso hasta que se entregue una señal llamada `"approved"`. El motor persiste el `ExecutionContext` en el `ExecutionPersistenceProvider` configurado; si el proceso se reinicia, rehidrata el contexto y reanuda desde la última capa completada.

`@compensation_step(for_step="compliance-review")` usa el argumento por palabra clave `for_step` (no posicional) y registra `release_review` como el manejador (handler) de compensación para el paso `compliance-review`.

`@workflow_query(name="status")` marca un método como un manejador de consulta del lado de lectura: invocable mientras el flujo de trabajo está suspendido sin hacer avanzar la ejecución.

Manejar el motor de flujos de trabajo

lumen/transfer/approval_controller.py

```
from __future__ import annotations

from pyfly.container import service
from pyfly.transactional.workflow.engine import WorkflowEngine
from pyfly.transactional.workflow.result import WorkflowResult

@service
class TransferApprovalService:

    def __init__(self, workflow_engine: WorkflowEngine) → None:
        self._wf = workflow_engine

    async def request_large_transfer(self, payload: dict) → str:
        result: WorkflowResult = await self._wf.start(
            "large-transfer-approval",
            input=payload,
        )
        # Returns immediately; workflow is now suspended at compliance-review.
        return result.correlation_id

    async def approve(self, correlation_id: str, reviewer_id: str) → None:
        await self._wf.deliver_signal(
            correlation_id,
            "approved",
            payload={"by": reviewer_id},
        )

    async def check_status(self, correlation_id: str) → str:
        return await self._wf.query(correlation_id, "status")
```

Listado 12.7 — Iniciar un flujo de trabajo y entregar una señal

Cómo funciona: `workflow_engine.start(workflow_id, input=payload)` ejecuta la primera capa (`enrich-request`) de forma síncrona, luego suspende en `compliance-review` por culpa de `@wait_for_signal` . Devuelve un `WorkflowResult` inmediatamente con un `correlation_id` : el invocador guarda este ID y consulta más tarde. Cuando se llama a `deliver_signal()` , el flujo de trabajo se reanuda y `settle-transfer` se ejecuta hasta completarse.

`WorkflowResult` lleva: `workflow_id` , `correlation_id` , `status` (un enum `ExecutionStatus`), `duration_ms` , `step_results` (diccionario) y `variables` . El booleano `result.successful` es `True` cuando `status` es `COMPLETED` o `CONFIRMED` .

El constructor programático

Cuando necesitas construir un flujo de trabajo de forma dinámica —a partir de una configuración en base de datos o de un motor de reglas— usa `WorkflowBuilder` :

lumen/transfer/dynamic_workflow.py

```

from pyfly.transactional.workflow.builder import WorkflowBuilder
from pyfly.transactional.workflow.definition import WorkflowDefinition

async def enrich_fn(payload: dict) → dict:
    return {**payload, "enriched": True}

async def settle_fn(payload: dict) → dict:
    return {"settled": True}

definition: WorkflowDefinition = (
    WorkflowBuilder("simple-transfer")
    .step("enrich", enrich_fn, depends_on=[])
    .wait_signal(
        "await-approval",
        "approved",
        depends_on=["enrich"],
        timeout_ms=3_600_000,
    )
    .step(
        "settle",
        settle_fn,
        depends_on=["await-approval"],
    )
    .build()
)

```

Listado 12.8 — Construir un flujo de trabajo de forma programática

`WorkflowBuilder.step(step_id, handler, *, depends_on, timeout_ms, max_retries, ...)` acepta un invocable y argumentos por palabra clave para dependencias, tiempos de espera y reintentos. `wait_signal(step_id, signal, *, depends_on, timeout_ms)` inserta un paso de compuerta de señal sin un manejador

real: crea una corrutina interna no-op que el motor sustituye por la lógica de espera de señal. `build()` devuelve un `WorkflowDefinition` que registras directamente con `WorkflowEngine`.

TCC: Try-Confirm-Cancel

El patrón saga ejecuta los pasos hacia delante y compensa hacia atrás. **TCC (Try-Confirm-Cancel)** adopta un enfoque diferente: todos los participantes primero *reservan tentativamente* sus recursos sin confirmar (Try), y luego todos *confirman* esas reservas (Confirm) o todos las *liberan* (Cancel). Esto te da una semántica fuerte de todo-o-nada entre participantes sin un bloqueo distribuido.

TCC encaja en escenarios donde cada participante puede mantener una reserva de forma barata; por ejemplo, preautorizar una retención en una tarjeta de pago en lugar de cobrarla inmediatamente.

Las tres fases

1. **Try** — cada participante reserva recursos. Las reservas son visibles internamente pero no son definitivas. Si algún Try falla, los participantes que tuvieron éxito cancelan sus reservas.
2. **Confirm** — si todas las fases Try tienen éxito, el coordinador instruye a cada participante a confirmar su reserva.
3. **Cancel** — si alguna fase Try falla, el coordinador instruye a cada participante que completó su Try a liberar su reserva.

Declarar una transacción TCC

lumen/transfer/transfer_tcc.py

```

from __future__ import annotations

from typing import Annotated

from pyfly.container import service
from pyfly.transactional.tcc.annotations import (
    FromTry,
    cancel_method,
    confirm_method,
    tcc,
    tcc_participant,
    try_method,
)
from pyfly.transactional.tcc.core.context import TccContext

from lumen.wallet.service import WalletService
from lumen.payments.service import PaymentsService

@tcc(
    name="wallet-transfer",
    timeout_ms=30_000,
    retry_enabled=True,
    max_retries=3,
    backoff_ms=500,
)
@service
class WalletTransferTcc:
    """Reserve funds and payment in lockstep; confirm or cancel together."""

    @tcc_participant(id="wallet-hold", order=1, timeout_ms=5_000)
    class WalletParticipant:

        def __init__(self, wallet_svc: WalletService) → None:
            self._wallet = wallet_svc

        @try_method(timeout_ms=4_000, retry=2, backoff_ms=200)

```

```

    async def try_hold(
        self,
        request: object,
        ctx: TccContext,
    ) → str:
        """Tentatively hold funds – does not debit yet."""
        return await self._wallet.hold_funds(
            wallet_id=getattr(request, "sender_id", ""),
            amount=getattr(request, "amount_cents", 0),
        ) # returns a hold_id

    @confirm_method(timeout_ms=5_000, retry=3)
    async def confirm_hold(
        self,
        hold_id: Annotated[str, FromTry()],
        ctx: TccContext,
    ) → None:
        await self._wallet.commit_hold(hold_id)

    @cancel_method(timeout_ms=3_000, retry=2)
    async def cancel_hold(
        self,
        hold_id: Annotated[str, FromTry()],
    ) → None:
        await self._wallet.release_hold(hold_id)

    @tcc_participant(id="payment-auth", order=2, timeout_ms=8_000)
    class PaymentParticipant:

        def __init__(self, payments_svc: PaymentsService) → None:
            self._payments = payments_svc

    @try_method(timeout_ms=6_000, retry=2, backoff_ms=300)
    async def try_auth(
        self,
        request: object,
        ctx: TccContext,
    ) → str:
        return await self._payments.pre_authorise(

```

```

        amount=getattr(request, "amount_cents", 0),
    ) # returns auth_id

@confirm_method(timeout_ms=8_000, retry=3)
async def confirm_auth(
    self,
    auth_id: Annotated[str, FromTry()],
    ctx: TccContext,
) → None:
    await self._payments.capture_auth(auth_id)

@cancel_method(timeout_ms=4_000, retry=2)
async def cancel_auth(
    self,
    auth_id: Annotated[str, FromTry()],
) → None:
    await self._payments.void_auth(auth_id)

```

Listado 12.9 — *WalletTransferTcc: Try-Confirm-Cancel para la reserva de pago*

Cómo funciona: `@tcc_participant(order=1)` le dice al motor TCC que ejecute la fase Try de `WalletParticipant` antes que la de `PaymentParticipant`: un `order` menor significa más temprano. `FromTry()` es el equivalente de `FromStep` en TCC: inyecta el valor devuelto por el `@try_method` del propio participante en su `@confirm_method` y su `@cancel_method`.

El motor ejecuta las fases Try de todos los participantes en secuencia de `order`. Si todos los Try tienen éxito, ejecuta todos los métodos Confirm. Si algún Try falla, ejecuta Cancel para cada participante que completó su Try, de nuevo en el orden declarado. Un participante `optional=True` que falla su Try no dispara un Cancel global; su fallo se registra y se omite.

Ejecutar una transacción TCC

lumen/transfer/tcc_service.py

```

from __future__ import annotations

from typing import Any

from pyfly.container import service
from pyfly.transactional.tcc.core.result import TccResult
from pyfly.transactional.tcc.engine.tcc_engine import TccEngine

from lumen.core.services.transfers.transfer_request import TransferRequest

@service
class TccTransferService:

    def __init__(self, tcc_engine: TccEngine) → None:
        self._engine = tcc_engine

    async def transfer(self, req: TransferRequest) → dict[str, Any]:
        result: TccResult = await self._engine.execute(
            tcc_name="wallet-transfer",
            input_data=req,
        )

        if result.success:
            hold_id = result.result_of("wallet-hold")
            return {
                "status": "confirmed",
                "hold_id": hold_id,
                "correlation_id": result.correlation_id,
            }

        failed = result.failed_participants()
        return {
            "status": "cancelled",
            "failed": list(failed.keys()),

```



```
    "error": str(result.error),  
}
```

Listado 12.10 — Ejecutar una transacción TCC

TCC frente a saga: elegir el patrón adecuado

Usa esta tabla para elegir entre los dos enfoques:

Pregunta	Saga	TCC
¿Los pasos se ejecutan independientemente?	Sí — cada uno confirma localmente	No — todas las fases Try deben tener éxito primero
¿Necesita lógica de compensación?	Sí, por paso	No — Cancel gestiona el rollback
¿Se necesita reserva de recursos?	No	Sí — los participantes mantienen recursos durante Try
Mejor para	Operaciones secuenciales largas	Bloqueos cortos de todo-o-nada

Persistencia: sobrevivir a una caída

El motor almacena el estado de la saga y de TCC a través del protocolo `TransactionalPersistencePort`, que implementa el bean `transactional_persistence_port` sobre el proveedor que elige `pyfly.transactional.persistence.provider`. El valor por defecto, `memory`, es rápido para desarrollo pero se pierde al reiniciar el proceso; los despliegues de producción eligen uno duradero (`redis`, `sqlalchemy` o `cache`). Con `sqlalchemy`, una saga iniciada dentro de `@transactional` escribe su registro en la transacción de

quien llama, así que inicia la saga fuera de la transacción de negocio cuando su registro deba sobrevivirla. (Antes de la v26.09.08 los motores de saga y de TCC guardaban siempre su estado en memoria, fuera cual fuera el proveedor.)

Cómo fluye el estado

El motor registra cuándo empieza una ejecución y cómo termina:

1. `persistence_port.persist_state(state)` — el registro `IN_FLIGHT` de la ejecución, al empezar.
2. `persistence_port.mark_completed(correlation_id, successful)` — el resultado final de la saga.

Los desenlaces de los pasos viajan en el `SagaResult`; `update_step_status` está ahí para el código que registra por sí mismo el progreso de los pasos.

En el arranque, `SagaRecoveryService` consulta

`persistence_port.get_stale(before)` para encontrar ejecuciones que empezaron pero nunca se completaron. Para cada saga obsoleta que sigue en estado `IN_FLIGHT`, marca la saga como `FAILED` y emite eventos de ciclo de vida para que los sistemas de observabilidad puedan alertar al equipo de guardia.

Configuración

```
pyfly:
  transactional:
    saga:
      persistence_enabled: true
      recovery_enabled: true
      recovery_interval_seconds: 60
      stale_threshold_seconds: 600
      cleanup_older_than_hours: 24
```

Con `recovery_enabled: true`, el framework ejecuta `SagaRecoveryService.recover_stale()` en una tarea en segundo plano cada `recovery_interval_seconds` segundos. Las sagas actualizadas por última vez hace más de `stale_threshold_seconds` segundos se consideran atascadas, se marcan como fallidas y se sacan a la superficie para investigación manual o reintento automático.

Implementar un adaptador de persistencia personalizado

Para persistir en una base de datos real, implementa `TransactionalPersistencePort` y registra tu implementación como un `@bean` o `@component`. Tu bean sustituye al `transactional_persistence_port` autoconfigurado:

lumen/infra/persistence/saga_postgres_adapter.py

```

from __future__ import annotations

from datetime import datetime
from typing import Any

from pyfly.container import component
from pyfly.transactional.shared.ports.outbound import (
    TransactionalPersistencePort,
)

@component
class SagaPostgresAdapter(TransactionalPersistencePort):

    async def persist_state(self, state: dict[str, Any]) → None:
        # INSERT INTO saga_executions ...
        ...

    async def get_state(
        self, correlation_id: str
    ) → dict[str, Any] | None:
        # SELECT * FROM saga_executions WHERE ...
        ...

    async def update_step_status(
        self,
        correlation_id: str,
        step_id: str,
        status: str,
    ) → None: ...

    async def mark_completed(
        self, correlation_id: str, successful: bool
    ) → None: ...

    async def get_in_flight(self) → list[dict[str, Any]]:
        return []

```

```

async def get_stale(
    self, before: datetime
) → list[dict[str, Any]]:
    return []

async def cleanup(self, older_than: datetime) → int:
    return 0

async def is_healthy(self) → bool:
    return True

```

Listado 12.11 — Esqueleto de un adaptador de persistencia para PostgreSQL

🔗 USA SAGARECOVERYSERVICE EN PRUEBAS DE INTEGRACIÓN

En pruebas que simulan una caída, crea un `SagaRecoveryService` con un `InMemoryPersistenceAdapter`, ejecuta una saga hasta un punto intermedio, márcala manualmente como obsoleta y luego llama a `await recovery.recover_stale(stale_threshold_seconds=0)`. Comprueba que `SagaResult.success` es `False` y que los pasos correctos están marcados como fallidos. Esto te da confianza en tu lógica de recuperación sin levantar una base de datos real.

El constructor programático de sagas

Cuando necesitas construir una saga a partir de configuración dinámica —cargando definiciones de paso desde una base de datos de reglas o un archivo de configuración — `SagaBuilder` te da la API fluida completa sin ningún decorador:

lumen/transfer/dynamic_saga.py

```

from __future__ import annotations

from pyfly.transactional.saga.registry.saga_builder import SagaBuilder
from pyfly.transactional.saga.core.result import SagaResult


async def debit_fn(req: object, ctx: object) → str:
    return "debit-ref-001"


async def capture_fn(req: object, ctx: object) → str:
    return "txn-001"


async def refund_fn(result: str) → None:
    pass    # undo debit


saga_def = (
    SagaBuilder("dynamic-transfer")
    .step("debit")
    .handler(debit_fn)
    .compensate(refund_fn)
    .retry(3)
    .backoff_ms(200)
    .timeout_ms(5_000)
    .jitter(enabled=True, factor=0.3)
    .add()
    .step("capture")
    .handler(capture_fn)
    .depends_on("debit")
    .retry(2)
    .backoff_ms(500)
    .add()
    .layer_concurrency(5)

```

```
.build()
)
```

Listado 12.12 — Construir una saga de forma programática con `SagaBuilder`

Cómo funciona: Cada llamada `.step(step_id)` devuelve un `StepBuilder`.

Encadena métodos de configuración

— `.handler()`, `.compensate()`, `.depends_on()`, `.retry()`, `.backoff_ms()`, `.timeout_ms()`, .

— y luego llama a `.add()` para finalizar el paso y devolver el `SagaBuilder` padre.

`.build()` ejecuta la misma validación del DAG que la vía de los decoradores: los manejadores que faltan, las referencias `depends_on` inexistentes y los ciclos lanzan todos `SagaValidationError` inmediatamente en el momento del registro.

✓ Lo que construiste

Empezaste con un problema concreto: una transferencia entre monederos a través de dos agregados independientes no puede usar una sola transacción de base de datos. Declaraste `MoneyTransferSaga` apilando `@saga` sobre `@service`, lo que hace que el `OrchestrationBeanPostProcessor` la registre en el `SagaEngine` autoconfigurado en el arranque. Cada paso usa instancias de marcador `Annotated[T, Input()]` y `Annotated[T, FromStep("step-id")]` —no clases desnudas— para la inyección de parámetros; `ctx: SagaContext` se inyecta por tipo. El método de compensación `recredit_source` no recibe la entrada de la saga; obtiene el `DebitResult` del paso hacia delante mediante `FromStep("debit-source")`. Cuando `credit-destination` lanza `AggregateNotFound`, el motor ejecuta automáticamente `recredit_source`, dejando ambos saldos sin cambios.

Exploraste la compensación en profundidad: cinco políticas que van desde la secuencial estricta hasta la paralela de mejor esfuerzo, los reintentos y tiempos de espera de compensación por paso, y el requisito innegociable de que todas las compensaciones sean idempotentes. Viste cómo `@workflow(id=...) @service` y `@wait_for_signal` suspenden un flujo de trabajo de larga duración hasta que un humano entrega una señal, y cómo `WorkflowResult.successful` informa del estado final. Recorraste TCC como una alternativa basada en reservas que bloquea recursos entre todos los participantes antes de confirmar ninguno. Por último, cableaste un `TransactionalPersistencePort` personalizado y configuraste `SagaRecoveryService` para detectar y sacar a la superficie ejecuciones obsoletas tras una caída.

Conceptos clave que llevarte:

- `@saga` **sobre** `@service` — la pila de decoradores que hace que una clase sea a la vez un bean de inyección de dependencias y una saga registrada; `@saga` por sí solo no basta.
- **Instancias de marcador** — `Input()`, `FromStep("step-id")` deben ser instancias (con paréntesis), no clases desnudas.
- **Compensación mediante `FromStep`** — los métodos de compensación reciben el resultado del paso hacia delante, nunca la entrada de la saga.
- `SagaEngine.execute(saga_name, input_data)` — la única llamada que devuelve `SagaResult` con `.success`, `.result_of()`, `.failed_steps()`, `.compensated_steps()`.
- `@workflow(id=...) @service` / `@wait_for_signal` — alternativa dirigida por señales y de larga duración; comprueba `WorkflowResult.successful` para el estado final.
- `@tcc` **sobre** `@service` / `@tcc_participant` — coordinación basada en reservas; `FromTry()` (instancia) inyecta el resultado del try en los métodos confirm y cancel.

- `TransactionalPersistencePort` — implementa y registra este protocolo para darle al motor estado duradero y recuperación ante caídas.

Pruébalo tú mismo

Ejercicio 1 — Validación de saldo en paralelo. Añade un paso `validate-source` a `MoneyTransferSaga` que compruebe que el monedero origen tiene fondos suficientes, sin realizar el cargo. El paso debería ejecutarse *en paralelo* con nada (sin `depends_on`), y `debit-source` debería depender de él. Extiende `credit-destination` para que dependa de `debit-source` como antes. Verifica la topología mediante `SagaRegistry.get("money-transfer")` en una prueba y comprueba que `definition.steps["debit-source"].depends_on == ["validate-source"]`.

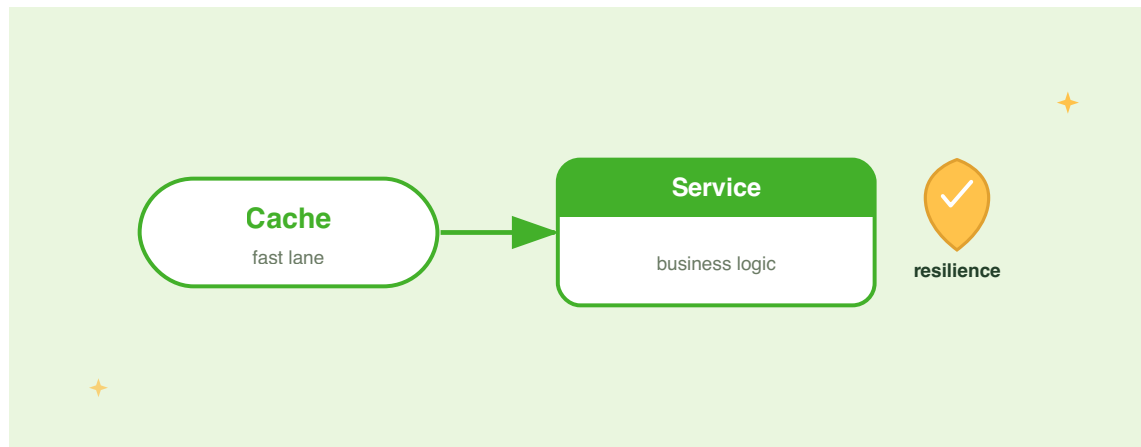
Ejercicio 2 — Manejador de errores de compensación. Cambia la política de compensación global de `MoneyTransferSaga` a `RETRY_WITH_BACKOFF` en `application.yaml`. Luego haz deliberadamente que `recredit_source` lance `RuntimeError` en la primera llamada y tenga éxito en la segunda. Escribe una prueba de pytest usando `AsyncMock` sobre `WalletRepository` que verifique que la saga acaba compensando con éxito y que `result.compensated_steps()` contiene `"debit-source"`.

Ejercicio 3 — Persistencia personalizada. Implementa `TransactionalPersistencePort` respaldado por un `dict` de Python plano que registre cada llamada. Regístralo como un `@service` y escribe una prueba que ejecute `TransferService`, luego llame a `get_state(correlation_id)` en tu adaptador y compruebe que el `status` registrado es `"COMPLETED"`. Extiende la prueba para

simular una saga obsoleta estableciendo manualmente `status = "IN_FLIGHT"` y un `started_at` pasado, y luego comprueba que `SagaRecoveryService.recover_stale(stale_threshold_seconds=0)` devuelve `1`.

CAPÍTULO 13

Caché y resiliencia



En el Capítulo 11 dividiste Lumen en servicios independientes y enseñaste a su manejador (handler) de monederos a llamar a un `AccountService` descendente a través de HTTP. En el Capítulo 12 añadiste una `DepositSaga` para coordinar operaciones de varios pasos a través de las fronteras de servicio, con transacciones de compensación listas para dispararse cuando cualquier paso fallara.

Esos dos capítulos introdujeron una nueva clase de problema: la latencia y la propagación de fallos. Cada salto HTTP a `AccountService` es un viaje de ida y vuelta que podría ser lento en una red congestionada, y cada llamada a la propia base de datos de Lumen compite con participantes de saga concurrentes. En un sistema distribuido, los fallos no son eventos excepcionales: son mantenimiento programado. `AccountService` se actualizará en plena carga de tráfico. Redis tendrá un tropiezo. Una pasarela de pago se disparará a tiempos de respuesta de tres segundos durante el pico de liquidación.

Sin protección, Lumen propaga esos fallos hacia arriba. Un `AccountService` lento atasca corrutinas, bloqueando lecturas de monederos para usuarios no relacionados. Una breve caída de Redis borra los saldos en caché y envía cada petición directamente a la base de datos, multiplicando la carga en el peor momento posible.

Este capítulo hace que Lumen sea **rápido** y **tolerante a fallos**. La primera mitad cubre la capa de caché declarativa de PyFly — `@cacheable`, `@cache_put` y `@cache_evict` — y muestra cómo respaldarlas con un `InMemoryCache` en proceso para desarrollo y un `RedisCacheAdapter` compartido con conmutación por error (failover) automática para producción. La segunda mitad incorpora el conjunto de herramientas de resiliencia: un **limitador de tasa** de cubo de tokens que limita el tráfico entrante, un **bulkhead** de semáforo que aísla la concurrencia, un **limitador de tiempo** que cancela corrutinas colgadas, un **fallback** que degrada con elegancia, y patrones de **reintento** y **cortacircuitos** (circuit breaker) que protegen las llamadas salientes. Una sección de cierre muestra cómo apilarlos todos en el orden correcto.

Al final del capítulo, cada ruta caliente de Lumen estará en caché y cada dependencia saliente estará envuelta en una valla de resiliencia.

❗ LO QUE CONSTRUIRÁS, EN TÉRMINOS SENCILLOS

Este capítulo introduce mucho vocabulario — *caché*, *cubo de tokens*, *bulkhead*, *cortacircuitos*. No dejes que la jerga te intimide. Cada una de estas es una herramienta pequeña y autocontenida que atornilla a una función con un único decorador. Presentaremos cada herramienta de una en una, la integraremos en Lumen paso a paso, la ejecutaremos y observaremos qué cambia. Al final tendrás una lista de comprobación mental: *¿es esta lectura caliente? cachéala; ¿va esta llamada por la red? ponle una valla*. La versión de PyFly usada a lo largo del libro es **v26.6.110** — cada comando y clave de configuración de abajo coincide con esa versión.

Cachear la ruta de lectura

¿Por qué cachear las lecturas de monedero?

NUEVO TÉRMINO: CACHÉ

Una *caché* es una pequeña y rápida zona de almacenamiento donde guardas la respuesta a una pregunta costosa para poder devolverla al instante la próxima vez que alguien la formule. La primera vez que Lumen calcula el saldo del monedero `w-001` almacena el resultado en la caché; las lecturas posteriores devuelven esa copia almacenada sin volver a ejecutar la consulta a la base de datos. La contrapartida — y siempre hay una contrapartida — es que la copia almacenada puede estar ligeramente desactualizada. El resto de esta sección trata de mantener esa obsolescencia dentro de límites aceptables.

La operación más frecuente de Lumen es la consulta de saldo: "¿cuál es el saldo actual del monedero `w-001` ?". Bajo carga normal esa consulta llega a la réplica de lectura. Bajo carga intensa compite con comandos de depósito, participantes de saga y escrituras de instantáneas. Un saldo en caché cuesta una búsqueda en Redis — un único viaje de ida y vuelta de red colocalizado — comparado con una consulta SQL completa que la réplica de lectura debe además analizar, planificar y ejecutar.

La economía es convincente, pero la caché introduce una preocupación de corrección: el saldo en caché puede ir por detrás del saldo confirmado hasta el límite del TTL. Para Lumen, un saldo obsoleto de cinco segundos es una contrapartida aceptable para el tráfico de consultas normal. Cuando un depósito se completa, el manejador invalida la entrada de caché inmediatamente, de modo que la siguiente lectura de saldo refleja el cambio. Las actualizaciones que pasan por la saga usan `@cache_put` para refrescar el valor en caché como efecto secundario de la escritura, eliminando cualquier ventana de obsolescencia visible.

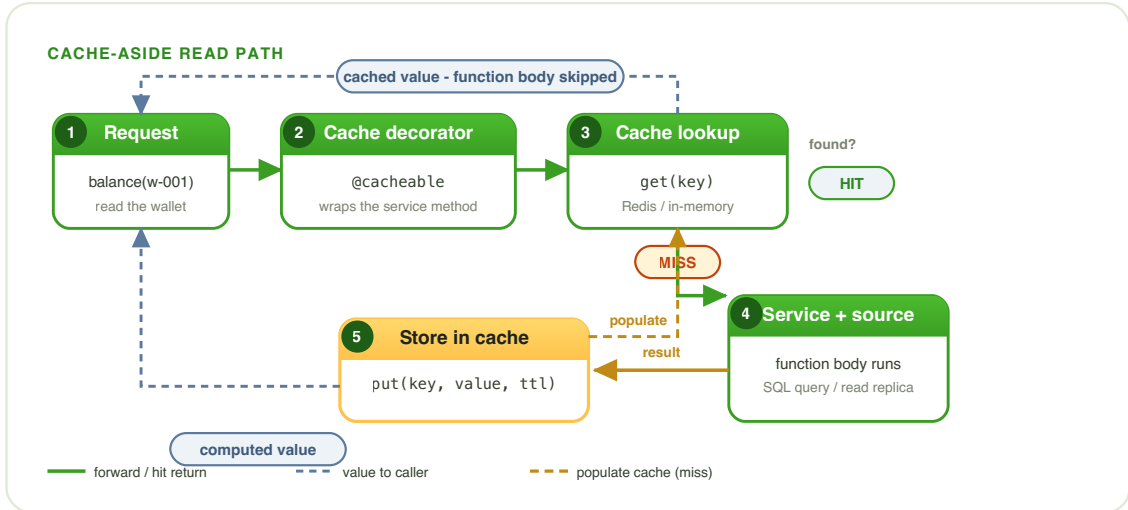


Figura 13.1 — Los decoradores de caché se sitúan delante de la capa de servicio. En un acierto el cuerpo de la función nunca se ejecuta; en un fallo se ejecuta y el resultado se almacena.

La abstracción de caché

La capa de caché de PyFly sigue el principio hexagonal que has visto a lo largo del libro: la lógica de negocio depende de un protocolo `CacheAdapter`, no de ningún backend específico. Las implementaciones concretas — `InMemoryCache` para desarrollo y `RedisCacheAdapter` para producción — se conectan a través del contenedor de inyección de dependencias. Cambiar de backend no requiere ningún cambio en la lógica de negocio.

El protocolo `CacheAdapter` define el contrato completo:

Método	Devuelve	Descripción
<code>get(key)</code>	<code>Any</code> <code>None</code>	Devuelve el valor en caché, o <code>None</code> si está ausente o ha expirado.
<code>put(key, value, ttl=None)</code>	<code>None</code>	Almacena un valor; <code>ttl</code> es un <code>timedelta</code> o <code>None</code> para que no expire.
<code>evict(key)</code>	<code>bool</code>	Elimina una clave; devuelve <code>True</code> si existía.
<code>exists(key)</code>	<code>bool</code>	Comprueba la presencia sin obtener el valor.
<code>clear()</code>	<code>None</code>	Vacía toda la caché.
<code>start()</code>	<code>None</code>	Se llama una vez al arrancar la aplicación.
<code>stop()</code>	<code>None</code>	Se llama una vez al apagar la aplicación.

Tanto `InMemoryCache` como `RedisCacheAdapter` implementan este contrato.

`InMemoryCache` almacena las entradas en un `OrderedDict` con expiración de TTL perezosa y acotación LRU opcional; es ideal para el desarrollo en un único proceso y para las suites de pruebas porque no tiene dependencias externas.

`RedisCacheAdapter` envuelve un cliente `redis.asyncio.Redis`, serializa los valores a JSON antes de almacenarlos y delega la gestión del TTL en el propio Redis — las claves expiradas desaparecen del lado del servidor sin sobrecarga de limpieza alguna por tu parte.

Configurar un backend de caché

Conectaremos dos backends: uno en proceso para desarrollo y uno compartido respaldado por Redis para producción. Tómalos de uno en uno.

Paso 1 — Elige un backend de desarrollo. Para desarrollo, lo único que necesitas es una sola importación:

lumen/cache/config_dev.py

```
from pyfly.cache.adapters.memory import InMemoryCache

wallet_cache = InMemoryCache(max_size=1000)
```

Listado 13.1 — *InMemoryCache* para desarrollo

`max_size=1000` acota la ventana de expulsión LRU: una vez que la caché contiene 1000 entradas, la entrada usada menos recientemente se descarta para hacer sitio. Pasa `None` (el valor por defecto) para dejar la caché sin límite y depender por completo de los TTL.

NUEVO TÉRMINO: LRU Y TTL

LRU significa *least-recently-used* (la usada menos recientemente) — cuando la caché está llena, la entrada que nadie ha tocado durante más tiempo es la que se expulsa para hacer sitio. *TTL* significa *time-to-live* (tiempo de vida) — cuánto tiempo permanece válida una entrada antes de expirar por sí sola. `InMemoryCache` admite ambos: `max_size` limita cuántas entradas contiene (LRU); cada `put` puede llevar un TTL que va envejeciendo la entrada hasta sacarla.

Paso 2 — Elige un backend de producción. Para producción, apunta `RedisCacheAdapter` a un cliente `redis.asyncio.Redis` y envuélvelo con un fallback en memoria para que un tropiezo de Redis nunca tumbe a Lumen:

lumen/cache/config_prod.py

```
import redis.asyncio as aioredis

from pyfly.cache import CacheAdapter, CacheManager
from pyfly.cache.adapters.memory import InMemoryCache
from pyfly.cache.adapters.redis import RedisCacheAdapter
from pyfly.container import bean, configuration

@configuration
class CacheConfig:

    @bean
    def wallet_cache(self) → CacheAdapter:
        client = aioredis.from_url("redis://localhost:6379/0")
        primary = RedisCacheAdapter(client)
        fallback = InMemoryCache(max_size=500)
        return CacheManager(primary=primary, fallback=fallback)
```

Listado 13.2 — RedisCacheAdapter para producción

Cómo funciona: `CacheManager` envuelve un backend primario de Redis y un fallback en memoria. Cada escritura va a ambas cachés, manteniendo el fallback caliente. En las lecturas, el gestor prueba primero con Redis; si Redis lanza una excepción registra un `WARNING` y recurre al almacén en proceso de forma silenciosa. Cuando Redis se recupera, las nuevas escrituras lo repueblan inmediatamente — sin intervención manual. El método `@bean` le indica al contenedor de inyección de dependencias de PyFly que cree un singleton y lo inyecte allá donde `CacheAdapter` se declare como dependencia.

Lo que acaba de pasar. Ahora tienes una interfaz `CacheAdapter` y dos formas de satisfacerla. En desarrollo le entregas al contenedor de inyección de dependencias un `InMemoryCache`; en producción le entregas un `CacheManager` que pone a Redis por

delante y recurre silenciosamente a la memoria cuando Redis es inalcanzable. Cada manejador en el resto de este capítulo pide `cache: CacheAdapter` en su constructor y nunca sabe ni le importa cuál de los dos recibió — esa es la recompensa hexagonal.

💡 AUTOCONFIGURACIÓN

No tienes por qué escribir la clase `@configuration` en absoluto. Añade lo siguiente a `pyfly.yaml` y la `CacheAutoConfiguration` de PyFly construye un bean `CacheAdapter` por ti al arrancar:

```
pyfly:
  cache:
    enabled: true          # required to switch the subsystem on
    provider: redis        # redis | postgres | memory | auto
    redis:
      url: redis://localhost:6379/0
    max-size: 1000        # used by the memory provider
```

Con `provider: redis` (o `auto`, que detecta un `redis.asyncio` instalado) la autoconfiguración conecta un `RedisCacheAdapter` apuntado a `pyfly.cache.redis.url`. Registra ese único adaptador como el bean `CacheAdapter` — **no** añade la capa de failover en memoria. Cuando quieras Redis *más* el fallback en proceso transparente que se muestra en el Listado 13.2, declara tú mismo el `CacheManager` en una clase `@configuration` como arriba. La autoconfiguración también se retira por completo si ya has definido tu propio bean `CacheAdapter` (`@conditional_on_missing_bean`), de modo que los dos enfoques nunca chocan.

@cacheable — saltarse la ejecución en un acierto

`@cacheable` es el decorador más común. En la primera llamada ejecuta el cuerpo de la función y almacena el valor de retorno. En cada llamada posterior con la misma clave devuelve el valor almacenado *sin ejecutar en absoluto el cuerpo de la función*.

El `GetBalanceHandler` de Lumen encaja de forma natural: las lecturas de saldo son frecuentes, baratas de cachear y toleran unos pocos segundos de obsolescencia. Le añadiremos caché en tres pequeños pasos.

Paso 1 — Acepta la caché. Añade un parámetro `cache: CacheAdapter` al constructor. El contenedor de inyección de dependencias de PyFly ve el tipo e inyecta el backend que hayas conectado en la sección anterior.

Paso 2 — Mueve el trabajo real a un método privado. Renombra el cuerpo que llega a la base de datos a `_fetch`. Esta es la función que la caché envolverá.

Paso 3 — Envuélvelo en el momento de la construcción. Dentro de `__init__`, establece `self.do_handle = cacheable(...)(self._fetch)`. Envolvemos dentro de `__init__` (en lugar de como un decorador `@cacheable` sobre el método) por una razón: el argumento `backend=cache` solo existe una vez que `cache` ha sido inyectado, y eso no ocurre hasta que se ejecuta `__init__`.

El manejador recibe `CacheAdapter` a través de su constructor — inyectado por PyFly — y envuelve `do_handle` en el momento de la construcción:

lumen/core/services/wallets/get_balance_handler.py

```
from datetime import timedelta

from lumen.core.mappers.wallet_mapper import entity_to_balance_dto
from lumen.core.services.wallets.get_balance_query import GetBalance
from lumen.interfaces.dtos.v1.balance_dto import BalanceDto
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.cache import CacheAdapter, cacheable
from pyfly.container import service
from pyfly.cqrs import QueryHandler, query_handler

@query_handler
@service
class GetBalanceHandler(QueryHandler[GetBalance, BalanceDto | None]):
    """Return a cached :class:`BalanceDto`; bypass the DB on a hit."""

    def __init__(
        self,
        repository: WalletRepository,
        cache: CacheAdapter,
    ) → None:
        super().__init__()
        self._repository = repository
        # Wrap do_handle at construction time so `cache` is in scope.
        self.do_handle = cacheable(
            backend=cache,
            key="wallet:balance:{query.wallet_id}",
            ttl=timedelta(seconds=5),
        )(self._fetch)

    async def _fetch(
        self, query: GetBalance
    ) → BalanceDto | None:
        entity = await self._repository.find_by_id(query.wallet_id)
        return entity_to_balance_dto(entity) if entity is not None else None
```

Listado 13.3 — @cacheable en GetBalanceHandler

❗ LA PLANTILLA DE CLAVE Y SELF

La plantilla `key` `"wallet:balance:{query.wallet_id}"` usa la sintaxis `str.format` de Python. PyFly enlaza los argumentos reales de la llamada con `inspect.signature(func).bind(*args, **kwargs)` y después llama a `key.format(**bound.arguments)`. Como `_fetch` se envuelve dentro de `__init__`, el primer argumento posicional es `query` — de modo que `{query.wallet_id}` se expande al id del monedero. Llamar con `GetBalance(wallet_id="wlt-001")` produce la clave de caché `"wallet:balance:wlt-001"`. La función de mapeo `entity_to_balance_dto` pasa por `Mapper.project` contra la interfaz `BalanceView` marcada con `@projection`, copiando solo los campos que la vista de saldo declara y calculando `balance` a partir de `balance_minor`.

`ttl=timedelta(seconds=5)` significa que la entrada de caché expira cinco segundos después de escribirse. Tras la expiración, la siguiente llamada vuelve a ejecutar el cuerpo de la función y refresca la entrada. Un TTL de `None` (el valor por defecto) significa que la entrada nunca expira — apropiado solo para datos verdaderamente inmutables.

Caché de nulos: Cuando la función devuelve `None`, PyFly aun así almacena la entrada y registra que la clave *existe*. Una llamada posterior encuentra la clave y devuelve `None` sin tocar la base de datos. Esto previene ataques de penetración de caché en los que un adversario inunda con peticiones de claves inexistentes, cada una de las cuales de otro modo se filtraría hasta la base de datos.

condition y unless: Tanto `@cache` como `@cacheable` aceptan predicados opcionales. `condition` es un invocable con la misma firma que la función decorada; si devuelve `False`, se omite la caché para esa llamada. `unless` es un invocable que recibe el *resultado*; si devuelve `True`, el resultado se devuelve pero no se almacena. Ambos son de solo palabra clave:

```
cacheable(  
    backend=cache,  
    key="wallet:balance:{query.wallet_id}",  
    ttl=timedelta(seconds=5),  
    condition=lambda query: not query.wallet_id.startswith("test-"),  
    unless=lambda result: result is None,  
) (self._fetch)
```

Ejecútalo — demuestra que la segunda lectura se salta la base de datos

La forma más limpia de *ver* un acierto de caché es una prueba unitaria que cuente cuántas veces se llama al repositorio. Usa un `InMemoryCache` real (sin necesidad de Redis) y un repositorio de prueba minúsculo:

tests/cache/test_get_balance_cache.py

```
from datetime import timedelta

import pytest

from pyfly.cache import cacheable
from pyfly.cache.adapters.memory import InMemoryCache

class _CountingRepo:
    """Stub repository that records how many times it is queried."""

    def __init__(self) → None:
        self.calls = 0

    async def find_by_id(self, wallet_id: str) → dict:
        self.calls += 1
        return {"wallet_id": wallet_id, "balance_minor": 500}

@pytest.mark.asyncio
async def test_second_read_is_a_cache_hit() → None:
    repo = _CountingRepo()
    cache = InMemoryCache(max_size=10)

    fetch = cacheable(
        backend=cache,
        key="wallet:balance:{wallet_id}",
        ttl=timedelta(seconds=5),
    )(repo.find_by_id)

    first = await fetch("wlt-001") # miss → runs the repo
    second = await fetch("wlt-001") # hit → repo NOT called again

    assert first == second
    assert repo.calls == 1 # the body ran exactly once
```

Listado 13.3a — Una prueba que demuestra que la segunda lectura es un acierto

Ejecuta solo esta prueba:

```
$ uv run --extra dev pytest tests/cache/test_get_balance_cache.py -q
.
[100%]
1 passed in 0.04s
```

El único `.` y el `1 passed` lo confirman: la segunda llamada devolvió el valor en caché y `repo.calls` se quedó en `1`, de modo que la base de datos se tocó exactamente una vez en dos lecturas. Eso es un acierto de caché, demostrado en lugar de afirmado en prosa.

Lo que acaba de pasar. Envolviste una función async sencilla con `cacheable`, la respaldaste con un `InMemoryCache` y confirmaste que claves idénticas cortocircuitan el cuerpo. En `GetBalanceHandler` la función envuelta es `_fetch` y el backend es el `CacheAdapter` inyectado, pero la mecánica es exactamente la que acabas de ejecutar.

🍃 EQUIVALENCIA CON SPRING

`@cacheable` refleja la `@Cacheable` de Spring. La plantilla `key` usa la sintaxis `str.format` de Python en lugar de SpEL, pero la semántica — saltarse en acierto, almacenar en fallo, `condition`, `unless` — es idéntica. `@cache` es un alias de más bajo nivel que se comporta igual; usa el nombre que mejor se lea en tu base de código.

@cache_put — ejecutar siempre, almacenar siempre

`@cacheable` es para lecturas: cortocircuita la función cuando la caché ya contiene un valor. `@cache_put` es para escrituras: *siempre* ejecuta la función y *siempre* almacena el resultado. Úsalo cuando la función es la fuente de la verdad — un manejador de comandos que modifica el monedero y debe mantener la caché actualizada.

`DepositFundsHandler` es el ejemplo canónico. Después de que un depósito tiene éxito, el nuevo saldo debe ser visible para la siguiente lectura sin esperar a que el TTL expire. El cableado refleja lo que hiciste para `@cacheable`, con un detalle crítico que vigilar:

Paso 1 — Acepta la caché en el constructor, exactamente como antes.

Paso 2 — Mueve la lógica de depósito a `_deposit` y conserva su decorador `@transactional()` para que la escritura siga confirmándose como una unidad de trabajo.

Paso 3 — Envuelve con `cache_put`, reutilizando la *misma* forma de clave. El manejador de depósitos debe escribir en la mismísima ranura de caché que el lector de saldos consulta. `@cacheable` usa `"wallet:balance:{query.wallet_id}"`; aquí el argumento se llama `command`, así que la plantilla es `"wallet:balance:{command.wallet_id}"`. Distintos nombres de parámetro, pero ambos resuelven a `wallet:balance:wlt-001`.

Envolver `do_handle` con `@cache_put` refresca la entrada de caché de forma atómica con la escritura:

lumen/core/services/wallets/deposit_funds_handler.py

```

from datetime import timedelta

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

from lumen.core.mappers.wallet_mapper import to_aggregate, to_entity
from lumen.core.services.wallets.deposit_funds_command import DepositFunds
from lumen.core.services.wallets.event_publishing import publish_domain_events
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.cache import CacheAdapter, cache_put
from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.data.relational.sqlalchemy import LockMode, transactional
from pyfly.domain import AggregateNotFound
from pyfly.eda import EventPublisher

@command_handler
@service
class DepositFundsHandler(CommandHandler[DepositFunds, int]):
    """Credit funds to an existing wallet; returns the new balance
    in minor units and refreshes the cached balance entry."""

    def __init__(
        self,
        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
        cache: CacheAdapter,
    ) → None:
        super().__init__()
        self._repository = repository
        self._events = events
        self._session_factory = session_factory
        # Wrap at construction time so `cache` is in scope.
        self.do_handle = cache_put(
            backend=cache,

```

```

        key="wallet:balance:{command.wallet_id}",
        ttl=timedelta(seconds=5),
    )(self._deposit)

    @transactional()
    async def _deposit(self, command: DepositFunds) → int:
        entity = await self._repository.find_by_id(
            command.wallet_id, lock=LockMode.PESSIMISTIC_WRITE
        )
        if entity is None:
            raise AggregateNotFound("Wallet", command.wallet_id)

        wallet = to_aggregate(entity)
        wallet.deposit(Money(amount=command.amount, currency=wallet.currency))
        await self._repository.upsert(to_entity(wallet))
        await publish_domain_events(self._events, wallet.clear_events())
        return wallet.balance.amount

```

Listado 13.4 — @cache_put refresca la caché en un depósito

Cómo funciona: `@cache_put` espera (await) a la función envuelta, después llama a `backend.put(resolved_key, result, ttl=ttl)`. Como la función siempre se ejecuta, el valor en caché tras un comando `DepositFunds` es el saldo recién confirmado — no una instantánea obsoleta anterior al depósito. `_deposit` se ejecuta dentro de `@transactional()`, de modo que la secuencia `find_by_id → to_aggregate → mutate → upsert` se confirma como una unidad de trabajo antes de que se refresque la caché. La siguiente lectura `@cacheable` en `GetBalanceHandler` recoge este valor fresco sin tocar la base de datos.

❗ LAS ESCRITURAS EN CACHÉ ESPERAN AL COMMIT

Desde la v26.09.08 los decoradores de caché pasan por `TransactionAwareCache`, la versión de PyFly del `TransactionAwareCacheDecorator` de Spring: un `put`, `evict` o `clear` hecho dentro de una unidad de trabajo se ejecuta cuando esa unidad ha hecho commit, y se descarta cuando se revierte, así que ninguna petición recibe un valor que nunca se confirmó. Aquí `@cache_put` envuelve todo el método `@transactional`, de modo que la unidad ya ha hecho commit cuando se ejecuta el `put`; envuelto dentro de una transacción mayor, el `put` esperaría al commit de esa transacción. Las lecturas y `put_if_absent` se ejecutan de inmediato, fuera de la unidad de quien llama.

❗ LA CLAVE DE CACHÉ DEBE COINCIDIR

La clave `@cache_put "wallet:balance:{command.wallet_id}"` debe coincidir con la clave `@cacheable "wallet:balance:{query.wallet_id}"` cuando ambas resuelven al mismo id de monedero. Claves que no coinciden significan que el depósito escribe en una ranura de caché distinta de la que consulta la lectura de saldo — la obsolescencia regresa.

Decorador	¿Se ejecuta la función?	En un acierto
<code>@cacheable</code> / <code>@cache</code>	Solo en un fallo	Devuelve el valor en caché
<code>@cache_put</code>	Siempre	Reemplaza el valor en caché con el resultado fresco

@cache_evict — eliminar tras un borrado

Cuando cierras un monedero o reviertes una transacción, la entrada de caché asociada debe eliminarse. `@cache_evict` ejecuta primero el cuerpo de la función y después elimina la clave indicada — o vacía toda la caché cuando `all_entries=True`.

lumen/core/services/wallets/close_wallet_handler.py

```
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.cache import CacheAdapter, cache_evict
from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.data.relational.sqlalchemy import transactional

@command_handler
@service
class CloseWalletHandler(CommandHandler["CloseWallet", None]):
    """Close a wallet and evict its cached balance entry."""

    def __init__(
        self,
        repository: WalletRepository,
        cache: CacheAdapter,
    ) → None:
        super().__init__()
        self._repository = repository
        self.do_handle = cache_evict(
            backend=cache,
            key="wallet:balance:{command.wallet_id}",
        )(self._close)

    @transactional()
    async def _close(self, command) → None:
        entity = await self._repository.find_by_id(command.wallet_id)
        if entity is not None:
            await self._repository.delete(entity)
```

Listado 13.5 — @cache_evict tras eliminar un monedero

Para vaciar de golpe cada saldo en caché — útil para un reinicio administrativo — pasa `all_entries=True`:

```
self.do_handle = cache_evict(  
    backend=cache,  
    all_entries=True,  
)(self._reset_all)
```

Cómo funciona: El cuerpo de la función se ejecuta primero —

`repository.delete(entity)` elimina la fila antes de la expulsión, de modo que un fallo no descarta prematuramente la entrada de caché. Después, o bien

`backend.evict(resolved_key)` elimina una clave o bien `backend.clear()` vacía todo. Con `CacheManager`, la expulsión se propaga tanto a la caché primaria como a la de fallback, de modo que no queda ninguna entrada obsoleta en ninguno de los dos niveles.

`all_entries=True` es un instrumento contundente reservado para reinicios administrativos. En el funcionamiento normal, prefiere la expulsión dirigida por clave.

Estrategia de invalidación

Una estrategia coherente empareja cada operación con el decorador correcto:

Operación	Decorador	Razón
Consulta <code>GetBalance</code>	<code>@cacheable</code>	Salta la BD en acierto; el TTL de 5 s acota la obsolescencia
Comando <code>DepositFunds</code>	<code>@cache_put</code>	Refresca la entrada de caché de forma atómica con la escritura
Comando <code>WithdrawFunds</code>	<code>@cache_put</code>	Igual — mantén caliente el saldo posterior a la retirada
Cerrar monedero	<code>@cache_evict</code>	Elimina la entrada; la siguiente lectura la reconstruye desde la BD
Truncado de admin	<code>@cache_evict(all_entries=True)</code>	Reinicio masivo; un vaciado completo de la caché es lo correcto

⚠ REQUISITO DE ASINCRONÍA

Los tres decoradores requieren que la función envuelta esté declarada `async`. Los adaptadores de caché son totalmente asíncronos (esperan con `await` las operaciones del backend), de modo que un objetivo síncrono fallará con un `TypeError` en el momento de la decoración — PyFly lanza el error inmediatamente para que cojas el error al arrancar en lugar de en tiempo de ejecución.

Patrones de resiliencia

Por qué importa la protección

NUEVO TÉRMINO: RESILIENCIA

Resiliencia aquí significa que el sistema sigue sirviendo las peticiones que *puede* servir incluso cuando una dependencia de la que depende está lenta, sobrecargada o caída. Las herramientas de esta sección no hacen que el servicio descendente sea más rápido — impiden que una dependencia enferma enferme a Lumen entero. Cada herramienta es, de nuevo, un decorador que apilas sobre la función que realiza la llamada arriesgada.

La caché hace rápido el camino feliz. Los patrones de resiliencia protegen a Lumen cuando el camino feliz no está disponible. Sin protección, un `AccountService` lento desencadena una cascada:

1. Las peticiones de los manejadores de monederos se acumulan, cada una esperando una respuesta HTTP.
2. El bucle de eventos `asyncio` de Lumen — de un solo hilo por defecto — procesa las tareas pendientes en orden; una acumulación de llamadas HTTP lentas retrasa cualquier otra operación.
3. La memoria y los descriptores de archivo abiertos suben a medida que las corrutinas se apilan.
4. Lumen se vuelve no disponible para peticiones que no tienen nada que ver con `AccountService`.

Cuatro patrones complementarios rompen esta cascada antes de que empiece:

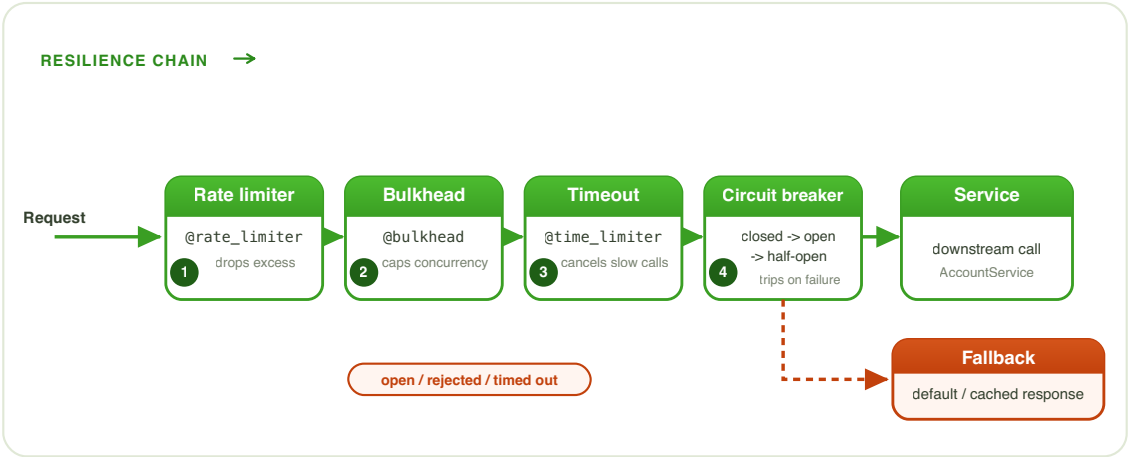


Figura 13.2 — Cuatro capas de resiliencia guardan la llamada saliente. El limitador de tasa descarta el tráfico excedente antes de que entre en el sistema; el bulkhead limita la concurrencia; el tiempo límite cancela las operaciones lentas; el fallback proporciona una respuesta segura cuando todo lo demás falla.

Patrón	Protege contra	¿Fallo rápido o espera?
Limitador de tasa	Picos de tráfico que abruman al servicio descendente	Fallo rápido (rechaza el exceso)
Bulkhead	Demasiadas llamadas concurrentes que atan recursos	Fallo rápido (rechaza por encima del límite)
Limitador de tiempo	Llamadas colgadas que nunca regresan	Cancela tras el tiempo límite
Fallback	Cualquier fallo que llegue al llamante	Devuelve un valor degradado

Los cuatro están en `pyfly.resilience` :

```
from pyfly.resilience import (
    RateLimiter, rate_limiter,
    Bulkhead, bulkhead,
    time_limiter,
    fallback,
)
```

Limitador de tasa — cubo de tokens

`RateLimiter` usa un **cubo de tokens**: el cubo contiene hasta `max_tokens` tokens y se rellena a `refill_rate` tokens por segundo. Cada llamada consume un token. Cuando el cubo está vacío, se lanza `RateLimitException` inmediatamente — sin cola, sin espera.

`lumen/resilience/rate_example.py`

```
from pyfly.resilience import RateLimiter, rate_limiter

# Sustained: 20 calls/s; burst: up to 40
account_limiter = RateLimiter(max_tokens=40, refill_rate=20.0)

@rate_limiter(account_limiter)
async def fetch_account(account_id: str) → dict:
    # This body is reached only when a token is available.
    ...
```

Listado 13.6 — Limitador de tasa de cubo de tokens en las búsquedas de cuentas

Cómo funciona: `@rate_limiter(limiter)` llama a `await limiter.acquire()` antes de cada invocación. `acquire()` rellena el cubo según el tiempo de reloj transcurrido (usando `time.monotonic()`), después comprueba y decrementa atómicamente el recuento de tokens bajo un `threading.Lock` — no un lock de asyncio — de modo que

tanto las tareas async como los llamantes síncronos comparten el mismo recuento sin condiciones de carrera. Si quedan menos de 1.0 tokens, `RateLimitException` se propaga al llamante.

La forma de cubo de tokens permite ráfagas controladas: un servicio que normalmente ve 10 llamadas por segundo puede absorber una ráfaga de 40 llamadas inmediatamente (recurriendo a tokens ahorrados) y después sostiene 20 llamadas por segundo a partir de ahí. Los limitadores de tasa de ventana fija no pueden expresar este matiz.

Ejecútalo — observa cómo el cubo se vacía

Un pequeño script hace concreto el comportamiento. Crea un limitador con un cubo diminuto, llama más allá de su capacidad y observa el rechazo:

scratch/rate_demo.py

```
import asyncio

from pyfly.kernel.exceptions import RateLimitException
from pyfly.resilience import RateLimiter, rate_limiter

# 3 tokens, refilling slowly so the burst is what we observe.
limiter = RateLimiter(max_tokens=3, refill_rate=1.0)

@rate_limiter(limiter)
async def ping(n: int) → str:
    return f"ok-{n}"

async def main() → None:
    for n in range(5):
        try:
            print(await ping(n))
        except RateLimitException:
            print(f"rejected-{n}")

asyncio.run(main())
```

Listado 13.6a — Vaciar el cubo de tokens a propósito

Ejecútalo directamente:

```
$ uv run python scratch/rate_demo.py
ok-0
ok-1
ok-2
rejected-3
rejected-4
```

Las primeras tres llamadas gastan cada una un token; la cuarta y la quinta llegan con un cubo vacío y se rechazan inmediatamente con `RateLimitException` — sin cola, sin espera. Ralentiza el bucle (o sube `refill_rate`) y los rechazos desaparecen porque los tokens se rellenan entre llamadas.

Lo que acaba de pasar. No cambiaste la función `ping` en absoluto — la decoraste. El decorador insertó un `await limiter.acquire()` antes de cada llamada, y `acquire()` lanzó cuando el cubo estaba vacío. Esta es la forma que adopta cada herramienta de resiliencia de este capítulo: un decorador que guarda la llamada sin que el cuerpo de la función sepa que existe.

Múltiples funciones que comparten una instancia `RateLimiter` imponen una tasa *global* a través de todas ellas — útil para limitar el tráfico total hacia un servicio descendente con independencia de qué método interno inicie la llamada.

Bulkhead — aislamiento de concurrencia

NUEVO TÉRMINO: BULKHEAD

El nombre viene de la construcción naval: el casco de un barco se divide en compartimentos estancos (*bulkheads*, mamparos) de modo que una brecha en uno no inunde toda la nave. Un bulkhead de software limita cuántas llamadas a una dependencia pueden ejecutarse a la vez, de modo que una avalancha de llamadas lentas a `AccountService` no pueda consumir cada corrutina y hundir peticiones no relacionadas.

`Bulkhead` es un semáforo: limita el número de llamadas *en vuelo al mismo tiempo*. Las llamadas que superen `max_concurrent` se rechazan inmediatamente con `BulkheadException`.

lumen/resilience/bulkhead_example.py

```
from pyfly.resilience import Bulkhead, bulkhead

# At most 5 concurrent calls to AccountService
account_bulkhead = Bulkhead(max_concurrent=5)

@bulkhead(account_bulkhead)
async def fetch_account(account_id: str) → dict:
    ...
```

Listado 13.7 — Bulkhead que limita las llamadas concurrentes al servicio de cuentas

Cómo funciona: El decorador adquiere un permiso (`_acquire_slot`) antes de entrar en la función y lo libera (`_release_slot`) en un bloque `finally` , de modo que la ranura siempre se devuelve incluso cuando la función lanza. Las ranuras se rastrean con un único contador entero protegido por lock compartido por las rutas de llamada async y síncrona, de modo que una instancia `Bulkhead` decora con seguridad una mezcla de corrutinas y funciones normales.

Este comportamiento de fallo rápido es intencionado: cuando hay 5 llamadas concurrentes en vuelo y llega una 6.^a, rechazarla inmediatamente permite al llamante reintentar o invocar un fallback — mucho mejor que ponerla en cola indefinidamente y provocar contrapresión en cascada.

💡 MONITORIZAR LA UTILIZACIÓN DEL BULKHEAD

`account_bulkhead.available_slots` devuelve el número de permisos libres en cualquier momento. Expón esto en un endpoint de salud o aliméntalo a tu pila de observabilidad para detectar la saturación persistente antes de que se convierta en una caída.

Limitador de tiempo — imponer una fecha límite

Un servicio descendente lento es a veces peor que uno caído: las llamadas que bloquean indefinidamente consumen recursos sin límite. `@time_limiter` cancela la corrutina si no se completa dentro de un `timedelta`:

`lumen/resilience/timeout_example.py`

```
from datetime import timedelta

from pyfly.resilience import time_limiter

@time_limiter(timeout=timedelta(seconds=2))
async def fetch_account(account_id: str) → dict:
    ...
```

Listado 13.8 — Fecha límite de 2 segundos en la búsqueda de cuenta

Cómo funciona: Internamente, `time_limiter` llama a `asyncio.wait_for(func(*args, **kwargs), timeout=timeout_seconds)`. Cuando pasa la fecha límite, `asyncio.wait_for` cancela la tarea subyacente, provocando que cualquier `await` dentro de la función lance `asyncio.CancelledError`. El decorador captura `TimeoutError` y lo relanza como `OperationTimeoutException` con un mensaje descriptivo:

```
OperationTimeoutException: fetch_account exceeded timeout of 2.0s
```

Los recursos adquiridos dentro de la función con límite de tiempo deberían guardarse con `try/finally` para que se liberen incluso en una cancelación:

```
@time_limiter(timeout=timedelta(seconds=2))
async def fetch_account(account_id: str) → dict:
    conn = await pool.acquire()
    try:
        return await conn.execute(query, account_id)
    finally:
        await pool.release(conn)
```

Fallback — degradación elegante

`@fallback` es la red de seguridad de la capa más externa: captura excepciones y devuelve una respuesta alternativa en lugar de propagar el error al llamante. El endpoint de resumen de saldo de Lumen puede devolver una respuesta degradada — el último saldo conocido, marcado como potencialmente obsoleto — en lugar de un HTTP 500 cuando `AccountService` está caído.

Hay dos modos disponibles. El primero devuelve un **valor estático**:

`lumen/resilience/fallback_static.py`

```
from pyfly.resilience import fallback

@fallback(fallback_value={"balance_minor": 0, "source": "fallback"})
async def fetch_account(account_id: str) → dict:
    ...
```

Listado 13.9 — Valor de fallback estático

El segundo invoca un **método de fallback** que recibe los argumentos originales más la excepción:

lumen/resilience/fallback_method.py

```
from pyfly.cache import CacheAdapter
from pyfly.resilience import fallback

_cache: CacheAdapter # injected elsewhere

async def account_from_cache(
    account_id: str,
    exc: Exception = None,
) → dict:
    cached = await _cache.get(f"account:{account_id}")
    if cached:
        return {**cached, "source": "cache"}
    return {"account_id": account_id, "balance_minor": 0, "source":
"fallback"}

@fallback(fallback_method=account_from_cache)
async def fetch_account(account_id: str) → dict:
    ...
```

Listado 13.10 — Método de fallback con datos en caché

Cómo funciona: Cuando la función primaria lanza uno de los tipos de excepción listados en `on` (por defecto: todas las subclases de `Exception`), el decorador llama a `fallback_method(*args, exc=exc, **kwargs)`. El argumento de palabra clave `exc` lleva la excepción capturada para que el fallback pueda registrarla, inspeccionar su tipo o devolver valores distintos para distintos modos de fallo. Si el método de fallback devuelve una corrutina, PyFly la espera con `await` automáticamente. Acota el filtro de excepciones con `on=(OperationTimeoutException, CircuitBreakerException)` para dejar que los errores de programación se propaguen con normalidad.

⚠ FIRMA DEL MÉTODO DE FALLBACK

El método de fallback debe aceptar `exc` como argumento de palabra clave. PyFly pasa la excepción capturada como `exc=<exception>`. Si la firma de tu método de fallback no incluye `exc`, verás un `TypeError` con un mensaje claro en el primer fallo — no en el momento de la decoración.

Reintento y cortacircuitos

@retry — reintentos acotados con backoff

Los errores de red a menudo son transitorios: se pierde un paquete, un pool de conexiones se agota momentáneamente, un pod descendente se reinicia. `@retry` reinvoca la función decorada hasta `max_attempts` veces con backoff exponencial entre intentos.

`max_attempts` es el único argumento posicional; todos los demás parámetros son de solo palabra clave:

lumen/resilience/retry_example.py

```
from pyfly.resilience import retry

@retry(
    max_attempts=3,
    delay=0.1,
    backoff=2.0,
    max_delay=2.0,
    exceptions=(IOError, TimeoutError),
)
async def fetch_account(account_id: str) → dict:
    ...
```

Listado 13.11 — Reintento con backoff exponencial

Cómo funciona: El decorador ejecuta la función, captura las excepciones que coinciden con `exceptions`, duerme `delay * backoff ** attempt` segundos (limitado a `max_delay`) y vuelve a intentar. En el último intento relanza la última excepción. La pausa usa `await asyncio.sleep(...)` para funciones async y `time.sleep(...)` para funciones síncronas — la misma implementación maneja ambas. El parámetro `jitter` añade aleatorización para evitar reintentos en estampida (thundering-herd) cuando muchas instancias se reinician simultáneamente.

Parámetro	Por defecto	Descripción
<code>max_attempts</code>	<code>3</code>	Total de intentos incluyendo el primero (≥ 1). Posicional.
<code>delay</code>	<code>0.0</code>	Pausa base en segundos antes del primer reintento. Solo palabra clave.
<code>backoff</code>	<code>1.0</code>	Multiplicador aplicado a <code>delay</code> en cada intento. Solo palabra clave.
<code>max_delay</code>	<code>None</code>	Tope de la pausa por intento. <code>None</code> significa sin tope. Solo palabra clave.
<code>jitter</code>	<code>0.0</code>	Fracción de aleatorización <code>[0, 1]</code> aplicada a cada espera. Solo palabra clave.
<code>exceptions</code>	<code>(Exception,)</code>	Tipos de excepción que disparan un reintento; los demás se propagan inmediatamente. Solo palabra clave.

⚠ LA IDEMPOTENCIA ES TU RESPONSABILIDAD

`@retry` llamará al cuerpo de la función varias veces. Si la operación no es idempotente — si llamarla dos veces tiene un efecto distinto de llamarla una vez — puedes aplicar cambios más de una vez. Los depósitos de monedero no son seguros de reintentar ingenuamente: reintentar un depósito fallido podría abonar la misma cantidad dos veces. Envuelve las operaciones no idempotentes en una comprobación de clave de idempotencia (almacena el ID de la operación antes de ejecutar; sáltala si el ID ya existe) o limita `exceptions` a errores que sean definitivamente previos a la ejecución (errores de conexión, tiempos de espera durante la fase de petición) en lugar de la ambigüedad posterior a la ejecución.

@circuit_breaker — fallo rápido bajo una caída sostenida

NUEVO TÉRMINO: CORTACIRCUITOS

Tomado prestado del cableado eléctrico: un cortacircuitos *salta* (se abre) cuando fluye demasiada corriente, cortando el circuito antes de que el cableado se sobrecaliente. Un cortacircuitos de software salta tras demasiados fallos, cortando las llamadas a una dependencia que falla para que dejes de machacarla — y para que tus propios llamantes fallen rápido en lugar de esperar en llamadas que de todos modos están condenadas a error.

Reintentar un servicio genuinamente no disponible amplifica la carga precisamente en el momento en que ese servicio más necesita alivio. El patrón cortacircuitos resuelve esto: tras un umbral de fallos consecutivos el circuito se **abre** y las llamadas posteriores se rechazan inmediatamente — sin intentar la llamada remota — hasta que transcurre un tiempo de espera de recuperación.

El cortacircuitos de PyFly tiene tres estados:

Estado	Comportamiento
CLOSED	Funcionamiento normal. Cada llamada pasa; los fallos se cuentan.
OPEN	Todas las llamadas lanzan <code>CircuitBreakerException</code> inmediatamente, sin E/S de red.
HALF_OPEN	Tras <code>recovery_timeout</code> segundos, se admite una llamada de sondeo limitada. Si tiene éxito el circuito se cierra; si falla el circuito se vuelve a abrir.

`@circuit_breaker` toma una **instancia** `CircuitBreaker` — no argumentos de palabra clave. Construye el `CircuitBreaker` por separado y pásalo:

lumen/resilience/cb_example.py

```
from pyfly.resilience import CircuitBreaker, circuit_breaker

account_cb = CircuitBreaker(
    failure_threshold=5,
    recovery_timeout=30.0,
    expected=(IOError, TimeoutError),
)

@circuit_breaker(account_cb)
async def fetch_account(account_id: str) → dict:
    ...
```

Listado 13.12 — Cortacircuitos alrededor de AccountService

Cómo funciona: Antes de cada llamada, `breaker.before_call()` comprueba el estado actual. Si está OPEN, lanza `CircuitBreakerException` inmediatamente. Si está HALF_OPEN y el presupuesto de sondeo está agotado, también lanza. En caso contrario la llamada procede. En caso de éxito, `breaker.on_success()` reinicia el contador de fallos consecutivos (o, en HALF_OPEN, cierra el circuito una vez que suficientes sondeos tienen éxito). En caso de fallo, `breaker.on_failure()` incrementa el contador y abre el circuito cuando se alcanza `failure_threshold`.

Solo las excepciones en `expected` disparan el cortacircuitos. Las excepciones de negocio — `ValueError`, `PermissionError` — se propagan con normalidad sin afectar al estado del circuito.

Parámetros del constructor de `CircuitBreaker` (`failure_rate_threshold`, `window_size` y `half_open_max_calls` son de solo palabra clave):

Parámetro	Por defecto	Descripción
<code>failure_threshold</code>	<code>5</code>	Fallos consecutivos que hacen saltar el circuito.
<code>recovery_timeout</code>	<code>30.0</code>	Segundos en OPEN antes de pasar a HALF_OPEN.
<code>expected</code>	<code>(Exception,)</code>	Tipos de excepción que cuentan como fallos.
<code>failure_rate_threshold</code>	<code>None</code>	Cambia al modo de tasa por ventana cuando se establece (p. ej. <code>0.5</code>).
<code>window_size</code>	<code>10</code>	Tamaño de la ventana de resultados para el salto basado en tasa.
<code>half_open_max_calls</code>	<code>1</code>	Llamadas de sondeo requeridas para cerrar desde HALF_OPEN.

Los parámetros `failure_rate_threshold` y `window_size` cambian del modo de recuento consecutivo al modo de tasa por ventana, igual que la ventana deslizante COUNT_BASED de Resilience4j. Establece `failure_rate_threshold=0.5` y `window_size=10` para abrir el circuito cuando más de la mitad de las últimas 10 llamadas fallen.

🌿 EQUIVALENCIA CON SPRING

`@retry` refleja la `@Retryable` de Spring Retry (con `maxAttempts`, `backoff`, `include`). `CircuitBreaker` refleja el `CircuitBreaker` de Resilience4j (umbral de fallos, tiempo de recuperación, máquina de estados CLOSED/OPEN/HALF_OPEN, llamadas de sondeo en half-open, filtro de excepciones esperadas). PyFly no usa la biblioteca Java de Resilience4j — es una reimplementación en Python puro con la misma semántica.

Configurar la resiliencia desde `pyfly.yaml`

Hasta ahora has construido cada `RateLimiter`, `Bulkhead` y `CircuitBreaker` en código. Eso es perfecto para una única pasarela, pero los equipos de operaciones suelen querer ajustar estos umbrales *sin un cambio de código* — subir un tiempo límite, ampliar un límite de tasa — y quieren un lugar evidente donde leer la configuración actual. PyFly v26.6.110 incluye un `ResilienceRegistry` dirigido por configuración para exactamente esto, dando paridad con el modelo de registro con nombre de Resilience4j.

Paso 1 — Declara instancias con nombre en `pyfly.yaml`. Cada entrada bajo `pyfly.resilience.*` se convierte en una instancia con nombre. Los nombres son tuyos a elegir; agrúpalos por el servicio descendente que protegen:

```
pyfly:
  resilience:
    circuit-breaker:
      account-api:
        failure-threshold: 5
        recovery-timeout: 30s
        # or switch to windowed-rate mode:
        # failure-rate-threshold: 0.5
        # window-size: 10
    rate-limiter:
      account-api:
        max-tokens: 50
        refill-rate: 20.0
    bulkhead:
      account-api:
        max-concurrent: 8
    time-limiter:
      account-api:
        timeout: 2s
```


Las duraciones aceptan sufijos amigables — `30s`, `500ms`, `1m`, `2h` — o un número desnudo leído como segundos. Las claves usan kebab-case (`failure-threshold`); el enlazado relajado de PyFly también acepta snake_case.

Paso 2 — Inyecta el registro y busca las instancias por nombre. La

`ResilienceAutoConfiguration` de PyFly registra un único bean `ResilienceRegistry` construido a partir de esas claves (siempre está activo, y devuelve un registro vacío cuando no hay claves presentes). Pídelo en cualquier constructor `@service`:

lumen/account/gateway_configured.py

```
from pyfly.container import service
from pyfly.resilience import (
    ResilienceRegistry,
    bulkhead,
    circuit_breaker,
    rate_limiter,
)

@service
class AccountGateway:

    def __init__(self, http_client, registry: ResilienceRegistry) → None:
        self._http = http_client
        # Look up the named instances declared in pyfly.yaml.
        cb = registry.circuit_breaker("account-api")
        rl = registry.rate_limiter("account-api")
        bh = registry.bulkhead("account-api")

        # Wrap the real call with the config-driven instances.
        guarded = circuit_breaker(cb)(self._raw_get)
        guarded = bulkhead(bh)(guarded)
        self.get_account = rate_limiter(rl)(guarded)

    async def _raw_get(self, account_id: str) → dict:
        resp = await self._http.get(f"/accounts/{account_id}")
        return resp.json()
```

Listado 13.12a — Obtener instancias de resiliencia del registro

Lo que acaba de pasar. Los umbrales ahora viven en la configuración, no en literales de Python. Un `CircuitBreaker` llamado `account-api` se materializa una vez al arrancar y se comparte por todo lo que lo busca — de modo que los recuentos de fallos y el estado OPEN/CLOSED son *globales* a través de todos los llamantes de ese nombre, exactamente como una instancia compartida en código. Buscar un nombre

desconocido lanza `KeyError` con la lista de nombres disponibles, de modo que una errata falla ruidosamente al arrancar en lugar de crear silenciosamente una ruta sin protección.

💡 EL LIMITADOR DE TIEMPO DEVUELVE UN `TIMEDELTA`

`registry.time_limiter("account-api")` devuelve el `timedelta` configurado, no un decorador — pásalo directamente a `time_limiter(timeout=registry.time_limiter("account-api"))`. Los otros tres accesorios (`circuit_breaker`, `rate_limiter`, `bulkhead`) devuelven la instancia que pasas al decorador correspondiente.

🍃 EQUIVALENCIA CON SPRING

El `ResilienceRegistry` refleja el `CircuitBreakerRegistry`, el `RateLimiterRegistry` y el `BulkheadRegistry` de `Resilience4j` — instancias con nombre declaradas en la configuración y buscadas en tiempo de ejecución. El `resilience4j.circuitbreaker.instances.<name>.*` de Spring Boot se convierte en `pyfly.resilience.circuit-breaker.<name>.*`; los nombres de las propiedades se alinean uno a uno.

Componer las capas

Orden de los decoradores

Los decoradores de resiliencia de `PyFly` se componen apilándose. Python aplica los decoradores de abajo arriba en el momento de la decoración pero los ejecuta de arriba abajo en el momento de la llamada. El orden recomendado, del más externo al más interno:

```
@fallback          ← 1. Catch any exception; return degraded response
@rate_limiter      ← 2. Reject excess traffic before it acquires resources
@bulkhead          ← 3. Limit concurrency of rate-limited calls
@time_limiter      ← 4. Cancel if execution takes too long
async def func()   ← 5. The actual operation
```

Este orden garantiza:

1. **Fallback** captura las excepciones de cada capa interna — incluyendo `RateLimitException`, `BulkheadException` y `OperationTimeoutException` — de modo que el llamante siempre recibe una respuesta utilizable.
2. **Limitador de tasa** descarta las peticiones excedentes antes de que consuman una ranura del bulkhead, evitando que una avalancha de tráfico agote el presupuesto de concurrencia.
3. **Bulkhead** limita cuántas llamadas permitidas por la tasa se ejecutan concurrentemente, protegiendo al servicio descendente de la sobrecarga.
4. **Limitador de tiempo** se aplica solo a la ejecución real; cuando se dispara, el bloque `finally` del bulkhead libera la ranura correctamente.

Añade `@retry` y `@circuit_breaker` en el lado más interno — envolviendo solo la llamada de E/S real — de modo que el fallback absorba sus excepciones y el limitador de tasa y el bulkhead contabilicen correctamente las llamadas reintentadas:

```
@fallback
@rate_limiter
@bulkhead
@time_limiter
@circuit_breaker(account_cb)
@retry(max_attempts=2, delay=0.05, backoff=2.0, exceptions=(IOError,))
async def fetch_account(account_id: str) → dict: ...
```

Con `@retry` por debajo de `@time_limiter`, el presupuesto de tiempo límite cubre toda la secuencia de reintentos, no cada intento individual. Para acotar cada intento de forma independiente, mueve `@time_limiter` por debajo de `@retry`.

Juntándolo todo — la pasarela de cuentas de Lumen

Aquí está el patrón completo ensamblado en una `AccountGateway` realista que los manejadores de monederos de Lumen usan para buscar información de cuentas:

lumen/account/gateway.py

```

from datetime import timedelta

from pyfly.cache import CacheAdapter, cacheable
from pyfly.container import service
from pyfly.kernel.exceptions import CircuitBreakerException,
OperationTimeoutException
from pyfly.resilience import (
    Bulkhead,
    CircuitBreaker,
    RateLimiter,
    bulkhead,
    circuit_breaker,
    fallback,
    rate_limiter,
    retry,
    time_limiter,
)

_limiter = RateLimiter(max_tokens=50, refill_rate=20.0)
_bh = Bulkhead(max_concurrent=8)
_cb = CircuitBreaker(
    failure_threshold=5,
    recovery_timeout=30.0,
    expected=(IOError, TimeoutError),
)

DEGRADED = {"status": "degraded", "balance_minor": None}

@service
class AccountGateway:

    def __init__(self, http_client, cache: CacheAdapter) → None:
        self._http = http_client
        self._cache = cache

    @cacheable(

```

```

        backend=None, # pass self._cache at runtime (see note below)
        key="account:{account_id}",
        ttl=timedelta(seconds=30),
    )
    @fallback(
        fallback_value=DEGRADED,
        on=(OperationTimeoutException, CircuitBreakerException, IOError),
    )
    @rate_limiter(_limiter)
    @bulkhead(_bh)
    @time_limiter(timeout=timedelta(seconds=2))
    @circuit_breaker(_cb)
    @retry(max_attempts=2, delay=0.05, backoff=2.0, exceptions=(IOError,))
    async def get_account(self, account_id: str) → dict:
        resp = await self._http.get(f"/accounts/{account_id}")
        return resp.json()

```

Listado 13.13 — AccountGateway con la pila de resiliencia completa

CABLEAR BACKEND EN UN MÉTODO DE CLASE

Como Python evalúa los decoradores del cuerpo de la clase antes de que se ejecute `__init__`, `self._cache` aún no está disponible ahí. El listado anterior pasa `backend=None` como marcador de posición para ilustrar el orden de apilamiento. En la práctica, envuelve `get_account` en `__init__` de la misma forma que en los ejemplos de manejador: `self.get_account = cacheable(backend=cache, key=..., ttl=...)(self._do_get_account)`. Como alternativa, usa una instancia `InMemoryCache` a nivel de módulo para las pruebas y cámbiala vía el contenedor de inyección de dependencias en producción.

Cómo fluye una llamada a través de las capas:

1. `@cacheable` comprueba la caché. En un acierto, cada capa interna se omite por completo.
2. En un fallo, `@fallback` se convierte en la red de seguridad más externa.

3. `@rate_limiter` comprueba el cubo de tokens; rechaza la llamada si está vacío.
4. `@bulkhead` comprueba el contador de permisos; rechaza si está a su capacidad.
5. `@time_limiter` establece una fecha límite de dos segundos para las capas de abajo.
6. `@circuit_breaker` rechaza inmediatamente si el circuito está OPEN.
7. `@retry` intenta la llamada HTTP hasta dos veces ante un `IOError`.
8. En caso de éxito, `@cacheable` almacena la respuesta durante 30 segundos.
9. Si un `IOError`, `OperationTimeoutException` o `CircuitBreakerException` escapa, `@fallback` lo captura y devuelve `DEGRADED`.

Fíjate en que `@cacheable` se sitúa *por encima* de `@fallback`. Eso significa:

- Una respuesta `DEGRADED` en caché de un ciclo de fallo anterior se devuelve tal cual durante hasta 30 segundos sin llegar a la red.
- Si no quieres cachear respuestas degradadas, mueve `@cacheable` por debajo de `@fallback`, o usa el predicado `unless`: `unless=lambda r: r.get("status") == "degraded"`.

Ejecútalo — haz que el servicio descendente falle y observa cómo la pila se degrada

No necesitas un `AccountService` en vivo para verificar la pila. Conecta las capas de resiliencia alrededor de un cliente HTTP de prueba que siempre lance, y verifica que el llamante aún obtiene una respuesta `DEGRADED` utilizable en lugar de una excepción:

tests/resilience/test_gateway_stack.py

```
import pytest

from pyfly.resilience import fallback, retry

DEGRADED = {"status": "degraded", "balance_minor": None}

class _BrokenClient:
    async def get(self, path: str) → dict:
        raise IOError("AccountService unreachable")

@fallback(fallback_value=DEGRADED, on=(IOError,))
@retry(max_attempts=2, delay=0.0, exceptions=(IOError,))
async def get_account(client: _BrokenClient, account_id: str) → dict:
    resp = await client.get(f"/accounts/{account_id}")
    return resp

@pytest.mark.asyncio
async def test_degrades_instead_of_raising() → None:
    result = await get_account(_BrokenClient(), "acc-1")
    assert result == DEGRADED
```

Listado 13.13a — La pila se degrada en lugar de lanzar

Ejecútalo:

```
$ uv run --extra dev pytest tests/resilience/test_gateway_stack.py -q
.
[100%]
1 passed in 0.05s
```

`@retry` intentó la llamada dos veces, ambos intentos lanzaron `IOException`, y `@fallback` capturó la excepción final y devolvió `DEGRADED`. El llamante nunca vio el error — exactamente el comportamiento que quieres cuando `AccountService` está teniendo un mal día.

Lo que acaba de pasar. Ensamblaste una porción de la pila completa — reintento por dentro, fallback por fuera — y demostraste que un fallo duro aflora como una respuesta degradada pero válida. Añade `@rate_limiter`, `@bulkhead`, `@time_limiter` y `@circuit_breaker` entre ellos en el orden mostrado arriba y cada uno se pliega en el mismo flujo: cada excepción que lanzan es capturada por el `@fallback` exterior, de modo que el llamante siempre recibe una respuesta que puede usar.

✓ Lo que construiste

Este capítulo cierra la Parte IV. En el Capítulo 11 dividiste Lumen en servicios independientes con clientes HTTP tipados. En el Capítulo 12 añadiste `DepositSaga` para coordinar operaciones de varios pasos con transacciones de compensación. Aquí hiciste que todo el sistema sea rápido y tolerante a fallos.

Concretamente, aprendiste:

- `@cacheable` cortocircuita las lecturas de saldo en un acierto de caché; el TTL de cinco segundos acota la obsolescencia a una ventana aceptable. Aplicado a `GetBalanceHandler` envolviendo `_fetch` en el momento de la construcción — `_fetch` llama a `repository.find_by_id` y proyecta el `WalletEntity` resultante sobre `BalanceDto` vía `entity_to_balance_dto` (`Mapper.project` + la `BalanceView` marcada con `@projection`).
- `@cache_put` refresca la caché como efecto secundario de cada comando `DepositFunds`. `_deposit` está decorado con `@transactional()`; hace

`find_by_id` → `to_aggregate` → `mutate` → `upsert` como una unidad de trabajo confirmada, y después actualiza la caché con el saldo devuelto. La plantilla de clave debe coincidir con la clave `@cacheable` para acertar en la misma ranura.

- `@cache_evict` elimina entradas al cerrar un monedero o en reinicios administrativos; `all_entries=True` vacía toda la caché en una sola llamada.
- `CacheManager` refleja las escrituras tanto en Redis (primario) como en `InMemoryCache` (fallback) y conmuta por error de forma transparente; es el valor por defecto correcto para cualquier despliegue de producción.
- `RateLimiter` + `@rate_limiter` limitan el tráfico entrante con un algoritmo de cubo de tokens que permite ráfagas controladas.
- `Bulkhead` + `@bulkhead` aíslan la concurrencia con un semáforo de fallo rápido que impide que una dependencia lenta consuma todos los recursos disponibles.
- `@time_limiter` impone fechas límite usando `asyncio.wait_for`, convirtiendo las llamadas colgadas indefinidamente en errores acotados `OperationTimeoutException`.
- `@fallback` proporciona una respuesta degradada pero funcional cuando todas las demás capas han fallado; el método de fallback recibe los argumentos originales y la excepción capturada vía el argumento de palabra clave `exc`.
- `@retry` toma `max_attempts` como su único argumento posicional; todos los demás parámetros (`delay`, `backoff`, `max_delay`, `jitter`, `exceptions`) son de solo palabra clave. Reinvoca operaciones un número acotado de veces con backoff exponencial.
- `@circuit_breaker` toma una **instancia** `CircuitBreaker` — no argumentos de palabra clave — y abre el circuito tras un umbral de fallos, cortocircuitando las llamadas posteriores durante la ventana de recuperación para que el servicio descendente tenga tiempo de recuperarse.

- **ResilienceRegistry** (PyFly v26.6.110) materializa instancias con nombre de **CircuitBreaker**, **RateLimiter**, **Bulkhead** y limitador de tiempo a partir de las claves de configuración `pyfly.resilience.*`, de modo que operaciones puede ajustar los umbrales en `pyfly.yaml` e inyectar el registro para buscar las instancias por nombre — paridad con el `resilience4j.*.instances.<name>.*` de Spring Boot.
- El **orden de los decoradores** importa: fallback el más externo, después limitador de tasa, bulkhead, limitador de tiempo, cortacircuitos y reintento el más interno — con la caché por encima del fallback para cachear incluso las respuestas degradadas.

Lumen es ahora un sistema multiservicio, coordinado por sagas, en caché y resiliente. La Parte V añade las preocupaciones finales de producción: observabilidad — métricas, trazas distribuidas y endpoints de salud — para que puedas ver exactamente qué está haciendo Lumen en producción.

Pruébalo tú mismo

Ejercicio 1 — Caché condicional. El manejador `GetBalance` se llama mucho más a menudo para los monederos activos que para los monederos de prueba. Añade `condition=lambda query: not query.wallet_id.startswith("test-")` a la llamada `cacheable(...)` dentro de `GetBalanceHandler.__init__` y verifica con una prueba unitaria que use `InMemoryCache` que las consultas para ids de monedero de prueba siempre llegan al repositorio.

Ejercicio 2 — Cortacircuitos con umbral basado en tasa. Reemplaza el cortacircuitos de recuento consecutivo en `AccountGateway` por uno basado en tasa: abre el circuito cuando al menos el 60 % de las últimas 20 llamadas fallen. Construye

```
CircuitBreaker(failure_rate_threshold=0.6, window_size=20,
recovery_timeout=60.0, expected=(IOError, TimeoutError))
```

. Dos sutilezas guían el diseño de la prueba. Primera, en modo de tasa el cortacircuitos permanece sin saltar hasta que la ventana está *llena* — requiere una ventana completa de 20 llamadas antes de juzgar la tasa — de modo que una ráfaga de fallos por sí sola nunca lo abre. Segunda, el cortacircuitos solo reevalúa su condición de salto en un *fallo* (un éxito nunca lo abre), de modo que la llamada que cruza el umbral debe ser ella misma una llamada fallida. Escribe una prueba que dispare 8 llamadas con éxito seguidas de 12 fallidas (20 llamadas en total = una ventana completa, terminando en un fallo). Verifica que el circuito permanece `CLOSED` hasta la llamada 19 (la ventana sigue siendo parcial), y luego `OPENS` en la 20.^a llamada, cuando la ventana se llena y la tasa de fallos alcanza exactamente $12 / 20 = 0,60$.

Ejercicio 3 — Expulsar por prefijo. Lumen a veces necesita invalidar todas las entradas de caché de un propietario de monedero dado (borrado RGPD). Añade un método `purge_owner(owner_id: str)` a un servicio de administración de monederos que llame a `backend.evict_by_prefix(f"wallet:balance:{owner_id}:")` directamente (sin un decorador), y escribe una prueba que prepoble tres claves de monedero para un propietario y una para otro, llame a `purge_owner` y verifique que solo las entradas del propietario objetivo han desaparecido.

Ejercicio 4 — Resiliencia dirigida por configuración. Mueve los umbrales codificados a mano de `AccountGateway` a `pyfly.yaml` bajo `pyfly.resilience.circuit-breaker.account-api`, `pyfly.resilience.rate-limiter.account-api` y `pyfly.resilience.bulkhead.account-api`. Inyecta `ResilienceRegistry` en la pasarela, busca las tres instancias por nombre y escribe una prueba que verifique que el `CircuitBreaker.failure_threshold` materializado coincide con el valor que estableciste en la configuración. Ten en cuenta que `ResilienceRegistry.from_config(...)` espera un `Config` de pyfly, no un dict plano

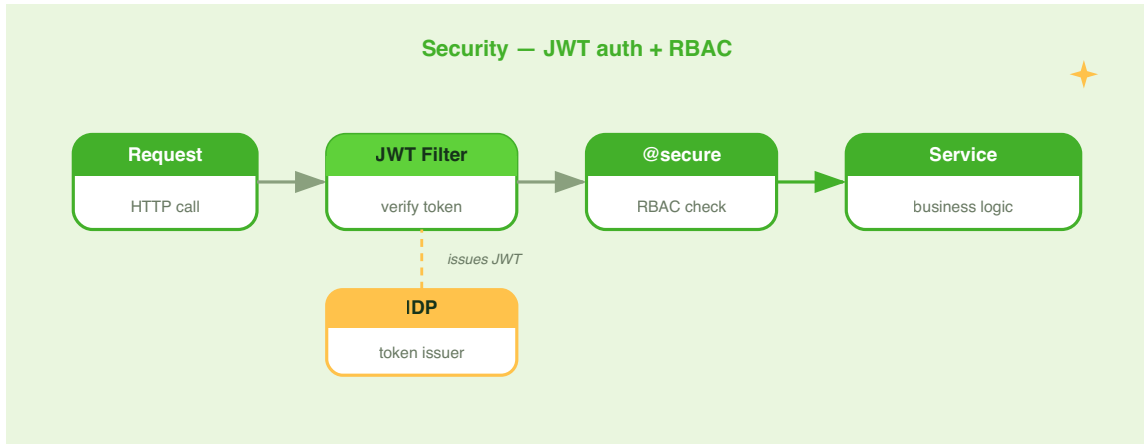
— llama a `config.get_section("pyfly.resilience.circuit-breaker")` internamente. Construye uno en la prueba a partir de un dict anidado, p. ej. `Config({"pyfly": {"resilience": {"circuit-breaker": {"account-api": {"failure-threshold": 5}}}}})` (importa `from pyfly.core.config import Config`), y después pasa ese `Config` a `from_config`. Confirma que buscar un nombre mal escrito lanza `KeyError` con la lista de nombres disponibles.

PARTE V

Asegurar, Observar y Desplegar

CAPÍTULO 14

Seguridad, sesiones e identidad



En el Capítulo 13 hiciste que Lumen fuera rápido y tolerante a fallos con caché y decoradores de resiliencia. La API de Lumen maneja ahora una alta concurrencia sin romperse bajo presión, pero está abierta de par en par. Cualquier llamante puede crear monederos, leer saldos o disparar depósitos. Antes de poder enviar las preocupaciones de producción restantes de la Parte V, necesitas cerrar esa puerta.

Este capítulo blindará Lumen. Vas a:

- **Autenticar** cada petición con un JWT firmado, usando `JWTService` para emitir y validar tokens y `SecurityMiddleware` para propagar el `SecurityContext` por todo el ámbito de la petición.
- **Autorizar** manejadores y comandos individuales con el decorador `@secure`, especificando roles, permisos o expresiones de seguridad completas.

- **Hashear contraseñas** de forma segura con `BcryptPasswordEncoder` para que tu almacén de usuarios nunca sea un pasivo.
- **Gestionar sesiones del lado del servidor** con `HttpSession`, un `SessionStore` conectable y un backend de Redis para escalado horizontal.
- **Federar la identidad** a un proveedor externo —Keycloak, AWS Cognito o Azure AD— a través del puerto `IdpAdapter` sin cambiar una sola línea de lógica de negocio.

Si has trabajado antes con Spring Security, la forma te resultará familiar: configura una cadena de filtros, anota métodos individuales y sustituye la fuente de detalles de usuario por un IDP. PyFly llama a esas piezas `SecurityMiddleware + HttpSecurity`, `@secure` e `IdpAdapter`, pero los conceptos se corresponden uno a uno.

Este capítulo está escrito como un tutorial guiado. Construimos la capa de seguridad pieza a pieza —emitir un token, validarlo, proteger un manejador, hashear una contraseña, almacenar una sesión, federar a un IDP— y después de cada pieza encontrarás un punto de control **Pruébalo** con el comando exacto que teclear y la salida que deberías esperar. Si nunca antes has conectado la autenticación a un servicio web, no pasa nada: cada término nuevo se glosa en lenguaje llano la primera vez que aparece, y puedes seguir el hilo editando el ejemplo de Lumen a medida que lees.

VERSIÓN

Los listados y las claves de configuración de este capítulo apuntan a PyFly **v26.6.110**. Si estás en una versión anterior, algunos nombres de propiedades difieren —sobre todo el puerto de la aplicación, que ahora es `pyfly.server.port` (el `server.port` de Spring), y el actuador y el panel de administración viven en un puerto de gestión separado, `pyfly.management.server.port` (por defecto `9090`).

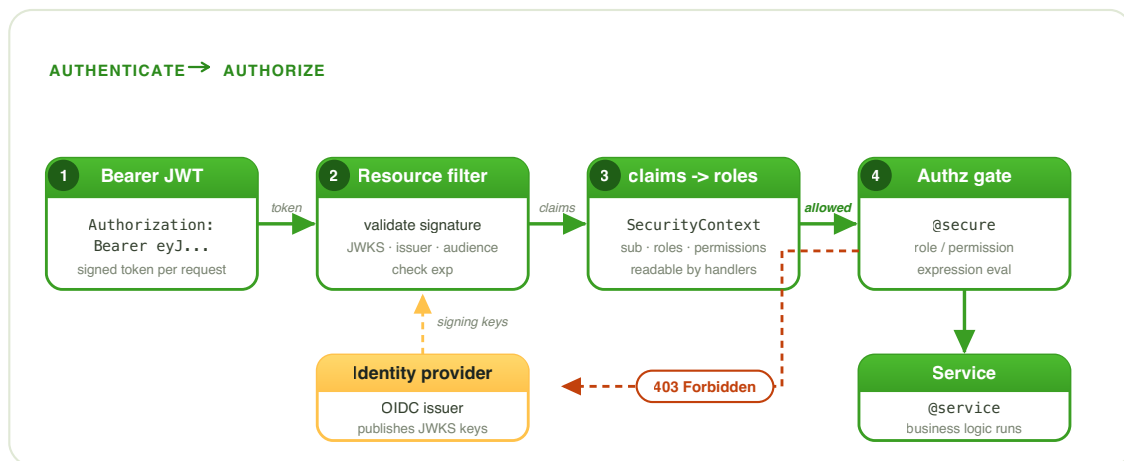


Figura 14.1 — Las capas de seguridad de Lumen. Un filtro JWT rellena el SecurityContext; HttpSecurity impone reglas a nivel de URL; @secure impone reglas a nivel de manejador; el puerto IDP delega la identidad a un proveedor externo.

Autenticación con JWT

¿Por qué JSON Web Tokens?

Lumen es una API sin estado. Las sesiones HTTP requerirían enrutamiento adhesivo (sticky) o un almacén de sesiones compartido en cada réplica. Los tokens JWT permiten que cada servicio valide las credenciales de forma independiente: sin estado compartido, sin coordinación, escalado horizontal por defecto.

Un **JWT** (JSON Web Token, que se pronuncia "yot") es una carga JSON firmada. Piénsalo como una credencial a prueba de manipulaciones: el servidor estampa un pequeño documento JSON —*quién eres, qué roles tienes, cuándo caduca*— y lo firma con una clave secreta. La credencial viaja con cada petición. Como la firma solo puede producirla alguien que posea el secreto, el servidor puede confiar en la credencial sin consultar nada en una base de datos. El servicio de autenticación de Lumen emite un

token al iniciar sesión; cada petición posterior lleva ese token en la cabecera

`Authorization` ; `SecurityMiddleware` valida la firma y desempaqueta el token en un `SecurityContext` que el resto de la petición puede leer.

JWTService

`JWTService` envuelve PyJWT con tres operaciones bien definidas:

Método	Descripción
<code>encode(payload)</code>	Firma un diccionario de carga, añadiendo <code>exp</code> si falta
<code>decode(token)</code>	Valida la firma + <code>exp</code> ; lanza <code>SecurityException</code> si falla
<code>to_security_context(token)</code>	Decodifica y extrae <code>sub</code> , <code>roles</code> , <code>permissions</code> en un <code>SecurityContext</code>

El servicio siempre exige un claim `exp` : un token sin caducidad se rechaza en el momento de decodificar. Ese invariante significa que todo token en circulación tiene un tiempo de vida acotado.

```
lumen/core/services/auth/auth_service.py
```

```
from pyfly.container import service
from pyfly.kernel.exceptions import UnauthorizedException
from pyfly.security import BcryptPasswordEncoder, JWTService, SecurityContext

@service
class AuthService:

    def __init__(
        self,
        jwt: JWTService,
        encoder: BcryptPasswordEncoder,
        user_repo,
    ) → None:
        self._jwt = jwt
        self._encoder = encoder
        self._users = user_repo

    async def login(self, username: str, password: str) → str:
        user = await self._users.find_by_username(username)
        if user is None or not self._encoder.verify(
            password, user.password_hash
        ):
            raise UnauthorizedException(
                "Invalid credentials", code="INVALID_CREDENTIALS"
            )
        # encode() auto-appends exp (default: 3 600 s from now)
        return self._jwt.encode({
            "sub": str(user.id),
            "roles": [user.role],
            "permissions": _permissions_for(user.role),
        })

    async def me(self, ctx: SecurityContext) → dict:
        user = await self._users.find_by_id(ctx.user_id)
        return {
            "id": str(user.id),
```

```

        "username": user.username,
        "role": user.role,
    }

def _permissions_for(role: str) → list[str]:
    MAP = {
        "USER": ["wallet:read", "wallet:deposit"],
        "ADMIN": [
            "wallet:read", "wallet:deposit",
            "wallet:create", "wallet:delete",
            "user:read", "user:write",
        ],
    }
    return MAP.get(role, [])

```

Listado 14.1 — Emitir un JWT en un inicio de sesión correcto

...

Cómo funciona. `login` obtiene el registro del usuario y llama a `BcryptPasswordEncoder.verify` para comparar la contraseña suministrada con el hash almacenado. Si tiene éxito, llama a `jwt.encode`, que añade automáticamente un claim `exp` a `expiration_seconds` segundos a partir de ahora (por defecto `3600`, una hora). Nunca importas `datetime`: el servicio calcula la caducidad como marca de tiempo Unix con `int(time.time()) + expiration_seconds`. El llamante recibe una cadena de token compacta y autocontenida.

Recorramos el método `login` paso a paso:

Paso 1 — Buscar al usuario. `find_by_username` devuelve el registro de usuario almacenado, o `None` si no existe ese nombre de usuario. Trataremos tanto "no existe ese usuario" como "contraseña incorrecta" como el mismo fallo, para que un atacante no pueda saber qué nombres de usuario están registrados.

Paso 2 — Verificar la contraseña. `self._encoder.verify(password, user.password_hash)` vuelve a hashear la contraseña suministrada con la sal incorporada en el hash almacenado y compara ambos en tiempo constante. Si no se encontró al usuario, o las contraseñas no coinciden, lanza `UnauthorizedException`: PyFly lo renderiza como un `401` con el código legible por máquina `INVALID_CREDENTIALS`.

Paso 3 — Construir los claims. Si tiene éxito, ensambla la carga JSON que se convertirá en el cuerpo del token: `sub` (el sujeto, el id del usuario), `roles` y los `permissions` que ese rol concede.

Paso 4 — Firmar y devolver. `self._jwt.encode({...})` firma la carga con el secreto configurado, estampa el claim obligatorio `exp` y devuelve la cadena de token compacta. Esa cadena es lo que el cliente almacena y reenvía en cada petición posterior.

JERGA: CLAIM

Un *claim* no es más que una afirmación clave/valor dentro del cuerpo JSON del token: `"sub": "42"` afirma que el sujeto es el usuario 42. El token lleva un saco de claims; el framework copia los relevantes para la seguridad (`sub` , `roles` , `permissions`) en el `SecurityContext`.

🔗 PRUÉBALO — EMITE UN TOKEN EN EL REPL

No necesitas un servidor en marcha para ver `JWTService` en acción. Con el entorno virtual de Lumen activo, abre un REPL de Python y firma una carga:

```
>>> from pyfly.security import JWTService
>>> jwt = JWTService(secret="dev-secret-change-me", algorithm="HS256")
>>> token = jwt.encode({"sub": "42", "roles": ["USER"]})
>>> token[:20]           # a compact "header.payload.signature" string
'eyJhbGciOiJIUzI1NiIs'
>>> ctx = jwt.to_security_context(token)
>>> ctx.user_id, ctx.roles
('42', ['USER'])
```

El token hace ida y vuelta: `encode` añadió el claim `exp` por ti, y `to_security_context` validó la firma y desempaquetó los claims en un `SecurityContext`. Prueba a manipular un carácter en mitad de la cadena y llama de nuevo a `jwt.decode(token)`: obtendrás una `SecurityException`, porque la firma ya no coincide con la carga.

El SecurityContext

`SecurityContext` es una dataclass inmutable que transporta los datos de autenticación y autorización de una única petición. El middleware lo crea a partir del token validado; tus manejadores lo reciben como un parámetro inyectado.

Campo	Tipo	Descripción
<code>user_id</code>	<code>str</code> <code>None</code>	Id del usuario autenticado; <code>None</code> si es anónimo
<code>roles</code>	<code>list[str]</code>	Roles concedidos en el token
<code>permissions</code>	<code>list[str]</code>	Permisos de grano fino
<code>attributes</code>	<code>dict[str, str]</code>	Claims adicionales (departamento, tenant, ...)

Métodos clave:

Método / Propiedad	Devuelve	Descripción
<code>is_authenticated</code>	<code>bool</code>	<code>True</code> cuando <code>user_id</code> no es <code>None</code>
<code>has_role(role)</code>	<code>bool</code>	Coincidencia exacta con la lista <code>roles</code>
<code>has_any_role(roles)</code>	<code>bool</code>	Intersección de conjuntos: cualquiera de los roles listados
<code>has_permission(perm)</code>	<code>bool</code>	Coincidencia exacta con la lista <code>permissions</code>
<code>SecurityContext.anonymous()</code>	<code>SecurityContext</code>	Crea un contexto no autenticado

El filtro de seguridad

`SecurityMiddleware` (ubicación canónica

`pyfly.web.adapters.starlette.security_middleware`, reexportado desde `pyfly.security`) se sitúa en la capa de middleware de Starlette. Para cada petición:

1. Comprueba si la ruta está en `exclude_paths`; si es así, establece un contexto anónimo y continúa.
2. Lee la cabecera `Authorization` y elimina el prefijo `Bearer`.
3. Llama a `jwt_service.to_security_context(token)`.
4. Si tiene éxito, almacena el contexto autenticado en `request.state.security_context`.
5. Ante cualquier `SecurityException` (token caducado, manipulado o sin `exp`), registra a nivel DEBUG y establece en su lugar un contexto anónimo.

El middleware **nunca rechaza peticiones**: el rechazo es trabajo de `@secure` y `HttpSecurity`. Un endpoint que requiere al usuario puede imponerlo; un endpoint de comprobación de salud puede ignorar el contexto por completo.

`lumen/app.py`

```
from pyfly.security import JWTService, SecurityMiddleware
from pyfly.web.adapters.starlette import create_app

def build_app(context):
    app = create_app(title="Lumen", context=context)

    jwt = context.get_bean(JWTService)
    app.add_middleware(
        SecurityMiddleware,
        jwt_service=jwt,
        exclude_paths=[
            "/docs",
            "/openapi.json",
            "/api/auth/login",
            "/api/auth/register",
        ],
    )
    return app
```

Listado 14.2 — Añadir el middleware de seguridad

...

Cómo funciona. `exclude_paths` enumera las rutas donde no se espera ningún token. El inicio de sesión y el registro no pueden requerir autenticación porque el token aún no existe; las rutas de la documentación se excluyen para que el explorador de la API funcione sin credenciales. Cualquier otra ruta pasa por la validación del token.

i JERGA: MIDDLEWARE

El *middleware* es código que envuelve cada petición, ejecutándose antes del manejador de la ruta a la entrada y después de él a la salida. `SecurityMiddleware` usa la pasada de "entrada" para leer el token y guardar un `SecurityContext` en `request.state` para que los manejadores posteriores puedan leerlo sin volver a parsear la cabecera.

Lo que acaba de ocurrir. Ya tienes conectadas las dos mitades de la autenticación.

`AuthService.login` *acuña* un token tras comprobar la contraseña;

`SecurityMiddleware` *lee* ese token en cada petición posterior y lo convierte en un

`SecurityContext`. Es crucial que el middleware sea permisivo: nunca devuelve un

`401`. Una petición con un token incorrecto o ausente simplemente llega como

anónima, y la decisión de permitirla o rechazarla se delega a las dos capas siguientes

que construirás: `HttpSecurity` (amplia, a nivel de URL) y `@secure` (precisa, por

manejador). Separar *quién eres* (autenticación) de *qué puedes hacer* (autorización) es

lo que mantiene cada capa pequeña y testeable.

Reglas a nivel de URL con `HttpSecurity`

`@secure` protege métodos manejadores individuales. `HttpSecurity` protege

subárboles de URL completos en la capa de filtro, antes de que se ejecute el

despachador de rutas. Las dos son complementarias: `HttpSecurity` aporta una

política rápida y amplia en el borde; `@secure` añade una imposición de grano fino, por

manejador, detrás de ella.

lumen/config/security_config.py

```

from pyfly.container import bean, configuration
from pyfly.security.http_security import HttpSecurity

@configuration
class SecurityConfig:

    @bean
    def http_security(self) → HttpSecurity:
        hs = HttpSecurity()
        hs.authorize_requests() \
            .request_matchers("/idp/admin/**").has_role("ADMIN") \
            .request_matchers("/api/v1/wallets/**").authenticated() \
            .request_matchers(
                "/health", "/docs", "/openapi.json",
                "/idp/login", "/idp/refresh",
            ).permit_all() \
            .any_request().permit_all()
        return hs

```

Listado 14.3 — El DSL de HttpSecurity

...

Cómo funciona. Las reglas se evalúan en el orden de declaración: gana la primera coincidencia. El `HttpSecurityFilter` se ejecuta en `HIGHEST_PRECEDENCE + 350`, después de que los filtros de autenticación hayan rellenado `request.state.security_context`, de modo que toda comprobación de rol y permiso dispone de un contexto totalmente hidratado para inspeccionar. Los métodos terminales — `has_role`, `has_any_role`, `has_permission`, `authenticated`, `permit_all` y `deny_all` — cubren toda política habitual; las reglas no satisfechas devuelven JSON de detalle de problema RFC 7807 (`application/problem+json`) con el estado HTTP apropiado.

Lee la cadena del DSL de arriba abajo: ese es exactamente el orden en que el filtro la evalúa:

Paso 1 — Abre la lista de reglas. `hs.authorize_requests()` inicia el constructor fluido. Cada llamada `.request_matchers(...)` que sigue registra una regla.

Paso 2 — Blinda el subárbol de administración. `.request_matchers("/idp/admin/**").has_role("ADMIN")` exige el rol `ADMIN` para cualquier cosa bajo `/idp/admin`. El glob `**` coincide con cualquier profundidad de segmentos de ruta.

Paso 3 — Exige inicio de sesión para los monederos. `.request_matchers("/api/v1/wallets/**").authenticated()` acepta a cualquier llamante con sesión iniciada, sin importar el rol, para el árbol de monederos. Las reglas más finas por manejador llegan después vía `@secure`.

Paso 4 — Permite las rutas públicas. `.request_matchers("/health", "/docs", ...).permit_all()` deja pasar sin token las comprobaciones de salud, el explorador de la documentación y los endpoints de inicio de sesión/refresco del IDP.

Paso 5 — Establece el valor por defecto. `.any_request().permit_all()` es el comodín para las rutas que ninguna regla anterior haya cubierto. Cámbialo a `.deny_all()` una vez que toda ruta esté explícitamente contemplada: "denegar por defecto" es la postura de producción más segura.

Paso 6 — Devuelve el constructor. Devuelve el propio `HttpSecurity` configurado; **no** llames aquí a `hs.build()`. Como `SecurityConfig` es una clase `@configuration`, el `HttpSecurityFilterAutoConfiguration` de la autoconfiguración (activo siempre que exista un bean `HttpSecurity`) recoge el bean `HttpSecurity`, llama a `.build()` por ti y registra en la cadena el `HttpSecurityFilter` resultante.

🔗 PRUÉBALO — OBSERVA CÓMO LA PUERTA ACEPTA Y RECHAZA

Arranca la aplicación con `uv run pyfly run` (sirve en `pyfly.server.port`, por defecto `8080`). En otra terminal, accede a una ruta protegida sin token:

```
curl -i http://localhost:8080/api/v1/wallets
```

Esperado — la puerta rechaza la petición anónima con un cuerpo RFC 7807:

```
HTTP/1.1 401 Unauthorized
content-type: application/problem+json

{"type":"about:blank","title":"Unauthorized","status":401,
 "detail":"Authentication is required to access this resource.",
 "instance":"/api/v1/wallets"}
```

Ahora reenvíala con un token (usa el `$TOKEN` que acuñaste en el REPL de arriba, o uno de `/idp/login`):

```
curl -i -H "Authorization: Bearer $TOKEN" http://localhost:8080/api/v1/
wallets
```

Esperado — `HTTP/1.1 200 OK` y una página JSON de monederos. La misma URL, dos resultados, decididos enteramente por el token: eso es la puerta haciendo su trabajo.

📖 DEFENSA EN DOS CAPAS

`HttpSecurity` aporta una política rápida a nivel de URL antes incluso de que las rutas se despachen, ideal para reglas generales como "todo lo que esté bajo `/api/v1/wallets` necesita autenticación". Los decoradores `@secure` sobre métodos individuales son la segunda capa, de grano más fino. Usa ambos juntos para una defensa en profundidad.

🍃 EQUIVALENCIA CON SPRING

`HttpSecurity` refleja la cadena `HttpSecurity.authorizeHttpRequests()` de Spring Security. `request_matchers` se corresponde con `requestMatchers`, `authenticated()` con `.authenticated()`, `has_role` con `hasRole`, y el `build()` que la autoconfiguración llama sobre tu bean `HttpSecurity` finaliza el filtro subyacente igual que `build()` finaliza la cadena de filtros de Spring. Los patrones glob de `fnmatch` (`/api/admin/**`) se comportan de forma idéntica a la coincidencia de rutas estilo Ant de Spring.

Autoconfiguración

No necesitas registrar `JWTService` ni `BcryptPasswordEncoder` manualmente. Añade las propiedades pertinentes a `pyfly.yaml` y la autoconfiguración lo conecta todo:

`lumen/resources/pyfly.yaml`

```
pyfly:
  security:
    enabled: true
    jwt:
      secret: "${JWT_SECRET}"
      algorithm: HS256
      filter:
        enabled: true
      exclude-patterns: >-
        /docs,/openapi.json,
        /api/auth/login,/api/auth/register
    password:
      bcrypt-rounds: 12
```

Listado 14.4 — Autoconfiguración de seguridad

...

Propiedad	Por defecto	Descripción
<code>pyfly.security.jwt.secret</code>	<code>change-me-in-production</code>	Clave de firma HMAC; debe sobreescribirse
<code>pyfly.security.jwt.algorithm</code>	<code>HS256</code>	Algoritmo de firma
<code>pyfly.security.jwt.filter.enabled</code>	<i>(ausente)</i>	Pon <code>true</code> para registrar el bean <code>SecurityFilter</code> automáticamente
<code>pyfly.security.jwt.exclude-patterns</code>	<i>(ausente)</i>	Rutas separadas por comas que se saltan
<code>pyfly.security.password.bcrypt-rounds</code>	<code>12</code>	Factor de coste de Bcrypt

Fíjate dónde vive cada clave: `filter.enabled` está anidada bajo `jwt.filter`, pero `exclude-patterns` se sitúa un nivel más arriba, directamente bajo `jwt`: esa es la clave que lee la autoconfiguración (`pyfly.security.jwt.exclude-patterns`). Ya no hay un `/actuator/health` en la lista de exclusión: a partir de la v26.6.110 el actuator y el panel de administración se ejecutan en un **puerto de gestión separado** (`pyfly.management.server.port`, por defecto `9090`), de modo que nunca pasan por el filtro JWT de la aplicación.

⚠ SECRETO DE PRODUCCIÓN

Nunca confirmes el secreto JWT real al control de versiones. Usa `${JWT_SECRET}` e inyecta el valor desde una variable de entorno o un gestor de secretos en el momento del despliegue.

⚠ EL PUERTO DE GESTIÓN ESTÁ ABIERTO POR DEFECTO

Por paridad con Spring, el servidor de gestión (`/actuator/*`) y el panel de administración, puerto por defecto `9090`) está **no autenticado por defecto**: no comparte la puerta de seguridad de la aplicación. En cualquier despliegue que no sea local, asegúralo explícitamente:

```
pyfly:
  management:
    security:
      enabled: true      # apply the security gate to the management
port
```

Establecer `pyfly.management.server.port: -1` deshabilita por completo los endpoints de gestión. La exposición HTTP por defecto del actuador es solo `health,info` ; amplíala con `pyfly.management.endpoints.web.exposure.include` .

🔍 PRUÉBALO — CONFIRMA LA CONEXIÓN AUTOMÁTICA EN EL ARRANQUE

Con las claves anteriores en `pyfly.yaml` , arranca la aplicación y observa el log de arranque:

```
uv run pyfly run
```

Esperado — verás que la aplicación se vincula a `:8080` y el servidor de gestión a `:9090` , y los beans de seguridad aparecen sin ningún código `@configuration` propio. Una prueba rápida de que el bean del encoder existe:

```
curl -s http://localhost:8080/api/auth/login \
  -d '{"username":"alice","password":"hunter2"}' \
  -H 'content-type: application/json'
```

Esperado — un cuerpo JSON que contiene un `access_token` (o un `401` con `INVALID_CREDENTIALS` si las credenciales son incorrectas). En cualquier caso, los beans `JWTService` y `BcryptPasswordEncoder` se conectaron automáticamente.

Autorización con `@secure`

`@secure` es un decorador de funciones que impone autenticación y autorización en manejadores y comandos individuales. Lee el argumento de palabra clave `security_context` que el middleware ya ha inyectado y luego evalúa las comprobaciones de rol, permiso y expresión antes de que se ejecute el cuerpo de la función.

Firma

```
def secure(  
    roles: list[str] | None = None,  
    permissions: list[str] | None = None,  
    expression: str | None = None,  
) → Callable: ...
```

La función decorada debe aceptar `security_context: SecurityContext` como argumento de palabra clave: así es como `@secure` alcanza al usuario actual.

Protección basada en roles

Los endpoints de monederos de Lumen (`/api/v1/wallets`) son el lugar natural para aplicar `@secure`. El `WalletController` real inyecta los buses de comandos y consultas; añade `security_context: SecurityContext` a cada método que necesite protección y apila `@secure` encima.

El controlador expone siete endpoints bajo `/api/v1/wallets` :

Método	Ruta	Manejador	Protección
POST	<code>""</code>	<code>open_wallet</code>	USER o ADMIN
POST	<code>/ {wallet_id} / deposit</code>	<code>deposit</code>	USER/ADMIN + <code>wallet:deposit</code>
POST	<code>/ {wallet_id} / withdraw</code>	<code>withdraw</code>	USER/ADMIN + <code>wallet:deposit</code>
GET	<code>""</code>	<code>list_wallets</code>	autenticado
GET	<code>/ rich</code>	<code>list_rich_wallets</code>	autenticado
GET	<code>/ {wallet_id} / balance</code>	<code>wallet_balance</code>	USER o ADMIN
GET	<code>/ {wallet_id}</code>	<code>wallet_detail</code>	solo ADMIN

Los manejadores de colección (`list_wallets` , `list_rich_wallets`) se nombran con el prefijo `list_` para que el framework los registre antes que los manejadores `wallet_detail` / `wallet_balance` : Starlette resuelve por primero-registrado-gana, así que el segmento literal `/ rich` se encuentra antes que la variable de ruta `/ {wallet_id}` .

lumen/web/controllers/wallet_controller.py

```

from lumen.core.services.wallets.deposit_funds_command import DepositFunds
from lumen.core.services.wallets.get_balance_query import GetBalance
from lumen.core.services.wallets.get_wallet_query import GetWallet
from lumen.core.services.wallets.list_rich_wallets_query import
ListRichWallets
from lumen.core.services.wallets.list_wallets_query import ListWallets
from lumen.core.services.wallets.open_wallet_command import OpenWallet
from lumen.core.services.wallets.withdraw_funds_command import WithdrawFunds
from lumen.interfaces.dtos.v1.balance_dto import BalanceDto
from lumen.interfaces.dtos.v1.deposit_request import DepositRequest
from lumen.interfaces.dtos.v1.open_wallet_request import OpenWalletRequest
from lumen.interfaces.dtos.v1.page_dto import PageDto
from lumen.interfaces.dtos.v1.wallet_dto import WalletDto
from pyfly.container import rest_controller
from pyfly.cqrs import DefaultCommandBus, DefaultQueryBus
from pyfly.data import Pageable, Sort
from pyfly.kernel import ResourceNotFoundException
from pyfly.security import SecurityContext, secure
from pyfly.web import (
    Body,
    PathVar,
    QueryParam,
    Valid,
    get_mapping,
    post_mapping,
    request_mapping,
)

_NEWEST_FIRST = Sort.by("created_at").descending()

@rest_controller
@request_mapping("/api/v1/wallets")
class WalletController:
    """Digital-wallet REST API: open, deposit, withdraw, list, inspect."""

    def __init__(

```

```

        self,
        commands: DefaultCommandBus,
        queries: DefaultQueryBus,
    ) → None:
        self._commands = commands
        self._queries = queries

# Any authenticated user may open a wallet.
@secure(roles=["USER", "ADMIN"])
@post_mapping("", status_code=201)
async def open_wallet(
    self,
    request: Valid[Body[OpenWalletRequest]],
    security_context: SecurityContext,
) → dict[str, str]:
    wallet_id = await self._commands.send(
        OpenWallet(
            owner_id=request.owner_id,
            currency=request.currency,
        )
    )
    return {"wallet_id": wallet_id}

# Deposit: USER/ADMIN role + wallet:deposit permission.
@secure(roles=["USER", "ADMIN"], permissions=["wallet:deposit"])
@post_mapping("/{wallet_id}/deposit")
async def deposit(
    self,
    wallet_id: PathVar[str],
    request: Valid[Body[DepositRequest]],
    security_context: SecurityContext,
) → dict[str, int | str]:
    balance = await self._commands.send(
        DepositFunds(wallet_id=wallet_id, amount=request.amount)
    )
    return {"wallet_id": wallet_id, "balance_minor": balance}

# Withdraw: same guard as deposit.
@secure(roles=["USER", "ADMIN"], permissions=["wallet:deposit"])

```

```

@post_mapping("/{wallet_id}/withdraw")
async def withdraw(
    self,
    wallet_id: PathVar[str],
    request: Valid[Body[DepositRequest]],
    security_context: SecurityContext,
) → dict[str, int | str]:
    balance = await self._commands.send(
        WithdrawFunds(wallet_id=wallet_id, amount=request.amount)
    )
    return {"wallet_id": wallet_id, "balance_minor": balance}

# Paged list - collection routes registered before /{wallet_id}.
@secure(roles=["USER", "ADMIN"])
@get_mapping("")
async def list_wallets(
    self,
    page: QueryParam[int] = 1,
    size: QueryParam[int] = 20,
    security_context: SecurityContext = None,
) → PageDto[WalletDto]:
    result = await self._queries.query(
        ListWallets(
            pageable=Pageable.of(page, size, _NEWEST_FIRST)
        )
    )
    return PageDto.from_page(result)

# Rich list: wallets filtered by minimum balance.
@secure(roles=["USER", "ADMIN"])
@get_mapping("/rich")
async def list_rich_wallets(
    self,
    min_minor: QueryParam[int] = 0,
    page: QueryParam[int] = 1,
    size: QueryParam[int] = 20,
    security_context: SecurityContext = None,
) → PageDto[WalletDto]:
    result = await self._queries.query(

```

```

        ListRichWallets(
            min_minor=min_minor,
            pageable=Pageable.of(page, size, _NEWEST_FIRST),
        )
    )
    return PageDto.from_page(result)

# Single-wallet balance: any authenticated user.
@secure(roles=["USER", "ADMIN"])
@get_mapping("/{wallet_id}/balance")
async def wallet_balance(
    self,
    wallet_id: PathVar[str],
    security_context: SecurityContext,
) → BalanceDto:
    result = await self._queries.query(
        GetBalance(wallet_id=wallet_id)
    )
    if result is None:
        raise ResourceNotFoundException(
            f"Wallet {wallet_id!r} not found",
            code="WALLET_NOT_FOUND",
            context={"wallet_id": wallet_id},
        )
    return result

# Full wallet view: ADMIN only.
@secure(roles=["ADMIN"])
@get_mapping("/{wallet_id}")
async def wallet_detail(
    self,
    wallet_id: PathVar[str],
    security_context: SecurityContext,
) → WalletDto:
    result = await self._queries.query(
        GetWallet(wallet_id=wallet_id)
    )
    if result is None:
        raise ResourceNotFoundException(

```

```

        f"Wallet {wallet_id!r} not found",
        code="WALLET_NOT_FOUND",
        context={"wallet_id": wallet_id},
    )
    return result

```

Listado 14.5 — Protecciones de rol y permiso en endpoints reales de Lumen

...

Cómo funciona. Apila `@secure` encima de `@post_mapping` / `@get_mapping` para que la autorización se ejecute antes que la vinculación de la ruta. El framework inyecta `security_context` desde `request.state.security_context`, que `SecurityMiddleware` ya rellenó.

Cuando listas varios roles, el usuario necesita **al menos uno** (semántica OR). Cuando listas varios permisos, el usuario necesita **todos** ellos (semántica AND). Cuando suministras tanto `roles` como `permissions`, ambas comprobaciones deben pasar de forma independiente.

Los manejadores de colección (`list_wallets`, `list_rich_wallets`) ordenan alfabéticamente antes que `wallet_balance` / `wallet_detail`, de modo que el framework registra primero las rutas literales `""` y `/rich`, garantizando que Starlette resuelva `/api/v1/wallets/rich` como un segmento fijo y no como un id de monedero.

Para añadir `@secure` a un manejador, la receta es siempre los mismos tres pasos:

Paso 1 — Añade el parámetro. Dale al método un parámetro de palabra clave `security_context: SecurityContext`. (Para los manejadores de listado `GET` que ya tienen parámetros de consulta con valor por defecto, dale como valor por defecto `None` para que el orden de los parámetros siga siendo válido: `security_context: SecurityContext = None`.)

Paso 2 — Apila el decorador. Pon `@secure(...)` encima de `@get_mapping` / `@post_mapping`. El orden importa: la comprobación de autorización debe ejecutarse antes de que lo haga la vinculación de la ruta.

Paso 3 — Elige la protección mínima. Escoge la regla de mínimo privilegio que aun así deje pasar a los usuarios correctos: `roles=["USER", "ADMIN"]` para las lecturas ordinarias, un `permissions=["wallet:deposit"]` adicional para las escrituras que mueven dinero, `roles=["ADMIN"]` para la vista de detalle completa.

🔍 PRUÉBALO — OBSERVA UN 403 POR EL ROL INCORRECTO

`wallet_detail` es solo para `ADMIN`. Llámalo con un token `USER` (la autenticación tiene éxito, pero la autorización falla):

```
curl -i -H "Authorization: Bearer $USER_TOKEN" \
http://localhost:8080/api/v1/wallets/abc-123
```

Esperado — la petición supera la puerta (sí está autenticada) pero `@secure(roles=["ADMIN"])` la rechaza. El manejador de excepciones renderiza un cuerpo de problema que incluye el `code` legible por máquina:

```
HTTP/1.1 403 Forbidden
content-type: application/problem+json

{"type":"about:blank","title":"Forbidden","status":403,
 "detail":"Insufficient roles: requires one of ['ADMIN']",
 "code":"FORBIDDEN"}
```

Observa la diferencia con el `401` de la puerta de antes: un token ausente o inválido en la puerta da `401` ("no sé quién eres"), mientras que un token válido sin el rol requerido da `403 FORBIDDEN` ("sé quién eres, y no puedes hacer esto"). El mismo token `USER` sí tendría éxito contra `GET /api/v1/wallets/abc-123/balance`, que está protegido con `roles=["USER", "ADMIN"]`.

IMPORTES EN UNIDADES MENORES

`DepositRequest.amount` es un `int` en **unidades menores** (céntimos). 10,50 € son 1050. Esta convención evita errores de redondeo de coma flotante en todo el dominio Money. `WalletDto.balance_minor` lleva el mismo entero; `WalletDto.balance` es un `float` renderizado solo para mostrar.

Autorización basada en expresiones

Para políticas que no pueden expresarse con una lista plana de roles, usa el parámetro `expression`. PyFly evalúa las expresiones mediante un parseo AST seguro: no hay `eval()` ni `exec()` en ningún punto de la cadena.

lumen/web/controllers/wallet_controller.py

```

from pyfly.security import SecurityContext, secure

# ADMIN, or a MANAGER who also holds wallet:write.
@secure(
    expression=(
        "hasRole('ADMIN')"
        " or (hasRole('MANAGER') and hasPermission('wallet:write'))"
    )
)
async def approve_large_deposit(
    self,
    deposit_id: str,
    security_context: SecurityContext,
) → None:
    ...

# Any authenticated, non-guest user can see the wallet dashboard.
@secure(expression="isAuthenticated and not hasRole('GUEST')")
async def dashboard(
    self,
    security_context: SecurityContext,
) → dict:
    ...

```

Listado 14.6 — Expresiones de seguridad en endpoints de monederos

:::

Vocabulario de expresiones soportado (conjunto completo):

Token	Descripción
<code>hasRole('X')</code>	El usuario tiene el rol <code>X</code>
<code>hasAnyRole('X', 'Y')</code>	El usuario tiene al menos uno de los roles listados
<code>hasAuthority('X')</code>	El usuario tiene el rol o el permiso <code>X</code>
<code>hasAnyAuthority('X', 'Y')</code>	Al menos uno de los roles/permisos listados
<code>hasPermission('X')</code>	El usuario tiene el permiso <code>X</code>
<code>isAuthenticated</code>	El usuario está autenticado
<code>isAnonymous</code>	El usuario no está autenticado
<code>permitAll</code>	Permite siempre
<code>denyAll</code>	Deniega siempre
<code>principal</code> / <code>authentication</code>	El objeto <code>SecurityContext</code> actual
<code>and</code> / <code>or</code> / <code>not</code>	Operadores booleanos
<code>(...)</code>	Agrupación

LAS EXPRESIONES SON SEGURAS

PyFly reduce cada constructo a `True` o `False` y luego evalúa solo un AST booleano puro. Cualquier nodo que no sea una constante, un `BoolOp` o un `UnaryOp(Not)` lanza `SecurityException(code="INVALID_EXPRESSION")`. Esto elimina por completo los riesgos de inyección.

Aplicar @secure a manejadores CQRS

`@secure` no se limita a los controladores REST. Puedes proteger manejadores de comandos CQRS exactamente de la misma forma: se dispara antes del cuerpo del manejador porque el contenedor de DI inyecta `security_context` desde `request.state.security_context` cuando resuelve el manejador:

`lumen/core/services/wallets/deposit_funds_handler.py`

```
from pyfly.cqrs import command_handler
from pyfly.security import SecurityContext, secure

from lumen.core.services.wallets.deposit_funds_command import DepositFunds

@command_handler
class DepositFundsHandler:

    @secure(roles=["USER", "ADMIN"], permissions=["wallet:deposit"])
    async def handle(
        self,
        command: DepositFunds,
        security_context: SecurityContext,
    ) → int:
        # command.amount is in minor units (cents)
        ...
```

Listado 14.7 — `@secure` en un manejador de comandos CQRS

...

La comprobación se dispara antes de que se ejecute cualquier lógica de negocio.

Contraseñas

¿Por qué bcrypt?

MD5 y SHA-256 están diseñados para ser rápidos: ideal para la integridad de datos, catastrófico para las contraseñas. Un atacante que robe tu tabla de usuarios puede probar miles de millones de conjeturas SHA-256 por segundo en hardware de consumo. **Bcrypt** es deliberadamente lento y de coste ajustable: el factor de coste (rounds) te permite afinar el algoritmo para que un ataque requiera órdenes de magnitud más de tiempo, sin afectar de forma apreciable a la latencia normal del inicio de sesión.

BcryptPasswordEncoder

`BcryptPasswordEncoder` implementa el protocolo `PasswordEncoder` (un Protocol `runtime_checkable` con métodos `hash` y `verify`):

`lumen/auth/password_service.py`

```
from pyfly.security import BcryptPasswordEncoder

encoder = BcryptPasswordEncoder(rounds=12)

# During registration – store only the hash, never the raw password
hashed = encoder.hash("correct-horse-battery-staple")

# During login – verify without storing the plaintext
is_match = encoder.verify("correct-horse-battery-staple", hashed)
# True

is_match = encoder.verify("wrong-password", hashed)
# False
```

Listado 14.8 — Hashear y verificar contraseñas

...

Parámetro	Por defecto	Notas
<code>rounds</code>	<code>12</code>	Cada incremento duplica el tiempo de hashing. 12 es el valor por defecto recomendado para producción.

Cómo funciona. `hash` llama a `bcrypt.gensalt(rounds=self._rounds)` para generar una sal aleatoria nueva y luego a `bcrypt.hashpw` para producir el hash. Tanto la sal como el hash quedan incrustados en la cadena devuelta: el prefijo `$2b$12$...` codifica la versión del algoritmo y el factor de coste, de modo que cada hash almacenado se autodescribe. `verify` llama a `bcrypt.checkpw`, que vuelve a derivar el hash a partir de la contraseña en claro y la sal incrustada, y compara el resultado con una comprobación de igualdad segura frente a temporización, evitando ataques de oráculo de tiempo.

El protocolo PasswordEncoder

`PasswordEncoder` es un Protocol `runtime_checkable`. Cualquier clase que implemente `hash(raw: str) → str` y `verify(raw: str, hashed: str) → bool` lo satisface, incluida `BcryptPasswordEncoder`. Puedes sustituirlo por `argon2` o `sCrypt` en cualquier momento sin tocar el código de servicio:

```
from pyfly.security import PasswordEncoder

class Argon2PasswordEncoder:
    def hash(self, raw: str) → str: ...
    def verify(self, raw: str, hashed: str) → bool: ...

isinstance(Argon2PasswordEncoder(), PasswordEncoder) # True
```

💡 AUTOCONFIGURACIÓN

Cuando `pyfly.security.enabled=true` y `bcrypt` está instalado, PyFly autoconfigura un bean `BcryptPasswordEncoder` con `rounds` leído de `pyfly.security.password.bcrypt-rounds`. Declara tú mismo el bean en una clase `@configuration` para sobrescribirlo sin tocar la autoconfiguración.

Sesiones

¿Por qué sesiones del lado del servidor?

Los tokens JWT son sin estado: una vez emitidos, no pueden revocarse antes de que caduquen. Si un usuario cierra sesión, su token sigue siendo válido hasta `exp`. Para muchas API ese compromiso es aceptable. Para aplicaciones de cara al navegador — el panel de administración de Lumen, por ejemplo — necesitas que el servidor sea la fuente autoritativa: cerrar sesión debe significar cerrar sesión.

Las **sesiones del lado del servidor** te dan ese control. Un `SessionStore` guarda los datos de sesión indexados por un ID de sesión aleatorio. El navegador recibe solo el ID de sesión en una cookie. El servidor revoca una sesión al instante eliminando su entrada del almacén.

HttpSession

`HttpSession` envuelve el diccionario de datos de una sesión con accesores tipados y rastrea el estado de mutación para que el filtro sepa cuándo persistir:

Propiedad / Método	Descripción
<code>id</code>	Identificador de sesión, cadena hex UUID
<code>is_new</code>	<code>True</code> si se creó durante esta petición
<code>created_at</code>	Marca de tiempo Unix (<code>float</code>) de la creación
<code>last_accessed</code>	Marca de tiempo Unix (<code>float</code>) del acceso más reciente
<code>modified</code>	<code>True</code> si se escribió algún atributo o la sesión es nueva
<code>invalidated</code>	<code>True</code> si se llamó a <code>invalidate()</code>
<code>previous_id</code>	Id anterior tras <code>rotate_id()</code> (lo usa el filtro para limpiar la entrada antigua)
<code>get_attribute(name)</code>	Devuelve el valor del atributo o <code>None</code>
<code>set_attribute(name, value)</code>	Escribe un atributo; marca la sesión como modificada
<code>remove_attribute(name)</code>	Elimina el atributo si existe
<code>get_attribute_names()</code>	Lista los nombres de atributos puestos por el usuario (excluye las claves internas <code>_*</code>)
<code>rotate_id()</code>	Asigna un id de sesión nuevo, preservando todos los datos
<code>invalidate()</code>	Marca para eliminación en la siguiente pasada del filtro
<code>get_data()</code>	Devuelve el diccionario de sesión en bruto (incluye los metadatos internos)

lumen/core/services/auth/session_handler.py

```

from pyfly.session import HttpSession

async def post_login(
    username: str,
    password: str,
    session: HttpSession,
    auth_service,
) → dict:
    user = await auth_service.authenticate(username, password)

    # Rotate the id before writing auth state (session-fixation prevention)
    session.rotate_id()
    session.set_attribute("user_id", str(user.id))
    session.set_attribute("role", user.role)

    return {"message": "Logged in"}

async def logout(session: HttpSession) → dict:
    session.invalidate()
    return {"message": "Logged out"}

```

Listado 14.9 — Usar la sesión tras el inicio de sesión

...

Cómo funciona. `rotate_id()` genera un ID de sesión UUID nuevo y registra el antiguo en `session.previous_id`. Cuando `SessionFilter` persiste la sesión al final de la petición, elimina la entrada antigua del almacén (el ID anterior ya no puede resolver a esta sesión) y guarda la nueva. Un atacante que hubiera obtenido el ID de sesión previo a la autenticación no puede arrastrarlo a la sesión autenticada: la clásica mitigación de la **fijación de sesión** (session-fixation).

El protocolo SessionStore

Todos los backends implementan el protocolo `SessionStore` :

```
class SessionStore(Protocol):
    async def get(
        self, session_id: str
    ) → dict[str, Any] | None: ...

    async def save(
        self, session_id: str,
        data: dict[str, Any],
        ttl: int,
    ) → None: ...

    async def delete(self, session_id: str) → None: ...

    async def exists(self, session_id: str) → bool: ...
```

Vienen dos adaptadores de serie:

Adaptador	Módulo	Notas
<code>InMemorySessionStore</code>	<code>pyfly.session.adapters.memory</code>	Protegido con <code>asyncio.Lock</code> ; solo monoproceso; los datos se pierden al reiniciar
<code>RedisSessionStore</code>	<code>pyfly.session.adapters.redis</code>	Serializado en JSON; claves con prefijo <code>pyfly:session:</code> ; TTL gestionado por Redis

SessionFilter

`SessionFilter` es un `OncePerRequestFilter` ordenado en `HIGHEST_PRECEDENCE + 150`. Enmarca cada petición:

1. Lee la cookie de sesión (`PYFLY_SESSION` por defecto).
2. Carga la sesión del almacén, o crea una nueva.
3. La adjunta a `request.state.session`.
4. Llama a `call_next(request)`: se ejecuta el resto de la cadena de filtros y el manejador.
5. A la vuelta, persiste una sesión modificada o nueva, elimina una invalidada y reemite la cookie con un `max_age` deslizante (el TTL se desplaza hacia adelante en cada petición).

Atributos de cookie que establece `SessionFilter`:

Atributo	Valor	Razón
<code>httponly</code>	<code>True</code>	Impide el acceso desde JavaScript; mitigación de XSS
<code>samesite</code>	<code>lax</code>	Bloquea la mayoría de los flujos de falsificación de petición entre sitios
<code>secure</code>	configurable	Pon <code>True</code> en producción (solo HTTPS)
<code>max_age</code>	<code>ttl</code>	Caducidad deslizante

Sesiones de Redis en producción

lumen/config/session_config.py

```
import redis.asyncio as aioredis

from pyfly.container import bean, configuration
from pyfly.session import SessionFilter
from pyfly.session.adapters.redis import RedisSessionStore

@configuration
class SessionConfig:

    @bean
    def session_store(self) → RedisSessionStore:
        client = aioredis.from_url(
            "redis://localhost:6379/0"
        )
        return RedisSessionStore(client=client)

    @bean
    def session_filter(
        self,
        store: RedisSessionStore,
    ) → SessionFilter:
        return SessionFilter(
            store=store,
            cookie_name="LUMEN_SESSION",
            ttl=1800,
            secure=True,
        )
```

Listado 14.10 — Almacén de sesiones en Redis

...

Cómo funciona. `RedisSessionStore.save` serializa en JSON el diccionario de sesión (incluidos atributos dataclass como `SecurityContext`, que hacen ida y vuelta mediante un mecanismo de etiquetas de tipo con lista de permitidos) y llama a `client.set(key, raw, ex=ttl)`. El TTL lo gestiona enteramente Redis, de modo que las sesiones caducadas desaparecen del lado del servidor sin ninguna sobrecarga de limpieza. Las claves usan el prefijo `pyfly:session:` para aislamiento de espacio de nombres. Al leer, solo se deserializan los tipos de la lista de permitidos, eliminando los riesgos de instanciación de objetos arbitrarios.

🔗 AUTOCONFIGURACIÓN

Añade `pyfly.session.enabled: true` y `pyfly.session.store: redis` a `pyfly.yaml`. PyFly autoconfigura `RedisSessionStore` (cuando `redis.asyncio` está instalado) y `SessionFilter` por ti. La `redis.url` por defecto es `redis://localhost:6379/0`; sobreescríbela con `pyfly.session.redis.url`.

🍃 EQUIVALENCIA CON SPRING

`SessionFilter` refleja el `SessionRepositoryFilter` de Spring Session. `HttpSession` refleja `javax.servlet.http.HttpSession` (o `jakarta.servlet.http.HttpSession` en Boot 3). `InMemorySessionStore` es equivalente al `MapSessionRepository` de Spring Session; `RedisSessionStore` es equivalente a `RedisSessionRepository`. Los atributos de cookie (`HttpOnly`, `SameSite=Lax`, `Max-Age` deslizante) coinciden exactamente con los valores por defecto de Spring Session.

Validar tokens de un IdP externo (servidor de recursos OAuth2)

Hasta ahora Lumen ha *acuñado sus propios tokens* con `JWTService` y un secreto HMAC compartido. Eso es perfecto para un servicio autocontenido. Pero en la mayoría de los despliegues del mundo real, los tokens los emite un proveedor de identidad dedicado —Keycloak, Microsoft Entra ID (antes Azure AD) o AWS Cognito— y el único trabajo de tu servicio es **validarlos**. Este es el rol de **servidor de recursos OAuth2**, y a partir de la v26.6.110 PyFly te lo da solo con configuración: sin código, y la misma configuración funciona en los tres proveedores.

JERGA: SERVIDOR DE RECURSOS

En términos de OAuth2, el IdP que emite tokens es el *servidor de autorización*; tu API, que custodia los datos que los llamantes quieren y comprueba sus tokens, es el *servidor de recursos*. Lumen es un servidor de recursos. Nunca ve la contraseña del usuario: solo verifica la credencial que el IdP ya firmó.

La diferencia con `JWTService` es el **esquema de firma**. Tus propios tokens se firman con un secreto simétrico (`HS256`): la misma clave firma y verifica, de modo que solo los servicios que poseen el secreto pueden validar. Los tokens del IdP se firman de forma *asimétrica* (`RS256`): el IdP guarda una clave privada y publica las claves públicas correspondientes en un endpoint **JWKS** (JSON Web Key Set). Tu servidor de recursos descarga esas claves públicas, las cachea y las usa para verificar la firma de cada token; nunca necesita el secreto del IdP en absoluto.

Activarlo

`JWKSTokenValidator` hace el trabajo, y `OAuth2ResourceServerAutoConfiguration` lo conecta desde `pyfly.security.oauth2.resource-server.*`. Aquí tienes la configuración completa de Keycloak:

```
lumen/resources/pyfly.yaml
```

```
pyfly:
  security:
    oauth2:
      resource-server:
        enabled: true
        # Either point at the JWKS endpoint directly...
        jwks-uri: "https://keycloak.example.com/realms/lumen/protocol/openid-
connect/certs"
        # ...or give an issuer-uri and let OIDC discovery find jwks-uri +
issuer:
        # issuer-uri: "https://keycloak.example.com/realms/lumen"
        issuer: "https://keycloak.example.com/realms/lumen"
        audiences: "lumen-backend"
        validate-audience: true
        algorithms: "RS256"
        clock-skew-seconds: 60
        exclude-patterns: "/idp/login,/idp/refresh,/docs,/openapi.json"
```

Listado 14.11 — Servidor de recursos OAuth2 multi-IdP (Keycloak)

...

Paso a paso:

Paso 1 — Actívalo. `enabled: true` activa

`OAuth2ResourceServerAutoConfiguration` (PyJWT debe estar instalado). Eso registra un bean `JWKSTokenValidator` y el filtro de token portador (bearer) en la cadena de la aplicación.

Paso 2 — Apunta a las claves. Establece `jwks-uri` directamente, o establece `issuer-uri` y deja que PyFly obtenga `<issuer-uri>/well-known/openid-configuration` para descubrir tanto el `jwks-uri` como el `issuer` autoritativo, exactamente como el `issuer-uri` de Spring.

Paso 3 — Fija la audiencia y el emisor. `audiences` enumera los valores con los que debe coincidir el claim `aud` del token (cualquiera de ellos). `issuer` (cuando se establece) se comprueba contra el `iss` del token. Juntos garantizan que un token acuñado para una aplicación o realm *diferente* no pueda reenviarse contra Lumen.

Paso 4 — Deja los valores por defecto para el mapeo de claims. No configuraste dónde viven los roles y, aun así, funciona para Keycloak, Entra y Cognito. Ese es el trabajo de `ClaimMappings`, que veremos a continuación.

Cómo los claims se convierten en un SecurityContext

Cada IdP pone los roles y scopes en sitios distintos. Keycloak anida los roles de realm bajo `realm_access.roles` y los roles por cliente bajo `resource_access.<client>.roles`; Entra usa un array plano `roles` (o `groups`); Cognito usa `cognito:groups`. `ClaimMappings` los reconcilia con un pequeño lenguaje de rutas, de modo que `has_role(...)` y `has_permission(...)` funcionan igual independientemente del proveedor.

Campo de mapeo	Orden de búsqueda por defecto	Se convierte en
<code>principal_claims</code>	<code>oid</code> , <code>sub</code>	<code>SecurityContext.user_id</code>
<code>authority_claims</code>	<code>roles</code> , <code>scopes</code> , <code>authorities</code> , <code>realm_access.roles</code> , <code>resource_access.*.roles</code> , <code>groups</code> , <code>cognito:groups</code>	<code>SecurityContext.roles</code>
<code>scope_claims</code>	<code>scp</code> , <code>scope</code> (separados por espacios)	<code>SecurityContext.permissions</code>
<code>attribute_claims</code>	(ninguno)	<code>SecurityContext.attributes</code>

Dos reglas hacen que los valores por defecto abarquen todos los proveedores:

- Las **rutas con puntos** descienden a objetos anidados: `realm_access.roles` lee `payload["realm_access"]["roles"]`.
- Un **comodín *** de un solo nivel itera sobre cada clave de ese nivel: `resource_access.*.roles` recopila el array `roles` de *cada* entrada de cliente bajo `resource_access`. Los segmentos de ruta se dividen solo por `.`, así que un nombre de claim que lleve dos puntos, como `cognito:groups`, se compara textualmente.

Los roles se recopilan de *todas* las rutas coincidentes y se deduplican, de modo que un token de Keycloak aporta tanto sus roles de realm como sus roles de cliente a una única lista plana `roles`.

JERGA: JWKS Y KID

Un *JWKS* es el directorio de claves públicas del IdP. Cada clave lleva un `kid` (key id); la cabecera de cada token nombra el `kid` con el que se firmó. El validador busca esa clave exacta, cachea el conjunto (por defecto 300 s) y rota automáticamente cuando el IdP publica claves nuevas: sin necesidad de redespliegue.

Si los valores por defecto integrados no coinciden con tu IdP, sobrescribe solo lo que difiera:

```
lumen/resources/pyfly.yaml
```

```
pyfly:
  security:
    oauth2:
      resource-server:
        enabled: true
        # Microsoft Entra ID (v2.0):
        issuer-uri: "https://login.microsoftonline.com/<tenant-id>/v2.0"
        audiences: "api://lumen-backend"
        principal-claim-names: "oid,sub"           # Entra's stable user id first
        authorities-claim-names: "roles,groups"
        scope-claim-names: "scp"                   # Entra delegated scopes
        attribute-claims: "tid,preferred_username"
        # AWS Cognito access tokens carry no 'aud' – disable audience checks:
        # validate-audience: false
        # authorities-claim-names: "cognito:groups"
```

Listado 14.12 — Ajustar el mapeo de claims para Entra ID y Cognito

...

Paso 5 (cuando sea necesario) — Cognito no tiene `aud`. Los tokens de acceso de Cognito llevan `client_id` en lugar de `aud`, así que establece `validate-audience: false` para ellos; las comprobaciones de firma, `iss` y `exp` siguen aplicándose.

🔗 PRUÉBALO — ACEPTA UN TOKEN REAL DEL IDP, RECHAZA UNO FALSIFICADO

Con el servidor de recursos activado, reenvía un token emitido por tu IdP contra una ruta protegida:

```
curl -i -H "Authorization: Bearer $KEYCLOAK_TOKEN" \
  http://localhost:8080/api/v1/wallets
```

Esperado — `HTTP/1.1 200 OK`. El filtro obtuvo el JWKS, encontró la coincidencia con el `kid` del token, verificó la firma `RS256`, el `iss`, el `aud` y el `exp` (con 60 s de tolerancia de desfase de reloj) y luego construyó un `SecurityContext` a partir de los claims del token.

Ahora cambia un carácter del token y reinténtalo. Esperado — `HTTP/1.1 401` (o un contexto anónimo que la puerta rechaza después), porque la firma ya no coincide con ninguna clave publicada. No escribiste ni desplegaste una sola línea de código de validación para conseguir ninguno de los dos resultados.

Lo que acaba de ocurrir. Has añadido un validador de tokens multi-IdP de nivel de producción solo con configuración. `JWKSTokenValidator` verifica la firma contra las claves publicadas del IdP (de modo que el secreto nunca abandona el IdP), comprueba `iss` / `aud` / `exp` y mapea los claims específicos de cada proveedor al mismo `SecurityContext` que el resto del capítulo ya usa. Tus decoradores `@secure` y tus reglas `HttpSecurity` no cambian en absoluto: leen `roles` y `permissions` exactamente como antes, tanto si el token vino del propio `JWTService` de Lumen como si vino de Keycloak.

⚠ MODO DE ERROR: ANÓNIMO VS 401

Por defecto (`authenticate-error-mode: anonymous`) un token *presente pero inválido* produce un contexto anónimo y la petición continúa hasta la puerta de `HttpSecurity` , que decide. Establece `authenticate-error-mode: "401"` para rechazar un token inválido directamente en el filtro con `WWW-Authenticate: Bearer error="invalid_token"` (RFC 6750). Un token *ausente* siempre cae hasta la puerta en cualquier caso.

🍃 EQUIVALENCIA CON SPRING

Este es el `spring-boot-starter-oauth2-resource-server` de PyFly. Las claves `resource-server.issuer-uri / jwks-uri` reflejan `spring.security.oauth2.resourceserver.jwt.*` ; el descubrimiento OIDC, la lista de audiencias aceptada, los `algorithms` configurables y el desfase de reloj por defecto de 60 segundos coinciden todos con el `JwtDecoder` y el `JwtTimestampValidator` de Spring Security. `ClaimMappings` es el equivalente de un `JwtAuthenticationConverter / JwtGrantedAuthoritiesConverter` personalizado, pero expresado de forma declarativa en YAML.

Identidad externa (IDP)

El problema de gestionar la identidad internamente

Lumen almacena actualmente las credenciales en su propia base de datos, lo que significa que Lumen debe implementar restablecimiento de contraseñas, MFA, verificación de correo, bloqueo de cuentas, borrado GDPR, inicio de sesión social y SSO: trabajo no diferenciador que todo servicio acaba necesitando. La respuesta del sector es delegar la identidad a un proveedor dedicado: Keycloak para entornos on-premise, AWS Cognito para pilas nativas de AWS, Azure AD para entornos de Microsoft.

El puerto `IdpAdapter` de PyFly hace esa delegación conectable tras una única interfaz. Cambia el adaptador y la capa de negocio nunca se entera.

IdpAdapter — el puerto

Todo adaptador debe satisfacer el protocolo `IdpAdapter`:

```

class IdpAdapter(Protocol):
    name: str

    # User management
    async def create_user(
        self, user: IdpUser, password: str
    ) → IdpUser: ...
    async def get_user(self, user_id: str) → IdpUser | None: ...
    async def find_by_username(
        self, username: str
    ) → IdpUser | None: ...
    async def update_user(self, user: IdpUser) → IdpUser: ...
    async def delete_user(self, user_id: str) → bool: ...
    async def list_users(self, *, limit: int = 100) → list[IdpUser]: ...

    # Authentication
    async def login(
        self, request: LoginRequest
    ) → AuthResult: ...
    async def logout(self, access_token: str) → bool: ...
    async def refresh(
        self, refresh_token: str
    ) → AuthResult: ...
    async def introspect(
        self, access_token: str
    ) → SessionIntrospection: ...

    # Password / MFA
    async def change_password(
        self, request: PasswordChangeRequest
    ) → bool: ...
    async def reset_password(self, user_id: str) → str: ...

    # Roles
    async def assign_role(
        self, user_id: str, role: str
    ) → bool: ...
    async def revoke_role(

```

```
        self, user_id: str, role: str
    ) → bool: ...
    async def list_roles(self) → list[IdpRole]: ...
```

DTOs clave:

Clase	Propósito
IdpUser	Registro de usuario: id , username , email , roles , attributes , ...
LoginRequest	username , password , mfa_code (opcional)
AuthResult	user , access_token , refresh_token , expires_in , token_type
SessionIntrospection	active , user_id , username , scopes , expires_at
PasswordChangeRequest	user_id , old_password , new_password
IdpRole	name , description , scopes

Adaptador de Keycloak

lumen/config/idp_config.py

```
from pyfly.container import bean, configuration
from pyfly.idp import IdpAdapter, KeycloakIdpAdapter

@Configuration
class IdpConfig:

    @bean
    def idp_adapter(self) → IdpAdapter:
        return KeycloakIdpAdapter(
            base_url="https://keycloak.example.com",
            realm="lumen",
            client_id="lumen-backend",
            client_secret="${KEYCLOAK_SECRET}",
            verify_ssl=True,
        )
```

Listado 14.13 — Conectar el adaptador de Keycloak

...

Cómo funciona. `KeycloakIdpAdapter` se comunica con la API REST de administración de Keycloak (`/admin/realms/{realm}/users`) y con el endpoint de tokens (`/realms/{realm}/protocol/openid-connect/token`) vía `httpx` . Cachea internamente el token de administración `client_credentials` y lo vuelve a obtener dentro de un margen de seguridad de diez segundos antes de la caducidad; el TTL por defecto de las client-credentials de Keycloak es de 60 s, así que sin esta caché cada llamada de administración requeriría dos viajes de ida y vuelta por red.

Usar el IDP en un servicio

```
lumen/core/services/auth/idp_auth_service.py
```

```
from pyfly.container import service
from pyfly.idp import IdpAdapter, IdpUser, LoginRequest
from pyfly.kernel.exceptions import UnauthorizedException

@service
class IdpAuthService:

    def __init__(self, idp: IdpAdapter) → None:
        self._idp = idp

    async def register(
        self,
        username: str,
        email: str,
        password: str,
        role: str = "USER",
    ) → str:
        user = IdpUser(
            username=username,
            email=email,
            roles=[role],
        )
        created = await self._idp.create_user(user, password)
        result = await self._idp.login(
            LoginRequest(
                username=username, password=password
            )
        )
        return result.access_token

    async def login(
        self, username: str, password: str
    ) → str:
        try:
            result = await self._idp.login(
                LoginRequest(
```

```

        username=username, password=password
    )
)
except PermissionError as exc:
    raise UnauthorizedException(
        "Invalid credentials",
        code="INVALID_CREDENTIALS",
    ) from exc
return result.access_token

async def introspect(self, token: str) → dict:
    info = await self._idp.introspect(token)
    return {
        "active": info.active,
        "user_id": info.user_id,
        "username": info.username,
        "scopes": info.scopes,
    }

```

Listado 14.14 — Usar `IdpAdapter` en el servicio de autenticación

...

Cómo funciona. `IdpAuthService` depende solo de `IdpAdapter`: el contenedor de DI resuelve el `KeycloakIdpAdapter` concreto en el arranque. La capa de servicio nunca importa código específico de Keycloak, Cognito ni Azure. Cambia de proveedor modificando una sola línea en `IdpConfig`; el servicio queda intacto.

Autoconfiguración y las rutas HTTP integradas

Activa el subsistema del IDP en `pyfly.yaml` y PyFly conecta el adaptador y un controlador REST ya hecho de forma automática:

lumen/resources/pyfly.yaml

```
pyfly:
  idp:
    enabled: true
    provider: keycloak
    keycloak:
      base-url: https://keycloak.example.com
      realm: lumen
      client-id: lumen-backend
      client-secret: "${KEYCLOAK_SECRET}"
```

Listado 14.15 — Autoconfiguración del IDP

...

Valor de provider	Adaptador
internal-db	InternalDbIdpAdapter (almacén bcrypt en memoria)
keycloak	KeycloakIdpAdapter
cognito / aws-cognito	AwsCognitoIdpAdapter
azure-ad / azuread / entra	AzureAdIdpAdapter

Cuando Starlette está presente, `IdpAutoConfiguration` también registra un bean `IdpController` que expone la API completa del IDP bajo `/idp`:

Ruta	Método	Descripción
<code>/idp/login</code>	POST	Autenticar (usuario + contraseña + MFA opcional)
<code>/idp/refresh</code>	POST	Refrescar un token de acceso
<code>/idp/logout</code>	POST	Revocar un token
<code>/idp/introspect</code>	POST	Inspeccionar una sesión activa
<code>/idp/admin/users</code>	POST	Crear un usuario
<code>/idp/admin/users</code>	GET	Listar usuarios
<code>/idp/admin/users/{user_id}</code>	GET / DELETE	Obtener o eliminar un usuario
<code>/idp/admin/users/{user_id}/roles/{role}</code>	POST / DELETE	Asignar o revocar un rol
<code>/idp/admin/roles</code>	GET	Listar todos los roles

🔗 ADAPTADORES PERSONALIZADOS

Cualquier clase que satisfaga el Protocol `IdpAdapter` puede conectarse como el bean `IdpAdapter`. Regístralo en una clase `@configuration` y el `@conditional_on_missing_bean` de PyFly se salta la autoconfiguración por completo. Este es el punto de extensión estándar para LDAP on-premise, SSO interno o adaptadores de doble de prueba (test-double).

🍃 EQUIVALENCIA CON SPRING

`IdpAdapter` es el equivalente en PyFly de la combinación `UserService` + `AuthProvider` de Spring Security. `IdpUser` se corresponde con `UserDetails`; `AuthResult` se corresponde con el objeto `Authentication` devuelto por `AuthenticationManager.authenticate()`. `KeycloakIdpAdapter` desempeña el papel del adaptador de Spring Security para Keycloak.

Iniciar sesión con un formulario

Todo lo visto hasta ahora daba por supuesto un cliente de máquina: acuñar un JWT, reenviarlo en la cabecera `Authorization` y validarlo a la entrada. Eso es exactamente lo correcto para la API de Lumen. Pero Lumen también tiene un panel de administración —un navegador— y los navegadores no llevan cabeceras Bearer. Llevan cookies. Para el panel quieres lo de siempre, lo aburrido y correcto: un formulario de usuario/contraseña que hace POST a una URL de inicio de sesión, y una cookie de sesión que viaja con cada petición posterior.

PyFly te da eso sin escribir un controlador. `FormLoginFilter` intercepta un POST a la URL de inicio de sesión, autentica las credenciales a través de un `ProviderManager` y —algo crucial— **rota el id de sesión** antes de vincular el `SecurityContext` autenticado a la sesión. La rotación es la defensa contra la fijación de sesión (session-fixation) de la sección de sesiones, aplicada automáticamente en el momento en que cambian los privilegios: cualquier id de sesión que un atacante pudiera haber plantado antes del inicio de sesión se descarta y se reemplaza. El contexto se almacena en la sesión, y `OAuth2SessionSecurityFilter` lo restaura en cada petición posterior, de modo que el panel permanece con la sesión iniciada.

Declaras los usuarios y las URLs en `pyfly.yaml`. Las contraseñas se almacenan **pre-hasheadas** —nunca en texto plano— y la autoconfiguración las verifica con `BcryptPasswordEncoder` a través de un `DaoAuthenticationProvider`:

```
pyfly:
  security:
    enabled: true
    form-login:
      enabled: true
      login-url: /login
      username-param: username
      password-param: password
      success-url: /dashboard
      failure-url: /login?error
      use-redirect: true
    users:
      alice:
        # hash bcrypt de "hunter2" — génalo con BcryptPasswordEncoder.hash
        password-hash: "$2b$12$Q9.../iJ8wq6Yk5fJ0b0"
        roles: ADMIN
        permissions: wallet:read,wallet:create
        enabled: true
    logout:
      enabled: true
      logout-url: /logout
      success-url: /login?logout
      delete-cookies: PYFLY_SESSION
      use-redirect: true
```

`FormLoginFilter` se ejecuta en `HIGHEST_PRECEDENCE + 230`, justo después del filtro que restaura la sesión, de modo que un inicio de sesión nuevo siempre prevalece sobre cualquier contexto anónimo previo. `LogoutFilter` corre justo detrás, en `+235`: un POST a la URL de cierre de sesión invalida la sesión, restablece el contexto de la petición a `SecurityContext.anonymous()` y elimina las cookies que nombres en `delete-cookies`. Ambos filtros hablan dos dialectos: pon `use-redirect: false` y

responderán con JSON (`{"authenticated": true}` / `204`) en lugar de un `302` , que es lo que quieres si el panel es una aplicación de página única (SPA) que llama a `fetch` .

La autenticación en sí es el `DaoAuthenticationProvider` de la SPI de autenticación de PyFly. Es deliberadamente cuidadoso en una cosa: un nombre de usuario desconocido aun así incurre en una verificación de contraseña contra un hash ficticio de usar y tirar, de modo que el endpoint de inicio de sesión no puede usarse como un oráculo de nombres de usuario: una contraseña incorrecta y un usuario inexistente tardan lo mismo y lanzan la misma `BadCredentialsException` . Una cuenta deshabilitada (`enabled: false`) lanza `DisabledException` .

🔗 PRUÉBALO — INICIA SESIÓN CON UNA COOKIE Y LUEGO CIERRA SESIÓN

Arranca la aplicación y haz POST del formulario (fíjate en `-d` , no JSON: esto es un envío de formulario HTML real):

```
curl -i -X POST http://localhost:8080/login \
  -d "username=alice&password=hunter2"
```

Esperado — `HTTP/1.1 302 Found` , una cabecera `location: /dashboard` y un `set-cookie: PYFLY_SESSION=...` que lleva el id de sesión nuevo (rotado). Reenvía una contraseña incorrecta y en su lugar obtendrás `302 → /login?error` . Ahora termina la sesión:

```
curl -i -X POST http://localhost:8080/logout -b "PYFLY_SESSION=<id>"
```

Esperado — `302 → /login?logout` y un `set-cookie` que caduca `PYFLY_SESSION` . La entrada de la sesión ya no está en el almacén; la cookie ya no resuelve a nada.

🍃 EQUIVALENCIA CON SPRING

Estos son los configuradores `formLogin()` y `logout()` de Spring Security.

`FormLoginFilter` se corresponde con `UsernamePasswordAuthenticationFilter`, `LogoutFilter` con el `LogoutFilter` de Spring, y el mapa `form-login.users` guiado por configuración es el equivalente de un `UserDetailsManager` en memoria. La rotación del id de sesión al iniciar sesión coincide con la estrategia de fijación de sesión `changeSessionId` por defecto de Spring.

Hashing de contraseñas más robusto

`bcrypt` es un buen valor por defecto, y es el que Lumen ha usado hasta ahora. Pero dos cosas son ciertas a la vez: `bcrypt` es suficientemente bueno hoy, y tarde o temprano querrás abandonarlo —en favor de `Argon2id`, el algoritmo que OWASP recomienda ahora— sin una migración de golpe que obligue a cada usuario a restablecer su contraseña. El obstáculo es que un hash almacenado no es más que una cadena; nada en `$2b$12$...` le dice a tu código *qué* algoritmo lo produjo una vez que tienes más de uno en juego.

`DelegatingPasswordEncoder` resuelve esto igual que Spring Security: antepone a cada hash que produce el id del encoder entre llaves — `{argon2}...`, `{bcrypt}...`, `{pbkdf2}...`, `{scrypt}...` —. Al verificar, lee el prefijo y delega en el encoder correspondiente, de modo que un único bean de encoder puede *leer* todos los hashes heredados mientras *escribe* solo el valor por defecto actual. Un valor almacenado con un prefijo desconocido o ausente nunca coincide: falla de forma cerrada (fail-closed) por construcción.

PyFly incluye cuatro adaptadores de `PasswordEncoder` — `BcryptPasswordEncoder`, `Pbkdf2PasswordEncoder`, `ScryptPasswordEncoder` y `Argon2PasswordEncoder` (Argon2id; instálalo con `pip install pyfly[argon2]`) — más la factoría `create_delegating_password_encoder`, que construye un encoder delegador con **bcrypt** como id por defecto y los otros tres conectados para la verificación. Para hacer de Argon2id el valor *por defecto* para los hashes nuevos sin dejar de leer la columna bcrypt existente de Lumen, construye el encoder delegador directamente y pásale `encoding_id="argon2"`:

```
from pyfly.container import bean, configuration
from pyfly.security import (
    Argon2PasswordEncoder,
    BcryptPasswordEncoder,
    DelegatingPasswordEncoder,
    Pbkdf2PasswordEncoder,
    ScryptPasswordEncoder,
)

@Configuration
class PasswordConfig:

    @bean
    def password_encoder(self) → DelegatingPasswordEncoder:
        return DelegatingPasswordEncoder(
            {
                "argon2": Argon2PasswordEncoder(), # valor por defecto
                "bcrypt": BcryptPasswordEncoder(rounds=12),
                "pbkdf2": Pbkdf2PasswordEncoder(),
                "scrypt": ScryptPasswordEncoder(),
            },
            encoding_id="argon2",
        )
```

La ventaja es la **migración transparente en el inicio de sesión**.

`DelegatingPasswordEncoder` expone `upgrade_encoding(stored)`, que devuelve `True` cuando un hash almacenado fue producido por algo distinto del valor por defecto actual. Llámalo dentro de `login`, justo después de que la contraseña se verifique, y vuelve a hashear in situ. Cada usuario se actualiza silenciosamente de bcrypt a Argon2id la próxima vez que inicie sesión: sin correo de restablecimiento, sin tiempo de inactividad, sin una segunda columna:

```
async def login(self, username: str, password: str) → str:
    user = await self._users.find_by_username(username)
    if user is None or not self._encoder.verify(password, user.password_hash):
        raise UnauthorizedException(
            "Invalid credentials", code="INVALID_CREDENTIALS"
        )

    # Re-hashea de forma transparente las credenciales heredadas ({bcrypt}...)
    al
    # valor por defecto actual ({argon2}...) ahora que tenemos el texto plano y
    se verificó.
    if self._encoder.upgrade_encoding(user.password_hash):
        user.password_hash = self._encoder.hash(password)
        await self._users.save(user)

    return self._jwt.encode({"sub": str(user.id), "roles": [user.role]})
```

💡 PRUÉBALO — OBSERVA EL PREFIJO Y LA ACTUALIZACIÓN EN EL REPL

```
>>> from pyfly.security import create_delegating_password_encoder
>>> enc = create_delegating_password_encoder() # bcrypt por defecto
>>> h = enc.hash("hunter2")
>>> h[:8]
'{bcrypt}'
>>> enc.verify("hunter2", h)
True
>>> enc.upgrade_encoding(h) # ya es el valor por defecto → no
necesita actualización
False
>>> enc.upgrade_encoding("{pbkdf2}sha256$600000$...") # heredado →
actualiza
True
```

El prefijo `{id}` es lo que hace que esto funcione: la verificación se enruta por él, y `upgrade_encoding` lo compara con el valor por defecto configurado.

❗ JERGA: ARGON2ID

Argon2id es un hash de contraseñas duro en memoria (memory-hard): obliga a un atacante a gastar *memoria*, no solo CPU, por cada conjetura, lo que neutraliza las granjas de GPU y ASIC que abaratan a gran escala el crackeo de bcrypt.

`Argon2PasswordEncoder` usa por defecto los parámetros interactivos de OWASP (coste de tiempo 3, 64 MiB, 4 carriles).

Lumen como su propio servidor de autorización

Hasta ahora Lumen ha sido un *consumidor* de tokens: acuñando JWTs simples para su propio inicio de sesión, o validando tokens emitidos por un IdP externo. Hay un tercer papel que Lumen puede desempeñar: un verdadero **servidor de autorización** OAuth2

que emite tokens a *otras* partes. Querrás esto en cuanto Lumen tenga más de una pieza en movimiento. Un worker nocturno de conciliación del libro mayor necesita llamar a la API de Lumen como sí mismo, sin ningún humano en el bucle. Una aplicación móvil propia necesita iniciar la sesión de un usuario y actuar en su nombre. Esos son dos tipos de concesión (grant) de OAuth2 distintos, y `AuthorizationServer` habla ambos.

Servicio a servicio: client_credentials

El worker de conciliación es un cliente confidencial: posee un secreto y pide un token en su propio nombre. Registra los clientes de Lumen en un `InMemoryClientRegistrationRepository` y activa el servidor de autorización en la configuración: el bean `AuthorizationServer` y un almacén de tokens en memoria se autoconfiguran por ti:

```

from pyfly.container import bean, configuration
from pyfly.security.oauth2 import (
    ClientRegistration,
    InMemoryClientRegistrationRepository,
)

@configuration
class LumenClientsConfig:

    @bean
    def client_repository(self) → InMemoryClientRegistrationRepository:
        return InMemoryClientRegistrationRepository(
            # Cliente de máquina – servicio a servicio, sin usuario.
            ClientRegistration(
                registration_id="lumen-reconciler",
                client_id="lumen-reconciler",
                client_secret="${RECONCILER_SECRET}",
                authorization_grant_type="client_credentials",
                scopes=["ledger:read"],
            ),
            # Aplicación móvil propia – cliente público (sin secreto), solo
            PKCE.
            ClientRegistration(
                registration_id="lumen-mobile",
                client_id="lumen-mobile",
                authorization_grant_type="authorization_code",
                redirect_uri="com.lumen.app://callback",
                scopes=["openid", "wallet:read", "wallet:deposit"],
            ),
        )

```

```

pyfly:
  security:
    oauth2:
      authorization-server:
        enabled: true
        secret: "${AS_SIGNING_SECRET}" # nunca el marcador de posición;
        HS256 ≥ 32 bytes
        issuer: "https://lumen.example.com"
        audience: "lumen-backend"
        access-token-ttl: 3600
        refresh-token-ttl: 86400
      token-store:
        provider: redis # memory | redis | postgres
        redis:
          url: "redis://localhost:6379/0"

```

La superficie HTTP es `AuthorizationServerEndpoints`, que construye las rutas estándar de OAuth2/OIDC a partir del bean `AuthorizationServer`. Móntalas en la aplicación:

```

from pyfly.security.oauth2 import AuthorizationServer,
AuthorizationServerEndpoints

def register_oauth2(app, context):
    server = context.get_bean(AuthorizationServer)
    app.router.routes.extend(AuthorizationServerEndpoints(server).routes())

```

Eso expone el conjunto completo: `GET /oauth2/authorize`, `POST /oauth2/par`, `POST /oauth2/token`, `POST /oauth2/introspect`, `POST /oauth2/revoke`, `POST /oauth2/register`, `GET /oauth2/jwks`, y los dos documentos de descubrimiento `GET /.well-known/oauth-authorization-server` y `GET /.well-known/openid-configuration`.

 PRUÉBALO — OBTÉN UN TOKEN DE MÁQUINA

El worker se autentica con HTTP Basic y pide el ámbito (scope) para el que fue registrado:

```
curl -s -X POST http://localhost:8080/oauth2/token \
  -u lumen-reconciler:$RECONCILER_SECRET \
  -d grant_type=client_credentials \
  -d scope="ledger:read"
```

Esperado — una respuesta de token JSON con `access_token`, `token_type`: "Bearer", `expires_in`: 3600, un `refresh_token` y `scope`: "ledger:read". Ahora pide un ámbito para el que el cliente **no** estaba registrado:

```
curl -s -X POST http://localhost:8080/oauth2/token \
  -u lumen-reconciler:$RECONCILER_SECRET \
  -d grant_type=client_credentials -d scope="ledger:write admin"
```

Esperado — `{"error": "invalid_scope", ...}` con HTTP 400. Un cliente solo puede obtener los ámbitos para los que está registrado; los ámbitos no registrados se rechazan por completo, nunca se devuelven en silencio. (Pedir `client_credentials` desde un cliente registrado solo para `authorization_code` se rechaza del mismo modo, con `unauthorized_client`.)

De cara al usuario: `authorization_code` + PKCE + un `id_token` OIDC

La aplicación móvil es un cliente público: no incluye ningún secreto, porque cualquier cosa empotrada en el binario de una app no es un secreto. Su prueba de posesión es **PKCE**: la app genera un `code_verifier` aleatorio, envía solo su hash SHA-256 (`code_challenge`) cuando pide un código de autorización, y revela el verificador solo cuando canjea el código. Un atacante que intercepte el código no puede canjearlo sin el verificador.

El flujo tiene dos tramos. Primero la app envía al usuario al endpoint de autorización:


```
GET /oauth2/authorize?response_type=code
    &client_id=lumen-mobile
    &redirect_uri=com.lumen.app://callback
    &scope=openid%20wallet:read
    &code_challenge=E9MeIhoa20wvFrEMTJguCHaoeK1t8URWbuGJSstw-cM
    &code_challenge_method=S256
    &state=xyz
```

Si el propietario del recurso aún no está autenticado, el endpoint lo redirige a la URL de inicio de sesión y vuelve. Una vez lo está, Lumen emite un código de autorización **de un solo uso** y redirige a `redirect_uri?code=...&state=xyz&iss=...`. El parámetro `iss` (RFC 9207) permite a la app confirmar qué servidor respondió: una defensa contra la confusión (mix-up). Luego la app canjea el código, presentando el verificador:

```
curl -s -X POST http://localhost:8080/oauth2/token \
  -d grant_type=authorization_code \
  -d client_id=lumen-mobile \
  -d code="$CODE" \
  -d redirect_uri="com.lumen.app://callback" \
  -d code_verifier="$VERIFIER"
```

La respuesta lleva el `access_token`, un `refresh_token` y —porque la petición incluía el ámbito `openid`— un `id_token` OIDC cuyo `aud` es el id del cliente. El servidor de autorización impone las barreras de seguridad por ti: el redirect URI debe coincidir con el registro **exactamente**, PKCE es **obligatorio y debe ser S256**, y el código es **de un solo uso**. Reenvía un código que ya se había canjeado y Lumen no se limita a rechazarlo: trata la repetición como un ataque de inyección y **revoca el token de actualización (refresh token) ya emitido a partir de ese código**.

Rotación de tokens de actualización y detección de reutilización

Los tokens de actualización (refresh tokens) rotan. Cada vez que la app refresca, obtiene un *nuevo* token de actualización y el antiguo se marca como consumido:

```
curl -s -X POST http://localhost:8080/oauth2/token \  
-d grant_type=refresh_token -d client_id=lumen-mobile \  
-d refresh_token="$OLD_REFRESH"
```

El token consumido no se elimina: se conserva precisamente para poder detectar una *repetición*. Si se presenta un token de actualización robado después de que el cliente legítimo ya lo haya rotado, el servidor detecta la reutilización y revoca la **familia de tokens completa** (el token original y todos sus descendientes), obligando tanto al atacante como a la víctima a pasar de nuevo por el inicio de sesión. Un cliente legítimo nunca reenvía un token ya rotado, así que la reutilización es prueba inequívoca de robo. Tanto el caso de reutilización como una presentación desde el cliente equivocado lanzan `INVALID_GRANT`.

Firma asimétrica y JWKS

Para los tokens de servicio a servicio, HS256 con un secreto compartido está bien: Lumen firma y Lumen verifica. Pero en cuanto *otros* servicios validan los tokens de Lumen, no querrás entregarles tu secreto de firma. Cambia a la firma asimétrica: Lumen guarda una clave privada, firma con `RS256` (o `ES256` / `PS256`) y publica solo la clave **pública** correspondiente en `/oauth2/jwks`. Proporciona tú mismo un bean `AuthorizationServer` para optar por el modo asimétrico (un bean declarado a mano suprime la autoconfiguración de HS256):

```

from pyfly.container import bean, configuration
from pyfly.security.oauth2 import (
    AuthorizationServer,
    InMemoryClientRegistrationRepository,
    InMemoryTokenStore,
)

@Configuration
class AsymmetricAuthServerConfig:

    @bean
    def authorization_server(
        self, clients: InMemoryClientRegistrationRepository
    ) → AuthorizationServer:
        return AuthorizationServer(
            secret="", # sin uso en la firma asimétrica
            client_repository=clients,
            token_store=InMemoryTokenStore(),
            algorithm="RS256",
            private_key=open("lumen-signing-key.pem").read(),
            key_id="lumen-2026", # se convierte en el `kid` de la
            # cabecera del JWT
            issuer="https://lumen.example.com",
        )

```

Ahora `GET /oauth2/jwks` devuelve el conjunto de claves JWK públicas (con el `kid` y el `alg`), y cualquier servidor de recursos PyFly puede validar los tokens de Lumen apuntando su `pyfly.security.oauth2.resource-server.jwks-uri` a él: exactamente la configuración de Keycloak vista antes en este capítulo, pero con Lumen como emisor. Lumen ha cerrado el círculo: emite los tokens y los valida, y las dos mitades no comparten nada salvo una clave pública.

🍃 EQUIVALENCIA CON SPRING

`AuthorizationServer` + `AuthorizationServerEndpoints` es el `spring-authorization-server` de PyFly. Las rutas de los endpoints, los documentos de descubrimiento, el PKCE obligatorio con S256, la rotación de tokens de actualización con detección de reutilización y el conjunto de claves de `/oauth2/jwks` coinciden todos con los valores por defecto de Spring Authorization Server.

Tokens restringidos al poseedor (sender-constrained)

Todos los tokens de este capítulo hasta ahora han sido tokens *portadores* (bearer): quien lo tiene, lo usa. Esa es toda la gracia, y todo el riesgo. Un token portador que se filtra por un log, un proxy o un dispositivo comprometido es una credencial totalmente utilizable en manos de un atacante. Para un monedero que mueve dinero, "quien lo tiene" no es un modelo de seguridad cómodo. La restricción al poseedor (sender-constraining) lo arregla vinculando el token a una clave que posee el cliente legítimo, de modo que un token robado es inerte sin esa clave.

PyFly soporta ambos mecanismos estándar, y la vinculación se transporta en el claim `cnf` (confirmación) del token:

- **DPoP** (RFC 9449) — el cliente firma un JWT de *prueba* (proof) nuevo por cada petición con su clave privada; el token de acceso lleva `cnf.jkt`, la huella SHA-256 de esa clave (RFC 7638). El servidor de recursos verifica la prueba y que la huella de su clave coincida con `jkt`.
- **mTLS** (RFC 8705) — el token de acceso lleva `cnf["x5t#S256"]`, la huella del certificado TLS del cliente. El servidor de recursos la compara con el certificado presentado en la conexión.

El servidor de autorización vincula el token automáticamente: si un cliente presenta una cabecera `DPoP` en la petición de token, Lumen estampa `cnf.jkt` en el token de acceso emitido. En el lado del servidor de recursos activas la imposición:

```
pyfly:
  security:
    oauth2:
      resource-server:
        enabled: true
        jwks-uri: "https://lumen.example.com/oauth2/jwks"
        enforce-sender-constraints: true
        mtls-cert-header: "x-client-cert"    # lo establece tu proxy que
termina el TLS
```

Con `enforce-sender-constraints: true`, cuando un token lleva un claim `cnf` el filtro *exige* la prueba correspondiente: un token `cnf.jkt` sin una cabecera de prueba `DPoP` válida se rechaza como `INVALID_TOKEN`; un token `cnf["x5t#$S256"]` cuya huella no coincida con el certificado de cliente presentado se rechaza igualmente. Un token portador simple (sin `cnf`) no se ve afectado: la imposición solo entra en juego para los tokens que se emitieron restringidos al poseedor, de modo que puedes desplegarla de forma gradual.

Las primitivas de verificación son públicas si las necesitas directamente:

`DPoPProofValidator`, `confirm_dpop_binding`, `confirm_mtls_binding`, `jwk_thumbprint`, `certificate_thumbprint` y `access_token_hash`. Para Lumen, la lección es la proporcional: protege los ámbitos que mueven dinero (`wallet:deposit`, `wallet:withdraw`) con tokens restringidos al poseedor, y un token de acceso filtrado deja de ser una retirada de fondos filtrada.

Seguro por defecto

Un tema recorre todo este capítulo: la opción segura es la que viene por defecto, y tienes que *desactivarla* explícitamente, no activarla. Eso es deliberado. Los valores de seguridad por defecto que hay que recordar son valores de seguridad por defecto que se olvidan. Vale la pena destacar juntos cuatro de los valores de endurecimiento por defecto de Lumen, porque cada uno cierra un error que, de otro modo, es fácil llevar a producción.

El secreto de marcador de posición falla rápido. Los valores por defecto de PyFly incluyen un secreto de firma literal, `change-me-in-production`. Si lo dejas en su sitio, la raíz de composición **se niega a arrancar**: tanto el `JWTService` simétrico (cuando su filtro está activado) como el servidor de autorización lanzan `INSECURE_SIGNING_SECRET` en lugar de firmar tokens que cualquiera podría falsificar. Y como una clave HS256 más corta que 32 bytes es más débil que el algoritmo al que alimenta, un secreto HMAC demasiado corto falla del mismo modo con `WEAK_SIGNING_SECRET`. No hay camino a producción con una clave adivinable.

La protección CSRF está activada. No la activaste tú; viene activada por defecto. El filtro está *condicionado por cookies* (cookie-gated): solo desafía las peticiones no seguras que de verdad llevan cookies, de modo que el tráfico de la API de Lumen, sin estado y con token Bearer, pasa sin tocarse mientras el panel de cara al navegador queda protegido. Desactívala por servicio con `pyfly.security.csrf.enabled: false`, o endurecéla hasta una imposición estricta y siempre activa con `pyfly.security.csrf.cookie-gated: false`.

PKCE está activado para el flujo de inicio de sesión.

`ClientRegistration.use_pkce` vale `True` por defecto, los clientes públicos (los que no tienen secreto) están *obligados* a usarlo de todos modos, y el paso de autorización del servidor de autorización exige el método S256: no hay forma de registrar un cliente de authorization-code que se salte la prueba de posesión.

La concesión de contraseña del propietario del recurso está desactivada. El flujo heredado ROPC (`grant_type=password`), en el que una app recoge la contraseña del usuario y la cambia por un token, está deshabilitado en los adaptadores de IdP de Keycloak, Cognito y Azure AD a menos que establezcas explícitamente `pyfly.idp.allow-password-grant: true`. La respuesta moderna es el flujo `authorization_code` + PKCE de arriba; ROPC queda detrás de una activación explícita para la rara migración heredada que todavía la necesita.

```
# Las desactivaciones, reunidas en un solo lugar – cada una te aparta de un
# valor por defecto seguro.
pyfly:
  security:
    csrf:
      enabled: false          # desactiva CSRF por completo (servicios solo
                              # sin estado)
      cookie-gated: false    # 0 mantenlo activado, pero imponlo
                              # estrictamente para todos
    idp:
      allow-password-grant: true
                              # reactiva el ROPC heredado (evítalo salvo que te obliguen)
```

La cuestión no es que estos valores por defecto nunca puedan relajarse: cada uno de ellos tiene una desactivación legítima. La cuestión es que relajarlos sea una *línea visible y deliberada en tu configuración*, revisada como cualquier otro cambio, en lugar de una salvaguarda que olvidaste añadir en silencio.

Juntándolo todo — la capa de autenticación de Lumen

El listado siguiente muestra la conexión completa: adaptador del IDP, filtro JWT, reglas a nivel de URL y un almacén de sesiones en Redis para el panel de administración.

lumen/config/security_full.py

```

from pyfly.container import bean, configuration
from pyfly.idp import IdpAdapter, KeycloakIdpAdapter
from pyfly.security.http_security import HttpSecurity

@configuration
class LumenSecurityConfig:

    @bean
    def idp_adapter(self) → IdpAdapter:
        return KeycloakIdpAdapter(
            base_url="https://keycloak.example.com",
            realm="lumen",
            client_id="lumen-backend",
            client_secret="${KEYCLOAK_SECRET}",
        )

    @bean
    def http_security(self) → HttpSecurity:
        hs = HttpSecurity()
        hs.authorize_requests() \
            .request_matchers(
                "/idp/login", "/idp/refresh",
                "/docs", "/openapi.json",
            ).permit_all() \
            .request_matchers(
                "/idp/admin/**"
            ).has_role("ADMIN") \
            .request_matchers(
                "/api/v1/wallets/**"
            ).authenticated() \
            .any_request().permit_all()
        return hs

```

Listado 14.16 — Configuración de seguridad completa

...

Con `pyfly.security.enabled=true`, `pyfly.session.enabled=true` y `pyfly.session.store=redis` en `pyfly.yaml`, la autoconfiguración se encarga de `JWTService`, `BcryptPasswordEncoder`, `SessionFilter` y `RedisSessionStore`. La clase `@configuration` de arriba aporta solo lo que la autoconfiguración no puede inferir: las coordenadas de Keycloak y la política a nivel de URL.

✓ Lo que construiste

Este capítulo abrió la Parte V cerrando la puerta de entrada abierta de Lumen. Tú:

- Usaste `JWTService` para emitir tokens firmados al iniciar sesión y para decodificarlos de vuelta a un `SecurityContext` en cada petición posterior. El claim `exp` es obligatorio: `encode()` lo añade automáticamente usando una marca de tiempo Unix, de modo que nunca necesitas importar `datetime`; los tokens sin `exp` se rechazan en la frontera.
- Añadiste `SecurityMiddleware` a la aplicación Starlette para que toda petición lleve un `SecurityContext` rellenado en el momento en que alcanza un manejador.
- Declaraste la política a nivel de URL con el constructor `HttpSecurity`: un DSL fluido que produce un `HttpSecurityFilter` evaluado antes del despachador de rutas, cubriendo el árbol real `/api/v1/wallets/**` de Lumen y las rutas de administración del IDP.
- Protegiste los endpoints de monederos reales de Lumen en `WalletController` con `@secure`, especificando roles, permisos o expresiones de seguridad completas. Las siete rutas — `open_wallet`, `deposit`, `withdraw`, `list_wallets` (listado paginado), `list_rich_wallets` (listado filtrado), `wallet_balance` y `wallet_detail` — llevan cada una la protección mínima: USER+ADMIN para la

mayoría, el permiso `wallet:deposit` para las mutaciones, solo ADMIN para la vista de detalle completa. Cuando `@secure` rechaza una llamada lanza `SecurityException` (código `AUTH_REQUIRED`) para un llamante no autenticado o `ForbiddenException` (código `FORBIDDEN`) para un llamante autenticado que carece del rol o permiso requerido: ambas se renderizan como HTTP `403` a través del manejador de excepciones. La puerta más amplia `HttpSecurity`, situada delante de esos manejadores, es la que devuelve el `401` escueto para un token ausente o inválido.

- Hasheaste contraseñas con `BcryptPasswordEncoder`, el adaptador por defecto del protocolo `PasswordEncoder`. El factor de coste es ajustable; el hash almacenado se autodescribe; la verificación es segura frente a temporización.
- Gestionaste sesiones del lado del servidor con `HttpSession` y el protocolo conectable `SessionStore`. En desarrollo, `InMemorySessionStore` no requiere dependencias; en producción, `RedisSessionStore` serializa a JSON y deja que Redis gestione el TTL. El `SessionFilter` desplaza el TTL de la cookie en cada petición y la elimina al invalidarse.
- Validaste tokens emitidos por un IdP externo con el **servidor de recursos OAuth2** guiado por configuración (`JWKSTokenValidator`). Un bloque de configuración `pyfly.security.oauth2.resource-server.*` acepta tokens de Keycloak, Microsoft Entra ID y AWS Cognito de serie: verifica la firma `RS256` contra las claves JWKS del IdP, comprueba `iss` / `aud` / `exp` con un desfase de reloj de 60 segundos y mapea los roles, scopes y principal de cada proveedor al mismo `SecurityContext` vía `ClaimMappings`, de modo que `@secure` y `HttpSecurity` quedan inalterados.
- Delegaste la identidad a un proveedor externo vía el puerto `IdpAdapter` y la implementación `KeycloakIdpAdapter`. El `IdpController` autoconfigurado expone inicio de sesión, refresco, cierre de sesión, introspección y gestión de usuarios de administración bajo `/idp` sin código adicional.

- Aprendiste que el actuador y el panel de administración se ejecutan en un **puerto de gestión separado** (`pyfly.management.server.port` , por defecto `9090`) que está **no autenticado por defecto** por paridad con Spring, y que lo aseguras con `pyfly.management.security.enabled: true` (o lo deshabilitas por completo con el puerto `-1`).

Pruébalo tú mismo

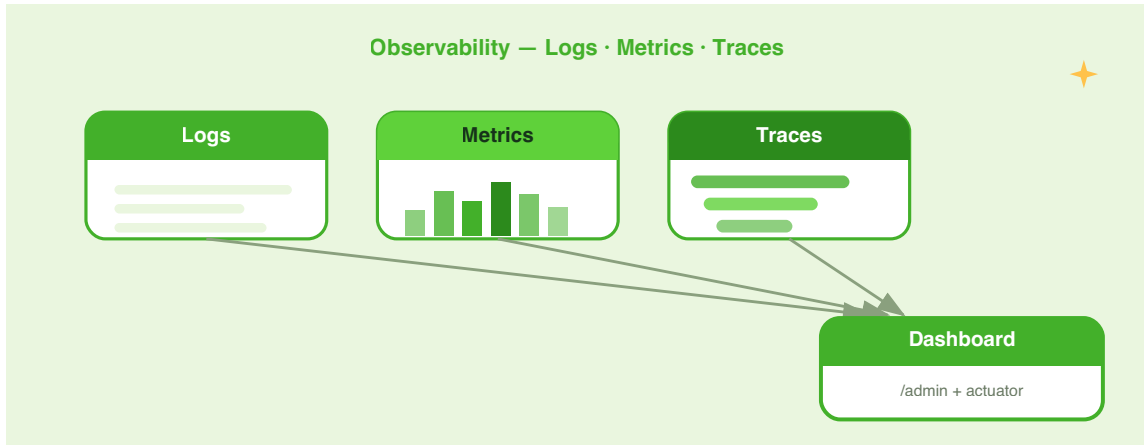
Ejercicio 1 — Jerarquía de roles. Lumen trata actualmente `ADMIN` y `USER` como roles independientes. Añade un rol de "superusuario" `SUPER` que posea implícitamente todos los privilegios de `ADMIN` . Implementa un envoltorio `RoleHierarchy` que pre-expanda los roles antes de almacenarlos en `SecurityContext` , y actualiza `_permissions_for` en el Listado 14.1 para que un token con rol `SUPER` pase toda comprobación `@secure(roles=["ADMIN"])` sin llevar explícitamente el rol `ADMIN` . Escribe una prueba unitaria que cree un `SecurityContext(roles=["SUPER"])` tras la expansión y afirme que `has_any_role(["ADMIN"])` devuelve `True` .

Ejercicio 2 — Control de concurrencia de sesiones. Los usuarios de administración de Lumen no deben estar con sesión iniciada en más de dos dispositivos al mismo tiempo (un requisito habitual de cumplimiento financiero). Activa `pyfly.session.concurrency.enabled: true` con `max-sessions: 2` y `strategy: evict-oldest` en `pyfly.yaml` . Escribe una prueba de integración que use `InMemorySessionStore` y que cree tres sesiones para el mismo `user_id` , llame al `SessionConcurrencyController` y afirme que la sesión más antigua ha sido expulsada mientras las dos más nuevas siguen siendo válidas.

Ejercicio 3 — Adaptador de IDP personalizado. El entorno de staging de Lumen usa un servidor OAuth2 casero. Implementa un `StagingIdpAdapter` que satisfaga `IdpAdapter`, respaldado por un diccionario de usuarios en memoria. El método `login` debería emitir un JWT firmado usando un `JWTService` inyectado a través del constructor. Conéctalo como el bean `IdpAdapter` en una clase `@configuration` etiquetada con `@conditional_on_property("lumen.env", having_value="staging")` y confirma que el bean de producción `KeycloakIdpAdapter` no se crea cuando esa propiedad está activa.

CAPÍTULO 15

Observabilidad y el panel de administración



En el Capítulo 13 rodeaste las rutas críticas de Lumen con cachés y envolviste las llamadas salientes en cortacircuitos (circuit breakers). En el Capítulo 14 aseguraste cada endpoint con autenticación JWT, guardas de rol y sesiones del lado del servidor.

Lumen ya es rápida y segura, pero sigue siendo una caja negra. Cuando un depósito en un monedero tarda tres segundos en producción necesitas saber *dónde* se fueron esos segundos. Cuando un servicio de pagos aguas abajo se degrada necesitas un panel que se ilumine en rojo *antes* de que se avise a tu ingeniero de guardia. Cuando un auditor de cumplimiento pregunta por qué se rechazó una transferencia concreta necesitas registros de log estructurados con suficiente contexto para reconstruir la decisión.

Este capítulo le añade ojos y oídos a Lumen. Los tres pilares de la observabilidad responden a tres preguntas complementarias sobre un sistema en ejecución:

Pilar	Pregunta	Módulo de PyFly
Logging	"¿Qué ocurrió y en qué contexto?"	<code>pyfly.logging</code>
Métricas	"¿Cuánto? ¿Con qué rapidez? ¿Con qué frecuencia?"	<code>pyfly.observability.metrics</code>
Trazas	"¿Qué camino siguió esta petición?"	<code>pyfly.observability.tracing</code>

Sobre esos pilares se asienta el **Actuator**: endpoints de gestión para producción que exponen la salud, el cableado de beans, el entorno y los niveles de logger en vivo, y el **panel de administración**, una interfaz de navegador embebida que lo une todo en un único panel de control.

NOVEDAD EN V26.6.110

Dos cambios de esta versión condicionan cómo accedes a todo lo de este capítulo. Primero, el Actuator y el panel de administración ahora escuchan en un **puerto de gestión separado** — `9090` por defecto — en lugar del puerto de aplicación `8080`. Segundo, ese puerto de gestión está **abierto y sin autenticar por defecto** (el modelo `management.server.port` de Spring Boot), de modo que puedes hacer curl a `http://localhost:9090/actuator/health` de inmediato, y lo blindas con `pyfly.management.security.enabled: true` cuando despliegas. Iremos señalando el puerto correcto en cada paso. Si has leído borradores antiguos que usaban `http://localhost:8080/actuator/...`, cambia el puerto a `9090`.

Por último, verás cómo la **programación orientada a aspectos** (AOP) aplica logging y métricas a cada método de servicio de forma declarativa, sin tocar los propios métodos.

Al final del capítulo Lumen producirá logs JSON estructurados con identificadores de correlación y enmascaramiento automático de PII, emitirá métricas Prometheus recolectadas por cualquier colector estándar, propagará spans de trazas OpenTelemetry a través de las fronteras entre servicios, responderá a las sondas de liveness y readiness de Kubernetes y mostrará todo lo anterior en un panel de configuración cero accesible en `/admin`.

APLICACIONES HTML Y ADMINISTRACIÓN DE MODELOS

El Apéndice E desarrolla el catálogo ejecutable: `ModelAndView`, `Form[T]`, `reverse`, `static_url`, errores HTML y CRUD de entidades/documentos existentes. La administración de datos siempre exige autenticación; en un puerto separado también exige `pyfly.management.security.enabled: true`.

Logging estructurado y redacción de PII

¿Por qué logging estructurado?

Las líneas de log tradicionales tienen este aspecto:

```
[INFO] Order ord-123 created for customer acme corp (email: sales@acme.com)
```

Buscar `ord-123` en Elasticsearch funciona, hasta que cambia el formato. Y que `sales@acme.com` acabe en un fichero de log puede vulnerar tu política de protección de datos sin que tu equipo siquiera lo note.

El **logging estructurado** sustituye la cadena interpolada por un nombre de evento y pares clave-valor explícitos. Los pares se renderizan como JSON en producción y como `clave=valor` legible en desarrollo. Un sistema de agregación de logs ingiere JSON de forma nativa; consultas sobre `wallet_id` u `owner_id` como campos de primera clase, con independencia del formato del mensaje.

get_logger

PyFly expone una única función factoría que devuelve un logger estructurado respaldado por `structlog` (cuando el extra `observability` está instalado) o, en caso contrario, un envoltorio de la biblioteca estándar sin dependencias. Ambos aceptan la misma firma de llamada.

❗ JERGA: FUNCIÓN FACTORÍA

Una *función factoría* no es más que una función cuya labor es construir y devolver un objeto configurado. `get_logger("lumen.wallet")` hace el cableado por ti: nunca construyes un logger a mano. La cadena que pasas (`"lumen.wallet"`) es el *nombre del logger*; por convención es la ruta de módulo con puntos, y es a lo que apuntan las anulaciones de nivel como `lumen.wallet: DEBUG`.

Construyamos un pequeño ejemplo que puedas ejecutar y luego pasemos al código real del monedero. Sigue estos pasos.

Paso 1 — Crea un módulo de demostración desechable. Dentro de tu proyecto Lumen, crea `src/lumen/logging_demo.py` con el contenido del listado siguiente. (Es un fichero de prueba para aprender; bórralo después: el logging de verdad vive dentro de los manejadores más adelante en el capítulo.)

lumen/logging_demo.py

```

from pyfly.logging import get_logger

logger = get_logger("lumen.wallet")

logger.info("wallet_opened", wallet_id="wlt-001", owner_id="usr-42")
logger.warning("balance_low", wallet_id="wlt-001", remaining=300)
logger.error(
    "deposit_rejected",
    wallet_id="wlt-001",
    reason="insufficient_funds",
)

```

Listado 15.1 — Uso del logger estructurado

Paso 2 — Ejecútalo. Ejecuta el módulo directamente:

```
uv run python -m lumen.logging_demo
```

En desarrollo, con `format: console`, la salida se lee de forma natural:

```

10:30:00 [info    ] wallet_opened   wallet_id=wlt-001 owner_id=usr-42
10:30:01 [warning ] balance_low    wallet_id=wlt-001 remaining=300
10:30:02 [error    ] deposit_rejected wallet_id=wlt-001
reason=insufficient_funds

```

Fíjate en la forma: el *primer* argumento (`"wallet_opened"`) es el **nombre del evento**, no una frase, y todo lo que viene después es un par `clave=valor`. No hay formateo de cadenas, ni f-string, ni `%s`. Ese es el sentido del logging estructurado: el nombre del evento permanece estable mientras los campos transportan los datos variables.

En producción, con `format: json`, cada línea es un objeto JSON autocontenido:

```
{ "event": "wallet_opened", "wallet_id": "wlt-001", "owner_id": "usr-42",
  "timestamp": "2026-06-07T10:30:00Z", "level": "info",
  "logger": "lumen.wallet" }
```

Configura el logging en `pyfly.yaml`:

```
pyfly:
  logging:
    level:
      root: INFO
      lumen.wallet: DEBUG
      sqlalchemy.engine: WARNING
    format: console          # console | json | logfmt
```

`level.<name>` anula el nivel raíz para cualquier logger cuyo nombre empiece por ese prefijo. `sqlalchemy.engine: WARNING` silencia los logs de consultas sin tocar tu código. Una variable de entorno `PYFLY_LOGGING_LEVEL_ROOT=WARNING` anula la clave de configuración, lo cual resulta útil para builds de staging.

Qué acaba de pasar. Llamaste a una sola función, `get_logger`, y obtuviste un logger que admite campos estructurados `clave=valor`. El ajuste `format` decide cómo se renderizan esos campos: `console` para humanos en tu terminal, `json` para máquinas en producción. No escribiste nada de código de handler, formateador ni `append`: PyFly los instaló en el logger raíz al arrancar. Cambia `format: console` por `format: json` en `pyfly.yaml` y vuelve a ejecutar la demostración para ver los mismos tres eventos emitidos como un objeto JSON por línea, listos para que un agregador de logs los ingiera.

💡 ¿POR QUÉ NO USAR DIRECTAMENTE EL LOGGING DE LA BIBLIOTECA ESTÁNDAR?

`logging.getLogger("x").info("event", wallet_id="wlt-001")` lanza `TypeError: la biblioteca estándar rechaza los argumentos con nombre`. `get_logger` garantiza que la firma estructurada funcione sea cual sea el adaptador activo.

Identificadores de correlación

Los **identificadores de correlación** enlazan cada línea de log emitida durante una única petición HTTP. PyFly vincula un `transaction_id` al contexto asíncrono actual de forma automática mediante `TransactionIdMiddleware`. Tus manejadores pueden vincular campos adicionales —como el usuario autenticado— para que esos campos fluyan a través de todas las llamadas de log subsiguientes sin pasarse de forma explícita:

`lumen/wallet/handler.py`

```
import structlog

from pyfly.logging import get_logger

logger = get_logger("lumen.wallet")

async def handle_deposit(wallet_id: str, amount: int, owner_id: str) → dict:
    structlog.contextvars.bind_contextvars(
        wallet_id=wallet_id,
        owner_id=owner_id,
    )

    logger.info("deposit_started", amount=amount)
    # ... business logic ...
    result = {"wallet_id": wallet_id, "new_balance": 1350}
    logger.info("deposit_completed", new_balance=result["new_balance"])

    structlog.contextvars.unbind_contextvars("wallet_id", "owner_id")
    return result
```

Listado 15.2 — Vinculación de contexto de correlación

Cada llamada a `logger.*` dentro de `handle_deposit` —incluidas las llamadas en lo más hondo de métodos de servicio aguas abajo— transporta automáticamente `wallet_id` y `owner_id` sin más fontanería.

Redacción de PII

PII son las siglas de *información de identificación personal* (personally identifiable information): correos electrónicos, números de tarjeta, números de documento nacional de identidad y similares. La **redacción de PII** está activada por defecto. Antes de que cualquier registro de log alcance un handler de salida, PyFly escanea el mensaje renderizado en busca de correos, números de tarjeta de crédito, IBAN, SSN, JWT, tokens bearer y credenciales en URL. Los patrones detectados se sustituyen por `<EMAIL>`, `<CREDIT_CARD>`, etc.

Puedes comprobarlo tú mismo en unos segundos. Añade una línea a la demostración del Paso 1:

```
logger.info("contact_logged", email="alice@example.com")
```

Ejecuta `uv run python -m lumen.logging_demo` de nuevo. La salida muestra el valor ya enmascarado: nunca tuviste que activarlo:

```
10:30:03 [info    ] contact_logged email=<EMAIL>
```

Esa pasada de redacción se ejecuta para *cada* logger del proceso, incluidos los que están dentro de bibliotecas de terceros, razón por la cual atrapa fugas que tú no escribiste.

El motor de expresiones regulares viene con cada instalación. El motor NER respaldado por Presidio —que además atrapa nombres y direcciones en texto libre— está disponible mediante el extra `[pii]` y se activa automáticamente cuando se instala:

```
uv add "pyfly[observability,pii]"
python -m spacy download en_core_web_sm  # lighter model; lg for higher
recall
```

Configura la redacción en `pyfly.yaml`:

```
pyfly:
  logging:
    redaction:
      enabled: true           # default; set false to disable entirely
      engine: auto           # regex | presidio | auto (presidio if installed)
      mask: placeholder      # placeholder (<EMAIL>) | partial (****@acme.com)
      deny-fields:
        - password
        - token
        - secret
      presidio:
        score-threshold: 0.6
        languages: [en, es]
```

`deny-fields` enumera los *nombres* de los campos de log estructurado cuyos valores se sustituyen incondicionalmente por `<REDACTED>`. Úsalo para campos como `password`, donde sabes que el valor es sensible sin inspeccionar el contenido.

🍃 EQUIVALENCIA CON SPRING

Spring Boot no incluye redacción de PII integrada; los equipos integran `MaskingMessageConverter` de Logback o appenders personalizados de forma manual. La redacción de PyFly se aplica a *todos* los loggers —incluidas las bibliotecas de terceros— mediante un único `ProcessorFormatter` / `RedactionFilter` instalado en el handler raíz. No hace falta configuración por biblioteca.

Appender de fichero con rotación

Cuando los logs van a un fichero en lugar de a stdout, configura la rotación en

`pyfly.yaml`:

```
pyfly:
  logging:
    file:
      name: lumen.log
      path: ./logs
    rolling:
      max-size: 50MB
      max-history: 14
```

PyFly escribe en `./logs/lumen.log` y rota a los 50 MB, conservando 14 ficheros rotados antes de descartar el más antiguo. La misma pasada de redacción de PII se aplica a la salida a fichero.

Métricas

El MetricsRegistry

Una **métrica** es un número que muestreas a lo largo del tiempo: un recuento de depósitos, una latencia en segundos, una cifra de memoria. **Prometheus** es la base de datos de métricas de código abierto de facto; *recolecta* (lee periódicamente) un endpoint HTTP que tu aplicación expone y almacena los números. `MetricsRegistry` es la pequeña puerta de entrada de PyFly a ese mundo: un fino envoltorio sobre la biblioteca `prometheus_client` que garantiza que cada nombre de métrica se registre solo una vez. Las llamadas duplicadas a `counter()` o `histogram()` con el mismo nombre devuelven la métrica existente en lugar de lanzar un `ValueError`. Inyéctalo desde el contenedor de DI (autoconfigurado cuando `prometheus_client` está instalado) o créalo manualmente:

lumen/observability/metrics.py

```
from pyfly.observability import MetricsRegistry

registry = MetricsRegistry()

deposits_total = registry.counter(
    name="lumen.deposits.total",
    description="Deposit operations completed",
    labels=["status"],
)

deposit_duration = registry.histogram(
    name="lumen.deposits.duration",
    description="Deposit processing time in seconds",
    labels=["status"],
    buckets=(0.01, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0),
)
```

Listado 15.3 — Creación de métricas

`counter()` y `histogram()` devuelven objetos nativos `prometheus_client.Counter` y `prometheus_client.Histogram`, de modo que cada herramienta del ecosistema Prometheus —paneles de Grafana, reglas de alerta, reglas de grabación— funciona sin modificación.

Método	Devuelve	Sirve para
<code>registry.counter(name, description, labels)</code>	<code>Counter</code>	Recuentos monótonamente crecientes
<code>registry.histogram(name, description, labels, buckets)</code>	<code>Histogram</code>	Duraciones, tamaños, percentiles de latencia
<code>registry.counter(...)</code> llamado de nuevo	El mismo <code>Counter</code>	Deduplicación segura

@timed — histograma de duración automático

`@timed` registra cuánto tarda en ejecutarse una función asíncrona o síncrona, usando un histograma etiquetado. Funciona sobre cualquier invocable y añade automáticamente las etiquetas `class`, `method` y `exception`.

❗ JERGA: COUNTER FRENTE A HISTOGRAM

Un **counter** (contador) solo crece: responde a "¿cuántos?" (depósitos atendidos, errores producidos). Un **histogram** (histograma) distribuye en cubetas los *valores* observados: responde a "¿cuánto tarda?" o "¿cómo de grande?" y permite a Prometheus calcular percentiles (latencia p95). Regla general: cuenta eventos con un counter; mide duraciones y tamaños con un histogram.

Ahora cableemos la primera métrica real en Lumen. Aquí está el manejador de depósitos que construiste en capítulos anteriores; el único cambio es el decorador sobre `do_handle`.

Paso 1 — Importa los ayudantes de métricas. En la parte superior de `src/lumen/core/services/wallets/deposit_funds_handler.py`, añade `MetricsRegistry` y `timed` al import de `pyfly.observability`.

Paso 2 — Crea un registro a nivel de módulo. Añade `registry = MetricsRegistry()` por encima de la clase. Como el registro deduplica por nombre, compartir uno por módulo es seguro.

Paso 3 — Decora `do_handle`. Coloca `@timed(...)` por encima de `@transactional()` para que el temporizador envuelva toda la unidad de trabajo transaccional.

lumen/core/services/wallets/deposit_funds_handler.py

```

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.data.relational.sqlalchemy import LockMode, transactional
from pyfly.domain import AggregateNotFound
from pyfly.eda import EventPublisher
from pyfly.observability import MetricsRegistry, timed

from lumen.core.mappers.wallet_mapper import to_aggregate, to_entity
from lumen.core.services.wallets.deposit_funds_command import DepositFunds
from lumen.core.services.wallets.event_publishing import publish_domain_events
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository

registry = MetricsRegistry()

@command_handler
@service
class DepositFundsHandler(CommandHandler[DepositFunds, int]):
    """Credit funds to an existing wallet; returns the new balance."""

    def __init__(
        self,
        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
    ) → None:
        super().__init__()
        self._repository = repository
        self._events = events
        self._session_factory = session_factory

    @timed(registry, "lumen.deposit.duration", "Deposit handler latency")
    @transactional()
    async def do_handle(self, command: DepositFunds) → int:

```

```

entity = await self._repository.find_by_id(
    command.wallet_id, lock=LockMode.PESSIMISTIC_WRITE
)
if entity is None:
    raise AggregateNotFound("Wallet", command.wallet_id)
wallet = to_aggregate(entity)
wallet.deposit(Money(amount=command.amount, currency=wallet.currency))
await self._repository.upsert(to_entity(wallet))
await publish_domain_events(self._events, wallet.clear_events())
return wallet.balance.amount

```

Listado 15.4 — `@timed` en `DepositFundsHandler`

El decorador captura `start = time.perf_counter()`, llama a la función y observa el tiempo transcurrido en el bloque `finally`. La etiqueta `exception` es `"none"` en caso de éxito y el nombre del tipo de excepción en caso de fallo, de modo que puedes desglosar la latencia por resultado en Grafana. Las etiquetas `class` y `method` se derivan automáticamente del nombre cualificado de la función.

Los nombres de los histogramas siguen la convención dot.case de Micrometer:

`"lumen.deposit.duration"` se convierte en `lumen_deposit_duration_seconds` en Prometheus, con un sufijo `_seconds` añadido si no está presente.

Paso 4 — Expón el endpoint de Prometheus. Por defecto el actuador expone en web solo `health` e `info` (consulta "El puerto de gestión" más adelante). Añade `prometheus` a la lista de exposición en `pyfly.yaml` para que aparezca el endpoint de recolección:

```

pyfly:
  management:
    endpoints:
      web:
        exposure:
          include: "health,info,prometheus"

```

Paso 5 — Ejecútalo y observa cómo aparece la métrica. Arranca Lumen, lanza un depósito a través de la API y luego recolecta el puerto de gestión.

```
# Terminal 1 – start the app (business API on 8080, management on 9090)
uv run pyfly run --server uvicorn

# Terminal 2 – open a wallet, then deposit into it
WALLET=$(curl -s -X POST localhost:8080/api/v1/wallets \
  -H 'Content-Type: application/json' \
  -d '{"owner_id":"usr-42","currency":"EUR"}' \
  | python -c "import sys,json;print(json.load(sys.stdin)['wallet_id'])")
curl -s -X POST localhost:8080/api/v1/wallets/$WALLET/deposit \
  -H 'Content-Type: application/json' -d '{"amount":1350}'

# Now scrape the metric – note the MANAGEMENT port 9090, not 8080
curl -s localhost:9090/actuator/prometheus | grep lumen_deposit_duration
```

Deberías ver líneas de histograma como estas (tus números diferirán):

```
lumen_deposit_duration_seconds_bucket{class="DepositFundsHandler",method="do_handle",exception="none"} 1.0
lumen_deposit_duration_seconds_count{class="DepositFundsHandler",method="do_handle",exception="none"} 1.0
lumen_deposit_duration_seconds_sum{class="DepositFundsHandler",method="do_handle",exception="none"} 0.013
```

La línea `_count` confirma una observación; la línea `_sum` es el total de segundos consumidos. Si `grep` no encuentra nada, es que aún no has lanzado un depósito: la métrica se crea de forma perezosa en la primera llamada.

@counted — contador de invocaciones automático

`@counted` incrementa un contador en cada llamada a la función. El `GetBalanceHandler` de Lumen encaja a la perfección: cada lectura de saldo incrementa el contador, etiquetado por resultado:

lumen/core/services/wallets/get_balance_handler.py

```

from pyfly.container import service
from pyfly.cqrs import QueryHandler, query_handler
from pyfly.observability import MetricsRegistry, counted

from lumen.core.mappers.wallet_mapper import entity_to_balance_dto
from lumen.core.services.wallets.get_balance_query import GetBalance
from lumen.interfaces.dtos.v1.balance_dto import BalanceDto
from lumen.models.repositories.wallet_repository import WalletRepository

registry = MetricsRegistry()

@query_handler
@service
class GetBalanceHandler(QueryHandler[GetBalance, BalanceDto | None]):

    def __init__(self, repository: WalletRepository) → None:
        super().__init__()
        self._repository = repository

    @counted(registry, "lumen.balance.reads", "Balance queries served")
    async def do_handle(self, query: GetBalance) → BalanceDto | None:
        entity = await self._repository.find_by_id(query.wallet_id)
        return entity_to_balance_dto(entity) if entity is not None else None

```

Listado 15.5 — @counted en GetBalanceHandler

En caso de éxito el contador se incrementa con las etiquetas

`class="GetBalanceHandler"`, `method="do_handle"`, `result="success"` y `exception="none"`. En caso de fallo usa `result="failure"` y `exception=<TypeName>`, y luego vuelve a lanzar la excepción original. El nombre del contador recibe automáticamente un sufijo `_total` en Prometheus, según la convención de nombres.

Puedes apilar ambos decoradores en el mismo método. El siguiente listado muestra el `WithdrawFundsHandler` de Lumen cronometrado y contado simultáneamente:

```
lumen/core/services/wallets/withdraw_funds_handler.py
```

```
from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.data.relational.sqlalchemy import LockMode, transactional
from pyfly.domain import AggregateNotFound
from pyfly.eda import EventPublisher
from pyfly.observability import MetricsRegistry, counted, timed

from lumen.core.mappers.wallet_mapper import to_aggregate, to_entity
from lumen.core.services.wallets.event_publishing import publish_domain_events
from lumen.core.services.wallets.withdraw_funds_command import WithdrawFunds
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository

registry = MetricsRegistry()

@command_handler
@service
class WithdrawFundsHandler(CommandHandler[WithdrawFunds, int]):
    """Debit funds from a wallet; returns the new balance in minor units."""

    def __init__(
        self,
        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
    ) → None:
        super().__init__()
        self._repository = repository
        self._events = events
        self._session_factory = session_factory

    @timed(registry, "lumen.withdrawal.duration", "Withdrawal latency")
    @counted(registry, "lumen.withdrawals", "Withdrawal attempts")
    @transactional()
```

```

async def do_handle(self, command: WithdrawFunds) → int:
    entity = await self._repository.find_by_id(
        command.wallet_id, lock=LockMode.PESSIMISTIC_WRITE
    )
    if entity is None:
        raise AggregateNotFound("Wallet", command.wallet_id)
    wallet = to_aggregate(entity)
    wallet.withdraw(Money(amount=command.amount,
currency=wallet.currency))
    await self._repository.upsert(to_entity(wallet))
    await publish_domain_events(self._events, wallet.clear_events())
    return wallet.balance.amount

```

Listado 15.6 — Apilando `@timed` y `@counted`

`amount` es un `int` en unidades menores (p. ej. 1050 = 10,50 €); el objeto de valor `Money` impone el tipo. Cada invocación produce tanto una observación de histograma como un incremento de contador.

Qué acaba de pasar. Añadiste medición transversal a tres manejadores sin cambiar nada salvo la pila de decoradores. `@timed` responde a "¿cuánto tardó?", `@counted` responde a "¿con qué frecuencia y tuvo éxito?", y apilarlos te da ambas cosas gratis. Los cuerpos de los manejadores —la lógica de negocio en sí— nunca mencionan métricas. Tras ejecutar unos cuantos depósitos y retiradas puedes recolectar `localhost:9090/actuator/prometheus` de nuevo y ver cómo `lumen_withdrawals_total` y `lumen_balance_reads_total` van subiendo.

Endpoint de recolección de Prometheus

El actuador (que se trata en la siguiente sección) expone el registro de métricas para su recolección. Cuando el actuador está habilitado y `prometheus_client` está instalado, se montan dos endpoints automáticamente sin código adicional:

- `GET /actuador/metrics` — JSON compatible con Micrometer que enumera todos los nombres de métrica
- `GET /actuador/prometheus` — formato de exposición de texto estándar para la recolección

Apunta tus `scrape_configs` de Prometheus a `/actuador/prometheus` y todas las métricas de `MetricsRegistry` aparecerán junto a las métricas de proceso integradas (CPU, memoria, hilos, GC). Por defecto el actuador escucha en el **puerto de gestión** (`9090`), no en el puerto de aplicación (`8080`), así que el destino de recolección es `http://<host>:9090/actuador/prometheus`. Consulta "El puerto de gestión" más adelante.

🍃 EQUIVALENCIA CON SPRING

`MetricsRegistry` refleja el `MeterRegistry` de Spring procedente de Micrometer. `@timed` corresponde al `@Timed` de Spring y `@counted` al `@Counted`. Los nombres `dot.case` (`lumen.deposit.duration`) coinciden con la convención de Micrometer que el `/actuador/metrics` de Spring Boot Actuator también expone.

Observabilidad de la capa de servidor

NOVEDAD EN V26.6.113

Hasta ahora toda la observabilidad de este capítulo ha sido de la **capa de aplicación**: el `MetricsFilter` que mide `http_server_requests_seconds`, los filtros de trazas/ correlación y `process_metrics`. Esta versión añade métricas sobre el **servidor en sí** — el servidor ASGI que ejecuta tu aplicación (uvicorn, granian o hypercorn)— para que puedas ver conexiones, peticiones en curso, workers y tiempo de actividad junto a las métricas de negocio.

Todas estas métricas se escriben en el mismo registro de Prometheus y se exponen automáticamente en `/actuator/prometheus`, sin código adicional. Tres mecanismos cooperan para producirlas:

1. **Un middleware ASGI puro** (`ServerMetricsASGIMiddleware`, en `pyfly/web/adapters/starlette/asgi_server_metrics.py`) envuelve la aplicación en la **capa más externa** y es la fuente **primaria**: corre en cada worker, para cada servidor y cualquier número de workers. Emite `server_active_connections`, `server_in_flight_requests` y `server_requests_total`.
2. **Un `ServerMetricsBinder`** (`pyfly/observability/server_metrics.py`), arrancado desde el lifespan ASGI dentro del worker (junto a `register_process_metrics` y el `ManagementServer`), emite `server_workers` (a partir de la variable de entorno `_PYFLY_WORKERS` que fija `pyfly run`), `server_uptime_seconds` (desde que el worker se vinculó al socket), `server_started_total`, `server_stopped_total` y, opcionalmente, `server_native_connections`.
3. **Un `ServerStatsPort` de mejor esfuerzo** (`pyfly/server/ports/server_stats.py`) implementado por cada adaptador: en la ruta en proceso `serve_async`, el adaptador de uvicorn aflora su recuento real de conexiones de socket y el total de peticiones desde `uvicorn.Server.server_state`; granian y

hypercorn solo reportan workers y tiempo de actividad (runtime en Rust / sin handle), de modo que ahí los campos de conexión son `None`.

❗ ¿POR QUÉ NO LEER SIMPLEMENTE LAS ESTADÍSTICAS NATIVAS DEL SERVIDOR?

En la ruta de producción `pyfly run`, `uvicorn.run(workers=N)` bifurca subprocesos worker que construyen cada uno su **propio** servidor; el bean del adaptador del worker no es el objeto que está sirviendo, así que `server_state` queda inalcanzable entre procesos. Por eso el middleware ASGI —que sí corre dentro del worker— es la fuente primaria uniforme; las estadísticas nativas son un enriquecimiento de mejor esfuerzo.

Catálogo de métricas

Todos los nombres son nombres de Prometheus y cada medidor lleva las etiquetas `server` (el tipo de servidor) y `worker_pid`:

Métrica	Tipo	Qué mide
<code>server_active_connections</code>	gauge	Conexiones ASGI abiertas (http + websocket); aproximado, no sockets reales (los sockets keep-alive ociosos que retiene el servidor son invisibles a ASGI)
<code>server_in_flight_requests</code>	gauge	Peticiones http que se están atendiendo ahora mismo
<code>server_requests_total</code>	counter	Peticiones http completadas en la capa de servidor
<code>server_workers</code>	gauge	Procesos worker configurados
<code>server_uptime_seconds</code>	gauge	Segundos desde que este worker se vinculó al socket (distinto de <code>process_uptime_seconds</code>)
<code>server_started_total</code> / <code>server_stopped_total</code>	counter	Ciclo de vida del worker
<code>server_native_connections</code>	gauge	Recuento real de conexiones de socket de uvicorn, incluido keep-alive ocioso (solo en la ruta <code>serve_async</code> ; ausente en granian/hypercorn)

Modo multi-worker

Cuando `workers > 1`, el **modo multiproceso de `prometheus_client`** se activa automáticamente: `pyfly run` fija `PROMETHEUS_MULTIPROC_DIR` antes de bifurcar los workers (`pyfly/observability/multiprocess.py`), cada worker escribe sus ficheros mmap y `/actuator/prometheus` agrega todos los workers mediante `MultiProcessCollector`. Así, una sola recolección refleja **todos** los workers.

⚠ LIMITACIÓN DEL MODO MULTIPROCESO

El modo multiproceso solo agrega valores de `Counter`, `Gauge`, `Histogram` y `Summary`. Los colectores Python personalizados (las métricas `process_*` y `system_*`) **no** se agregan entre workers. Los medidores `server_*` y `http_server_requests_*` sí se agregan correctamente.

Configuración

```
pyfly:
  server:
    observability:
      enabled: true           # default; lo activan los starters web y
core                          # default
      sample-interval-seconds: 5.0
      access-log: false      # default; logging de acceso nativo opt-
in
```

`pyfly.server.observability.enabled` está activado por defecto por los starters web y core, igual que `pyfly.observability.metrics.enabled`. Requiere el extra `observability` (`prometheus_client`); sin él, degrada a no-op.

Exposición y panel

Los medidores `server_*` aparecen en `/actuator/prometheus` y `/actuator/metrics`. El panel de administración gana una nueva sección **Observability** en vivo (dentro de Monitoring): tarjetas de estadística (workers, tiempo de actividad, conexiones activas, peticiones en curso, peticiones/segundo), gráficos en movimiento, una tabla de desglose por worker y enlaces a las vistas de Metrics y Traces. Está respaldada por `GET /admin/api/observability` y el SSE `/admin/api/sse/observability`.

❗ ALCANCE: SOLO ASGI POR AHORA

Esta versión **no** añade gunicorn: la pila sigue siendo asíncrona, solo ASGI (granian > uvicorn > hypercorn). Aun así, el diseño de `ServerStatsPort` y del modo multiproceso está preparado para gunicorn de cara a un futuro adaptador.

Stack local — Prometheus y Grafana

Para que puedas ver estas métricas en vivo, `docker-compose.yml` incorpora servicios de **prometheus** y **grafana** que recolectan `/actuator/prometheus` (la configuración vive en `ops/prometheus/prometheus.yml`). Levanta el stack, lanza unas cuantas peticiones contra la API de negocio y observa cómo `server_in_flight_requests` y `server_requests_total` se mueven en Grafana.

Trazas distribuidas

@span — decorador de span de OpenTelemetry

❗ JERGA: TRACE, SPAN, OPENTELEMETRY

Un **span** es un paso de trabajo único, cronometrado y con nombre —"obtener el monedero", "persistir el depósito"—. Un **trace** (traza) es el árbol completo de spans de una única petición, de la raíz a las hojas. **OpenTelemetry** (a menudo "OTel") es el estándar neutral respecto a proveedores para producir trazas; una vez que tus spans hablan OTel, cualquier visor compatible —Jaeger, Tempo, Honeycomb— puede mostrarlos. PyFly emite spans OTel, así que nunca quedas atado a una sola herramienta.

`@span` envuelve una función asíncrona o síncrona en un span de OpenTelemetry.

Cada span es una unidad de trabajo cronometrada y con nombre. Los spans se anidan automáticamente gracias a la propagación de contexto de OpenTelemetry, de modo que una función decorada con `@span` llamada desde dentro de otra función decorada con `@span` produce una relación padre-hijo en tu visor de trazas:

lumen/wallet/service.py

```
from pyfly.observability import span

class DepositFundsHandler:

    @span("deposit-funds")
    async def do_handle(self, command):
        balance = await self._fetch_wallet(command.wallet_id)
        await self._persist_deposit(command.wallet_id, command.amount)
        return balance + command.amount

    @span("fetch-wallet")
    async def _fetch_wallet(self, wallet_id: str) → int:
        # ... repository.find(wallet_id) ...
        return 1000

    @span("persist-deposit")
    async def _persist_deposit(self, wallet_id: str, amount: int) → None:
        # ... repository.add(wallet) ...
        pass
```

Listado 15.7 — `@span` en métodos de manejador CQRS

En un visor de trazas esto aparece así:

```
deposit-funds [120 ms]
  +-- fetch-wallet [15 ms]
  +-- persist-deposit [90 ms]
```

`@span` crea un tracer llamado "pyfly" mediante `trace.get_tracer("pyfly")`.

Cuando la función decorada lanza, el span registra automáticamente el error: pone el estado en `ERROR`, llama a `current_span.record_exception(exc)` y luego vuelve a lanzar para que los llamantes vean la excepción original sin modificar. Las funciones síncronas se admiten de forma idéntica: no hay `await` en el lado decorado.

Autoconfiguración de OpenTelemetry

PyFly cablea un `TracerProvider` con un `BatchSpanProcessor` de forma automática cuando `opentelemetry-api` y `opentelemetry-sdk` están instalados:

```
uv add opentelemetry-api opentelemetry-sdk opentelemetry-exporter-otlp
```

Configura el exportador en `pyfly.yaml`:

```
pyfly:
  observability:
    tracing:
      enabled: true
      service-name: "${pyfly.app.name}"
      exporter: otlp
      otlp:
        endpoint: "http://localhost:4318"
```

Ejecútalo — observa spans sin un colector. Levantar Jaeger o Tempo es excesivo mientras estás aprendiendo. Configura en su lugar el **exportador de consola**, que imprime cada span por stdout. Lumen viene con `tracing.enabled: false`; actívalo y elige `console`:


```
pyfly:
  observability:
    tracing:
      enabled: true
      exporter: console
```

Reinicia con `uv run pyfly run --server uvicorn`, lanza un depósito a través de la API como en el paso de métricas y observa la terminal. Cada `@span` imprime un bloque JSON que muestra su nombre, el id del span padre y el tiempo transcurrido: la misma estructura padre-hijo que esboza el diagrama anterior, solo que en forma de texto. Vuelve a `exporter: otlp` (o elimina la anulación) en cuanto tengas un colector real.

Reglas de selección del exportador:

- `exporter: otlp` — usa OTLP/HTTP; requiere `opentelemetry-exporter-otlp`
- `exporter: console` — imprime los spans por stdout; útil para depuración local
- `exporter: none` — registra los spans pero los descarta (útil en pruebas)
- Sin definir — selecciona OTLP automáticamente si está configurado `pyfly.observability.tracing.otlp.endpoint` o la variable de entorno `OTEL_EXPORTER_OTLP_ENDPOINT`; en caso contrario registra una única línea informativa y descarta los spans en silencio

💡 TRACERPROVIDER PERSONALIZADO

Si necesitas exportación gRPC o una configuración del SDK no estándar, registra tu propio `TracerProvider` antes de que PyFly arranque: `trace.set_tracer_provider(...)`. PyFly detecta un proveedor global existente y omite la autoconfiguración.

Propagación de contexto — entrante y saliente

Los spans permanecen correlacionados *dentro de* un proceso automáticamente.

Mantener la misma traza *a través de* varios servicios requiere extraer el contexto de traza aguas arriba de las cabeceras HTTP entrantes e inyectar el contexto actual en cada llamada saliente.

PyFly gestiona ambos extremos sin nada de código por manejador.

Entrante — TracingFilter:

`TracingFilter` se cablea en la cadena de filtros de Lumen inmediatamente después de `CorrelationFilter` por `create_app()`. Para cada petición lee la cabecera W3C `traceparent`, abre un span SERVER como hijo del contexto aguas arriba y mantiene ese span activo durante toda la vida de la petición. Cada `@span` creado durante la petición —y cada línea de log— pertenece a la traza distribuida del llamante:

```
# Simplified view of what TracingFilter does per-request:
parent = extract_context(request.headers) # parse W3C traceparent
with tracer.start_as_current_span(
    f"{request.method} {request.url.path}",
    context=parent,
    kind=trace.SpanKind.SERVER,
) as span:
    response = await call_next(request)
    span.set_attribute("http.response.status_code", response.status_code)
```

Cuando OpenTelemetry no está instalado, el filtro es un pasarela transparente.

Saliente — HttpxClientAdapter:

`HttpxClientAdapter` llama a `inject_headers()` en cada petición saliente para que los servicios aguas abajo puedan continuar la misma traza:

lumen/client/inventory_client.py

```

from pyfly.client.adapters.httpx_adapter import HttpxClientAdapter

class InventoryClient:

    def __init__(self) → None:
        self._http = HttpxClientAdapter(
            base_url="http://inventory-service:8080"
        )

    async def check_stock(self, sku: str) → dict:
        # The active traceparent is injected automatically into
        # the outbound request headers – no manual plumbing required.
        resp = await self._http.request("GET", f"/skus/{sku}")
        return resp.json()

```

*Listado 15.8 — Propagación de trazas***Los logs llevan trace_id y span_id:**

`StructLogAdapter` registra un procesador que estampa los IDs del span activo en cada registro de log. No hace falta ningún cambio de código: cualquier llamada `get_logger(...)` dentro de un span activo obtiene los campos `trace_id` y `span_id` automáticamente:

```
{
  "event": "deposit_completed",
  "wallet_id": "wlt-001",
  "new_balance": 1350,
  "trace_id": "1a4b3145ed8f2dd11172ee3584123f4a",
  "span_id": "d2a62aaa81b0ad66",
  "timestamp": "2026-06-07T10:30:00Z",
  "level": "info",
  "logger": "lumen.wallet"
}
```

Con `trace_id` en cada registro de log puedes saltar desde una búsqueda en Grafana Loki por `wallet_id=wlt-001` directamente a la vista de traza correlacionada en Tempo, y desde ahí a los gráficos de latencia de Prometheus de esa ventana temporal: los tres pilares unidos por un único identificador.

Los ayudantes de propagación de bajo nivel están disponibles por si alguna vez los necesitas directamente:

```
from pyfly.observability.propagation import (
    extract_context, # parse traceparent from inbound headers
    inject_headers,  # write traceparent into outbound headers
    current_trace_ids, # → (trace_id, span_id) hex, or None
    has_otel,         # True if opentelemetry is importable
)
```

Comprobaciones de salud y el Actuador

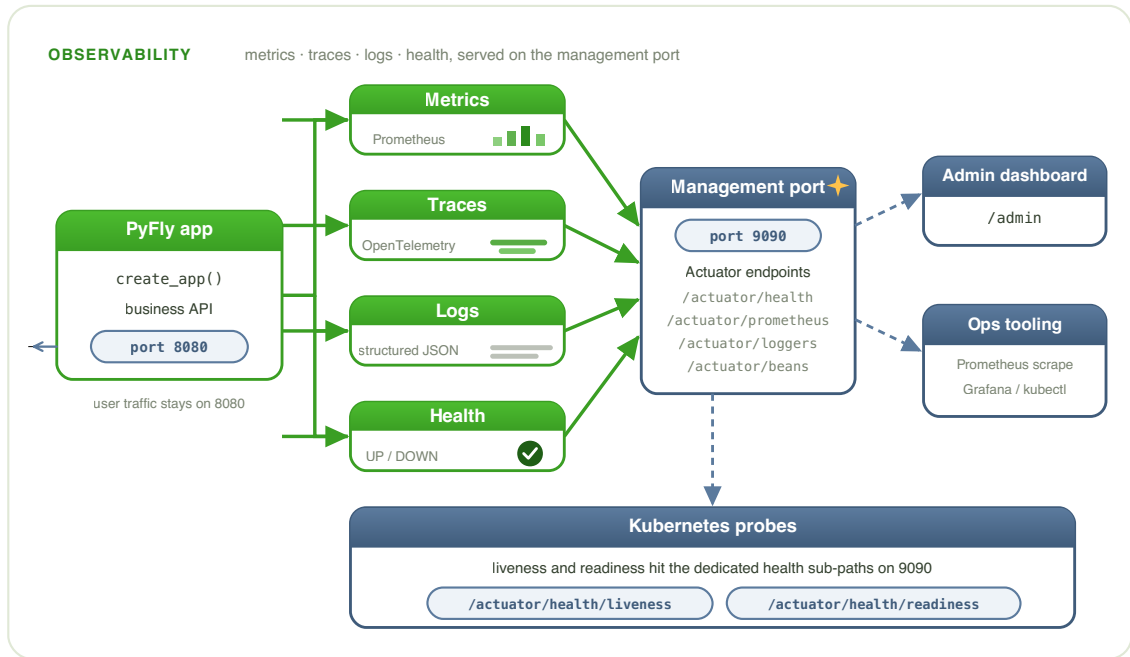


Figura 15.1 — El Actuador expone la salud, los beans, los loggers y las métricas de Prometheus por HTTP. Las sondas de liveness y readiness de Kubernetes acceden a las subrutinas dedicadas.

El **Actuador** le da a Kubernetes y a tu instrumental de operaciones un contrato estable: un conjunto de endpoints de gestión que exponen la salud, el cableado de beans, el estado del entorno y los recolectores de métricas. Lo configuras una vez y cada herramienta, desde `kubectl` hasta Grafana, puede consumirlo sin código a medida.

Habilitar el Actuador

El Actuador está **activado por defecto** cuando hay presente un contexto de PyFly: no tienes que habilitarlo en absoluto para obtener `/actuator/health` y `/actuator/info`. Solo tocas el flag para *desactivarlo* o para ser explícito. Pasa `actuator_enabled=True` a `create_app()`, o establece el flag en `pyfly.yaml`:

```

pyfly:
  management:
    enabled: true          # default; the actuator is on unless you set
false
  app:
    name: lumen
    version: 1.0.0
    description: Lumen wallet service

```

❗ LA CLAVE DE CONFIGURACIÓN HA CAMBIADO

El flag de habilitación ahora es `pyfly.management.enabled`. El antiguo `pyfly.web.actuator.enabled` sigue funcionando como alias heredado, pero el código nuevo debería usar el espacio de nombres `pyfly.management.*`, que es donde también viven los ajustes de puerto, seguridad y exposición de endpoints.

Cuando está habilitado, `create_app()` escanea automáticamente el contenedor de DI en busca de beans `HealthIndicator`, crea un `HealthAggregator`, instancia todos los endpoints integrados y los monta en `/actuator/*`.

Ejecútalo — tu primera comprobación de salud. Lumen ya habilita el actuator, así que basta con arrancar la aplicación y hacer curl al endpoint de salud en el **puerto de gestión**:

```

uv run pyfly run --server uvicorn
curl -s localhost:9090/actuator/health

```

Una aplicación sana devuelve HTTP 200 con:

```

{"status": "UP"}

```

Si obtienes "connection refused", confirma que usaste `9090` (gestión) y no `8080` (negocio). El mismo `/actuator/health` en `8080` devuelve 404: el puerto de negocio deliberadamente no transporta endpoints de gestión.

El puerto de gestión

Por defecto, estos endpoints `/actuator/*` —y el panel `/admin` de la siguiente sección— se sirven en un **puerto de gestión separado** (`9090`), no en el puerto de aplicación (`8080`). Este es el modelo `management.server.port` de Spring Boot: mantener las comprobaciones de salud, la recolección de Prometheus y la consola de administración fuera del puerto público, exponiendo solo la API de negocio a internet mientras el instrumental de operaciones accede a `9090` desde dentro del clúster.

El puerto de gestión es un segundo listener en el mismo proceso (no workers adicionales), por lo que no añade complejidad de despliegue. Configúralo mediante `pyfly.management.server.port` (env `PYFLY_MANAGEMENT_SERVER_PORT`): ponlo **igual** a `pyfly.server.port` para servirlo todo en un solo puerto, o a `-1` para deshabilitar los endpoints web de gestión. Por tanto, un despliegue de Kubernetes apunta las sondas de liveness/readiness y el `ServiceMonitor` de Prometheus al puerto `9090`, y el `Service` / `Ingress` para el tráfico de usuario al `8080`.

⚠ EL PUERTO DE GESTIÓN ESTÁ ABIERTO POR DEFECTO

A partir de v26.6.110 el puerto de gestión está **sin autenticar por defecto**: las rutas `/actuador/*` y `/admin` responden a cualquier llamante que pueda alcanzar `9090`. Esto es intencionado (el modelo de Spring Boot): el puerto está pensado para ser accesible solo desde dentro de tu clúster, tras aislamiento de red, nunca expuesto en la internet pública. Si no puedes garantizar ese aislamiento, activa también los filtros de seguridad de la aplicación para el puerto de gestión:

```
pyfly:
  management:
    security:
      enabled: true
```

Con ese flag activado, la misma autenticación, guardas de rol y reglas CSRF que protegen tu API de negocio también blindan el puerto de gestión.

Por defecto el actuador expone en web solo `health` e `info`, de nuevo en consonancia con Spring Boot, que mantiene los endpoints potencialmente sensibles (`beans`, `env`, `threaddump`, `prometheus`) fuera de la red hasta que des tu consentimiento. Amplía el conjunto con

```
pyfly.management.endpoints.web.exposure.include:
```

```
pyfly:
  management:
    endpoints:
      web:
        exposure:
          include: "health,info,metrics,prometheus,loggers"  # or "*" for all
```

Así pues, los pasos de recolección de Prometheus y de loggers en tiempo de ejecución, más adelante en este capítulo, dan por hecho que has añadido `prometheus` y `loggers` a esta lista. El atajo `*` expone todo y es cómodo en desarrollo.

Endpoints integrados

Endpoint	Método	Descripción
<code>/actuator</code>	GET	Índice estilo HAL de todos los endpoints habilitados
<code>/actuator/health</code>	GET	Estado agregado: <code>UP</code> (200) o <code>DOWN</code> (503)
<code>/actuator/health/liveness</code>	GET	Subruta de la sonda de liveness de Kubernetes
<code>/actuator/health/readiness</code>	GET	Subruta de la sonda de readiness de Kubernetes
<code>/actuator/beans</code>	GET	Todos los beans de DI registrados con estereotipo y ámbito
<code>/actuator/env</code>	GET	Perfiles de configuración activos
<code>/actuator/info</code>	GET	Nombre, versión y descripción de la aplicación
<code>/actuator/loggers</code>	GET, POST	Lista loggers; cambia niveles en tiempo de ejecución
<code>/actuator/metrics</code>	GET	Nombres de métrica en JSON compatible con Micrometer
<code>/actuator/prometheus</code>	GET	Destino de recolección en formato de exposición de texto de Prometheus
<code>/actuator/threaddump</code>	GET	Instantánea de todos los hilos vivos y sus trazas de pila
<code>/actuator/refresh</code>	POST	Desaloja los beans de ámbito refresh; revincula la configuración

HealthIndicator personalizado

Cualquier bean `@component` con un método `async def health(self) → HealthStatus` se descubre y registra automáticamente como indicador de salud. El `WalletRepository` de Lumen es un buen candidato: `count()` emite un ligero `SELECT COUNT(*)` en una unidad de lectura breve propia, sin mutar ningún dato:

lumen/health/indicators.py

```

from sqlalchemy import text
from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

from pyfly.actuator import HealthStatus
from pyfly.container import component

from lumen.models.repositories.wallet_repository import WalletRepository

@Component
class WalletRepositoryHealthIndicator:
    """Checks the wallet store is reachable with a lightweight probe."""

    def __init__(self, repository: WalletRepository) → None:
        self._repository = repository

    async def health(self) → HealthStatus:
        try:
            # count() issues SELECT COUNT(*) – fast, read-only probe.
            total = await self._repository.count()
            return HealthStatus(
                status="UP",
                details={"store": "wallet-repository", "rows": total},
            )
        except Exception as exc:
            return HealthStatus(
                status="DOWN",
                details={"error": str(exc)},
            )

@Component
class DatabaseHealthIndicator:
    """Checks database connectivity via a lightweight SELECT 1."""

    def __init__(
        self, session_factory: async_sessionmaker[AsyncSession]
    )
```

```

) → None:
    self._factory = session_factory

    async def health(self) → HealthStatus:
        try:
            async with self._factory() as session:
                await session.execute(text("SELECT 1"))
            return HealthStatus(
                status="UP",
                details={"type": "postgresql", "pool_active": 3},
            )
        except Exception as exc:
            return HealthStatus(
                status="DOWN",
                details={"error": str(exc)},
            )

```

Listado 15.9 — Beans HealthIndicator

PyFly ya aporta un indicador de base de datos propio a la sonda de preparación (readiness): `SqlAlchemyHealthIndicator`, el bean `db_health_indicator`, que comprueba cada origen de datos del registro con un `SELECT 1` acotado por `pyfly.data.relational.health.timeout` (2 s). `DatabaseHealthIndicator`, más arriba, muestra el contrato; en una aplicación real quédate con el integrado.

`HealthStatus.status` admite cuatro valores: `"UP"`, `"DOWN"`, `"OUT_OF_SERVICE"` o `"UNKNOWN"`. El agregador aplica un orden de severidad (`DOWN > OUT_OF_SERVICE > UP > UNKNOWN`) y devuelve el estado del peor caso entre todos los indicadores. Si el método `health()` de algún indicador lanza, ese indicador se trata como `"DOWN"` con `details={"error": "check failed"}`; la excepción se registra pero no hace caer el endpoint de salud.

Ejecútalo — observa cómo aflora un indicador personalizado. Coloca el listado anterior en `src/lumen/health/indicators.py`, reinicia Lumen y pide el informe de salud detallado (la forma `?show-details`, o simplemente recolectándolo del puerto de gestión):

```
curl -s localhost:9090/actuador/health
```

Una respuesta sana ahora incluye tu componente por su nombre:

```
{
  "status": "UP",
  "components": {
    "WalletRepositoryHealthIndicator": {
      "status": "UP",
      "details": {"store": "wallet-repository", "rows": 42}
    },
    "DatabaseHealthIndicator": {
      "status": "UP",
      "details": {"type": "postgresql", "pool_active": 3}
    }
  }
}
```

No escribiste código de registro: el estereotipo `@component` más el método `async def health()` constituyen todo el contrato. Al arrancar, el actuador escaneó el contenedor, encontró todo lo que parecía un indicador de salud y lo integró en el agregador.

❗ CONSTRUIR SONDAS A MANO — `APP.STATE.PYFLY_HEALTH_AGGREGATOR`

Novedad en v26.6.110: el `HealthAggregator` en vivo es accesible en `app.state.pyfly_health_aggregator` en el objeto de aplicación de Starlette. Es el mismo agregador que usa la ruta `/actuator/health`, tanto si el actuador corre en la aplicación principal como en el puerto de gestión separado. Si alguna vez necesitas una puerta de readiness a medida —por ejemplo, una ruta ASGI personalizada que devuelva 503 hasta que termine un calentamiento único—, puedes leer este agregador directamente o registrar indicadores adicionales en él después de `create_app()`, en lugar de pasar por HTTP. Solo lo expone el adaptador de Starlette.

Cambiar niveles de log en tiempo de ejecución

El endpoint de loggers te permite inspeccionar y cambiar los niveles de log sin reiniciar Lumen, algo inestimable cuando un incidente de producción necesita salida DEBUG de exactamente un paquete. Primero añade `Loggers` a la lista de exposición (`pyfly.management.endpoints.web.exposure.include: "health,info,loggers"`) y luego manéjalo desde el **puerto de gestión** `9090`:

```
# List all loggers with configured and effective levels
curl http://localhost:9090/actuator/loggers

# Enable DEBUG for the wallet module – takes effect immediately
curl -X POST http://localhost:9090/actuator/loggers/lumen.wallet \
  -H "Content-Type: application/json" \
  -d '{"configuredLevel": "DEBUG"}'

# Reset to inherit from parent
curl -X POST http://localhost:9090/actuator/loggers/lumen.wallet \
  -H "Content-Type: application/json" \
  -d '{"configuredLevel": null}'
```

Qué acaba de pasar. Cambiaste el nivel de un logger en un proceso *en marcha*. El POST surtió efecto de inmediato: sin reinicio, sin redesplicue. En un incidente real cambiarías exactamente un paquete a DEBUG, capturarías la salida ruidosa que necesitas y luego harías POST de `null` para devolverlo a como estaba, todo sin perturbar el resto del servicio.

El endpoint usa el vocabulario de niveles de Spring Boot (`OFF` , `ERROR` , `WARN` , `INFO` , `DEBUG` , `TRACE`) y es compatible sin cambios con el instrumental de Spring Boot Actuator.

Endpoint de actuador personalizado

Para exponer un endpoint personalizado, implementa el protocolo `ActuatorEndpoint` y anota la clase con `@component` . PyFly lo descubre durante el arranque del contexto y lo monta en `/actuator/{endpoint_id}` automáticamente:

lumen/actuator/git_info.py

```

from pyfly.container import component

@Component
class GitInfoEndpoint:
    """Exposes build metadata at /actuator/git."""

    @property
    def endpoint_id(self) → str:
        return "git"

    @property
    def enabled(self) → bool:
        return True

    async def handle(self, context=None) → dict:
        return {
            "branch": "main",
            "commit": {
                "id": "5c6f83b",
                "time": "2026-06-07T08:30:00Z",
            },
            "build": {
                "version": "1.0.0",
            },
        }

```

Listado 15.10 — Endpoint de actuador personalizado

Configuración de las sondas de Kubernetes

Apunta la especificación de tu pod a las subrutinas dedicadas de liveness y readiness para que Kubernetes pueda tomar decisiones independientes de reinicio y de tráfico. Como el actuador vive en el puerto de gestión, las sondas apuntan a **9090**, mientras que tu **Service** enruta el tráfico de usuario a **8080**:


```

livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 9090
  initialDelaySeconds: 10
  periodSeconds: 30
readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 9090
  initialDelaySeconds: 5
  periodSeconds: 10

```

Las subrutinas separadas te permiten agrupar indicadores de forma independiente: una migración en curso que degrade la readiness no tiene por qué disparar un reinicio de liveness ni la recreación del contenedor.

🍃 EQUIVALENCIA CON SPRING

El Actuator de PyFly refleja el de Spring Boot. `HealthIndicator`, `HealthStatus`, `HealthAggregator`, `ActuatorEndpoint` y `ActuatorRegistry` corresponden directamente a sus contrapartes de Spring. El endpoint de loggers usa el mismo vocabulario de niveles de Spring Boot y la misma forma de respuesta `configuredLevel / effectiveLevel`, lo que lo hace compatible con Spring Boot Admin y el instrumental compatible con Actuator de fábrica. `MetricsAutoConfiguration` y `MetricsActuatorAutoConfiguration` reflejan la autoconfiguración de Micrometer de Spring Boot: cuando `prometheus_client` está instalado, `/actuator/prometheus` aparece sin ningún cableado manual.

El panel de administración

El **panel de administración** es una interfaz de navegador sin build y sin dependencias servida directamente desde el paquete `pyfly.admin`. Una sola línea de configuración la habilita; navega a `/admin`: sin servidor aparte, sin paso de build de `npm`.

Habilitar el panel

```
pyfly:
  admin:
    enabled: true
    title: "Lumen Admin"
    theme: auto          # auto | light | dark
    refresh_interval: 5000
```

El panel autodescubre beans, indicadores de salud, loggers, tareas programadas, mapeos HTTP, cachés, manejadores CQRS, sagas y métricas del `ApplicationContext` en ejecución. Los presenta en **15 vistas integradas** con actualizaciones en tiempo real mediante Server-Sent Events (SSE): sin WebSocket, sin bucle de sondeo en tu código.

❗ DÓNDE ENCONTRARLO: EL PUERTO DE GESTIÓN

Como el actuador, el panel se sirve en el **puerto de gestión**. Con los valores por defecto de Lumen eso es `http://localhost:9090/admin`, no `8080/admin`. Pon `pyfly.management.server.port` igual a `pyfly.server.port` si prefieres servir el panel en el mismo puerto que tu API. Y recuerda: ese puerto está abierto por defecto; blíndalo con `pyfly.management.security.enabled` o con el propio `require_auth` del panel (mostrado en "Seguridad") antes de exponerlo en cualquier lugar no fiable.

Ejecútalo — abre el panel. Habilítalo en `pyfly.yaml`, arranca Lumen y abre la URL en un navegador:

```
pyfly:
  admin:
    enabled: true
    title: "Lumen Admin"
```

```
uv run pyfly run --server uvicorn
# then visit http://localhost:9090/admin
```

Deberías ver cómo se rellena la vista Overview en un segundo o dos: nombre de la aplicación y tiempo de actividad, una insignia de salud verde y los recuentos de beans agrupados por estereotipo. Lanza unos cuantos depósitos a través de `localhost:8080` y observa cómo los paneles de Salud y Métricas se actualizan en vivo: la página nunca se recarga, porque los datos llegan por SSE.

JERGA: SSE (SERVER-SENT EVENTS)

SSE es un canal de streaming unidireccional: el navegador abre una única conexión HTTP de larga duración y el servidor *empuja* eventos por ella a medida que ocurren. Es más simple que un WebSocket (que es bidireccional) y es justo la herramienta adecuada para un panel que solo necesita *recibir* actualizaciones. No escribes ningún bucle de sondeo; el framework gestiona el flujo.

Vistas integradas

Sección Dashboard:

Vista	Descripción
Overview	Información de la aplicación, tiempo de actividad, insignia de salud, recuentos de beans por estereotipo
Health	Estado de los componentes con insignias UP / DOWN / UNKNOWN codificadas por color; SSE en vivo

Sección Application:

Vista	Descripción
Beans	Todos los beans de DI con estereotipo, ámbito y grafo de dependencias
Environment	Perfiles activos y variables de entorno enmascaradas
Configuration	Árbol de configuración resuelta para todos los espacios de nombres con seguimiento de origen
Loggers	Niveles de logger con interfaz de cambio de nivel en tiempo de ejecución; se admiten TRACE y OFF

Sección Monitoring:

Vista	Descripción
Metrics	CPU, memoria, hilos, GC, tiempo de actividad; métricas de Prometheus opcionales; gráfico de tendencia en vivo
Scheduled Tasks	Todas las tareas <code>@scheduled</code> con expresiones cron y estado
HTTP Traces	Trazas de petición/respuesta con latencia p50/p90/p95/p99 y barra de tasa de error
Log Viewer	Cola en vivo con filtros de nivel, búsqueda y pausa/reanudación

Sección Infrastructure:

Vista	Descripción
Mappings	Todas las rutas HTTP con manejador, parámetros, tipo de retorno y docstring
Caches	Tipo de adaptador, recuento de entradas, desalojo por clave, desalojo masivo
CQRS	Manejadores de comandos y consultas con introspección del pipeline del bus
Transactions	DAG de pasos de saga y cobertura de participantes TCC; recuento en curso

Sección Fleet (modo servidor):

Vista	Descripción
Instances	Todas las instancias de aplicación remotas registradas con su estado de salud

Flujos SSE en tiempo real

El panel nunca sondea el backend con `setInterval`. Abre una única conexión `EventSource` y el servidor empuja los eventos a medida que ocurren:

Endpoint SSE	Nombre de evento	Qué transmite
<code>/admin/api/sse/health</code>	<code>health</code>	Cambio de estado cada vez que cambia la salud agregada
<code>/admin/api/sse/metrics</code>	<code>metrics</code>	Lista completa de nombres de métrica en cada intervalo de refresco
<code>/admin/api/sse/traces</code>	<code>trace</code>	Trazas HTTP individuales a medida que llegan
<code>/admin/api/sse/logfile</code>	<code>log</code>	Nuevos registros de log del buffer circular en memoria
<code>/admin/api/sse/beans</code>	<code>beans</code>	Instantánea del registro de beans en cada intervalo de refresco

El buffer circular del visor de logs guarda 2.000 registros; el buffer circular de las trazas HTTP guarda 500. Las rutas de admin y actuador (`/admin/*` , `/actuator/*`) quedan excluidas de la captura de trazas automáticamente, para que no contaminen el panel de trazas.

Gestión de loggers en tiempo de ejecución

La vista Loggers usa el mismo endpoint `/admin/api/loggers/{name}` que el actuador. Haz clic en una fila de logger para abrir un selector de nivel en línea y envía: el nuevo nivel surte efecto de inmediato, y la interfaz vuelve a obtener los datos para confirmar el cambio. El botón Reset envía `null` para devolver el logger a `NOTSET` (heredar del padre).

Extensión de vista personalizada

Para añadir tu propia vista de barra lateral, implementa `AdminViewExtension` y anota con `@component` . El panel la descubre al arrancar:

lumen/admin/deployment_view.py

```

from pyfly.container import component

@Component
class DeploymentView:
    """Shows deployment metadata in the admin sidebar."""

    @property
    def view_id(self) → str:
        return "deployments"

    @property
    def display_name(self) → str:
        return "Deployments"

    @property
    def icon(self) → str:
        return "upload-cloud"

    async def get_data(self, context=None) → dict:
        return {
            "last_deploy": "2026-06-07T08:00:00Z",
            "version": "1.0.0",
            "environment": "production",
        }

```

Listado 15.11 — Vista de administración personalizada

`view_id` define el fragmento de URL de la barra lateral (`#deployments`), `display_name` aparece en el menú de la barra lateral e `icon` se mapea a un icono de Feather. `get_data()` es llamado por `GET /admin/api/views` y puede consultar el contenedor de DI, una base de datos o cualquier fuente externa.

Seguridad

Restringe el acceso al panel a los operadores en producción:

```
pyfly:
  admin:
    enabled: true
    require_auth: true
    allowed_roles:
      - ADMIN
      - OPS
```

Cuando `require_auth: true`, cada ruta `/admin/api/*` —datos, mutación, SSE y endpoints del registro de instancias— requiere un principal autenticado cuyos roles se solapan con `allowed_roles`. Las peticiones no autenticadas reciben `401`; los usuarios autenticados que carezcan de todos los roles listados reciben `403`. El shell SPA estático sigue siendo público para que el panel pueda arrancar y mostrar el mensaje de error.

Monitorización de flota — modo servidor

Para una flota de instancias de Lumen, ejecuta un servidor de administración dedicado y apunta cada instancia de aplicación hacia él:


```
# Admin server instance
pyfly:
  admin:
    enabled: true
  server:
    enabled: true
    poll_interval: 10000
  instances:
    - name: lumen-1
      url: http://lumen-1:8080
    - name: lumen-2
      url: http://lumen-2:8080
```

```
# Each application instance
pyfly:
  admin:
    enabled: true
  client:
    url: http://admin-server:8080
    auto_register: true
```

`StaticDiscovery` siembra el registro a partir de la lista YAML.

`AdminClientRegistration` registra la instancia al arrancar y la elimina al apagar. Las llamadas HTTP usan `httpx` cuando está disponible y recurren a `urllib.request`; los errores de registro se tragan en silencio, de modo que un servidor de administración inalcanzable nunca aborta el arranque de la aplicación.

🍃 EQUIVALENCIA CON SPRING

PyFly Admin se mapea directamente a Spring Boot Admin. `server.enabled: true` sustituye a `@EnableAdminServer`. `client.url` sustituye a `spring.boot.admin.client.url`. El frontend de Vaadin/React se sustituye por una SPA en JavaScript puro que no requiere ningún instrumental de build. Los flujos SSE sustituyen a las notificaciones por WebSocket de Spring Boot Admin. El Log Viewer integrado sustituye al visor de logfile de Spring Boot Admin respaldado por `/actuator/logfile`; el enfoque de buffer circular de PyFly evita la configuración de ruta de fichero que Spring Boot Admin requiere.

AOP para preocupaciones transversales

¿Qué es la AOP?

La **programación orientada a aspectos** separa las preocupaciones transversales — logging, métricas, seguridad, auditoría— de la lógica de negocio. Sin AOP, cada método de servicio empieza con `logger.info(...)` y termina con `metrics.increment(...)`. Con AOP, escribes esa lógica una sola vez en una clase `@aspect` y la aplicas a cada método coincidente mediante una expresión de pointcut: los propios métodos quedan limpios.

El módulo de AOP de PyFly trae cinco tipos de advice:

Advice	Decorador	Se ejecuta
Before	<code>@before</code>	Antes del método destino
After returning	<code>@after_returning</code>	Después de que el método tenga éxito
After throwing	<code>@after_throwing</code>	Después de que el método lance
After (finally)	<code>@after</code>	Siempre, con éxito o fallo
Around	<code>@around</code>	Envuelve toda la llamada; debe llamar a <code>jp.proceed()</code>

@aspect — declarar un aspecto

❗ JERGA: ASPECTO, ADVICE, POINTCUT, WEAVING

Cuatro palabras viajan juntas en AOP. Un **aspecto** es la clase que agrupa una preocupación transversal. El **advice** es una pieza concreta de comportamiento dentro de él (el cuerpo de `@before`, `@around`, etc.). Un **pointcut** es la cadena de patrón —como `"service.*.*"`— que decide a qué métodos se aplica el advice. El **weaving** (tejido) es el acto de coser el advice en esos métodos al arrancar. Tú escribes los aspectos; PyFly hace el weaving.

`@aspect` marca una clase como aspecto de PyFly. La clase se registra automáticamente en el contenedor de DI como singleton y recibe las dependencias inyectadas vía `__init__`. No se requiere ninguna clase base explícita.

Construye el aspecto de logging en tres movimientos.

Paso 1 — Declara la clase. Crea `src/lumen/aspects/logging_aspect.py`, marca la clase con `@aspect` y dale un `logger` a nivel de módulo.

Paso 2 — Añade métodos de advice. Cada método se decora con un tipo de advice (`@before` , `@after_returning` , `@after_throwing`) y una cadena de pointcut. El argumento `jp: JoinPoint` transporta los detalles de la llamada interceptada.

Paso 3 — Establece el orden. `@order(-50)` hace que este aspecto se ejecute antes que los de número más alto, útil cuando quieres que el logging enmarque el aspecto de métricas que escribes a continuación.

lumen/aspects/logging_aspect.py

```

from pyfly.aop import aspect, before, after_returning, after_throwing,
JoinPoint
from pyfly.container.ordering import order
from pyfly.logging import get_logger

logger = get_logger("lumen.audit")

@aspect
@order(-50)
class AuditLoggingAspect:
    """Logs entry, exit, and failure for every service method."""

    @before("service.*")
    def log_entry(self, jp: JoinPoint) → None:
        logger.info(
            "method_called",
            cls=type(jp.target).__name__,
            method=jp.method_name,
        )

    @after_returning("service.*")
    def log_return(self, jp: JoinPoint) → None:
        logger.info(
            "method_returned",
            cls=type(jp.target).__name__,
            method=jp.method_name,
        )

    @after_throwing("service.*")
    def log_error(self, jp: JoinPoint) → None:
        logger.error(
            "method_raised",
            cls=type(jp.target).__name__,
            method=jp.method_name,

```

```
exc=type(jp.exception).__name__,
)
```

Listado 15.12 — Un aspecto de logging

El pointcut `"service.*.*"` coincide con cada método público de cada bean con estereotipo `@service`. `*` coincide con exactamente un segmento separado por puntos; `**` coincide con uno o más. Se admiten globs parciales dentro de un segmento: `"service.*.do_handle"` coincide con todos los métodos `do_handle` de todos los manejadores con estereotipo de servicio.

Los nombres cualificados siguen el patrón `"{stereotype}.{ClassName}.{method_name}"`, de modo que `service.DepositFundsHandler.do_handle` identifica de forma unívoca el método `do_handle` de `DepositFundsHandler`.

💡 LOS HANDLERS `@before` DEBEN SER SÍNCRONOS

Los handlers `@before`, `@after_returning`, `@after_throwing` y `@after` siempre son llamados de forma síncrona por el weaver. Solo los handlers `@around` pueden ser asíncronos (y deben hacer `await jp.proceed()` cuando aconsejan un método asíncrono).

@around — métricas sin decoradores

`@around` es el tipo de advice más potente. Envuelve toda la ejecución del método; llama a `await jp.proceed()` para invocar el método original (o el siguiente advice de la cadena) y añade comportamiento a ambos lados:

lumen/aspects/metrics_aspect.py

```

import time

from pyfly.aop import JoinPoint, around, aspect
from pyfly.container.ordering import order
from pyfly.observability import MetricsRegistry

registry = MetricsRegistry()

@aspect
@order(50)
class MetricsAspect:
    """Records duration and call counts for every service method."""

    @around("service.*")
    async def record_metrics(self, jp: JoinPoint):
        start = time.perf_counter()
        exc_name = "none"
        try:
            result = await jp.proceed()
            return result
        except Exception as exc:
            exc_name = type(exc).__name__
            raise
        finally:
            elapsed = time.perf_counter() - start
            histogram = registry.histogram(
                f"service.{jp.method_name}.duration",
                f"Duration of {jp.method_name}",
                labels=["exception"],
            )
            histogram.labels(exception=exc_name).observe(elapsed)

```

Listado 15.13 — Aspecto de métricas con @around

`@order(-50)` en `AuditLoggingAspect` y `@order(50)` en `MetricsAspect` garantizan que el aspecto de logging se dispare primero en la cadena de advice.

`HIGHEST_PRECEDENCE = -(2^31)` se ejecuta el primero; `LOWEST_PRECEDENCE = 2^31 - 1` se ejecuta el último.

Weaving automático — `AspectBeanPostProcessor`

En producción nunca llamas a `weave_bean()` manualmente. `AopAutoConfiguration` registra `AspectBeanPostProcessor` incondicionalmente. Durante el arranque del contexto, el post-procesador:

1. Recopila en un `AspectRegistry` cada bean cuya clase tiene `__pyflyl__ = True`.
2. Para cada bean que no sea aspecto, comprueba si algún pointcut registrado coincide con algún método público.
3. Envuelve los métodos coincidentes en su sitio con la cadena de advice completa mediante `weave_bean()`.

El resultado es AOP de configuración cero: define aspectos, define servicios, arranca la aplicación, y el weaver los cablea entre sí.

Qué acaba de pasar. Escribiste dos aspectos —uno para logging, otro para métricas— y no editaste ni un solo manejador. Al arrancar, `AspectBeanPostProcessor` hizo coincidir sus pointcuts contra tus beans de servicio y tejió el advice en los métodos coincidentes en su sitio. A partir de ahora, cada método `@service` emite logs de entrada/salida y un histograma de duración de forma automática. Añade un manejador nuevo mañana y heredará la misma observabilidad en el momento en que su pointcut coincida: nada que recordar, nada que copiar y pegar. Esa es la recompensa de la AOP: el comportamiento transversal vive en un solo lugar, y el código de negocio queda limpio.

Referencia de JoinPoint

Cada handler de advice recibe una dataclass `JoinPoint` :

Atributo	Disponible en	Descripción
<code>target</code>	Todos	La instancia del bean interceptada
<code>method_name</code>	Todos	Nombre del método interceptado
<code>args</code>	Todos	Argumentos posicionales pasados al método
<code>kwargs</code>	Todos	Argumentos con nombre pasados al método
<code>return_value</code>	<code>@after_returning</code> , <code>@after</code>	Valor de retorno (tras el éxito)
<code>exception</code>	<code>@after_throwing</code> , <code>@after</code>	La excepción lanzada (o <code>None</code>)
<code>proceed</code>	Solo <code>@around</code>	Invocable; con <code>await</code> para métodos asíncronos

Juntándolo todo — observabilidad completa en `DepositFundsHandler`

El manejador de depósitos de Lumen con los tres pilares de observabilidad aplicados; cero código de observabilidad dentro de la lógica de negocio:

lumen/core/services/wallets/deposit_funds_handler.py

```

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.data.relational.sqlalchemy import LockMode, transactional
from pyfly.domain import AggregateNotFound
from pyfly.eda import EventPublisher
from pyfly.logging import get_logger
from pyfly.observability import MetricsRegistry, counted, span, timed

from lumen.core.mappers.wallet_mapper import to_aggregate, to_entity
from lumen.core.services.wallets.deposit_funds_command import DepositFunds
from lumen.core.services.wallets.event_publishing import publish_domain_events
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository

logger = get_logger("lumen.wallet")
registry = MetricsRegistry()

@command_handler
@service
class DepositFundsHandler(CommandHandler[DepositFunds, int]):
    """
    Credits funds to an existing wallet and returns the new balance
    (in minor units, e.g. 1350 = €13.50).

    Logging, metrics, and tracing are applied by decorators and aspects;
    the business logic here stays free of cross-cutting concerns.
    """

    def __init__(
        self,
        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
    ) → None:

```

```

    super().__init__()
    self._repository = repository
    self._events = events
    self._session_factory = session_factory

    @timed(registry, "lumen.wallet.deposit.duration", "Deposit latency")
    @counted(registry, "lumen.wallet.deposits", "Total deposit attempts")
    @span("wallet-deposit")
    @transactional()
    async def do_handle(self, command: DepositFunds) → int:
        logger.info("deposit_started", wallet_id=command.wallet_id,
                    amount=command.amount)
        entity = await self._repository.find_by_id(
            command.wallet_id, lock=LockMode.PESSIMISTIC_WRITE
        )
        if entity is None:
            raise AggregateNotFound("Wallet", command.wallet_id)
        wallet = to_aggregate(entity)
        wallet.deposit(Money(amount=command.amount, currency=wallet.currency))
        await self._repository.upsert(to_entity(wallet))
        await publish_domain_events(self._events, wallet.clear_events())
        logger.info("deposit_completed", wallet_id=command.wallet_id,
                    new_balance=wallet.balance.amount)
        return wallet.balance.amount

```

Listado 15.14 — DepositFundsHandler con observabilidad completa

Cómo funciona. `@span` abre un span de OpenTelemetry. `@timed` registra la duración de `do_handle`. `@counted` incrementa el contador de llamadas. `AuditLoggingAspect` dispara `@before` y `@after_returning` en cada método de servicio. `MetricsAspect` añade su propio histograma `@around`. La redacción de PII elimina automáticamente los valores sensibles de la salida de log. El actuador expone `/actuator/health`, `/actuator/prometheus` y `/actuator/loggers`. El panel de administración muestra trazas y registros de log en vivo.

`command.amount` es un `int` en unidades menores, impuesto por el validador del comando `DepositFunds` (`amount > 0`). El objeto de valor `Money` lo envuelve con la `Currency` del monedero, evitando la aritmética entre divisas distintas en la frontera del dominio.

Siete líneas de decoradores y una llamada a `get_logger`, y la ruta de depósito de Lumen es totalmente observable.

Ejecútalo — confirma que nada se rompió. Los decoradores de observabilidad envuelven comportamiento alrededor de tus manejadores; no deben cambiar el resultado que esos manejadores devuelven. Ejecuta la suite existente de Lumen para demostrar que la ruta de depósito sigue comportándose igual:

```
uv run --extra dev pytest -q
```

Todas las pruebas deberían seguir en verde:

```
..S.S..... [100%]
43 passed, 2 skipped in 0.3s
```

Luego da una vuelta manual completa con la aplicación en marcha: lanza un depósito en `8080`, confirma que `/actuator/health` está `UP` en `9090`, recolecta `/actuator/prometheus` y comprueba que `lumen_wallet_deposits_total` se incrementa, y abre `http://localhost:9090/admin` para ver el mismo flujo de eventos pasar por los paneles en vivo de Métricas y Log Viewer. Los tres pilares —logs, métricas, trazas— describen ahora un único depósito, unidos por un solo `trace_id`.

✓ Lo que construiste

Empezaste con un servicio listo para producción pero opaco. Al final de este capítulo Lumen:

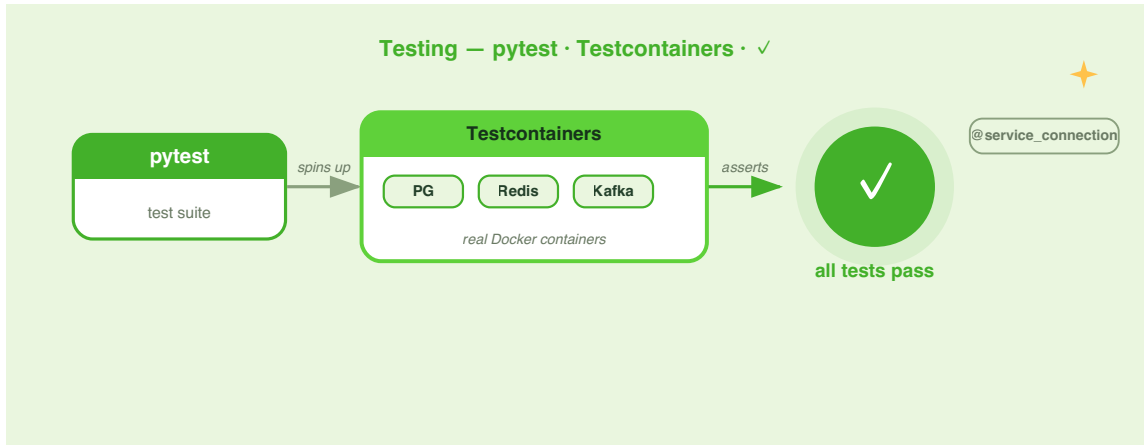
- Emite **logs JSON estructurados** con identificadores de correlación, campos vinculados al contexto asíncrono y redacción automática de PII mediante expresiones regulares y, opcionalmente, NER con Presidio.
- Exporta **métricas Prometheus** desde `MetricsRegistry`, etiquetadas con histogramas de duración `@timed` y contadores de invocación `@counted`, recolectadas en `/actuator/prometheus`.
- Propaga **trazas OpenTelemetry** de extremo a extremo: `TracingFilter` abre un span SERVER a partir de la cabecera `traceparent` entrante, `@span` crea spans hijos para las llamadas a manejadores, `HttpClientAdapter` inyecta el contexto en las peticiones salientes y `StructlogAdapter` estampa cada registro de log con `trace_id` y `span_id`.
- Responde a las **sondas de salud de Kubernetes** en `/actuator/health/liveness` y `/actuator/health/readiness` mediante beans `HealthIndicator` autodescubiertos, incluido un `WalletRepositoryHealthIndicator` que sondea el almacén de monederos.
- Muestra todo lo anterior en el **panel de administración embebido** en `/admin`, con 15 vistas integradas, flujos SSE en tiempo real, una cola de logs en vivo, un panel de analítica de trazas HTTP y gestión de niveles de logger en tiempo de ejecución.
- Aplica logging y métricas de forma **transversal** mediante `@aspect`, `@before`, `@after_returning`, `@after_throwing` y `@around`, tejidos automáticamente por `AspectBeanPostProcessor` sin tocar el código de los manejadores.

Pruébalo tú mismo

- 1. Auditoría de redacción de PII.** Añade una sentencia de log a `DepositFundsHandler.do_handle` que incluya una dirección de correo falsa como valor de campo (`customer_email="alice@example.com"`) y un campo `token` con una cadena arbitraria. Ejecuta Lumen localmente con `format: console`, observa que el correo se sustituye por `<EMAIL>` y que el valor del campo token se sustituye por `<REDACTED>`. Luego cambia a `engine: presidio` (tras instalar `pyfly[pii]` y `en_core_web_sm`) y compara la salida.
- 2. HealthIndicator personalizado.** Escribe un `StripeHealthIndicator` que llame a `https://status.stripe.com/api/v2/status.json` con `httpx`, parsee `indicator.status` y devuelva `UP` si el valor es `"none"` o `DOWN` en caso contrario. Regístralo como `@component` y verifica que `/actuator/health` incluye un componente `StripeHealthIndicator`. Pruébalo con una llamada `httpx` simulada que lance `httpx.ConnectError` y verifica que el estado agregado pasa a `DOWN`.
- 3. Aspecto de métricas con umbrales por método.** Extiende `MetricsAspect` del Listado 15.13 con un `slow_threshold` configurable (0,5 s por defecto) inyectado desde `pyfly.yaml` vía `@config_properties`. Cuando un método de servicio supere el umbral, emite un `logger.warning("slow_method", ...)` con los campos `method_name` y `elapsed`. Escribe una prueba de pytest que use un `FakeClock` o `unittest.mock.patch("time.perf_counter")` para simular una llamada lenta y aserte que se registra la advertencia.

CAPÍTULO 16

Pruebas de aplicaciones PyFly



El monedero (wallet) funciona. Los depósitos entran, los saldos se actualizan, los eventos se propagan por el bus y el coordinador de la saga revierte limpiamente ante un fallo. Lo que todavía no tienes es la **confianza** de que seguirá funcionando tras la próxima refactorización. Esa confianza viene de las pruebas: pruebas que se ejecutan en milisegundos, que demuestran que el modelo de dominio hace cumplir sus invariantes, que verifican que la canalización CQRS despacha correctamente, que ejercitan las consultas derivadas y los predicados de tipo Specification del repositorio contra una base de datos SQLite real, y que arrancan el contexto de aplicación completo en una prueba de integración que demuestra la composición entera de inyección de dependencias + persistencia.

PyFly trata las pruebas como un asunto de primer orden. El módulo `pyfly.testing` incluye ayudantes de más alto nivel — `PyFlyTestCase`, `create_test_container`, `assert_event_published`, el cableado de Testcontainers— a los que puedes recurrir

cuando los necesites. La propia suite de pruebas de Lumen no los usa: cablea componentes reales directamente desde `conftest.py`, emplea fixtures estándar de pytest y cubre cada nivel de la pirámide sin código repetitivo. Ese es el enfoque que enseña este capítulo.

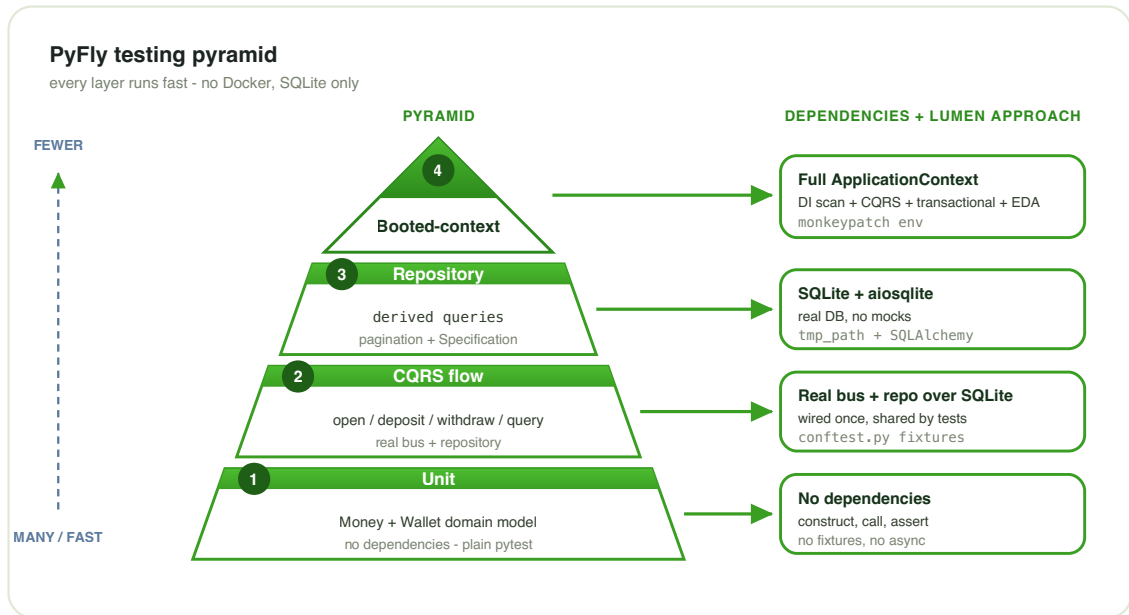


Figura 16.1 — La pirámide de pruebas de PyFly. Las pruebas unitarias rápidas forman la base ancha; las pruebas de integración ocupan el centro; las pruebas de adaptador contra una BD real y una prueba de integración con el contexto arrancado coronan la cima.

La pirámide tiene cuatro niveles. Las **pruebas unitarias** están en la base —muchas de ellas, ejecutándose en milisegundos, ejercitando el modelo de dominio sin dependencias—. Las **pruebas de flujo CQRS** ocupan el siguiente escalón: el ciclo completo de apertura/depósito/retiro/consulta enrutado a través del bus real y del repositorio real, todo cableado en `conftest.py`. Las **pruebas de repositorio** ejercitan las consultas derivadas, la paginación y los predicados de tipo Specification contra SQLite. En la cúspide, una **prueba de integración con el contexto arrancado** inicia el `ApplicationContext` real —escaneo de inyección de dependencias,

autoconfiguración de CQRS, `RepositoryBeanPostProcessor`, la costura `@transactional`, la arquitectura orientada a eventos (EDA)— y recorre el ciclo de vida completo.

Nivel	Dependencias	Velocidad	Enfoque de Lumen
Unitario	Ninguna	Rápido	pytest puro, sin fixtures
Flujo CQRS	Bus real + repositorio sobre SQLite	Rápido	fixtures de <code>conftest.py</code>
Repositorio	SQLite + <code>aiosqlite</code>	Rápido	<code>tmp_path</code> + SQLAlchemy
Contexto arrancado	ApplicationContext completo + SQLite	Rápido	sustitución de entorno <code>monkeypatch</code>

El proyecto usa pytest con `pytest-asyncio` en **modo automático**. Actívalo una vez en `pyproject.toml` y cada función `async def test_*` se recopila automáticamente:

```
[tool.pytest.ini_options]
asyncio_mode = "auto"
testpaths = ["tests"]
pythonpath = ["src"]
```

Una glosa rápida antes de empezar, ya que tres términos se repiten en este capítulo. Un **fixture** es una pieza reutilizable de preparación de prueba: pytest la construye una vez, se la entrega a tu prueba y la desmonta después. Un **conftest.py** es un archivo especial que pytest descubre automáticamente; cualquier fixture que declares ahí pasa a estar disponible para cada prueba del paquete sin necesidad de importarla. Y el **modo automático de `pytest-asyncio`** simplemente significa que puedes escribir `async def test_...` y pytest hará el `await` por ti: no hace falta un decorador por prueba. Cada uno de estos vuelve a aparecer con un ejemplo concreto más adelante; esto es solo el mapa.

Instala las dependencias de desarrollo y ejecuta la suite:

```
uv run --extra dev pytest -q
```

El `uv sync` a secas (sin `--extra dev`) omite el grupo de desarrollo, así que `pytest` no queda instalado. Incluye siempre `--extra dev` al ejecutar las pruebas.

Ejecútalo. Desde el directorio `samples/lumen`, ejecuta toda la suite una vez ahora para que tengas una referencia de base antes de cambiar nada:

```
uv run --extra dev pytest -q
```

Deberías ver una hilera de puntos —uno por prueba— seguida de una línea de resumen:

```
..S.S.....
[100%]
43 passed, 2 skipped in 0.28s
```

Cuarenta y tres pruebas superadas, en menos de un tercio de segundo, sin Docker ni proceso externo alguno. (Las dos omitidas son las ejecuciones sobre PostgreSQL de la prueba de concurrencia del Capítulo 5; necesitan `LUMEN_TEST_POSTGRES_URL`.) Esa velocidad es el sentido entero de la pirámide: la base rápida atrapa la mayoría de las regresiones antes de que lleguen a ejecutarse las capas de integración más lentas. Si en cambio ves `No module named pytest`, es que olvidaste `--extra dev`; vuelve a ejecutarlo con esa opción.

Pruebas unitarias del dominio

El modelo de dominio — `Money` y `Wallet` — no tiene dependencias del framework. Nunca toca una base de datos, un bus de mensajes ni un cliente HTTP. Esa pureza lo convierte en el objetivo ideal para pruebas unitarias rápidas: construye objetos, llama a métodos, comprueba resultados. Sin mocks, sin fixtures, sin `async`.

Pruebas de Money

`Money` es una `dataclass` congelada (frozen). Cada operación o bien tiene éxito y devuelve una nueva instancia de `Money`, o bien lanza `BusinessRuleViolation`. Cada violación lleva una cadena `.rule` que nombra el invariante incumplido, útil para verificar la regla exacta en las pruebas.

Una glosa rápida sobre dos términos. Un **objeto de valor** es un objeto definido por completo por sus valores: dos instancias de `Money(1050, EUR)` son iguales porque sus campos son iguales, no porque sean el mismo objeto en memoria. Una **`dataclass` congelada (frozen)** es la forma que tiene Python de hacer inmutable ese objeto: una vez construido, no puedes reasignar sus campos. Juntos hacen que `Money` sea seguro de pasar libremente: ningún consumidor puede mutarlo a tus espaldas, así que nunca necesita copia defensiva.

Construyamos el archivo de pruebas un grupo de aserciones a la vez. Cada paso de abajo se corresponde con una función `def test_...` del listado que sigue.

Paso 1 — la igualdad es estructural. `test_value_equality_is_structural` comprueba que dos valores `Money` con el mismo importe y la misma moneda son iguales, y que diferir en cualquiera de los dos campos los hace distintos. Este es el contrato del objeto de valor.

Paso 2 — la inmutabilidad se hace cumplir. `test_money_is_immutable` intenta asignar a `money.amount` y espera una excepción. La dataclass congelada lanza `FrozenInstanceError`, demostrando que no puedes mutar un valor tras su construcción.

Paso 3 — la aritmética devuelve valores nuevos.

`test_add_and_subtract_same_currency` comprueba que `add` y `subtract` producen el `Money` esperado, sin mutar nunca los operandos.

Paso 4 — la superficie de conveniencia. `test_zero_factory_and_major_units` cubre la fábrica `Money.zero(currency)`, la propiedad `major_units` y el formateo de `__str__`.

Paso 5 — los invariantes rechazan la entrada inválida. Las dos últimas pruebas comprueban que mezclar monedas y pasar un importe no entero lanzan cada una `BusinessRuleViolation` con una cadena `.rule` específica: `"money-currency-mismatch"` y `"money-amount-integer"`.

tests/test_money.py

```

from __future__ import annotations

import pytest
from lumen.interfaces.enums.v1.currency import Currency
from lumen.models.entities.v1.money import Money

from pyfly.domain import BusinessRuleViolation


def test_value_equality_is_structural() → None:
    assert Money(1050, Currency.EUR) == Money(1050, Currency.EUR)
    assert Money(1050, Currency.EUR) != Money(1050, Currency.USD)
    assert Money(1050, Currency.EUR) != Money(999, Currency.EUR)


def test_money_is_immutable() → None:
    money = Money(1050, Currency.EUR)
    with pytest.raises(Exception): # frozen dataclass → FrozenInstanceError
        money.amount = 0 # type: ignore[misc]


def test_add_and_subtract_same_currency() → None:
    a = Money(1050, Currency.EUR)
    b = Money(450, Currency.EUR)
    assert a.add(b) == Money(1500, Currency.EUR)
    assert a.subtract(b) == Money(600, Currency.EUR)


def test_zero_factory_and_major_units() → None:
    assert Money.zero(Currency.USD) == Money(0, Currency.USD)
    assert Money(1050, Currency.EUR).major_units == 10.5
    assert str(Money(1050, Currency.EUR)) == "10.50 EUR"


def test_currency_mismatch_is_rejected() → None:
    with pytest.raises(BusinessRuleViolation) as exc:
        Money(100, Currency.EUR).add(Money(100, Currency.USD))

```

```

assert exc.value.rule == "money-currency-mismatch"

def test_non_integer_amount_is_rejected() → None:
    with pytest.raises(BusinessRuleViolation) as exc:
        Money(10.5, Currency.EUR) # type: ignore[arg-type]
    assert exc.value.rule == "money-amount-integer"

```

Listado 16.1 — Pruebas unitarias puras del objeto de valor Money

Cada prueba es síncrona: sin `async`, sin `await`, sin fixtures. Pytest recopila las funciones a nivel de módulo automáticamente. `Currency.EUR` es un valor de enumeración, no una cadena simple, ajustándose exactamente al contrato de tipos del modelo de dominio.

Ejecútalo. Ejecuta solo este archivo para ver en acción la base unitaria de la pirámide:

```
uv run --extra dev pytest tests/test_money.py -q
```

Salida esperada:

```

.....
[100%]
6 passed in 0.02s

```

Seis pruebas, veinte milisegundos. Sin base de datos conectada, sin bus de eventos iniciado: estas pruebas construyen un objeto simple y comprueban su comportamiento. Eso es lo que hace que la base de la pirámide sea tan ancha y tan rápida.

Qué acaba de pasar. Has demostrado todo el contrato de `Money` —igualdad, inmutabilidad, aritmética y cada invariante— sin tocar una sola pieza de infraestructura del framework. El código de dominio puro permanece probable como dominio puro. Cuando una de estas falla, sabes que el error está en el propio objeto de valor, no en el cableado, una sesión o el bus.

💡 ARITMÉTICA DE UNIDADES MENORES

`Money` almacena los importes en **unidades menores** (céntimos enteros). `Money(1050, Currency.EUR)` representa 10,50 € —verificado por `major_units == 10.5` y `str(...) == "10.50 EUR"` —. La fábrica `Money.zero(currency)` devuelve un `Money(0, currency)`, útil para inicializar los saldos de los monederos.

Pruebas de la raíz de agregado Wallet

`Wallet` hace cumplir varios invariantes: el propietario debe ser una cadena no vacía, los depósitos deben ser positivos, los retiros no deben dejar el saldo en descubierto y los importes deben coincidir con la moneda del monedero. Cada violación lleva un atributo `.rule` para una verificación precisa.

Un término primero. Un **agregado** (o **raíz de agregado**) es un grupo de objetos de dominio tratado como una sola unidad de coherencia; aquí, el `Wallet` y su saldo. Todo cambio pasa por los métodos del agregado, así que el agregado es el único lugar que garantiza que sus invariantes se cumplen. Eso lo convierte en la unidad natural para probar: recórrelo a través de sus métodos públicos y comprueba que las reglas nunca se rompen.

El patrón que sigue cada prueba de agregado es **organizar, actuar, comprobar** (arrange, act, assert). Organizar: construir el monedero en un estado conocido. Actuar: llamar a un método. Comprobar: verificar el saldo, el evento emitido o la violación lanzada. Búscalo en cada prueba:

Paso 1 — la apertura emite un evento. `test_open_creates_empty_wallet` abre un monedero y comprueba que el saldo es cero y que se encola exactamente un evento `WalletOpened`.

Paso 2 — la apertura valida sus argumentos. `test_open_requires_owner` pasa un propietario en blanco y espera `BusinessRuleViolation` con la regla `"wallet-owner-required"`.

Paso 3 — el camino feliz de depósito y luego retiro.

`test_deposit_then_withdraw_happy_path` deposita, comprueba el saldo y el evento `FundsDeposited`, luego retira y comprueba el saldo y el evento `FundsWithdrawn`.

Fíjate en la llamada a `clear_events()` en el paso de organización; más sobre esto justo debajo del listado.

Paso 4 — los invariantes rechazan las operaciones inválidas. Las tres pruebas finales demuestran que un retiro no puede dejar el saldo en descubierto, que un depósito debe ser positivo y que un importe debe coincidir con la moneda del monedero. Cada una comprueba la cadena `.rule` exacta, y la prueba de descubierto también comprueba que el saldo quedó sin cambios y que no se lanzó ningún evento, prueba de que el invariante se disparó *antes* de que cambiara ningún estado.

tests/test_wallet_aggregate.py

```

from __future__ import annotations

import pytest
from lumen.interfaces.enums.v1.currency import Currency
from lumen.models.entities.v1.money import Money
from lumen.models.entities.v1.wallet_entity import (
    FundsDeposited,
    FundsWithdrawn,
    Wallet,
    WalletOpened,
)

from pyfly.domain import BusinessRuleViolation


def test_open_creates_empty_wallet() → None:
    wallet = Wallet.open("wlt-1", "owner-1", Currency.EUR)
    assert wallet.owner_id == "owner-1"
    assert wallet.currency is Currency.EUR
    assert wallet.balance == Money.zero(Currency.EUR)
    [event] = wallet.pending_events()
    assert isinstance(event, WalletOpened)
    assert event.wallet_id == "wlt-1"
    assert event.currency == "EUR"


def test_open_requires_owner() → None:
    with pytest.raises(BusinessRuleViolation) as exc:
        Wallet.open("wlt-x", " ", Currency.EUR)
    assert exc.value.rule == "wallet-owner-required"


def test_deposit_then_withdraw_happy_path() → None:
    wallet = Wallet.open("wlt-2", "owner-2", Currency.EUR)
    wallet.clear_events()

    wallet.deposit(Money(1000, Currency.EUR))

```

```

assert wallet.balance == Money(1000, Currency.EUR)
[event] = wallet.clear_events()
assert isinstance(event, FundsDeposited)
assert event.amount == 1000
assert event.balance == 1000

wallet.withdraw(Money(400, Currency.EUR))
assert wallet.balance == Money(600, Currency.EUR)
[event] = wallet.clear_events()
assert isinstance(event, FundsWithdrawn)
assert event.amount == 400
assert event.balance == 600

def test_withdraw_cannot_overdraw() → None:
    wallet = Wallet.open("wlt-3", "owner-3", Currency.EUR)
    wallet.deposit(Money(500, Currency.EUR))
    wallet.clear_events()
    with pytest.raises(BusinessRuleViolation) as exc:
        wallet.withdraw(Money(501, Currency.EUR))
    assert exc.value.rule == "wallet-insufficient-funds"
    # invariant held: balance unchanged, no event raised
    assert wallet.balance == Money(500, Currency.EUR)
    assert wallet.pending_events() == []

def test_deposit_must_be_positive() → None:
    wallet = Wallet.open("wlt-4", "owner-4", Currency.EUR)
    with pytest.raises(BusinessRuleViolation) as exc:
        wallet.deposit(Money(0, Currency.EUR))
    assert exc.value.rule == "wallet-deposit-positive"

def test_currency_mismatch_is_rejected() → None:
    wallet = Wallet.open("wlt-5", "owner-5", Currency.EUR)
    with pytest.raises(BusinessRuleViolation) as exc:
        wallet.deposit(Money(100, Currency.USD))
    assert exc.value.rule == "wallet-currency-mismatch"

```

Listado 16.2 — Pruebas unitarias de la raíz de agregado Wallet

Tres detalles merecen atención. Primero, `Wallet.open` toma tres argumentos posicionales: un `wallet_id` pregenerado, un `owner_id` y un valor de enumeración `Currency`; el agregado no genera su propio ID. Segundo, `pending_events()` devuelve los eventos almacenados en búfer sin vaciarlos; `clear_events()` los devuelve y los vacía. La prueba `test_deposit_then_withdraw_happy_path` llama a `clear_events()` tras la apertura para que cada aserción vea exactamente un evento. Tercero, `FundsDeposited` y `FundsWithdrawn` llevan `amount` (el importe de la operación en unidades menores) y `balance` (el saldo acumulado tras la operación), no `new_balance`. Verifica siempre los campos reales de la dataclass del evento antes de comprobarlos.

Ejecútalo.

```
uv run --extra dev pytest tests/test_wallet_aggregate.py -q
```

Salida esperada:

```
.....
[100%]
6 passed in 0.02s
```

Qué acaba de pasar. La línea más difícil de leer es la asignación `[event] = wallet.clear_events()`. Eso es un desempaqueado de lista: comprueba que la lista devuelta tiene **exactamente un** elemento y lo enlaza a `event` en un solo paso. Si el agregado hubiera lanzado cero o dos eventos, el propio desempaqueado lanzaría un `ValueError` y la prueba fallaría, así que la forma del flujo de eventos se comprueba gratis. Por eso la prueba del camino feliz llama a `clear_events()` justo después de la apertura: vacía el evento `WalletOpened` para que el siguiente desempaqueado vea solo el evento `FundsDeposited` que realmente estás comprobando.

🍃 EQUIVALENCIA CON SPRING

Probar un agregado DDD de forma aislada es la misma disciplina en cualquier stack. En Spring / jMolecules llamarías a los métodos del agregado directamente y comprobarías `aggregate.domainEvents()` (proporcionado por `AbstractAggregateRoot`) antes de llamar a `afterDomainEventPublication()` para vaciar el búfer. El `clear_events()` de PyFly cumple el mismo papel: vaciar, comprobar, seguir.

Cableado del stack de pruebas con conftest.py

Las pruebas de CQRS y de listeners de eventos necesitan infraestructura real: un `WalletRepository` respaldado por SQLite, un bus de eventos, manejadores de comandos y consultas y un bus en marcha. En lugar de recrear esto en cada módulo de prueba, Lumen declara el cableado una sola vez en `tests/conftest.py`. Pytest descubre el archivo automáticamente y pone los fixtures a disposición de cada prueba del paquete.

La diferencia clave respecto a una prueba de adaptador montada a mano es que el fixture `repository` usa el `WalletRepository` **real del framework** —la misma clase al estilo Spring Data que arranca la aplicación— y lo ejecuta a través del `RepositoryBeanPostProcessor` **real**, que compila los esbozos de consulta derivada a partir de los nombres de método al arrancar.

Este archivo es el corazón del capítulo, así que lo leeremos de arriba abajo como una serie de pasos pequeños y por capas. Cada fixture se construye sobre el anterior. Lo que hay que tener presente: pytest enlaza los fixtures por **nombre**. Cuando una función de fixture declara un parámetro, pytest busca un fixture con ese nombre, lo construye y lo inyecta. Así es como un pequeño grafo de fixtures independientes se compone en un stack de pruebas completo.

Paso 1 — hacer que la muestra sea importable. Las primeras líneas añaden el `src/` de la muestra a `sys.path` para que `import lumen...` se resuelva. El ajuste `pythonpath = ["src"]` de `pyproject.toml` hace lo mismo para la recopilación de la propia pytest; esta línea cubre las importaciones directas dentro de `conftest.py` antes de que se apliquen los ajustes de ruta de pytest.

Paso 2 — `session_factory` : un motor de base de datos. Este fixture asíncrono crea un motor SQLite en memoria, ejecuta `Base.metadata.create_all` para construir el esquema y entrega un `async_sessionmaker`. Una **fábrica de sesiones** es un invocable que reparte sesiones de base de datos nuevas; la autoconfiguración relacional del framework crea una exactamente así al arrancar. El único motor compartido mantiene la base de datos en memoria viva durante toda la prueba: ciérralo y los datos se esfuman.

Paso 3 — `repository` : el repositorio real al estilo Spring Data. Construye el `WalletRepository` del framework, luego llama a `RepositoryBeanPostProcessor().after_init(repo, "walletRepository")`. Un **post-procesador** es un gancho que se ejecuta contra un bean recién creado; aquí lee nombres de método como `find_by_owner_id` y los compila en consultas reales. Si te saltas esta llamada, esos métodos siguen siendo esbozos que lanzan `NotImplementedError`.

Paso 4 — `event_bus` : el bus de eventos en memoria. Un fixture de una línea que entrega un `InMemoryEventBus`, el mismo publicador que usa la aplicación en despliegues sin Kafka.

Paso 5 — `audit_listener` : un suscriptor en ese bus. Crea el

`WalletAuditListener` y suscribe su manejador al bus, leyendo los patrones de evento directamente del atributo `__pyfly_event_patterns__` del método decorado: exactamente lo que hace el `ApplicationContext` cuando autocablea los listeners al arrancar.

Paso 6 — `command_bus` y `query_bus` : los despachadores de CQRS. Cada uno

construye un `HandlerRegistry`, registra los manejadores reales (pasándoles el `repository`, el `event_bus` y la `session_factory` que necesitan) y entrega un bus.

CQRS —Command/Query Responsibility Segregation— significa simplemente que las escrituras van por un bus de comandos y las lecturas por un bus de consultas; un

manejador (handler) es la función que de hecho procesa un comando o una consulta.

tests/conftest.py

```

from __future__ import annotations

import sys
from collections.abc import AsyncIterator
from pathlib import Path

import pytest_asyncio
from sqlalchemy.ext.asyncio import (
    AsyncSession,
    async_sessionmaker,
    create_async_engine,
)

# Make the sample's `src/` importable
_HERE = Path(__file__).resolve().parent
_SRC = _HERE.parent / "src"
sys.path.insert(0, str(_SRC))

from lumen.core.services.listeners import WalletAuditListener
from lumen.core.services.wallets import (
    DepositFundsHandler,
    GetBalanceHandler,
    GetWalletHandler,
    ListRichWalletsHandler,
    ListWalletsHandler,
    OpenWalletHandler,
    WithdrawFundsHandler,
)
from lumen.models.entities.v1.wallet_orm import WalletEntity
from lumen.models.repositories import WalletRepository

from pyfly.cqrs import DefaultCommandBus, DefaultQueryBus, HandlerRegistry
from pyfly.data.relational.sqlalchemy import Base
from pyfly.data.relational.sqlalchemy.post_processor import (
    RepositoryBeanPostProcessor,
)
from pyfly.eda.adapters.memory import InMemoryEventBus

```

```

@pytest_asyncio.fixture
async def session_factory() →
    AsyncIterator[async_sessionmaker[AsyncSession]]:
        """An in-memory SQLite engine + session factory, schema created.

        Mirrors the framework's relational auto-configuration: build the
        async engine and run Base.metadata.create_all.
        """

        engine = create_async_engine("sqlite+aiosqlite:///memory:")
        async with engine.begin() as conn:
            await conn.run_sync(Base.metadata.create_all)
        factory = async_sessionmaker(engine, expire_on_commit=False)
        try:
            yield factory
        finally:
            await engine.dispose()

@pytest_asyncio.fixture
async def repository(
    session_factory: async_sessionmaker[AsyncSession],
) → AsyncIterator[WalletRepository]:
    """The framework WalletRepository, post-processed.

    RepositoryBeanPostProcessor compiles derived-query stubs
    (e.g. find_by_owner_id) onto the bean – the same step the
    ApplicationContext runs at startup.
    """

    session = session_factory()
    repo = WalletRepository(WalletEntity, session)
    RepositoryBeanPostProcessor().after_init(repo, "walletRepository")
    try:
        yield repo
    finally:
        await session.close()

```



```

@pytest_asyncio.fixture
async def event_bus() → AsyncIterator[InMemoryEventBus]:
    """A real in-memory EDA bus – the same EventPublisher used in
    production."""
    yield InMemoryEventBus()

@pytest_asyncio.fixture
async def audit_listener(
    event_bus: InMemoryEventBus,
) → AsyncIterator[WalletAuditListener]:
    """The wallet audit projection, subscribed to the bus exactly as the
    ApplicationContext auto-wires it at startup."""
    listener = WalletAuditListener()
    method = listener.on_wallet_event
    for pattern in method.__pyfly_event_patterns__:
        event_bus.subscribe(pattern, method)
    yield listener

@pytest_asyncio.fixture
async def command_bus(
    repository: WalletRepository,
    event_bus: InMemoryEventBus,
    session_factory: async_sessionmaker[AsyncSession],
) → AsyncIterator[DefaultCommandBus]:
    registry = HandlerRegistry()
    registry.register_command_handler(
        OpenWalletHandler(
            repository=repository,
            events=event_bus,
            session_factory=session_factory,
        )
    )
    registry.register_command_handler(
        DepositFundsHandler(
            repository=repository,
            events=event_bus,
            session_factory=session_factory,
        )
    )

```

```

    )
)
registry.register_command_handler(
    WithdrawFundsHandler(
        repository=repository,
        events=event_bus,
        session_factory=session_factory,
    )
)
yield DefaultCommandBus(registry=registry)

@pytest_asyncio.fixture
async def query_bus(
    repository: WalletRepository,
) → AsyncIterator[DefaultQueryBus]:
    registry = HandlerRegistry()
    registry.register_query_handler(GetWalletHandler(repository=repository))
    registry.register_query_handler(GetBalanceHandler(repository=repository))
    registry.register_query_handler(
        ListWalletsHandler(repository=repository)
    )
    registry.register_query_handler(
        ListRichWalletsHandler(repository=repository)
    )
    yield DefaultQueryBus(registry=registry)

```

Listado 16.3 — *conftest.py*: componentes reales del framework cableados sin mocks

Cada fixture se declara con `@pytest_asyncio.fixture` (no con el `@pytest.fixture` pelado) para que `pytest-asyncio` gestione el ciclo de vida del iterador asíncrono.

`asyncio_mode = "auto"` en `pyproject.toml` hace que los fixtures y las pruebas asíncronas funcionen sin decoradores por función, pero el propio decorador del fixture debe seguir siendo `pytest_asyncio.fixture`.

El fixture `session_factory` es compartido. Tanto `repository` como `command_bus` lo reciben, así que el mismo motor SQLite en memoria respalda las lecturas, las escrituras y la frontera `@transactional` que abren los manejadores. El repositorio se construye con una sesión propia —su *modo manual*— y usa esa única sesión dentro y fuera de `@transactional`, que es lo que mantiene una base de datos `:memory:` (viva solo en una conexión) compartida entre los manejadores y las consultas. La aplicación arrancada construye el repositorio sin sesión, así que cada llamada se une a la unidad de trabajo del manejador que la hace; `test_app_context_integration.py` y `test_concurrent_balance_changes.py` ejercitan ese camino. Los fixtures `audit_listener` y `command_bus` reciben ambos `event_bus`; pytest lo instancia una vez por prueba y lo comparte entre ellos, así que los eventos publicados por los manejadores de comandos son visibles para el listener.

Qué acaba de pasar. Has cableado un stack de pruebas completo, con forma de producción —motor, repositorio, bus de eventos, listener y ambos buses de CQRS— enteramente desde fixtures, sin mocks. Los dos hechos que lo hacen funcionar merecen retenerse. Primero, **los fixtures se componen por nombre**: `command_bus` pide `repository`, `event_bus` y `session_factory` simplemente nombrándolos como parámetros, y pytest entrelaza el grafo por ti. Segundo, **un fixture solicitado por otros dos se construye una vez por prueba**: tanto `repository` como `command_bus` nombran `session_factory`, así que comparten un solo motor; la escritura que hace un comando es la lectura que ve una consulta. Acierta con este archivo y cada prueba de las siguientes cuatro secciones será una llamada de dos líneas.

🍃 EQUIVALENCIA CON SPRING

`conftest.py` es el `@TestConfiguration` de PyFly más el compartido `application-test.properties` de un proyecto Spring Boot: un único lugar que declara los beans que cada prueba reutiliza. Un `@pytest_asyncio.fixture` es el equivalente aproximado de un método `@Bean` en esa configuración: pytest lo construye de forma perezosa, lo inyecta donde se solicita su nombre y lo desmonta después, igual que el contexto de pruebas de Spring gestiona el ciclo de vida y la inyección de los beans.

🔍 NADA DE MOCKS EN NINGÚN SITIO

Cada componente de `conftest.py` es la implementación real de producción. `WalletRepository` es la misma clase que arranca la aplicación. `RepositoryBeanPostProcessor` es el mismo post-procesador que el `ApplicationContext` ejecuta al arrancar para compilar los esbozos de consulta derivada. `InMemoryEventBus` es el mismo bus que se usa en despliegues sin Kafka. El objetivo es probar los caminos de código reales, no el cableado.

Pruebas del flujo CQRS de extremo a extremo

Con los fixtures de `conftest.py`, ejercitar el ciclo completo de comando/consulta es cuestión de llamar a `command_bus.send(...)` y `query_bus.query(...)`. No se instancia ningún manejador en el cuerpo de la prueba: el bus despacha al manejador ya registrado en el fixture.

Fíjate en lo corto que se vuelve el cuerpo de cada prueba ahora que el cableado vive en `conftest.py`. Una prueba declara los fixtures que necesita como parámetros y luego se lee como una narración llana:

Paso 1 — solicitar los buses. La firma de cada prueba enumera `command_bus` o `query_bus`. pytest ve esos nombres, construye el grafo de fixtures desde `conftest.py` e inyecta los buses listos.

Paso 2 — enviar comandos. `await command_bus.send(OpenWallet(...))` devuelve el id del nuevo monedero; los comandos `DepositFunds` y `WithdrawFunds` posteriores devuelven el saldo acumulado. Compruebas cada valor de retorno sobre la marcha.

Paso 3 — consultar el lado de lectura. `await query_bus.query(GetWallet(...))` y `GetBalance(...)` recargan el estado persistido y devuelven DTO (objetos de transferencia de datos, modelos de lectura simples). Compruebas que sus campos coinciden con lo que escribieron los comandos.

Paso 4 — demostrar los caminos de error. Las pruebas restantes envían un comando que debe fallar —un descubierto, un depósito no positivo, un monedero desconocido— envuelto en `pytest.raises(CommandProcessingException)`. Ese gestor de contexto comprueba que el bloque lanza la excepción nombrada; si no lo hace, la prueba falla.

tests/test_cqrs_flow.py

```

from __future__ import annotations

import pytest
from lumen.core.services.wallets.deposit_funds_command import DepositFunds
from lumen.core.services.wallets.get_balance_query import GetBalance
from lumen.core.services.wallets.get_wallet_query import GetWallet
from lumen.core.services.wallets.open_wallet_command import OpenWallet
from lumen.core.services.wallets.withdraw_funds_command import WithdrawFunds
from lumen.interfaces.enums.v1.currency import Currency

from pyfly.cqrs import DefaultCommandBus, DefaultQueryBus

@pytest.mark.asyncio
async def test_full_wallet_lifecycle(
    command_bus: DefaultCommandBus,
    query_bus: DefaultQueryBus,
) → None:
    wallet_id = await command_bus.send(
        OpenWallet(owner_id="u-1", currency=Currency.EUR)
    )
    assert isinstance(wallet_id, str) and wallet_id.startswith("wlt-")

    balance = await command_bus.send(
        DepositFunds(wallet_id=wallet_id, amount=1500)
    )
    assert balance == 1500

    balance = await command_bus.send(
        WithdrawFunds(wallet_id=wallet_id, amount=500)
    )
    assert balance == 1000

    wallet = await query_bus.query(GetWallet(wallet_id=wallet_id))
    assert wallet is not None
    assert wallet.id == wallet_id
    assert wallet.owner_id == "u-1"

```

```

assert wallet.currency is Currency.EUR
assert wallet.balance_minor == 1000
assert wallet.balance == 10.0

balance_dto = await query_bus.query(GetBalance(wallet_id=wallet_id))
assert balance_dto is not None
assert balance_dto.balance_minor == 1000
assert balance_dto.balance == 10.0

@pytest.mark.asyncio
async def test_get_wallet_returns_none_for_unknown_id(
    query_bus: DefaultQueryBus,
) → None:
    assert await query_bus.query(
        GetWallet(wallet_id="wlt-does-not-exist")
    ) is None

@pytest.mark.asyncio
async def test_overdraw_is_rejected_through_the_bus(
    command_bus: DefaultCommandBus,
) → None:
    from pyfly.cqrs.exceptions import CommandProcessingException

    wallet_id = await command_bus.send(
        OpenWallet(owner_id="u-2", currency=Currency.EUR)
    )
    await command_bus.send(DepositFunds(wallet_id=wallet_id, amount=100))

    with pytest.raises(CommandProcessingException):
        await command_bus.send(
            WithdrawFunds(wallet_id=wallet_id, amount=999)
        )

@pytest.mark.asyncio
async def test_validation_rejects_non_positive_deposit(
    command_bus: DefaultCommandBus,

```

```

) → None:
    from pyfly.cqrs.exceptions import CommandProcessingException

    wallet_id = await command_bus.send(
        OpenWallet(owner_id="u-3", currency=Currency.EUR)
    )
    with pytest.raises(CommandProcessingException):
        await command_bus.send(DepositFunds(wallet_id=wallet_id, amount=0))

@pytest.mark.asyncio
async def test_deposit_to_unknown_wallet_is_rejected(
    command_bus: DefaultCommandBus,
) → None:
    from pyfly.cqrs.exceptions import CommandProcessingException

    with pytest.raises(CommandProcessingException):
        await command_bus.send(
            DepositFunds(wallet_id="wlt-nope", amount=100)
        )

```

Listado 16.4 — Pruebas CQRS de extremo a extremo a través del bus real

`test_full_wallet_lifecycle` es la prueba de humo principal: envía cada comando en el orden natural y luego consulta tanto el DTO completo del monedero como el DTO de saldo. El DTO del monedero expone `balance_minor` (unidades menores enteras) y `balance` (unidades mayores como float); ambos derivan de la misma fila `WalletEntity` almacenada a través del repositorio.

Las pruebas de los caminos de error verifican que el bus aflora correctamente las violaciones de dominio. `CommandProcessingException` es el envoltorio del bus para cualquier excepción lanzada dentro de un manejador, incluida `BusinessRuleViolation` del agregado. El código que llama nunca ve la excepción de dominio en bruto; siempre ve el envoltorio del bus.

Ejecútalo.

```
uv run --extra dev pytest tests/test_cqrs_flow.py -q
```

Salida esperada:

```
.....
[100%]
5 passed in 0.05s
```

Qué acaba de pasar. Esta es la primera capa que toca infraestructura real, y aun así se ejecuta en milisegundos. El comando pasó por el bus real, el manejador real abrió una unidad de trabajo `@transactional` real sobre una base de datos SQLite real, la confirmó (commit), y la consulta la leyó de vuelta: el camino que se ejecuta en producción, menos la capa HTTP y con la sesión compartida del fixture. Como el manejador se registra en el fixture en lugar de construirse en la prueba, estás probando el *despacho* además de la lógica: si el enrutamiento de comando a manejador se rompiera, estas pruebas lo atraparían.

 ASYNCIO_MODE = "AUTO" Y @PYTEST.MARK.ASYNCIO

Con `asyncio_mode = "auto"`, cada prueba asíncrona se recopila y ejecuta automáticamente. El decorador `@pytest.mark.asyncio` **no es obligatorio** pero es inocuo y hace que la intención asíncrona resulte explícita de un vistazo. Lumen lo conserva por claridad.

Pruebas del adaptador del repositorio

Las pruebas de flujo CQRS demuestran la canalización completa de apertura/depósito/retiro/consulta. Las pruebas del adaptador del repositorio van un nivel más hondo: ejercitan directamente la API de `WalletRepository` —CRUD, **consultas derivadas** compiladas a partir de nombres de método, paginación con `Pageable / Page` y predicados de tipo `Specification` — contra una base de datos SQLite en un archivo temporal. Sin Docker, sin proceso externo, sin red. El fixture integrado `tmp_path` de pytest proporciona un directorio temporal que se limpia automáticamente tras cada prueba.

El fixture local `_make_repo` refleja lo que hace el `ApplicationContext` al arrancar: construir el repositorio y ejecutar `RepositoryBeanPostProcessor.after_init` para compilar los esbozos de consulta derivada. Sin esa llamada, métodos como `find_by_owner_id` lanzarían `NotImplementedError`.

Dos términos antes del listado. Una **consulta derivada** es un método de repositorio cuyo cuerpo se genera a partir de su *nombre*: `find_by_owner_id` se convierte en `WHERE owner_id = :value`, sin SQL escrito a mano. Una **Specification** es un objeto predicado reutilizable y componible que pasas a una consulta —

`balance_at_least(1000)` es uno— para filtros demasiado dinámicos como para incrustarlos en un nombre de método. La **paginación** envuelve ambos: `Pageable.of(page, size, sort)` describe qué porción quieres, y la consulta devuelve una `Page` que lleva los elementos más `total`, `total_pages` y `has_next`.

Esta sección usa una base de datos SQLite **basada en archivo** (vía `tmp_path`) en lugar de la de en memoria, para poder demostrar una propiedad más fuerte: la durabilidad tras una reconexión. Esta es la forma de cada prueba:

Paso 1 — `sqlite_factory` : un motor sobre archivo temporal. El fixture construye un motor SQLite respaldado por un archivo real bajo el `tmp_path` de pytest (un directorio temporal nuevo por prueba, autoeliminado después), crea el esquema y entrega tanto la fábrica como la URL. Entregar la URL es lo que permite que una prueba se reconecte con un motor totalmente nuevo.

Paso 2 — CRUD y persistencia. `test_upsert_inserts_then_updates_and_persists` hace un upsert de una fila, lo repite con un nuevo saldo para demostrar la actualización en sitio, hace commit, la lee de vuelta y luego desecha el motor y se reconecta con uno *nuevo* para demostrar que los datos realmente llegaron al disco.

Paso 3 — el camino de id desconocido. `test_find_by_id_unknown_returns_none` confirma que un fallo devuelve `None` , no un error.

Paso 4 — consulta derivada. `test_derived_find_by_owner_id` inserta monederos para dos propietarios y comprueba que `find_by_owner_id("alice")` devuelve solo los de Alice: prueba de que el post-procesador compiló correctamente la convención de nombres de método.

Paso 5 — Specification + paginación.

`test_specification_find_rich_paged_and_sorted` y `test_find_all_pageable_counts_and_pages` ejercitan el camino del predicado `find_rich` / `find_all_by_spec` y el camino de paginación `find_all(pageable)` , comprobando `total` , `total_pages` , `has_next` y el orden exacto de `page.items` .

tests/test_sql_wallet_repository.py

```

from __future__ import annotations

from collections.abc import AsyncIterator
from datetime import UTC, datetime, timedelta
from pathlib import Path

import pytest
import pytest_asyncio
from sqlalchemy.ext.asyncio import (
    AsyncSession,
    async_sessionmaker,
    create_async_engine,
)

from lumen.models.entities.v1.wallet_orm import WalletEntity
from lumen.models.repositories.wallet_repository import (
    WalletRepository,
    balance_at_least,
)

from pyfly.data import Pageable, Sort
from pyfly.data.relational.sqlalchemy import Base
from pyfly.data.relational.sqlalchemy.post_processor import (
    RepositoryBeanPostProcessor,
)

def _entity(
    wid: str,
    owner: str,
    minor: int,
    *,
    currency: str = "EUR",
    age_days: int = 0,
) → WalletEntity:
    created = datetime.now(UTC) - timedelta(days=age_days)
    return WalletEntity(
        id=wid,

```

```

        owner_id=owner,
        currency=currency,
        balance_minor=minor,
        created_at=created,
    )

def _make_repo(session: AsyncSession) → WalletRepository:
    repo = WalletRepository(WalletEntity, session)
    # Mirror the ApplicationContext: compile derived-query stubs.
    RepositoryBeanPostProcessor().after_init(repo, "walletRepository")
    return repo

@pytest_asyncio.fixture
async def sqlite_factory(
    tmp_path: Path,
) → AsyncIterator[tuple[async_sessionmaker[AsyncSession], str]]:
    """A temp-file SQLite engine + session factory, schema created.

    Yields the session factory and the database URL so the test can
    reconnect with a fresh engine to verify true persistence.
    """
    db_url = f"sqlite+aiosqlite:/// {tmp_path / 'wallets.db'}"
    engine = create_async_engine(db_url)
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
    factory = async_sessionmaker(engine, expire_on_commit=False)
    try:
        yield factory, db_url
    finally:
        await engine.dispose()

@pytest.mark.asyncio
async def test_upsert_inserts_then_updates_and_persists(
    sqlite_factory: tuple[async_sessionmaker[AsyncSession], str],
) → None:
    factory, db_url = sqlite_factory

```

```

async with factory() as session:
    repo = _make_repo(session)
    await repo.upsert(_entity("wlt-1", "owner-42", 0, currency="USD"))
    # update: same PK, new balance
    await repo.upsert(_entity("wlt-1", "owner-42", 2500, currency="USD"))
    await session.commit()

    got = await repo.find_by_id("wlt-1")
    assert got is not None
    assert got.owner_id == "owner-42"
    assert got.currency == "USD"
    assert got.balance_minor == 2500
    assert await repo.count() == 1

# prove persistence: reconnect with a brand-new engine/session
fresh_engine = create_async_engine(db_url)
fresh_factory = async_sessionmaker(fresh_engine, expire_on_commit=False)
try:
    async with fresh_factory() as fresh_session:
        fresh_repo = _make_repo(fresh_session)
        persisted = await fresh_repo.find_by_id("wlt-1")
        assert persisted is not None, "wallet should survive a reconnect"
        assert persisted.balance_minor == 2500
finally:
    await fresh_engine.dispose()

@pytest.mark.asyncio
async def test_find_by_id_unknown_returns_none(
    sqlite_factory: tuple[async_sessionmaker[AsyncSession], str],
) → None:
    factory, _ = sqlite_factory
    async with factory() as session:
        repo = _make_repo(session)
        assert await repo.find_by_id("wlt-nope") is None

@pytest.mark.asyncio

```

```

async def test_derived_find_by_owner_id(
    sqlite_factory: tuple[async_sessionmaker[AsyncSession], str],
) → None:
    factory, _ = sqlite_factory
    async with factory() as session:
        repo = _make_repo(session)
        await repo.upsert(_entity("wlt-1", "alice", 100))
        await repo.upsert(_entity("wlt-2", "alice", 200))
        await repo.upsert(_entity("wlt-3", "bob", 300))
        await session.commit()

        owned = await repo.find_by_owner_id("alice")
        assert sorted(w.id for w in owned) == ["wlt-1", "wlt-2"]
        assert await repo.find_by_owner_id("nobody") == []

@pytest.mark.asyncio
async def test_specification_find_rich_paged_and_sorted(
    sqlite_factory: tuple[async_sessionmaker[AsyncSession], str],
) → None:
    factory, _ = sqlite_factory
    async with factory() as session:
        repo = _make_repo(session)
        # age_days drives created_at so we can assert newest-first ordering.
        await repo.upsert(_entity("wlt-poor", "a", 50, age_days=3))
        await repo.upsert(_entity("wlt-mid", "b", 1000, age_days=2))
        await repo.upsert(_entity("wlt-rich", "c", 5000, age_days=1))
        await session.commit()

        # Specification: balance_minor ≥ 1000, newest first, page size 1.
        newest_first = Sort.by("created_at").descending()
        page = await repo.find_rich(1000, Pageable.of(1, 1, newest_first))
        assert page.total == 2 # mid + rich
        assert page.total_pages == 2
        assert page.has_next is True
        assert [w.id for w in page.items] == ["wlt-rich"]

        page2 = await repo.find_rich(1000, Pageable.of(2, 1, newest_first))
        assert [w.id for w in page2.items] == ["wlt-mid"]

```

```

# The bare predicate also works through find_all_by_spec.
rich = await repo.find_all_by_spec(balance_at_least(5000))
assert [w.id for w in rich] == ["wlt-rich"]

@pytest.mark.asyncio
async def test_find_all_pageable_counts_and_pages(
    sqlite_factory: tuple[async_sessionmaker[AsyncSession], str],
) → None:
    factory, _ = sqlite_factory
    async with factory() as session:
        repo = _make_repo(session)
        for i in range(5):
            await repo.upsert(
                _entity(f"wlt-{i}", "owner", i * 100, age_days=5 - i)
            )
        await session.commit()

    page = await repo.find_all(
        Pageable.of(1, 2, Sort.by("created_at").descending())
    )
    assert page.total == 5
    assert page.total_pages == 3
    assert len(page.items) == 2
    # newest first → wlt-4 (age 1 day), then wlt-3
    assert [w.id for w in page.items] == ["wlt-4", "wlt-3"]

```

Listado 16.5 — Pruebas de repositorio: CRUD, consulta derivada, paginación, Specification

Cuatro cosas a destacar. Primero, `_make_repo` llama a `RepositoryBeanPostProcessor().after_init(repo, ...)`; sin esto, `find_by_owner_id` sigue siendo un esbozo y lanza `NotImplementedError`. El post-procesador compila el nombre de método en una cláusula `WHERE owner_id = :owner_id` de SQLAlchemy. Segundo, `upsert` es la inserción-o-actualización del repositorio; tras cada lote de upserts, `await session.commit()` vuelca a SQLite.

Tercero, `find_rich` toma un saldo mínimo y un `Pageable` ; delega en `find_all_by_spec_paged(balance_at_least(min), pageable)` . Cuarto, el patrón de dos motores en `test_upsert_inserts_then_updates_and_persists` demuestra la durabilidad verdadera: los datos confirmados a través de un motor son legibles por un motor y una sesión completamente nuevos.

Ejecútalo.

```
uv run --extra dev pytest tests/test_sql_wallet_repository.py -q
```

Salida esperada:

```
.....
[100%]
5 passed in 0.06s
```

Qué acaba de pasar. Lo más notable es la reconexión de dos motores en la primera prueba. Muchas pruebas de "persistencia" pasan incluso cuando no se ha escrito nada en el disco, porque la misma sesión cachea el objeto en memoria y lo devuelve en la lectura. Al desechar el motor por completo y abrir un *segundo* motor contra la misma URL de archivo, esta prueba fuerza un viaje de ida y vuelta real al almacenamiento; si `upsert` o `commit` no estuvieran persistiendo silenciosamente, la reconexión devolvería `None` y la prueba fallaría. Esa es la diferencia entre probar tu código y probar tu caché.

EQUIVALENCIA CON SPRING

Esta capa de prueba es el equivalente en Python de `@DataJpaTest` con una base de datos H2 embebida en Spring Boot. `@DataJpaTest` carga solo la capa JPA (entidades, repositorios, Flyway) y cablea una H2 en memoria nueva para cada clase de prueba. El fixture `sqlite_factory` hace lo mismo: crear el esquema, ejecutar las pruebas, desechar el motor. Sin Docker, sin proceso externo.

💡 LAS CONSULTAS DERIVADAS SON CONVENCIONES DE NOMBRE DE MÉTODO

`WalletRepository.find_by_owner_id` se declara como un esbozo (`raise NotImplementedError`). `RepositoryBeanPostProcessor` inspecciona el nombre del método al arrancar — `find_by_owner_id` → `WHERE owner_id = :value` — y reemplaza el esbozo por una corrutina real. Por tanto, probar este método también prueba que la convención del post-procesador funciona correctamente.

Pruebas del listener de eventos

`WalletAuditListener` escucha los eventos de dominio publicados por los manejadores de comandos. Probarlo de extremo a extremo —el comando se ejecuta en el bus, el manejador publica eventos, el listener los recibe— requiere que los tres componentes compartan el mismo `InMemoryEventBus`. Los fixtures de `conftest.py` ya lo disponen: tanto `command_bus` como `audit_listener` aceptan un argumento `event_bus`, y pytest inyecta la misma instancia en ambos.

Un **listener de eventos** es simplemente un método que el framework suscribe al bus para que se ejecute cada vez que se publica un evento coincidente. El

`WalletAuditListener` de Lumen mantiene un registro de auditoría en memoria y un saldo acumulado por monedero: una pequeña **proyección** (un modelo de lectura construido plegando eventos). Probarlo es la demostración más clara del truco del fixture compartido: como `command_bus` y `audit_listener` nombran el mismo `event_bus`, un evento que publica un comando es un evento que observa el listener, sin pagamento alguno en el cuerpo de la prueba.

Las pruebas siguen un solo ritmo:

Paso 1 — recorrer comandos. Abrir un monedero, depositar, retirar; todo a través de `command_bus`.

Paso 2 — leer la proyección. Llamar a `audit_listener.entries_for(wallet_id)` y comprobar los tipos de evento registrados, en orden, más el `running_total`.

Paso 3 — comprobar el negativo. Una prueba deja deliberadamente el saldo en descubierto —un comando que debe fallar— y comprueba que el registro de auditoría no anota nada de él. Una operación fallida no deja rastro.

tests/test_event_listener.py

```

from __future__ import annotations

import pytest
from lumen.core.services.listeners import WalletAuditListener
from lumen.core.services.wallets.deposit_funds_command import DepositFunds
from lumen.core.services.wallets.open_wallet_command import OpenWallet
from lumen.core.services.wallets.withdraw_funds_command import WithdrawFunds
from lumen.interfaces.enums.v1.currency import Currency

from pyfly.cqrs import DefaultCommandBus

@pytest.mark.asyncio
async def test_listener_observes_wallet_events(
    command_bus: DefaultCommandBus,
    audit_listener: WalletAuditListener,
) → None:
    wallet_id = await command_bus.send(
        OpenWallet(owner_id="u-1", currency=Currency.EUR)
    )
    await command_bus.send(DepositFunds(wallet_id=wallet_id, amount=1500))
    await command_bus.send(WithdrawFunds(wallet_id=wallet_id, amount=400))

    entries = audit_listener.entries_for(wallet_id)
    assert [e.event_type for e in entries] == [
        "WalletOpened",
        "FundsDeposited",
        "FundsWithdrawn",
    ]

    # The payload carried the real domain-event fields.
    deposited = entries[1]
    assert deposited.payload["amount"] == 1500
    assert deposited.payload["currency"] == "EUR"
    assert deposited.payload["balance"] == 1500
    assert deposited.event_id # the aggregate's DomainEvent.event_id

```

```

# The running-total projection reflects deposit - withdrawal.
assert audit_listener.running_total(wallet_id) == 1100

@pytest.mark.asyncio
async def test_listener_records_nothing_before_any_command(
    audit_listener: WalletAuditListener,
) → None:
    assert audit_listener.entries == []
    assert audit_listener.running_total("anything") == 0

@pytest.mark.asyncio
async def test_event_type_matches_domain_event_class_names(
    command_bus: DefaultCommandBus,
    audit_listener: WalletAuditListener,
) → None:
    # A rejected withdrawal raises no event – it must not appear in the log.
    wallet_id = await command_bus.send(
        OpenWallet(owner_id="u-2", currency=Currency.USD)
    )
    await command_bus.send(DepositFunds(wallet_id=wallet_id, amount=100))

    from pyfly.cqrs.exceptions import CommandProcessingException

    with pytest.raises(CommandProcessingException):
        await command_bus.send(
            WithdrawFunds(wallet_id=wallet_id, amount=9999)
        )

    types = [e.event_type for e in audit_listener.entries_for(wallet_id)]
    assert types == ["WalletOpened", "FundsDeposited"]
    assert audit_listener.running_total(wallet_id) == 100

```

Listado 16.6 — Pruebas del listener de eventos: el comando publica, el listener observa

`test_listener_observes_wallet_events` es la prueba de integración central: tres comandos producen tres eventos, el listener los registra los tres en orden, los campos del payload coinciden con los campos de la dataclass de evento del agregado y la proyección `running_total` es igual al resultado aritmético. Sin mock del bus, sin lista de captura de eventos: el listener de producción se ejecuta sobre el bus de producción.

`test_event_type_matches_domain_event_class_names` demuestra un invariante de dominio: un comando rechazado (descubierto) no lanza ningún evento. El registro de auditoría nunca debe anotar un efecto secundario de una operación fallida.

Ejecútalo.

```
uv run --extra dev pytest tests/test_event_listener.py -q
```

Salida esperada:

```
...
[100%]
3 passed in 0.04s
```

Qué acaba de pasar. Ninguna parte de la prueba conectó el listener al bus: el fixture `audit_listener` hizo eso en `confptest.py`, suscribiendo el manejador al mismo `event_bus` por el que publica el `command_bus`. Así que enviar un comando y luego leer `entries_for(...)` ejercita el camino real de publicación/suscripción de extremo a extremo. La prueba negativa es la sutil: demuestra que el registro de auditoría se rige por *eventos*, no por *intentos*; un retiro rechazado lanza un `BusinessRuleViolation` antes de que se emita ningún evento, así que nada llega al listener.

💡 **EVENT_TYPE ES EL NOMBRE DE LA CLASE**

El publicador de eventos de PyFly fija `event_type` al nombre de la clase del evento de dominio: `"WalletOpened"`, `"FundsDeposited"`, `"FundsWithdrawn"`. El decorador `@event_listener(event_types=["WalletOpened", "FundsDeposited", "FundsWithdrawn"])` sobre `WalletAuditListener.on_wallet_event` nombra esos tres tipos explícitamente; el framework los almacena en el método como `__pyfly_event_patterns__`, que el fixture `audit_listener` lee para suscribirse. La prueba comprueba directamente las cadenas con los nombres de clase.

Prueba de integración con el contexto arrancado

Las pruebas unitarias, las de flujo CQRS y las de repositorio cablean cada una una capa del stack. La prueba de integración con el contexto arrancado las cablea todas a la vez: inicia el `ApplicationContext` real —escaneo de componentes de inyección de dependencias, autoconfiguración de CQRS, autoconfiguración relacional, `RepositoryBeanPostProcessor`, la costura `@transactional`, el bus de eventos de la EDA— y luego resuelve el `DefaultCommandBus` y el `DefaultQueryBus` desde el contexto y recorre el ciclo de vida completo del monedero.

El **ApplicationContext** es el contenedor en tiempo de ejecución de PyFly: el objeto que escanea en busca de componentes, construye beans, ejecuta post-procesadores y mantiene unida la aplicación cableada. Arrancarlo es la prueba más fiel que puedes escribir sin llegar a iniciar un servidor HTTP: cada pieza de cableado que el framework hace al arrancar ocurre de verdad.

La URL de la base de datos se sustituye mediante una variable de entorno para que la prueba nunca toque el `Lumen.db` del desarrollador. Este es el plan:

Paso 1 — aislar la base de datos. El fixture `booted_context` usa el fixture `monkeypatch` de pytest para fijar `PYFLY_DATA_RELATIONAL_URL` a una ruta SQLite de archivo temporal bajo `tmp_path`, *antes* de que la aplicación arranque. `monkeypatch` es la forma segura de pytest de fijar una variable de entorno durante una sola prueba y restaurarla automáticamente después, así que esta prueba nunca puede pisar tu `lumen.db` real.

Paso 2 — arrancar la aplicación real. Construye

`PyFlyApplication(LumenApplication, config_path=...)` y `await app.startup()`.

Esa única llamada ejecuta toda la secuencia de arranque: escaneo de componentes, todas las autoconfiguraciones, el `RepositoryBeanPostProcessor` y el bus de eventos. El fixture entrega `app.context` y, en su bloque `finally`, cierra la sesión compartida y llama a `app.shutdown()`.

Paso 3 — resolver beans y recorrer el ciclo de vida. La prueba llama a

`ctx.get_bean(DefaultCommandBus)` y `ctx.get_bean(DefaultQueryBus)` —

obteniendo los *mismos* buses que usaría la aplicación— y luego ejecuta apertura → depósito → retiro → listado → ricos → saldo, comprobando cada resultado.

tests/test_app_context_integration.py

```

from __future__ import annotations

import logging
import sys
from collections.abc import AsyncIterator
from pathlib import Path

import pytest
import pytest_asyncio
from sqlalchemy.ext.asyncio import AsyncSession

_HERE = Path(__file__).resolve().parent
_SRC = _HERE.parent / "src"
sys.path.insert(0, str(_SRC))

@pytest_asyncio.fixture
async def booted_context(
    tmp_path: Path,
    monkeypatch: pytest.MonkeyPatch,
) → AsyncIterator[object]:
    """Boot the full LumenApplication against an isolated SQLite file."""
    db_path = tmp_path / "lumen-it.db"
    monkeypatch.setenv(
        "PYFLY_DATA_RELATIONAL_URL",
        f"sqlite+aiosqlite:/// {db_path}",
    )
    # Silence the pool-GC warning that aiosqlite emits at teardown.
    logging.getLogger(
        "sqlalchemy.pool.impl.AsyncAdaptedQueuePool"
    ).setLevel(logging.CRITICAL)

    from lumen.app import LumenApplication
    from pyfly.core import PyFlyApplication

    app = PyFlyApplication(
        LumenApplication,

```

```

        config_path=str(_HERE.parent / "pyfly.yaml"),
    )
    await app.startup()
    try:
        yield app.context
    finally:
        await app.context.get_bean(AsyncSession).close()
        await app.shutdown()

@pytest.mark.asyncio
async def test_full_lifecycle_through_booted_context(
    booted_context: object,
) → None:
    from lumen.core.services.wallets.deposit_funds_command import DepositFunds
    from lumen.core.services.wallets.get_balance_query import GetBalance
    from lumen.core.services.wallets.get_wallet_query import GetWallet
    from lumen.core.services.wallets.list_rich_wallets_query import (
        ListRichWallets,
    )
    from lumen.core.services.wallets.list_wallets_query import ListWallets
    from lumen.core.services.wallets.open_wallet_command import OpenWallet
    from lumen.core.services.wallets.withdraw_funds_command import
WithdrawFunds
    from lumen.interfaces.enums.v1.currency import Currency
    from pyfly.cqrs import DefaultCommandBus, DefaultQueryBus
    from pyfly.data import Pageable

    ctx = booted_context
    commands = ctx.get_bean(DefaultCommandBus)
    queries = ctx.get_bean(DefaultQueryBus)

    # --- open → deposit → withdraw, each a committed unit of work -----
    w1 = await commands.send(OpenWallet(owner_id="u-1",
currency=Currency.EUR))
    w2 = await commands.send(OpenWallet(owner_id="u-2",
currency=Currency.EUR))
    assert w1.startswith("wlt-") and w2.startswith("wlt-")

```

```

    assert await commands.send(DepositFunds(wallet_id=w1, amount=5000)) ==
5000
    assert await commands.send(WithdrawFunds(wallet_id=w1, amount=1500)) ==
3500
    assert await commands.send(DepositFunds(wallet_id=w2, amount=100)) == 100

# --- persistence survived: reload the aggregate via the query side ---
reloaded = await queries.query(GetWallet(wallet_id=w1))
assert reloaded is not None
assert reloaded.owner_id == "u-1"
assert reloaded.balance_minor == 3500

# --- paged list (find_all(pageable) + Page.map) -----
page = await queries.query(ListWallets(pageable=Pageable.of(1, 10)))
assert page.total == 2
assert {w.id for w in page.items} == {w1, w2}

# --- Specification: only wallets with balance ≥ 1000 -----
rich = await queries.query(
    ListRichWallets(min_minor=1000, pageable=Pageable.of(1, 10))
)
assert rich.total == 1
assert [w.id for w in rich.items] == [w1]

everyone = await queries.query(
    ListRichWallets(min_minor=0, pageable=Pageable.of(1, 10))
)
assert everyone.total == 2

# --- projection-backed balance -----
balance = await queries.query(GetBalance(wallet_id=w1))
assert balance is not None
assert balance.balance_minor == 3500
assert balance.balance == 35.0

```

Listado 16.7 — Integración con el contexto arrancado: composición completa de inyección de dependencias + persistencia

`booted_context` usa el fixture integrado `monkeypatch` de pytest para fijar `PYFLY_DATA_RELATIONAL_URL` antes de que la aplicación arranque. El framework lee esta variable de entorno durante la autoconfiguración relacional, así que el contexto usa la base de datos SQLite de archivo temporal aislada durante la vida de la prueba, y luego la desecha cuando el fixture se desmonta.

`test_full_lifecycle_through_booted_context` ejercita cada tipo de consulta que la aplicación expone: `GetWallet` (recarga del agregado), `ListWallets` (listado paginado usando `find_all(pageable)`), `ListRichWallets` (predicado de tipo Specification usando `find_all_by_spec_paged`) y `GetBalance` (saldo respaldado por proyección). Demuestra que el `RepositoryBeanPostProcessor`, la frontera `@transactional` alrededor de cada manejador de comandos y el cableado de inyección de dependencias se componen todos correctamente en un solo arranque.

Ejecútalo.

```
uv run --extra dev pytest tests/test_app_context_integration.py -q
```

Salida esperada:

```
.
[100%]
1 passed in 0.15s
```

Una prueba, pero la más pesada de la suite: realmente arrancó el framework. Si el escaneo de inyección de dependencias se saltó un bean, una autoconfiguración cableó mal o la frontera `@transactional` no consiguió hacer commit, esta es la prueba que lo atrapa, lo cual es exactamente por lo que se sitúa en la cima de la pirámide y por lo que solo hay una de ella.

Qué acaba de pasar. La sustitución de la variable de entorno es el truco que lo sostiene todo. El framework lee `pyfly.data.relational.url` de la configuración durante la autoconfiguración relacional, y PyFly mapea cualquier clave de configuración a una variable de entorno `PYFLY_*` (los puntos y los guiones se vuelven guiones bajos, en mayúsculas), así que `PYFLY_DATA_RELATIONAL_URL` sustituye la `url` de `pyfly.yaml`. Fijarla con `monkeypatch` antes de `app.startup()` es lo que redirige toda la aplicación arrancada a una base de datos desechable, y `monkeypatch` deshace el cambio cuando el fixture se desmonta, así que ninguna otra prueba se ve afectada.

🔗 UN PERFIL DE PRUEBA DEDICADO (V26.6.110)

Fijar una variable está bien para una sola sustitución. Cuando un proyecto necesita un bloque entero de ajustes solo de prueba, el mecanismo de **perfiles** de PyFly (equivalencia con Spring) es más limpio: deja un `pyfly-test.yaml` junto a `pyfly.yaml` con tus sustituciones de prueba, luego actívalo fijando `PYFLY_PROFILES_ACTIVE=test` (o `pyfly.profiles.active: test` en la configuración). Al arrancar, PyFly superpone `pyfly-test.yaml` sobre el `pyfly.yaml` base, así que valores como la URL de la base de datos o `ddl-auto` se aplican solo bajo ese perfil. Lumen no necesita un perfil —una sola sustitución de entorno cubre su único ajuste solo de prueba—, pero recurre a uno a medida que la configuración de pruebas crezca.

🍃 EQUIVALENCIA CON SPRING

Esta prueba es el equivalente en Python de `@SpringBootTest` con una base de datos H2 embebida. `@SpringBootTest` carga el contexto de aplicación completo, incluidas todas las autoconfiguraciones y la capa JPA; fijas `spring.datasource.url` en `application-test.properties` para redirigir a H2. La sustitución de variable de entorno de PyFly (`monkeypatch.setenv`) cumple el mismo papel. Ambos enfoques demuestran que la aplicación compuesta funciona de extremo a extremo sin infraestructura externa alguna.

Ayudantes de pruebas del framework

Las pruebas de arriba cubren la pirámide completa de Lumen con primitivas estándar de pytest y los componentes reales de producción de PyFly. Para aplicaciones más grandes o equipos que prefieren más estructura, `pyfly.testing` incluye ayudantes de más alto nivel que reflejan las anotaciones de pruebas de Spring Boot.

`PyFlyTestCase` + `mock_bean(T)` funcionan como `@MockBean` en `@SpringBootTest`. Declara `repo = mock_bean(WalletDomainRepository)` en el cuerpo de la clase; `setup()` instala un `AsyncMock(spec=T)` en el contexto de aplicación y lo cablea en cualquier colaborador que dependa de él.

`create_test_container(overrides={Interface: Implementation})` construye un contenedor de inyección de dependencias con dobles (fakes) registrados para interfaces concretas. Resuelve desde él la clase bajo prueba y sus dependencias ya quedan inyectadas.

`assert_event_published(events, event_type, payload_contains=...)` rastrea una lista capturada de `EventEnvelope` en busca del primer sobre del tipo dado, opcionalmente comprueba claves del payload y devuelve el sobre para aserciones posteriores. `assert_no_events_published(events)` falla si la lista no está vacía.

Integración con Testcontainers (`postgres_container()`, `redis_container()`, `pyfly_config(container, base={...})`) es el equivalente de PyFly a `@Testcontainers` + `@ServiceConnection` de Spring Boot. Arranca un contenedor Postgres real; `pyfly_config` reescribe la URL síncrona `psycopg2://` a `postgresql+asyncpg://`, activa la capa relacional y la fusiona en un `Config` listo para arrancar un `ApplicationContext`. En un servidor de base de datos `ddl-auto` vale `none` por defecto, así que pasa `create` en `base` cuando la prueba necesite las tablas de sus modelos. Instala el soporte con:

```
pip install 'pyfly[testcontainers]'
```

Protege cada prueba de Testcontainers con `@requires_docker` para que se omita limpiamente en máquinas sin Docker y se ejecute automáticamente en los ejecutores de CI que sí lo tengan:

```
from pyfly.testing import postgres_container, pyfly_config, requires_docker

@requires_docker
async def test_wallet_round_trip_against_real_postgres():
    with postgres_container() as pg:
        config = pyfly_config(
            pg, base={"pyfly.data.relational.ddl-auto": "create"}
        )
        assert config.get("pyfly.data.relational.url").startswith(
            "postgresql+asyncpg://"
        )
    ...
```

`@DataTest` es el `@DataJpaTest` de PyFly. Ejecuta cada prueba de una clase en un slice de datos arrancado con los beans que indiques, entrega el contexto a la prueba como el fixture `data_context` y **revierte todas las unidades de trabajo de la prueba** al terminar, de modo que las pruebas que comparten una base de datos nunca ven las filas de las demás:

```

from lumen.models.entities.v1.wallet_orm import WalletEntity
from lumen.models.repositories.wallet_repository import WalletRepository

from pyfly.context.application_context import ApplicationContext
from pyfly.testing import DataTest

@DataTest(beans=[WalletRepository])
class TestWalletRepositorySlice:
    async def test_upsert_then_count(
        self, data_context: ApplicationContext
    ) → None:
        wallets = data_context.get_bean(WalletRepository)
        await wallets.upsert(
            WalletEntity(id="wlt-1", owner_id="alice", currency="EUR")
        )
        assert await wallets.count() == 1

    async def test_starts_empty(
        self, data_context: ApplicationContext
    ) → None:
        # The row of the test above was rolled back.
        assert await data_context.get_bean(WalletRepository).count() == 0

```

Por defecto cada prueba recibe un fichero SQLite en su `tmp_path` con la capa relacional activada. Sobrescribe el fixture `pyfly_data_config` en `conftest.py` para ejecutar todas las pruebas de datos sobre PostgreSQL, MySQL (`mysql_container()`), con el driver asyncmy de `pyfly[mysql]`) o MariaDB (`mariadb_container()`), y la reversión mantiene separadas las pruebas en el servidor compartido; `data_slice(..., rollback=True)` da la misma reversión a una sola prueba. Dos cosas difieren de producción por construcción: el trabajo desacoplado (los pasos de una saga, las llamadas `@async_method`) se rechaza mientras hay abierta una unidad

de la prueba, y solo se revierten los orígenes de datos relacionales. Las transacciones de MongoDB se prueban sobre `mongodb_replica_set_container()` (Apéndice B). Hasta la v26.09.07 `@DataTest` solo marcaba la clase.

Lumen no usa estos ayudantes: SQLite cubre la capa de persistencia sin Docker, y el bus en memoria cubre el enrutamiento de eventos. Recurre a ellos cuando tu proyecto tenga infraestructura que no pueda reproducirse sin un demonio real.

Ejecútalo. Ya has recorrido cada capa archivo a archivo. Ejecuta toda la suite una vez más para confirmar que la pirámide completa está en verde en conjunto:

```
uv run --extra dev pytest -q
```

Salida esperada: el mismo `43 passed` con el que empezaste, ahora con un modelo mental de exactamente lo que demuestra cada punto:

```
..S.S.....  
[100%]  
43 passed, 2 skipped in 0.28s
```

✓ Lo que construiste

Los seis archivos de prueba que construyó este capítulo suman 26 pruebas superadas, ejercitando cada capa de la pirámide. Junto con las pruebas de la saga del Capítulo 12, las pruebas de event sourcing del Capítulo 9 y la prueba de concurrencia del Capítulo 5, la suite completa de Lumen son **43 pruebas superadas** (y dos casos de PostgreSQL que se omiten sin servidor): el recuento que viste cuando ejecutaste `uv run --extra dev pytest -q` al principio.

En la base, `test_money.py` y `test_wallet_aggregate.py` demuestran la aritmética, la inmutabilidad y las reglas de invariante del modelo de dominio. Todas las pruebas son funciones síncronas de Python puro, sin fixtures, sin inyección de dependencias, sin `async`. El atributo `BusinessRuleViolation.rule` hace que cada aserción sea específica del invariante exacto incumplido.

En el centro, `conftest.py` cablea los componentes reales —el `WalletRepository` del framework sobre un motor SQLite en memoria, `InMemoryEventBus`, los cinco manejadores de comandos y consultas (incluidos `ListWalletsHandler` y `ListRichWalletsHandler`) y `WalletAuditListener` — en fixtures asíncronos reutilizables que pytest comparte automáticamente entre módulos. El `RepositoryBeanPostProcessor` se aplica al fixture del repositorio exactamente como el `ApplicationContext` lo aplica al arrancar. `test_cqrs_flow.py` despacha comandos y consultas a través del bus real y comprueba cada campo de los DTO de consulta. `test_event_listener.py` demuestra que el listener de auditoría observa exactamente los eventos producidos por comandos exitosos y nada de los rechazados.

`test_sql_wallet_repository.py` ejercita el `WalletRepository` directamente contra un archivo SQLite temporal, cubriendo la superficie CRUD completa, la consulta derivada `find_by_owner_id` (compilada a partir del nombre del método por `RepositoryBeanPostProcessor`), la API `find_all(pageable)` que devuelve una `Page` con recuento total y metadatos de página, y el camino del predicado de tipo `Specification` vía `find_rich` / `find_all_by_spec`. El patrón de reconexión de dos motores demuestra la durabilidad verdadera.

En la cima, `test_app_context_integration.py` arranca la `LumenApplication` real con la URL de la base de datos sustituida por un archivo SQLite aislado, y luego recorre el ciclo de vida completo de apertura → depósito → retiro → listado → ricos → saldo a través de los buses resueltos por el contexto. Esta única prueba demuestra que

el escaneo de inyección de dependencias, la autoconfiguración de CQRS, el `RepositoryBeanPostProcessor` y la frontera `@transactional` se componen todos correctamente.

En concreto, aprendiste:

- `asyncio_mode = "auto" + pythonpath = ["src"]` en `pyproject.toml`: todas las pruebas asíncronas se ejecutan sin decoradores; la disposición `src/` es importable.
- `uv run --extra dev pytest -q`: el `uv sync` a secas omite el grupo de desarrollo; incluye siempre `--extra dev` para obtener pytest.
- `@pytest_asyncio.fixture`: ciclo de vida del fixture asíncrono gestionado por pytest-asyncio; el `@pytest.fixture` simple no maneja generadores asíncronos.
- **Instancias de fixture compartidas**: cuando dos fixtures solicitan el mismo nombre de fixture (p. ej., `event_bus`, `session_factory`), pytest lo resuelve una vez por prueba y comparte la instancia.
- `RepositoryBeanPostProcessor().after_init(repo, name)`: debe llamarse en las pruebas que ejercitan consultas derivadas; sin él, los esbozos de método lanzan `NotImplementedError`.
- **Consultas derivadas** (`find_by_owner_id`): declaradas como esbozos; el post-procesador las compila a `WHERE owner_id = :value` al arrancar.
- `Pageable.of(page, size, sort) + Page`: la API `find_all(pageable)` devuelve una `Page` con `total`, `total_pages`, `has_next` e `items`; comprueba cada campo para verificar la corrección de la paginación.
- **Predicados de tipo Specification**: `balance_at_least(n)` se pasa a `find_rich` / `find_all_by_spec` para filtrar por un predicado arbitrario sin añadir un nuevo método de consulta derivada.

- `pending_events()` frente a `clear_events()` : `pending_events()` lee sin vaciar; `clear_events()` vacía. Llama siempre a `clear_events()` en los pasos de organización para que las aserciones solo vean eventos del paso de actuación.
 - `BusinessRuleViolation.rule` : comprueba la cadena de regla exacta, no solo la clase de excepción, para demostrar que se disparó el invariante correcto.
 - `monkeypatch.setenv` : sustituye la configuración antes de arrancar el contexto en las pruebas de integración; el framework lee variables de entorno durante la autoconfiguración.
 - **Sustituciones de configuración** `PYFLY_*` : cada clave de configuración se mapea a una variable de entorno (`pyfly.data.relational.url` → `PYFLY_DATA_RELATIONAL_URL`); fíjala con `monkeypatch.setenv` antes de arrancar para redirigir toda la aplicación.
 - **Perfil de prueba (v26.6.110)**: para un bloque de ajustes solo de prueba, añade un superpuesto `pyfly-test.yaml` y actívalo con `PYFLY_PROFILES_ACTIVE=test` (equivalencia con el `application-test.yaml` de Spring).
 - `@DataTest` — un slice de datos cuyas pruebas revierten todas sus unidades de trabajo, sobre SQLite por defecto o sobre un servidor compartido mediante `pyfly_data_config`.
 - **Ayudantes del framework** (`PyFlyTestCase` , `mock_bean` , `create_test_container` , `Testcontainers`): disponibles en `pyfly.testing` para proyectos que los necesiten; Lumen lo mantiene simple con componentes reales.
-

Pruébalo tú mismo

1. Añade una prueba para el retiro de importe cero. En

`test_wallet_aggregate.py`, añade `test_withdraw_zero_is_rejected`. Abre un monedero, deposita 500 EUR y luego intenta `wallet.withdraw(Money(0, Currency.EUR))`. Comprueba que se lanza una `BusinessRuleViolation` y verifica su atributo `.rule`. Compara el nombre de la regla con el equivalente del depósito: ¿son simétricas las reglas?

2. Prueba un monedero desconocido vía el bus de CQRS. En

`test_cQRS_flow.py`, añade `test_withdraw_from_unknown_wallet_is_rejected`. Envía solo un comando `WithdrawFunds(wallet_id="wlt-ghost", amount=50)` sin abrir un monedero antes. Comprueba que se lanza `CommandProcessingException`. Confirma que el repositorio sigue sin contener monederos consultando `GetWallet(wallet_id="wlt-ghost")` y comprobando `None`.

3. Amplía la prueba del listener con un segundo monedero. En

`test_event_listener.py`, añade una prueba que abra dos monederos (`u-A` y `u-B`), deposite importes distintos en cada uno y luego llame a `audit_listener.entries_for(wallet_id_A)` y `audit_listener.entries_for(wallet_id_B)` por separado. Comprueba que cada uno devuelve exactamente dos entradas (`WalletOpened` + `FundsDeposited`) y que los valores de `amount` del payload difieren. Esto demuestra que `entries_for` filtra por ID de monedero, no por tipo de evento.

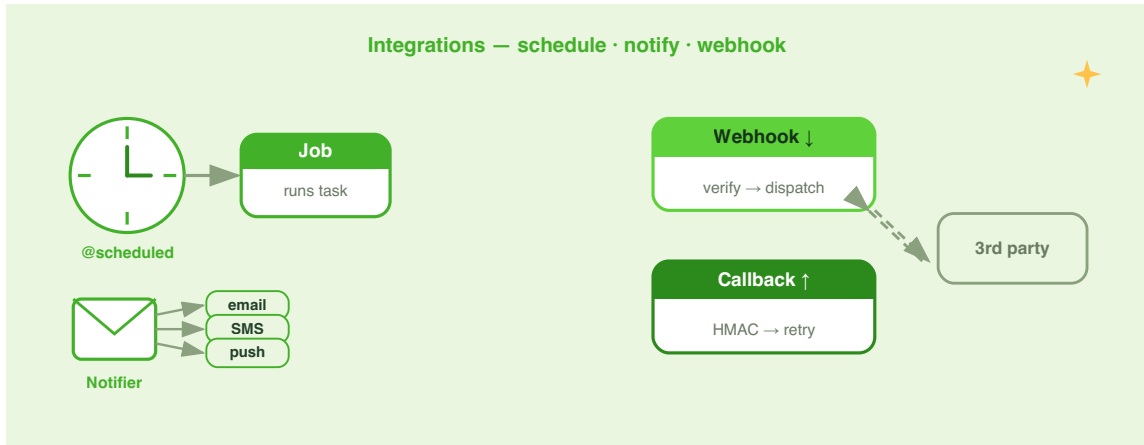
4. Añade una prueba de paginación multipropietario. En

`test_sql_wallet_repository.py`, añade una prueba que inserte diez monederos con dos propietarios distintos, llame a `find_by_owner_id` para cada propietario y luego llame a `find_all` con `Pageable.of(1, 3,`

`Sort.by("balance_minor").descending())`. Comprueba que `page.total = 10`, `page.total_pages = 4` y que el primer elemento de `page.items` tiene el `balance_minor` más alto. Esto demuestra que la paginación es independiente del filtro de consulta derivada.

CAPÍTULO 17

Programación, notificaciones, webhooks y callbacks



En el Capítulo 16 dotaste a Lumen de un arnés de pruebas completo: pruebas unitarias para el dominio, pruebas de flujo CQRS a través del bus real y una prueba del adaptador de SQLite que demuestra una persistencia real. Lumen está ahora bien probado y es resiliente. Pero sigue viviendo enteramente dentro de su propio proceso: reacciona a las peticiones, pero nunca toma la iniciativa por sí mismo.

Las plataformas financieras reales son distintas. Envían extractos de cuenta nocturnos, disparan un SMS en el momento en que los fondos llegan, reciben webhooks de estado de pago de un proveedor de pagos a medianoche y llaman de vuelta a sistemas de socios para confirmar que un desembolso quedó registrado. Eso son cuatro patrones de integración distintos —programación, notificaciones, webhooks entrantes y callbacks salientes— y este capítulo los cubre todos.

Al final del capítulo Lumen:

- ejecutará un **trabajo programado nocturno** que totaliza los saldos diarios de los monederos usando `@scheduled` con una expresión cron;
- enviará un **correo electrónico y una notificación push de "fondos recibidos"** a través de los puertos enchufables `EmailService` y `PushService` de PyFly, disparados por el evento de dominio real `FundsDeposited`;
- aceptará **webhooks entrantes** de un proveedor de pagos ilustrativo, verificando la firma HMAC-SHA256, deduplicando reintentos y despachando a un listener tipado;
- despachará **callbacks salientes** a sistemas de socios, firmando cada carga útil, reintentando ante fallos transitorios y registrando cada intento de entrega.

Instala los dos extras opcionales antes de empezar:

```
uv add "pyfly[scheduling,notifications]"
```

❗ TÉRMINO NUEVO: EXTRAS OPCIONALES

Un *extra* es una porción opt-in de las dependencias de un paquete. `pyfly` mantiene su núcleo ligero y entrega capacidades más pesadas —programación, notificaciones y el resto— detrás de extras con nombre, de modo que solo instalas lo que usas.

`pyfly[scheduling,notifications]` trae ambos. La sintaxis de corchetes es estándar en el empaquetado de Python; `uv add` lo registra en tu `pyproject.toml` para que el siguiente `uv sync` los reinstale. Si más adelante ves un `ModuleNotFoundError` para `croniter` o para un proveedor de notificaciones, te saltaste esta línea: vuelve a ejecutarla.

Este capítulo está dirigido a PyFly **v26.6.110**. Cada listado de código de abajo coincide con el código real de Lumen bajo `samples/lumen/src/lumen`, y cada API del framework se comprobó contra el propio `pyfly`, así que lo que construyas aquí se ejecuta sin cambios.

Tareas programadas

¿Por qué programar en lugar de disparar?

Muchas operaciones de una plataforma financiera no pueden esperar a que llegue una petición HTTP. La conciliación nocturna debe ejecutarse a las 02:00 independientemente de si hay algún usuario activo. Una pasada de calentamiento de caché debería dispararse 30 segundos después del arranque —antes de que llegue el tráfico real— y no cuando la primera petición lenta provoque un fallo de caché. Una métrica de latido debería emitirse cada 10 segundos para que el panel de operaciones muestre una señal en vivo, no una lectura obsoleta.

El módulo de programación de PyFly proporciona una forma declarativa, basada en decoradores, de definir los tres patrones sin gestionar manualmente hilos, bucles de eventos ni ruedas de temporizadores.

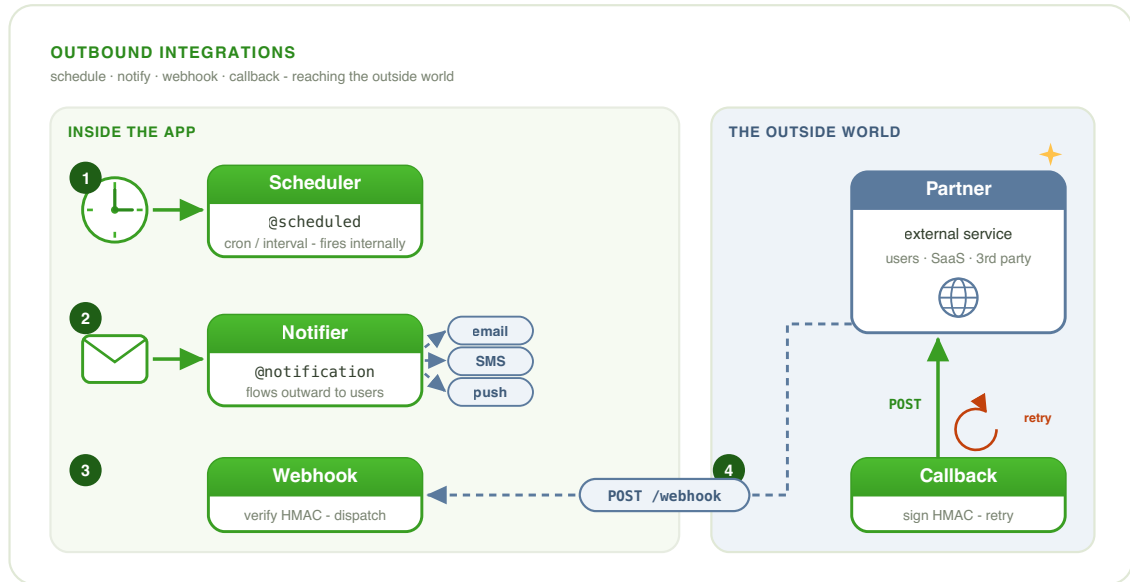


Figura 17.1 — Las cuatro\

capas de integración añadidas en este capítulo. Las tareas programadas se disparan\ internamente; las notificaciones fluyen hacia afuera, hacia los usuarios; los webhooks entrantes\ llegan desde los socios; los callbacks salientes cierran el bucle de realimentación.

El decorador `@scheduled`

`@scheduled` marca cualquier método `async` de un bean `@service` para su ejecución periódica. Acepta exactamente un *disparador* (trigger): `fixed_rate`, `fixed_delay` o `cron`. Proporcionar cero o más de un disparador lanza un `ValueError` en el momento de la decoración, de modo que los errores afloran al arrancar y no silenciosamente a las tres de la madrugada.

❗ TÉRMINO NUEVO: DISPARADOR

Un *disparador* (trigger) es la regla que decide *cuándo* se ejecuta un método programado. Eliges exactamente uno: `cron` (una expresión de calendario como "todos los días a las 02:00"), `fixed_rate` (un intervalo constante como "cada 10 segundos") o `fixed_delay` (un hueco medido después de que cada ejecución termina). Un método, un disparador.

Construyamos el resumen nocturno una decisión a la vez.

Paso 1 — Crea el archivo de servicio. En el árbol de Lumen, añade

`daily_rollup.py` bajo un paquete `ledger`. La clase es un `@service` corriente: una clase Python sencilla que PyFly registra en su contenedor de inyección de dependencias y construye por ti. Como es un bean gestionado, puedes pedir el `WalletRepository` en el constructor y el framework te lo entrega; nunca llamas a `new` tú mismo.

Paso 2 — Elige el disparador. Este trabajo debe ejecutarse una vez por noche, a hora fija, así que el disparador es `cron`. La expresión `"0 2 * * *"` se lee campo a campo como *minuto 0, hora 2, cada día del mes, cada mes, cada día de la semana* —es decir, las 02:00 cada día.

Paso 3 — Escribe el trabajo. Dentro del método, carga cada monedero, suma los enteros `balance_minor` persistidos y (por ahora) registra el total. En producción escribirías la instantánea en una tabla de informes o la enviarías aguas abajo; el registro mantiene el ejemplo centrado en la *programación*, no en la contabilidad.

lumen/ledger/daily_rollup.py

```

from datetime import timedelta

from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.scheduling import scheduled

@service
class DailyRollupService:
    """Tallies all wallet balances once per night at 02:00 UTC."""

    def __init__(self, wallet_repo: WalletRepository) → None:
        self._wallets = wallet_repo

    @scheduled(cron="0 2 * * *")
    async def run(self) → None:
        wallets = await self._wallets.find_all()
        # find_all() returns WalletEntity rows; balance_minor is the
        # persisted integer (cents).
        total_minor_units = sum(w.balance_minor for w in wallets)
        # Persist or ship the nightly snapshot; here we log it.
        print(
            f"[rollup] {len(wallets)} wallets, "
            f"total {total_minor_units / 100:.2f} "
            f"(minor units: {total_minor_units})"
        )

```

Listado 17.1 — Resumen nocturno de saldos de monederos con @scheduled

Cómo funciona. `@scheduled(cron="0 2 * * *")` se dispara todos los días a las 02:00 UTC. El planificador calcula `seconds_until_next()` mediante `CronExpression`, duerme exactamente ese tiempo y luego envía el método al ejecutor.

Eso es todo el cableado que Lumen necesita. Con `pyfly[scheduling]` instalado, `SchedulingAutoConfiguration` automáticamente:

1. registra un bean `TaskScheduler` ;
2. escanea cada bean `@service` en busca de métodos `@scheduled` ;
3. arranca el planificador durante el arranque del `ApplicationContext` ;
4. lo detiene con elegancia al apagarse.

No se requiere ningún `SchedulerManager` .

TÉRMINO NUEVO: AUTOCONFIGURACIÓN

La *autoconfiguración* es PyFly fijándose en lo que hay en tu classpath y cableando por ti la maquinaria correspondiente. `SchedulingAutoConfiguration` solo se activa cuando `croniter` (incluido por el extra `scheduling`) es importable, así que el planificador aparece en el momento en que instalas el extra y permanece ausente en caso contrario. Es la misma idea de "convención sobre configuración" que hizo famosa Spring Boot; siempre puedes reemplazar un bean declarando el tuyo propio.

Ejecútalo. Esperar hasta las 02:00 para ver tu primer tic no tiene gracia, así que cambia temporalmente el disparador para que se ejecute cada minuto — `@scheduled(cron="* * * * *")` — e inicia la aplicación desde el directorio `samples/Lumen` :

```
uv run pyfly run --server uvicorn
```

Al comienzo del siguiente minuto deberías ver tu línea de resumen en los logs (una base de datos vacía simplemente informa de cero monederos):

```
[rollup] 0 wallets, total 0.00 (minor units: 0)
```

Abre un monedero y deposita en él (consulta las recetas de curl en el Capítulo 7), espera al siguiente minuto y los totales se mueven:

```
[rollup] 1 wallets, total 15.00 (minor units: 1500)
```

Detén la aplicación con Ctrl-C y **vuelve a poner el disparador en** `"0 2 * * *"` antes de confirmar el cambio: el cron de cada minuto era solo una sonda.

QUÉ ACABA DE PASAR

No arrancaste un hilo, ni abriste un bucle de eventos, ni registraste un temporizador. Escribiste un `@service` con un método `@scheduled`, y el framework lo descubrió al arrancar, calculó el siguiente momento de disparo a partir de la expresión cron, durmió hasta entonces y ejecutó tu corrutina, repitiendo para siempre. El planificador es *declarativo*: tú indicas *cuándo*, PyFly se encarga del *cómo*.

fixed_rate frente a fixed_delay

`fixed_rate` mide desde el **inicio** de una ejecución hasta el inicio de la siguiente. `fixed_delay` mide desde el **final** de una ejecución hasta el inicio de la siguiente. Usa `fixed_rate` para latidos y métricas donde necesitas una cadencia constante con independencia del tiempo de ejecución. Usa `fixed_delay` cuando necesitas un hueco de respiro garantizado; por ejemplo, al sondear una API aguas arriba que limita la tasa por frecuencia de peticiones. Desde la v26.09.08 un trabajo `fixed_rate` o `cron` nunca se solapa consigo mismo: cuando una ejecución dura más que su periodo, la siguiente la espera (`@scheduled(concurrent=N)` permite que se solapen hasta *N* ejecuciones), y `cron` calcula su siguiente disparo cuando ha terminado una ejecución.

lumen/health/monitor.py

```

from datetime import timedelta

from pyfly.container import service
from pyfly.scheduling import scheduled

@service
class HealthMonitor:

    @scheduled(
        fixed_rate=timedelta(seconds=10),
        initial_delay=timedelta(seconds=5),
    )
    async def heartbeat(self) → None:
        """Emit a liveness metric every 10 s, starting 5 s after startup."""
        # metrics.gauge("lumen.up", 1)
        pass

@service
class ExchangeRatePoller:

    def __init__(self, fx_repo) → None:
        self._repo = fx_repo

    @scheduled(fixed_delay=timedelta(minutes=5))
    async def poll(self) → None:
        """Fetch the latest exchange rates, then wait 5 min before
        repeating."""
        rates = await self._repo.fetch_latest()
        await self._repo.store(rates)

```

Listado 17.2 — Latido *fixed_rate* y sondeo *fixed_delay*

`initial_delay` pospone la primera ejecución; está disponible tanto para `fixed_rate` como para `fixed_delay` (se ignora en los disparadores `cron`, que siempre esperan al siguiente instante de calendario coincidente).

CronExpression

`CronExpression` también es utilizable directamente: resulta conveniente cuando necesitas mostrar próximos horarios de programación en una interfaz o validar una expresión proporcionada por el usuario antes de almacenarla.

`lumen/ledger/schedule_preview.py`

```
from pyfly.scheduling import CronExpression

def preview_rollup_schedule(expression: str, n: int = 5) → list[str]:
    """Return the next N fire times for a given cron expression."""
    cron = CronExpression(expression)
    return [str(t) for t in cron.next_n_fire_times(n)]
```

Listado 17.3 — Uso de `CronExpression` de forma independiente

Ejecútalo. `CronExpression` no necesita una aplicación en marcha, así que la forma más rápida de construir intuición es el REPL. Desde `samples/lumen`:

```
uv run python -c "from pyfly.scheduling import CronExpression; \
print(*CronExpression('0 2 * * *').next_n_fire_times(3), sep='\n')"
```

Deberías ver los tres próximos instantes de las 02:00, uno por línea (tus fechas serán distintas):

```
2026-06-16 02:00:00+00:00
2026-06-17 02:00:00+00:00
2026-06-18 02:00:00+00:00
```


Fíjate en el `+00:00` : los momentos de disparo son UTC salvo que pases una `zone` (que se cubre a continuación). Esta es también la forma más limpia de comprobar la cordura de una expresión proporcionada por el usuario antes de almacenarla: una cadena no válida lanza `ValueError` de inmediato.

`CronExpression` acepta tanto el formato estándar de 5 campos (`min hour dom month dow`) como el formato de 6 campos al estilo Spring con los segundos primero (`sec min hour dom month dow`). El comodín `?` de Spring se normaliza a `*` de forma transparente.

Expresión	Se dispara
<code>* * * * *</code>	Cada minuto
<code>0 * * * *</code>	Cada hora, en punto
<code>0 2 * * *</code>	Cada día a las 02:00
<code>0 9 * * 1-5</code>	Días laborables a las 09:00
<code>30 2 1 * *</code>	El día 1 de cada mes a las 02:30
<code>*/15 * * * *</code>	Cada 15 minutos
<code>0 0 12 * * *</code>	Mediodía cada día (6 campos, segundos primero)

Cron consciente de la zona horaria

Las expresiones cron se evalúan en **UTC** por defecto. Pasa `zone` con un nombre de zona horaria IANA para evaluar los momentos de disparo en una zona específica en su lugar:

```
@scheduled(cron="0 2 * * *", zone="America/New_York")
async def close_books(self) → None:
    """02:00 New York time – follows DST automatically."""
    ...
```

El mismo parámetro `zone` está disponible en `CronExpression`:

```
cron = CronExpression("0 9 * * *", zone="Europe/Madrid")
next_run = cron.next_fire_time() # zone-aware datetime
```

Las transiciones de horario de verano (DST) las gestiona la base de datos `zoneinfo`; no se requiere ningún ajuste manual de desfase.

Bloqueo distribuido

Cuando varias instancias de Lumen se ejecutan detrás de un balanceador de carga, cada instancia programa los mismos métodos `@scheduled`. Sin coordinación, el resumen nocturno se dispara una vez por instancia y escribe registros duplicados. El parámetro `lock` resuelve esto de la misma manera que `@SchedulerLock` (ShedLock) de Spring: antes de cada tic el planificador intenta adquirir un bloqueo con nombre y **omite la ejecución** si el bloqueo ya está retenido en otro lugar.

```
@scheduled(cron="0 2 * * *", lock=True, lock_ttl=timedelta(minutes=5))
async def run(self) → None:
    """lock=True auto-names the lock 'DailyRollupService.run'."""
    ...
```

- `lock=True` — deriva el nombre del bloqueo de `"ClassName.method_name"`.
- `lock="shared-name"` — nombre explícito; útil cuando dos métodos deben ser mutuamente excluyentes.
- `lock_ttl` — TTL de válvula de seguridad; ponlo cómodamente más largo que el peor tiempo de ejecución del trabajo.

Por defecto `TaskScheduler` usa `LocalLock`, que siempre adquiere: el comportamiento de instancia única no cambia. Para la coordinación entre procesos, implementa `DistributedLock` y regístralo como un bean:

`lumen/infra/redis_lock.py`

```
from pyfly.container import bean, configuration
from pyfly.scheduling import DistributedLock

class RedisLock:
    """Best-effort named lock backed by Redis SET NX PX."""

    def __init__(self, redis) → None:
        self._redis = redis

    async def try_acquire(self, name: str, ttl: float) → bool:
        ok = await self._redis.set(
            f"pyfly:lock:{name}", "1",
            nx=True, px=int(ttl * 1000),
        )
        return ok is True

    async def release(self, name: str) → None:
        await self._redis.delete(f"pyfly:lock:{name}")

@configuration
class LockConfig:

    @bean
    def distributed_lock(self) → DistributedLock:
        import redis.asyncio as aioredis
        client = aioredis.from_url("redis://localhost:6379/1")
        return RedisLock(client)
```

Listado 17.4 — `DistributedLock` respaldado por Redis

`SchedulingAutoConfiguration` detecta el bean `DistributedLock` en el contenedor automáticamente y se lo pasa al `TaskScheduler`. Cualquier objeto con corrutinas `try_acquire` y `release` conformes satisface el protocolo.

@async_method

`@async_method` marca un método para ejecución en segundo plano a través del `TaskExecutorPort`, al estilo de `@Async` de Spring. Esperar la llamada la envía y retorna de inmediato con la `asyncio.Task` que la ejecuta; espera esa tarea cuando necesites el resultado:

```
from pyfly.scheduling import async_method

@service
class AlertService:

    @async_method
    async def send_alert(self, msg: str) → None:
        """Runs in a task of its own, outside the caller's transaction."""
        ...

    async def notify(alerts: AlertService) → None:
        # The call returns the asyncio.Task at once; await it for the result.
        task = await alerts.send_alert("funds received")
        await task
```

Bajo el capó `@async_method` establece `__pyfly_async__ = True` en la función; el contexto de la aplicación sustituye el método del bean por un despachador que envía la llamada al `TaskExecutorPort`. La tarea empieza con el estado transaccional limpio: un método `@transactional` abre su propia unidad de trabajo, así que su fallo nunca

revierte a quien llama y el commit de quien llama no lo espera. Una excepción no capturada va al bean `AsyncUncaughtExceptionHandler`, que por defecto la registra en el log. (Hasta la v26.09.07 esperar la llamada devolvía el resultado del método.)

🍃 EQUIVALENCIA CON SPRING

`@scheduled(fixed_rate=...)` refleja el `@Scheduled(fixedRate=...)` de Spring.
`@scheduled(fixed_delay=...)` refleja `@Scheduled(fixedDelay=...)`.
`@scheduled(cron=...)` refleja `@Scheduled(cron=...)`. `zone=` refleja el atributo `zone` de Spring. `lock=True` refleja el `@SchedulerLock` de ShedLock. `@async_method` refleja el `@Async` de Spring.

Referencia de configuración

```
pyfly:
  scheduling:
    enabled: true           # set false to disable all loops
    executor:
      type: asyncio         # 'asyncio' (default, in-loop) or 'thread'
      max-workers: 4        # worker threads when type is 'thread'
    lock:
      provider: none        # none | memory | redis | database | postgres
```

Cuando `enabled` es `false`, `TaskScheduler` no arranca ningún bucle y todos los métodos `@scheduled` se omiten en silencio.

El `executor.type` elige cómo se ejecuta cada tic. El valor por defecto `asyncio` ejecuta la corrutina en el bucle de eventos de la aplicación —ideal para trabajos cortos, ligados a E/S, como el resumen—. Cambia a `thread` (un grupo de `executor.max-workers` hilos) cuando un trabajo realice trabajo intensivo de CPU o llame a una biblioteca bloqueante, de modo que no pueda atascar el bucle.

💡 CÓMO ELEGIR UN PROVEEDOR DE BLOQUEO

`lock.provider` selecciona el backend detrás de `@scheduled(lock=...)`, descrito a continuación: `none` (el valor por defecto: sin coordinación), `memory` (exclusión mutua dentro de un proceso), `redis`, `database` o `postgres` (verdadera coordinación entre instancias sin cambios en el código). `database` —y `postgres`, que es el mismo bloqueo — es una tabla de concesiones portable (`LeaseLock`, la tabla `pyfly_locks`) en la base de datos de la aplicación, sea cual sea su backend; los bloqueos consultivos de PostgreSQL son opcionales (`pyfly.scheduling.lock.postgres.advisory: true`). En `redis` PyFly construye el bloqueo a partir de `pyfly.scheduling.lock.redis.url`; el `@bean` artesanal del Listado 17.4 es la alternativa de hazlo-tú-mismo cuando necesitas semánticas personalizadas.

Notificaciones

Lumen necesita decirles a los clientes que su dinero ha llegado: un correo electrónico para la confirmación del saldo y, opcionalmente, un SMS o un push móvil para la alerta en tiempo real.

El módulo de notificaciones de PyFly define tres **protocolos de puerto** y tres **servicios por defecto**. Tu lógica de negocio depende de los protocolos; los adaptadores concretos de proveedor —SMTP, SendGrid, Twilio, Firebase— viven detrás de la frontera del puerto y pueden intercambiarse sin tocar una sola línea de código de dominio.

❗ TÉRMINO NUEVO: PUERTO Y ADAPTADOR

Un *puerto* es una interfaz con la que tu código conversa —aquí, "algo capaz de enviar un correo electrónico"—. Un *adaptador* es una implementación concreta de ese puerto:

`SmtpEmailProvider`, `SendGridEmailProvider`, etcétera. El patrón (también llamado *arquitectura hexagonal*) significa que tu lógica de depósito depende solo del puerto `EmailService`, nunca de un proveedor específico. Cambiar SMTP por SendGrid es un cambio de una línea en una clase de configuración; el código de dominio nunca se entera.

La jerarquía de puertos

Protocolo	Clase de servicio	Método
<code>EmailProvider</code>	<code>DefaultEmailService</code>	<code>send(EmailMessage) → NotificationResult</code>
<code>SmsProvider</code>	<code>DefaultSmsService</code>	<code>send(SmsMessage) → NotificationResult</code>
<code>PushProvider</code>	<code>DefaultPushService</code>	<code>send(PushMessage) → NotificationResult</code>

`DefaultEmailService`, `DefaultSmsService` y `DefaultPushService` son envoltorios finos: cada uno delega en un proveedor, captura cualquier excepción del proveedor y devuelve un `NotificationResult` estructurado con `status=FAILED` y la cadena del error en lugar de propagar la excepción. Una caída transitoria de SendGrid no tumba el manejador (handler) de depósitos.

Mensajes y resultados

`FundsDeposited` lleva `amount` y `balance` como **unidades menores enteras** (céntimos). La propiedad `Money.major_units` las convierte para su visualización —`Money(25000, Currency.EUR).major_units` es `250.0`—. Tenlo en cuenta al formatear los cuerpos de las notificaciones.

lumen/notifications/models_overview.py

```

from pyfly.notifications import (
    EmailMessage,
    NotificationResult,
    PushMessage,
    SmsMessage,
)

# Email – full field set
email = EmailMessage(
    to=["alice@example.com"],
    sender="no-reply@lumenbank.com",
    subject="Funds received",
    body_text="EUR 250.00 has been credited to your wallet.",
    body_html=(
        "<p><strong>EUR 250.00</strong> has been credited "
        "to your wallet.</p>"
    ),
)

# SMS – compact
sms = SmsMessage(
    to="+34600000001",
    body="Lumen: EUR 250.00 received. New balance: EUR 750.00.",
    sender="LUMEN",
)

# Push – structured payload
push = PushMessage(
    device_tokens=["FCM_TOKEN_GOES_HERE"],
    title="Funds received",
    body="EUR 250.00 credited",
    data={"wallet_id": "w-001", "amount_minor": 25000},
)

```

Listado 17.5 — Los DTO principales

`NotificationResult` lleva `id`, `provider`, `status` (`EmailStatus.SENT` | `DELIVERED` | `FAILED` | ...), un `provider_id` opcional (p. ej. el ID de mensaje de SendGrid) y un `error` opcional.

Cableando el proveedor SMTP

Para desarrollo y despliegues autoalojados, `SmtpEmailProvider` ejecuta `smtplib` desde un grupo de hilos para que el bucle de eventos asíncrono nunca se bloquee.

Paso 1 — Construye el proveedor como un `@bean`. Una clase `@configuration` es el lugar de PyFly para ensamblar objetos que el contenedor no puede construir por sí solo —aquí, un cliente SMTP que necesita un host, credenciales y ajustes de TLS—. Cada método `@bean` devuelve un objeto listo para usar; el framework lo cachea y lo inyecta allá donde se solicite el tipo de retorno.

Paso 2 — Envuélvelo en un `DefaultEmailService`. El segundo `@bean` toma el proveedor y lo devuelve como un `EmailService` —el *puerto* del que depende tu código de dominio—. El tipo de retorno declarado importa: al devolver `EmailService`, cada clase que pide un `EmailService` recibe este envoltorio, y ninguna de ellas se entera de que SMTP está detrás.

lumen/notifications/config.py

```

from pyfly.container import bean, configuration
from pyfly.notifications import DefaultEmailService, EmailService
from pyfly.notifications.providers.smtp import SmtplibEmailProvider

@configuration
class NotificationConfig:

    @bean
    def email_provider(self) → SmtplibEmailProvider:
        return SmtplibEmailProvider(
            "smtp.lumenbank.internal",
            port=587,
            username="notifications",
            password="s3cr3t",
            use_tls=True,
        )

    @bean
    def email_service(
        self, provider: SmtplibEmailProvider,
    ) → EmailService:
        return DefaultEmailService(provider=provider)

```

Listado 17.6 — Proveedor SMTP cableado como un @bean

`SmtplibEmailProvider` acepta `host`, `port` (por defecto `587`), `username`, `password` y `use_tls` (por defecto `True`). Cámbialo por `SendGridEmailProvider` o `ResendEmailProvider` modificando un único método `@bean: DefaultEmailService` es indiferente al proveedor que tenga detrás.

💡 PROVEEDORES DISPONIBLES

`pyfly.notifications` incluye ocho adaptadores integrados: `DummyEmailProvider` / `DummySmsProvider` / `DummyPushProvider` (solo registran, para desarrollo/pruebas), `SmtplibEmailProvider`, `SendGridEmailProvider`, `ResendEmailProvider` (correo electrónico), `TwilioSmsProvider` (SMS) y `FirebasePushProvider` (push). Todos satisfacen su respectivo protocolo `*Provider`.

Enviando una notificación de "fondos recibidos"

Lumen publica un evento de dominio `FundsDeposited` cada vez que el comando `deposit()` tiene éxito (consulta el Capítulo 8). El lugar adecuado para disparar la notificación es un listener de EDA suscrito a ese evento, no el propio manejador del comando, lo que mantiene la ruta de depósito libre de preocupaciones de notificación.

`FundsDeposited` lleva `wallet_id: str`, `amount: int` (unidades menores), `currency: str` y `balance: int` (nuevo saldo, unidades menores). El listener convierte `amount` en una cadena de visualización mediante `amount / 100`.

Paso 1 — Suscríbete al evento. Apila

`@event_listener(event_types=["FundsDeposited"])` sobre un método `async` de un `@service`. Al arrancar, PyFly descubre el método marcado y lo autosuscribe al bus `EventPublisher` —exactamente el mismo mecanismo que `WalletAuditListener` usó allá en el Capítulo 8—. Nunca cableas un bus a mano.

Paso 2 — Lee la carga útil. El manejador recibe un `EventEnvelope`. Su `payload` es un dict sencillo de los campos del evento, así que extraes `wallet_id`, `amount`, `currency` y `balance` con `.get(...)` y coaccionas los importes a enteros. Como los importes están en unidades menores, dividir por 100 da el valor de visualización: `25000` se convierte en `250.00`.

Paso 3 — Envía a través de los puertos. Inyecta `EmailService` y `PushService` en el constructor y llama a `.send(...)` en cada uno. Ambos devuelven un `NotificationResult` en lugar de lanzar una excepción: un proveedor inestable se degrada con elegancia en lugar de hacer caer el listener.

lumen/wallet/deposit_notification_listener.py

```

from pyfly.container import service
from pyfly.eda import EventEnvelope, event_listener
from pyfly.notifications import (
    EmailMessage,
    EmailService,
    PushMessage,
    PushService,
)

@service
class DepositNotificationListener:
    """Sends email + push when a FundsDeposited event is observed."""

    def __init__(
        self,
        email_service: EmailService,
        push_service: PushService,
    ) → None:
        self._email = email_service
        self._push = push_service

    @event_listener(event_types=["FundsDeposited"])
    async def on_funds_deposited(
        self, envelope: EventEnvelope,
    ) → None:
        payload = envelope.payload
        wallet_id = str(payload.get("wallet_id", ""))
        amount_minor = int(payload.get("amount", 0))
        currency = str(payload.get("currency", "EUR"))
        balance_minor = int(payload.get("balance", 0))
        amount_str = f"{amount_minor / 100:.2f} {currency}"
        balance_str = f"{balance_minor / 100:.2f} {currency}"

        # Fetch contact details from a wallet profile service in prod;
        # hardcoded here for illustration.
        email = "customer@example.com"

```

```

device_token = "FCM_TOKEN_GOES_HERE"

await self._email.send(EmailMessage(
    to=[email],
    sender="no-reply@lumenbank.com",
    subject=f"Funds received: {amount_str}",
    body_text=(
        f"{amount_str} has been credited to wallet "
        f"{wallet_id}. New balance: {balance_str}."
    ),
))

await self._push.send(PushMessage(
    device_tokens=[device_token],
    title="Funds received",
    body=f"{amount_str} credited",
    data={
        "wallet_id": wallet_id,
        "amount_minor": amount_minor,
        "currency": currency,
    },
))

```

Listado 17.7 — Notificando ante FundsDeposited

Ambas llamadas devuelven un `NotificationResult`; inspecciona el campo `status` para registrar fallos o programar reintentos.

Ejecútalo. No quieres un servidor SMTP real mientras desarrollas, así que cambia el proveedor por el `DummyEmailProvider`, que solo registra. En tu clase `@configuration`, devuelve un `DummyEmailProvider` (y un `DummyPushProvider`) en lugar del de SMTP, luego inicia la aplicación y dispara un depósito:

```

uv run pyfly run --server uvicorn
# in a second terminal, open a wallet and deposit (see Chapter 7), e.g.:
curl -s -X POST localhost:8080/api/v1/wallets/<wallet-id>/deposit \
  -H 'content-type: application/json' -d '{"amount":25000}'

```

El depósito publica `FundsDeposited`, el listener se dispara y los proveedores de prueba registran los mensajes que "enviaron":

```
[dummy email] to=['customer@example.com'] subject=Funds received: 250.00 EUR  
[dummy push] tokens=1 title=Funds received
```

El `DummyEmailProvider` también conserva cada mensaje que recibió en una lista `.sent`, que es exactamente contra lo que se afirma la prueba del Ejercicio 2, sin necesidad de servidor SMTP.

QUÉ ACABA DE PASAR

El comando de depósito no sabía nada del correo electrónico. Simplemente hizo su trabajo y lanzó un evento de dominio `FundsDeposited`. La lógica de notificación vive en un listener aparte que *reacciona* a ese evento, así que la ruta de depósito se mantiene limpia y puedes añadir, eliminar o cambiar notificaciones sin tocar el manejador del comando. Esa separación —publicar un hecho, dejar que las partes interesadas reaccionen— es la razón de ser de la arquitectura orientada a eventos.

EQUIVALENCIA CON SPRING

`EmailService` / `SmsService` / `PushService` son los equivalentes en Python del `JavaMailSender` (correo electrónico) de Spring y de las integraciones de terceros de Spring para SMS y push. La división hexagonal puerto/adaptador es idéntica: tu código de dominio depende del protocolo; los adaptadores concretos de proveedor viven en la capa de infraestructura.

Webhooks entrantes

Un proveedor de pagos ilustrativo envía por POST un evento

`payment_intent.succeeded` a Lumen cada vez que un cliente recarga su monedero con tarjeta. Lumen debe:

1. **verificar la firma HMAC-SHA256** para rechazar cargas útiles falsificadas;
2. **deduplicar** los reintentos usando la clave de idempotencia para que un reintento no acredite un monedero dos veces;
3. **despachar** el evento verificado a un listener tipado.

El módulo `pyfly.webhooks` de PyFly se encarga de los tres pasos.

TÉRMINOS NUEVOS: WEBHOOK, HMAC, IDEMPOTENCIA

Un *webhook* es un POST HTTP que un sistema externo te envía *a ti* cuando ocurre algo: el espejo entrante de los callbacks salientes que veremos más adelante en este capítulo. Como cualquiera puede hacer POST a una URL pública, el proveedor firma cada petición con un secreto compartido usando *HMAC* (un hash con clave); recalculamos el hash sobre los bytes exactos recibidos y compararlo demuestra que la carga útil es genuina y no fue manipulada. *Idempotencia* significa "seguro de recibir más de una vez": los proveedores reintentan ante fallos de red, así que almacenamos una clave de idempotencia e ignoramos una repetición —de lo contrario una sola recarga con tarjeta podría acreditar un monedero dos veces—.

WebhookEvent y AbstractWebhookEventListener

Cada evento entrante se modela como una dataclass `WebhookEvent`:


```
@dataclass
class WebhookEvent:
    id: str          # auto-generated UUID
    source: str       # e.g. "payment-provider"
    event_type: str   # from body["type"]
    headers: dict[str, str]
    body: dict[str, Any]
    raw_body: bytes
    received_at: datetime
    idempotency_key: str | None
```

Subclasifica `AbstractWebhookEventListener` y establece `source` con el nombre que pasarás a `WebhookProcessor.process()`:

lumen/webhooks/payment_listener.py

```

from pyfly.container import service
from pyfly.webhooks import AbstractWebhookEventListener, WebhookEvent

@service
class PaymentWebhookListener(AbstractWebhookEventListener):
    source = "payment-provider"

    def __init__(self, deposit_handler) → None:
        self._handler = deposit_handler

    async def handle(self, event: WebhookEvent) → None:
        if event.event_type == "payment_intent.succeeded":
            pi = event.body.get("data", {}).get("object", {})
            wallet_id = pi.get("metadata", {}).get("wallet_id")
            amount_minor = int(pi.get("amount_received", 0))
            currency_code = pi.get("currency", "EUR").upper()
            if wallet_id and amount_minor > 0:
                # Delegate to the CQRS command handler so the aggregate
                # enforces the balance invariant and raises FundsDeposited.
                from lumen.core.services.wallets.deposit_funds_command import
(
                    DepositFunds,
                )
                await self._handler.handle(
                    DepositFunds(
                        wallet_id=wallet_id,
                        amount=amount_minor,
                    )
                )

    async def on_error(
        self, event: WebhookEvent, error: BaseException,
    ) → None:
        # Override to DLQ or page on-call.
        pass

```

Listado 17.8 — Listener de webhook del proveedor de pagos

`on_error` se invoca cuando `handle` lanza una excepción; por defecto es un no-op. Reescríbelo para publicar en una cola de mensajes muertos o emitir una métrica.

WebhookProcessor: verificar, deduplicar, despachar

`WebhookProcessor` ensambla un validador de firmas, un almacén de idempotencia y una lista de listeners. Móntalo en una clase `@configuration` para que sea un único bean compartido:

- `listeners` es la lista de subclases de `AbstractWebhookEventListener` a las que abrir en abanico los eventos (de momento solo `PaymentWebhookListener`);
- `signature_validators` asocia cada nombre de `source` con el validador que demuestra que sus peticiones son genuinas —aquí un `HmacSignatureValidator` con clave del secreto del webhook que te dio tu proveedor—;
- un `event_store` (omitido aquí, así que se usa el por defecto en memoria) recuerda las claves de idempotencia.

lumen/webhooks/processor_config.py

```

from pyfly.container import bean, configuration
from pyfly.webhooks import (
    HmacSignatureValidator,
    WebhookProcessor,
)
from lumen.webhooks.payment_listener import PaymentWebhookListener

@configuration
class WebhookConfig:

    @bean
    def webhook_processor(
        self, payment_listener: PaymentWebhookListener,
    ) → WebhookProcessor:
        return WebhookProcessor(
            listeners=[payment_listener],
            signature_validators={
                "payment-provider": HmacSignatureValidator(
                    secret="whsec_REPLACE_ME",
                ),
            },
        )

```

Listado 17.9 — Ensamblando WebhookProcessor

`HmacSignatureValidator` espera el formato de cabecera `sha256=<hex>` y usa `hmac.compare_digest` para una comparación en tiempo constante. Cambia el parámetro `header_prefix` si tu proveedor usa un esquema diferente.

Manejando un webhook en un manejador HTTP

Llama a `processor.process()` desde tu manejador HTTP entrante. Pasa el cuerpo de la petición sin procesar (bytes sin modificar): el validador calcula el HMAC sobre los bytes exactos recibidos.

⚠ LEE EL CUERPO COMO BYTES EN CRUDO, NO COMO JSON PARSEADO

La firma se calcula sobre los *bytes exactos* que envió el proveedor. Si parseas el JSON y lo vuelves a serializar, el orden de las claves o los espacios en blanco pueden cambiar y el HMAC ya no coincidirá —toda petición legítima sería rechazada—. Pasa siempre `await request.body()` (los bytes intactos) a `process()`, como hace el manejador de abajo.

lumen/webhooks/payment_handler.py

```

from pyfly.container import rest_controller
from pyfly.web import post_mapping, request_mapping
from pyfly.webhooks import WebhookProcessor
from starlette.requests import Request
from starlette.responses import Response

@rest_controller
@request_mapping("/webhooks")
class PaymentWebhookHandler:

    def __init__(self, processor: WebhookProcessor) → None:
        self._processor = processor

    @post_mapping("/payment")
    async def receive(self, request: Request) → Response:
        # Read the untouched bytes – the HMAC is computed over exactly
        # what the provider sent (see the warning above).
        raw_body = await request.body()
        headers = {
            "X-Signature": request.headers.get(
                "X-Webhook-Signature", ""
            ),
            "X-Idempotency-Key": request.headers.get(
                "X-Idempotency-Key", ""
            ),
        }
        try:
            await self._processor.process(
                source="payment-provider",
                raw_body=raw_body,
                headers=headers,
            )
        except ValueError:
            return Response(content=b"invalid signature", status_code=400)
        return Response(content=b"ok", status_code=200)

```

Listado 17.10 — Endpoint de webhook entrante del proveedor de pagos

La firma de `process()` acepta los argumentos por palabra clave `signature_header` e `idempotency_header` para reemplazar los nombres de cabecera por defecto (`X-Signature` y `X-Idempotency-Key`).

Qué ocurre dentro de `process()` :

1. El validador se busca por `source` ; si no hay ninguno registrado, se usa un `NoOpSignatureValidator` —que siempre pasa, seguro para desarrollo pero no para producción—.
2. Si la validación de la firma falla, se lanza `ValueError` de inmediato y no se invoca a ningún listener.
3. El cuerpo en crudo se decodifica como JSON; si falla, los bytes en crudo se almacenan bajo `body["_raw"]` .
4. Si `idempotency_key` está presente y ya se ha visto, el evento se devuelve pero **no** se invoca a los listeners.
5. Cada listener para el source se invoca en el orden de registro; si uno lanza una excepción, el error se registra y se invoca `on_error` antes de continuar con el siguiente listener.

Ejecútalo. Prueba el endpoint como lo haría un proveedor real: firmando el cuerpo exacto. Elige el mismo secreto que pusiste en `HmacSignatureValidator` (`whsec_REPLACE_ME` en el Listado 17.9), calcula el HMAC con `openssl` y haz POST. Inicia la aplicación (`uv run pyfly run --server uvicorn`), luego en un segundo terminal:

terminal

```
BODY='{ "type": "payment_intent.succeeded", "data": { "object":
{ "amount_received": 25000, "currency": "eur", "metadata": { "wallet_id": "<wallet-
id>" } } } }'
SIG=$(printf '%s' "$BODY" | openssl dgst -sha256 -hmac "whsec_REPLACE_ME" |
sed 's/^.* //')

curl -s -X POST localhost:8080/webhooks/payment \
-H "X-Webhook-Signature: sha256=$SIG" \
-H "X-Idempotency-Key: evt-001" \
-H 'content-type: application/json' \
-d "$BODY"
```

Listado 17.10a — Firma y POST de un webhook

Una petición correctamente firmada devuelve **ok** y acredita el monedero (verás también dispararse los logs de notificación de **FundsDeposited** de antes):

```
ok
```

Ahora demuestra las dos garantías. Haz POST del **mismo** comando otra vez con la misma **X-Idempotency-Key**: sigue devolviendo **ok**, pero el monedero *no* se acredita una segunda vez (el duplicado se descarta antes de que se ejecute ningún listener). Luego altera un byte de **\$BODY** sin recalcular **\$SIG** y vuelve a hacer POST: la firma ya no coincide, así que el manejador devuelve:

```
invalid signature
```

Esa es la tubería verificar-deduplicar-despachar funcionando de extremo a extremo, y refleja el Ejercicio 3 casi exactamente.

❗ QUÉ ACABA DE PASAR

Un desconocido en internet hizo POST de JSON a tu servicio, y tres guardianes se ejecutaron antes que una sola línea de tu lógica de negocio: la comprobación de la firma rechazó falsificaciones, el almacén de idempotencia rechazó reintentos, y solo entonces el listener tipado tradujo el evento en un comando CQRS `DepositFunds` —así que la raíz de agregado del monedero siguió haciendo cumplir sus propios invariantes—. Tú escribiste el cuerpo de `handle()`; PyFly proporcionó el desafío que lo rodea.

❗ ALMACÉN DE IDEMPOTENCIA EN MEMORIA

El `InMemoryWebhookEventStore` por defecto guarda las claves vistas en un `set` de Python. Para clústeres de producción, implementa el protocolo `WebhookEventStore` respaldado por Redis o una base de datos:

```
class RedisEventStore:
    async def already_processed(
        self, key: str,
    ) → bool: ...
    async def remember(self, key: str) → None: ...
```

Pásalo como `event_store=RedisEventStore(...)` a `WebhookProcessor`.

Callbacks salientes

Cuando Lumen registra un desembolso a un banco socio, ese socio espera un POST `DisbursementSettled` a su URL de webhook —firmado, reintentado ante fallos y auditable—. El módulo `pyfly.callbacks` de PyFly se encarga del lado saliente.

❗ TÉRMINO NUEVO: CALLBACK SALIENTE

Un *callback* aquí es lo inverso del webhook entrante que acabas de construir: ahora *Lumen* es el remitente, que hace POST de un evento a la URL de un socio. El módulo te da la misma maquinaria de confianza y fiabilidad en la salida: firma cada carga útil (para que el socio pueda verificarla), reintentos los fallos transitorios con retroceso (backoff) y registra cada intento para que tengas un rastro de auditoría cuando un socio pregunte "¿nos llegasteis a avisar de la transacción X?".

Suscripciones y configuración

Cada socio se modela como una `CallbackConfig` —un registro con alcance de inquilino (tenant) que contiene el secreto del webhook, las suscripciones a eventos y la política de reintentos—.

❗ TÉRMINO NUEVO: INQUILINO (TENANT)

Un *inquilino* (tenant) es un cliente u organización aislada dentro de una aplicación compartida —aquí, `"lumen"`—. Los callbacks tienen alcance de inquilino para que un despliegue multiinquilino pueda mantener las URLs de socios, secretos y política de reintentos de cada inquilino por separado y nunca cruzar los cables. Con un solo inquilino simplemente pasas el mismo `tenant_id` en todas partes.

Paso 1 — Describe cada suscripción. Una `CallbackSubscription` empareja un `event_type` con la `target_url` a la que hacerle POST. Usa el nombre exacto del evento (`"DisbursementSettled"`) para encaminar un evento, o `"*"` como comodín que coincide con todos los eventos del inquilino —útil para un endpoint de auditoría que quiere el flujo completo—.

Paso 2 — Envuélvelas en una `CallbackConfig` y guárdala. La configuración lleva el `secret` compartido (usado para firmar cada carga útil), la política de reintentos (`max_attempts`, `backoff_ms`) y la lista de suscripciones. Persístela a través de un `CallbackConfigRepository` — `InMemoryCallbackConfigRepository` por ahora, uno respaldado por base de datos en producción—.

lumen/callbacks/register_partner.py

```

from pyfly.callbacks import (
    CallbackConfig,
    CallbackSubscription,
    InMemoryCallbackConfigRepository,
    InMemoryCallbackExecutionRepository,
)

async def register_clearance_bank(configs) → None:
    await configs.save(CallbackConfig(
        tenant_id="lumen",
        name="clearance-bank",
        secret="cb-secret-xyz",
        max_attempts=5,
        backoff_ms=2_000,
        subscriptions=[
            CallbackSubscription(
                event_type="DisbursementSettled",
                target_url=(
                    "https://api.clearancebank.example.com"
                    "/hooks/lumen"
                ),
            ),
            CallbackSubscription(
                event_type="*",
                target_url=(
                    "https://audit.clearancebank.example.com"
                    "/all-events"
                ),
            ),
        ],
    ))

```

Listado 17.11 — Registrando un callback de socio

`event_type="*"` es un comodín: todos los eventos despachados para el inquilino coinciden. Los tipos con nombre solo coinciden con la cadena exacta del tipo de evento.

Despachando un evento

`CallbackDispatcher.dispatch()` abre el evento en abanico hacia cada suscripción coincidente:

lumen/callbacks/dispatcher_config.py

```

from pyfly.callbacks import (
    CallbackDispatcher,
    InMemoryCallbackConfigRepository,
    InMemoryCallbackExecutionRepository,
)
from pyfly.container import bean, configuration

@configuration
class CallbackConfig_:

    @bean
    def callback_configs(self) → InMemoryCallbackConfigRepository:
        return InMemoryCallbackConfigRepository()

    @bean
    def callback_executions(
        self,
    ) → InMemoryCallbackExecutionRepository:
        return InMemoryCallbackExecutionRepository()

    @bean
    def callback_dispatcher(
        self,
        configs: InMemoryCallbackConfigRepository,
        executions: InMemoryCallbackExecutionRepository,
    ) → CallbackDispatcher:
        return CallbackDispatcher(
            configs=configs,
            executions=executions,
        )

```

Listado 17.12 — Cableando e invocando CallbackDispatcher

Luego, en el servicio de dominio —fíjate en que la carga útil usa `amount` en unidades menores (céntimos), así que `50_000` son EUR 500.00—:

```
results = await dispatcher.dispatch(
    "lumen",          # tenant_id
    "DisbursementSettled",
    {"id": "txn-009", "amount": 50_000, "currency": "EUR"},
)
```

`dispatch()` devuelve un registro `CallbackExecution` por cada suscripción coincidente, cada uno con `status`, `attempts`, `response_status` y `delivered_at`.

Ejecútalo. El emisor HTTP por defecto del despachador no llama realmente a la red — registra la petición que *haría* y devuelve `200` —, lo cual es perfecto para ver el cableado sin levantar un servidor de socio. Suelta esto en un script (o en `uv run python`) desde `samples/lumen`:

lumen/callbacks/try_dispatch.py

```

import asyncio

from pyfly.callbacks import (
    CallbackConfig,
    CallbackDispatcher,
    CallbackSubscription,
    InMemoryCallbackConfigRepository,
    InMemoryCallbackExecutionRepository,
)

async def main() → None:
    configs = InMemoryCallbackConfigRepository()
    await configs.save(CallbackConfig(
        tenant_id="lumen",
        name="clearance-bank",
        secret="cb-secret-xyz",
        subscriptions=[CallbackSubscription(
            event_type="DisbursementSettled",
            target_url="https://api.clearancebank.example.com/hooks/lumen",
        )],
    ))
    dispatcher = CallbackDispatcher(
        configs=configs,
        executions=InMemoryCallbackExecutionRepository(),
    )
    results = await dispatcher.dispatch(
        "lumen",
        "DisbursementSettled",
        {"id": "txn-009", "amount": 50_000, "currency": "EUR"},
    )
    for r in results:
        print(r.status, r.attempts, r.response_status, r.target_url)

asyncio.run(main())

```


Listado 17.12a — Despacho contra el emisor por defecto (solo registra)

Deberías ver una ejecución entregada, con la petición firmada registrada justo encima de ella:

```
would POST https://api.clearancebank.example.com/hooks/lumen headers={'X-
Pyfly-Signature': 'sha256=...', 'Content-Type': 'application/json'}
body={'id': 'txn-009', 'amount': 50000, 'currency': 'EUR'}
DELIVERED 1 200 https://api.clearancebank.example.com/hooks/lumen
```

`DELIVERED 1 200` se lee como: estado `DELIVERED`, con éxito en el intento `1`, HTTP `200`. Para enviar de verdad, pasa tu propio emisor `http=` (un POST de `httpx / aiohttp`) a `CallbackDispatcher`; la lógica de firma, reintento y auditoría se mantiene idéntica.

❗ QUÉ ACABA DE PASAR

Una sola llamada a `dispatch()` buscó todas las suscripciones que el inquilino tiene para ese tipo de evento, firmó la carga útil, hizo POST y escribió un registro `CallbackExecution` para cada una —todo ello sin que tu servicio de dominio supiera cuántos socios están escuchando ni cómo funcionan los reintentos—. Añadir un socio más adelante es solo otra `CallbackConfig` guardada; el código de desembolso nunca cambia.

Firma HMAC y lógica de reintentos

Cuando `CallbackConfig.secret` está establecido, `CallbackDispatcher` firma la carga útil JSON canónica antes de cada POST usando HMAC-SHA256:

```
# canonical body — compact, keys sorted
canonical = json.dumps(payload, separators=(",", ":"), sort_keys=True)
sig = hmac.new(secret, canonical.encode(), hashlib.sha256).hexdigest()
headers["X-Pyfly-Signature"] = f"sha256={sig}"
```

El destinatario puede verificar la firma usando el propio `HmacSignatureValidator` de PyFly —la misma clase usada para los webhooks entrantes—.

Política de reintentos:

- El despachador reintentará hasta `max_attempts` veces (por defecto `5`).
- Entre reintentos aplica retroceso exponencial: `delay = min(backoff_ms * 2^(attempt-1), 300_000) ms`.
- Solo los códigos de estado HTTP *transitorios* disparan un reintento: `408`, `429`, `500`, `502`, `503`, `504`, o cualquiera ≥ 500 .
- Los errores de cliente permanentes (`4xx` salvo `408 / 429`) marcan la ejecución como `FAILED` de inmediato sin reintentar.
- Ante el éxito (`2xx`) la ejecución se marca como `DELIVERED` y se estampa `delivered_at`.

`lumen/callbacks/models_overview.py`

```
from pyfly.callbacks import CallbackStatus

# After a successful delivery:
assert execution.status == CallbackStatus.DELIVERED
assert execution.delivered_at is not None
assert execution.response_status == 200

# After all retries are exhausted:
assert execution.status == CallbackStatus.FAILED
assert execution.attempts == 5
assert execution.last_error is not None
```

Listado 17.13 — Ciclo de vida del estado de `CallbackExecution`

Protección contra SSRF: dominios autorizados

El campo `authorized_domains` de `CallbackConfig` actúa como una lista de permitidos. Cuando está establecido, `CallbackDispatcher` comprueba que el nombre de host de la URL de destino coincide con uno de los dominios permitidos antes de realizar cualquier petición saliente. Una URL que no supera la comprobación se marca de inmediato como `FAILED` con `last_error="Domain not authorized"` —no se realiza ninguna petición HTTP—.

```
from pyfly.callbacks import AuthorizedDomain, CallbackConfig

config = CallbackConfig(
    tenant_id="lumen",
    name="safe-config",
    secret="s3cr3t",
    authorized_domains=[
        AuthorizedDomain(domain="clearancebank.example.com"),
    ],
    subscriptions=[...],
)
```

Los subdominios de los dominios permitidos también se aceptan (p. ej.

`api.clearancebank.example.com`).

❗ TÉRMINO NUEVO: SSRF

La *falsificación de peticiones del lado del servidor* (Server-Side Request Forgery) es un ataque en el que un valor malicioso engaña a tu servidor para que haga una petición HTTP que no debería —por ejemplo, una URL de socio que apunta a `http://169.254.169.254/` (un endpoint de metadatos de la nube) para robar credenciales—. Como las URLs de callback pueden venir de configuración proporcionada por el socio, la lista de permitidos `authorized_domains` cierra esa puerta: un host que no está en la lista se marca como `FAILED` antes de que ninguna petición salga del proceso. Para verificarlo, añade un `AuthorizedDomain` para un host, luego despacha una suscripción cuya `target_url` apunte a otro sitio: el `CallbackExecution` devuelto mostrará `status=FAILED`, `attempts=0` y `last_error="Domain not authorized"`, y no se enviará nada.

🍃 EQUIVALENCIA CON SPRING

El trío `@scheduled` / `CronExpression` / `TaskScheduler` de PyFly refleja el `@Scheduled` / `CronExpression` / `ThreadPoolTaskScheduler` de Spring. Los puertos de notificación se corresponden con los `MailSender` / `JavaMailSender` de Spring. `WebhookProcessor` se corresponde con un `@RestController` de Spring + el `HmacRequestMatcher` de HMAC de Spring Security. `CallbackDispatcher` con firma HMAC y reintentos refleja el patrón `WebhookPublisher` de Spring Modulith.

Ejecútalo — confirma que la suite sigue en verde. Añadiste cuatro nuevos patrones de integración; asegúrate de que nada más sufrió una regresión. Desde el directorio `samples/lumen`:

```
uv run --extra dev pytest -q
```

Deberías ver que todas las pruebas existentes siguen pasando:

```
..S.S.....
[100%]
43 passed, 2 skipped in 0.28s
```

Los tres ejercicios de abajo añaden sus propias pruebas de programación, notificación y webhook — vuelve a ejecutar este comando después de cada uno para ver subir el recuento—. Recuerda la bandera `--extra dev`; el `uv sync` a secas omite pytest.

✓ Lo que construiste

Extendiste Lumen hasta convertirlo en un sistema conectado que opera con independencia de las peticiones entrantes:

- **Tareas programadas** — `@scheduled` con los disparadores `cron`, `fixed_rate` y `fixed_delay` ejecuta trabajo según un calendario o un temporizador. `CronExpression` impulsa los cálculos de los momentos de disparo, incluida la programación consciente de la zona horaria con el horario de verano gestionado automáticamente. `lock=True` serializa la ejecución en todo el clúster a través del puerto `DistributedLock`. `@async_method` descarga el trabajo fire-and-forget en el ejecutor.
- **Notificaciones** — Los protocolos de puerto `EmailService`, `SmsService` y `PushService` desacoplan la lógica de negocio de los adaptadores de proveedor (SMTP, SendGrid, Resend, Twilio, Firebase). `DefaultEmailService` y sus hermanos capturan los errores del proveedor y devuelven valores `NotificationResult` estructurados en lugar de propagar excepciones. `DepositNotificationListener` se suscribe al evento de EDA real `FundsDeposited` y convierte los importes en unidades menores en cadenas de visualización antes de enviar.

- **Webhooks entrantes** — `AbstractWebhookEventListener` define consumidores tipados. `WebhookProcessor` filtra cada evento con `HmacSignatureValidator` y `WebhookEventStore` antes de despacharlo a los listeners. Los ganchos `on_error()` permiten la integración con una cola de mensajes muertos sin romper el bucle de despacho.
- **Callbacks salientes** — `CallbackDispatcher` abre los eventos en abanico hacia los destinos de `CallbackSubscription`, firma las cargas útiles con HMAC-SHA256 bajo la cabecera `X-Pyfly-Signature` y reintenta ante fallos transitorios con retroceso exponencial. Los registros `CallbackExecution` proporcionan un rastro de auditoría de entrega completo. `authorized_domains` previene la SSRF.

Pruébalo tú mismo

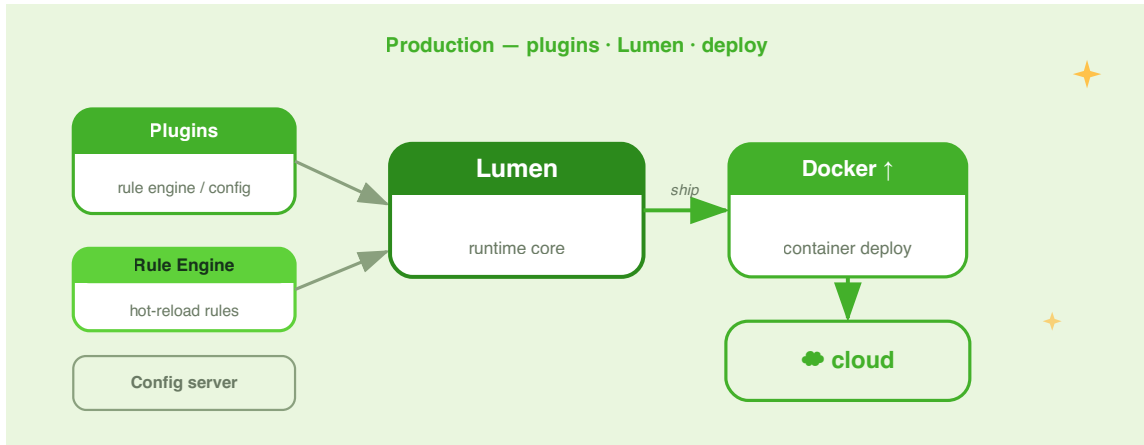
1. **Bloqueo de clúster.** Añade una segunda instancia de `DailyRollupService` a una prueba que levante dos instancias de `ApplicationContext` compartiendo un `FakeDistributedLock` (uno que solo devuelva `True` para el primer adquirente). Afirma que `run()` se invoca exactamente una vez en ambas instancias para un único tic de cron.
2. **Cambio de proveedor.** Reemplaza `SmtplibEmailProvider` por un `DummyEmailProvider` en la suite de pruebas. Escribe una prueba que deposite EUR 100 (10 000 unidades menores) en el monedero `w-001` a través del manejador de depósitos, disparando un evento `FundsDeposited`. El proveedor registra cada mensaje que recibió en su lista `.sent`, así que afirma que `provider.sent[-1].body_text` contiene el ID del monedero y el importe formateado (`100.00 EUR`).

3. Ataque de reenvío de firma. Escribe una prueba que invoque

`PaymentWebhookHandler.receive()` dos veces con el mismo cuerpo en crudo, cabeceras y clave de idempotencia. Afirmar que la primera llamada devuelve 200 y acredita el monedero (disparando `FundsDeposited`), y que la segunda llamada también devuelve 200 (el duplicado lo ignora silenciosamente `InMemoryWebhookEventStore`) pero **no** acredita el monedero una segunda vez.

CAPÍTULO 18

Extender PyFly y llevarlo a producción



Lumen ya no es un juguete. A lo largo de los diecisiete capítulos anteriores construiste un servicio de monedero (wallet) desde una única clase anotada hasta un microservicio completo orientado a eventos: CQRS, sagas, EDA, clientes HTTP, caché, resiliencia, observabilidad, seguridad y un conjunto de pruebas respaldado por contenedores reales. Sabes cómo ejecutarlo, depurarlo y desplegarlo.

Este capítulo final trata sobre la distancia que separa el "funciona en mi portátil" del "funciona de forma fiable para usuarios reales". Esa distancia tiene tres dimensiones. Primera: la **extensibilidad**, la capacidad de añadir comportamiento sin bifurcar el framework. Segunda: las funcionalidades que tu dominio puede necesitar ahora mismo: un motor de reglas, configuración centralizada, varios idiomas, push en tiempo real, una CLI. Tercera: los hábitos operativos que separan un proyecto de fin de semana de un servicio en producción.

Avanzarás rápido. Cada sección escoge un tema, muestra la API mínima que funciona y la conecta con Lumen. Al final tendrás una imagen completa del ecosistema PyFly y una lista de comprobación de producción que merece la pena tener a mano.

CONVENCIONES DE ESTE CAPÍTULO

Los listados y las claves de configuración de aquí apuntan a PyFly **v26.6.110**. Dos hechos de despliegue de esa versión recorren toda la sección "Llevarlo a producción", así que conviene conocerlos de antemano:

- La aplicación escucha en `pyfly.server.port` (por defecto `8080`), la clave equivalente a `server.port` de Spring. Las antiguas claves `pyfly.web.port` / `PYFLY_WEB_PORT` se eliminaron en v26.6.102.
- El Actuador y el panel de administración se ejecutan en un **puerto de gestión independiente** (`pyfly.management.server.port`, por defecto `9090`), abierto y sin autenticación por defecto. Lo cubrimos en detalle cuando conectemos las sondas de salud.

Cada funcionalidad de abajo se construye igual: un breve "por qué", el código y, a continuación, un punto de control **Ejecútalo** que muestra el comando exacto y la salida que deberías ver. Escribe los comandos a medida que avanzas: así es como las ideas se asientan.

Plugins y puntos de extensión

¿Por qué un sistema de plugins?

El framework central es pequeño de forma intencionada. Las funcionalidades opcionales (formateadores, sumideros de auditoría, canales de notificación) deberían poder conectarse como plugins para que los equipos compongan solo lo que necesitan y publiquen añadidos sin tocar las interioridades del framework.

El módulo `pyfly.plugins` de PyFly refleja el registro de plugins de Spring: declaras un **punto de extensión** (una ranura con nombre), **extensiones** (contribuciones concretas) y las agrupas en un **plugin** con un ciclo de vida.

La nueva jerga, en términos sencillos: un **punto de extensión** es un hueco etiquetado en tu aplicación: "todo lo que quiera ser un sumidero de auditoría se conecta aquí".

Una **extensión** es una cosa que llena el hueco. Un **plugin** es el paquete que entrega una o más extensiones juntas y que se puede arrancar y detener como una unidad. Si alguna vez has definido una interfaz de Java y has dejado que quienes la llaman registren implementaciones de ella, ya conoces la forma: los decoradores de abajo simplemente hacen el cableado declarativo.

Construiremos el ejemplo útil más pequeño: un sumidero de auditoría de consola que imprime cada evento que se le entrega.

Paso 1: declara el punto de extensión. Este es el contrato. Todo sumidero de auditoría debe prometer un método `record(event)`. La cadena `id="audit-sinks"` es el nombre que usa el resto del código para encontrar más tarde cada sumidero.

Paso 2: escribe un plugin que aporte un sumidero. El decorador `@plugin` envuelve una clase con un `id` y una `version` obligatorios; la clase interna `@extension` es la contribución real. Hereda de la interfaz del punto de extensión (`AuditSink`) para que el registro pueda confirmar que respeta el contrato.

Paso 3: dale al plugin un ciclo de vida. Los métodos `start` y `stop` son donde abres y cierras recursos (un descriptor de archivo, una conexión de red). El gestor de plugins los llama por ti.

Aquí tienes los tres pasos en un único archivo.

lumen/plugins/audit.py

```

from pyfly.plugins import (
    PluginManager,
    extension,
    extension_point,
    plugin,
)

@extension_point(id="audit-sinks")
class AuditSink:
    """Interface that all audit sinks must implement."""

    def record(self, event: dict) → None: ...

@plugin(id="console-audit", version="1.0.0")
class ConsoleAuditPlugin:

    @extension(point="audit-sinks", priority=10)
    class ConsoleSink(AuditSink):
        name = "console"

        def record(self, event: dict) → None:
            print(f"[AUDIT] {event}")

        async def start(self) → None:
            print("ConsoleAuditPlugin started")

        async def stop(self) → None:
            print("ConsoleAuditPlugin stopped")

```

Listado 18.1 — Declarar un plugin de sumidero de auditoría

Cómo funciona. `@extension_point(id="audit-sinks")` registra una ranura con nombre y declara la interfaz que toda contribución debe implementar.

`@plugin(id="console-audit", version="1.0.0")` declara una clase de plugin con un `id` y una `version` obligatorios. `@extension(point="audit-sinks", priority=10)` marca una clase interna como contribución a esa ranura; la clase interna debe heredar de la interfaz del punto de extensión para que el registro pueda validarla. La prioridad más alta gana la primera posición al iterar los resultados.

Cargar y ejecutar el plugin:

`lumen/plugins/runner.py`

```
import asyncio

from pyfly.plugins import PluginManager

from lumen.plugins.audit import ConsoleAuditPlugin


async def main() → None:
    manager = PluginManager()
    await manager.add(ConsoleAuditPlugin)
    await manager.start_all()

    sinks = await manager.registry.get("audit-sinks")
    for sink in sinks:
        sink.record({"action": "deposit", "amount_minor": 100})

    await manager.stop_all()

asyncio.run(main())
```

Listado 18.2 — Conducir el ciclo de vida del plugin

`PluginManager.add()` inspecciona la clase en busca de declaraciones `@extension_point` anidadas y luego registra cada contribución `@extension`. `start_all()` invoca los hooks `init` y después `start` de cada plugin en orden de dependencias; `stop_all()` invierte la secuencia, llamando a `stop` y luego a `unload`. Las dependencias circulares lanzan `PluginResolutionError` antes de que se ejecute código alguno.

🔗 EJECÚTALO

Guarda ambos listados bajo `src/lumen/plugins/` y ejecuta el módulo runner directamente:

```
uv run python -m lumen.plugins.runner
```

Deberías ver dispararse los hooks del ciclo de vida, registrarse el único evento y producirse el apagado limpio:

```
ConsoleAuditPlugin started
[AUDIT] {'action': 'deposit', 'amount_minor': 100}
ConsoleAuditPlugin stopped
```

El orden lo es todo: `start` se ejecuta antes de usar ningún sumidero, tu código itera los sumideros registrados y `stop` se ejecuta el último. Si añades un segundo plugin más adelante, `start_all()` arranca ambos en orden de dependencias y `stop_all()` los apaga en orden inverso: nunca gestionas ese orden a mano.

Lo que acaba de ocurrir. Añadiste una capacidad a la aplicación —el registro de auditoría— sin editar una sola línea de código del framework. El framework descubrió tu plugin, validó que su extensión respeta el contrato `AuditSink`, ejecutó su ciclo de vida y te entregó los sumideros registrados para que los llamaras. Esa es la promesa central de un sistema de plugins: abierto a la extensión, cerrado a la modificación.

Método	Descripción
<code>await manager.add(cls)</code>	Escanea y registra una clase de plugin
<code>await manager.start_all()</code>	<code>init</code> → <code>start</code> en orden de dependencias
<code>await manager.stop_all()</code>	<code>stop</code> → <code>unload</code> en orden inverso
<code>await manager.remove(plugin_id)</code>	Descarga un plugin; devuelve <code>False</code> si es desconocido
<code>await manager.registry.get(point_id)</code>	Extensiones de una ranura, ordenadas por prioridad

🌿 EQUIVALENCIA CON SPRING

`@plugin` / `@extension_point` / `@extension` reflejan `@Plugin` / `ExtensionPoint` / `@Extension` de la API de plugins de Spring. `PluginManager.start_all()` desempeña el papel de la gestión del ciclo de vida del contenedor de plugins de Spring. El arranque en orden de dependencias y el apagado en orden inverso son idénticos en semántica.

Reglas de negocio con el motor de reglas

La mayoría de los servicios del mundo real arrastran lógica que pertenece al negocio, no al código: "marca los pedidos de más de 500.000 céntimos", "bloquea los envíos a regiones sancionadas", "aplica un recargo fuera de horario". Codificar esos umbrales en Python a fuego significa recompilar cada vez que el negocio cambia de opinión.

El módulo `pyfly.rule_engine` de PyFly ofrece a los responsables de producto un dial YAML que pueden girar sin tocar el código fuente.

Un **motor de reglas**, en términos sencillos, es un pequeño intérprete de sentencias "si esto, entonces aquello" que viven en datos en lugar de en código. Le proporcionas un saco de hechos (el *contexto*) y un conjunto de reglas; comprueba la condición de cada regla contra los hechos y, para las que coinciden, ejecuta las acciones listadas, normalmente escribiendo una marca de vuelta en el contexto. La ventaja es que las *reglas* las pueden editar quienes no programan, mientras que el *motor* que las ejecuta es fijo y está probado.

Definir reglas en YAML

Construiremos una comprobación de fraude y límite en dos pasos: primero escribir las reglas como datos, luego conectar un servicio que las ejecuta.

Paso 1: escribe las reglas como YAML. Cada regla nombra una condición **when** y una lista de acciones **then**. Como las reglas son datos planos, un responsable de producto puede revisarlas en una pull request y un servidor de configuración puede intercambiarlas en caliente en tiempo de ejecución.

Las reglas viven en un archivo YAML separado que cualquier miembro del equipo puede revisar. El evaluador analiza este archivo una vez al arranque, o en cada obtención si lo recargas en caliente desde un Config Server (consulta la siguiente sección).

```
lumen/rules/transaction_rules.yaml
```

```
id: transaction-rules
name: Lumen transaction rules

rules:
  - id: daily-limit
    priority: 20
    when:
      op: ge
      field: transaction.daily_total
      value: 500000
    then:
      - type: set
        target: flags.limit_exceeded
        value: true
      - type: log
        value: "daily limit exceeded"

  - id: fraud-country
    priority: 10
    when:
      op: in
      field: transaction.country
      value: ["XX", "YY", "ZZ"]
    then:
      - type: set
        target: flags.fraud_risk
        value: true
      - type: log
        value: "high-risk country detected"

  - id: high-value
    priority: 5
    when:
      op: ge
      field: transaction.amount
      value: 100000
    then:
```



```
- type: set
  target: flags.high_value
  value: true
```

Listado 18.3 — Reglas de fraude y límite diario (importes en unidades menores)

Cada regla tiene una condición `when` y una lista de acciones `then`. Los importes son siempre **unidades menores enteras** (céntimos), así que `100000` son 1.000,00 €. Las condiciones usan estos operadores:

Comparación	Lógicos
<code>eq</code> , <code>ne</code> , <code>gt</code> , <code>ge</code> , <code>lt</code> , <code>le</code>	<code>and</code> , <code>or</code> , <code>not</code>
<code>in</code> , <code>not_in</code> , <code>regex</code>	(con <code>conditions: [...]</code>)

Las acciones son `set` (escribir en una ruta del contexto), `increment` o `log`. Crea una subclase de `RuleEvaluator` y sobrescribe `_execute_action` para añadir `call`, `calculate` o cualquier verbo personalizado.

Paso 2: conecta un servicio que cargue y ejecute las reglas. El servicio analiza el YAML una vez en la construcción y luego expone un método `assess()` que construye un contexto, ejecuta el evaluador y devuelve las marcas que las reglas establecieron.

lumen/rules/risk_service.py

```

from pathlib import Path

from pyfly.container import service
from pyfly.rule_engine import RuleSetEvaluator, RuleSetLoader

@service
class RiskService:
    """Evaluate transaction-level rules and return risk flags."""

    def __init__(self) → None:
        yaml_text = (
            Path(__file__).parent / "transaction_rules.yaml"
        ).read_text()
        self._ruleset = RuleSetLoader.from_yaml(yaml_text)
        self._evaluator = RuleSetEvaluator()

    def assess(
        self,
        amount: int,
        daily_total: int,
        country: str,
    ) → dict:
        ctx = {
            "transaction": {
                "amount": amount,
                "daily_total": daily_total,
                "country": country,
            },
            "flags": {},
        }
        self._evaluator.evaluate(self._ruleset, ctx)
        return ctx["flags"]

```

Listado 18.4 — Evaluar reglas contra una transacción

Cómo funciona. `RuleSetLoader.from_yaml(text)` analiza el YAML en un AST.

`RuleSetEvaluator.evaluate(ruleset, ctx)` recorre cada regla en orden de prioridad, evalúa la cláusula `when` y aplica las acciones `then` que coinciden, mutando `ctx` en el sitio y devolviendo una `list[EvaluationResult]`. El diccionario `flags` es la salida autoritativa: un manejador (handler) posterior rechaza, encola o marca la transacción en función de las claves que se hayan establecido. `amount` y `daily_total` están en unidades menores (céntimos) para coincidir con el dominio de Lumen.

🔗 EJECÚTALO

Ejercita el servicio desde un REPL de Python para ver dispararse las reglas. Una transferencia de 6.000,00 € (600000 unidades menores) dispara a la vez los umbrales de alto valor y de límite diario:

```
uv run python -c "
from lumen.rules.risk_service import RiskService
print(RiskService().assess(amount=600000, daily_total=600000,
country='US'))
"
```

Salida esperada: las marcas que establecen las reglas coincidentes, y nada más:

```
{'high_value': True, 'limit_exceeded': True}
```

Ahora baja el importe a 50000 (500,00 €) con un país limpio y recibes de vuelta un `{}` vacío: ninguna regla coincidió, así que no se marcó nada. Cambiaste el *resultado* sin tocar Python: ese es el dial que el YAML da a tus responsables de producto.

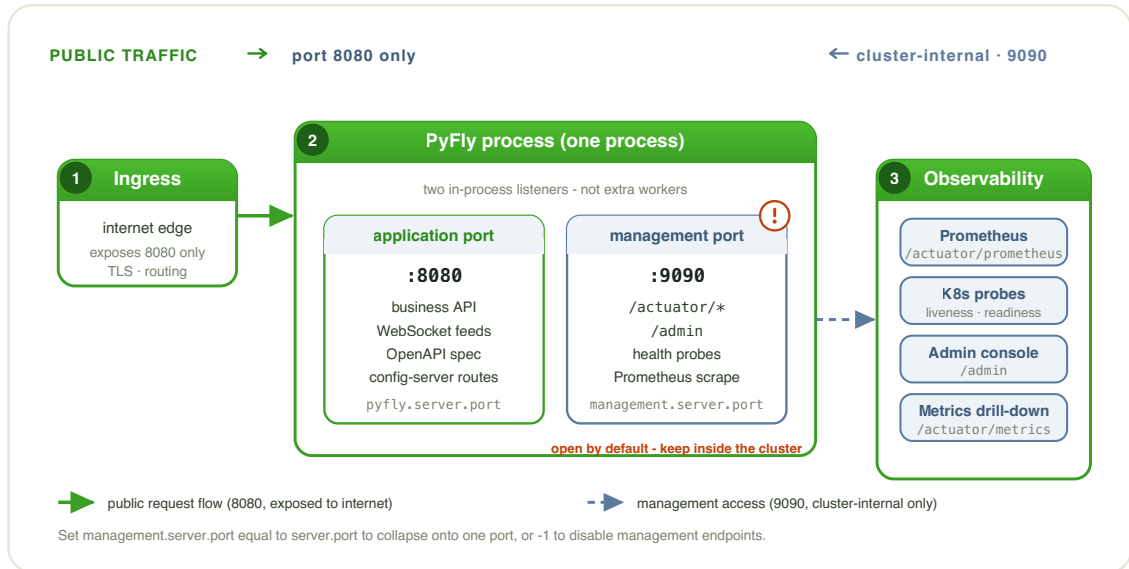


Figura 18.1 — Evaluación de reglas en la frontera del servicio. Las reglas YAML se analizan una vez al arranque; cada transacción pasa por el evaluador como un diccionario de contexto mutable.

💡 RECARGAR REGLAS EN CALIENTE SIN REDESPLEGAR

Almacena `transaction_rules.yaml` en el Config Server (consulta la siguiente sección) y vuelve a analizarlo en cada obtención. Tu motor de reglas se convierte en un dial en vivo que controla el negocio.

Configuración centralizada (Config Server)

A medida que Lumen crece a múltiples servicios, cada uno arrastra su propia copia de URLs de base de datos, tiempos de espera y feature flags. El módulo Config Server elimina esa duplicación: un servicio posee la verdad; todos los demás la obtienen al arrancar.

Un **config server**, en términos sencillos, es un pequeño servicio HTTP cuyo único trabajo es repartir configuración. En lugar de incrustar tiempos de espera y URLs en el `pyfly.yaml` de cada servicio, los almacenas en un único lugar y cada servicio pide su paquete al arrancar. Cambia el valor una vez, reinicia (o refresca) a los consumidores y toda la flota se mueve junta.

Ejecutar el servidor

Habilita el servidor en `pyfly.yaml`:

```
pyfly:
  config-server:
    enabled: true
    backend:
      root: /etc/lumen/config
```

Eso es todo. PyFly autoconfigura un `ConfigServer` respaldado por un `FilesystemConfigBackend` y monta las rutas HTTP automáticamente:

Método	Ruta	Propósito
GET	<code>/app/{profile}</code>	Obtener el paquete de configuración combinado
GET	<code>/app/{profile}/{label}</code>	Obtener para una etiqueta concreta
POST	<code>/app/{profile}</code>	Guardar un paquete de configuración
GET	<code>/_list</code>	Listar todos los paquetes almacenados

La forma de la respuesta es compatible con Spring Cloud Config, así que un servicio Spring existente puede consumir el mismo endpoint sin cambios.

Guardar y obtener configuración de forma programática

lumen/config/seed.py

```
import asyncio

from pyfly.config_server import (
    ConfigClient,
    ConfigServer,
    FilesystemConfigBackend,
)

async def seed() → None:
    server = ConfigServer(FilesystemConfigBackend("/etc/lumen/config"))
    await server.save(
        "wallet",
        "prod",
        {
            "db.url": "postgres://db:5432/lumen",
            "cache.ttl": 30,
        },
    )
    bundle = await server.fetch("wallet", "prod")
    print(bundle)

asyncio.run(seed())
```

Listado 18.5 — Sembrar y leer un paquete de configuración

 EJECÚTALO

Con `pyfly.config-server.enabled: true` en `pyfly.yaml`, arranca la app y haz un curl al paquete que sembraste. Recuerda: las rutas del config-server se sirven en el puerto de **aplicación** (8080), no en el puerto de gestión.

```
uv run pyfly run
# en otra terminal:
curl http://localhost:8080/wallet/prod
```

La respuesta tiene la forma de Spring Cloud Config: tus claves sembradas llegan dentro de un array `propertySources`:

```
{
  "name": "wallet",
  "profiles": ["prod"],
  "propertySources": [
    {"name": "wallet-prod", "source": {"db.url": "postgres://db:5432/
lumen", "cache.ttl": 30}}
  ]
}
```

Como la forma coincide con Spring Cloud Config, un servicio Spring Boot existente puede apuntar su `spring.config.import=configserver:` a esta misma URL y consumirla sin cambios.

Los servicios cliente la obtienen al arrancar con:

lumen/config/bootstrap.py

```

from pyfly.config_server import ConfigClient

async def load_remote() → dict:
    client = ConfigClient(
        url="http://config:8888",
        application="wallet",
        profile="prod",
        label="main",
    )
    return await client.fetch()

```

Listado 18.6 — Obtener configuración remota al arranque

`ConfigClient.fetch()` hace un GET a `{url}/{application}/{profile}/{label}`, combina el array `propertySources` (la prioridad más alta primero) y devuelve un diccionario plano `{dotted_key: value}`. En el funcionamiento normal nunca llamas a `ConfigClient` directamente: establece `pyfly.cloud.config.uri` en `pyfly.yaml` y `PyFlyApplication` lo llama automáticamente durante el bootstrap, combinando el resultado en el `Config` de la aplicación como una fuente de alta precedencia.

PRIORIDAD DE RESPALDO

El servidor ensambla hasta cuatro capas de superposición: `{app}/{profile}`, `{app}/default`, `application/{profile}`, `application/default`. Un cliente las combina con la primera fuente ganando, así que las anulaciones específicas de entorno siempre batan a los valores por defecto compartidos.

Internacionalización (i18n)

Los mensajes de error y las notificaciones de Lumen viven actualmente como literales de cadena en Python. Cuando los usuarios hablan idiomas diferentes, ese enfoque no escala.

i18n es la abreviatura de *internacionalización* (las 18 letras entre la "i" y la "n"). En la práctica significa sacar cada cadena visible para el usuario de tu código hacia **paquetes de recursos** por idioma, y luego elegir el paquete correcto en tiempo de ejecución según el idioma preferido de quien llama. Tu código se refiere a una cadena mediante una clave estable como `wallet.deposit_ok`; el framework busca la traducción.

Habilita el subsistema i18n con un único flag:

```
pyfly:
  i18n:
    enabled: true
    base-path: i18n/
    default-locale: en
```

Escribir paquetes de recursos

Paso 1: escribe un paquete por idioma. Los archivos se nombran

`messages_<locale>.yaml`. Las claves se comparten entre idiomas; solo cambian los valores. Los marcadores `{0}`, `{1}` son marcadores de posición posicionales que el framework rellena en tiempo de renderizado.

```
# i18n/messages_en.yaml
wallet:
  deposit_ok: "Deposited {0} minor units to wallet {1}."
  limit_exceeded: "Daily limit exceeded. Maximum is {0} minor units."

# i18n/messages_es.yaml
wallet:
  deposit_ok: "Se depositaron {0} unidades menores en la billetera {1}."
  limit_exceeded: "Se superó el límite diario. El máximo es {0} unidades menor
es."
```

`ResourceBundleMessageSource` resuelve claves con notación de puntos y sustituye los marcadores de posición `{n}` (base cero) siguiendo la semántica de `MessageFormat`. Los códigos ausentes recurren al `default-locale`; si tampoco están allí, `get_message` lanza `KeyError`.

Usar MessageSource en un servicio

Paso 2: inyecta `MessageSource` y resuelve el locale a partir de la petición. El

servicio de abajo lee la cabecera `Accept-Language` de quien llama, escoge el paquete que coincide y renderiza el mensaje con los argumentos en tiempo de ejecución.

lumen/i18n/notification_service.py

```

from pyfly.container import service
from pyfly.i18n import AcceptHeaderLocaleResolver, MessageSource

@service
class NotificationService:
    """Renders user-facing messages in the caller's preferred locale."""

    def __init__(
        self,
        messages: MessageSource,
        locale_resolver: AcceptHeaderLocaleResolver,
    ) → None:
        self._messages = messages
        self._resolver = locale_resolver

    def deposit_confirmation(
        self,
        request,
        amount_minor: int,
        wallet_id: str,
    ) → str:
        locale = self._resolver.resolve_locale(request)
        return self._messages.get_message_or_default(
            "wallet.deposit_ok",
            default="Deposit successful.",
            args=(amount_minor, wallet_id),
            locale=locale,
        )

```

Listado 18.7 — Servicio de notificación consciente del locale

`AcceptHeaderLocaleResolver` analiza la cabecera `Accept-Language` y devuelve el subtag primario con la `q` más alta. Usa `FixedLocaleResolver` para despliegues de un solo idioma o para pruebas. La autoconfiguración registra ambos cuando `pyfly.i18n.enabled: true`; inyecta o bien `MessageSource` (el protocolo del puerto) o bien el `ResourceBundleMessageSource` concreto: ambos funcionan.

🔗 EJECÚTALO

La forma más rápida de demostrar que la búsqueda funciona es una prueba que resuelve el paquete directamente, sin necesidad de un servidor HTTP. Guarda el Listado 18.7a bajo `tests/` y luego ejecuta solo esta prueba:

```
uv run --extra dev pytest tests/test_messages.py -q
```

Esperado:

```
1 passed in 0.05s
```

Cambia `locale="es"` por `locale="en"` y la aserción necesitaría la cadena en inglés en su lugar: misma clave, paquete distinto. Ese es justo el objetivo de `i18n`: tu código nunca cambia, solo cambia el locale resuelto.

tests/test_messages.py

```

from pyfly.i18n.adapters.resource_bundle import ResourceBundleMessageSource

def test_deposit_message_in_spanish() → None:
    messages = ResourceBundleMessageSource(base_path="i18n/",
    default_locale="en")
    text = messages.get_message(
        "wallet.deposit_ok", args=(100, "w-001"), locale="es"
    )
    assert text == "Se depositaron 100 unidades menores en la billetera
    w-001."

```

*Listado 18.7a — Afirmar un mensaje traducido***🍃 EQUIVALENCIA CON SPRING**

`MessageSource`, `ResourceBundleMessageSource`, `AcceptHeaderLocaleResolver` y `FixedLocaleResolver` son equivalentes directos de nombre del stack i18n de Spring MVC. La API difiere solo en el uso de marcadores de posición posicionales `{n}` en lugar de SpEL dentro de las cadenas de mensaje.

Actualizaciones en tiempo real con WebSocket

El panel de administración de Lumen actualmente sondea en busca de cambios de saldo. Un endpoint WebSocket elimina el sondeo: el servidor empuja una actualización en el instante en que un depósito se confirma.

Un **WebSocket** es una conexión bidireccional que permanece abierta. Una petición HTTP normal es de un solo disparo: el cliente pregunta, el servidor responde, la línea se cierra. Con un WebSocket la línea se mantiene activa, así que el *servidor* puede enviar datos cuando le apetezca: perfecto para actualizaciones en vivo. El esquema de URL es `ws://` (o `wss://` sobre TLS) en lugar de `http://`.

lumen/web/balance_ws_controller.py

```

import asyncio

from pyfly.container import rest_controller
from pyfly.web import request_mapping
from pyfly.websocket import WebSocketSession, websocket_mapping

@rest_controller
@request_mapping("/ws")
class BalanceFeedController:
    """Streams balance updates to connected clients."""

    def __init__(self, wallet_service) → None:
        self._wallet = wallet_service
        self._clients: set[WebSocketSession] = set()

    @websocket_mapping("/balance/{wallet_id}")
    async def balance_feed(self, session: WebSocketSession) → None:
        wallet_id = session.path_params["wallet_id"]
        await session.accept()
        self._clients.add(session)
        try:
            while True:
                balance = await self._wallet.get_balance(wallet_id)
                await session.send_json(
                    {"wallet_id": wallet_id, "balance_minor": balance}
                )
                await asyncio.sleep(1)
        finally:
            self._clients.discard(session)

    async def on_disconnect(self, session: WebSocketSession) → None:
        self._clients.discard(session)

```

Listado 18.8 — Flujo de saldo en vivo vía WebSocket

Cómo funciona. `@websocket_mapping("/balance/{wallet_id}")` monta el endpoint en `ws://<host>/ws/balance/{wallet_id}`. La ruta completa es la base `@request_mapping` del controlador (`/ws`) concatenada con la ruta del decorador.

💡 EJECÚTALO

Arranca la app y luego abre el flujo en vivo con cualquier cliente WebSocket. Usando `websocat` (`brew install websocat`):

```
uv run pyfly run
# en otra terminal:
websocat ws://localhost:8080/ws/balance/w-001
```

Deberías ver llegar un frame JSON aproximadamente una vez por segundo, empujado por el servidor sin que vuelvas a pedirlo:

```
{"wallet_id": "w-001", "balance_minor": 5000}
{"wallet_id": "w-001", "balance_minor": 5000}
```

Deposita en `w-001` desde otra terminal y observa cómo el valor de `balance_minor` salta en el siguiente frame: sin sondeo, sin refresco. Pulsa `Ctrl+C` para cerrar; se ejecuta el `on_disconnect` del controlador y la sesión se elimina del conjunto de difusión.

`WebSocketSession` expone el ciclo de vida de la conexión:

Método	Descripción
<code>await accept(subprotocol=None)</code>	Completar el handshake
<code>await send_json(data)</code>	Serializar y enviar un mensaje JSON
<code>await send_text(data)</code>	Enviar una cadena plana
<code>await receive_text()</code>	Bloquear hasta que llegue un mensaje de texto
<code>await receive_json()</code>	Bloquear hasta que llegue un mensaje JSON
<code>await close(code=1000, reason=None)</code>	Cerrar la conexión limpiamente

`session.path_params`, `session.query_params` y `session.headers` exponen los metadatos de la conexión. Las rutas WebSocket se descubren automáticamente junto con las rutas HTTP: no se requiere configuración adicional.

El método opcional `on_disconnect` lo invoca automáticamente el registrador después de que el manejador `@websocket_mapping` retorne o el cliente cierre la conexión (solo si la conexión se aceptó antes), dando a los controladores un lugar seguro donde liberar recursos.

💡 DIFUSIÓN (BROADCASTING)

Mantén un `set[WebSocketSession]` en el controlador y difunde con `for client in list(self._clients): await client.send_json(payload)`. Como los beans de los controladores son singletons, el conjunto vive durante toda la vida de la aplicación.

Comandos de shell y runners de arranque

No toda funcionalidad vive detrás de un endpoint HTTP. Los scripts de siembra de bases de datos, las migraciones de datos puntuales y los trabajos por lotes programados se expresan mejor como comandos CLI que se ejecutan dentro del mismo contenedor de DI, compartiendo servicios, configuración y repositorios con la aplicación principal.

La idea clave aquí: un **comando de shell** es solo un método que se ejecuta desde la terminal pero que aún tiene acceso completo a los servicios cableados de tu aplicación. No escribes un script aparte que recree la conexión a la base de datos a mano: el comando recibe el mismo `WalletService` que usan tus controladores HTTP, porque vive dentro del mismo contenedor de DI.

@shell_component y @shell_method

lumen/cli/wallet_commands.py

```

from pyfly.shell import (
    shell_argument,
    shell_component,
    shell_method,
    shell_option,
)

@shell_component
class WalletCommands:
    """Operational commands for the Lumen wallet service."""

    def __init__(self, wallet_service) → None:
        self._wallet = wallet_service

    @shell_method(group="wallet", help="Deposit funds into a wallet")
    @shell_argument("wallet_id", help="Target wallet identifier")
    @shell_option("--amount", help="Amount in minor units (integer cents)")
    async def deposit(
        self, wallet_id: str, amount: int = 100
    ) → str:
        result = await self._wallet.deposit(wallet_id, amount)
        return f"New balance: {result['balance_minor']} minor units"

    @shell_method(group="wallet", help="Show current balance")
    @shell_argument("wallet_id", help="Wallet to inspect")
    async def balance(self, wallet_id: str) → str:
        data = await self._wallet.get_balance(wallet_id)
        return f"{wallet_id}: {data['balance_minor']} minor units"

```

Listado 18.9 — Comandos de shell con DI

Habilita el shell en `pyfly.yaml`:

```
pyfly:
  shell:
    enabled: true
```

PyFly autoconfigura un `ClickShellAdapter` y cablea cada `@shell_method` al arranque. El nombre del grupo se convierte en un subcomando:

```
python -m lumen wallet deposit w-001 --amount 500
python -m lumen wallet balance w-001
python -m lumen          # sin argumentos → entra en modo REPL
```

🔗 EJECÚTALO

Deposita 500 unidades menores en un monedero directamente desde la línea de comandos: el comando ejecuta tu `WalletService` real contra la base de datos real:

```
uv run python -m lumen wallet deposit w-001 --amount 500
```

Salida esperada (el valor de retorno de tu método `deposit`, impreso por el adaptador de shell):

```
New balance: 500 minor units
```

Ejecuta `uv run python -m lumen wallet balance w-001` y verás reflejado de vuelta el mismo `500`: prueba de que el comando y la API HTTP hablan con el mismo estado persistente, no con una copia desechable en memoria.

CommandLineRunner: tareas puntuales tras el arranque

Para tareas que se ejecutan una vez al arranque —siembra, calentamiento, comprobaciones de conexión— implementa `CommandLineRunner`:

lumen/runners/seed_runner.py

```

from pyfly.container import service
from pyfly.shell import CommandLineRunner

@service
class SeedRunner(CommandLineRunner):
    """Seed the database with a default admin wallet on first boot."""

    def __init__(self, wallet_service) → None:
        self._wallet = wallet_service

    async def run(self, args: list[str]) → None:
        if "--seed" in args:
            await self._wallet.ensure_default_wallet()
            print("Default wallet ensured.")

```

Listado 18.10 — Sembrador de base de datos tras el arranque

Cualquier bean cuya clase implemente `async def run(self, args: list[str]) → None` satisface estructuralmente el protocolo `CommandLineRunner`. El framework lo detecta vía `isinstance()` (el protocolo es `@runtime_checkable`) después de que se dispare `ApplicationReadyEvent`, y luego lo invoca con los argumentos CLI en crudo. Usa `@order(n)` para controlar el orden de ejecución cuando coexisten varios runners.

💡 EJECÚTALO

El runner se dispara durante el arranque de la aplicación y recibe el `sys.argv[1:]` del proceso, así que lanza la app a través de su punto de entrada CLI con el flag añadido:

```
uv run python -m lumen --seed
```

Tras el banner y la tabla de rutas, verás la línea de confirmación del runner:

```
Default wallet ensured.
```

Arranca sin `--seed` y la línea no aparece: la guarda `if "--seed" in args` se salta el trabajo. Esa es la diferencia entre un *runner* (se ejecuta en cada arranque, lo controlas tú) y un comando de shell puntual (se ejecuta solo cuando invocas su nombre).

🍃 EQUIVALENCIA CON SPRING

`@shell_component`, `@shell_method`, `@shell_option`, `@shell_argument` y `CommandLineRunner` son equivalentes directos de `@ShellComponent`, `@ShellMethod`, `@ShellOption`, `@ShellArgument` de Spring Shell y de la interfaz `CommandLineRunner` de Spring Boot. Click reemplaza a JLine como librería de terminal, pero el modelo de programación es idéntico.

Generar un SDK a partir de la especificación OpenAPI

Cuando Lumen expone una API HTTP, los servicios posteriores deberían llamarla mediante un cliente generado, no mediante llamadas `httpx` escritas a mano que se desincronizan. PyFly construye y sirve una especificación OpenAPI 3.1 automáticamente en `/openapi.json`.

Una **especificación OpenAPI** es una descripción legible por máquina de tu API HTTP: cada ruta, cada parámetro, cada forma de petición y respuesta. Un **SDK** (software development kit) es la librería cliente que una herramienta genera *a partir de* esa especificación: métodos tipados que envuelven las llamadas HTTP por ti. La cadena es: tus controladores → la especificación → un cliente generado. Como cada eslabón es mecánico, el cliente nunca puede desincronizarse silenciosamente del servidor.

`OpenAPIGenerator` ensambla la especificación a partir de los metadatos de ruta recopilados por `ControllerRegistrar`:

- **Info:** poblada a partir de `title`, `version` y `description` pasados a `create_app()`.
- **Paths:** una operación por cada `@get_mapping` / `@post_mapping`, etc., con parámetros inferidos a partir de las anotaciones de tipo `PathVar[T]`, `QueryParam[T]`, `Header[T]` y `Body[BaseModel]`.
- **Schemas:** modelos de Pydantic registrados en `components.schemas` vía `model_json_schema()` y referenciados con `$ref`.

Con la especificación disponible, genera un cliente Python en dos pasos: descarga la especificación y luego ejecuta el generador.

Paso 1: descarga la especificación de una instancia en ejecución. La especificación vive en el puerto de aplicación (`8080`), junto a tus rutas de negocio.

Paso 2: ejecuta el generador de OpenAPI para convertir ese JSON en un paquete Python instalable.

```
# Download the spec from a running instance
curl http://localhost:8080/openapi.json -o lumen-spec.json

# Generate a Python client package
openapi-generator-cli generate \
  -i lumen-spec.json \
  -g python \
  -o lumen-client \
  --package-name lumen_client
```

El paquete `lumen_client` generado contiene modelos tipados y un `DefaultApi` con un método por operación. Los servicios consumidores lo añaden como dependencia y lo llaman sin saber nada de HTTP:

`payment/services/wallet_client.py`

```
from lumen_client import ApiClient, Configuration, DefaultApi

class WalletGateway:
    """Typed façade over the generated Lumen client SDK."""

    def __init__(self, base_url: str) → None:
        cfg = Configuration(host=base_url)
        self._api = DefaultApi(ApiClient(cfg))

    def get_balance(self, wallet_id: str) → int:
        result = self._api.get_wallet_balance(wallet_id)
        return result.balance_minor
```

Listado 18.11 — Consumir el SDK de Lumen generado

 MANTÉN LA ESPECIFICACIÓN VERSIONADA

Incluye `lumen-spec.json` en el repositorio de Lumen y regenera los paquetes cliente en CI cada vez que cambie la especificación. Los equipos posteriores se fijan a una versión concreta de la especificación a través de su gestor de dependencias: la misma disciplina que usan los equipos Java con las versiones de artefactos de Maven.

Llevarlo a producción

Empaquetar con Docker

Contenerizar, en términos sencillos, significa congelar tu app y todo lo que necesita para ejecutarse —Python, dependencias, tu código, tu configuración— en una única imagen que se ejecuta de forma idéntica en tu portátil, en CI y en producción. Un `Dockerfile` es la receta para construir esa imagen.

`pyfly new` genera un `Dockerfile` para cada arquetipo. Para un servicio web tiene este aspecto tras el endurecimiento de producción. Léelo de arriba abajo: cada línea es un paso del build:

```

FROM python:3.12-slim

WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN pip install uv && \
    uv sync --no-dev --extra web --extra data-relational \
        --extra security --extra observability

COPY src/ src/
COPY pyfly.yaml .

# 8080 = application traffic; 9090 = the management port (actuator + admin).
EXPOSE 8080 9090
CMD ["pyfly", "run", "--host", "0.0.0.0", \
    "--port", "8080", "--server", "granian", "--workers", "2"]

```

Recorriendo la receta: **FROM** escoge una imagen base de Python pequeña; **COPY** + **uv sync** instalan exactamente los extras de dependencias que nombras (y nada más); el segundo **COPY** añade tu fuente y tu configuración; **EXPOSE** documenta los dos puertos en los que escucha el contenedor; **CMD** es el comando que se ejecuta cuando arranca el contenedor. El flag **--port 8080** de aquí establece **pyfly.server.port** para este proceso: el puerto de gestión se queda en su valor por defecto **9090** salvo que anules **pyfly.management.server.port**.

Instala solo los extras que tu servicio realmente usa: el meta-extra **full** arrastra los drivers de Kafka, RabbitMQ y MongoDB incluso cuando no necesitas ninguno de ellos.

💡 EJECÚTALO

Construye la imagen y ejecútala, mapeando ambos puertos fuera del contenedor:

```
docker build -t lumen:local .
docker run --rm -p 8080:8080 -p 9090:9090 lumen:local
```

Luego confirma que ambos listeners responden: el puerto de negocio en **8080** y el puerto de gestión en **9090** :

```
curl http://localhost:8080/openapi.json # app: returns the spec
curl http://localhost:9090/actuator/health # management: {"status": "UP"}
```

Dos puertos, un proceso. Esa separación es sobre la que se construye el resto de esta sección.

Variables de entorno y secretos

Nunca incrustes secretos en `pyfly.yaml`. PyFly resuelve marcadores de posición `${ENV_VAR}` en cualquier parte de la configuración:

```
pyfly:
  data:
    relational:
      url: ${DATABASE_URL}

  security:
    jwt:
      secret: ${JWT_SECRET}
```

PyFly lee los valores reales del entorno del contenedor al arranque. En Kubernetes, respalda esas variables con un `Secret`; en Docker Compose, usa un archivo `.env` que nunca se commitea. El comando `pyfly doctor` comprueba que las herramientas requeridas están presentes, pero no valida secretos: esa responsabilidad sigue siendo tuya.

Apagado ordenado

PyFly respeta el apagado ordenado por defecto. Establece `pyfly.server.graceful-timeout` (en segundos) para controlar cuánto espera el servidor a que se completen las peticiones en vuelo antes de forzar la salida:

```
pyfly:
  server:
    graceful-timeout: 30
```

SIGTERM dispara la secuencia de apagado: el servidor deja de aceptar nuevas conexiones, `ApplicationContext.stop()` ejecuta los hooks `@pre_destroy` y `stop_all()` para los plugins, y el proceso sale limpiamente. En Kubernetes, establece `terminationGracePeriodSeconds` al menos cinco segundos por encima de `graceful-timeout`.

Selección de servidor

El servidor ASGI se selecciona por prioridad en tiempo de ejecución:

Prioridad	Servidor	Extra de instalación
1	Granian (Rust/tokio)	<code>granian</code>
2	Uvicorn	<code>web</code> (por defecto)
3	Hypercorn	<code>hypercorn</code>

Para producción, prefiere Granian: ofrece aproximadamente 3x el rendimiento de Uvicorn con HTTP/2 nativo. Combínalo con uvloop en Linux para un aceleramiento adicional de 2 a 4x del bucle de eventos:

```
uv add "pyfly[web-fast]" # granian + uvloop in one shot
```

Fija la elección en YAML para evitar sorpresas en máquinas donde resulta que hay varios servidores instalados:

```
pyfly:
  server:
    type: granian
    event-loop: uvloop
    workers: 4
    graceful-timeout: 30
```

El despliegue de dos puertos

Este es el hecho de despliegue que más necesitas interiorizar para v26.6.110. PyFly se ejecuta en **dos puertos**, ambos dentro de un único proceso:

- el **puerto de aplicación** — `pyfly.server.port`, por defecto `8080` — sirve tu API de negocio, tus flujos WebSocket, la especificación OpenAPI y las rutas del config-server;
- el **puerto de gestión** — `pyfly.management.server.port`, por defecto `9090` — sirve el Actuator (`/actuator/*`) y el panel de administración (`/admin`).

El puerto de gestión es un segundo listener en proceso, no procesos worker adicionales, así que casi no cuesta nada. Dos opciones de ajuste importan en el momento del despliegue: establece `pyfly.management.server.port` **igual** a `pyfly.server.port` para colapsar todo en un único puerto, o ponlo a `-1` para deshabilitar por completo los endpoints web de gestión. La anulación por entorno es `PYFLY_MANAGEMENT_SERVER_PORT`.

¿Por qué separarlos? Porque te permite exponer solo `8080` a internet manteniendo las comprobaciones de salud, los scrapes de Prometheus y la consola de administración en `9090`, accesibles únicamente desde dentro de tu clúster.

⚠ EL PUERTO DE GESTIÓN ESTÁ ABIERTO POR DEFECTO

A partir de v26.6.110, el puerto de gestión está **sin autenticación por defecto** (el modelo `management.server.port` de Spring Boot): cualquier cosa que pueda alcanzar `9090` puede leer `/actuator/*` y `/admin`. Eso es intencionado: el puerto está pensado para situarse tras aislamiento de red, nunca en el internet público. Si no puedes garantizar ese aislamiento, aplica también los filtros de seguridad de la app al puerto de gestión:

```
pyfly:
  management:
    security:
      enabled: true
```

Con ese flag activado, la misma autenticación, las guardas de roles y las reglas CSRF que protegen tu API de negocio también protegen `9090`.

Endpoints de salud

PyFly expone endpoints de actuator al estilo de Spring Boot de fábrica. Viven en el **puerto de gestión** (`9090`):

Endpoint	Propósito
<code>GET /actuator/health</code>	Salud agregada (UP / DOWN)
<code>GET /actuator/health/liveness</code>	Sonda de liveness de Kubernetes
<code>GET /actuator/health/readiness</code>	Sonda de readiness de Kubernetes
<code>GET /actuator/metrics</code>	Desglose por métrica (p. ej. <code>/actuator/metrics/http.server.requests</code>)
<code>GET /actuator/prometheus</code>	Endpoint de scrape de Prometheus

Solo `health` e `info` se exponen sobre HTTP por defecto (el valor por defecto seguro de Spring Boot). Para publicar las métricas y el endpoint de scrape de Prometheus, añádelos a la lista de exposición en `pyfly.yaml`:

```
pyfly:
  management:
    endpoints:
      web:
        exposure:
          include: health,info,metrics,prometheus
    endpoint:
      health:
        show-details: when-authorized
```

🔗 EJECÚTALO

Golpea las sondas en el puerto de gestión: responden de inmediato porque ese puerto está abierto por defecto:

```
curl http://localhost:9090/actuator/health
curl http://localhost:9090/actuator/health/readiness
```

Esperado: un objeto de estado JSON que el orquestador puede analizar:

```
{"status": "UP"}
```

Una comprobación de readiness que falla (una base de datos aún calentándose, por ejemplo) devuelve `{"status": "DOWN"}` con HTTP `503`, que es exactamente lo que Kubernetes necesita para retener el tráfico hasta que el pod esté listo.

Luego conecta tu deployment de Kubernetes: fíjate en que cada sonda y cada scrape apuntan al **puerto de gestión** `9090`, no a `8080`:

```

livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 9090
  initialDelaySeconds: 10
  periodSeconds: 15
readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 9090
  initialDelaySeconds: 5
  periodSeconds: 10

```

🍃 EQUIVALENCIA CON SPRING

La división de dos puertos es un port directo del `management.server.port` de Spring Boot: tráfico de la app en `server.port`, actuator y la UI de gestión en un `management.server.port` dedicado. La postura de abierto por defecto, la convención de `-1` para deshabilitar y `management.security.enabled` para bloquearlo reflejan todos el comportamiento de Spring Boot.

📄 SONDA PERSONALIZADA DESDE EL HEALTHAGGREGATOR EN VIVO

Nuevo en v26.6.110, el `HealthAggregator` en vivo es accesible en `app.state.pyfly_health_aggregator` (solo adaptador Starlette). Registra un indicador de readiness adicional después de `create_app()` —por ejemplo una comprobación que haga ping a un servicio posterior— y aparece en `/actuator/health` independientemente de si el actuator se ejecuta en el puerto de gestión compartido o en el separado.

Lo que acaba de ocurrir. Ahora tienes la forma completa de un despliegue de PyFly: un contenedor, dos puertos (`8080` para usuarios, `9090` para operaciones), secretos inyectados como variables de entorno, un servidor Granian ajustado con workers

explícitos, una ventana de apagado ordenado que drena las peticiones en vuelo y sondas de Kubernetes conectadas al puerto de gestión. La lista de comprobación de abajo es la misma imagen en forma de lista que puedes pegar en una plantilla de pull request.

La lista de comprobación de producción

- [] Todos los secretos son variables de entorno: ninguno está en `pyfly.yaml` ni en el control de versiones.
- [] La imagen Docker instala solo los extras que el servicio usa.
- [] El servidor está fijado a Granian + uvloop; `workers` está establecido explícitamente (no dejado en `1` para máquinas multinúcleo).
- [] `graceful-timeout` es de al menos 15 s; el `terminationGracePeriodSeconds` de Kubernetes es al menos 5 s más.
- [] Las sondas de liveness y readiness apuntan al **puerto de gestión** (`9090`), no al puerto de aplicación, y están probadas.
- [] `/actuator/health` (en `9090`) devuelve UP antes de que se envíe tráfico.
- [] El puerto de gestión está aislado a nivel de red, o `pyfly.management.security.enabled: true` está establecido: está abierto por defecto.
- [] El endpoint de scrape de Prometheus (`/actuator/prometheus` en `9090`) está añadido a la lista de exposición y lo scrapea el stack de monitorización.
- [] El logging estructurado está habilitado (`pyfly[observability]`) y el nivel de log es `INFO` en producción, no `DEBUG` .
- [] El exportador de OpenTelemetry apunta al colector de producción.
- [] Las migraciones de base de datos se ejecutan antes de que llegue el tráfico: `pyfly db upgrade` en un paso previo al despliegue, o `pyfly.data.relational.migrations.enabled: true` , que las serializa entre

instancias al arrancar. `ddl-auto` vale `none` (su valor por defecto en un servidor de base de datos) o `validate`.

- [] La especificación OpenAPI generada está versionada en CI y los paquetes SDK posteriores se fijan a una revisión concreta de la especificación.
- [] `pyfly doctor` pasa en cada máquina de desarrollo y en cada runner de CI.

✓ Lo que construiste

Hace diecisiete capítulos escribiste `@pyfly_application` y viste cómo el contenedor de DI cableaba tu primer `@service`. Hoy ese mismo contenedor impulsa una plataforma de monedero en producción. Esto es lo que construiste por el camino.

Capítulos 1–3 te dieron los cimientos: un contenedor de DI de primera clase, configuración flexible (YAML, env, perfiles, expresiones SpEL) y una capa HTTP completa con request mapping, filtros, negociación de contenido y una capa de serialización JSON que puedes reemplazar sin tocar un solo controlador.

Capítulos 4–6 introdujeron la persistencia. Mapeaste entidades con SQLAlchemy, gestionaste la evolución del esquema con Alembic y estructuraste el dominio con patrones tácticos de DDD: agregados, objetos de valor y repositorios que mantienen la lógica de negocio fuera de la infraestructura.

Capítulos 7–9 dieron vida a la arquitectura. CQRS separó las lecturas de las escrituras a nivel del manejador. EDA dejó que los servicios reaccionaran a eventos sin sondear. El event sourcing convirtió cada cambio de estado en un hecho de primera clase: reproducible, auditable y el cimiento de las proyecciones.

Capítulos 10–12 enseñaron a Lumen a salir de su propio proceso. Clientes HTTP resilientes llamaban a servicios posteriores sin fallos en cascada. Las sagas orquestaban transacciones de múltiples pasos a través de las fronteras de los servicios con compensación automática cuando algún paso salía mal.

Capítulos 13–15 endurecieron la plataforma. La caché redujo la presión sobre la base de datos. Limitadores de tasa, bulkheads, tiempos de espera y cortacircuitos (circuit breakers) convirtieron cada dependencia en un radio de impacto controlado. Las trazas distribuidas, el logging estructurado y un panel de administración en vivo te dieron ojos dentro del sistema en cada capa.

Capítulos 16–17 cerraron el bucle de retroalimentación. Un conjunto de pruebas estructurado —pruebas unitarias, de integración y de persistencia respaldadas por Testcontainers— hizo seguro cambiar la plataforma. Tareas programadas, notificaciones push, webhooks y callbacks dejaron que Lumen llegara al mundo según su propio calendario.

Capítulo 18 te mostró lo que hay más allá del núcleo: un sistema de plugins para la extensión abierta, un motor de reglas YAML para la lógica propiedad del negocio, un Config Server para la configuración de toda la flota, i18n para audiencias globales, WebSocket para UX en tiempo real, un módulo Shell para herramientas operativas, una especificación OpenAPI que genera SDKs cliente tipados automáticamente y los hábitos de producción que mantienen todo eso en marcha, empaquetado como un único contenedor que escucha en dos puertos, 8080 para usuarios y 9090 para operaciones.

PyFly no es magia. Toda abstracción de este libro tiene un coste, y ahora entiendes cuál es ese coste: un contenedor de DI que arranca en milisegundos pero te obliga a pensar en los ámbitos de los beans; un servidor HTTP asíncrono que maneja miles de

conexiones concurrentes pero te obliga a evitar llamadas bloqueantes; un motor de sagas que sobrevive a fallos parciales pero te obliga a escribir transacciones de compensación.

Entender el coste es lo que separa a un practicante de un lector que se limita a copiar patrones. Ahora eres un practicante.

Pruébalo tú mismo

- 1. Añade un plugin personalizado.** Implementa un `AuditPlugin` que aporte una extensión a un punto de extensión `"audit-sinks"`. La clase de extensión debe implementar una interfaz `AuditSink` y escribir cada evento en un archivo. En una prueba, llama a `PluginManager.registry.get("audit-sinks")` y afirma que tu extensión se devuelve con el `name` esperado.
 - 2. Despliega un cambio de reglas sin redespregar.** Almacena `transaction_rules.yaml` en el Config Server (`pyfly.config-server.enabled: true`). Escribe un `RiskService` que obtenga el YAML vía `ConfigClient` en cada llamada a `assess()` (o que lo cachee con un TTL corto). Actualiza el umbral `value` a través de la ruta `POST /{app}/{profile}` y verifica que `assess()` recoge el nuevo valor sin reiniciar el servicio.
 - 3. Localiza un mensaje de rechazo.** Añade `wallet.limit_exceeded` a `i18n/messages_en.yaml` e `i18n/messages_es.yaml`. Conecta un `NotificationService` que lea el locale de la cabecera `Accept-Language` y devuelva la cadena correcta. Escribe dos pruebas —una con `Accept-Language: en`, otra con `Accept-Language: es`— y afirma que cada una devuelve el mensaje adecuado.
-

Lumen está lista para producción. Para lo que viene a continuación —nuevos módulos, plugins de la comunidad y notas de versión— visita la documentación del framework en github.com/fireflyframework/fireflyframework-pyfly. Cada concepto de este libro vive en ese repositorio; el código fuente es la referencia definitiva.

Apéndices

APÉNDICE A

Chuleta Spring Boot → PyFly

Si has puesto en producción servicios con Spring Boot, los conceptos de PyFly te resultarán inmediatamente familiares: estereotipos, inyección por constructor, factorías `@configuration` + `@bean`, vinculación tipada de configuración, consultas derivadas, orquestación de sagas — están todos aquí. Lo que cambia es la sintaxis (decoradores de Python en lugar de anotaciones de Java), el modelo de ejecución (`async/await` nativo en lugar de Project Reactor o hilos de servlet) y un puñado de decisiones de diseño deliberadas para encajar con el Python idiomático. Esta chuleta asocia cada concepto de Spring Boot que ya conoces con su equivalente en PyFly, de modo que puedas empezar a leer y escribir código PyFly sin reaprender la arquitectura desde cero.

Arranque de la aplicación y estereotipos

Spring Boot	PyFly	Notas
<code>@SpringBootApplication</code>	<code>@pyfly_application</code>	Combina <code>@EnableAutoConfiguration</code> + <code>@ComponentScan</code> . <code>scan_packages</code> sustituye al escaneo del classpath — enumera los paquetes explícitamente.
<code>SpringApplication.run(...)</code>	<code>PyFlyApplication(App); await app.startup()</code>	El punto de entrada es asíncrono.
<code>@Component</code>	<code>@component</code>	Bean singleton genérico gestionado.
<code>@Service</code>	<code>@service</code>	Capa de lógica de negocio.
<code>@Repository</code>	<code>@repository</code>	Capa de acceso a datos.
<code>@RestController</code>	<code>@rest_controller</code>	Clase de endpoint de API. No hay <code>@Controller</code> (no se renderizan vistas).
<code>@Configuration</code>	<code>@configuration</code>	Clase factoría de beans.
<code>@Bean</code>	<code>@bean</code>	Método factoría dentro de una clase <code>@configuration</code> . La anotación del tipo de retorno es el tipo registrado del bean.
<code>@Primary</code>	<code>@primary</code> o <code>@bean(primary=True)</code>	Opción por defecto cuando existen varios candidatos.
<code>@Order(N)</code>	<code>@order(N)</code>	Ordenación del ciclo de vida y de la inyección.

Spring Boot	PyFly	Notas
@Lazy	@lazy	El bean no se crea hasta su primera resolución.

Inyección de dependencias

Spring Boot	PyFly	Notas
Constructor <code>@Autowired</code> (implícito en el Spring moderno)	Constructor normal con parámetros tipados	El contenedor lee las anotaciones de tipo de <code>__init__</code> automáticamente.
Campo <code>@Autowired</code>	<code>field: T = Autowired()</code>	<code>Autowired(required=False)</code> para campos opcionales.
<code>@Qualifier("name")</code>	<code>Annotated[T, Qualifier("name")]</code>	El <code>Annotated</code> de Python transporta el cualificador sin perder el tipo base.
Inyección de <code>Optional<T></code>	Parámetro <code>Optional[T]</code>	Se resuelve a <code>None</code> cuando no hay ningún bean registrado.
Inyección de <code>List<T></code>	Parámetro <code>list[T]</code>	Recopila todas las implementaciones registradas de <code>T</code> .
Inyección de <code>Map<String, T></code>	Parámetro <code>dict[str, T]</code>	<code>{nombre-bean: bean}</code> para cada bean con nombre de tipo <code>T</code> .
Inyección genérica <code>Repository<User></code>	Parámetro <code>Repository[User, int]</code>	El contenedor casa por los argumentos de tipo genérico y respeta <code>@primary</code> para resolver empates.
<code>ObjectFactory<T></code> / <code>Provider<T></code>	<code>Provider[T]</code>	Resolución diferida; cada <code>.get()</code> vuelve a resolver — seguro para beans <code>TRANSIENT</code> .

 PREFIERE LA INYECCIÓN POR CONSTRUCTOR

La inyección por constructor mantiene las dependencias visibles en la firma de la clase, evita errores por dependencias ausentes en el arranque en lugar de en tiempo de ejecución, y te permite escribir pruebas unitarias en Python puro sin contenedor: `svc = WalletService(repo=MockRepo(), events=MockEvents())`.

Condiciones y autoconfiguración

Spring Boot	PyFly	Notas
<code>@ConditionalOnProperty</code>	<code>@conditional_on_property</code>	Registra cuando una clave de configuración es igual a un valor concreto.
<code>@ConditionalOnClass</code>	<code>@conditional_on_class("module")</code>	Registra cuando un módulo de Python es importable.
<code>@ConditionalOnMissingBean</code>	<code>@conditional_on_missing_bean(T)</code>	Registra cuando aún no existe ningún bean de tipo <code>T</code> .
<code>@ConditionalOnBean</code>	<code>@conditional_on_bean(T)</code>	Registra solo si hay presente un bean de tipo <code>T</code> .
<code>@ConditionalOnSingleCandidate</code>	<code>@conditional_on_single_candidate(T)</code>	Exactamente un candidato, o uno marcado como <code>@primary</code> .
<code>@ConditionalOnWebApplication</code>	<code>@conditional_on_web_application()</code>	Pila web (Starlette/FastAPI) presente.

Spring Boot	PyFly	Notas
<code>@ConditionalOnResource</code>	<code>@conditional_on_resource(path)</code>	Existe una ruta del sistema de archivos.
<code>@ConditionalOnExpression</code>	<code>@conditional_on_expression("#{...}")</code>	Expresión tipo SpEL ligera — admite <code>\$</code> <code>{key:default}</code> + aritmética, comparación y booleanos. Analizada como AST, sin <code>eval</code> .

Hooks del ciclo de vida y ámbitos

Spring Boot	PyFly	Notas
<code>@PostConstruct</code>	<code>@post_construct</code>	Se invoca después de la inyección de dependencias; puede ser <code>async def</code> .
<code>@PreDestroy</code>	<code>@pre_destroy</code>	Se invoca en el apagado ordenado; puede ser <code>async def</code> .
Ámbito por defecto (singleton)	Por defecto (singleton)	Una instancia por aplicación.
<code>@SessionScope</code>	<code>@component(scope=Scope.SESSION)</code>	Una instancia por <code>HttpSession</code> .
SPI de <code>Scope</code> personalizado	<code>Container.register_scope(name, handler)</code>	Implementa el protocolo <code>ScopeHandler</code> .
<code>@RefreshScope</code> (Spring Cloud)	<code>@refresh_scope</code>	Se descarta y se reconstruye al recibir <code>POST /actuator/refresh</code> .
<code>ContextRefresher.refresh()</code>	<code>ContextRefresher.refresh()</code> (injectable)	Descarta los beans de ámbito refresh, reinicia <code>@config_properties</code> y devuelve las claves modificadas.
<code>ApplicationEventPublisher</code>	<code>ApplicationEventPublisher</code> (injectable)	<code>await publisher.publish(event)</code> .
<code>@EventListener</code>	<code>@app_event_listener</code>	Despacho por <code>isinstance</code> ; se permiten listeners síncronos.

Configuración y perfiles

Spring Boot	PyFly	Notas
<code>application.yml</code>	<code>pyfly.yaml</code>	Misma estructura jerárquica.
<code>application-{profile}.yml</code>	<code>pyfly-{profile}.yaml</code>	Superposiciones por perfil.
<code>spring.profiles.active=dev</code>	<code>PYFLY_PROFILES_ACTIVE=dev</code> (variable de entorno) o <code>pyfly.profiles.active: dev</code> en <code>pyfly.yaml</code>	La activación es idéntica en el orden de prioridad.
<code>@ConfigurationProperties(prefix=...)</code>	<code>@config_properties(prefix=...)</code> sobre un <code>@dataclass</code> (<code>from pyfly.core import config_properties</code>)	Respaldado por Pydantic: validado y congelado en el arranque.
<code>@Value("\${key}")</code>	<code>field: str = Value("\${key}")</code> (<code>from pyfly.core import Value</code>)	Lanza una excepción si falta la clave.
<code>@Value("\${key:default}")</code>	<code>field: str = Value("\${key:default}")</code>	Valor por defecto tras los dos puntos.
<code>@Value("#{expr}")</code> SpEL	<code>Value("#{...}")</code> SpEL ligero	Aritmética, comparación, booleanos, sustitución <code>\${...}</code> y mapeo <code>env</code> . Inyección por constructor: <code>Annotated[bool, Value("#{...}")]</code> .

Spring Boot	PyFly	Notas
<code>@Bean @Profile("dev")</code>	<code>@bean(profile="dev")</code>	Expresión de perfil (<code>&</code> <code>!</code> <code>()</code>) sobre cualquier <code>@bean</code> .
<code>@Profile("prod & cloud")</code> booleano	<code>profile="prod & cloud"</code>	Operadores de Spring Boot 2.4+; la antigua forma de OR con comas sigue funcionando.
<code>spring.application.name</code>	<code>pyfly.app.name</code>	Clave del nombre de la aplicación en <code>pyfly.yaml</code> .

Prioridad de las fuentes de propiedades (de menor → mayor):

1. `pyfly-defaults.yaml` (valores integrados del framework)
2. `pyfly.yaml` (valores por defecto de la aplicación)
3. `pyfly-{profile}.yaml` (superposiciones por perfil)
4. Variables de entorno (en tiempo de ejecución, máxima prioridad)

Esto es idéntico al orden de las fuentes de propiedades de Spring Boot.

Capa web

```
Imports: from pyfly.web import (Body, PathVar, QueryParam, Valid,
get_mapping, post_mapping, put_mapping, delete_mapping, patch_mapping,
request_mapping). Estereotipos: from pyfly.container import rest_controller,
service, repository, component, configuration. 404: from pyfly.kernel
import ResourceNotFoundException.
```

Spring Boot	PyFly	Notas
<code>@RestController</code>	<code>@rest_controller</code>	
<code>@RequestMapping("/path")</code>	<code>@request_mapping("/path")</code>	Prefijo a nivel de clase.
<code>@GetMapping</code>	<code>@get_mapping</code>	Todos los métodos manejadores son <code>async def</code> .
<code>@PostMapping</code>	<code>@post_mapping</code>	
<code>@PutMapping</code>	<code>@put_mapping</code>	
<code>@DeleteMapping</code>	<code>@delete_mapping</code>	
<code>@PatchMapping</code>	<code>@patch_mapping</code>	
<code>@PathVariable Long id</code>	Parámetro <code>id:</code> <code>PathVar[int]</code>	Anotación de tipo obligatoria; casa por nombre.
<code>@RequestParam(defaultValue="0") int page</code>	<code>page:</code> <code>QueryParam[int] = 0</code>	El valor por defecto de Python sustituye a <code>defaultValue</code> .
<code>@RequestBody @Valid T body</code>	<code>body:</code> <code>Valid[Body[T]]</code>	Deserialización Pydantic + validación combinadas.
<code>@RequestHeader("X-Token") String t</code>	<code>t: Header[str]</code>	
<code>@ResponseStatus(HttpStatus.CREATED)</code>	<code>@post_mapping("/", status_code=201)</code>	Código de estado en el decorador del mapeo.
<code>@ControllerAdvice + @ExceptionHandler</code>	<code>@exception_handler</code> o jerarquía de excepciones integrada	<code>ResourceNotFoundException</code> → 404, <code>ValidationException</code> → 422, etc.

Spring Boot	PyFly	Notas
<code>spring.mvc.problemdetails.enabled</code>	<code>pyfly.web.problem-details.enabled: true</code>	Respuestas RFC 7807 <code>application/problem+json</code> .

JSON y negociación de contenido

Spring Boot	PyFly	Notas
<code>spring.jackson.property-naming-strategy</code>	<code>pyfly.web.json.property-naming-strategy</code>	<code>as-is</code> (por defecto) o <code>camelCase</code> .
<code>spring.jackson.default-property-inclusion: non_null</code>	<code>pyfly.web.json.exclude-none: true</code>	
<code>ObjectMapper</code> de Jackson	<code>PyFlyJsonSerializer</code>	Frontera central de serialización.
<code>Module</code> de Jackson / serializador personalizado	<code>JsonSerializers.register(Type, encode=fn)</code>	Codificadores para tipos no Pydantic.
<code>@JsonNaming(CamelCase...)</code>	Clase base <code>CamelModel</code>	Modelo camelCase opcional — <code>order_id</code> se serializa como <code>orderId</code> .
<code>HttpMessageConverter</code>	<code>MessageConverter</code> / <code>MessageConverterRegistry</code>	Integrados: JSON (primero) y luego XML; añade conversores personalizados con <code>registry.add(...)</code> .

Acceso a datos

Imports: `from pyfly.data.relational.sqlalchemy import (Repository, BaseEntity, Base, Specification, transactional, Propagation, Isolation)`
`from pyfly.data import Page, Pageable, Sort` `from pyfly.data.query import query` `from pyfly.container import repository`

Repositorios

Spring Boot	PyFly	Notas
<code>JpaRepository<E, ID> / CrudRepository</code>	<code>Repository[E, ID]</code>	<code>from pyfly.data.relational.sqla import Repository</code> ; decora con <code>@repository</code> . Crea una subclase para parámetros de tipo concretos. No inyecta ninguna sesión: cada instancia se une a la unidad de trabajo o tarea en curso, o confirma en la unidad breve propia.
<code>@Repository</code> <code>interface WalletRepo</code> <code>extends JpaRepository<...></code>	<code>@repository class WalletRepository(Repository[WalletEntity, str])</code>	Clase, no interfaz; el cuerpo de la clase contiene esbozos de consultas derivadas y métodos personalizados.
<code>findByOwnerId(String id)</code>	<code>async def find_by_owner_id(self, owner_id: str) → list[WalletEntity]: ...</code>	El cuerpo esbozado <code>...</code> desencadena la compilación por <code>RepositoryBeanPostProcessor</code> en el arranque. Prefijos: <code>find_by_</code> , <code>count_by_</code> , <code>exists_by_</code> , <code>delete_by_</code> .
<code>@Query("SELECT u FROM User u WHERE ...")</code>	<code>@query("SELECT u FROM User u WHERE ...")</code> (tipo JPQL) o <code>@query("SELECT ...", native=True)</code> (SQL en bruto)	<code>from pyfly.data.query import query</code> . Los parámetros usan la sintaxis <code>:name</code> .
<code>Specification<T></code>	<code>Specification(lambda root, q: q.where(root.status == "ACTIVE"))</code>	Compón con <code>&</code> / <code> </code> / <code>~</code> ; ej: <code>repo.find_all_by_spec(spec1 & spec2)</code> o <code>repo.find_all_by_spec_pageable(spec1 spec2, pageable)</code> .

Entidades

Spring Boot	PyFly	Notas
<code>@Entity</code> <code>class Order</code>	<code>class Order(Base)</code>	from <code>pyfly.data.relational.sqlalchemy</code> <code>import Base</code> . Registra la tabla en <code>Base.metadata</code> .
<code>@Entity</code> + PK UUID subrogada + columnas de auditoría	<code>class Order(BaseEntity)</code>	from <code>pyfly.data.relational.sqlalchemy</code> <code>import BaseEntity</code> . Hereda <code>id: UUID</code> , <code>created_at</code> , <code>updated_at</code> , <code>created_by</code> , <code>updated_by</code> — todas mapeadas como <code>Mapped / mapped_column</code> de SQLAlchemy 2.0.
<code>@Column</code> / <code>@Id</code>	<code>id: Mapped[str] =</code> <code>mapped_column(String(64),</code> <code>primary_key=True)</code>	Columnas tipadas de SQLAlchemy 2.0. <code>Base</code> (sin auditoría) deja que la entidad sea dueña de su propio tipo de PK, como hace <code>WalletEntity</code> de Lumen con un id de tipo <code>str</code> .
<code>@SoftDelete</code> (Hibernate 6)	<code>SoftDeleteMixin</code> + <code>SoftDeleteRepository</code>	from <code>pyfly.data.relational.sqlalchemy</code> <code>import SoftDeleteMixin</code> . Añade <code>deleted_at</code> ; el repositorio lo filtra automáticamente.
Bloqueo optimista <code>@Version</code>	<code>VersionedMixin</code>	Añade una columna <code>version: int</code> ; un conflicto lanza <code>OptimisticLockingFailureException</code> (HTTP 409).

Paginación y ordenación

Spring Boot	PyFly	Notas
<code>Pageable /</code> <code>PageRequest.of(page,</code> <code>size,</code> <code>Sort.by(...).descending())</code>	<code>Pageable.of(page, size,</code> <code>Sort.by("created_at").descending())</code>	<code>from pyfly.data</code> <code>import Pageable,</code> <code>Sort.</code> <code>Sort.by(*fields)</code> devuelve orden ascendente; <code>.descending()</code> lo invierte todo.
<code>Page<T></code>	<code>Page[T]</code>	<code>from pyfly.data</code> <code>import Page.</code> Atributos: <code>.items</code> , <code>.total</code> , <code>.p</code> transforma los elementos preservando los metadatos.
<code>page.getContent() /</code> <code>page.getTotalElements()</code>	<code>page.items / page.total</code>	Nomenclatura de Python; <code>.total_pages</code> se deriva como <code>ceil(total /</code> <code>size)</code> .
<code>repo.findAll(pageable)</code>	<code>await repo.find_all(pageable)</code>	Devuelve <code>Page[T]</code> .
<code>repo.findAll(spec,</code> <code>pageable)</code>	<code>await</code> <code>repo.find_all_by_spec_paged(spec,</code> <code>pageable)</code>	Aplica WHERE, ORDER BY y LIMIT/OFFSET en una sola llamada.

Transacciones y save

Spring Boot	PyFly	Notas
<code>@Transactional</code>	<code>@transactional()</code>	<code>from py</code> <code>import t</code> trabajo a en el orig <code>self._s</code> defecto); ella. Cua <code>rollbac</code> reglas; un hace que <code>Unexpec</code>
<code>@Transactional(propagation = REQUIRES_NEW)</code>	<code>@transactional(propagation=Propagation.REQUIRES_NEW)</code>	Enum Pr <code>REQUIRE</code> guardado <code>NEVER ,</code>
<code>@Transactional(isolation = READ_COMMITTED)</code>	<code>@transactional(isolation=Isolation.READ_COMMITTED)</code>	El enum niveles d admite fa con <code>Tran</code>
<code>@Transactional(readOnly = true)</code>	<code>@transactional(read_only=True)</code>	Una unid la réplica configura
<code>TransactionTemplate</code>	<code>TransactionTemplate</code>	<code>from py</code> <code>Transact</code> <code>template</code> <code>template</code>

Spring Boot	PyFly	Notas
<code>repo.save(entity)</code>	<code>await repo.save(entity)</code>	El <code>save</code> fusiona o gestiona y la unidad propia un
<code>session.merge(entity)</code> (upsert)	<code>await repo.upsert(entity)</code> (extiéndelo) o <code>session.merge(entity)</code> + flush	No hay una entidad <code>WalletR</code> dejar clar gestiona función d
<code>AbstractRoutingDataSource</code>	<code>@transactional(read_only=True)</code> o <code>RoutingSessionFactory</code>	Una unidad factory fuerzan u
<code>@Lock(PESSIMISTIC_WRITE)</code>	<code>find_by_id(id, lock=LockMode.PESSIMISTIC_WRITE)</code>	<code>SELECT</code> <code>@transa</code> toma su l IMMEDIAT
Varios beans <code>DataSource</code>	<code>NamedDataSources</code>	Configura <code>pyfly.d</code> inyecta <code>.get("<</code>

Proyecciones y mapeador

Spring Boot	PyFly	Notas
Proyección por interfaz <code>interface</code> <code>OrderSummary</code> <code>{ ... }</code>	<code>@projection class</code> <code>OrderSummary: id: str;</code> <code>status: str</code>	<code>from pyfly.data.projection import projection</code> . Un dataclass concreto (no un <code>Protocol</code>), no un proxy de la JDK; se registra con <code>mapper.register_projection(src, proj)</code> .
<code>repo.findAll(cls, Projection.class)</code>	<code>mapper.project(entity, OrderSummary)</code>	<code>from pyfly.data.mapper import Mapper</code> .
<code>@Mapper</code> de <code>MapStruct</code>	<code>Mapper</code> + decorador <code>@mapping</code>	Reflexión en tiempo de ejecución; sin generación de código. <code>mapper.map(obj, TargetDTO)</code> , <code>mapper.map_list(...)</code> .

Migraciones

Spring Boot	PyFly	Notas
Migraciones de Flyway	<code>pyfly.data.migrations.*</code>	Mismo concepto: scripts SQL versionados que se ejecutan en el arranque.

Caché

Spring Boot	PyFly	Notas
<code>@Cacheable(value="cache", key="#id")</code>	<code>@cacheable(backend=cache, key="item:{id}")</code>	Sintaxis de plantilla <code>{param}</code> en la clave.
<code>@Cacheable(condition=..., unless=...)</code>	<code>@cacheable(condition=..., unless=...)</code>	<code>condition</code> omite según los argumentos; <code>unless</code> evita almacenar según el resultado.
<code>@CacheEvict</code>	<code>@cache_evict(backend=cache, key="item:{id}")</code>	
<code>@CachePut</code>	<code>@cache_put(backend=cache, key="item:{id}")</code>	Actualiza la caché tras la mutación.
<code>CacheManager</code> (autoconfigurado)	<code>InMemoryCache</code> / <code>RedisCacheAdapter</code> (autoconfigurados)	Se selecciona Redis cuando el extra <code>redis</code> está instalado; recae en memoria en caso contrario.

Mensajería y eventos

Spring Boot	PyFly
<code>@KafkaListener(topics=..., groupId=...)</code>	<code>@message_listener(topic=..., group=...)</code>
<code>KafkaTemplate.send(topic, event)</code>	<code>await publisher.publish(dest, event_type, payload)</code>
<code>@RetryableTopic</code> / DLT	<code>@message_listener(retries=3, retry_delay=1.0, dead_letter_topic="...")</code>
<code>ApplicationEvent</code>	<code>EventEnvelope</code>
<code>@EventListener</code>	<code>@event_listener(event_types=["TypeName"])</code>
<code>ApplicationEventPublisher</code>	<code>ApplicationEventPublisher</code> (injectable)

Spring Boot	PyFly
<code>@TransactionalEventListener(phase = AFTER_COMMIT)</code>	<code>@app_event_listener(phase=TransactionPhase.AFTER_COMMIT)</code>
Outbox transaccional (Spring Modulith)	<code>pyfly.eda.outbox.enabled: true</code>

💡 MENSAJERÍA FRENTE A EDA

PyFly separa la **mensajería por broker** (`pyfly.messaging` — transporte Kafka/ RabbitMQ) de los **eventos de dominio** (`pyfly.eda` — `EventEnvelope` + `EventBus`).
Empieza con `InMemoryEventBus` dentro de un monolito; cambia a un adaptador de Kafka más adelante modificando una sola clave de configuración, no tus manejadores.

Seguridad

Spring Boot	PyFly	Notas
<code>SecurityAutoConfiguration</code>	<code>JwtAutoConfiguration</code> + <code>PasswordEncoderAutoConfiguration</code>	Dividido por depend
Cadena de filtros JWT	<code>JwtAutoConfiguration</code> autocableado	Actívalo con <code>pyfly.security.jw</code>
<code>@PreAuthorize("hasRole('ADMIN')")</code>	<code>@pre_authorize("hasRole('ADMIN')")</code>	Mismo subconjunto <code>hasRole</code> , <code>hasAnyRo</code> <code>hasAuthority</code> , <code>isA</code> <code>permitAll</code> , <code>denyAT</code> <code>or</code> / <code>not</code> . Recorrido <code>eval</code> .
<code>@PostAuthorize("returnObject.owner == principal")</code>	<code>@post_authorize("returnObject.owner == principal")</code>	<code>returnObject</code> se v retorno del método.
Bean <code>RoleHierarchy</code>	<code>RoleHierarchy.from_string("ADMIN > USER")</code> + <code>set_role_hierarchy(...)</code>	<code>expand(roles)</code> es <code>hasRole</code> / <code>hasAnyRo</code>
<code>ClientRegistration</code> (OAuth2 con código de autorización)	<code>ClientRegistration(...)</code>	PKCE: añade <code>use_L</code> genera <code>code_verif</code> <code>code_challenge</code> (S automáticamente.
<code>maximumSessions</code> / <code>SessionRegistry</code>	<code>SessionConcurrencyController</code> + <code>SessionRegistry</code>	Limita las sesiones principal (<code>evict-oL</code> <code>reject-new</code>). Activ <code>pyfly.session.com</code> <code>true</code> .

Programación

Spring Boot	PyFly	Notas
<code>@Scheduled(fixedRate = 5000)</code>	<code>@scheduled(fixed_rate=5.0)</code>	Spring usa milisegundos; PyFly usa segundos .
<code>@Scheduled(fixedDelay = 1000)</code>	<code>@scheduled(fixed_delay=1.0)</code>	
<code>@Scheduled(cron = "0 0 2 * * ?")</code>	<code>@scheduled(cron="0 2 * * *")</code>	Spring: 6 campos (segundos primero). PyFly: 5 campos estándar; también acepta la forma de 6 campos de Spring y <code>?</code> .
<code>@Scheduled(cron=..., zone=...)</code>	<code>@scheduled(cron=..., zone="America/New_York")</code>	Zona horaria IANA; se ignora para <code>fixed_rate</code> / <code>fixed_delay</code> .
ShedLock / <code>@SchedulerLock</code>	<code>@scheduled(lock=True, lock_ttl=30)</code> + bean <code>DistributedLock</code>	Omite un tick cuando el bloqueo está retenido en otro lugar. Por defecto usa el <code>LocalLock</code> en proceso; registra un <code>DistributedLock</code> de Redis para un disparo único entre procesos.
<code>@EnableScheduling</code>	<code>SchedulingAutoConfiguration</code>	Se activa automáticamente cuando <code>croniter</code> está instalado. No hace falta <code>@Enable...</code> explícito.

Resiliencia

```
from pyfly.resilience import retry, CircuitBreaker, circuit_breaker,
RateLimiter, rate_limiter, Bulkhead, bulkhead, time_limiter, fallback
```

Spring (Resilience4j)	PyFly	Notas
@Retry	@retry(max_attempts=3, *, delay=0.1, backoff=2.0, jitter=0.1, exceptions=(IOError,))	delay / backoff / jitter son solo por palabra clave. jitter es una fracción float en [0,1] .
@CircuitBreaker	breaker = CircuitBreaker(...); @circuit_breaker(breaker)	Pasa una <i>instancia</i> de CircuitBreaker . Basado en conteo (failure_threshold) o en tasa (failure_rate_threshold + window_size).
@RateLimiter	limiter = RateLimiter(max_tokens=100, refill_rate=100/60); @rate_limiter(limiter)	Cubo de tokens; pasa una <i>instancia</i> .
@Bulkhead	bh = Bulkhead(max_concurrent=10); @bulkhead(bh)	Límite de concurrencia; pasa una <i>instancia</i> .
@TimeLimiter	@time_limiter(timeout=timedelta(seconds=2))	Lanza asyncio.TimeoutError al superarse.
fallbackMethod	@fallback(fallback_method=fn) o @fallback(fallback_value=v)	Valor de reserva estático o invocable.

AOP

Spring Boot	PyFly	Notas
<code>@Aspect + @Component</code>	<code>@aspect + @component</code>	
<code>@Before("execution(...)")</code>	<code>@before("execution(...)")</code>	
<code>@After</code>	<code>@after</code>	Se ejecuta siempre (como <code>finally</code>).
<code>@Around</code>	<code>@around</code>	Llama a <code>await</code> <code>join_point.proceed()</code> para continuar.
<code>@AfterReturning</code>	<code>@after_returning</code>	
<code>@AfterThrowing</code>	<code>@after_throwing</code>	
<code>@EnableAspectJAutoProxy</code>	<code>AopAutoConfiguration</code>	Siempre activo; no hace falta optar por él.

DSL de pointcuts: `execution(* pkg.services.*.*(..))` para patrones de métodos; `annotation(timed)` para casar por decorador.

Observabilidad y Actuator

Spring Boot	PyFly
<code>management.endpoints.web.exposure.include</code>	<code>pyfly.management.endpoints.web.exposure.include</code>
<code>/actuator/health</code>	<code>/actuator/health</code>
<code>/actuator/info</code>	<code>/actuator/info</code>
<code>/actuator/beans</code>	<code>/actuator/beans</code>
<code>/actuator/env</code>	<code>/actuator/env</code>
<code>POST /actuator/refresh</code> (Spring Cloud)	<code>POST /actuator/refresh</code>
<code>@Timed</code> de Micrometer	<code>@timed("metric_name")</code>
<code>@Counted</code> de Micrometer	<code>@counted("metric_name")</code>
<code>MetricsRegistry</code> de Prometheus	<code>MetricsRegistry</code> (backend de Prometheus)

Spring Boot	PyFly
Sleuth / Micrometer Tracing (W3C)	<code>TracingFilter</code> (entrante) + <code>HttpxClientAdapter</code> (saliente)

CQRS y sagas

```
from pyfly.cqrs import (Command, CommandHandler, DefaultCommandBus, Query,
QueryHandler, DefaultQueryBus, command_handler, query_handler)
from pyfly.transactional.saga.annotations import (saga, saga_step, Input,
FromStep)
```

Spring Boot / Axon	PyFly	Notas
<code>@CommandHandler</code>	<code>@command_handler</code> + <code>@service</code> apilados	Ambos decoradores son obligatorios; <code>@service</code> registra el bean. Sobrescribe <code>do_handle(self, cmd)</code> .
<code>@QueryHandler</code>	<code>@query_handler</code> + <code>@service</code> apilados	Misma regla. Sobrescribe <code>do_handle(self, qry)</code> . Método del bus: <code>.query(...)</code> .
Inyección del bus en el controlador	<code>commands: DefaultCommandBus,</code> <code>queries: DefaultQueryBus</code>	Inyecta las clases concretas, no el protocolo. <code>commands.send(cmd)</code> , <code>queries.query(qry)</code> .
<code>@Repository</code> + puerto	<code>@repository</code> sobre una clase que hereda del puerto <code>Protocol</code> con <code>@runtime_checkable</code>	El puerto es un <code>Protocol</code> ; la clase adaptadora lo hereda.
<code>@EventHandler</code> (event sourcing)	<code>@event_handler</code>	Procede del <code>EventStore</code> .
<code>@Saga</code>	<code>@saga(name="...",</code> <code>layer_concurrency=N)</code> + <code>@service</code>	Ambos decoradores son obligatorios para la inyección de dependencias + el registro en el motor.
<code>@SagaStep(id=...,</code> <code>compensate=...)</code>	<code>@saga_step(id="...",</code> <code>compensate="method_name")</code>	
<code>@Input</code>	<code>Annotated[T, Input()]</code>	El marcador es una instancia : <code>Input()</code> . Inyecta la carga inicial de la saga.

Spring Boot / Axon	PyFly	Notas
<code>@FromStep("id")</code>	<code>Annotated[T, FromStep("id")]</code>	El marcador es una instancia : <code>FromStep("step-id")</code> . Inyecta el resultado de un paso anterior.
<code>@Tcc</code>	<code>@tcc(name="...")</code>	Clase de transacción TCC (Try-Confirm-Cancel).
<code>@TccParticipant</code>	<code>@tcc_participant(id="...", order=N)</code>	
<code>@TryMethod</code> / <code>@ConfirmMethod</code> / <code>@CancelMethod</code>	<code>@try_method</code> / <code>@confirm_method</code> / <code>@cancel_method</code>	Métodos de las tres fases de TCC.
<code>@FromTry</code>	<code>Annotated[T, FromTry]</code>	Inyecta el resultado de la fase try en confirm/cancel.

Event sourcing (gestión de estado mediante secuencias de eventos): `from pyfly.eventsourcing import AggregateRoot, EventSourcedRepository` . La raíz de agregado usa `self.when(EventType, handler_fn)` para registrar manejadores de aplicación. Datos: `from pyfly.data.relational.sqlalchemy import Base` (requiere `pyfly[data-relational]`).

Pruebas de integración

Spring Boot	PyFly	Notas
<code>@SpringBootTest</code>	<code>service_slice(*beans) / slice_context(...)</code>	Contexto mínimo ya iniciado; <code>overrides</code> acepta una clase o una instancia ya construida.
<code>@WebMvcTest</code>	<code>web_slice(*controllers, overrides=...) → (context, client)</code>	Inicia un contexto mínimo + <code>PyFlyTestClient</code> .
<code>@DataJpaTest</code>	<code>@DataTest(beans=[...])</code> o <code>data_slice(*beans, rollback=True) → context</code>	Slice de la capa de datos; todas las unidades de trabajo de la prueba se revierten.
<code>@Testcontainers + @Container</code>	<code>with postgres_container() as pg:</code>	El context manager de Python gestiona el ciclo de vida.
<code>@ServiceConnection</code>	<code>pyfly_config(pg) / pyfly_config_for(pg)</code>	Asocia los detalles de conexión del contenedor a las claves de configuración de PyFly.
<code>@DynamicPropertySource</code>	<code>pyfly_config(*containers, base=...)</code>	Un <code>Config</code> en una sola llamada para varios contenedores.
<code>PostgreSQLContainer</code>	<code>postgres_container()</code>	La URL se reescribe automáticamente a <code>asyncpg</code> .

Spring Boot	PyFly	Notas
<code>MySQLContainer</code>	<code>mysql_container() / mariadb_container()</code>	La URL se reescribe a <code>mysql+asyncmy:// / mariadb+asyncmy:// (pyfly[mysql])</code> .
<code>MongoDBContainer</code> (conjunto de réplicas)	<code>mongodb_replica_set_container()</code>	Conjunto de réplicas <code>rs0</code> de un solo nodo que ejecuta transacciones.
<code>GenericContainer</code> (Redis)	<code>redis_container()</code>	URLs de caché + sesión cableadas.
<code>KafkaContainer</code>	<code>kafka_container()</code>	
<code>@requires_docker</code>	<code>@requires_docker</code>	Omite la prueba limpiamente cuando el demonio de Docker no está presente.

Instálalo con: `pip install 'pyfly[testcontainers]'`

Servidor embebido

Spring Boot	PyFly	Notas
<code>server.port</code>	<code>pyfly.server.port</code>	Puerto HTTP de escucha de la aplicación (por defecto 8080).
<code>server.address</code>	<code>pyfly.server.host</code>	Dirección de enlace de la aplicación.
<code>management.server.port</code>	<code>pyfly.management.server.port</code>	Puerto de gestión independiente (actuador + panel de administración); por defecto 9090.
Tomcat (por defecto)	Granian (por defecto)	Runtime HTTP en Rust/tokio; máxima prioridad.
Jetty (alternativa de reserva)	Uvicorn (alternativa de reserva)	Reserva ASGI estándar del ecosistema.
Undertow (alternativa)	Hypercorn (alternativa)	Soporte de protocolos avanzados (HTTP/3).
<code>server.tomcat.*</code>	<code>pyfly.server.granian.*</code>	Ajuste específico del servidor.
Interfaz <code>WebServer</code>	Protocolo <code>ApplicationServerPort</code>	Contrato del servidor ASGI embebido.
<code>EventLoopGroup</code> (Netty)	Protocolo <code>EventLoopPort</code>	Contrato del runtime de E/S.
<code>server.type: auto</code>	<code>pyfly.server.type: auto</code> (por defecto)	Cascada Granian → Uvicorn → Hypercorn.

 CASCADA DE AUTOCONFIGURACIÓN

La autoconfiguración de PyFly usa el mismo patrón de beans condicionales que Spring Boot: si Granian está instalado, gana `GranianServerAdapter` ; si no, se prueba Uvicorn a continuación; luego Hypercorn. Anúlalo en cualquier punto aportando tu propio bean `ApplicationServerPort` .

HTML nativo y recursos estáticos

Spring / Django	PyFly
Spring <code>ModelAndView</code>	<code>ModelAndView(template, model)</code>
Django <code>reverse</code> / template URL tag	<code>reverse(request, name, **params)</code> / <code>reverse(name, **params)</code>
Django static helper / template tag	<code>static_url(request, path)</code> / <code>static_url(path)</code>
Django <code>ModelAdmin</code>	Explicit <code>ModelAdmin</code> over existing SQLAlchemy/Beanie models

El Apéndice E explica configuración, seguridad y diferencias de persistencia.

APÉNDICE B

MongoDB y datos documentales

La capa de datos documentales de PyFly envuelve MongoDB mediante **Beanie ODM** y la API asíncrona de PyMongo (`AsyncMongoClient`). La API replica deliberadamente la del adaptador relacional: la misma clase base `MongoRepository[T, ID]` , la misma convención de nombres para consultas derivadas, el mismo vocabulario `Page / Pageable / Sort` , de modo que cambiar entre almacenamiento relacional y documental solo afecta a la definición de la clase de documento y a la clase base del repositorio, no a la capa de servicio. También se apoya en el mismo modelo de unidad de trabajo: `@transactional` , las reglas de reversión y las propagaciones funcionan igual (salvo `NESTED` : MongoDB no tiene puntos de guardado).

Todos los tipos concretos viven en `pyfly.data.document.mongodb` . Los tipos compartidos (`Page` , `Pageable` , `Sort`) provienen de `pyfly.data` .

Instalación y configuración

Instala el extra:

```
terminal
```

```
uv add "pyfly[data-document]"
```

Listado B.1 — Instalar el extra data-document

Habilita el adaptador en `pyfly.yaml` :

`pyfly.yaml`

```
pyfly:
  data:
    document:
      enabled: true
      uri: "mongodb://localhost:27017"
      database: "myapp"
      min_pool_size: 5
      max_pool_size: 50
```

Listado B.2 — Configuración mínima de MongoDB

Referencia de configuración

Clave de <code>pyfly.yaml</code>	Tipo	Valor por defecto	Descripción
<code>pyfly.data.document.enabled</code>	bool	<code>false</code>	Habilita el adaptador de MongoDB
<code>pyfly.data.document.uri</code>	str	<code>mongodb://localhost:27017</code>	URI de conexión
<code>pyfly.data.document.database</code>	str	<code>pyfly</code>	Nombre de la base de datos
<code>pyfly.data.document.min_pool_size</code>	int	<code>0</code>	Mínimo del pool de conexiones (<code>minPoolSize</code>)
<code>pyfly.data.document.max_pool_size</code>	int	<code>100</code>	Máximo del pool de conexiones (<code>maxPoolSize</code>)
<code>pyfly.data.document.datasources</code>	str	<code>document</code>	Nombre del origen de datos de las unidades de trabajo documentales
<code>pyfly.data.document.tz_aware</code>	bool	<code>true</code>	Las fechas vuelven como valores UTC con zona horaria

Clave de <code>pyfly.yaml</code>	Tipo	Valor por defecto	Descripción
<code>pyfly.data.document.transaction.default</code>	bool	sin fijar	Si el origen documental es el de por defecto de <code>@transactional</code> (por defecto, cuando la capa relacional está desactivada)

Cada clave tiene una variable de entorno equivalente: sustituye los puntos por guiones bajos y pásala a mayúsculas; por ejemplo, `PYFLY_DATA_DOCUMENT_URI`. Para MongoDB Atlas o un conjunto de réplicas (replica set):

pyfly.yaml

```
# Atlas
pyfly:
  data:
    document:
      enabled: true
      uri: >-
        mongodb+srv://user:secret@cluster.mongodb.net/
        ?retryWrites=true&w=majority
      database: production_db

# Replica set (required for transactions)
# pyfly:
#   data:
#     document:
#       uri: "mongodb://m1:27017,m2:27017,m3:27017/?replicaSet=rs0"
```

Listado B.3 — URIs de Atlas y de conjunto de réplicas

BaseDocument

`BaseDocument` extiende `beanie.Document` con un rastro de auditoría. Toda clase de documento en una aplicación PyFly debería heredar de ella.

Campo	Tipo	Valor por defecto	Descripción
<code>id</code>	<code>PydanticObjectId</code>	Autogenerado	Clave primaria del documento (ObjectId)
<code>created_at</code>	<code>datetime</code>	<code>datetime.now(UTC)</code>	Marca de tiempo de inserción
<code>updated_at</code>	<code>datetime</code>	<code>datetime.now(UTC)</code>	Marca de tiempo de la última actualización
<code>created_by</code>	<code>str</code> <code>None</code>	<code>None</code>	Identificador del creador
<code>updated_by</code>	<code>str</code> <code>None</code>	<code>None</code>	Identificador de quien actualizó por última vez

`use_state_management = True` se establece en la clase base `Settings`, lo que habilita el seguimiento de cambios de Beanie para que `save_changes()` produzca actualizaciones parciales eficientes.

Una clase de documento típica:

catalog/product_document.py

```

from pydantic import BaseModel, Field
from beanie import Indexed, PydanticObjectId
from pyfly.data.document.mongodb import BaseDocument

class Dimensions(BaseModel):
    width_cm: float
    height_cm: float
    depth_cm: float

class ProductDocument(BaseDocument):
    name: str
    sku: Indexed(str, unique=True)
    description: str = ""
    price: float = Field(gt=0)
    category: Indexed(str)
    tags: list[str] = Field(default_factory=list)
    dimensions: Dimensions | None = None
    active: bool = True

class Settings:
    name = "products"

```

Listado B.4 — ProductDocument con índice y modelo anidado

El atributo `Settings.name` establece el nombre de la colección de MongoDB. Si lo omites, Beanie deriva el nombre a partir del nombre de la clase, lo cual rara vez es lo que deseas.

Para índices compuestos o descendentes usa `Settings.indexes`:

catalog/product_document.py

```

from pymongo import IndexModel, ASCENDING, DESCENDING
from pyfly.data.document.mongodb import BaseDocument

class OrderDocument(BaseDocument):
    customer_id: str
    status: str
    total: float
    region: str

    class Settings:
        name = "orders"
        indexes = [
            IndexModel(
                [("customer_id", ASCENDING), ("status", ASCENDING)],
                name="idx_customer_status",
            ),
            IndexModel(
                [("region", ASCENDING), ("total", DESCENDING)],
                name="idx_region_total",
            ),
        ]

```

Listado B.5 — Índice compuesto mediante `Settings.indexes`

Correspondencia entre Spring Data y MongoRepository

La siguiente tabla muestra cómo los conceptos de Spring Data MongoDB se corresponden con la capa documental de PyFly. La superficie es intencionadamente idéntica a la de `Repository[T, ID]` del adaptador relacional, de modo que los mismos patrones de la capa de servicio se aplican a ambos.

Spring Data MongoDB	PyFly	Notas
<code>MongoRepository<E, ID></code>	<code>MongoRepository[E, ID]</code>	<code>from pyfly.data.document.mongodb import MongoRepository ;</code> decora con <code>@repository</code> .
<code>@Document class Product + @Id</code>	<code>class ProductDocument(BaseDocument)</code>	<code>from pyfly.data.document.mongodb import BaseDocument</code> . Hereda <code>id</code> (<code>PydanticObjectId</code> de Beanie), <code>created_at</code> , <code>updated_at</code> , <code>created_by</code> , <code>updated_by</code> . El nombre de la colección se fija en <code>class Settings: name = "products"</code> .
<code>findByCategory(String c)</code>	<code>async def find_by_category(self, category: str) → List[ProductDocument]: ...</code>	El cuerpo vacío <code>...</code> lo compila <code>MongoRepositoryBeanPostProcessor</code> al arrancar. Mismos prefijos que en el relacional: <code>find_by_</code> , <code>count_by_</code> , <code>exists_by_</code> , <code>delete_by_</code> .
<code>@Query("{ 'status': ? @ }")</code>	<code>@query('{"status": ":status"}')</code>	<code>from pyfly.data.query import query</code> . Filtro JSON o pipeline de agregación; sustitución de <code>:param</code> .
<code>MongoSpecification</code>	<code>MongoSpecification(lambda root, q: {"active": True})</code>	<code>from pyfly.data.document.mongodb import MongoSpecification</code> . Compón con <code>&</code> / <code>\ </code> / <code>~</code> ; ejecuta mediante <code>find_all_by_spec(spec)</code> o <code>find_all_by_spec_paged(spec, pageable)</code> .
<code>PageRequest.of(page, size, Sort.by(...))</code>	<code>Pageable.of(page, size, Sort.by("name").descending())</code>	<code>from pyfly.data import Pageable, Sort</code> , idéntico al adaptador relacional.

Spring Data MongoDB	PyFly	Notas
<code>Page<T></code>	<code>Page[T]</code>	<code>.items</code> , <code>.total</code> , <code>.page</code> , <code>.size</code> , <code>.total</code> igual que en el relacional.

MongoRepository[T, ID]

Crea una subclase de `MongoRepository[T, ID]` y anótala con `@repository`. El framework extrae el tipo de documento y el tipo de ID a partir de los parámetros genéricos en el momento de definir la clase, mediante `__init_subclass__`. No se requiere ningún `__init__`.

catalog/product_repository.py

```

from beanie import PydanticObjectId
from pyfly.container import repository
from pyfly.data.document.mongodb import MongoRepository

from catalog.product_document import ProductDocument

@repository
class ProductRepository(MongoRepository[ProductDocument, PydanticObjectId]):

    # --- derived query method stubs (compiled at startup) ---

    async def find_by_category(
        self, category: str
    ) → list[ProductDocument]: ...

    async def find_by_active_and_category(
        self, active: bool, category: str
    ) → list[ProductDocument]: ...

    async def find_by_price_greater_than_order_by_price_desc(
        self, min_price: float
    ) → list[ProductDocument]: ...

    async def find_by_name_containing(
        self, fragment: str
    ) → list[ProductDocument]: ...

    async def count_by_category(self, category: str) → int: ...

    async def exists_by_sku(self, sku: str) → bool: ...

    async def delete_by_active(self, active: bool) → int: ...

```

Listado B.6 — ProductRepository: CRUD + consultas derivadas

Métodos CRUD integrados

Método	Tipo de retorno	Descripción
<code>save(entity)</code>	<code>T</code>	Inserta un documento nuevo (<code>insert_one</code>) o actualiza uno guardado, en un solo comando
<code>find_by_id(id)</code>	<code>T None</code>	Busca por clave primaria
<code>find_all(**filters)</code>	<code>list[T]</code>	Busca todos; los argumentos por palabra clave se convierten en filtros de igualdad
<code>find_all(sort)</code>	<code>list[T]</code>	Recupera todos los documentos, ordenados por un <code>Sort</code>
<code>find_all(pageable)</code>	<code>Page[T]</code>	Consulta paginada: cuenta el total, aplica el orden del Pageable, recorta con skip/limit y devuelve <code>Page[T]</code>
<code>stream_all(sort)</code>	<code>AsyncIterator[T]</code>	Transmite todos los documentos (el análogo de <code>Flux<T></code>); admite un <code>Sort</code> y filtros de igualdad opcionales
<code>delete(entity)</code>	<code>None</code>	Elimina una instancia de documento ya cargada
<code>delete_by_id(id)</code>	<code>None</code>	Elimina por clave primaria; no hace nada si no se encuentra

Método	Tipo de retorno	Descripción
<code>count()</code>	<code>int</code>	Cuenta todos los documentos de la colección
<code>exists_by_id(id)</code>	<code>bool</code>	True si existe un documento con este ID
<code>save_all(entities)</code>	<code>list[T]</code>	Una escritura masiva ordenada para documentos nuevos y guardados (atómica en un conjunto de réplicas)
<code>find_all_by_id(ids)</code>	<code>list[T]</code>	Busca todos cuyos ID estén en una lista
<code>delete_all_by_id(ids)</code>	<code>None</code>	Elimina todos cuyos ID estén en una lista
<code>delete_all(entities=None)</code>	<code>None</code>	Elimina los documentos indicados; sin argumentos, vacía la colección entera
<code>delete_all_in_batch(entities=None)</code>	<code>None</code>	Borrado masivo que se salta las acciones de evento de borrado
<code>find_all_by_spec(spec)</code>	<code>list[T]</code>	Busca los que coincidan con una <code>MongoSpecification</code>

Método	Tipo de retorno	Descripción
<code>find_all_by_spec_paged(spec, pageable)</code>	<code>Page[T]</code>	Busca los que coincidan con una <code>MongoSpecification</code> , con paginación y orden

`find_all(**filters)` traduce los argumentos por palabra clave en filtros de igualdad de MongoDB:

```
# {"status": "PENDING", "customer_id": "abc"}
orders = await repo.find_all(status="PENDING", customer_id="abc")
```

El repositorio nunca guarda una sesión. Dentro de una unidad de trabajo de su origen de datos (`@transactional`, una entrega de mensaje) cada llamada pasa la sesión de la unidad al driver, así que sus escrituras forman parte de la transacción. Fuera de ella, cada llamada se ejecuta en una unidad breve propia: una lectura sin transacción, una escritura que envía un solo comando (`save`, `delete`) por su cuenta, y un método que escribe más de una vez (`save_all`) en una transacción que se confirma al final de la llamada. Un guardado fallido devuelve a los documentos el id y la revisión que tenían, así que los mismos objetos se pueden volver a guardar; un documento nuevo se inserta con `insert_one`, así que la lógica de una sobrescritura de `insert` / `save` debe pasar a acciones `@before_event` / `@after_event`.

Métodos de consulta derivados

PyFly compila los métodos vacíos (stubs) de las subclases de `MongoRepository` en consultas reales de MongoDB al arrancar. La convención de nombres es idéntica a la del adaptador relacional y a la de Spring Data: `{prefix}_by_{predicates}[_order_by_{fields}]`.

Prefijos: `find_by`, `count_by`, `exists_by`, `delete_by`

Conectores: `_and_`, `_or_`

Correspondencia de operadores

Sufijo del método	Filtro de MongoDB	Argumentos consumidos
<i>(ninguno, por defecto)</i>	<code>{field: value}</code>	1
<code>_not</code>	<code>{field: {"\$nin": [value, None]}}</code> (null nunca coincide, como en SQL)	1
<code>_greater_than</code>	<code>{field: {"\$gt": value}}</code>	1
<code>_greater_than_equal</code>	<code>{field: {"\$gte": value}}</code>	1
<code>_less_than</code>	<code>{field: {"\$lt": value}}</code>	1
<code>_less_than_equal</code>	<code>{field: {"\$lte": value}}</code>	1
<code>_between</code>	<code>{field: {"\$gte": low, "\$lte": high}}</code>	2
<code>_like</code>	<code>{field: {"\$regex": "^...\$"}}</code> (el <code>LIKE</code> de SQL: anclado, distingue mayúsculas)	1
<code>_containing</code>	<code>{field: {"\$regex": "<valor escapado>"}}</code> (distingue mayúsculas)	1
<code>_in</code>	<code>{field: {"\$in": values}}</code>	1 (lista)
<code>_is_null</code>	<code>{field: None}</code>	0
<code>_is_not_null</code>	<code>{field: {"\$ne": None}}</code>	0

Añade `_ignore_case` tras un predicado para una coincidencia que no distinga mayúsculas. (Antes de la v26.09.08, `_containing` ignoraba las mayúsculas y `_like` no estaba anclado.) Ordenación: añade `_order_by_{field}_{asc|desc}`. Varios campos de ordenación se encadenan:

```
# sort=[("name", ASC), ("created_at", DESC)]
async def find_by_active_order_by_name_asc_created_at_desc(
    self, active: bool
) → list[ProductDocument]: ...
```

El `MongoRepositoryBeanPostProcessor` detecta los stubs (cuerpos que solo contienen `...` o `pass`) y los reemplaza por invocables compilados. Fuente: `src/pyfly/data/document/mongodb/post_processor.py` y `src/pyfly/data/document/mongodb/query_compiler.py`.

Consultas personalizadas con `@query`

Para las consultas que no pueden expresarse mediante convenciones de nombres, `@query` acepta un documento de filtro de MongoDB (`{...}`) o un pipeline de agregación (`[...]`) como cadena JSON. Los parámetros con nombre usan la sintaxis `:param_name`.

catalog/order_repository.py

```

from pyfly.container import repository
from pyfly.data.document.mongodb import MongoRepository
from pyfly.data.query import query

from catalog.order_document import OrderDocument

@repository
class OrderRepository(MongoRepository[OrderDocument, str]):

    @query('{"status": ":status", "total": {"$gte": ":min_total"}}')
    async def find_by_status_min_total(
        self, status: str, min_total: float
    ) → list[OrderDocument]: ...

    @query(
        ' [{"$match": {"customer_id": ":cid"}}, '
        ' {"$group": {"_id": "$category", '
        ' "total": {"$sum": "$amount"}}}] '
    )
    async def totals_by_category(
        self, cid: str
    ) → list[dict]: ...

```

Listado B.7 — Ejemplos de filtro y agregación con @query

Reglas de sustitución:

- Un valor de cadena JSON que sea *exactamente* `:param_name` se reemplaza por el valor de Python, conservando su tipo (`int`, `bool`, `list`, etc.).
- Un `:param_name` incrustado dentro de una cadena mayor se reemplaza mediante `str(value)`.
- Los diccionarios y las listas se recorren recursivamente. Los valores JSON no textuales pasan sin cambios.

`MongoQueryExecutor` analiza la plantilla de la consulta una sola vez al arrancar, detecta si se trata de un filtro o de un pipeline, y sustituye los parámetros en el momento de la llamada.

Paginación

catalog/product_service.py

```
from pyfly.data import Page, Pageable, Sort
from pyfly.data.document.mongodb import MongoRepository

from catalog.product_document import ProductDocument

async def list_products(
    repo: MongoRepository[ProductDocument, str],
    page: int = 1,
    size: int = 20,
) → Page[ProductDocument]:
    pageable = Pageable.of(
        page=page,
        size=size,
        sort=Sort.by("name"),
    )
    return await repo.find_all(pageable)
```

Listado B.8 — Listado de productos paginado

`find_all(pageable)` cuenta el total, aplica el orden del `Pageable`, recorta con `.skip((page-1)*size)` y `.limit(size)` sobre la consulta de Beanie, y devuelve `Page[T]`. `Pageable` parte de 1, así que la página 1 es la primera página.

Gestión de transacciones

Las transacciones multidocumento requieren un despliegue con **conjunto de réplicas** (replica set); basta con uno de un solo nodo. Una instancia de MongoDB independiente (standalone) no las admite: allí `@transactional` lanza `IllegalTransactionStateError` en lugar de ejecutarse sin transacción.

pyfly.yaml

```
# Start MongoDB: mongod --replSet rs0 --bind_ip localhost
# Init (once, in mongosh): rs.initiate()
pyfly:
  data:
    document:
      enabled: true
      uri: "mongodb://localhost:27017/?replicaSet=rs0"
      database: myapp
```

Listado B.9 — Conjunto de réplicas de un solo nodo para desarrollo local

MongoDB usa el **mismo** `@transactional` que el adaptador relacional. Su `MongoTransactionManager` liga una `ClientSession` de pymongo a la tarea en curso, y cada llamada al repositorio dentro del método se la pasa al driver:

billing/transfer.py

```

from pyfly.container import service
from pyfly.data import transactional

from billing.account_repository import AccountRepository

@service
class TransferService:
    def __init__(self, accounts: AccountRepository) → None:
        self._accounts = accounts

    @transactional(datasource="document")
    async def transfer(
        self, from_id: str, to_id: str, amount: float
    ) → None:
        src = await self._accounts.find_by_id(from_id)
        dst = await self._accounts.find_by_id(to_id)
        if src is None or dst is None or src.balance < amount:
            raise ValueError("Invalid transfer")
        src.balance -= amount
        dst.balance += amount
        await self._accounts.save(src)
        await self._accounts.save(dst) # a failure here undoes the debit

```

Listado B.10 — Transferencia de fondos atómica con `@transactional`

Si todo va bien, la transacción se confirma (commit); ante cualquier excepción se aborta y se vuelve a lanzar. `datasource="document"` nombra el origen de datos documental; en una aplicación sin capa relacional es el de por defecto, y un `@transactional` sin argumentos lo encuentra. Un servicio con un `_session_factory` relacional y un `_motor_client` a la vez debe nombrar su origen de datos, o la llamada lanza `IllegalTransactionStateError`. El resto sigue las reglas relacionales: `REQUIRES_NEW` suspende la unidad, `NESTED` lanza `NestedTransactionNotSupportedError`, solo se acepta `Isolation.DEFAULT`, un fallo

capturado de un participante hace que el commit lance `UnexpectedRollbackError`, y un commit cuyo resultado el driver no puede conocer lanza `CommitOutcomeUnknownError`. El código que llama directamente a Beanie o a pymongo pasa la sesión: un parámetro `session` la recibe, y `current_session()` la devuelve.

El outbox transaccional también funciona sobre MongoDB: con `pyfly.eda.outbox.enabled: true`, una aplicación solo de MongoDB añade sus eventos a colecciones `pyfly_outbox_*` en la propia transacción de la unidad (`pyfly.eda.outbox.store: mongo`, que `auto` elige allí).

⚠ SE REQUIERE UN CONJUNTO DE RÉPLICAS

`@transactional` lanza `IllegalTransactionStateError` contra una instancia de MongoDB independiente. Usa el fragmento de URI `?replicaSet=rs0` (consulta el Listado B.9) incluso en desarrollo local. En una aplicación solo de MongoDB sobre un servidor independiente, fija además `pyfly.messaging.listener.transactional: false`: si no, cada entrega de mensaje abriría una unidad documental y fallaría.

❗ MOTOR YA NO ESTÁ

Desde la v26.09.08 solo se acepta el `AsyncMongoClient` de PyMongo: el gestor de transacciones rechaza un cliente de Motor o de mongomock con `TypeError`. `mongo_transactional` aún se puede importar, como alias obsoleto de `@transactional`.

Autoconfiguración

`DocumentAutoConfiguration` se activa cuando:

1. `beanie` es importable (`@conditional_on_class("beanie")`), y

2. `pyfly.data.document.enabled` vale `"true"` en la configuración.

Registra estos beans automáticamente:

Bean	Tipo	Función
<code>mongo_client</code>	<code>AsyncMongoClient</code>	El cliente y su pool de conexiones (tu propio bean singleton <code>AsyncMongoClient</code> lo sustituye)
<code>mongo_post_processor</code>	<code>MongoRepositoryBeanPostProcessor</code>	Compila los stubs de consultas derivadas
<code>odm_initializer</code>	<code>BeanieInitializer</code>	Llama a <code>init_beanie()</code> al arrancar
<code>mongo_transaction_manager</code>	<code>MongoTransactionManager</code>	Ejecuta las unidades de trabajo documentales
<code>mongo_health_indicator</code>	<code>MongoHealthIndicator</code>	Comprobación de preparación (<code>ping</code> , 2 s)

`BeanieInitializer` descubre por sí mismo las clases de documento: el documento de cada bean `MongoRepository`, cada documento de Beanie registrado en el contenedor, las clases o módulos listados en `pyfly.data.document.models`, y los documentos que nombran sus campos `Link`. Esto significa que definir un repositorio es suficiente: no necesitas registrar los modelos de documento por separado.

Archivos fuente: `src/pyfly/data/document/auto_configuration.py`, `src/pyfly/data/document/mongodb/initializer.py`.

Pruebas

Prueba el código transaccional contra un conjunto de réplicas real.

`mongodb_replica_set_container()` arranca en Docker un conjunto de réplicas `rs0` de un solo nodo, y `pyfly_config` activa la capa documental con su URI:

tests/test_transfer.py

```

import pytest

from pyfly.testing import (
    data_slice,
    mongodb_replica_set_container,
    pyfly_config,
    requires_docker,
)

from billing.account_document import AccountDocument
from billing.account_repository import AccountRepository
from billing.transfer import TransferService

@pytest.fixture(scope="module")
def mongo():
    with mongodb_replica_set_container() as container:
        yield container

@requires_docker
async def test_transfer_commits_both_sides(mongo) → None:
    config = pyfly_config(
        mongo, base={"pyfly.data.document.database": "billing_test"}
    )
    async with await data_slice(
        AccountRepository, TransferService, config=config
    ) as ctx:
        accounts = ctx.get_bean(AccountRepository)
        transfers = ctx.get_bean(TransferService)
        await accounts.delete_all()
        src = await accounts.save(AccountDocument(balance=100))
        dst = await accounts.save(AccountDocument(balance=0))

        await transfers.transfer(str(src.id), str(dst.id), 60)
        with pytest.raises(ValueError):
            await transfers.transfer(str(src.id), str(dst.id), 60)

```

```
assert (await accounts.find_by_id(src.id)).balance == 40
assert (await accounts.find_by_id(dst.id)).balance == 60
```

Listado B.11 — Una prueba de transacción sobre un conjunto de réplicas

`@requires_docker` omite la prueba donde Docker no está disponible. La reversión del slice solo cubre los orígenes de datos relacionales, así que la prueba vacía antes su colección. Instala el soporte con `pip install 'pyfly[testcontainers]'`; mongomock y Motor ya no se admiten.

APÉNDICE C

ECM: contenido y firma electrónica

`pyfly.ecm` proporciona abstracciones hexagonales para la gestión documental empresarial: almacenamiento de blobs, metadatos de documentos, jerarquías de carpetas y flujos de firma electrónica. El módulo incluye adaptadores completamente cableados para almacenamiento local, AWS S3 y Azure Blob Storage, además de adaptadores de firma electrónica para DocuSign, Adobe Sign y Logalty — todos seleccionables sin cambiar una sola línea del código de la aplicación, puramente mediante YAML.

❗ EL MÓDULO EN SÍ NO REQUIERE DEPENDENCIAS ADICIONALES

El módulo ECM central (`pyfly.ecm`) no tiene dependencias de terceros más allá de la biblioteca estándar. Los adaptadores de almacenamiento en la nube (`boto3` , `azure-storage-blob`) y las llamadas a los SDK de firma electrónica se realizan a través de envoltorios asíncronos ligeros en cada clase de adaptador; instala solo lo que necesites.

Modelo de dominio

Todos los tipos de ECM son dataclasses sencillas de Python definidas en `src/pyfly/ecm/models.py` .

Clase	Campos clave	Descripción
<code>Document</code>	<code>id</code> , <code>name</code> , <code>folder_id</code> , <code>content_type</code> , <code>size_bytes</code> , <code>metadata</code> , <code>versions</code>	Registro lógico de documento
<code>DocumentVersion</code>	<code>version</code> , <code>content_hash</code> , <code>size_bytes</code> , <code>storage_uri</code>	Instantánea inmutable de versión
<code>Folder</code>	<code>id</code> , <code>name</code> , <code>parent_id</code> , <code>path</code>	Nodo de carpeta en la jerarquía
<code>Recipient</code>	<code>name</code> , <code>email</code> , <code>role</code>	Destinatario de firma electrónica
<code>SignatureRequest</code>	<code>document_id</code> , <code>recipients</code> , <code>subject</code> , <code>message</code>	Carga útil de la solicitud de firma
<code>ESignatureEnvelope</code>	<code>id</code> , <code>provider</code> , <code>status</code> , <code>provider_envelope_id</code>	Registro de seguimiento del sobre

`ESignatureStatus` es un `StrEnum` con los valores `DRAFT`, `SENT`, `SIGNED`, `DECLINED` y `EXPIRED`.

Puertos (contratos)

Cuatro clases `Protocol` definidas en `src/pyfly/ecm/ports.py` forman el límite hexagonal. Los servicios de tu aplicación dependen de estos contratos; los adaptadores los implementan.

DocumentStoragePort

```
class DocumentStoragePort(Protocol):
    name: str
    async def upload(
        self, document: Document, content: bytes
    ) → DocumentVersion: ...
    async def download(
        self, document: Document, version: int | None = None
    ) → bytes: ...
    async def delete(
        self, document: Document, version: int | None = None
    ) → bool: ...
```

MetadataStoragePort

```
class MetadataStoragePort(Protocol):
    async def save(self, document: Document) → Document: ...
    async def get(self, document_id: str) → Document | None: ...
    async def list(
        self, folder_id: str | None = None, *, limit: int = 100
    ) → list[Document]: ...
    async def delete(self, document_id: str) → bool: ...
```

FolderRepositoryPort

```
class FolderRepositoryPort(Protocol):
    async def save(self, folder: Folder) → Folder: ...
    async def get(self, folder_id: str) → Folder | None: ...
    async def list(self, parent_id: str | None = None) → list[Folder]: ...
    async def delete(self, folder_id: str) → bool: ...
```


ESignatureAdapter

```
class ESignatureAdapter(Protocol):
    name: str
    async def send(
        self, request: SignatureRequest
    ) → ESignatureEnvelope: ...
    async def get(self, envelope_id: str) → ESignatureEnvelope | None: ...
    async def cancel(self, envelope_id: str) → bool: ...
```

DocumentService

`DocumentService` orquesta los puertos de almacenamiento y de metadatos. Es el punto de entrada principal para la subida, descarga, listado y eliminación de documentos.

contracts/ecm_usage.py

```

from pyfly.ecm import (
    DocumentService,
    LocalFilesystemStorageAdapter,
)
from pyfly.ecm.in_memory import (
    InMemoryFolderRepository,
    InMemoryMetadataStorage,
)

service = DocumentService(
    storage=LocalFilesystemStorageAdapter("/var/ecm/docs"),
    metadata=InMemoryMetadataStorage(),
    folders=InMemoryFolderRepository(),
)

async def demo() → None:
    # Upload
    doc = await service.upload(
        name="contract.pdf",
        content=b"%PDF-1.4 ...",
        content_type="application/pdf",
        metadata={"department": "legal", "year": "2026"},
    )

    # Download (latest version)
    content = await service.download(doc.id)

    # Download a specific version
    v1_content = await service.download(doc.id, version=1)

    # List documents in a folder
    docs = await service.list(folder_id=doc.folder_id)

```

```
# Delete (returns False if blob delete partially fails)
ok = await service.delete(doc.id)
```

Listado C.1 — DocumentService: subir, descargar, eliminar

Semántica de `delete`

`DocumentService.delete(document_id)` elimina tanto el blob almacenado como el registro de metadatos y devuelve el **AND lógico** de los dos resultados de eliminación. Si el ID del documento es desconocido, devuelve `False` de inmediato. Si la eliminación del blob falla, el registro de metadatos se elimina igualmente (de modo que el documento lógico desaparece), pero el método devuelve `False` para señalar una eliminación parcial. Este contrato se verifica en el código fuente en `src/pyfly/ecm/services.py`.

Gestión de carpetas

contracts/ecm_folders.py

```
from pyfly.ecm import DocumentService, Folder

async def setup_folders(service: DocumentService) → None:
    root = await service.create_folder(
        Folder(name="contracts", path="/contracts")
    )
    legal = await service.create_folder(
        Folder(name="legal", parent_id=root.id, path="/contracts/legal")
    )
    doc = await service.upload(
        name="nda.pdf",
        content=b"... ",
        folder_id=legal.id,
        content_type="application/pdf",
    )
```

Listado C.2 — Creación de jerarquías de carpetas

❗ EL PUERTO DE CARPETAS ES OPCIONAL

Pasa `folders=None` a `DocumentService` si no necesitas soporte de carpetas. Llamar a `create_folder` en una instancia así lanza `RuntimeError`.

ESignatureService

`ESignatureService` envuelve un `ESignatureAdapter` y expone tres operaciones: `request`, `get` y `cancel`.

contracts/esignature_usage.py

```

from pyfly.ecm import (
    ESignatureService,
    NoOpESignatureAdapter,
    Recipient,
    SignatureRequest,
)

esig = ESignatureService(adapter=NoOpESignatureAdapter())

async def send_for_signature(document_id: str) → str:
    envelope = await esig.request(
        SignatureRequest(
            document_id=document_id,
            recipients=[
                Recipient(
                    name="Alice Johnson",
                    email="alice@example.com",
                    role="signer",
                ),
            ],
            subject="Please sign your employment contract",
            message="Review and sign at your earliest convenience.",
        )
    )
    return envelope.id

```

Listado C.3 — Solicitar una firma electrónica

Adaptadores de almacenamiento

Clase de adaptador	Valor de <code>pyfly.ecm.storage.provider</code>	Notas
<code>LocalFilesystemStorageAdapter</code>	<code>local</code> (por defecto)	Almacena los blobs como <code><base_dir>/<id>/v<n></code>
<code>AwsS3StorageAdapter</code>	<code>s3</code> o <code>aws</code>	Requiere <code>boto3</code> ; asíncrono mediante <code>run_in_executor</code>
<code>AzureBlobStorageAdapter</code>	<code>azure</code>	Requiere <code>azure-storage-blob</code>

Los tres implementan `DocumentStoragePort`. `LocalFilesystemStorageAdapter` crea el directorio base en el momento de la construcción y almacena cada versión como un archivo independiente, lo que lo hace adecuado para desarrollo y pruebas sin servicios externos.

Adaptadores de firma electrónica

Clase de adaptador	Valor de <code>pyfly.ecm.esignature.provider</code>	Notas
<code>NoOpESignatureAdapter</code>	<code>noop</code> (por defecto)	Devuelve sobres ficticios; seguro para dev/test
<code>DocuSignESignatureAdapter</code>	<code>docusign</code>	Llamadas a la API REST mediante <code>httpx</code>
<code>AdobeSignESignatureAdapter</code>	<code>adobe</code>	Llamadas a la API REST mediante <code>httpx</code>
<code>LogaltyESignatureAdapter</code>	<code>logalty</code>	Llamadas a la API REST mediante <code>httpx</code>

Implementa `ESignatureAdapter` para añadir un proveedor personalizado (por ejemplo, un servicio de firma interno). Registra tu implementación como un bean e inyéctala en `ESignatureService`.

Autoconfiguración

`EcmAutoConfiguration` se activa cuando se establece `pyfly.ecm.enabled=true` en la configuración. Registra cinco beans:

Bean	Implementación por defecto
<code>metadata_storage</code>	<code>InMemoryMetadataStorage</code>
<code>folder_repository</code>	<code>InMemoryFolderRepository</code>
<code>document_storage</code>	<code>LocalFilesystemStorageAdapter</code> (directorio temporal)
<code>document_service</code>	<code>DocumentService</code> cableando los tres anteriores
<code>esignature_adapter</code>	<code>NoOpESignatureAdapter</code>
<code>esignature_service</code>	<code>ESignatureService</code> envolviendo el adaptador

Los adaptadores de almacenamiento y de firma electrónica se seleccionan desde la configuración, de modo que cambias de proveedor puramente mediante YAML — sin necesidad de cambiar código:

pyfly.yaml

```

pyfly:
  ecm:
    enabled: true
    storage:
      provider: s3
      local:
        base-dir: /var/ecm
      s3:
        bucket: my-docs-bucket
        region: eu-west-1
        key-prefix: documents/
      azure:
        container: my-container
        connection-string: "DefaultEndpointsProtocol=https;..."
        account-url: "https://acct.blob.core.windows.net"
        key-prefix: documents/
    esignature:
      provider: docusign
      docusign:
        base-url: "https://demo.docusign.net/restapi"
        account-id: "your-account-id"
        access-token: "your-access-token"
      adobe:
        api-base: "https://api.na4.adobesign.com"
        access-token: "your-token"
      logalty:
        api-base: "https://api.logalty.com"
        api-key: "your-api-key"

```

Listado C.4 — YAML de ECM completo con almacenamiento S3 y DocuSign

💡 EMPIEZA CON LOCAL + NOOP

En desarrollo, omite las claves `provider` y deja que actúen los valores por defecto: el almacenamiento `local` usa un directorio temporal nuevo y la firma electrónica `noop` devuelve datos ficticios deterministas. No se necesitan servicios externos.

Implementar un adaptador de almacenamiento personalizado

Implementa `DocumentStoragePort` y regístralo como un `@bean` :

infra/gcs_storage.py

```

from pyfly.ecm.models import Document, DocumentVersion
from pyfly.ecm.ports import DocumentStoragePort

class GcsStorageAdapter:
    """Google Cloud Storage adapter (skeleton)."""

    name = "gcs"

    def __init__(self, bucket: str, key_prefix: str = "") → None:
        self._bucket = bucket
        self._prefix = key_prefix

    async def upload(
        self, document: Document, content: bytes
    ) → DocumentVersion:
        version_no = (
            (document.versions[-1].version + 1) if document.versions else 1
        )
        key = f"{self._prefix}{document.id}/v{version_no}"
        # ... upload content to GCS ...
        return DocumentVersion(
            version=version_no,
            content_hash="sha256-placeholder",
            size_bytes=len(content),
            storage_uri=f"gs://{self._bucket}/{key}",
        )

    async def download(
        self, document: Document, version: int | None = None
    ) → bytes:
        target = version or document.versions[-1].version
        key = f"{self._prefix}{document.id}/v{target}"
        # ... download from GCS ...
        return b""

    async def delete(

```

```

        self, document: Document, version: int | None = None
    ) → bool:
        # ... delete object(s) from GCS ...
        return True

```

Listado C.5 — Esqueleto de un adaptador de almacenamiento personalizado

Archivos fuente: - `src/pyfly/ecm/models.py` — dataclasses de dominio - `src/pyfly/ecm/ports.py` — contratos Protocol - `src/pyfly/ecm/services.py` — `DocumentService`, `ESignatureService` - `src/pyfly/ecm/adapters/` — adaptadores de almacenamiento y firma electrónica incluidos - `src/pyfly/ecm/in_memory.py` — `InMemoryMetadataStorage`, `InMemoryFolderRepository` - `src/pyfly/ecm/auto_configuration.py` — `EcmAutoConfiguration`

APÉNDICE D

CLI y resolución de problemas

La CLI `pyfly` proporciona andamiaje de proyectos, arranque de la aplicación, migraciones de base de datos y diagnóstico del entorno. Está construida con **Click** para el análisis de comandos y con **Rich** para la salida coloreada en la terminal.

Instalación: `uv add "pyfly[cli]"` (o `uv sync --extra cli`). Requiere Click, Rich, Jinja2 y questionnaire.

Punto de entrada: `pyfly` — registrado como script de consola en `pyproject.toml`.

Referencia de comandos

Comando	Descripción
<code>pyfly new [NAME]</code>	Genera el andamiaje de un nuevo proyecto (interactivo o directo)
<code>pyfly run</code>	Arranca el servidor de aplicación ASGI
<code>pyfly info</code>	Muestra la versión de Python y los extras instalados
<code>pyfly doctor</code>	Diagnostica el entorno (Python, herramientas, PyFly)
<code>pyfly db init</code>	Inicializa un entorno de migraciones de Alembic
<code>pyfly db migrate [-m MSG]</code>	Genera automáticamente una revisión de migración
<code>pyfly db upgrade [REVISION]</code>	Aplica las migraciones pendientes (por defecto: <code>head</code>)
<code>pyfly db downgrade REVISION</code>	Revierte a una revisión anterior
<code>pyfly license</code>	Muestra el texto de la licencia Apache 2.0
<code>pyfly sbom [--json]</code>	Tabla del Software Bill of Materials (inventario de software)
<code>pyfly --version</code>	Imprime la versión instalada de PyFly
<code>pyfly --help</code>	Imprime el banner y todos los comandos

pyfly new

Crea un directorio de proyecto completo a partir de uno de siete arquetipos.

```
pyfly new <name> [--archetype ARCHETYPE] [--features FEAT,...] [--directory DIR]
pyfly new                                     # interactive mode (questionary TUI)
```

Arquetipos

Arquetipo	Características por defecto	Qué se genera
<code>core</code>	<i>(ninguna)</i>	Contenedor de DI, configuración, Dockerfile
<code>web-api</code>	<code>web</code>	Controladores REST, servicios, repositorios (CRUD de Todo)
<code>fastapi-api</code>	<code>fastapi</code>	Stack FastAPI, OpenAPI nativo
<code>web</code>	<code>web</code>	HTML renderizado en servidor con plantillas Jinja2
<code>hexagonal</code>	<code>web</code>	Puertos y adaptadores, capas dominio/aplicación/infra/api
<code>library</code>	<i>(ninguna)</i>	Paquete PEP 561 <code>py.typed</code> , sin runtime de PyFly
<code>cli</code>	<code>shell</code>	Comandos de shell con DI, sin punto de entrada ASGI

Valores disponibles para `--features`

Característica	Qué añade
<code>web</code>	Servidor HTTP Starlette, controladores REST, documentación OpenAPI
<code>fastapi</code>	Servidor HTTP FastAPI, OpenAPI nativo
<code>granian</code>	Servidor ASGI Granian (Rust/tokio)
<code>hypercorn</code>	Servidor ASGI Hypercorn (HTTP/2, HTTP/3)
<code>data-relational</code>	SQLAlchemy ORM (asíncrono), migraciones Alembic
<code>data-document</code>	Beanie ODM sobre el cliente asíncrono de PyMongo (MongoDB)
<code>eda</code>	Bus de eventos en memoria, soporte de Kafka + RabbitMQ
<code>cache</code>	Capa de caché (en memoria por defecto; Redis si está instalado)
<code>client</code>	Cliente HTTP resiliente (httpx + reintentos/cortacircuitos)
<code>security</code>	Autenticación JWT, hashing de contraseñas con bcrypt
<code>scheduling</code>	Programación de tareas basada en cron
<code>observability</code>	Métricas de Prometheus, trazas de OpenTelemetry
<code>cqrs</code>	Segregación de Responsabilidad entre Comandos y Consultas (CQRS)
<code>shell</code>	Comandos de shell interactivos potenciados con DI

Ejemplos

terminal

```
# Minimal core service
pyfly new wallet-service

# REST API with SQL data layer
pyfly new lumen --archetype web-api --features web,data-relational

# Hexagonal service with cache and security
pyfly new payment-svc --archetype hexagonal \
    --features web,data-relational,cache,security

# MongoDB-backed microservice
pyfly new catalog-svc --archetype web-api \
    --features web,data-document

# CLI application with interactive shell
pyfly new admin-tool --archetype cli

# Interactive mode (arrow keys + checkboxes)
pyfly new
```

Listado D.1 — Ejemplos de `pyfly new`

En el modo interactivo, el asistente solicita el nombre, el nombre del paquete (precargado a partir del nombre del proyecto), el arquetipo (selección única con las flechas) y las características (selección múltiple con la barra espaciadora). Antes de la creación se muestra un resumen de confirmación. Pulsa `Ctrl+C` en cualquier prompt para salir de forma limpia.

Los nombres de proyecto que contienen guiones se convierten automáticamente a nombres de paquete de Python válidos (`my-service` → `my_service`). Usa el prompt **Package name** en el modo interactivo para sobrescribir este valor derivado.

pyfly run

Arranca la aplicación ASGI usando el servidor seleccionado automáticamente.

```
pyfly run [--host HOST] [--port PORT] [--server SERVER]
          [--workers N] [--reload] [--app MODULE:VAR]
```

Opción	Por defecto	Descripción
<code>--host</code>	<code>0.0.0.0</code>	Dirección de enlace
<code>--port</code>	Config o <code>8080</code>	Puerto de la app (CLI → <code>pyfly.server.port</code> → 8080)
<code>--server</code>	Autodetección	<code>granian</code> , <code>uvicorn</code> o <code>hypercorn</code>
<code>--workers</code>	Config o <code>0</code>	Procesos worker (<code>0</code> = número de CPUs)
<code>--reload</code>	desactivado	Recarga automática al cambiar el código
<code>--app</code>	Autodescubrimiento	Ruta de importación, p. ej. <code>myapp.main:app</code>

Prioridad de servidores (cuando se omite `--server`): Granian › Uvicorn › Hypercorn.
Gana el primer servidor que se pueda importar.

Descubrimiento de la aplicación (cuando se omite `--app`):

1. `pyfly.app.module` en `pyfly.yaml`
2. `app.module` en `pyfly.yaml` (disposición plana)
3. Autoescaneo de `src/<package>/main.py`

`pyfly run` añade `src/` a `sys.path` automáticamente en los proyectos con disposición src.

terminal

```
# Development with auto-reload
pyfly run --reload

# Production: Granian, bind all interfaces, all CPU cores
pyfly run --host 0.0.0.0 --port 80 --server granian --workers 0

# Explicit application path
pyfly run --app my_service.main:app
```

Listado D.2 — Ejemplos de pyfly run

pyfly info

Muestra dos tablas de Rich: versión de Python, plataforma y arquitectura; además del estado de instalación de cada módulo opcional de PyFly. Resulta útil tras una instalación selectiva para confirmar qué adaptadores están disponibles.

```
pyfly info
```

pyfly doctor

Ejecuta una comprobación integral de salud del entorno.

```
pyfly doctor
```

Comprobación	Condición de aprobado	Impacto del fallo
Versión de Python	<code>>= 3.12</code>	Fatal — doctor informa de fallo
Entorno virtual	<code>sys.prefix != sys.base_prefix</code>	Solo advertencia
<code>git</code> en el PATH	<code>shutil.which("git")</code> devuelve una ruta	Fatal
<code>uv</code> en el PATH	<code>shutil.which("uv")</code> devuelve una ruta	Fatal
<code>uvicorn</code> en el PATH	presente	Informativo (-)
<code>alembic</code> en el PATH	presente	Informativo (-)
<code>ruff</code> en el PATH	presente	Informativo (-)
<code>mypy</code> en el PATH	presente	Informativo (-)
PyFly importable	<code>import pyfly</code> tiene éxito	Fatal

Finaliza con **"All checks passed!"** o **"Some issues found."**.

pyfly db

Comandos de migración de base de datos respaldados por [Alembic](#).

REQUIERE EL EXTRA DATA-RELATIONAL

Todos los subcomandos `pyfly db` requieren Alembic (`pyfly[data-relational]`).

pyfly db init

```
pyfly db init
```

Crea `alembic/` y `alembic.ini` en el directorio actual. Sobrescribe `alembic/env.py` con una plantilla de PyFly que importa los módulos que declaran tus entidades, lista `Base.metadata` y las tablas propias del framework (`framework_metadata`) como `target_metadata` , y se ejecuta sobre el origen de datos principal de la aplicación, de modo que autogenerate nunca propone borrar ninguna de ellas. Finaliza con un error si `alembic/` ya existe. Regenera un `env.py` creado antes de la v26.09.08: no listaba ningún modelo, y su autogenerate borraría todas las tablas (PyFly rechaza ahora esa revisión).

pyfly db migrate

```
pyfly db migrate [-m "description"]
```

Ejecuta `alembic revision --autogenerate` . Compara el `Base.metadata` actual con la base de datos en vivo y escribe un nuevo archivo de versión en `alembic/versions/` . Requiere `alembic.ini` (ejecuta primero `pyfly db init`).

pyfly db upgrade / downgrade

```
pyfly db upgrade [REVISION]      # default: head
pyfly db downgrade REVISION      # e.g. -1, base, abc123
```

Flujo de trabajo típico de migración

terminal

```
# 1. Initialise Alembic (once per project)
pyfly db init

# 2. Generate initial migration from your entity models
pyfly db migrate -m "initial schema"

# 3. Apply migrations
pyfly db upgrade

# 4. After modifying entities, generate a new revision
pyfly db migrate -m "add order status column"
pyfly db upgrade

# 5. Roll back one step
pyfly db downgrade -1

# 6. Revert all migrations
pyfly db downgrade base
```

Listado D.3 — Ciclo de vida de las migraciones de base de datos

pyfly sbom

Imprime una tabla de Rich con cada dependencia de PyFly junto con las versiones requeridas e instaladas. La opción `--json` produce JSON legible por máquina para pipelines de cumplimiento normativo.

```
pyfly sbom
pyfly sbom --json
```

Flujo de trabajo de desarrollo típico

terminal

```
# Verify environment
pyfly doctor

# Scaffold the project
pyfly new lumen --archetype web-api \
  --features web,data-relational,security

cd lumen

# Check what was installed
pyfly info

# Initialise migrations
pyfly db init
pyfly db migrate -m "initial schema"
pyfly db upgrade

# Develop with auto-reload
pyfly run --reload

# Add a new column, migrate, upgrade
pyfly db migrate -m "add shipment_id to orders"
pyfly db upgrade
```

Listado D.4 — Flujo de trabajo de extremo a extremo, de la configuración al desarrollo

Resolución de problemas

Problemas comunes y soluciones

Síntoma	Causa probable	Solución
<code>command not found: pyfly</code>	El PATH no se actualizó tras la instalación	<code>source ~/.zshrc</code> (o <code>~/.bashrc</code>); o <code>export PATH="\$HOME/.pyfly/bin:\$PATH"</code>
<code>Python 3.11 found (3.12 required)</code>	Python antiguo en el PATH	<code>brew install python@3.12</code> (macOS) o <code>sudo apt install python3.12</code> (Ubuntu)
<code>ModuleNotFoundError: No module named 'starlette'</code>	El extra <code>web</code> no está instalado	<code>uv add "pyfly[web]"</code> o <code>uv sync --extra web</code>
<code>ModuleNotFoundError: No module named 'beanie'</code>	Falta el extra <code>data-document</code>	<code>uv add "pyfly[data-document]"</code>
<code>ModuleNotFoundError: No module named 'sqlalchemy'</code>	Falta el extra <code>data-relational</code>	<code>uv add "pyfly[data-relational]"</code>
<code>alembic is not installed</code>	Falta Alembic	<code>uv add "pyfly[data-relational]"</code>
<code>No application found</code>	No hay <code>pyfly.yaml</code> ni opción <code>--app</code>	Añade <code>pyfly.app.module: myapp.main:app</code> a <code>pyfly.yaml</code> , o pasa <code>--app myapp.main:app</code>
<code>No ASGI server found</code>	No hay ningún extra de servidor instalado	<code>uv add "pyfly[web]"</code> (añade Uvicorn)
<code>venv module not found</code>	Python en paquetes separados (Debian/Ubuntu)	<code>sudo apt install python3.12-venv</code>

Síntoma	Causa probable	Solución
<code>Directory 'alembic' already exists</code>	<code>db init</code> ejecutado dos veces	<code>rm -rf alembic alembic.ini</code> y vuelve a ejecutar <code>pyfly db init</code>
<code>uv cache clean</code> arregla instalaciones lentas	Caché de uv corrupta	Ejecuta <code>uv cache clean</code> y reintenta
Ctrl+C en <code>pyfly new</code> no deja archivos	Modo interactivo cancelado limpiamente	Vuelve a ejecutar <code>pyfly new</code> ; no hace falta limpiar nada

Desinstalación

Para una instalación basada en el instalador (`bash install.sh`):

terminal

```
# Remove the installation directory
rm -rf ~/.pyfly

# Remove the PATH line from your shell profile
# Delete the line: export PATH="$HOME/.pyfly/bin:$PATH" # PyFly Framework
```

Listado D.5 — Desinstalar PyFly

Para una instalación manual con `uv` / `pip` : `uv remove pyfly` o `pip uninstall pyfly` .

Extender la CLI

La CLI está construida sobre Click. Añade comandos personalizados importando el grupo `cli` y llamando a `cli.add_command` :

myapp/generate.py

```
import click
from pyfly.cli.main import cli

@click.command()
@click.argument("entity_name")
def generate(entity_name: str) → None:
    """Generate boilerplate for a new entity."""
    click.echo(f"Generating {entity_name} entity ...")

cli.add_command(generate, name="generate")
```

Listado D.6 — Añadir un comando de CLI personalizado

Registra el módulo del comando en los entry points de tu `pyproject.toml` o impórtalo en el `__init__.py` de tu aplicación antes de que se invoque la CLI.

APÉNDICE E

Aplicaciones Web Nativas y Administración de Modelos

PyFly sirve páginas HTML y administra los mismos datos que utilizan sus API REST. Este apéndice sigue el catálogo ejecutable de `samples/webapp`. Su `Product` es una entidad PyFly `BaseEntity` normal: los formularios y el panel leen y modifican las mismas filas. Puedes registrar entidades SQLAlchemy o documentos Beanie existentes sin definir un segundo modelo de persistencia.

Lumen sigue siendo el servicio de monederos del libro. El catálogo es otro ejemplo, más pequeño y centrado en el navegador. No expongamos saldos o asientos de Lumen como registros genéricos editables: el movimiento de dinero debe pasar por los comandos del dominio. Un proveedor de administración personalizado puede delegar en ellos.

E.1. Ejecutar el catálogo completo

Desde el repositorio del framework, entra en `samples/webapp`. Su proyecto resuelve PyFly desde el repositorio que lo contiene y ejecuta esta implementación.

```

cd samples/webapp
uv sync --group dev
export WEBAPP_ADMIN_PASSWORD=\
    "$(python -c 'import secrets; print(secrets.token_urlsafe(24))')"
export WEBAPP_EDIT_TOKEN_KEY=\
    "$(python -c 'import secrets; print(secrets.token_urlsafe(48))')"
export WEBAPP_DATABASE_URL="sqlite+aiosqlite:///catalog.db"
export PYFLY_SECURITY_CSRF_COOKIE_SECURE=false
uv run python -m catalog.init_db
uv run uvicorn catalog.app:create_webapp --factory \
    --host 127.0.0.1 --port 8080

```

Abre <http://127.0.0.1:8080/>, identifícate como `admin` con la contraseña generada y crea un producto. En `/admin`, selecciona **Datasources** → **Products**: encontrarás el mismo registro. Cambia el precio y recarga el catálogo. Reinicia el servidor y comprueba que el archivo SQLite conserva el producto.

El comando de inicialización crea la tabla del ejemplo únicamente si no existe. El servidor no crea ni migra el esquema. El ejemplo sitúa administración en el puerto 8080 para compartir autenticación; el puerto de administración separado predeterminado de PyFly es 9090. En despliegues utiliza migraciones, HTTPS y cookies seguras. La variable que desactiva la cookie segura es solo para HTTP local.

E.2. Un modelo, dos interfaces de navegador

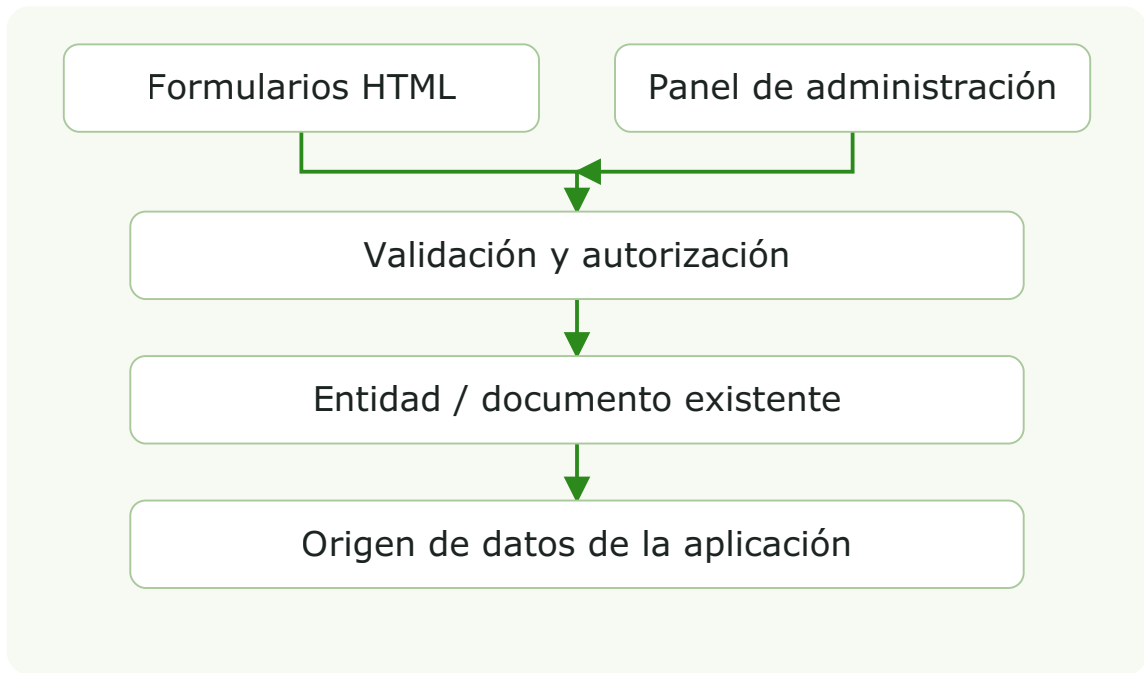


Figura E.1 — Ambas interfaces usan los mismos modelos y origen de datos.

Tipo	Responsabilidad
<code>Product(BaseEntity)</code>	Columnas persistidas, claves, restricciones y auditoría.
<code>ProductWrite(BaseModel)</code>	Entrada y validación compartidas por ambas interfaces.
<code>ModelAdmin</code>	Campos visibles y editables, operaciones y selección del origen.

`ProductForm` añade al esquema de entrada un token de edición. Los esquemas de entrada no crean tablas y `ModelAdmin` no sustituye la entidad. El catálogo es propietario de su fábrica de sesiones; el controlador HTML y el panel utilizan un

servicio de administración sobre esa misma fábrica. En otras aplicaciones las páginas públicas pueden conservar sus servicios habituales y la administración puede tener un registro independiente con permisos más restringidos.

catalog/models.py

```
from decimal import Decimal

from pydantic import BaseModel, Field
from sqlalchemy import Numeric, String
from sqlalchemy.orm import Mapped, mapped_column

from pyfly.data.relational.sqlalchemy.entity import BaseEntity


class Product(BaseEntity):
    __tablename__ = "webapp_products"
    name: Mapped[str] = mapped_column(String(80), unique=True)
    price: Mapped[Decimal] = mapped_column(Numeric(10, 2))


class ProductWrite(BaseModel):
    name: str = Field(min_length=1, max_length=80)
    price: Decimal = Field(ge=0, max_digits=10, decimal_places=2)


class ProductForm(ProductWrite):
    edit_version: str = ""
```

Listado E.1 — La entidad existente y el esquema de entrada compartido

E.3. Renderizar páginas y resolver URLs

Instala `pyfly[webapp]` para Starlette, Jinja y seguridad de navegador; añada `data-relational` para SQL o `data-document` para MongoDB. Los mismos contratos de vista funcionan con el extra `fastapi`. Activa el renderizado explícitamente:

pyfly.yaml

```

pyfly:
  web:
    templates:
      enabled: true
      directories: []
      packages: ["catalog:templates"]
    static:
      enabled: true
      directories: []
      packages: ["catalog:static"]
    errors:
      html-enabled: true

```

Listado E.2 — Recursos de paquete utilizados por el catálogo

Los directorios del sistema de archivos son relativos al directorio de trabajo. Los recursos de paquete viajan con la aplicación instalada. Usa `directories: []` si solo utilizas paquetes; de lo contrario se comprueban también los directorios locales predeterminados. Jinja admite herencia, inclusiones, funciones asíncronas y escape HTML automático. Las variables inexistentes producen un error por defecto.

Utiliza `@controller` y devuelve `ModelAndView("catalog.html", contexto)` para una página; también puedes indicar `status_code` y `headers`. `@rest_controller` conserva el comportamiento JSON. Asigna nombres únicos mediante `@get_mapping("/", name="catalog")` para evitar rutas escritas a mano.

En plantillas, `reverse('catalog')` genera una ruta local y `static_url('catalog.css')` resuelve un recurso estático. En Python, importa `reverse` y `static_url` de `pyfly.web` y pasa `request` como primer argumento. Por ejemplo, `Redirect(reverse(request, "catalog"))` devuelve un 303 local después

de un POST. Ambas funciones respetan los prefijos de montaje y la ruta estática configurada. `url_for('catalog')` sigue disponible cuando necesitas una URL absoluta.

Pasa nombres y parámetros sin codificación previa: las funciones codifican espacios, Unicode, `#` y `?`. Los recursos se indican respecto a la raíz estática, por ejemplo `images/logo.svg` o `downloads/manual.pdf`, sin `/` inicial ni segmentos `..`. Una ruta inexistente produce un error de resolución; generar una URL no comprueba que el archivo exista. Jinja `{% include 'partials/menu.html' %}` carga otra plantilla desde sus raíces; no genera una URL ni publica archivos del servidor.

Registra un bean `TemplateEngine` para sustituir Jinja o beans `TemplateContextProcessor` para añadir valores por petición. El modelo de la vista prevalece sobre los procesadores; las funciones de URL y CSRF las reserva el framework. Las redirecciones HTTP(S) externas requieren `allow_external=True`.

E.4. Vincular formularios y mostrar errores

La acción de creación recibe `product: Form[ProductForm]`. PyFly analiza los campos, convierte los tipos y valida el modelo Pydantic antes de invocar el controlador.

La acción pasa `product.model_dump(exclude={"edit_version"})` a `AdminDataService.create` y redirige. Para editar, pasa también el identificador y `product.edit_version` a `AdminDataService.update`.

Cada formulario POST debe incluir el token CSRF:

catalog/templates/form.html

```

<input type="hidden" name="{{ csrf_field }}" value="{{ csrf_token }}">
<input type="hidden" name="edit_version"
      value="{{ values.get('edit_version', '') }}">
<label for="name">Name</label>
<input id="name" name="name" value="{{ values.get('name', '') }}"
      required maxlength="80">
<label for="price">Price</label>
<input id="price" name="price" inputmode="decimal"
      value="{{ values.get('price', '') }}" required>

```

Listado E.3 — Campos del formulario del ejemplo

Captura `FormValidationException` mediante `@exception_handler` y devuelve la plantilla con `error.values`, `error.errors` y estado 422. El controlador completo está en `catalog/controllers.py`. Los campos identificados como secretos y los tipos secretos de Pydantic se eliminan de los valores mostrados; los mensajes omiten la entrada enviada. Conserva el escape HTML. Un GET normal no invalida el token de otra pestaña; la rotación o invalidación de sesión sí lo cambia.

Se admiten alias y campos de lista repetidos; los escalares repetidos se rechazan. Una casilla desmarcada no envía valor: utiliza `False` como valor predeterminado y `"true"` cuando esté marcada. Texto multipart y `File[UploadedFile]` comparten un análisis limitado: 1.000 campos, 20 archivos, 1 MiB por parte y 16 MiB por petición de forma predeterminada, configurables bajo `pyfly.web.forms`.

E.5. Personalizar las páginas de error

Añade `errors/404.html` y `errors/500.html` a las raíces de plantillas. Para asociar un estado explícitamente, configura `pyfly.web.errors.templates`, por ejemplo `{"404": "errors/missing.html"}`. Se intenta primero esa asociación, después el

estado exacto, la familia (`errors/4xx.html`) y `default-template` , cuyo valor es `errors/error.html` . Una plantilla ausente permite probar la siguiente; una rota produce la página integrada con contenido escapado.

Las plantillas de error reciben `status` , `title` , `message` , `path` y `transaction_id` , además de `reverse` , `static_url` y `url_for` para navegación y recursos compartidos. Puedes usar `static_url('catalog.css')` , pero el diseño no debe depender de procesadores de contexto ni funciones CSRF/formulario. Los errores de servidor ocultan detalles de excepciones de forma predeterminada. Los errores del navegador negocian HTML mediante `Accept` ; REST y la API de datos siguen devolviendo JSON. Los fallos de autenticación y CSRF conservan estado y cabeceras de seguridad. Las páginas usan `no-store` y `Vary: Accept` .

Identidad visual y página de bienvenida

El framework incluye `pyfly/base.html` , `pyfly/welcome.html` y `pyfly/error.html` , con el logotipo de PyFly, su paleta verde, diseño adaptable y colores claros u oscuros según el navegador. Solo el arquetipo `web` activa estas funciones automáticamente. Los proyectos de API, microservicios, arquitectura hexagonal, bibliotecas y CLI las mantienen desactivadas; instalar un extra no activa páginas. Una API existente puede activarlas de forma deliberada mediante configuración.

Con plantillas activadas, la bienvenida aparece en `/` cuando no existe un controlador de inicio. Los controladores descubiertos durante el arranque tienen prioridad, como ocurre en el catálogo. `pyfly.web.welcome.enabled: false` desactiva esta página; `welcome.template` selecciona otra plantilla. Una webapp mínima que solo utiliza plantillas integradas configura `templates.enabled: true` y `templates.directories: []` .

`pyfly.web.branding` configura vistas y errores conjuntamente. `name` vale `PyFly`; `tagline`, `Build something that matters.`; `primary-color`, `#4cbb2f`; y `accent-color`, `#c2e85f`. Los colores aceptan valores hexadecimales de seis dígitos. `logo-url` y `favicon-url` aceptan URLs HTTP(S) o rutas relativas a la aplicación como `/static/catalog-logo.svg`; se añade el prefijo del montaje ASGI. Un logo vacío usa el de PyFly y un favicon vacío omite el icono. Personaliza `documentation-url`, `support-url` y `footer`; una URL vacía oculta su enlace.

Las plantillas reciben `branding`, `home_url` y `web_asset_url`. El framework publica logo y estilos bajo `branding.assets-path`, cuyo valor es `/_pyfly/web`, solo al activar plantillas o errores HTML. Permite ese prefijo reservado en las reglas de seguridad si las páginas anteriores al inicio de sesión lo necesitan. Los estilos externos funcionan con `style-src 'self'`. Los recursos de la aplicación usan `static_url`; `web_asset_url('web.css')`, `web_asset_url('theme.css')` y `web_asset_url('logo.png')` resuelven los recursos integrados.

Extiende una plantilla integrada y reemplaza los bloques `title`, `head`, `navigation`, `content` o `footer`. Los errores del catálogo extienden `pyfly/error.html` y añaden el enlace al catálogo en navegación. Las raíces de la aplicación tienen prioridad sobre las integradas. Mantén la plantilla hija en una ruta distinta de la del padre para evitar herencia recursiva. Un motor personalizado debe ofrecer la plantilla de bienvenida seleccionada o desactivar esa página.

Trazas durante el desarrollo

`pyfly.web.errors.include-stacktrace` acepta `never`, `on-debug` (predeterminado) y `always`. Activa `pyfly.web.debug: true` en un perfil local, o pasa `debug=True` a la factoría del adaptador, para mostrar trazas con la política predeterminada. También funciona `PYFLY_WEB_DEBUG=true`. En producción el panel permanece oculto; los

parámetros de la URL no lo activan. Con errores HTML habilitados, PyFly controla las páginas de depuración negociadas y los controladores REST siguen devolviendo JSON.

Las plantillas reciben `show_stacktrace`, `exception_type`, `exception_message`, `stack_trace` y `stack_frames`. Cada marco contiene `filename`, `lineno`, `function` y `source`. La traza corresponde a la excepción original, incluso cuando un conversor modifica el estado HTTP. Si el diagnóstico está desactivado, se reciben cadenas y lista vacías. Los rechazos de seguridad tempranos pueden no tener excepción.

`max-stack-frames` vale 30 (rango 1–200); `max-trace-length`, 20.000 caracteres (rango 256–100.000). Se incluyen los marcos más recientes de la excepción actual, sin cadenas de excepciones ni variables locales. No se recopilan cabeceras, cookies, parámetros de consulta ni cuerpos de peticiones. El mensaje y las líneas de código pueden contener secretos: mantén las trazas desactivadas en producción pública. Jinja escapa su texto; no apliques `safe`.

`pyfly/error.html` incluye el panel. En un diseño independiente usa `{% if show_stacktrace %}<pre>{{ stack_trace }}</pre>{% endif %}`. Si falla una plantilla de error de la aplicación, el renderizador integrado independiente conserva la traza original. También existe una respuesta con identidad visual y contenido escapado cuando Jinja no está instalado.

E.6. Registrar la entidad existente

En una aplicación con escaneo, publica un bean `ModelAdmin`. Esta alternativa sustituye el registro explícito y el proveedor que configura el ejemplo:

catalog/administration.py

```

from catalog.models import Product, ProductWrite
from pyfly.admin import ModelAdmin
from pyfly.container import bean, configuration

@Configuration
class Administration:
    @bean
    def products(self) → ModelAdmin:
        return ModelAdmin(
            "products", Product, label="Products",
            datasource="primary",
            fields=("id", "name", "price", "updated_at"),
            editable_fields=("name", "price"),
            search_fields=("name",), filter_fields=("name",),
            ordering=("name",),
            operations=("list", "read", "create", "update", "delete"),
            create_schema=ProductWrite, update_schema=ProductWrite,
        )

```

Listado E.4 — Registrar el Product existente

El origen primario reutiliza el bean `async_session_factory` ; uno con nombre usa `NamedDataSources.get(name)` . Para un documento Beanie ya inicializado, indica `datasource="document"` . Un `provider` explícito tiene prioridad. Administración no crea clientes, migra esquemas ni cierra recursos propiedad de la aplicación.

Activa `pyfly.admin.data.enabled` , configura `allowed-roles` y proporciona `edit-token-key` mediante un secreto de entorno de al menos 32 caracteres si habilitas escrituras. Comparte la clave entre procesos; rotarla invalida ediciones abiertas. Por defecto, un registro solo permite listar y leer. Las tablas no se exponen automáticamente. Las claves primarias y los campos de auditoría, versión y binarios son de solo lectura; selecciona explícitamente los campos editables.

E.7. Permisos, conflictos y reglas de negocio

La administración de datos exige un `SecurityContext` autenticado y un rol permitido aunque la monitorización use `require-auth: false`. Un puerto de administración separado debe activar `pyfly.management.security.enabled`; el arranque falla si falta. Configura autenticación y reglas de rutas para ese puerto. Toda mutación exige la cookie CSRF y su cabecera `X-XSRF-TOKEN`, incluso con bearer.

Las operaciones permitidas son la intersección de configuración global, registro y `has_permission(operation, context, record=None)`. El método puede restringir, pero no ampliar esas listas. Usa `scope(context)` para restricciones de igualdad definidas por el servidor, como el tenant. Se aplican a recuentos, listas, lecturas y mutaciones; las altas heredan el ámbito y las modificaciones no pueden salir de él. Los permisos por objeto no bastan para ocultar filas de una lista.

El panel agrupa orígenes y recursos y permite paginación, búsqueda, filtros y orden. Los formularios muestran controles tipados, nulos explícitos y errores de validación. `relations={"category_id": "categories"}` obtiene opciones autorizadas y comprueba la selección antes de escribir. Las escrituras anidadas requieren lógica propia.

El token de edición representa todo el estado persistido, incluidos campos ocultos. PATCH lo envía como `editToken`; DELETE, en `If-Match`. PostgreSQL bloquea la fila, SQLite mantiene una transacción inmediata y MongoDB ejecuta una sustitución o borrado condicionado al BSON original. Una edición obsoleta devuelve 409 y conserva la entrada hasta una recarga explícita. Las restricciones conflictivas también devuelven 409; la entrada inválida, 422. Verifica bloqueos equivalentes antes de habilitar escrituras concurrentes en otros dialectos SQL.

Los decimales y enteros escalares grandes conservan precisión como cadenas ante el navegador. `AdminFieldAdapter(type, serialize, parse)` adapta tipos propios. Si necesitas reglas de servicio o action hooks de Beanie, implementa `AdminDataProvider` y pásalo como `provider`. El proveedor Mongo integrado valida el documento resultante, pero no ejecuta action hooks arbitrarios. Un proveedor propio debe conservar ámbito, validación y comprobaciones atómicas de edición. La auditoría de éxito registra actor, recurso, identificador, operación, nombres de campos y correlación, sin los valores enviados.

E.8. Configuración, pruebas y diagnóstico

Las propiedades utilizan YAML/TOML, perfiles y entorno como el resto de PyFly.

`PYFLY_WEB_TEMPLATES_AUTO_RELOAD=true` activa recarga local de plantillas;
`PYFLY_ADMIN_DATA_OPERATIONS=list,read` restringe globalmente a lectura. Usa perfiles para diferencias entre entornos y secretos para credenciales. Plantillas, estáticos, errores HTML y administración de modelos requieren activación explícita.

Ejecuta `uv run pytest` en `samples/webapp`: usa una base temporal y comprueba HTML, validación, CSRF, persistencia y los mismos IDs en administración. El framework añade pruebas de ambos adaptadores, PostgreSQL/MongoDB reales, CRUD en navegador e instalación del wheel. `docs/modules/webapps.md` contiene comandos y referencia completa.

Síntoma	Comprobación
Falla la raíz de plantillas	Directorio existente, paquete instalado y raíces no usadas desactivadas.
POST local devuelve 403	Cookie segura sobre HTTP, campo CSRF ausente o sesión rotada.
Datasources no aparece o está vacío	Activación, registro descubierto, rol y permiso para listar.
Falla el arranque de administración	Activa su seguridad y configura autenticación real.
Guardar devuelve 409	Recarga el registro; revisa restricciones y valores únicos.
Enlaces rotos bajo un prefijo	Usa <code>reverse / static_url</code> y configura el root path ASGI.

Pruébalo tú mismo

1. Crea un producto en el catálogo, editalo en el panel y verifica ID y precio desde una sesión nueva de base de datos.
2. Edita un registro en dos pestañas. Guarda la primera y después la segunda: comprueba que 409 impide sobrescribir silenciosamente el valor más reciente.
3. Limita las operaciones globales a `list,read`. Comprueba tanto los controles del panel como el rechazo de peticiones de mutación directas.
4. Cambia la plantilla 404 y compara una ruta inexistente con `Accept: text/html` y `Accept: application/json`.

APÉNDICE

Glosario

Adaptador (adapter) — Una clase concreta que implementa un puerto delegando en una biblioteca o tecnología de infraestructura específica (PostgreSQL, Redis, Kafka, etc.). Los adaptadores viven en el borde de la arquitectura hexagonal y pueden intercambiarse sin tocar las capas de dominio o de aplicación. En PyFly, la factoría de sesiones asíncronas de SQLAlchemy, `RedisCacheAdapter` y `KafkaMessageBroker` son todos adaptadores (Capítulos 5, 10, 13).

Raíz de agregado (aggregate root) — El único punto de entrada a un grupo de objetos de dominio que deben permanecer consistentes entre sí; todos los cambios de estado se canalizan a través de sus métodos, que imponen las invariantes y emiten eventos de dominio. El código externo carga y guarda solo la raíz, nunca sus objetos internos. En PyFly, `AggregateRoot[ID]` es la clase base con estado almacenado; `Wallet` es la raíz de agregado con estado almacenado de Lumen y `LedgerAccount` es su contrapartida con event sourcing (Capítulos 6, 9).

AOP (Programación Orientada a Aspectos) — Una técnica para añadir preocupaciones transversales —registro de logs, métricas, comprobaciones de seguridad— de forma declarativa a los métodos sin modificar su código fuente. Los decoradores de aviso `@aspect` y `@before` / `@after` / `@around` de PyFly implementan AOP; el Capítulo 15 lo usa para asociar observabilidad a cada método de servicio.

ApplicationContext — El contenedor de inyección de dependencias central que descubre los beans, resuelve las dependencias y gestiona su ciclo de vida desde el arranque hasta el apagado. Dispara `ContextRefreshedEvent` una vez que todos los

beans están cableados y `ContextClosedEvent` en un apagado ordenado.

`ApplicationContext` es el equivalente en PyFly del `ApplicationContext` de Spring (Capítulo 2).

Async/await — La sintaxis nativa de corrutinas de Python para E/S no bloqueante. PyFly está construido de forma nativamente asíncrona: cada manejador HTTP, método de repositorio, llamada al bus y cliente de servicio se declara `async def` y se planifica en el bucle de eventos. Las llamadas bloqueantes dentro de una función `async def` congelan el bucle y deben evitarse (Capítulo 1).

Autocableado (autowiring) — El mecanismo por el cual el contenedor de inyección de dependencias resuelve los argumentos del constructor de un bean únicamente a partir de las anotaciones de tipo, sin necesidad de código de factoría. PyFly inspecciona las firmas de `__init__` en el arranque e inyecta automáticamente los beans coincidentes; la anotación `@primary` selecciona la implementación preferida cuando existen varios candidatos (Capítulo 2).

Bean — Cualquier objeto Python que el contenedor de inyección de dependencias crea, cablea y gestiona. Declaras una clase como bean aplicando un decorador de estereotipo (`@service` , `@repository` , `@component` , `@configuration` o `@rest_controller`), o anotando un método de factoría con `@bean` dentro de una clase `@configuration` (Capítulo 2).

BFF (Backend for Frontend) — Una capa de pasarela de API que se sitúa frente a varios microservicios y compone sus capacidades en una API enfocada al recorrido del usuario, adaptada a un cliente de frontend específico. En el Capítulo 11, el nivel BFF de Lumen agrega los datos del monedero y de los pagos para que la aplicación móvil haga una sola petición en lugar de dos.

Contexto delimitado (bounded context) — Un ámbito a nivel de DDD dentro del cual un modelo de dominio tiene un significado único e inequívoco. Los servicios en una arquitectura de microservicios a menudo se corresponden uno a uno con contextos delimitados: `WalletService` posee el modelo del monedero; `PaymentsService` posee el modelo de pagos. Se necesitan núcleos compartidos o capas anticorrupción cuando dos contextos deben intercambiar datos (Capítulo 6).

Mamparo (bulkhead) — Un patrón de resiliencia que limita el número de llamadas concurrentes a un recurso o servicio dependiente concreto, evitando que una dependencia lenta agote el pool de hilos o de corrutinas y degrade rutas no relacionadas. El decorador `@bulkhead(max_concurrent=N)` de PyFly implementa el mamparo basado en semáforo (Capítulo 13).

Cortacircuitos (circuit breaker) — Un patrón de resiliencia que cuenta los fallos consecutivos hacia una dependencia remota; una vez que se alcanza el umbral de fallos, el cortacircuitos salta al estado abierto y cortocircuita las llamadas posteriores con un error rápido, dando tiempo a la dependencia para recuperarse antes de reanudar las llamadas. El decorador `@circuit_breaker` y `@service_client` de PyFly inyectan este comportamiento automáticamente (Capítulos 11, 13).

Comando (CQRS) — Un dataclass inmutable y congelado que expresa una única intención de escritura — «abrir un monedero», «depositar fondos» — y lleva los datos exactos necesarios para satisfacerla. Los comandos heredan de `Command[R]`, donde `R` es el tipo de retorno del manejador, y fluyen a través del `CommandBus`. Opcionalmente pueden implementar `validate()` y `authorize()` (Capítulo 7).

CommandBus — La canalización que recibe un `Command`, ejecuta la validación y la autorización, lo despacha al `CommandHandler` coincidente y luego publica cualquier evento de dominio almacenado en búfer por el agregado. Se registra un manejador (handler) por cada tipo de comando; el bus impone esta restricción en el arranque (Capítulo 7).

Compensación — Una operación hacia adelante que revierte semánticamente el efecto de un paso de saga completado previamente cuando un paso posterior falla. A diferencia de un rollback de base de datos, la compensación es una nueva escritura que deshace explícitamente el cambio anterior (por ejemplo, «reembolsar pago» compensa «capturar pago»). Cada `@saga_step` nombra su método de compensación (Capítulo 12).

Component — Un estereotipo genérico para un bean gestionado que no encaja en los roles más específicos de `@service`, `@repository` o `@rest_controller`. `@component` registra la clase con el contenedor y habilita la inyección, pero no aporta ningún significado semántico adicional (Capítulo 2).

Clase de configuración — Una clase decorada con `@configuration` que agrupa métodos de factoría `@bean`. El contenedor la trata como una fuente de definiciones de beans, llamando a cada método de factoría y registrando el valor de retorno como un bean nombrado y tipado. Se pueden aplicar guardas de perfil y anotaciones condicionales a la clase o a métodos de factoría individuales (Capítulo 3).

Contenedor (DI) — Véase *ApplicationContext*.

Convención sobre configuración — El principio según el cual unos valores predeterminados sensatos eliminan el código repetitivo: una clase anotada con `@service` queda automáticamente con ámbito singleton, es descubierta por el

escaneo de componentes e inyectada por tipo sin ningún XML ni registro explícito. PyFly adopta esto como valor de diseño central, exigiendo anulaciones explícitas solo cuando el valor predeterminado es incorrecto (Capítulo 1).

CQRS (Segregación de Responsabilidad entre Comandos y Consultas) — Un patrón arquitectónico que separa las operaciones de escritura (comandos) de las operaciones de lectura (consultas) en rutas de código, buses y, potencialmente, modelos de datos distintos. Las lecturas pueden almacenarse en caché de forma independiente de las escrituras; los manejadores pueden probarse de forma aislada; las preocupaciones transversales se aplican uniformemente por el bus respectivo (Capítulo 7).

Cola de mensajes fallidos (DLQ) — Una cola o topic de mensajes dedicado que recibe los mensajes que no pudieron procesarse tras el número configurado de reintentos. El `@message_listener` de PyFly enruta automáticamente los mensajes envenenados a la DLQ, evitando que bloqueen los mensajes sanos (Capítulo 10).

Inyección de dependencias (DI) — Un patrón de diseño en el que un objeto declara sus colaboradores como parámetros del constructor y un contenedor externo proporciona las instancias concretas. La inyección de dependencias desacopla la construcción del uso, lo que facilita intercambiar implementaciones y probar clases con dobles (fakes) o mocks (Capítulo 2).

Evento de dominio — Un registro inmutable de un hecho de negocio que ya ha ocurrido — «monedero abierto», «fondos depositados» —. Los eventos de dominio son producidos por las raíces de agregado mediante `raise_event()`, publicados a través del puerto `EventPublisher` y consumidos por escuchadores independientes. En event sourcing, son además la fuente de verdad del estado del agregado (Capítulos 6, 8, 9).

DTO (Objeto de Transferencia de Datos) — Un objeto plano y serializable usado para transportar datos a través del límite de una capa —entre el controlador HTTP y el servicio, o entre un servicio y su cliente de API— sin exponer los tipos internos del dominio. En PyFly, las subclases de `BaseModel` de Pydantic sirven como DTO de petición/respuesta (Capítulo 4).

EDA (Arquitectura Orientada a Eventos) — Un estilo arquitectónico en el que los servicios se comunican publicando y suscribiéndose a eventos en lugar de mediante llamadas síncronas directas. Productores y consumidores están desacoplados: un productor no sabe qué consumidores existen. El `EventPublisher` y `@event_listener` de PyFly son las primitivas de EDA intra-proceso; `MessageBrokerPort` extiende el patrón a través de los límites de proceso (Capítulos 8, 10).

Entidad — Un objeto de dominio con una identidad estable que persiste a lo largo del tiempo y a través de los cambios de estado. Dos entidades son iguales si y solo si comparten el mismo `id` no nulo. En PyFly, `Entity[TID]` rastrea la identidad; `BaseEntity` añade columnas de auditoría (`created_at` , `updated_at`) para la capa de persistencia (Capítulos 5, 6).

Event sourcing (abastecimiento de eventos) — Una estrategia de persistencia en la que cada cambio de estado se almacena como un evento de dominio inmutable en un flujo de solo adición. El estado actual de un agregado se calcula reproduciendo todos los eventos del flujo. El módulo `pyfly.eventsourcing` de PyFly proporciona `AggregateRoot` , `EventStore` , `EventSourcedRepository` , soporte para instantáneas y un `ProjectionRunner` (Capítulo 9).

EventEnvelope — El envoltorio de metadatos que empaqueta la carga útil de un evento de dominio para su entrega: ID del evento, tipo de evento, ID del flujo del agregado, número de secuencia, marca de tiempo, ID de correlación e ID de causalidad. Cada evento que llega a un escuchador llega dentro de un `EventEnvelope`; nunca construyes uno manualmente (Capítulos 8, 9).

EventStore — La capa de persistencia de solo adición para un sistema con event sourcing. Registra los eventos de dominio indexados por ID de flujo y número de secuencia, impone concurrencia optimista mediante el token `version` y soporta consultas por rango para la reproducción. PyFly incluye `SQLAlchemyEventStore` e `InMemoryEventStore` (Capítulo 9).

Arquitectura hexagonal — Un estilo arquitectónico que coloca la lógica de dominio y de aplicación en el centro, rodeada de puertos (interfaces), con adaptadores en los bordes. El código de negocio depende únicamente de los puertos; los adaptadores implementan esos puertos usando tecnologías específicas. Este es el principio organizador de cada módulo de PyFly (Capítulos 1, 2, 5).

Idempotencia — La propiedad por la cual realizar la misma operación varias veces produce el mismo resultado que realizarla una sola vez. Los manejadores idempotentes son esenciales en la mensajería y la compensación de sagas: los reintentos de red o las garantías de entrega al-menos-una-vez implican que un mensaje puede llegar más de una vez. Se usan claves de deduplicación o tokens de idempotencia para detectar y descartar ejecuciones duplicadas (Capítulos 10, 12).

Migración — Un script versionado y ordenado que hace evolucionar el esquema de una base de datos relacional sin destruir datos. PyFly integra Alembic para la gestión de migraciones; `pyfly db migrate` autogenera una migración a partir de los cambios en las entidades y `pyfly db upgrade` aplica las migraciones pendientes (Capítulo 5).

Patrón outbox — Una técnica para publicar eventos de dominio de forma fiable junto con una escritura en la base de datos: tanto el cambio de estado como los registros de eventos se escriben en la misma transacción local; un proceso de retransmisión en segundo plano lee los eventos no enviados de la tabla outbox y los reenvía al broker. Esto elimina el commit en dos fases entre la base de datos y el broker de mensajes. Con `pyfly.eda.outbox.enabled: true` PyFly pone un outbox delante de cualquier `EventPublisher`, sobre tablas SQL o colecciones de MongoDB; `TransactionalOutbox` hace lo mismo para un almacén de eventos (Capítulos 8, 9, 10).

Puerto (port) — Una clase `Protocol` de Python que define la interfaz de la que depende una pieza de lógica de negocio, sin especificar ninguna implementación. El contenedor de inyección de dependencias cablea en el arranque el adaptador concreto que satisface el protocolo. Los puertos habilitan la arquitectura hexagonal y hacen que los adaptadores sean intercambiables sin ningún cambio en la lógica de negocio (Capítulos 1, 2, 5).

Bean primario — Cuando varios beans satisfacen el mismo tipo, se prefiere para la inyección el que está anotado con `@primary`. Sin una anotación `@primary`, el contenedor lanza un error de ambigüedad. `@primary` es la forma de designar el adaptador de producción entre varias alternativas (Capítulo 2).

Perfil — Una etiqueta de activación nombrada que selecciona qué beans y valores de configuración están activos en tiempo de ejecución. PyFly carga `pyfly-{profile}.yaml` sobre el `pyfly.yaml` base y activa los beans anotados con `@profile("prod")` solo cuando `prod` está en la lista de perfiles activos. El perfil activo se establece mediante `PYFLY_PROFILES_ACTIVE` o `pyfly.yaml` (Capítulo 3).

Proyección — Un modelo de lectura derivado del consumo de un flujo de eventos.

Una proyección se suscribe a tipos de eventos específicos y construye incrementalmente una vista consultable —una caché de saldos, una tabla de auditoría, un agregado para un panel— sin tocar el modelo de escritura. En event sourcing, `ProjectionRunner` reproduce el `EventStore` para reconstruir las proyecciones desde cero (Capítulos 8, 9).

Consulta (CQRS) — Un dataclass inmutable que expresa una intención de lectura —«obtener el saldo del monedero», «listar transacciones»—. Las consultas heredan de `Query[R]` y fluyen a través del `QueryBus`, que puede almacenar los resultados en caché de forma transparente. Las consultas nunca mutan el estado (Capítulo 7).

QueryBus — La canalización que recibe una `Query`, opcionalmente devuelve un resultado en caché, la despacha al `QueryHandler` coincidente y opcionalmente almacena el resultado en caché. Separar el bus de consultas del bus de comandos permite un comportamiento transversal diferente —caché, enrutamiento a réplicas de lectura— para las rutas de lectura (Capítulo 7).

Limitador de tasa (rate limiter) — Un componente de resiliencia que limita el número de peticiones que un endpoint o cliente de servicio puede aceptar dentro de una ventana de tiempo, evitando la sobrecarga por tráfico en ráfagas. El `RateLimiter(max_tokens=N, refill_rate=r)` + `@rate_limiter(limiter)` de PyFly implementa un limitador de cubo de tokens (Capítulo 13).

Repositorio — Una abstracción similar a una colección sobre la capa de persistencia que permite a la aplicación cargar y guardar agregados o entidades sin nada de SQL en el código de negocio. El `CrudRepository[E, ID]` de PyFly proporciona `find_by_id`, `save`, `delete` tipados y ayudantes de consultas derivadas; las implementaciones personalizadas anotadas con `@repository` reemplazan a las predeterminadas en memoria (Capítulos 2, 5).

Reintento (retry) — Un patrón de resiliencia que vuelve a ejecutar una operación fallida tras un retardo, hasta un número máximo de intentos configurado. Los reintentos manejan fallos transitorios —breves fallos de red, errores 503 momentáneos— sin intervención del operador. El `@retry(max_attempts=N, backoff=...)` de PyFly implementa retroceso exponencial con jitter; `@service_client` incluye reintentos por defecto (Capítulos 11, 13).

Saga — Una secuencia de transacciones locales coordinadas por un orquestador central. Cada paso confirma en la base de datos de su propio servicio; si un paso posterior falla, el orquestador llama a la transacción compensatoria de cada paso ya confirmado en orden inverso. Los decoradores `@saga` y `@saga_step` de PyFly implementan el patrón saga orquestado con un DAG de ejecución en paralelo (Capítulo 12).

Serialización — El proceso de convertir un objeto en memoria a un formato de transmisión (JSON, Protobuf, Avro) para respuestas HTTP o publicación de mensajes, y la deserialización en sentido inverso. El bean central `PyFlyJsonSerializer` de PyFly (`from pyfly.web import PyFlyJsonSerializer`) configura una vez la serialización JSON respaldada por Pydantic y la aplica a cada respuesta HTTP y a cada envoltorio de mensaje (Capítulos 4, 10).

Servicio — Un bean gestionado decorado con `@service` que alberga la lógica de negocio y orquesta las llamadas a repositorios, publicadores de eventos y otros servicios. Los servicios son la capa de aplicación en la arquitectura hexagonal: traducen los comandos entrantes en operaciones de dominio y persisten los resultados (Capítulo 2).

Instantánea (snapshot) — Una copia serializada en un instante concreto del estado de un agregado con event sourcing, almacenada junto al flujo de eventos para acelerar la reproducción. Al cargar, el repositorio restaura la instantánea y reproduce solo los eventos ocurridos después de la versión de la instantánea, acotando el tiempo de reproducción independientemente de la longitud del flujo (Capítulo 9).

Estereotipo — Un decorador que registra una clase como bean y señala su rol arquitectónico: `@service`, `@repository`, `@component`, `@configuration` o `@rest_controller`. Todos los estereotipos son técnicamente equivalentes en el contenedor; la diferencia semántica es para los lectores humanos y las herramientas (Capítulo 2).

TCC (Try-Confirm-Cancel) — Un patrón de transacciones distribuidas en el que cada participante primero *reserva* un recurso (Try), y luego el coordinador o bien *confirma* todas las reservas o las *cancela* en función de si todos los Try tuvieron éxito. TCC es útil cuando se requiere una semántica de reserva exacta e inmediata —por ejemplo, retener fondos antes de capturar un pago—. Los decoradores `@tcc(name="...")` + `@tcc_participant(id="...", order=N)` de PyFly implementan el protocolo TCC junto al motor de sagas (Capítulo 12).

Testcontainers — Una biblioteca que arranca contenedores Docker reales (PostgreSQL, Redis, Kafka) para las pruebas de integración y los detiene cuando la suite termina. El módulo `pyfly.testing` de PyFly proporciona factorías como `postgres_container`, `mysql_container`, `mariadb_container`, `mongodb_replica_set_container` y `redis_container` que cablean Testcontainers en el contenedor de inyección de dependencias mediante una configuración al estilo `@ServiceConnection` (Capítulo 16).

Unidad de trabajo (unit of work) — Un lote de cambios de todo o nada: cada escritura que contiene se confirma junta, o ninguna lo hace. En PyFly cada frontera transaccional es una `UnitOfWork` ligada a la tarea en curso (`pyfly.data.transaction`): `@transactional` empieza una o se une a ella, las llamadas al repositorio dentro de ella se unen, y una llamada al repositorio fuera de ella se ejecuta en una unidad breve propia que confirma la escritura antes de devolver (Capítulos 5, 7).

Objeto de valor (value object) — Un objeto de dominio inmutable identificado por su valor en lugar de por un campo de identidad. Dos objetos de valor son iguales si todos sus campos son iguales. En PyFly, `ValueObject` es la clase base; aplica `@dataclass(frozen=True)` para imponer la inmutabilidad. `Money` — un importe entero en **unidades menores** (céntimos) y un `Currency` `StrEnum` — es el objeto de valor canónico de Lumen; `Wallet` es la raíz de agregado que lo posee (Capítulo 6).

Webhook — Una llamada de retorno HTTP entrante que un proveedor externo (Stripe, Twilio, etc.) invoca para notificar a tu servicio de un evento asíncrono, como un cambio en el estado de un pago. El decorador `@webhook_listener` de PyFly verifica la firma HMAC-SHA256, deduplica las repeticiones mediante una caché de nonces y enruta la carga útil a un manejador tipado (Capítulo 17).

Flujo de trabajo (workflow) — Una variante de larga duración del patrón saga en la que los pasos pueden pausarse durante minutos, horas o a la espera de aprobación humana antes de reanudarse. Los flujos de trabajo persisten su estado entre pasos para sobrevivir a los reinicios del proceso; los decoradores `@workflow` y `@workflow_step` de PyFly proporcionan esta capacidad sobre el mismo motor de sagas (Capítulo 12).